

INTRODUCCIÓN AL DISEÑO Y CONFIGURACIÓN DE
SOFTWARE EN REDES LOCALES

75 PROYECTOS BÁSICOS

INCLUYE EL CAPÍTULO
PYTHON y SQLite en ENTORNOS MULTIUSUARIO

POR JAVIER GARCÍA

García Programming Books
www.brownbook.cloud
Tomás Morales 118, Las Palmas 35004, Spain

Texto (c) de Javier García 2026
javier928@gmail.com

Cubierta e ilustraciones interiores de Javier García Estévez

Esta es una obra de ficción. Toda la información sobre períodos históricos es correcta en el momento de la publicación.

Android, Google Play y el logo de Google Play son marcas registradas de Google LLC.
Apple, iOS, macOS, iPhone y iPad son marcas registradas de Apple Inc.
en EE.UU., otros países y regiones.
Microsoft, Windows, Excel, PowerPoint, Word, Access, FoxPro y Notepad son marcas registradas de Microsoft Corporation.
WordPress es una marca registrada de la Fundación WordPress.
LiveCode (anteriormente Runtime Revolution) es una marca registrada de LiveCode Ltd.

Introducción al diseño y configuración de software en redes locales
© 2026 by Javier García is licensed under Creative Commons
Attribution-NonCommercial-NoDerivatives 4.0 International.
To view a copy of this license, visit:
<https://creativecommons.org/licenses/by-nc-nd/4.0/>

Prefacio

En este libro presentaremos al usuario problemas y soluciones en diferentes lenguajes de programación: Python, Microsoft Visual FoxPro, C# y SQL. Por ejemplo, veremos cómo utilizar estos lenguajes para construir y gestionar bases de datos relacionales en una red local de computadoras personales.

Para utilizar de manera eficiente cualquier lenguaje, es muy recomendable aprender con ejemplos básicos y sencillos, como los módulos que permiten la gestión de contenido en este tipo de aplicaciones.

SQL, que significa Structured Query Language (Lenguaje de Consulta Estructurada), fue creado en 1974. Su desarrollo se atribuye a Edgar F. Codd, un investigador de IBM, quien introdujo el modelo relacional de datos en 1969.

Python fue creado en 1989 por Guido van Rossum, un informático neerlandés. Desde entonces, ha evolucionado y se ha convertido en uno de los lenguajes de programación más populares del mundo.

C# se lanzó en el año 2000 como parte de la plataforma .NET de Microsoft. El lenguaje de programación C#, creado por Anders Hejlsberg, Scott Wiltamuth y Peter Golde. Este lenguaje de programación de alto nivel se caracteriza por las múltiples plataformas de desarrollo que ofrece. Actualmente, ya no es necesario aprender lenguajes especializados como C o Fortran para desarrollar aplicaciones o videojuegos en diferentes entornos. Lo que necesitaremos, sin embargo, es saber qué herramientas o librerías permiten construir software para los diferentes sistemas operativos que podemos encontrar en el mercado.

En la última década, el mercado de aplicaciones móviles ha transformado radicalmente la forma en que interactuamos con la tecnología. Desde herramientas de productividad hasta entretenimiento, las apps y los libros en formato electrónico se han convertido en un negocio multimillonario.

El ecosistema Android ha experimentado una expansión enorme. Con millones de aplicaciones disponibles y miles de millones de descargas, se ha consolidado como una plataforma accesible tanto para jóvenes como para adultos. A diferencia del entorno creado por la multinacional americana Apple, Android permite a los desarrolladores publicar sus creaciones con mayor libertad, lo que ha impulsado la innovación y la competencia global.

Según un artículo de *The Economist* titulado “The World in 2015”, las aplicaciones móviles generarían más de 77 mil millones de dólares en ingresos para finales de 2016. Esta cifra puso de manifiesto el valor comercial de las apps y los servicios digitales.

Según el informe “Mercado de Aplicaciones Móviles, Informe 2025-2034”, el mercado global de aplicaciones móviles alcanzó un valor aproximado de 230 mil millones de dólares en 2024, y se prevé que alcance los 754 mil millones de dólares para 2034 (Análisis redactado por la empresa de Wyoming, *Informe de Expertos*). Este crecimiento deja entrever el valor comercial de las apps y su conexión directa con la publicidad digital, el comercio electrónico, los servicios financieros y la salud. Las aplicaciones móviles se han convertido en herramientas esenciales para la vida diaria, impulsando modelos de negocio como las suscripciones y las compras dentro de la app, mientras que tecnologías emergentes como la realidad aumentada y la inteligencia artificial continúan redefiniendo la experiencia del usuario.

Para los programadores, analistas y científicos de datos, el mercado de apps representa una oportunidad única. La posibilidad de crear, publicar y vender aplicaciones en cualquier parte del mundo plantea un reto para la monetización y la sostenibilidad. El verdadero desafío no es solo la publicación, sino diseñar modelos de negocio escalables en el largo plazo para interactuar con clientes tanto particulares como empresariales.

Si nos planteásemos crear una app para nuestra tableta o teléfono móvil, como por ejemplo, una tienda virtual o un sitio web para publicar relatos sería necesario conocer los conceptos básicos de la programación, estudiar las diferentes estructuras de datos, aprender a manejar y resolver errores en nuestro código, instalar diferentes librerías y familiarizarnos con diferentes entornos de desarrollo.

Siguiendo los problemas de programación escritos en SQL, Python (nivel básico) y C# (nivel intermedio) de este libro, aprenderá a utilizar un sencillo entorno de desarrollo para ver cómo funcionan los ejemplos en su ordenador personal y así aprender la lógica necesaria para desarrollar aplicaciones y poder distribuirlas en un entorno multiusuario.

¿Para qué se usa Python en el desarrollo de software para redes locales? Se utiliza para automatización de tareas y análisis de datos con entornos de desarrollo web o intranet como Django o Flask.

¿Para qué se usa SQL o Microsoft Visual FoxPro en el desarrollo de software para redes de ordenadores? Para almacenar y recuperar información de productos, clientes, pedidos, empleados, salarios, etc.

Este libro consta de 3 partes. El contenido de la primera parte de este libro (Parte 1) está dividido, a su vez, en tres apartados: Introducción a la programación, Instalación y más de 50 ejercicios prácticos. La segunda parte (Parte 2) trata de cómo crear bases de datos simples con los lenguajes SQL y Microsoft Visual FoxPro. La tercera parte (Parte 3) versa sobre programación en C# (lenguaje necesario para crear aplicaciones con Unity 6).

Introducción a la programación.

Describe los pasos básicos que el usuario necesita conocer para empezar a utilizar el entorno de programación, y desarrollar y probar (ejecutar) los ejemplos de los programas incluidos en este libro.

Instalación.

Indica cómo instalar las herramientas que se utilizarán para desarrollar las diferentes aplicaciones (ejemplos incluidos).

Ejercicios prácticos.

Esta sección se conforma de más de 50 ejemplos prácticos (también llamados **prácticas**). Éstas muestran al desarrollador los pasos para desarrollar cada programa.

¿Qué ejercicios prácticos realizaremos?

1. Hola Mundo. Primera aplicación para explicar la estructura de las aplicaciones.
2. Textos. Aplicación que realiza preguntas al usuario y muestra las respuestas en pantalla.
3. Cálculos. Sumas y restas. Introducción a las variables numéricas.
4. Funciones. La función `int()`. Cálculo del área de un rectángulo.
5. Cálculo del área de un triángulo.
6. Creación de un programa que hace preguntas a los alumnos de una clase.
7. Creación de un test básico sobre Microsoft Windows.
8. Implementación de funciones definidas por el usuario (Parte 1).
9. Implementación de funciones definidas por el usuario (Parte 2).
10. Implementación de funciones con parámetros definidas por el usuario.
11. Creación de un test sobre Microsoft Windows (implementando funciones con parámetros).
12. Creación de una función que suma 4 números (implementando una función con parámetros).
13. Cálculo del volumen de un cubo.
14. Cálculo del volumen de una esfera (utilizando la función `multiplica`).
15. Cálculo del volumen de una esfera (utilizando la librería **`math`**).
16. Cálculo del volumen de un cilindro.
17. Hola Mundo (en una ventana en entorno gráfico utilizando la librería **`flet`**).
18. Hola Mundo (con mensaje centrado).
19. Hola Mundo (con cambio de color).
20. Hola Mundo (con cambios de color).
21. Creación de una tabla.
22. Creación de una tabla 5 x 2 con bordes.
23. Creación de una tabla 5 x 3 con bordes.
24. Creación de una tabla 5 x 5 con bordes.
25. Creación de una tabla 5 x 5 y cálculo de la suma de valores de la fila uno de la tabla.
26. Creación de una aplicación que imprima texto en el centro de la pantalla con fondo azul.
27. Creación de una aplicación que obtenga una entrada por parte del usuario e imprima textos en la pantalla.
28. Creación de una aplicación que obtenga una entrada por parte del usuario e imprima textos (bloqueados o sin bloquear) en la pantalla.
29. Creación de una aplicación que imprima el resultado de **sumar** dos números.

30. Creación de una aplicación que imprima el resultado de sumar dos números y añada dos líneas para dar más información al usuario.
31. Creación de una aplicación que imprima el resultado de sumar dos números e introduzca el resultado en un objeto `TextField`.
32. Creación de una aplicación que imprima el resultado de sumar o restar dos números e introduzca el resultado en un objeto `TextField`.
33. Creación de una aplicación que genere una clave (password) combinando dos palabras introducidas por el usuario.
34. Creación de una aplicación que genere una clave (password) combinando números, símbolos y dos palabras introducidas por el usuario.
35. Creación de una aplicación que genere una clave (password) con una longitud mínima de 13 caracteres (combinando números, símbolos y dos palabras introducidas por el usuario).
36. Creación de una aplicación que genere una clave con una longitud de 17 caracteres (combinando números, símbolos y palabras introducidas por el usuario).
37. Creación de una aplicación que genere un número aleatorio.
38. Creación de una aplicación que genere 5 números aleatorios.
39. Juego de los dados. Creación de una aplicación que genere 2 números aleatorios.
40. Creación de una presentación con una diapositiva compatible con Microsoft PowerPoint utilizando la librería **`python-pptx`**.
41. Presentación de una historia con 2 diapositivas utilizando librería PPT para crear un archivo compatible con Microsoft PowerPoint.
42. Creación de un archivo Microsoft Excel utilizando la librería **`openpyxl`**.
43. Creación de un archivo Microsoft Excel con sumas y restas.
44. Creación de un archivo con datos compatible con Microsoft Excel para realizar la suma del contenido de varias celdas.
45. Creación de un archivo con datos de una lista de contactos compatible con Microsoft Excel.
46. Creación de textos: Tabla con dos filas y dos columnas con nombres de pueblos de las Islas Canarias.
47. Creación de textos: Tabla con dos filas y dos columnas con números y palabras.
48. Creación de textos: Tabla con dos filas y dos columnas con las coordenadas de cada celda de la tabla.
49. Creación de textos con números: Tabla con dos filas y dos columnas con números enteros. Realización de la suma de los números enteros con los valores de la tabla.
50. Creación de una tabla con dos filas y tres columnas a partir de los datos de una tabla (con dos filas y dos columnas).
51. Creación del juego “Preguntas y respuestas” utilizando la librería **`tabulate`**.

Apéndices:

- Código fuente de un programa muy simple que hace un test de repaso al usuario de 49 preguntas sobre Microsoft Windows y el lenguaje de programación Python.
- Código fuente de un programa (optimizando el anterior programa mediante la implementación de listas) que hace un test al usuario de 2 preguntas sobre Python
- Las 12 funciones de Excel (y LibreOffice Calc) más utilizadas en el mundo empresarial.
- Código fuente de un proyecto sencillo para visualizar registros de una base de datos SQLite utilizando Python.
- Algunos aspectos básicos sobre redes y seguridad informática.
- La comunicación en entornos TIC.

Todos los ejemplos desarrollados en este libro con C# 12 y Python 3.8 fueron probados en un ordenador equipado con Microsoft Windows 10 y la plataforma de desarrollo .NET versión 8 (Long-term support).

A quién va dirigido este libro.

Este libro va dirigido a cualquier persona que desee adquirir las competencias digitales básicas necesarias para programar aplicaciones sencillas en Python 3, aplicaciones CMS básicas o juegos simples creados en C# (lenguaje necesario para desarrollar aplicaciones en Unity 6).

No es necesario haber programado en ningún lenguaje de programación previamente pero tener un nivel de inglés A2 (CEFR) o superior puede ayudarle a entender mejor la documentación y los ejemplos comentados de este libro.

En este material educativo se tratan los conceptos básicos de redes locales de computadoras, bases de datos, SQL, programación de aplicaciones con los lenguajes C# y Python 3.

Para seguir y entender la complicación básica de cualquier aplicación informática se requiere paciencia, tiempo de estudio, lógica y capacidad de análisis para la realización de cada ejercicio.

Los primeros ejemplos escritos en Python incluidos en el libro son simples, para más adelante, encontrar ejercicios escritos en C#, Microsoft Visual FoxPro y SQL con un nivel de complejidad más elevado.

Para facilitar la lectura y el entendimiento de los ejercicios, se han establecido las siguientes convenciones:

- Uso de distintas fuentes para destacar conceptos. Para facilitar la identificación de los términos, se utilizan fuentes y estilos de texto distintos. Estos son los siguientes:
- `Courier New`. Fuente general utilizada en el código de las aplicaciones.
- *Cursiva*. Indican las propiedades de los objetos en Python o nombres de librerías utilizadas en este libro.
- **Negritas**. Señalan las palabras del diccionario de Python.
- “Palabras entre comillas”. Se refieren a los nombres asignados a las variables, handlers y stacks que se utilizan en los ejemplos. Veremos estas definiciones más adelante.
- **Resaltado de códigos**. Los códigos utilizados en las prácticas son señalados como figuras y resaltados en rectángulos.
- [Corchetes]. Notas del escritor-desarrollador.

Introducción

Los lenguajes de programación son el soporte para la creación de aplicaciones para redes locales, móviles y servidores. Un lenguaje de programación es similar a un idioma que le permite hablarle a una computadora. Es un conjunto de instrucciones ordenadas que siguen un orden específico y que le dicen a la computadora qué hacer, paso a paso.

Para programar aplicaciones móviles o aplicaciones para computadoras personales o redes locales necesitamos introducir el concepto de librería o paquete de software externo (estos paquetes se instalan fácilmente).

Para crear aplicaciones sencillas, como las de este libro, no es imprescindible adquirir componentes o paquetes adicionales. Con el paquete de configuración estándar, es suficiente.

En este libro utilizaremos C#, Microsoft Visual FoxPro, SQL, Python. (No instale ni utilice Python 2.5. Los ejemplos de este libro funcionarán correctamente únicamente con Python, versión 3.8 o superior).

En aquellas prácticas en las que necesitemos añadir o conectar software externo, paquetes externos, o librerías específicas como **flet** o **flask**, se menciona claramente.

Flet está diseñada para Python 3. Es una biblioteca (librería o módulo) que ayuda a los nuevos programadores a crear interfaces gráficas de usuario (GUI) simples para dispositivos móviles, tablets u ordenadores de sobremesa. Si está comenzando con Python, esta librería es una gran herramienta para hacer programas más interactivos sin lidiar con las complejidades de otras bibliotecas GUI.

Los sistemas operativos en los que podemos ejecutar los ejemplos descritos en este libro son Kali Linux 2023.2, MacOS 15.3.2 y Windows 10.

Si desea desarrollar aplicaciones con Unity 6 y Linux, deberá tener en cuenta lo siguiente:

Es posible escribir código y compilar aplicaciones C# para Linux con Microsoft Windows. También es posible compilar aplicaciones C# para Windows directamente con Linux y probarlas con Wine o con Mono.

La última versión del entorno de trabajo (framework) .NET con soporte oficial de Wine en Linux es .NET Framework 4.8, lanzado en 2019. Sin embargo, la compatibilidad con versiones más recientes, como .NET 5, 6 u 8, es limitada, ya que Wine se centra principalmente en la compatibilidad con aplicaciones de Windows anteriores.

Para aplicaciones .NET modernas, podría considerar alternativas como:

- Mono: Una implementación de código abierto de .NET que funciona de forma nativa en Linux.
- .NET Core/.NET 5+: Compila aplicaciones directamente para Linux, sin necesidad de Wine. Puede compilar su código con VSCodium, una versión para Linux basada en Microsoft Visual Studio Code.

PARTE 1

INTRODUCCIÓN A LA PROGRAMACIÓN para REDES LOCALES

Introducción al Python Shell

Para trabajar con Python, debemos descargar Python 3 e instalarlo en nuestro PC. El equipo utilizado en los ejemplos es un ordenador Acer Aspire 5742G con 4 Gb de RAM, 320 GB de disco duro y un procesador Intel Core i3-370M. El sistema operativo es el Microsoft Windows 10.

Podríamos empezar a explicar los conceptos básicos sobre programación con pseudocódigo (forma no oficial de describir algoritmos) pero según mi experiencia, Python ofrece un entorno ideal para empezar a programar y realizar pruebas para ir adquiriendo conocimientos de forma gradual.

Python fue desarrollado para enfatizar la legibilidad del código, y su sintaxis permite a los programadores expresar conceptos en menos líneas de código. Una vez haya hecho y entendido los ejemplos con Python y SQL, estará listo para estudiar C# y posteriormente, crear aplicaciones con Unity.

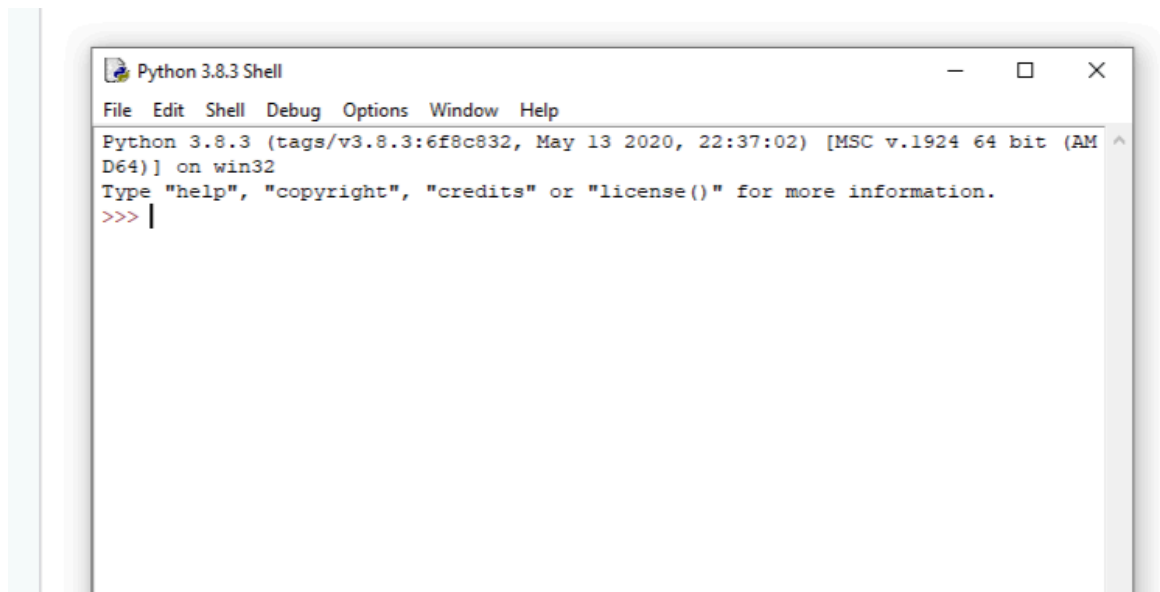
Importante: Si ya has utilizado Python 2.5, debes saber que la compatibilidad con Python 3 no es completa, así que podrías tener problemas si intentas utilizar ejemplos creados para Python 2.5.

Para utilizar el Python Shell o intérprete de comandos, necesitamos descargar Python 3 de la web oficial. El Python Shell, llamado también IDLE, podemos encontrarlo en la ruta que se describe unas líneas más abajo.

Utiliza tu navegador para visitar <http://www.python.org/download>

Descarga e instala Python 3. Una vez descargado e instalado, puedes comprobar que está correctamente instalado al hacer clic en:

Start > Programs> Python 3.x > IDLE

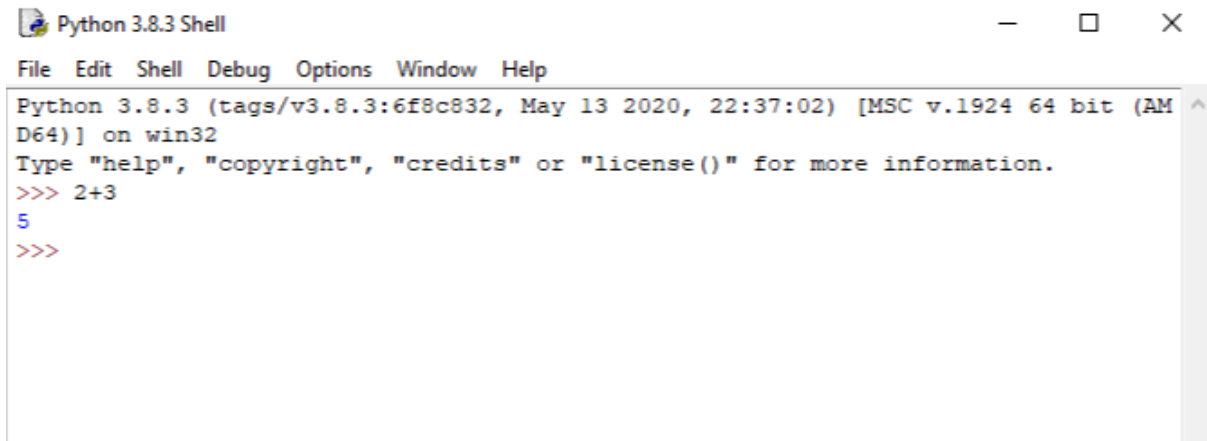


En la ventana anterior se puede ver el Python Shell. El Python Shell es un intérprete interactivo de comandos.

Un intérprete analiza cada comando u orden que se teclea, y lo ejecuta.

En el ejemplo de esta página hemos introducido el número “2”, el símbolo “+” y el número “3”. A continuación hemos pulsado <Enter> y el intérprete ha realizado el cálculo, mostrando el resultado, que es 5.

Ejemplo de suma de dos números:

A screenshot of a Python 3.8.3 Shell window. The window has a title bar that says "Python 3.8.3 Shell" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area shows the following text: "Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32", "Type 'help', 'copyright', 'credits' or 'license()' for more information.", and a prompt ">>>". Below the prompt, the user has entered "2+3", and the interpreter has responded with "5". The prompt ">>>" is shown again on the next line.

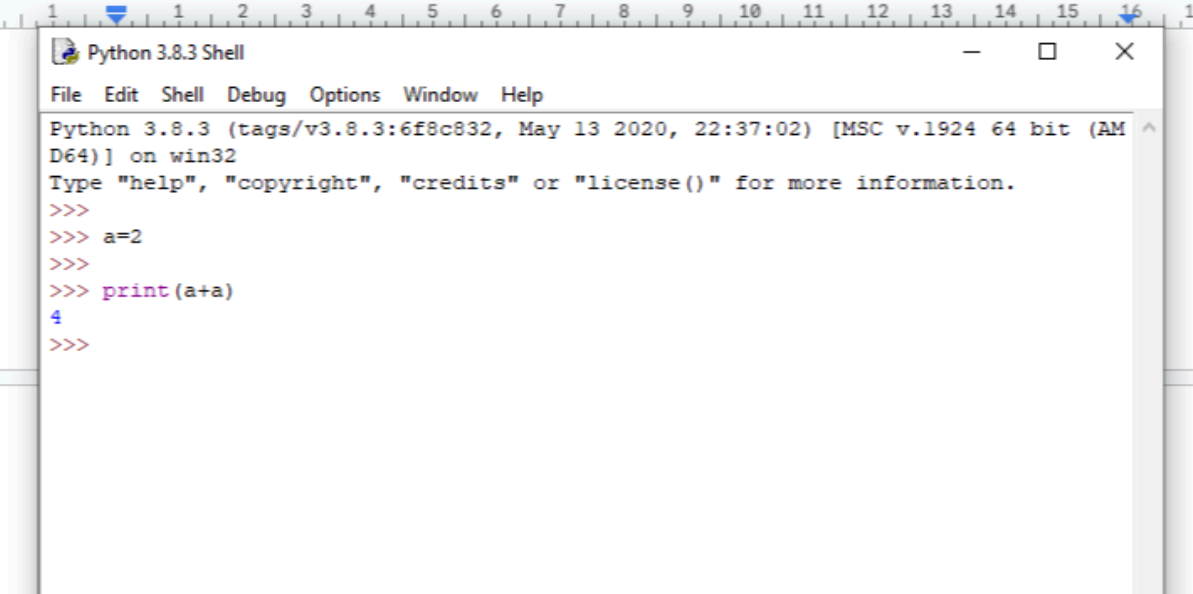
```
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> 2+3
5
>>>
```

Si ve (>>>), quiere decir que el intérprete de Python está preparado para recibir órdenes y crear programas escritos en lenguaje Python.

Si no desea utilizar el editor de código incluido en IDLE, puede utilizar cualquier editor de textos como Notepad, Microsoft Word, VSCodium o el entorno más moderno Microsoft Visual Studio Code.

Nuestra recomendación es que utilice por el momento IDLE hasta que se familiarice con el entorno de trabajo.

En el próximo ejemplo, mostramos otro ejemplo de cálculo. En este caso, hemos asignado el valor del número 2 a una variable llamada “a”. Más adelante introduciremos el concepto de “variables de memoria”. Por ahora, solo es necesario que practique este cálculo con diferentes letras: la letra “a”, que llamaremos variable “a”, almacena el valor 2 para después hacer un cálculo con esa variable y mostrar el resultado.



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
>>> a=2
>>>
>>> print(a+a)
4
>>>
```

Observe que hemos introducido el valor de “a” con la orden:

a=2

y acto seguido, hemos tecleado la instrucción para sumar el valor de “a”:

print(a+a)

A continuación, escribiremos nuestro primer programa utilizando IDLE.

Ejemplos Prácticos

Existen dos modos de interactuar con Python: El modo interactivo (sin crear programas) y escribiendo programas (un programa es una secuencia de instrucciones que persiguen un fin específico).

Utilizaremos el segundo método, escribiendo programas con un editor de textos.

Para poder escribir un programa necesitamos un editor de texto como el programa Notepad de Windows, aunque también podemos utilizar el editor interno del Python Shell.

Además, en el mercado existen numerosos editores de texto y con funciones diseñadas específicamente para ayudar a los programadores a ser más eficientes.

Es importante señalar que los programas deben ser copiados con sus puntos, sus comas, paréntesis, etc.. Un error en un símbolo hará que nuestro programa no funcione como esperamos, o que simplemente, no funcione.

Revise siempre dos veces su código.

Ejemplo 1. Hola Mundo.

Objetivo: Entender la estructura de un programa en Python mediante el desarrollo del ejemplo más básico: “Hola Mundo”.

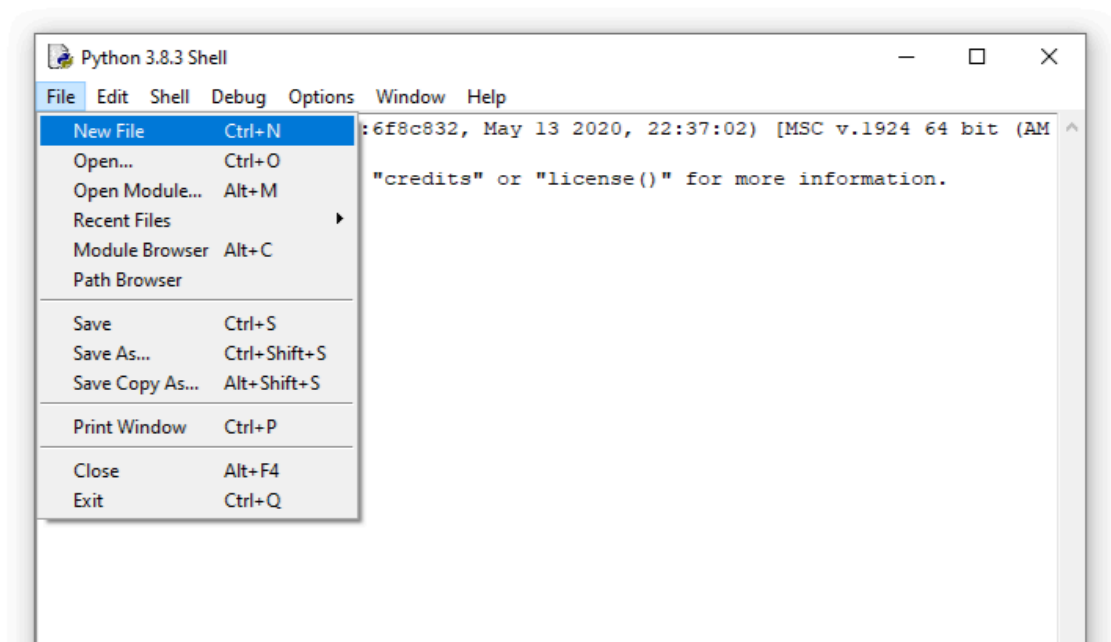
Para empezar, abra el Python Shell (el Python Shell es un intérprete interactivo de comandos).

Haga clic en:

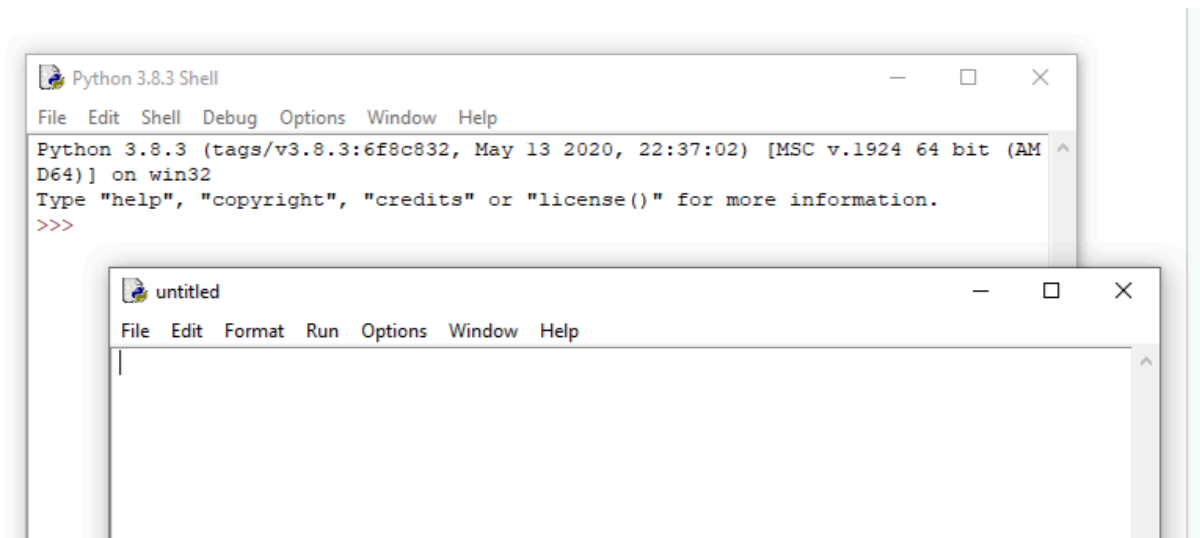
Start > Programs > Python 3.x > IDLE

Para crear un nuevo programa, haga clic en:

File > New File



A continuación, verá la pantalla de edición, llamada “untitled”, de su código:



En esta pantalla “untitled”, teclearemos algo que nos debe sonar muy familiar:

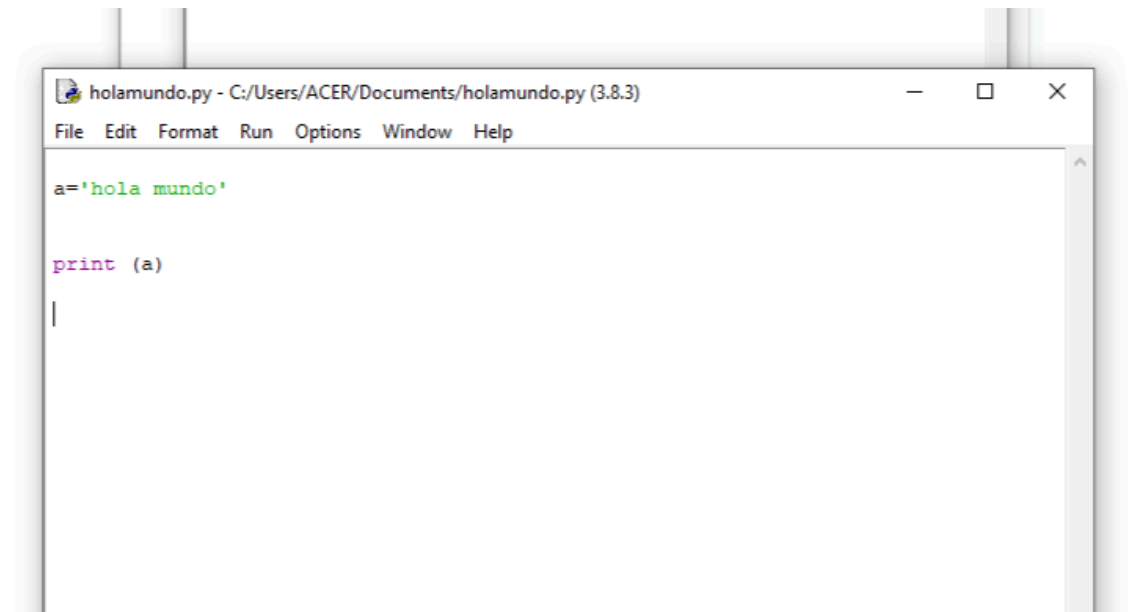
```
a="hola mundo"
```

```
print (a)
```

Y a continuación, haga clic en:

File > Save File as

Grabe su programa con el nombre de archivo “holamundo” en su PC en la carpeta “Mis Documentos”



Acaba de crear su primer programa en lenguaje Python llamado “holamundo.py”.

Recuerde grabar sus programas antes de ejecutarlos para no perder información.

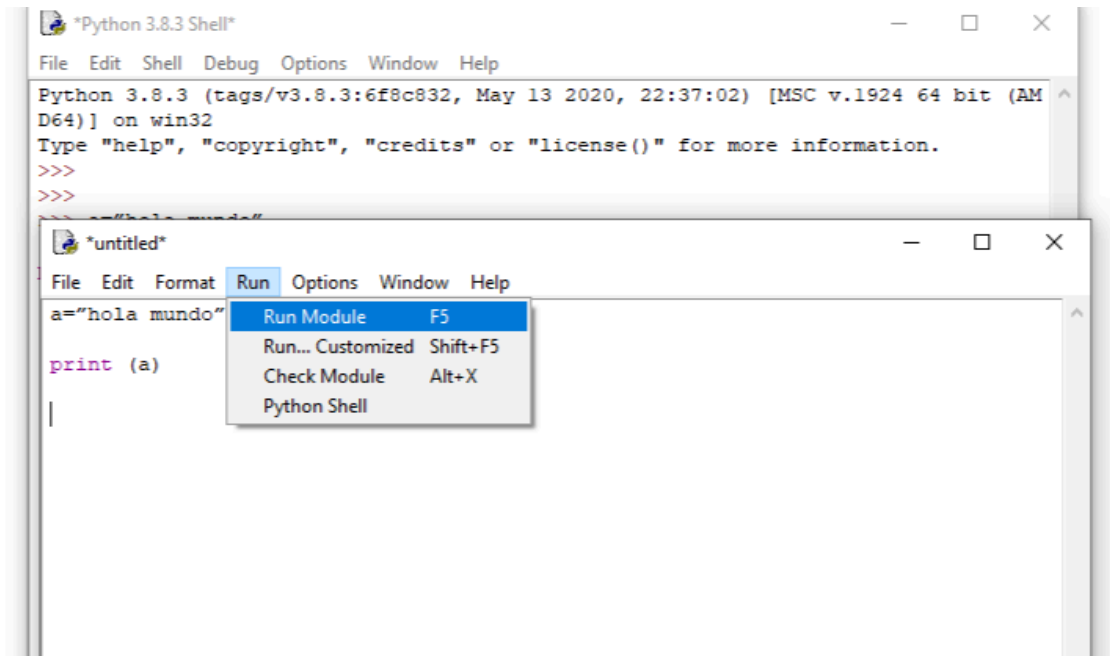
Recordatorio: La extensión de los archivos creados con Microsoft Word es .doc y la extensión de los programas creados con Python es .py

Si utiliza Microsoft Word para crear sus programas, recuerde cambiar la extensión .doc por la extensión .py.

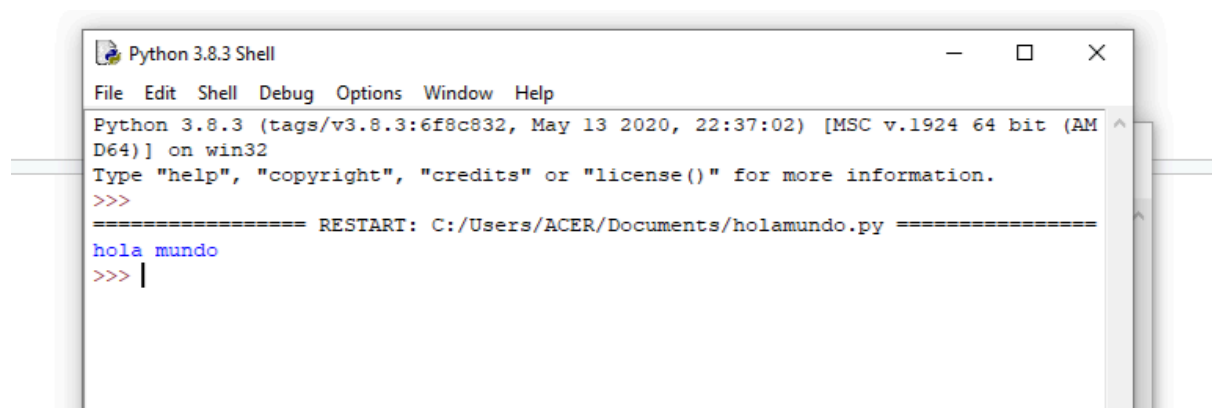
A medida que nuestro proyecto avance, iremos conociendo más extensiones.

Es muy útil escribir una lista de los programas que va terminando y otra lista con las extensiones que utiliza en un cuaderno de notas.

A continuación, pulse F5 para ejecutar su programa “holamundo”:



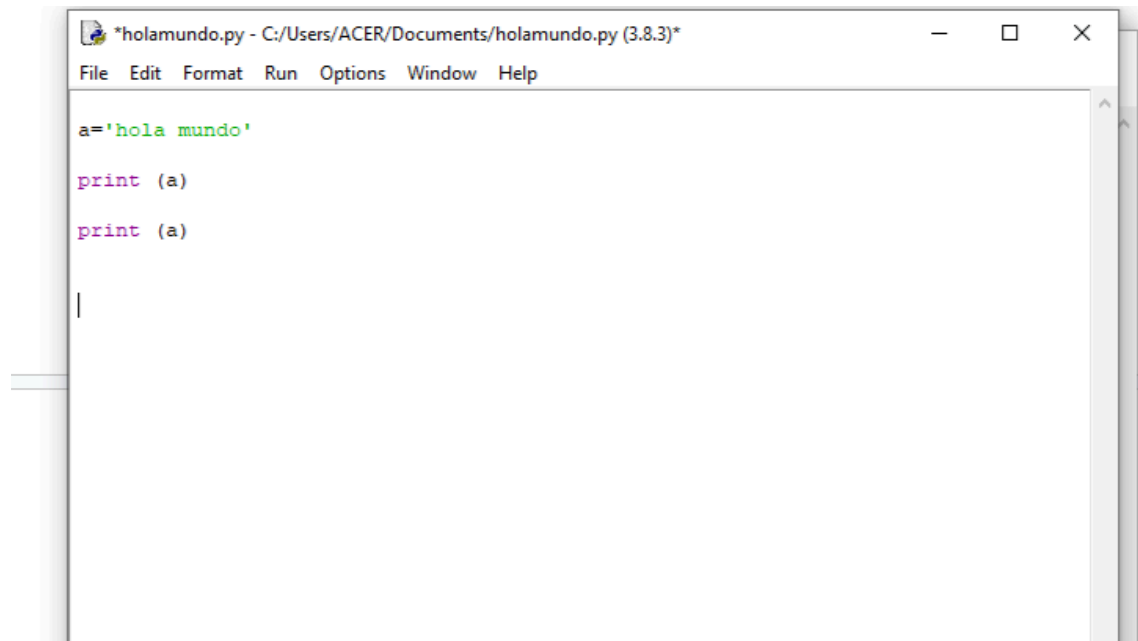
Este es el resultado:



El ordenador ha impreso en pantalla en color azul las palabras “hola mundo”.

Ahora modificaremos el programa para que imprima las palabras “hola mundo” dos veces.

Abra el archivo **holamundo.py** y añada otra orden **print(a)** justo debajo de la orden **print(a)** anterior:



```
*holamundo.py - C:/Users/ACER/Documents/holamundo.py (3.8.3)*
File Edit Format Run Options Window Help

a='hola mundo'
print (a)
print (a)
|
```

Importante:

No olvide grabar sus programas antes de ejecutarlos para no perder información. Puede usar la combinación de teclas CTRL+S para grabar el archivo antes de ejecutarlo.

A continuación, pulse F5 para ejecutar su programa modificado “holamundo”. Este es el resultado del programa que imprime las palabras “hola mundo” dos veces:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ACER/Documents/holamundo.py =====
hola mundo
hola mundo
>>> |
```

El ordenador ha impreso en pantalla en color azul las palabras “hola mundo” dos veces. Es decir, ha ejecutado la orden **print** dos veces.

Fin del Ejemplo 1.

Ejemplo 2. Textos y preguntas.

Objetivo: Aplicación que realiza preguntas al usuario y muestra las respuestas en pantalla.

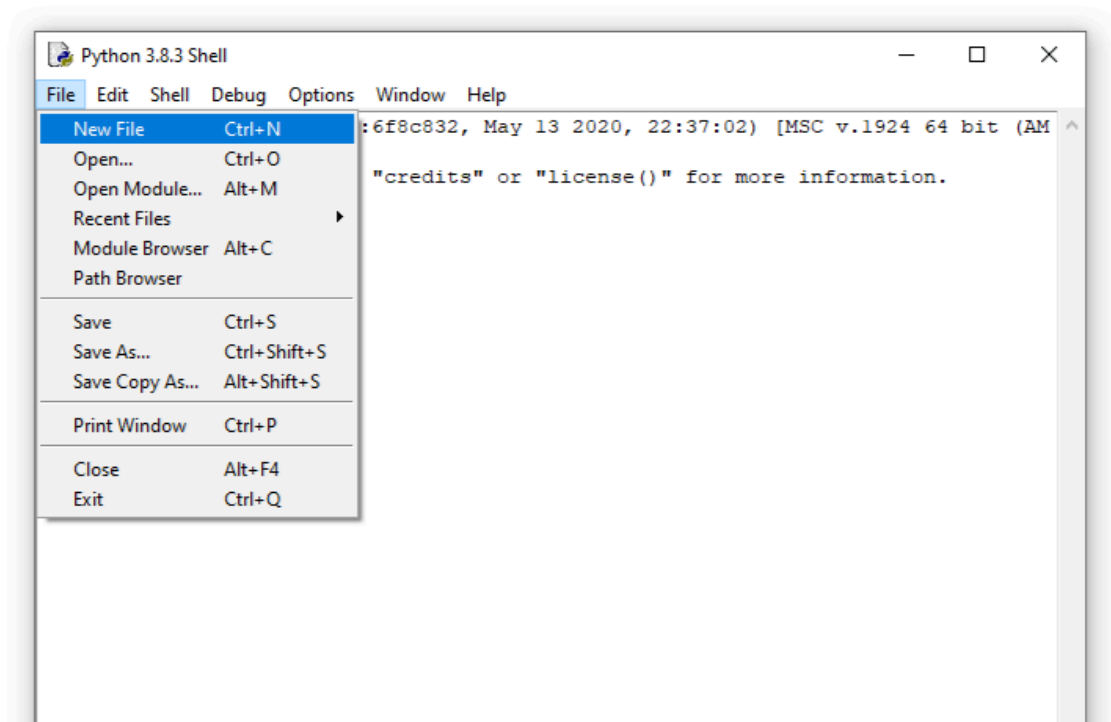
Empecemos por abrir el Python Shell (intérprete interactivo de comandos de Python):

Haga clic en:

Start > Programs > Python 3.x > IDLE

A continuación, haga clic en:

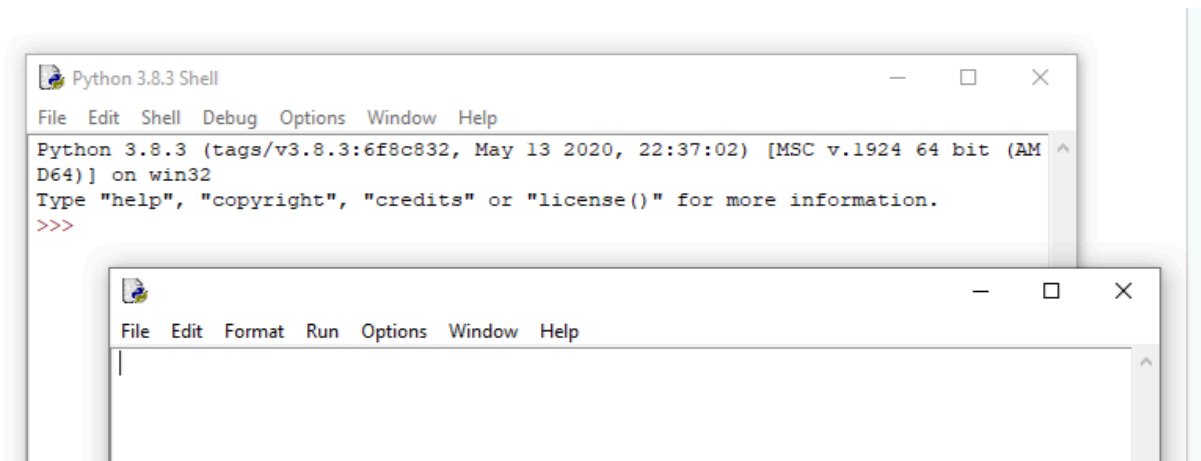
File > New File



Y, seguidamente, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**textos**” en su ordenador en la carpeta “Mis Documentos”



Ahora añada las siguientes en el archivo **textos.py**, que acaba de crear.

```
print ("- - - - -")  
  
print("Hola,")  
  
xname = input("Dime tu nombre, por favor: ")  
  
print("Hola, " + xname + "!")
```

A continuación, grabe el archivo (pulsando CTRL+S) y ejecute su programa.

Este es el resultado del programa que le pregunta su nombre y lo imprime en la pantalla:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ACER/Documents/textos.py =====
- - - - -
Hola,
-
Dime tu nombre, por favor: Juan
-
Hola, Juan!
- - - - -
>>>
```

Importante:

Hemos utilizado una variable tipo **String**. En Python existen cuatro estructuras básicas de datos: **Boolean**, **Integer**, **Float** y **String**. En el ejemplo 2 hemos utilizado una variable tipo **String** llamada **xname**. Esta variable puede almacenar cadenas de caracteres alfanuméricas. Más adelante trabajaremos con el resto de estructuras básicas de datos.

Fin del Ejemplo 2.

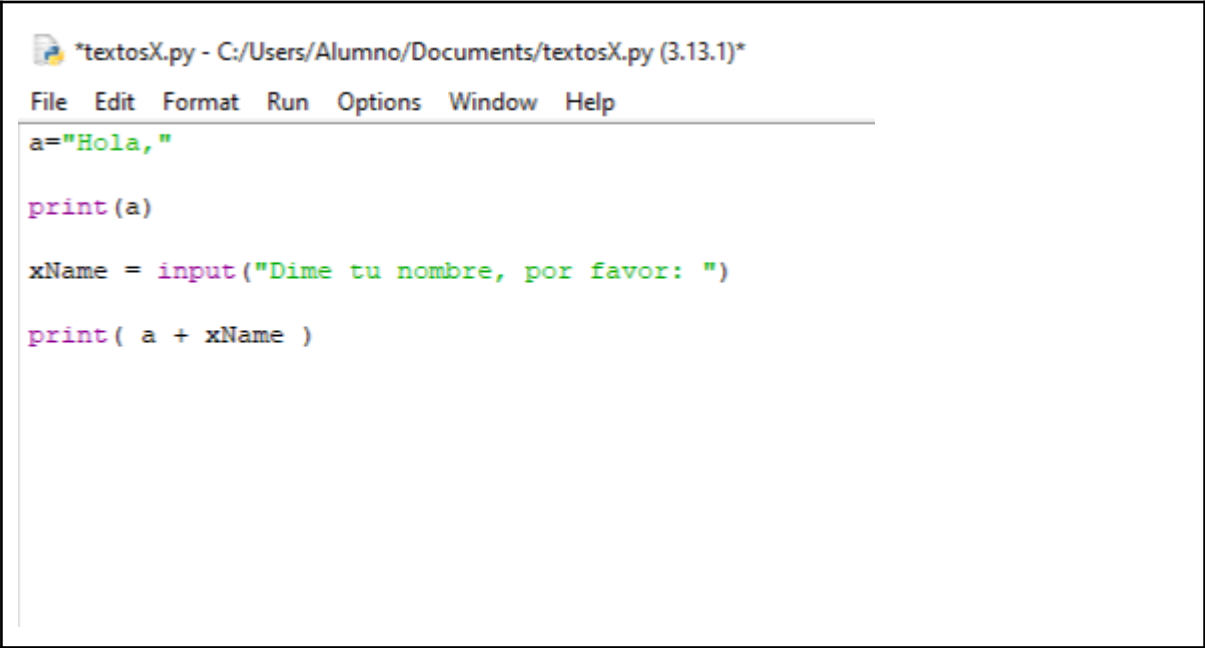
Reto:

Corrija los errores en el siguiente ejemplo de código escrito en Python:

```
a="Hola,  
  
print(a)  
  
xName = input("Dime tu nombre, por favor: ")  
  
prin( a + xName )
```

Grabe su archivo con el nombre [textosX.py](#).

Solución:



The screenshot shows a Python IDE window titled "*textosX.py - C:/Users/Alumno/Documents/textosX.py (3.13.1)*". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following corrected Python code:

```
a="Hola,"  
  
print(a)  
  
xName = input("Dime tu nombre, por favor: ")  
  
print( a + xName )
```

Ejemplo 3. Sumas y restas.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Haga clic en:

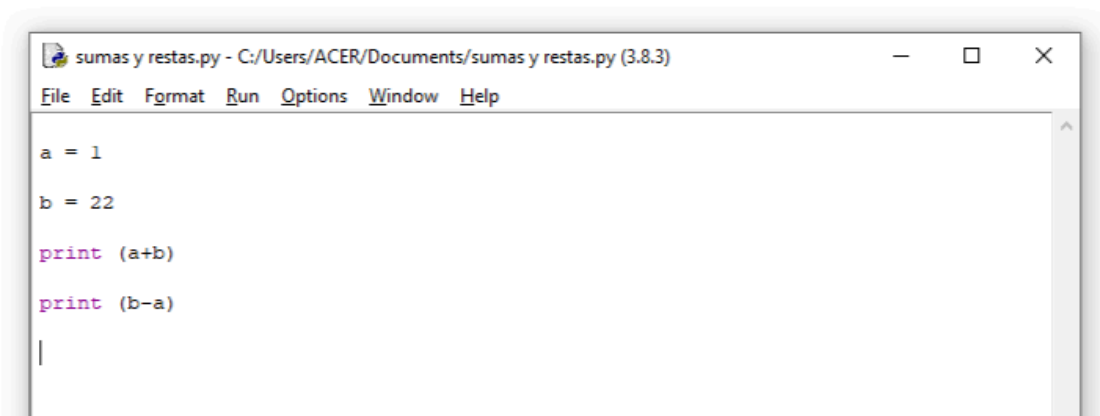
File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**sumas y restas.py**” en su ordenador en la carpeta “Mis Documentos”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
a = 1
b = 22
print (a+b)
print (b-a)
|
```

Preguntas: ¿Qué tipo de variables crees que hemos utilizado? ¿Integer (entero) o String (cadena)? ¿Has ejecutado el programa? ¿Cuál ha sido el resultado?

Fin del Ejemplo 3.

Ejemplo 4. Cálculo del área de un rectángulo.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Haga clic en:

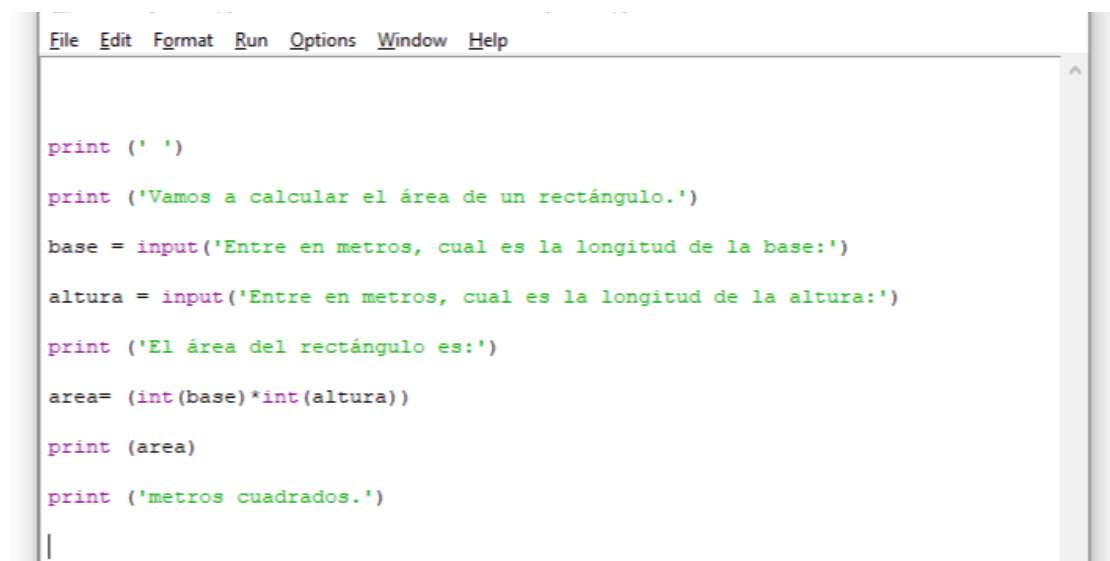
File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “rectángulo.py” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
File Edit Format Run Options Window Help

print ( ' ')
print ('Vamos a calcular el área de un rectángulo.')
base = input('Entre en metros, cual es la longitud de la base:')
altura = input('Entre en metros, cual es la longitud de la altura:')
print ('El área del rectángulo es:')
area= (int(base)*int(altura))
print (area)
print ('metros cuadrados.')
```

Preguntas: ¿Qué tipo de variables hemos utilizado? ¿Has ejecutado el programa?
¿Qué crees que hace la función **int (base)** ?

Fin del Ejemplo 4.

Ejemplo 5. Cálculo del área de un triángulo.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Haga clic en:

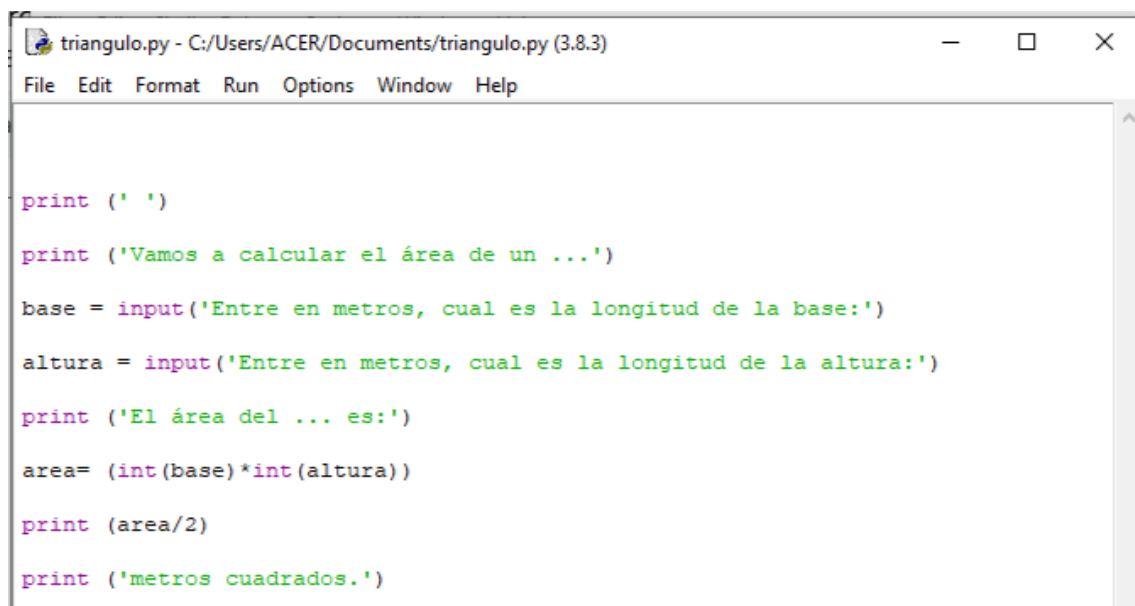
File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “triángulo.py” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes. Modifique el programa para que calcule el área de un triángulo.



```
print(' ')
print('Vamos a calcular el área de un ...')
base = input('Entre en metros, cual es la longitud de la base:')
altura = input('Entre en metros, cual es la longitud de la altura:')
print('El área del ... es:')
area = (int(base)*int(altura))
print(area/2)
print('metros cuadrados.')
```

Preguntas: ¿Qué variables hemos utilizado? ¿Has ejecutado el programa? ¿Ya dominas la función `print( )`?

Fin del Ejemplo 5.

Ejemplo 6. Creación de un programa que hace una pregunta a los alumnos de una clase de informática en una escuela de programación (creación de un test básico sobre Microsoft Windows).

Objetivo: Entender la sentencia **if** del lenguaje Python.

Una sentencia **if** esencialmente dice:

"Si la expresión evaluada, resulta ser verdadera (True), entonces ejecuta una vez el código en la expresión."

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo "**test.py**" en su ordenador en la carpeta "Mis Documentos"

Ahora que ya tiene creado el archivo **test.py** creado, añade las siguientes órdenes:

```
xname = input("Dime tu nombre, por favor: ")

print("Vamos a hacer un test sobre Microsoft Windows.")

puntos=0

xx = input("El comando COPY sirve para copiar archivos. (V/F)? ")

if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos = puntos+1
#endif

print("Tu nota final en este test es:")

print(puntos)
```

Preguntas: ¿Qué variables hemos utilizado para acumular puntos en nuestro test? ¿Cómo hemos comprobado que la respuesta es correcta? ¿Ya dominas la función **input()**?

Fin del Ejemplo 6.

Ejemplo 7. Creación de un programa que hace preguntas a los alumnos de una clase de informática (creación de un test básico sobre Microsoft Windows).

Objetivo: Entender la secuencia de la programación estructurada.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**test2.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
#Test básico escrito en Python

puntos=0

#PREGUNTA 1
xx= input("El comando COPY sirve para copiar archivos. (V/F)? ")

if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 2
xx= input("El comando DIR sirve para mover o renombrar archivos. (V/F)? ")

if xx=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#Imprimir resultado del test
print(puntos)
```

Fin del Ejemplo 7.

Reto:

Busque y corrija los errores en el siguiente ejemplo:

```
#Test básico escrito en Python

xname = input("Dime tu nombre, por favor: ")

a="Vamos a hacer un test de dos preguntas sobre"
b=" Microsoft Windows."
print(a+b)
print(" ")

#puntos=0
p=0

#PREGUNTA 1
xx= input("El comando COPY sirve para copiar archivos. (V/F)? ")

if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    p=p+1
#endif

#PREGUNTA 2
print(" ")
xx= input("La orden DIR sirve para mover o renombrar archivos. (V/F)?")

if xx=="F":
    print("Respuesta correcta. Tienes 1 punto."); p=p+1
#endif

print(" ")
print("Tu nota final en este test es:")
print(p)
```

Ejemplo 7r. Creación de un programa que hace preguntas sobre el sistema operativo Linux a los alumnos de una clase de informática.

Objetivo: Entender el funcionamiento de una estructura ordenada así como algunos conceptos básicos del sistema operativo Linux.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**test7r.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# test7r.py
# Importante:
# Linux es un sistema que permite que múltiples usuarios utilicen la
# misma computadora a la vez, manteniendo sus archivos totalmente
# seguros y separados en áreas diferentes.
# Cada usuario tiene una clave para su propia área de trabajo.
# El administrador del sistema Linux, que se llama root, tiene una
# clave maestra. El root puede reparar el sistema y crear reglas que
# se aplicarán en todos los equipos dependientes instalados en la red
# administradas por él.

puntos=0

#PREGUNTA 1
xx= input("El comando cp (en Linux) sirve para copiar archivos. (V/F)? : ")

if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 2
xx= input("El comando DIR (en Linux) sirve para mover archivos. (V/F)? : ")

if xx=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#Imprimir resultado del test7r
print(puntos)
```

Fin del Ejemplo 7r.

Ejemplo 7s. Creación de un programa que hace preguntas sobre el sistema operativo Linux a los alumnos de una clase de informática.

Objetivo: Entender el funcionamiento de una estructura ordenada así como algunos conceptos básicos del sistema operativo Linux.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**test7s.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# test7s.py
# Nota:
# Recuerde que Linux es un sistema multiusuario.
# Un usuario puede estar sentado frente a una pantalla utilizando el
# ordenador, mientras otros 3 usuarios están conectadas a ese mismo
# ordenador desde sus despachos en diferentes pisos a través de la
# red del edificio sin interferir en los diferentes archivos del
# área de trabajo de cada uno.
# El administrador del sistema Linux, que se llama root, tiene una
# clave maestra. Puede asignar prioridades y velocidad de proceso a
# las tareas y programas ejecutados por la máquina principal.

puntos=0

#PREGUNTA 1
xx= input("El comando mv (en Linux) sirve para mover archivos. (V/F)? : ")

if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA ____
xx= input("El comando lc (en Linux) sirve para borrar archivos. (V/F)? : ")

if xx=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#Imprimir resultado del _____
print(puntos)
```

Fin del Ejemplo 7s.

Ejemplo 7ss. Creación de un programa que hace preguntas sobre el sistema operativo Linux.

Objetivo: Entender el funcionamiento de una estructura ordenada así como algunos conceptos del sistema operativo Linux y la función “servidor de archivos”.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa con el nombre de archivo “**test7ss.py**” en su ordenador.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# test7ss.py
# Nota:
# Recuerde que Linux es un sistema multiusuario.
# Un usuario puede estar sentado trabajando en sus archivos en su
# PC mientras otros usuarios están conectados a ese PC u
# ordenador central, llamado servidor, desde sus despachos.
# Entre ellos debe haber una conexión física (cable) o una conexión
# inalámbrica por radiofrecuencia (WIFI).
# El administrador del sistema Linux, que se llama root, tiene una
# clave maestra para conectar diferentes PC en la red.
# El administrador puede dar permisos especiales a cada usuario para
# almacenar archivos en el servidor de archivos (ordenador central) y
# que nunca queden grabados en sus respectivos ordenadores.
# Cada usuario tendrá sus propios archivos en una carpeta del servidor
# de archivos (ordenador central).
# Si un usuario necesita leer un archivo de otro compañero, éste deberá
# imprimirlo, copiarlo en un pendrive o mandarlo por correo. Si el
# usuario hace un cambio en el archivo original, el resto de usuarios no
# verá el cambio realizado.

puntos=0

#PREGUNTA 1
xx= input("El comando ls (en Linux) sirve para borrar archivos. (V/F)?:")

if xx=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA ____
xx= input("cat (en Linux) sirve para mostrar el contenido de un archivo:")
if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#Imprimir resultado del ____
print(puntos)
```

Fin del Ejemplo 7ss.

Ejemplo 7s. Creación de un programa que hace preguntas sobre el sistema operativo Linux a los alumnos de una clase de informática.

Objetivo: Entender el funcionamiento de una estructura ordenada así como algunos conceptos básicos del sistema operativo Linux.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**test7s.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# test7s.py
# Nota:
# Recuerde que Linux es un sistema multiusuario.
# Un usuario puede estar sentado frente a una pantalla utilizando el
# ordenador, mientras otros 3 usuarios están conectados a ese mismo
# ordenador desde sus despachos en diferentes pisos a través de la
# red del edificio sin interferir en los diferentes archivos del
# área de trabajo de cada uno.
# El administrador del sistema Linux, que se llama root, tiene una
# clave maestra. Puede asignar prioridades y velocidad de proceso a
# las tareas y programas ejecutados por la máquina principal.

puntos=0

#PREGUNTA 1
xx= input("El comando rm (en Linux) sirve para borrar archivos. (V/F)? : ")

if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA ____
xx= input("El comando lss (en Linux) sirve para borrar archivos. (V/F)? :")

if xx=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#Imprimir resultado del _____
print(puntos)
```

Fin del Ejemplo 7s.

Ejemplo 7t. Creación de un programa que hace preguntas sobre el sistema operativo Linux y la compartición de archivos.

Objetivo: Entender el funcionamiento básico de la compartición de archivos del sistema operativo Linux.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**test7t.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# test7t.py
# Lea atentamente:
# Linux es un sistema multiusuario que puede compartir archivos.
# Un usuario puede estar sentado frente a una pantalla utilizando el
# ordenador, mientras otros usuarios están conectados a ese mismo
# ordenador central en diferentes pisos a través de la
# red del edificio sin interferir en los diferentes archivos del
# área de trabajo de cada uno.
# El administrador del sistema, que se llama root, tiene una
# clave maestra. Puede asignar permisos a los diferentes programas
# ejecutados o archivos leídos en la máquina principal.

puntos=0

#PREGUNTA 1
print("Lee y responde si la siguiente afirmación es verdadera:")
print("Si un usuario trabaja en un proyecto con tres compañeros")
print("en una oficina conectados en una red WIFI a un servidor de ")
print("archivos donde se almacena el documento del proyecto en una")
print("carpeta compartida, todos podrán editar el archivo si el")
print("administrador ha otorgado a todos permiso para editar el")
print("archivo del proyecto.")

xx= input("Verdadero o Falso (V/F)? : ")
if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA ____
print("Lee y responde si la siguiente afirmación es verdadera:")
print("Si tres usuarios desean ver un archivo de video de un")
print("documental en 3 ordenadores diferentes, el archivo debe ")
print("guardarse con los permisos adecuados en una carpeta")
print("compartida para ahorrar espacio, donde todos podrán leerlo si el")
print("administrador ha otorgado permisos de lectura para cada")
print("archivo de esa carpeta.")

xx= input("Verdadero o Falso (V/F)? : ")
if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#Imprimir resultado del _____
print(puntos)
```

Fin del Ejemplo 7t.

Ejemplo 7tv. Creación de un programa que hace preguntas sobre el sistema operativo Linux y las copias de seguridad.

Objetivo: Entender la importancia de las copias de seguridad en el sistema operativo Linux.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre “**test7tv.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# test7tv.py
# Lea atentamente:
# Un usuario puede estar sentado frente a una pantalla haciendo
# copias de seguridad en un ordenador central mientras otros usuarios
# están conectados a Internet.
# El administrador del sistema, que se llama root, tiene una
# clave maestra. Puede asignar permisos a los diferentes programas
# ejecutados o archivos leídos en la máquina principal.

puntos=0

#PREGUNTA 1
print("Lee y responde si la siguiente afirmación es verdadera:")
print("Es más fácil hacer copias de seguridad (backups) en Linux")
print("de una máquina central (servidor de archivos) que ir ")
print("a cada PC de cada usuario para copiar esos archivos")
print("en un disco de respaldo.")

xx= input("Verdadero o Falso (V/F)? : ")
if xx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA ____
print("Responde si la siguiente afirmación es verdadera o falsa:")
print("Un administrador de red no puede decidir exactamente quién")
print("entra a una unidad de red, carpeta o ve determinados archivos.")

xx= input("Verdadero o Falso (V/F)? : ")
if xx=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#Imprimir resultado del _____
print(puntos)
```

Fin del Ejemplo 7tv.

Ejemplo 8. Cálculo del volumen de un cubo.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla implementando una función definida por el usuario.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**cubo.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
# Definición de funciones

def multiplica(a,b,c):
    print("Multiplicaremos los siguientes valores:")
    print(a,b,c)
    resultado=a*b*c
    return resultado

##### Programa principal (main program)

ax="Vamos a calcular el volumen de un cubo (poliedro regular)."
print(ax)
bx="Todas las aristas miden lo mismo -en metros-"
print(bx)

total= multiplica(3, 3, 3)
print("El total es:")
print(total)
print("metros cubicos.")
```

Preguntas: ¿Qué variables hemos utilizado? ¿Has ejecutado el programa? ¿Ya dominas la función **print()**?

Fin del Ejemplo 8.

Ejemplo 9. Creación de un test básico sobre Microsoft Windows (implementando funciones definidas por el usuario).

Objetivo: Entender el funcionamiento de una función definida por el usuario. Entender cómo se define una función y llamada a la función definida.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**función.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# Definición de funciones función.py

def examina():
    puntos=0
    #PREGUNTA 1
    x= input("El comando COPY en Windows sirve para copiar archivos.(V/F)?: ")
    if x=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 2
    xx= input("El comando DIR sirve para mover o renombrar archivos.(V/F)?: ")
    if xx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    print("Tu nota final es:")
    print(puntos)

##### Programa principal (main program)

xname = input("Dime tu nombre, por favor: ")
print("Hola, " + xname + "!")
a="Vamos a hacer un test dos preguntas sobre"
b=" Microsoft Windows."
print(a+b)
examina()
print("Fin de la prueba.")
```

Explicación:

El programa está dividido en dos partes: La definición de la función, llamada **examina**, y el programa principal. La función siempre debe ser escrita antes del programa principal.

Podemos diseñar funciones para ejecutar una tarea o varias. Dentro de una función pueden existir una o varias órdenes. Un programa puede tener más de una función.

Veamos un ejemplo:

Imagina que trabajas con un asistente personal con inteligencia artificial como Alexa o Siri. En este ejemplo, vamos a llamar “**Examina**” a nuestro asistente.

Ahora necesitas que **Examina** (que es nuestro asistente personal) ejecute una acción por tí. Para que el asistente complete la tarea, necesitas decir su nombre y ordenarle que ejecute esa tarea programada. Los programadores hacemos algo similar con las funciones.

Primero, creamos el asistente personal (función) en el bloque “definición de funciones”:

```
def examina() :  
    print("Hola, vamos a hacer un test")
```

Acto seguido, para que el asistente haga su trabajo, simplemente escribimos su nombre seguido de paréntesis en el “programa principal” (main program):

```
examina()
```

Cuando tecleamos **examina()** en la segunda parte de nuestro código, el asistente (función) hace lo que le pedimos, que en este caso es decir “¡Hola, vamos a hacer un test”.

Fin del Ejemplo 9.

Ejemplo 10. Creación de un test sobre Microsoft Windows (implementando dos funciones definidas por el usuario).

Objetivo: Entender el funcionamiento de una función definida por el usuario. Llamada a varias funciones desde el programa principal.

Cree un nuevo fichero para el próximo ejercicio y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**función2.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# Definición de funciones
```

función2.py

```
def examen():
    puntos=0
    #PREGUNTA 1
    x= input("El comando TREE en Windows sirve para copiar archivos.(V/F)?: ")
    if x=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 2
    xx= input("El comando DEL sirve para mover o renombrar archivos.(V/F)?: ")
    if xx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    print("Tu nota final es:")
    print(puntos)
```

```
def felicitacion():
    print("Felicitaciones, siga estudiando así.")
    print("Estamos orgullosos de usted.")
```

```
##### Programa principal (main program)
```

```
xname = input("Dime tu nombre, por favor: ")
print("Hola, " + xname + " !")
a="Vamos a hacer un test dos preguntas sobre "
b="Microsoft Windows."
print(a+b)
examen()
felicitacion()
print("Fin de la prueba.")
```

Observe que hemos creado dos funciones en el bloque “definición de funciones”:

```
def examen() :  
    . . .  
  
def felicitacion() :  
    . . .
```

Acto seguido, para que el programa se ejecute correctamente, simplemente escribimos el nombre seguido de paréntesis en el “programa principal” de las funciones que queremos ejecutar:

```
examen()  
felicitacion()
```

El hecho de dividir los programas en diferentes funciones crea un código con un número total de líneas menor. Esto implica hacer un proyecto más limpio y legible. Si necesitáramos hacer modificaciones a nuestro programa, sería más fácil identificar la función que necesita ser modificada.

¡Recuerde! Revise dos veces siempre los símbolos en todas las instrucciones `input()` en su código.

Fin del Ejemplo 10.

Ejemplo 11. Creación de un test sobre Microsoft Windows (implementando funciones con parámetros).

Objetivo: Entender el funcionamiento de una función definida por el usuario. Llamada a una función con parámetros.

Cree un nuevo fichero para el próximo ejercicio y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**función3.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```

# Definición de funciones
función3.py

def examen():
    puntos=0
    #PREGUNTA 1
    xxx = input("El comando FORMAT en Windows sirve para copiar archivos. (V/F)? ")
    if xxx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 2
    xxx = input("El comando REN sirve para copiar archivos. (V/F)? ")
    if xxx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    felicitacion(puntos)

def felicitacion(p):
    print("Felicitaciones, siga estudiando así.")
    print("Estamos orgullosos de usted.")
    print("Tu nota final es:")
    print(p)

#### Programa principal (main program)

xname = input("Dime tu nombre, por favor: ")
print("Hola, " + xname + "!")
a="Vamos a hacer un test de dos preguntas sobre "
b="Microsoft Windows."
print(a+b)
examen()
print("Fin de la prueba.")

```


Observe que hemos creado dos funciones en el bloque “definición de funciones”:

```
def examen() :
    . . .

def felicitacion(p) :
    . . .
```

Acto seguido, para que el programa se ejecute correctamente, simplemente escribimos el nombre seguido de paréntesis en el “programa principal” de la función que queremos ejecutar (no incluimos la llamada a la segunda función en el programa principal porque será llamada desde la función **examen()**).

El hecho de dividir los programas en diferentes funciones con parámetros crea un código con un número total de líneas menor, debido a que podemos reutilizar las funciones.

Observe que en este programa la función **examen()** llama en la última línea a la función **felicitación()**. Ahora la función **felicitación()** es la que imprime los puntos del test. Para hacerlo, la forma en la que hacemos la llamada a la función **felicitación()** ha cambiado. Ahora debe incluir la variable **puntos** entre paréntesis (que es el valor que le pasa a la función **felicitación**). La utilizaremos de la siguiente forma:

```
felicitación(puntos)
```

Nota: Las variables utilizadas como argumentos en la definición de la función **felicitación** deben ser distintas a las enviadas en la llamada a la función definida. Para hacerlo, hemos utilizado la variable **puntos** como argumento y la variable **p** en la definición de la función.

Fin del Ejemplo 11.

Reto 1:

Corrija y modifique el siguiente código para que los parámetros en la función actúen correctamente e impriman en pantalla el mensaje “Hola Mundo”.

```
##
## Definimos la función
##

def hello_funcion(parametro1, parametro2):
    print(parametro1, parametro2)    ### La función utiliza dos parámetros

#####

##
## Programa principal: Llamamos a la función definida anteriormente
##

hello_funcion("Hola", "Mundo")      ### Llamamos a la función y pasamos los argumentos
```

Nota: No olvide que al usar parámetros al definir una función, los parámetros deben ir entre paréntesis. De la misma manera, los argumentos pasados a la función, en este caso, **“Hola”** y **“Mundo”**, deben ir entre paréntesis.

Reto 2:

Modifica el siguiente código para que el parámetro **p** en la función **examen(p)** actúe como un interruptor para controlar qué preguntas se incluyen en el test.

```
##
##
## Test de 2 preguntas (se sumará un punto por cada respuesta correcta introducida)
##
## El test utiliza la función examen con un interruptor, llamado p, para elegir preguntas:
##
## si p=1, entonces solo hacemos pregunta 1
## si p=2, entonces hacemos preguntas 1 y 2
##
##
def examen(p):
    puntos = 0
    if p==1 or p==2:
        print(" ")
        ##Pregunta 1
        x = input("El S.O. más usado en el mundo en 2025 fue Microsoft Windows (V/F)? ")
        if x.upper == "V":
            print("Respuesta correcta. Tienes 1 punto.")
            puntos = puntos + 1
        #endif

    if p==2:
        ## Hacemos la pregunta 2 si el parámetro pasado es 2.
        ##Pregunta 2
        x = input("El S.O. más usado en el mundo en 2025 no fue Linux (V/F)? ").upper()
        if x == "V":
            print("Respuesta correcta. Tienes 1 punto.")
            puntos = puntos + 1

    print("Tu nota final en este test es:", puntos)

##
## Programa principal. Main program.
##
print(" ")
print("Vamos a hacer un test de 2 preguntas sobre tecnología."); print(" ")
examen(2)
```

Nota: No olvide que al usar parámetros al definir una función, el parámetro debe ir entre paréntesis. De la misma manera, el argumento pasado a la función, en este caso, **2**, debe ir entre paréntesis.

Reto 3:

Modifica el siguiente ejemplo de código para que se ejecute correctamente. El usuario debe hacer un test de 3 preguntas sobre Internet, CMS, Wordpress y Tiendas Virtuales en España.

```
# Vamos a hacer un programa que realiza un test de dos preguntas sobre
# Microsoft Windows, CMS, Wordpress y Tiendas Virtuales en España.
# Un CMS (Sistema de Gestión de Contenidos) es una herramienta que permite
# crear y administrar contenido digital sin necesidad de saber programar.
# Wordpress.com es una plataforma de blogging y creación de
# sitios web basada en el CMS Wordpress, pero alojada y gestionada por
# Wordpress, lo que facilita su uso y mantenimiento.

# Definición de funciones

def examen():
    puntos=0

    #PREGUNTA 1
    A=" virtual, no es necesario comunicar la apertura a la Agencia Tributaria. (V/F)?: "
    xx=input("Si no se tienen beneficios en el primer mes de actividad de una tienda "+A)
    if xx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 2
    A="autónomo en España está ligada a la habitualidad y la finalidad lucrativa de la..."
    B="actividad o grado de discapacidad?"
    xxx = input("Completa la frase: La obligación de darse de alta como ""+A+B)
    if xxx=="actividad":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 3
    A="ha sido publicado con wordpress.com correctamente, debemos visitar la "
    B="dirección "
    C="siguiente:"
    F=A+B+C
    xxx = input("Completa la frase: Para comprobar que nuestro sitio web ""+F)
    if xxx=="https://wordpress.com/sites":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    print("Tu nota final es:"); print(puntos); print("Fin de la prueba.")

##### Programa principal (main program)

xName = input("Dime tu nombre, por favor: ")
print("Hola, " + xName + " !")
a="Vamos a hacer un test de dos preguntas sobre"
b=" Microsoft Windows, CMS, Wordpress y"
c=" Tiendas Virtuales en España."
f=a+b+c
print(frase)
examen()
```

Reto 4:

Modifica el siguiente ejemplo de código para que se ejecute correctamente. El usuario debe hacer un test de 4 preguntas sobre Microsoft Windows, Internet, CMS, Wordpress y Tiendas Virtuales en España.

```
# Vamos a hacer un programa que realiza un SEGUNDO test de dos preguntas sobre
# Microsoft Windows, Internet, CMS, Wordpress y Tiendas Virtuales en España.
# Un CMS (Sistema de Gestión de Contenidos) es una herramienta que permite
# crear y administrar contenido digital sin necesidad de saber programar.
# WordPress.com es una plataforma de blogging y creación de
# sitios web basada en el CMS WordPress, pero alojada y gestionada por
# WordPress, lo que facilita su uso y mantenimiento.

# Definición de funciones

def examen2():
    puntos=0

    #PREGUNTA 1
    A=" virtual, no es necesario comunicar la apertura a la Agencia Tributaria. (V/F)? "
    xx=input("Si no se tienen beneficios en el primer mes de actividad de una tienda "+A)
    if xx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 2
    A="autónomo en España está ligada a la habitualidad y la finalidad lucrativa de la..."
    B="actividad o grado de discapacidad?"
    xxx = input("Completa la frase: La obligación de darse de alta como ""+A+B)
    if xxx=="actividad":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 3
    A="ha sido publicado con wordpress.com correctamente, debemos visitar la "
    B="dirección "
    C="siguiente:"
    F=A+B+C
    xxx = input("Completa la frase: Para comprobar que nuestro sitio web ""+F)
    if xxx=="https://wordpress.com/sites":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 4
    A="
    B=" "
    C=" "
    F=A+B+C
    xxx = input
    if. . .

    print("Tu nota final es:"); print(puntos); print("Fin de la prueba.")

##### Programa principal (main program)
xName = input("Dime tu nombre, por favor: "); print("Hola, " + xName + "!")
a="Vamos a hacer un test de dos preguntas sobre"
b=" Microsoft Windows, CMS, Wordpress y"; c=" Tiendas Virtuales en España."
f=a+b+c; print(frase)
examen2(
```

Ejemplo 12. Creación de una función que suma 4 números (implementando una función con parámetros).

Objetivo: Entender el funcionamiento de una función definida por el usuario. Llamada a una función con parámetros.

Cree un nuevo fichero para el próximo ejercicio y a continuación, haga clic en:

File > Save File as

Grabe su programa (vacío, por ahora) con el nombre de archivo “**función4.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# Definicion de funciones función4.py

def sumar(*args):
    resultado = sum(args)
    print("Sumaremos los siguientes valores:")
    print(args)
    return resultado

#### Programa principal (main program)

a="Vamos a calcular la suma de 4 valores."
print(a)

total=sumar(5, 5, 5, 5)
print("El total es:")
print (total)
```

Observe que hemos creado una función en el bloque “definición de funciones”:

```
def sumar():
    . . .
```

Acto seguido, para que el programa haga correctamente su trabajo, simplemente escribimos el nombre seguido de paréntesis en el “programa principal” de la función que queremos ejecutar.

Para calcular la suma de cuatro valores, llamaremos a la función de la siguiente forma:

```
total= sumar(3,3,3,3)
```

Para imprimir el valor del total de la suma, escribiremos:

```
print(total)
```

Ejemplo 13. Cálculo del volumen de un cubo.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla utilizando una función.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe el archivo con el nombre “**cubo.py**” en la carpeta “Mis Documentos”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
# Definición de funciones

def multiplica(a,b,c):
    print("Multiplicaremos los siguientes valores:")
    print(a,b,c)
    resultado=a*b*c
    return resultado

##### Programa principal (main program)

ax="Vamos a calcular el volumen de un cubo (poliedro regular). "
print(ax)
bx="Todas las aristas miden lo mismo -en metros-"
print(bx)

total=multiplica(3, 3, 3)
print("El total es:");    print (total);    print("metros cubicos.")
```

Preguntas: ¿Qué variables hemos utilizado? ¿Ya domina la función **print**?

Fin del Ejemplo 13.

Ejemplo 14. Cálculo del volumen de una esfera (I) .

Objetivo: Crear una aplicación que realiza cálculos utilizando una función.

Haga clic en:

File > New File

Y a continuación, haga clic en:

File > Save File as

Grabe su programa con el nombre de archivo “**esfera.py**”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
# definicion de funciones

# volume= 4/3 * pi * r * r * r

def multiplica(a,b,c):
    resultado=a*b*c
    return resultado

#### programa principal (main program)

ax="Vamos a calcular el volumen de una esfera."
print(ax)
bx="El radio mide 5 metros."
print(bx)

total=multiplica(5, 5, 5)
print("El total es:")
volume=4/3*3.14*total
print (volume)
print("metros cubicos.")
```

Pregunta: ¿Ya dominas la creación de funciones definidas por el usuario?

Ejemplo 15. Cálculo del volumen de una esfera (II) .

Objetivo: Crear un programa que realiza cálculos con variables numéricas tipo `float` utilizando la biblioteca `math`.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre de archivo “**esfera2.py**”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
# importamos la libreria matematica de Python llamada "math"

# esta libreria matematica contiene el valor de pi. La llamaremos math.pi

import math

# definicion de funciones

def volume(radio):
    volume = (4/3) * math.pi * radio * radio * radio
    return volume

#### programa principal (main program)

# Preguntamos la longitud del radio al usuario con INPUT
radio = float(input("Entre el radio de la esfera: "))

# Calculamos el volumen llamando a la funcion "volume"
volumen = volume(radio)

print(f"El volumen de la esfera es: {volumen:.2f}")
```

En Python, una biblioteca (library) es una colección de código preescrito que podemos usar para realizar tareas comunes, haciendo la programación más eficiente y fácil. Las bibliotecas contienen módulos, que son archivos que incluyen funciones, clases y variables. En este ejemplo hemos importado la biblioteca matemática `math` para poder utilizar el valor de `pi` en nuestros cálculos y variables numéricas tipo `float` (para poder utilizar decimales).

Ejemplo 16a. Cálculo del volumen de un cilindro .

Objetivo: Crear una aplicación que realiza cálculos utilizando la librería **math**.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre de archivo “cilindro.py”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
# Importamos la libreria matematica de Python llamada "math"
# Esta libreria matematica contiene el valor de pi.
# Ademas esta libreria tiene la funcion pow(numero,potencia) que
# calcula la potencia de un numero.
# Ejemplo num. 1: pow(2,2) = 2 * 2 = 4
# Ejemplo num. 2: pow(2,3) = 2 * 2 * 2 = 8
# Ejemplo num. 3: pow(4,2) = 4 * 4 = 16
# Para calcular 2 elevado a 2, podemos escribir en el
# interprete de Python: print(math.pow(2,2))

import math

# definicion de funciones

def calcular_volumen(radius, height):
    resultado_volumen = math.pi * math.pow(radius, 2) * height
    return resultado_volumen

#### programa principal (main program)

radio = float(input("Entre el radio del cilindro: "))
altura = float(input("Entre la altura del cilindro: "))
volumen = calcular_volumen(radio, altura)
print(f"El volumen de un cilindro con radio {radio} y altura {altura} es: {volumen:.2f}")
```

En este ejemplo hemos importado la biblioteca (library) matemática **math** para poder utilizar el valor de **pi** y la función **pow(num,pot)**.

Pregunta: ¿Cómo calcularías 4^3 con la función **pow()**?

Ejemplo 16b. Cálculo del tiempo que se tarda en duplicar un capital con un interés compuesto del 2% anual.

Objetivo: Crear una aplicación que realiza cálculos financieros utilizando la librería **math**.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre de archivo “**capital.py**”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
import math

# Tasa de interés anual
interes = 0.02                ### ojo! 2%

# Tiempo para duplicar el dinero (capital):
tiempo = math.log(2) / math.log(1 + interes)

print(f"Tardarás aproximadamente {tiempo:.2f} años en duplicar el")
print(f"capital al {interes*100}% de interés anual.")
```

Ejemplo 16c. Cálculo del tiempo que se tarda en duplicar un capital con un interés compuesto del 9% anual.

Objetivo: Crear una aplicación que realiza cálculos financieros utilizando la librería **math**.

Haga clic en:

File > New File

Grabe su archivo con el nombre de archivo “**capital2.py**”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
import math

# Tasa de interés anual
inte = 0.09

tiempo = math.log(2) / math.log(1 + inte)

print(f"Tardarás {tiempo:.2f} años en duplicar un capital al {inte*100}% de")
print("interés anual.")
```

Ejemplo 16d. Convertir una tasa de interés mensual a tasa de interés anual.

Objetivo: Crear una aplicación que realiza conversiones de tasas de interés para cálculos financieros utilizando la librería **math**.

Haga clic en:

File > New File

Grabe su archivo con el nombre de archivo “**capital3.py**”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
# Tasa de interés mensual (2%)
interes_mensual = 0.02

# Convertir a tasa anual compuesta
interes_anual = (1 + interes_mensual) ** 12 - 1

print(f"Una tasa mensual del 2% equivale a un {interes_anual*100:.2f}% anual")
print(" con interés compuesto.")

# Ojo! El interés compuesto hace que se pague mucho más a largo plazo si no
# se amortiza rápido. Veamos algunos ejemplos:

# Préstamos personales y tarjetas de crédito:
# Interés compuesto: Se calcula sobre el saldo pendiente, y si no pagas,
# los intereses se acumulan sobre los intereses anteriores.

# Hipotecas:
# Aunque el cálculo puede parecer simple, el sistema de amortización utilizado
# por la mayoría de bancos en muchos países, entre ellos, Francia y España,
# implica pagos mensuales que incluyen capital e intereses,
# y el interés se recalcula sobre el saldo restante. Es una forma
# de interés compuesto disfrazado.

# Préstamos para vehículos o consumo:
# Algunos usan interés simple, especialmente si el préstamo tiene pagos fijos
# y cortos. Pero los bancos aplican interés compuesto si hay cuotas variables
# o refinanciación.
```

Reto 1:

Cree una función para sumar dos valores enteros. El primer valor debe ser declarado como variable global y el segundo valor debe ser declarado como variable local.

Compruebe el siguiente bloque de código. Corrija los errores:

```
global_var= 1010

def so_function():
    local_var= 555
    print(global_var + local_var)

so_function(
```

¿Qué implica utilizar variables globales en lugar de locales en Python?

Las implicaciones más importantes afectan la modularidad de nuestras aplicaciones, la legibilidad, la depuración y la gestión de la memoria.

Las variables locales se definen dentro de una función y sólo existen mientras esa función se está ejecutando (es lo que llamamos “Alcance” o “Scope” en inglés).

Las variables globales se definen fuera de cualquier función y son accesibles desde cualquier parte del código, tanto dentro como fuera de las funciones.

Reto 2:

Cree una función para sumar **tres** valores enteros. El primer valor debe ser declarado como variable global y el segundo y tercer valor deben ser declarados como variables locales.

Compruebe el siguiente bloque de código. Corrija los errores:

```
# Definición de variables

#local_var2=1

global_var= 10


# Definición de funciones

def so_function3():
    local_var1= 55
    print(global_var + local_var1 +)


# Llamada a la función

so_function3(
```


Cómo funcionan las direcciones *IP* en una red

Imaginemos que queremos enviar una carta o un regalo a un amigo y necesitamos una dirección postal única para que llegue a su casa; las computadoras hacen algo muy parecido en internet usando una *IP address*. Esta dirección IP ayuda a que los dispositivos, ordenadores o teléfonos móviles, se encuentren unos a otros en internet.

Esta dirección está formada por cuatro números separados por puntos, a los que se les llama *octets* (octetos). Cada uno de estos números puede tener un valor desde el 0 hasta el 255.

Si tomamos como ejemplo el número 192.168.1.10, cada parte cumple una función especial:

- El primer número, **192**, es una *Network Address* que identifica la red específica en internet.
- El segundo número, **168**, también es una *Network Address* que ayuda a guiar los datos hacia la red correcta.
- El tercer número, **1**, es una *Subnet Address* que identifica una subdivisión de la red.
- El cuarto número, **10**, es una *Host Address* que identifica un dispositivo específico en la red.

Todos estos números, juntos, identifican de forma única a un dispositivo en una red, ya sea un ordenador o un teléfono móvil, en una red. Es de esta manera que los datos usan la dirección para encontrar y comunicarse con el dispositivo correcto.

El protocolo *TCP/IP*

Para que puedan comunicarse dos o más dispositivos en una red, existen reglas. Un protocolo es, en esencia, un conjunto de reglas o normas que indican cómo deben hacerse las cosas para que todos se entiendan.

El protocolo TCP permite a dos dispositivos establecer una conexión e intercambiar paquetes de datos. Este protocolo garantiza la entrega de los paquetes (de datos) eliminando la posibilidad de pérdida de información.

Un aspecto importante de este protocolo es que garantiza que los paquetes sean entregados en el mismo orden en que fueron enviados.

El protocolo IP utiliza las direcciones de cuatro números mencionadas en la página anterior.

Los protocolos HTTP y FTP, de los cuales hablaremos más adelante, utilizan el protocolo *TCP/IP*.

El protocolo *HTTP*

Para que puedan comunicarse dos o más dispositivos en Internet (World Wide Web), estos deben también seguir unas reglas. Este conjunto de normas indica cómo debe hacerse la transmisión de información entre ordenadores o dispositivos en la red de redes y permiten a los servidores y dispositivos conectados comunicarse mediante intercambio de datos.

Cada petición de un cliente es analizada y respondida por un servidor.

Cada URL es el identificador del **recurso** (información transmitida).

El cliente que hace la petición mediante un software especial (navegador web o *browser*) se llama agente del usuario.

Un navegador web (o *browser*) es un programa diseñado para interpretar el código fuente de las páginas y mostrarlo de forma visual e interactiva.

Para entenderlo mejor, veamos un ejemplo:

- Cuando escribimos una dirección web (como www.yahoo.com), el navegador traduce esa orden en una petición oficial (petición HTTP/HTTPS), viaja a través de los cables y antenas WIFI y antenas de los operadores de la red para entregarla al ordenador que guarda esa información (el servidor).
- El servidor de Yahoo recibe la orden, prepara los datos y se los devuelve al usuario.
- El navegador procesa la respuesta: El servidor le devuelve al navegador archivos en formato HTML (la estructura), CSS (el diseño y los colores) y JavaScript (la interactividad y animaciones). El navegador lee ese código en menos de un segundo y lo convierte en la página web que vemos en la pantalla del ordenador, iPad o teléfono móvil.

El protocolo *DHCP*

El protocolo *DHCP* (significa Protocolo de Configuración Dinámica de Host) es un sistema que se encarga de repartir las direcciones *IP* de forma automática a cada computadora, teléfono o tablet que se conecta al internet en una red.

Sin *DHCP*, tendríamos que ir dispositivo por dispositivo escribiendo manualmente los números uno por uno y escribirlos en una lista para no repetirlos y cometer errores al asignarlos a cada dispositivo.

Con *DHCP*, en cuanto un ordenador o dispositivo se conecta, el sistema *DHCP* le asigna su dirección al instante para que pueda empezar a navegar y hablar con los demás dispositivos.

El protocolo *FTP*

El protocolo ***FTP*** (File Transfer Protocol) es un protocolo de red diseñado para el envío de archivos entre dispositivos de una red TCP. Se suele utilizar para subir archivos .html o .jpg a un servidor web. La inmensa mayoría de sitios web contienen archivos de este tipo. Estos archivos son necesarios para que las páginas de un sitio web funcionen correctamente.

Para entender cómo funciona este protocolo, la clave está en que utiliza dos conexiones independientes (canales) al mismo tiempo entre un ordenador (cliente) y el ordenador remoto (servidor):

- Canal de control (para las órdenes -solo viajan comandos y texto-): Con esta conexión indicamos al servidor qué se desea hacer ("iniciar sesión", "mostrar carpetas", "descargar esta foto", "borrar este documento").
- Canal de datos (para la carga): Es la vía por la que viajan los archivos. Cada vez que pedimos descargar o subir un fichero, el protocolo abre este canal exclusivo para transmitir la información y lo cierra en cuanto termina.

¿Cómo funciona?

1. Conexión y autenticación: Nos conectamos al servidor introduciendo una dirección IP o dominio, un usuario y una contraseña (algunos servidores, cada vez menos, admiten como usuario: anonymous -si es público-).
2. Petición: Navegamos por las carpetas del servidor y seleccionamos un archivo. El canal de control envía la orden de transferencia.
3. Transferencia: Se abre el canal de datos, el archivo se envía entero en bloques de información y, al finalizar, recibes la confirmación en el canal de control.

Importante:

El protocolo FTP tradicional **no cifra** la información. Esto significa que los usuarios, las contraseñas y los archivos viajan en texto plano por la red. Para evitar problemas de seguridad, deberemos utilizar una versión segura del protocolo FTP, ya sea SFTP o FTPS, que añaden una capa de cifrado.

Los programas de servidor web más utilizados

Una sola máquina física (computadora) puede ejecutar un software que aloje cientos de sitios web al mismo tiempo. El mercado de software para servidores web se divide principalmente entre:

- **Apache HTTP Server:** El estándar histórico en servidores Linux.
- **Google / Microsoft IIS:** Infraestructuras corporativas y de gran escala cloud.
- **Cloudflare:** Lidera la infraestructura global mediante redes CDN (aceleración mediante caché) y servicios *proxy* masivos. Un servicio proxy masivo (o *reverse proxy* a gran escala) es una red gigante de servidores intermediarios que se colocan entre los usuarios de todo el mundo y los sitios web para filtrar, proteger y acelerar el tráfico de Internet.

Se calcula que entre 2026 y 2027 Internet alojará más de 1500 millones de dominios configurados (Hostnames) para responder peticiones HTTP/HTTPS/FTP.

Protocolos para correo electrónico

1. El protocolo *POP*

Este protocolo se utiliza para obtener los correos electrónicos (mensajes) almacenados en un servidor remoto. Este protocolo únicamente recibe mensajes (no puede enviarlos).

2. El protocolo *IMAP*

Este protocolo permite el acceso a mensajes almacenados en un servidor de correo en Internet. Su principal ventaja es que permite utilizar carpetas en el servidor. Sin embargo, su funcionamiento es diferente ya que permite ver el contenido de los mensajes sin descargar la información completa contenido en el correo electrónico.

3. El protocolo *SMTP*

El protocolo **SMTP** (*Simple Mail Transfer Protocol*) es el encargado de **enviar** correos electrónicos de un punto a otro.

1. El cliente de correo redacta el mensaje y contacta a su servidor SMTP local.
2. El servidor SMTP consulta el DNS para encontrar la dirección IP del servidor del destinatario.
3. Ambos servidores conversan entre sí en texto plano para validar las direcciones y transferir el mensaje.
4. El servidor del destinatario recibe el correo y lo guarda para que la otra persona pueda descargarlo.

Notas:

Un correo electrónico (o email) es una carta tradicional, pero generada con un dispositivo electrónico como un ordenador o un teléfono móvil.

En lugar de escribir en papel el mensaje y enviarlo por correo postal, redactamos un mensaje con un ordenador o teléfono y se envía a través de internet con un software específico a través de la línea telefónica, satelital o inalámbrica (WIFI). Para enviar o recibir un email, se necesita:

- Dirección única: Para comunicarnos con un usuario remoto, usamos su dirección de correo electrónico (como pablo@email.com).
- Contenido: Además de texto, podemos adjuntar fotos, documentos, vídeos o enlaces.
- Un ordenador con conexión a internet y Software especial: Microsoft Outlook o Gmail para ordenadores con Microsoft Windows o Mozilla Thunderbird para usuarios de Linux son los más utilizados.

Importante: Sin un servidor de correo que utiliza uno o varios de estos protocolos, sería imposible enviar o recibir correos electrónicos a través de Internet.

Acceso a Internet mediante VPN

Una **VPN** (*Virtual Private Network* o **Red Privada Virtual**) es una herramienta que crea una conexión cifrada y segura entre un dispositivo (computadora, teléfono o tablet) e Internet.

En lugar de conectar el dispositivo directamente a sitios web o servicios que utilizamos, la VPN redirige el tráfico a través de un servidor privado remoto antes de enviarlo a su destino.

[Dispositivo] <== (Túnel Cifrado) ==> [Servidor VPN] <====> [Internet]

Funciones principales

- **Cifrado de datos:** Protege la información sensible (contraseñas, datos bancarios) cifrando todo el tráfico de red, lo que resulta esencial al usar redes Wi-Fi públicas.
- **Ocultación de IP y ubicación:** Reemplaza la dirección IP real por la del servidor VPN. Esto ayuda a proteger la privacidad en línea y evita que rastreen la ubicación geográfica exacta.
- **Acceso remoto seguro:** Permite a empleados conectarse a la red interna de una empresa de forma segura desde cualquier lugar.
- **Elusión de restricciones geográficas:** Permite acceder a sitios o servicios de streaming cuya disponibilidad varía según la región.

¿Cómo funciona?

- **Autenticación:** El cliente VPN se conecta y verifica su identidad con el servidor VPN.
- **Cifrado (El "Túnel"):** Se establece un protocolo de tunelización cifrado (como *WireGuard*, *OpenVPN* o *IPsec*).

Enrutamiento: Las solicitudes de red viajan por el túnel cifrado hasta el servidor VPN, el cual procesa la petición en Internet y devuelve los datos de forma segura.

Acceso a *WAN*

Conectar un ordenador a una *WAN* (*Wide Area Network*) conlleva comunicar una red local (de una oficina o de una casa particular) con Internet en la mayoría de los casos. Los pasos lógicos son conectar el *router* de la red local (*port forwarding* o apertura de puertos) y manejar la dirección IP pública.

Por razones de seguridad, se aconseja utilizar un cortafuegos (*firewall*), aunque sea de manera básica. Esto es debido a que el *router* es básicamente un muro (teóricamente seguro) que bloquea peticiones de datos o ataques desde el exterior.

El router se encarga de abrir la conexión si un equipo desde el exterior quiere ver archivos o una **página web** (puerto 80) alojada en nuestro equipo.

Ejemplo

1. Fijaremos la IP local del ordenador (servidor) o Raspberry Pi: Para que el router no se confunda si el servidor se reinicia.

Asegúrate de que tu Raspberry Pi siempre tenga la misma dirección IP privada (por ejemplo, 192.168.1.50). Puedes hacerlo desde la propia configuración de la Raspberry Pi (editando el archivo `/etc/dhcpd.conf`) o reservando la IP en los ajustes de tu router. Si la IP cambia mañana, la regla que vamos a crear dejará de funcionar.

2. Accede a la configuración de tu router:

Abre un navegador en tu ordenador y escribe la **IP de tu router** (suele ser 192.168.1.1 o 192.168.0.1). Te pedirá un usuario y contraseña. Si nunca los has cambiado, suelen estar en una pegatina debajo del propio router (por ejemplo, admin/admin o admin/1234).

3.Busca la sección de Port Forwarding:

Cada marca de router tiene un menú distinto. Busca apartados que se llamen:

- **Port Forwarding**
- **Reenvío de puertos**
- **NAT / Virtual Server**
- **Juegos y Aplicaciones**

4.Crea la regla para el servidor web:Abriendo el puerto 80 (HTTP).

Añade una nueva regla con estos datos básicos:

- **Nombre:** Servidor Web Raspberry (o Servidor_Web).
 - **Protocolo:** TCP.
 - **Puerto Externo (WAN):** 80.
 - **Puerto Interno (LAN):** 80.
 - **Dirección IP Interna:** La IP de tu Raspberry Pi (ej. 192.168.1.50).
- Guarda los cambios.

5.Prueba el acceso desde el exterior

No uses el Wi-Fi de tu red para esta prueba. Para comprobar que funciona, no puedes estar conectado a la misma red de la Raspberry.

- Desconecta tu móvil del Wi-Fi (usa solo datos móviles, 4G/5G).
- Busca cuál es tu **IP Pública** (puedes buscar "cuál es mi IP" en Google desde tu PC para verla, será un número como 85.20.134.X).
- En el navegador de tu móvil, escribe tu IP Pública: <http://85.20.134.X>

Importante: Si al hacer la prueba desde tu móvil usando datos ves tu página web, la Raspberry Pi funciona como un servidor web real accesible desde Internet.

La app *Tailscale* (gratuita, aunque con limitaciones) conecta dispositivos de forma directa a través de Internet creando una **VPN en malla (Mesh VPN)**, eliminando la necesidad de configurar reglas de reenvío de puertos (*Port Forwarding*) en el router.

Funciona combinando el protocolo de encriptación *WireGuard* con técnicas automáticas para sortear los cortafuegos.

¿Cómo logra *Tailscale* conectar en solo 4 pasos dos equipos (por ejemplo, el móvil con datos y el servidor en la oficina) de forma transparente?

1. El Servidor de Coordinación (Control Plane)

Tailscale posee servidores que actúan exclusivamente como un directorio telefónico seguro. Su única función es autenticar tus dispositivos e intercambiar sus **claves públicas criptográficas** y sus **direcciones IP actuales**. En teoría, el servidor central de *Tailscale* facilita el contacto inicial, pero **nunca ve, procesa ni almacena tu tráfico de datos**.

2. Perforación de Cortafuegos (NAT Traversal)

El mayor obstáculo en Internet es que los routers domésticos bloquean las conexiones entrantes. Para evitar abrir puertos manualmente, *Tailscale* utiliza técnicas agresivas de *NAT Traversal*. Ambos dispositivos intentan enviarse paquetes de datos simultáneamente hacia afuera, "perforando" sus respectivos *firewalls* desde el interior. Esto engaña a los routers para que mantengan los canales abiertos, creyendo que son respuestas a tráfico legítimo saliente.

3. Encriptación Punto a Punto (*WireGuard*)

Una vez que se han enviado paquetes con éxito, los dispositivos establecen un túnel directo entre ellos utilizando *WireGuard*. De esta manera, los datos viajan directamente de un dispositivo a otro (Punto a Punto) sin pasar por ningún intermediario, utilizando un cifrado de extremo a extremo extremadamente rápido y ligero.

WireGuard es un protocolo de red privada virtual (VPN) moderno, de código abierto y de alto rendimiento que permite establecer conexiones seguras y cifradas entre dispositivos.

4. La Red de Seguridad (Servidores DERP)

Si las redes son demasiado restrictivas (por ejemplo, el Wi-Fi público de un hotel o corporativo que bloquea el tráfico no estándar) y la conexión directa falla, *Tailscale* utiliza un plan alternativo.

¿Cómo funciona este plan?

Los relés **DERP** (*Designated Encrypted Relay for Packets*). Los servidores de *Tailscale* rebotan el tráfico encriptado entre los dos dispositivos a través del puerto 443 (un puerto web seguro estándar que nunca se bloquea).

La conexión será más lenta por la triangulación, pero el túnel se mantendrá activo. Es importante señalar que los servidores DERP no manejan ni obtienen de ningún modo las claves para descifrar la información enviada.

Reto 3:

Compruebe el siguiente bloque de código. Corrija los errores. El usuario debe hacer un test de 3 preguntas sobre Linux e Internet (la red de redes).

```
# Vamos a hacer un programa que realiza un test de tres preguntas sobre Linux e Internet.

# Definición de funciones

def examen():
    puntos=0

    #PREGUNTA 1
    A="Una dirección IP es el número de identificación único que tiene cada "
    B="dispositivo conectado a una. . .:"
    xxx = input("Completa la frase correctamente: ""+A+B)
    if xxx=="red":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 2
    A="Los dos tipos de dirección IP que se utilizan en una red son "
    B="pública y. . .:"
    xxx = input("Completa la frase correctamente: ""+A+B)
    if xxx=="privada":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 3
    A="un proveedor de Internet asigna a un router. Es la cara visible, "
    B="de toda la red hacia el "
    C="mundo. . ."
    F=A+B+C
    xxx = input("Completa la frase: Una IP pública es la dirección que ""+F)
    if xxx=="exterior":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    print("Tu nota final es:"); print(puntos); print("Fin de la prueba.")

##### Programa principal (main program)

xName = input("Dime tu nombre, por favor: ")
print("Hola, " + xName + " !")
a="Vamos a hacer un test de tres preguntas"
b=" sobre"
c=" Linux."
frase= a+b+c
print(frase)
examen()

# Nota: La IP privada es la dirección única que el router le asigna a cada
# dispositivo dentro de una red (PC, servidor, impresora, tablet, móvil, etc.)
# Suele tener el formato 192.168.x.x
# Ejemplo:
# [ Internet ] ----- ( IP Pública ) ----- [ Router ] ----- (IPs Privadas: )
#
# PC: 192.168.1.15
# Tablet: 192.168.1.55
```

Reto 4:

Modifica el siguiente ejemplo de código para que se ejecute correctamente. El usuario debe hacer un test de 3 preguntas sobre Linux.

```
# Vamos a hacer un programa que realiza un test de tres preguntas sobre
# Linux.

# Definición de funciones

def examen():
    puntos=0

    #PREGUNTA 1
    A="tu nombre de "
    B="usuario. . .:"
    xxx = input("Escribe correctamente la instrucción que nos dirá en Linux ""+A+B)
    if xxx=="whoami":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 2
    A="el nombre de la computadora y que versión del sistema operativo "
    B="estamos utilizando. . .:"
    xxx = input("Escribe correctamente la instrucción que nos dirá en Linux ""+A+B)
    if xxx=="uname -a":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 3
    A="ha sido publicado con wordpress.com correctamente en nuestro servidor Linux, "
    B="debemos visitar la dirección "
    C="siguiente:"
    F=A+B+C
    xxx = input("Completa la frase: Para comprobar que nuestro sitio web ""+F)
    if xxx=="https://wordpress.com/sites":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    print("Tu nota final es:"); print(puntos); print("Fin de la prueba.")

#### Programa principal (main program)

xName = input("Dime tu nombre, por favor: ")
print("Hola, " + xName + "!")
a="Vamos a hacer un test de tres preguntas"
b=" sobre"
c=" Linux."
frase= a+b+c
print(frase)
examen()
```

Reto 5:

Modifica el siguiente ejemplo de código para que se ejecute correctamente. El usuario debe hacer un test de 3 preguntas sobre la seguridad en una red local de computadoras con sistema operativo Linux.

```
# Vamos a hacer un programa que realiza un test de tres preguntas sobre seguridad en
# una red local con computadoras que utilizan el S.O. Linux.

# Definición de funciones

def examen():
    puntos=0

    #PREGUNTA 1
    A="qué puertos están "
    B="esperando una llamada. . .:"
    xxx = input("Escribe correctamente la instrucción que nos dirá en Linux "+A+B)
    if xxx=="ss -tulnp":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 2
    A="los procesos que se están ejecutando en forma "
    B="de árbol. . .:"
    xxx = input("Escribe correctamente la instrucción que nos mostrará "+A+B)
    if xxx=="ps aux -forest":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 3
    A="qué puertos están "
    B="en estado LISTEN. . .:"
    xxx = input("Escribe correctamente la instrucción que mostrará en Linux "+A+B)
    if xxx=="ss -tulnp":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    print("Tu nota final es:"); print(puntos); print("Fin de la prueba.")

##### Programa principal (main program)

xName = input("Dime tu nombre, por favor: ")
print("Hola, " + xName + "!")
a="Vamos a hacer un test de tres preguntas"
b=" sobre"
c=" seguridad en Linux."
frase= a+b+c
print(frase)
examen()
```


Reto 6:

Modifica el siguiente ejemplo de código para que se ejecute correctamente. El usuario debe hacer un test de 3 preguntas sobre el control de nuestro servidor web con sistema operativo Linux.

```
# Vamos a hacer un programa que realiza un test de tres preguntas sobre el control de
# procesos en una red local con computadoras que utilizan el S.O. Linux.

# Definición de funciones

def examenLinux():
    puntos=0

    #PREGUNTA 1
    A="qué puertos están "
    B="esperando una llamada. . .:"
    xxx = input("Escribe correctamente la instrucción que nos dirá en Linux ""+A+B)
    if xxx=="ss -tulnp":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 2
    A="las tareas programadas para ejecutarse en "
    B="el sistema cada hora, día o semana. . .:"
    xxx = input("Escribe correctamente la instrucción que nos mostrará ""+A+B)
    if xxx=="cat /etc/crontab":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 3
    A="si nuestro sistema está configurado como"
    B="servidor web. . .:"
    xxx = input("Escribe correctamente la instrucción que mostrará ""+A+B)
    if xxx=="curl -I localhost":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    Lea la nota importante sobre SERVIDORES en la siguiente página

    print("Tu nota final es:"); print(puntos); print("Fin de la prueba.")

##### Programa principal (main program)

xName = input("Dime tu nombre de usuario, por favor: ")
print("Hola, " + xName + "!")
a="Vamos a hacer un test de tres preguntas"
b=" sobre"
c=" control de procesos en Linux."
frase= a+b+c
print(frase)
examen(
```

Nota importante

Para comprobar si nuestro ordenador, Servidor o Raspberry Pi responde como servidor web desde otro ordenador conectado a la misma red local (Wi-Fi o cable), deberemos seguir estos sencillos pasos:

Paso 1: Obtener la dirección IP local del Servidor o la Raspberry Pi

Una dirección IP (Internet Protocol) es el número de identificación único que tiene cada dispositivo conectado a una red.

Primero necesitamos saber qué dirección IP tiene asignada el servidor dentro de la red. Ejecutaremos este comando en la terminal:

```
hostname -I
```

- Resultado posible: 192.168.1.50 (Estos números representan la dirección IP local).

Paso 2: Probar la conexión desde otro ordenador de la misma red

Una vez que tengamos la IP (por ejemplo, 192.168.1.50), tenemos dos opciones sencillas desde otro ordenador:

Opción A: Usar el navegador web (la más fácil)

- Abre cualquier navegador (Chrome, Firefox, Edge, Safari) en el otro equipo.
- En la barra de direcciones escribiremos:

`http://192.168.1.50`

(Sustituiremos 192.168.1.50 por la IP real de tu Raspberry Pi o nuestro servidor).

- Resultado posible: Si todo está bien, verás la página web principal de tu servidor (por ejemplo, la página por defecto de Apache, Nginx o tu propia aplicación web).

Opción B: Usar la terminal o símbolo del sistema (CMD o PowerShell)

Abriremos la terminal o consola de comandos en otro ordenador y ejecutaremos:

○ `curl -I http://192.168.1.50`

- Si devuelve HTTP/1.1 200 OK, el servidor web está respondiendo correctamente a la red.

¿Qué hacer si no logramos conectar?

- Debemos probar un **ping** básico:

Un **ping** es la forma más rápida de saber si dos ordenadores pueden comunicarse entre sí en una red (Internet o red local). Es una instrucción que devolverá el tiempo (normalmente en milisegundos) que tarda el otro ordenador en responder.

En la terminal del otro equipo ejecutaremos **ping 192.168.1.50**. Si no hay respuesta, ambos dispositivos no están en la misma red o hay un problema de conectividad local.

- Revisa las direcciones IP:

Debemos asegurarnos de que hemos tecleado las direcciones correctamente.

- Revisa el cortafuegos (Firewall):

Debemos asegurarnos de que la Raspberry Pi (o el servidor) no tenga un firewall bloqueando el puerto 80 (puerto utilizado normalmente para servicios web).

Notas sobre otros protocolos de comunicación

Ejemplo 17. Hola Mundo (utilizando la librería **flet**).

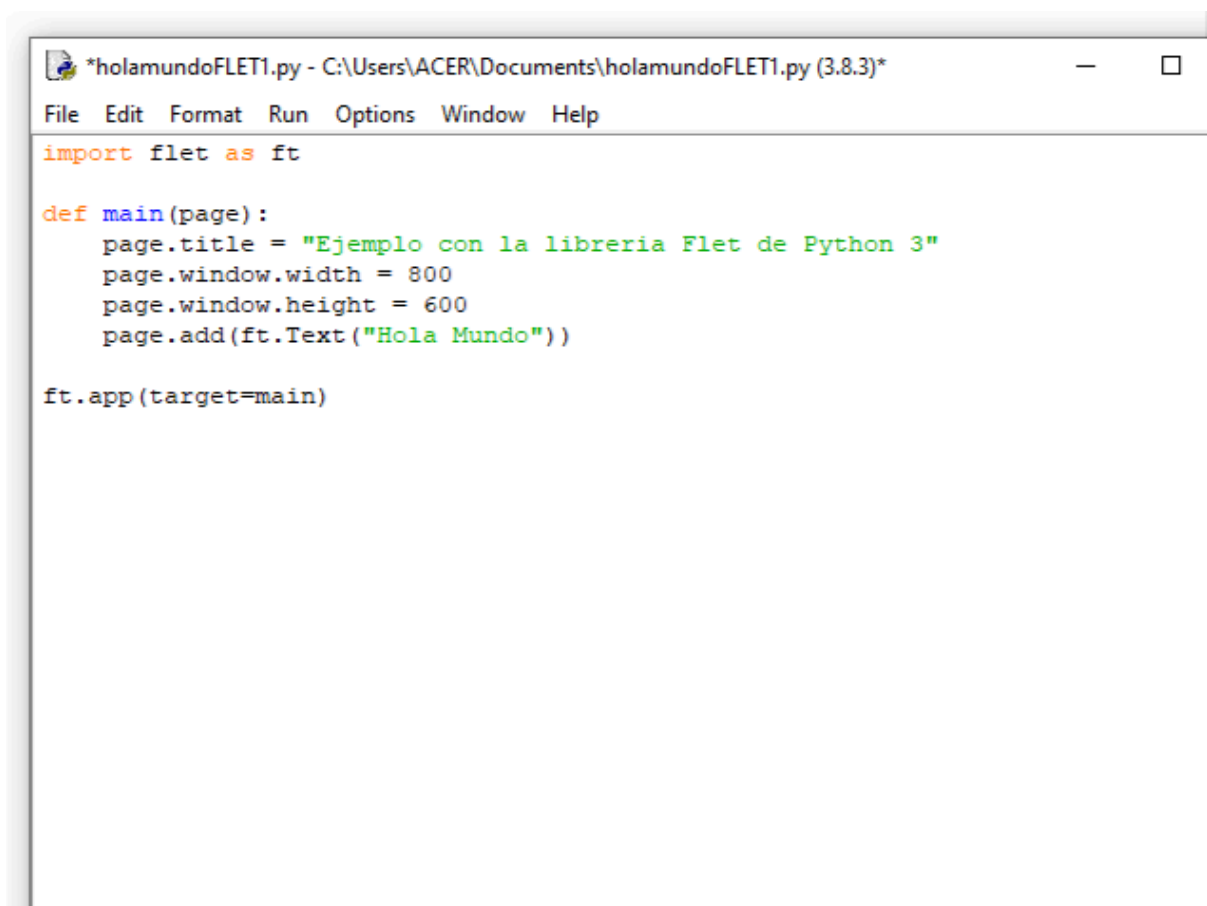
Objetivo: Crear una aplicación que imprime en pantalla “hola mundo” en una ventana nueva utilizando la librería **flet**.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre “holamundoFLET1.py”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

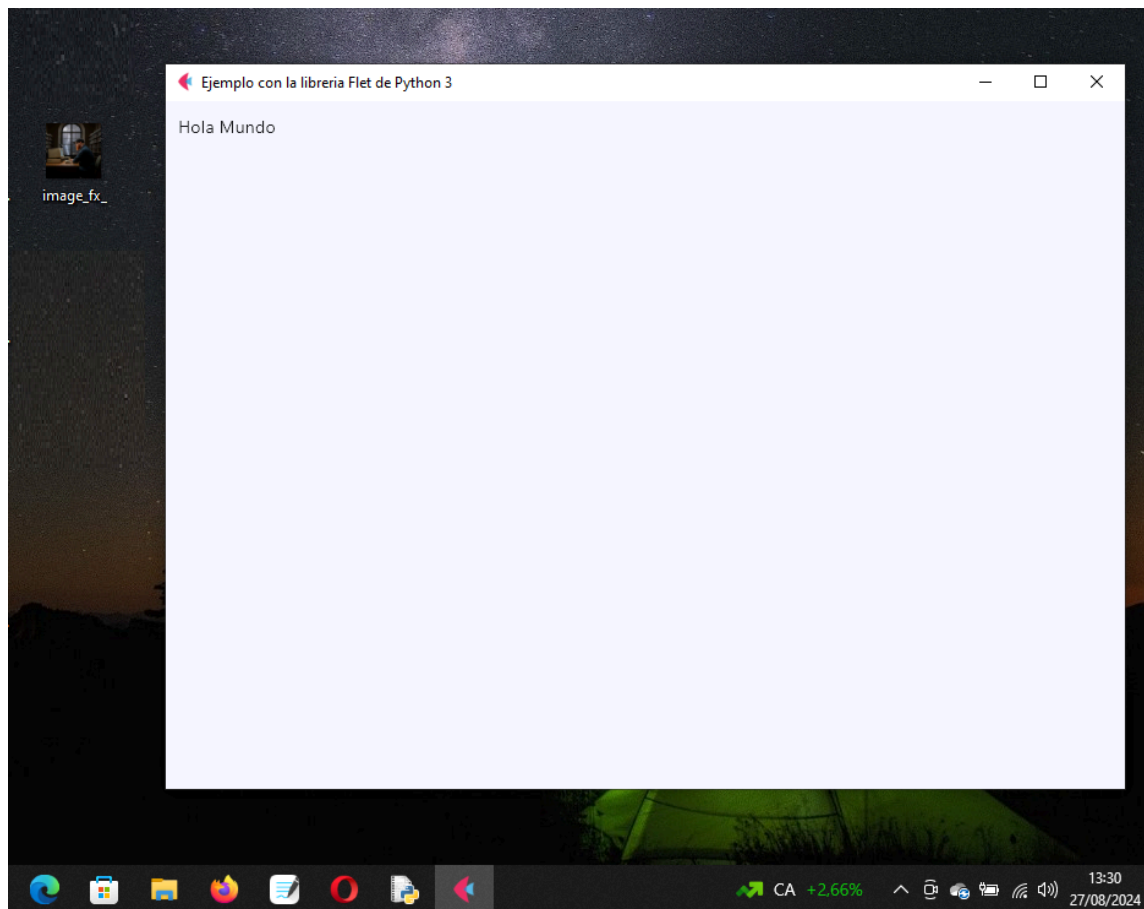
A screenshot of a Python IDE window titled '*holamundoFLET1.py - C:\Users\ACER\Documents\holamundoFLET1.py (3.8.3)*'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor contains the following Python code:

```
import flet as ft

def main(page):
    page.title = "Ejemplo con la libreria Flet de Python 3"
    page.window.width = 800
    page.window.height = 600
    page.add(ft.Text("Hola Mundo"))

ft.app(target=main)
```

Este es el resultado del programa escrito anteriormente:



La librería Flet ayuda al programador a construir aplicaciones móviles, web y de escritorio en Python sin experiencia previa en desarrollo “frontend”.

El desarrollo “frontend” se refiere a la creación de la parte visible e interactiva de la aplicación. Los desarrolladores de “frontend” deben crear aplicaciones que se vean bien en diferentes dispositivos o sistemas operativos.

Con esta librería podemos construir una interfaz de usuario (UI) para apps con controles que están basados en Flutter (framework de código abierto creado por Google). Flet tiene widgets (objetos) más pequeños, lo que simplifica el desarrollo. Esto asegura que las aplicaciones sean más elegantes sin tener que aplicar conceptos de diseño más complicados.

Fin del Ejemplo 17.

Ejemplo 18. Hola Mundo (con mensaje centrado y utilizando la librería `flet`).

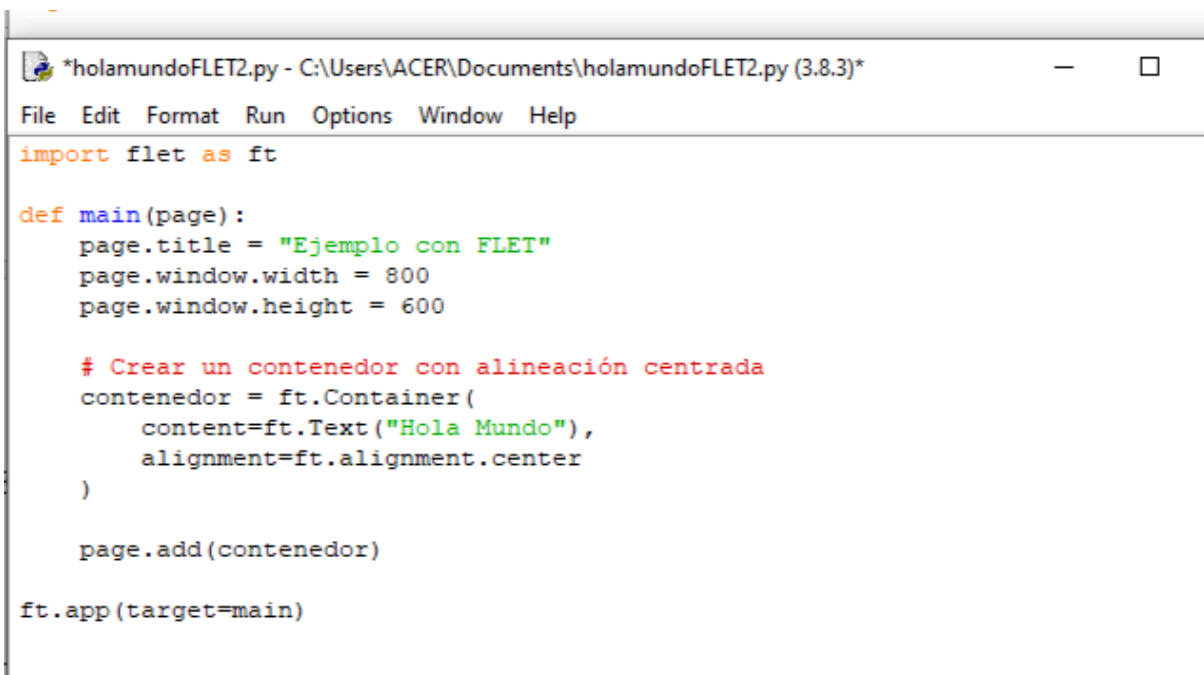
Objetivo: Crear una aplicación que imprime en pantalla “hola mundo” en una ventana nueva utilizando la librería `flet` cambiando atributos para centrar el mensaje.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre “holamundoFLET2.py”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

A screenshot of a Python IDE window titled '*holamundoFLET2.py - C:\Users\ACER\Documents\holamundoFLET2.py (3.8.3)*'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor contains the following Python code:

```
import flet as ft

def main(page):
    page.title = "Ejemplo con FLET"
    page.window.width = 800
    page.window.height = 600

    # Crear un contenedor con alineación centrada
    contenedor = ft.Container(
        content=ft.Text("Hola Mundo"),
        alignment=ft.alignment.center
    )

    page.add(contenedor)

ft.app(target=main)
```

Fin del Ejemplo 18.

Ejemplo 19. Hola Mundo (con cambio de color y utilizando la librería **flet**).

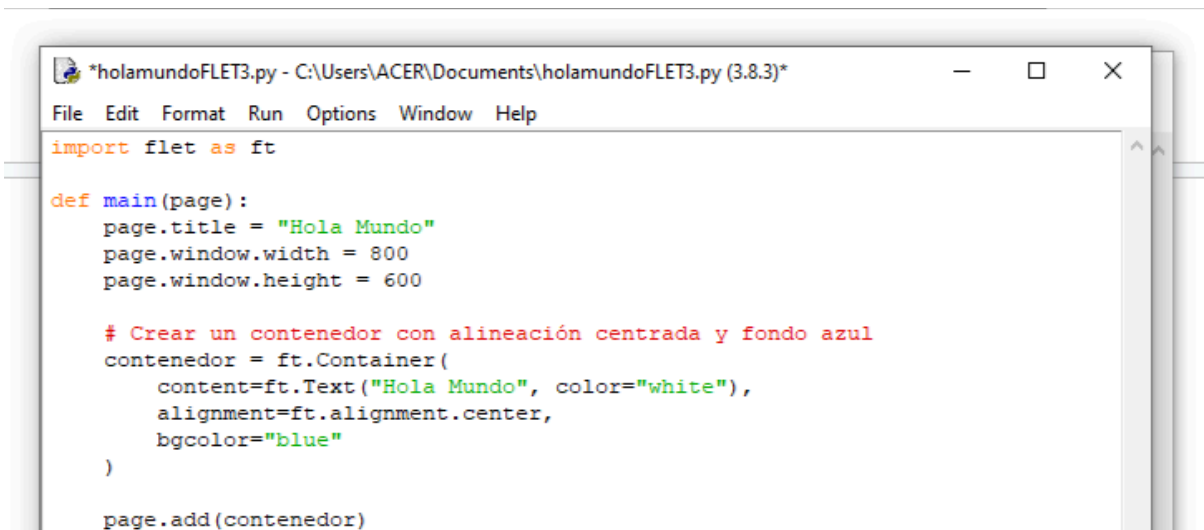
Objetivo: Crear una aplicación que imprime en pantalla “hola mundo” en una ventana nueva utilizando la librería **flet** cambiando atributos para centrar el mensaje.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre “holamundoFLET3.py”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

A screenshot of a Python IDE window titled '*holamundoFLET3.py - C:\Users\ACER\Documents\holamundoFLET3.py (3.8.3)*'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor shows the following Python code:

```
import flet as ft

def main(page):
    page.title = "Hola Mundo"
    page.window.width = 800
    page.window.height = 600

    # Crear un contenedor con alineación centrada y fondo azul
    contenedor = ft.Container(
        content=ft.Text("Hola Mundo", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor)
```

Pregunta: ¿Has notado que no funciona el ejemplo 19? Comprueba que las últimas líneas del código son correctas comparando con el ejemplo 18.

Fin del Ejemplo 19.

Ejemplo 20. Hola Mundo (con cambios de color y utilizando la librería **flet**).

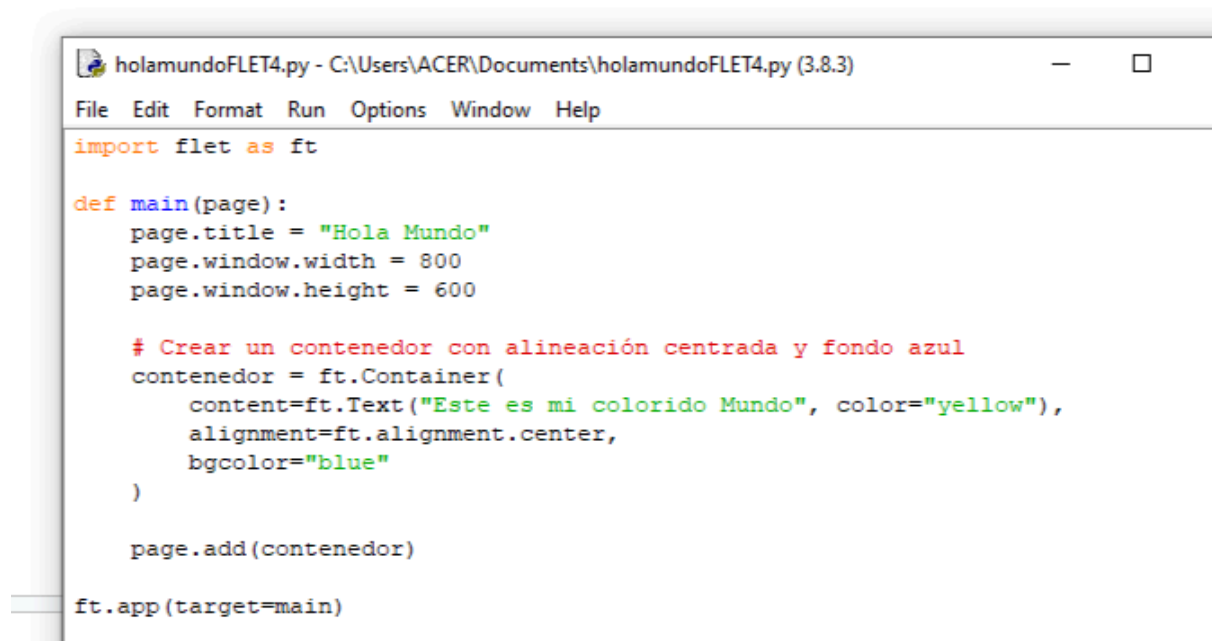
Objetivo: Crear una aplicación que imprime en pantalla “hola mundo” en una ventana nueva utilizando la librería **flet** cambiando atributos para centrar el mensaje.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre “holamundoFLET4.py”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

A screenshot of a Python IDE window titled 'holamundoFLET4.py - C:\Users\ACER\Documents\holamundoFLET4.py (3.8.3)'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor contains the following Python code:

```
import flet as ft

def main(page):
    page.title = "Hola Mundo"
    page.window.width = 800
    page.window.height = 600

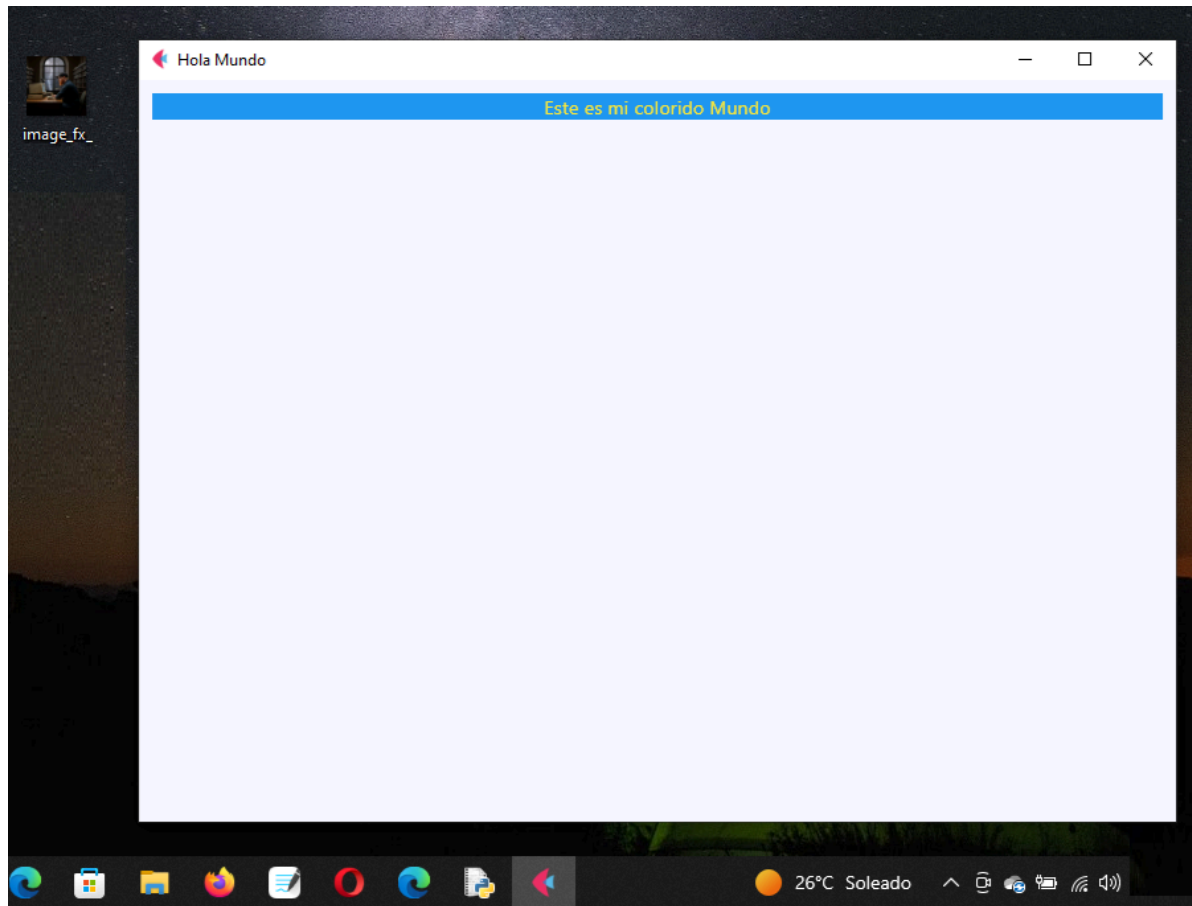
    # Crear un contenedor con alineación centrada y fondo azul
    contenedor = ft.Container(
        content=ft.Text("Este es mi colorido Mundo", color="yellow"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor)

ft.app(target=main)
```

Pregunta: ¿Has comprobado que funciona correctamente? Comprueba que las líneas del código que contienen los colores están en idioma inglés.

Este es el resultado del ejemplo 20:



Fin del Ejemplo 20.

Ejemplo 21. Creación de una tabla utilizando la librería **flet**.

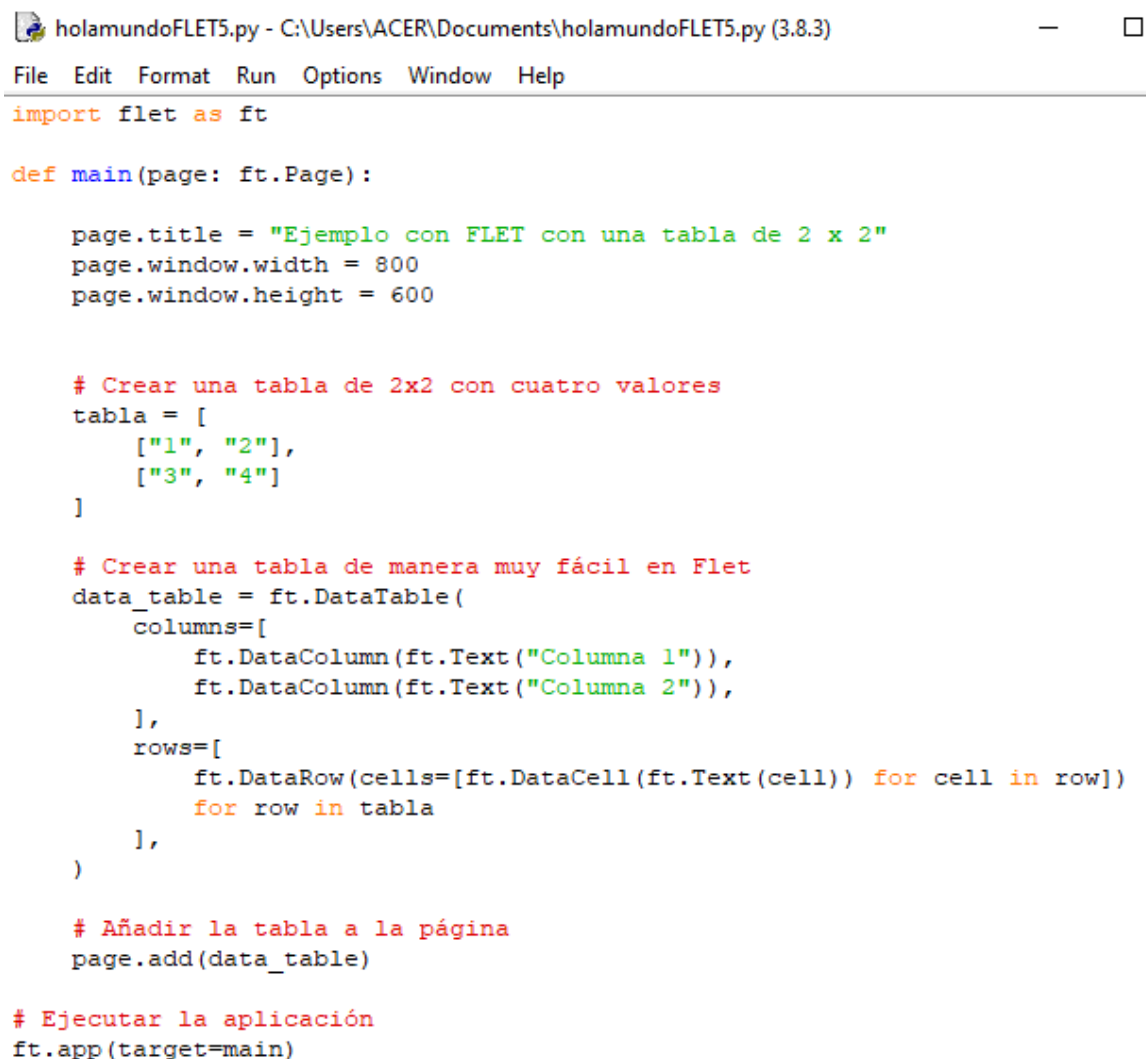
Objetivo: Crear una aplicación que imprime una tabla en una ventana nueva utilizando la librería **flet** cambiando atributos para centrar el mensaje.

Haga clic en:

File > New File

Grabe su archivo (vacío, por ahora) con el nombre “holamundoFLET5.py”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
holamundoFLET5.py - C:\Users\ACER\Documents\holamundoFLET5.py (3.8.3)
File Edit Format Run Options Window Help
import flet as ft

def main(page: ft.Page):

    page.title = "Ejemplo con FLET con una tabla de 2 x 2"
    page.window.width = 800
    page.window.height = 600

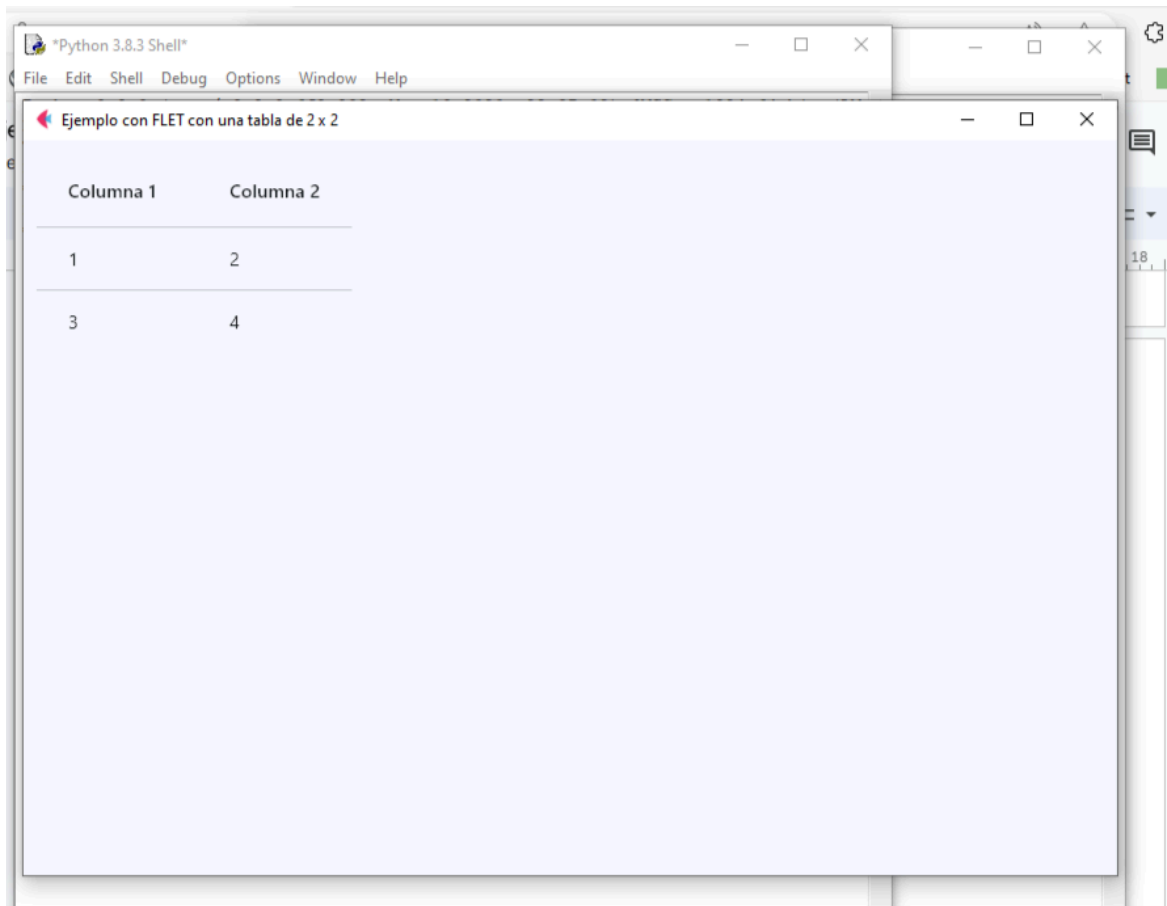
    # Crear una tabla de 2x2 con cuatro valores
    tabla = [
        ["1", "2"],
        ["3", "4"]
    ]

    # Crear una tabla de manera muy fácil en Flet
    data_table = ft.DataTable(
        columns=[
            ft.DataColumn(ft.Text("Columna 1")),
            ft.DataColumn(ft.Text("Columna 2")),
        ],
        rows=[
            ft.DataRow(cells=[ft.DataCell(ft.Text(cell)) for cell in row])
            for row in tabla
        ],
    )

    # Añadir la tabla a la página
    page.add(data_table)

# Ejecutar la aplicación
ft.app(target=main)
```

Este es el resultado del ejemplo 21:



Fin del Ejemplo 21.

Importante

Las **listas** se usan en Python para representar datos. Imaginemos que queremos imprimir 5 colores utilizando una **lista**. La **lista** permite definir un conjunto de valores de diferentes tipos bajo el mismo nombre de variable. El conjunto de valores se delimita con “[“ y ”]”.

Veamos un ejemplo con 6 valores, 5 de tipo **string** y 1 de tipo **entero**:

```
colores= [ "blanco" , "negro" , "rojo" , "azul" , "verde",5 ]
```

Los datos de la lista se pueden modificar durante la ejecución de nuestro programa, tanto los colores (string) como las variables numéricas.

La orden siguiente imprimiría el color ‘blanco’ (recuerde que el primer valor de la lista es siempre 0):

```
print(colores[0])
```

La orden siguiente imprimiría ‘blanco negro’ (recuerde que el primer valor es siempre 0):

```
print(colores[0],colores[1])
```

Pregunta 1:

¿Qué cree que pasaría al ejecutar el siguiente bloque de código? Complete el “script” para que funcione correctamente.

```
colores= [ "blanco" , "negro" , "rojo" , "azul" , "verde",5 ]  
for color in colores:  
    print(color
```

Pregunta 2:

¿Qué cree que pasaría al ejecutar el siguiente bloque? Complete el “script” para que funcione correctamente.

```
for num in range(0,10)  
    print num
```

Pregunta 3:

Corrija los errores en el siguiente programa para mostrar en pantalla únicamente el primer elemento de la lista de motocicletas con su precio correspondiente.

```
lista_de_motos=["Kawasaki","Honda","Suzuki","HD","Triumph"]  
for i in lista_de_motos:  
    print(i)  
  
print(" ")  
  
lista_de_precios=["50000","40000","30000","20000","10000"]  
for j in lista_de_precios:  
    print(j)  
  
print(lista_de_coches[0],int(lista_de_precios[0])
```

Pregunta 4a:

Lea el ejercicio y corrija los errores en el programa para imprimir en pantalla el resultado de la ***pregunta** planteada dentro del recuadro siguiente:

```
#
# Resuelve el siguiente problema utilizando "listas" en Python:
# Juan quiere ir a la playa el próximo Domingo.
# Para ello necesita una toalla y un bañador.
# Los precios son: La toalla cuesta 6 eur y el bañador cuesta 15 eur.
# *Pregunta:
# ¿Cuánto le cuesta a Juan ir a la playa el Domingo?

# Elementos que necesita Juan para ir a la playa:
elementos= [ "toalla" , "bañador" ]

# Método 1 para imprimir los valores de la lista (con un bucle)
for c in elementos:
    print (c)

# Método 2 para imprimir los valores de la lista (con un puntero)
#print(" ")
#print(elementos[0])
#print(elementos[1])

lista_de_precios= [ "6" , "15" ]
print(" ")
print("Veamos los precios:")
print(int(lista_de_precios[0]))
print(int(lista_de_precios[1]))

# Busque y corrija los errores en este bloque:
print("¿Cuánto le cuesta a Juan ir a la playa el Domingo?")
print(int(lista_de_precios[0])+int(lista_de_precios[0]
print("euros")

# Busque y corrija los errores:
# print("¿Cuánto le cuesta a Ana, la amiga de Juan, ir a la playa el
# Domingo si también necesita un bañador y una toalla")
# print("El precio de los artículos es el mismo para Juan y para Ana")
# print(int(lista_de_precios[0])+int(lista_de_precios[
#
```


Pregunta 4b:

Lea el ejercicio y corrija los errores en el programa para imprimir en pantalla el resultado de la ***pregunta** planteada dentro del recuadro siguiente:

```
#
# Resuelve el siguiente problema utilizando "listas" en Python:
# John quiere ir a la playa el próximo Domingo.
# Para ello necesita bronceador, una toalla y un bañador.
# Los precios son: La toalla cuesta 6 eur y el bañador cuesta 19 eur.
# El bronceador cuesta justo la mitad que una toalla.
# *Pregunta:
# ¿Cuánto le cuesta a John ir a la playa el Domingo?

# Elementos que necesita John para ir a la playa:
elementos= [ "bronceador", "toalla" , "bañador" ]

# Método 1 para imprimir los valores de la lista (con un bucle)
for c in elementos:
    print (c)

# Método 2 para imprimir los valores de la lista (con un puntero)
#print(" ")
#print(elementos[0])
#print(elementos[1])
#print(elementos[2])

# Busque y corrija los errores en cada bloque:
lista_de_precios= [ "X", "6" , "19" ]
print(" ")
print("Veamos los precios:")
print(int(lista_de_precios[0]))
print(int(lista_de_precios[1]))
print(int(lista_de_precios[1]

# Busque y corrija los errores:
print("¿Cuánto le cuesta a John ir a la playa el Domingo?")
print(int(lista_de_precios[0])+int(lista_de_precios[0
print("euros")
```

Pregunta 5a:

Imprima los valores de la lista de elementos del ejercicio anterior con un bucle "while".

```
# Inicializamos la lista con 3 elementos:
elementos = ["bronceador", "toalla", "bañador"]

# Primero necesitaremos inicializar un contador:
i = 0

# Creamos el bucle:
while i < len(elementos):
    print(elementos[i])
    i=i+1                # Incrementamos el contador i += 1
```

Pregunta 5b:

Imprima el elemento mayor entre los valores de la lista del siguiente grupo de precios. Utilice un bucle "while". Busque y corrija los errores del siguiente bloque de código:

```
lista= [ 123, 12111, 1201 ]

maximo = lista[0]
i = 1

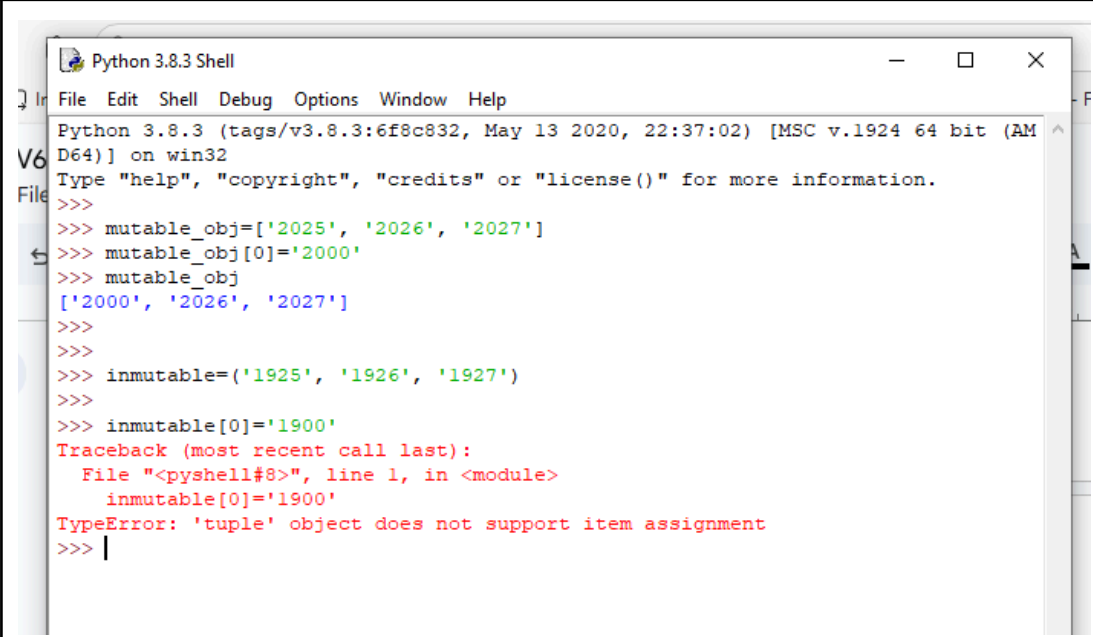
while i < len(lista):
    if lista[i] > maximo:
        maximo = lista[i]
    i += 1

print(maximo
```

Solución: Sustituya la última línea de código: `print(maximo)`

Pregunta 6:

Estudie el ejercicio siguiente y observe las diferencias entre las listas (la segunda lista es inmutable):



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
>>> mutable_obj=['2025', '2026', '2027']
>>> mutable_obj[0]='2000'
>>> mutable_obj
['2000', '2026', '2027']
>>>
>>>
>>> immutable=('1925', '1926', '1927')
>>>
>>> immutable[0]='1900'
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    immutable[0]='1900'
TypeError: 'tuple' object does not support item assignment
>>> |
```

¿Cuándo usar estructuras inmutables en lugar de listas normales (list)?

Cuando los datos no deben modificarse como en el ejemplo siguiente:

```
dias_semana = ("Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")
### dias_semana[0] = "Monday" # Error: las tuplas no permiten asignación
```

¿En qué otro caso podríamos utilizar listas inmutables?

Reto 1:

Juan, Antonio y Yolanda tienen 20, 30 y 40 años de edad respectivamente. Crea una lista mutable llamada `lista_mutable1` con las edades de las tres personas.

A continuación crea una segunda lista mutable donde cada elemento de esta lista sea el doble de cada elemento de la primera lista.

```
# Corrija los errores y complete el código:

lista_mutable1=[20,30,40]
juan   = lista_mutable1[0]*2
antonio= . . .
yolanda= . . .

lista_mutable2= . . .
print("Los elementos de la segunda lista son:")
print(lista_mutable2[0])
print(. . .
print(. . .
```

Solución:

```
lista_mutable1=[20,30,40]
juan   = lista_mutable1[0]*2
antonio= lista_mutable1[1]*2
yolanda= lista_mutable1[2]*2

lista_mutable2= [juan,antonio,yolanda]
print('Los elementos de la segunda lista son:')
print(lista_mutable2[0])
print(lista_mutable2[1])
print(lista_mutable2[2])
```

Reto 2: Cree una tercera lista mutable donde cada elemento de esta lista sea el doble de cada elemento de la segunda lista.

Reto 3:

Estela, Juan, Antonio y Yolanda tienen 10, 20, 30 y 40 años de edad respectivamente. Crea una lista mutable llamada `lista_mutable1` con las edades de las cuatro personas.

A continuación crea una segunda lista mutable donde cada elemento de esta lista sea el doble de cada elemento de la primera lista. Utiliza la instrucción “`while`”.

Solución:

```
## Listas mutables

lista_mutable1 = [10, 20, 30, 40]
lista_mutable2 = []          ### creamos una lista vacía

i = 0

while i < len(lista_mutable1):
    doble_valor = lista_mutable1[i] * 2
    lista_mutable2.append(doble_valor)
    i=i+ 1

print('Primera lista:')
print(lista_mutable1)
print(' ')
print('Segunda lista:')
print(lista_mutable2)

## Recuerda que la función append en Python se utiliza para añadir
## un elemento al final de una lista mutable. Es una forma sencilla
## de extender una lista existente con un elemento nuevo.
```

Ejemplo 22. Creación de una tabla 5x2 con bordes utilizando la librería **flet**.

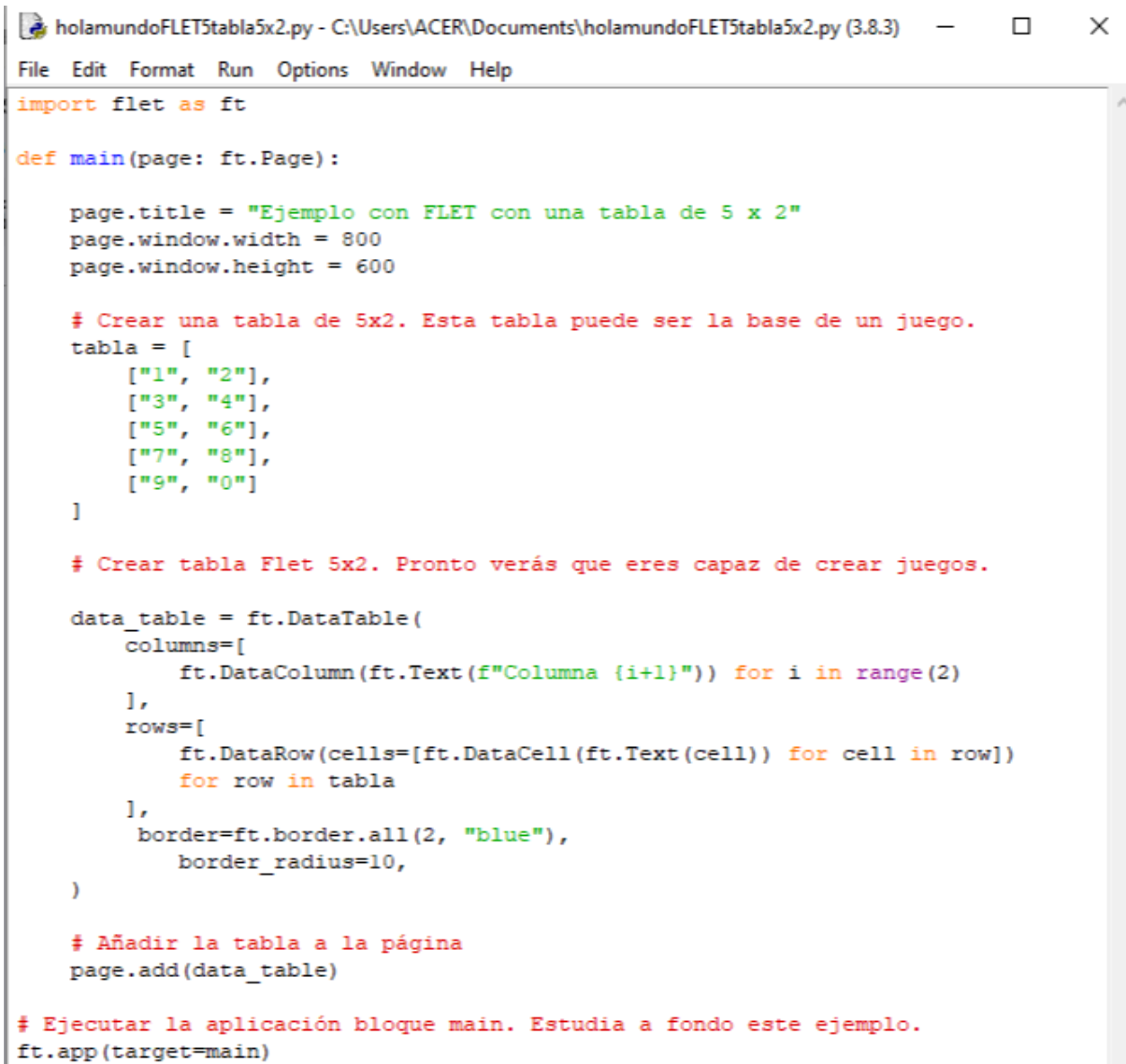
Objetivo: Crear una aplicación que imprima una tabla con bordes azules (blue) en una ventana nueva utilizando la librería **flet**.

Haga clic en:

File > New File

Grabe su archivo vacío con el nombre “holamundoFLET5tabla5x2.py”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
holamundoFLET5tabla5x2.py - C:\Users\ACER\Documents\holamundoFLET5tabla5x2.py (3.8.3)
File Edit Format Run Options Window Help
import flet as ft

def main(page: ft.Page):

    page.title = "Ejemplo con FLET con una tabla de 5 x 2"
    page.window.width = 800
    page.window.height = 600

    # Crear una tabla de 5x2. Esta tabla puede ser la base de un juego.
    tabla = [
        ["1", "2"],
        ["3", "4"],
        ["5", "6"],
        ["7", "8"],
        ["9", "0"]
    ]

    # Crear tabla Flet 5x2. Pronto verás que eres capaz de crear juegos.

    data_table = ft.DataTable(
        columns=[
            ft.DataColumn(ft.Text(f"Columna {i+1}")) for i in range(2)
        ],
        rows=[
            ft.DataRow(cells=[ft.DataCell(ft.Text(cell)) for cell in row])
            for row in tabla
        ],
        border=ft.border.all(2, "blue"),
        border_radius=10,
    )

    # Añadir la tabla a la página
    page.add(data_table)

    # Ejecutar la aplicación bloque main. Estudia a fondo este ejemplo.
    ft.app(target=main)
```

Este es el resultado del ejemplo 22:

Columna 1	Columna 2
1	2
3	4
5	6
7	8
9	0

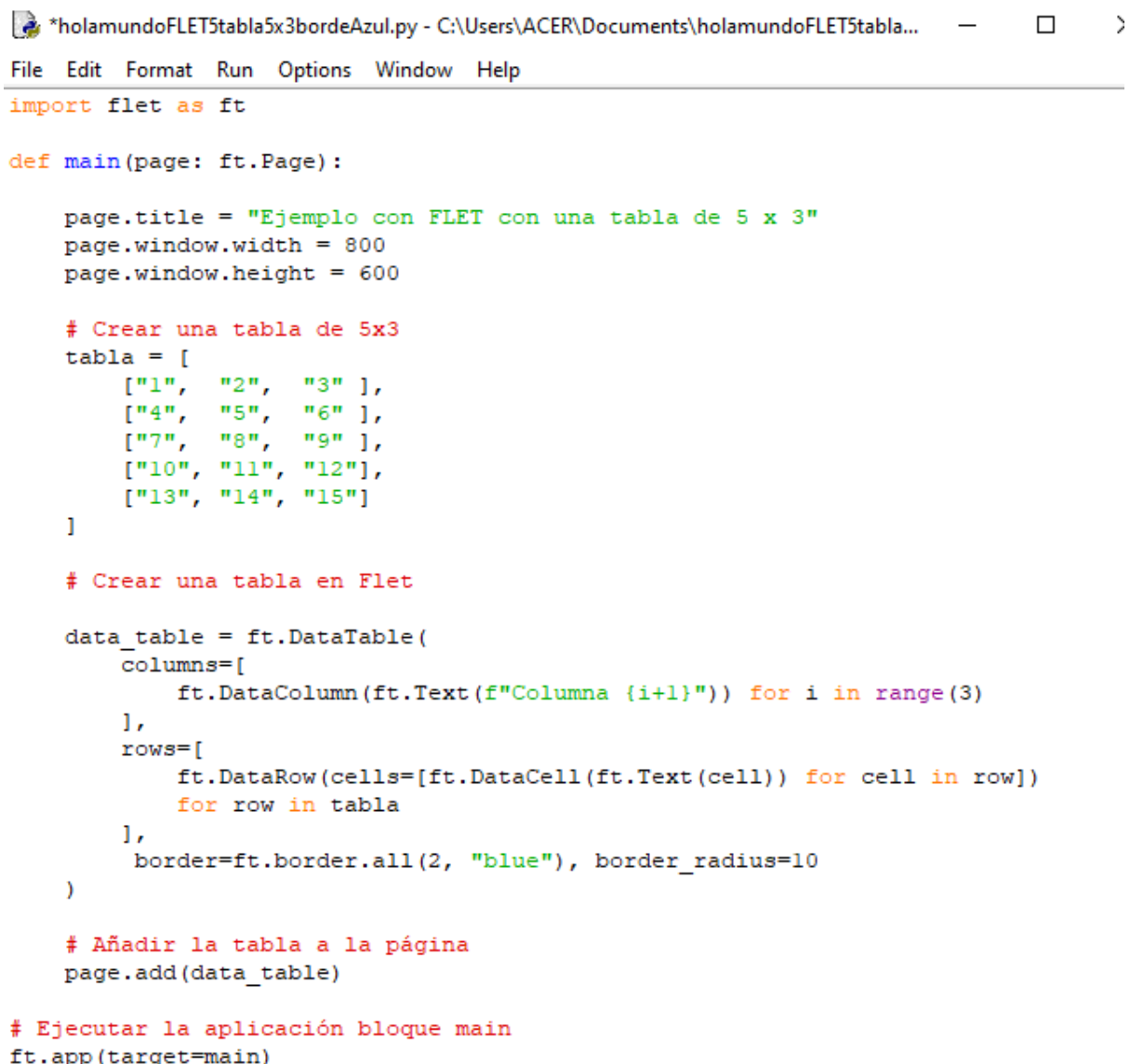
Fin del Ejemplo 22.

Ejemplo 23. Creación de una tabla 5 x 3 con bordes utilizando la librería **flet**.

Objetivo: Crear una aplicación que imprima una tabla con cinco líneas y tres columnas con bordes azules en una ventana nueva utilizando la librería **flet**.

Cree un archivo nuevo con el nombre: “**holamundoFLET5tabla5x3bordeazul.py**”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
*holamundoFLET5tabla5x3bordeAzul.py - C:\Users\ACER\Documents\holamundoFLET5tabla...
File Edit Format Run Options Window Help
import flet as ft

def main(page: ft.Page):

    page.title = "Ejemplo con FLET con una tabla de 5 x 3"
    page.window.width = 800
    page.window.height = 600

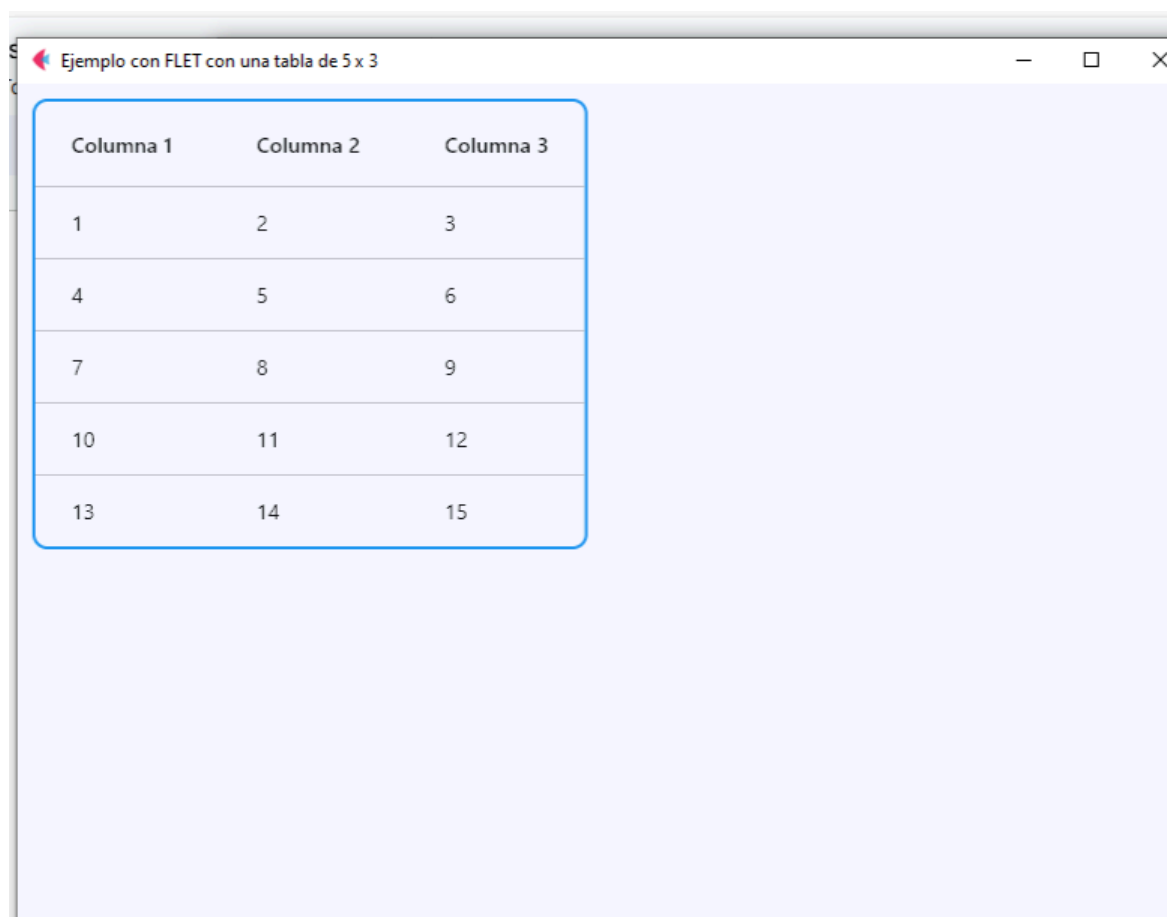
    # Crear una tabla de 5x3
    tabla = [
        ["1", "2", "3"],
        ["4", "5", "6"],
        ["7", "8", "9"],
        ["10", "11", "12"],
        ["13", "14", "15"]
    ]

    # Crear una tabla en Flet
    data_table = ft.DataTable(
        columns=[
            ft.DataColumn(ft.Text(f"Columna {i+1}")) for i in range(3)
        ],
        rows=[
            ft.DataRow(cells=[ft.DataCell(ft.Text(cell)) for cell in row])
            for row in tabla
        ],
        border=ft.border.all(2, "blue"), border_radius=10
    )

    # Añadir la tabla a la página
    page.add(data_table)

# Ejecutar la aplicación bloque main
ft.app(target=main)
```


Este es el resultado del ejemplo 23:



Columna 1	Columna 2	Columna 3
1	2	3
4	5	6
7	8	9
10	11	12
13	14	15

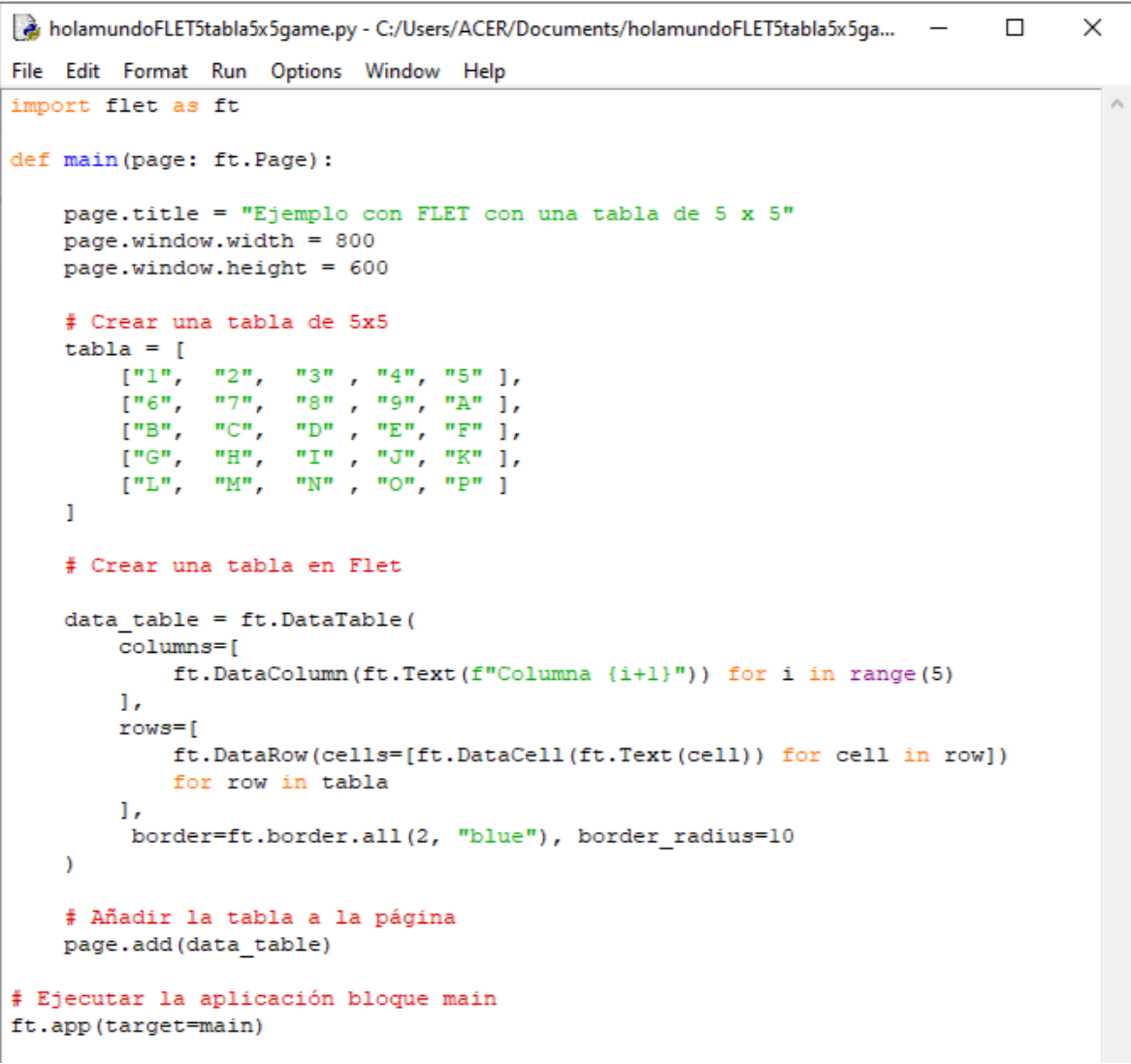
Fin del Ejemplo 23.

Ejemplo 24. Creación de una tabla 5 x 5 con bordes utilizando la librería **flet**.

Objetivo: Crear una aplicación que imprima una tabla con cinco líneas y cinco columnas con bordes azules en una ventana nueva utilizando la librería **flet**.

Cree un archivo nuevo con el nombre: “**holamundoFLET5tabla5x5game.py**”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
holamundoFLET5tabla5x5game.py - C:/Users/ACER/Documents/holamundoFLET5tabla5x5ga...
File Edit Format Run Options Window Help

import flet as ft

def main(page: ft.Page):

    page.title = "Ejemplo con FLET con una tabla de 5 x 5"
    page.window.width = 800
    page.window.height = 600

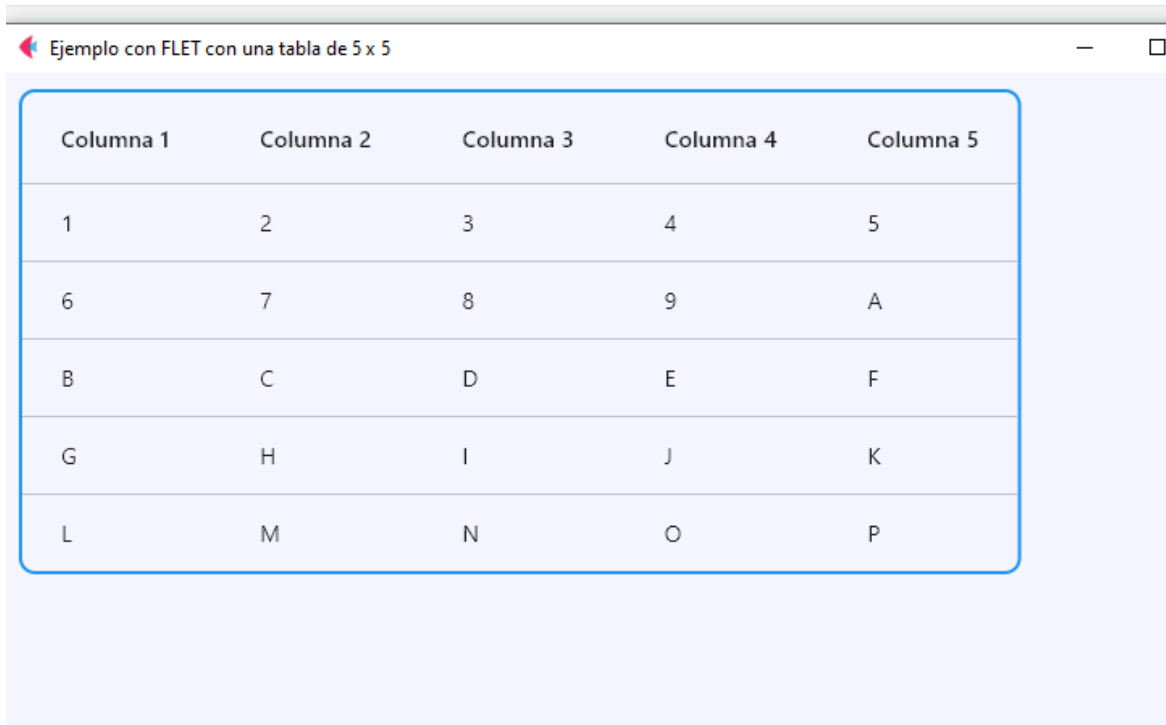
    # Crear una tabla de 5x5
    tabla = [
        ["1", "2", "3", "4", "5"],
        ["6", "7", "8", "9", "A"],
        ["B", "C", "D", "E", "F"],
        ["G", "H", "I", "J", "K"],
        ["L", "M", "N", "O", "P"]
    ]

    # Crear una tabla en Flet
    data_table = ft.DataTable(
        columns=[
            ft.DataColumn(ft.Text(f"Columna {i+1}")) for i in range(5)
        ],
        rows=[
            ft.DataRow(cells=[ft.DataCell(ft.Text(cell)) for cell in row])
            for row in tabla
        ],
        border=ft.border.all(2, "blue"), border_radius=10
    )

    # Añadir la tabla a la página
    page.add(data_table)

# Ejecutar la aplicación bloque main
ft.app(target=main)
```

Este es el resultado del ejemplo 24:



The screenshot shows a web browser window with the title "Ejemplo con FLET con una tabla de 5 x 5". The browser window has a light blue background. Inside the window, there is a table with 5 columns and 5 rows. The columns are labeled "Columna 1", "Columna 2", "Columna 3", "Columna 4", and "Columna 5". The rows contain the following data:

Columna 1	Columna 2	Columna 3	Columna 4	Columna 5
1	2	3	4	5
6	7	8	9	A
B	C	D	E	F
G	H	I	J	K
L	M	N	O	P

Fin del Ejemplo 24.

Ejemplo 25. Creación de una tabla 5 x 5 y cálculo de la suma de valores de la fila uno de la tabla utilizando la librería **flet**.

Objetivo: Utilizando la librería **flet**, crear una aplicación que imprima una tabla con cinco líneas y cinco columnas en una ventana nueva y realice la suma de los valores de la primera fila para después mostrar el resultado en pantalla.

Cree un archivo nuevo con el nombre: “**holamundoFLET5tabla5x5gameadd.py**”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
holamundoFLET5tabla5x5gameadd.py - C:/Users/ACER/Documents/holamundoFLET5tabla5x5gameadd.py (3.8.3)
File Edit Format Run Options Window Help

import flet as ft

def main(page: ft.Page):

    page.title = "Ejemplo con FLET con una tabla de 5 x 5"
    page.window.width = 700
    page.window.height = 600

    # Crear un contenedor con alineación centrada y fondo azul
    contenedor1 = ft.Container(
        content=ft.Text("Cálculos", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor1)

    # Crear una tabla de 5x5
    tabla = [
        ["1", "2", "3", "4", "5"],
        ["6", "7", "8", "9", "A"],
        ["B", "C", "D", "E", "F"],
        ["G", "H", "I", "J", "K"],
        ["L", "M", "N", "O", "P"]
    ]

    # Crear una tabla en Flet
    data_table = ft.DataTable(
        columns=[
            ft.DataColumn(ft.Text(f"Columna {i+1}")) for i in range(5)
        ],
        rows=[
            ft.DataRow(cells=[ft.DataCell(ft.Text(cell)) for cell in row])
            for row in tabla
        ],
        border=ft.border.all(2, "blue"), border_radius=10
    )

    # Añadir la tabla a la página
    page.add(data_table)
```

#el código continúa en la siguiente página#

```

# Imprimir algunos valores de la tabla en la consola para comprobaciones
prim = int(tabla[0][0])
print("El primer valor de la tabla es:", prim)
segu = int(tabla[0][1])
terc = int(tabla[0][2])
cuar = int(tabla[0][3])
quin = int(tabla[0][4])
print("El quinto valor de la tabla es:", quin)
suma=prim+segu+terc+cuar+quin
suma_texto=str(suma)

# Crear otro contenedor para mostrar resultados de sumas de valores
contenedor2 = ft.Container(
    content=ft.Text("La suma de los valores de la primera fila es: "+suma_texto, color="white"),
    alignment=ft.alignment.center,
    bgcolor="blue"
)

page.add(contenedor2)

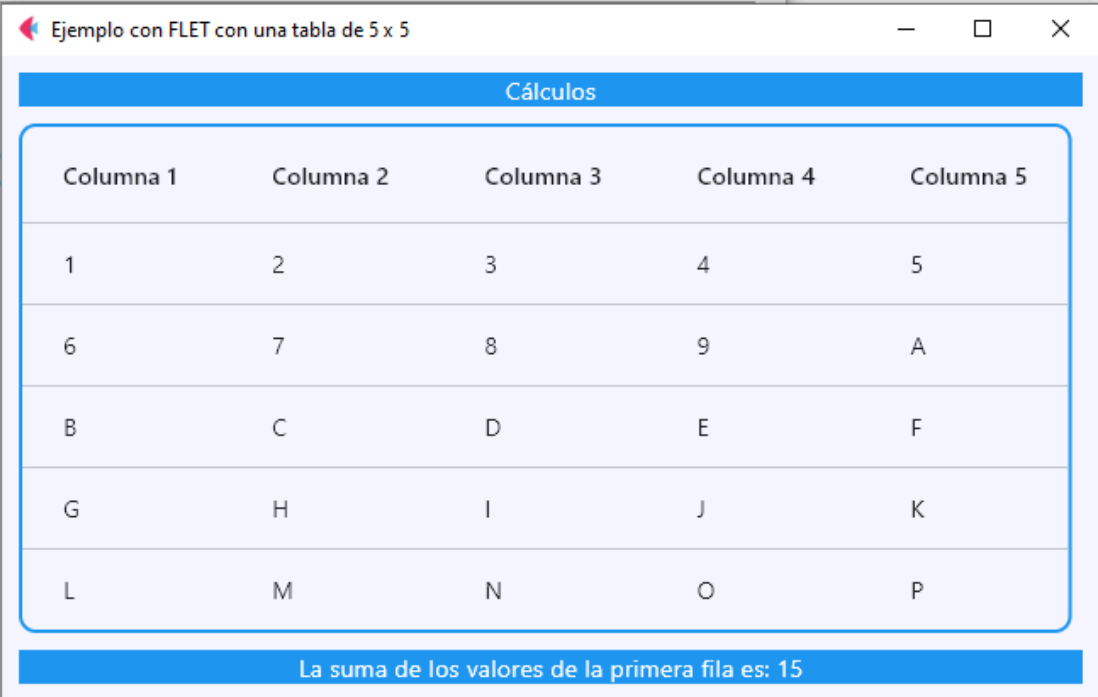
# Ejecutar la aplicación bloque main
ft.app(target=main)

```

Observe que se han creado las variables **prim**, **segu**, **terc**, **cuar** y **quin** para almacenar el valor numérico de los valores (tipo texto) de la tabla. Para convertir cada texto (valores de la tabla) a valores numéricos, hemos utilizado la función **int()**.

Después de esto, necesitamos convertir el resultado numérico de la suma a texto. Lo hemos conseguido con la función **str()**, que convierte el resultado numérico a texto, para así añadirlo en el **contenedor2**.

Este es el resultado del ejemplo 25:



Cálculos				
Columna 1	Columna 2	Columna 3	Columna 4	Columna 5
1	2	3	4	5
6	7	8	9	A
B	C	D	E	F
G	H	I	J	K
L	M	N	O	P

La suma de los valores de la primera fila es: 15

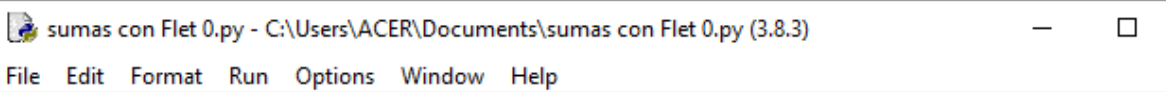
Fin del Ejemplo 25.

Ejemplo 26. Creación de una aplicación que imprima texto en el centro de la pantalla con fondo azul.

Objetivo: Utilizando la librería **flet**, crear una aplicación que imprima una frase en el centro de la pantalla con fondo azul.

Cree un archivo nuevo con el nombre: “sumas con Flet 0.py”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
import flet as ft

#define main

def main(page: ft.Page):
    page.title = "Hola mundo Flet"
    page.window.width=640
    page.window.height = 480
    page.vertical_alignment= 'center'
    page.horizontal_alignment= ft.CrossAxisAlignment.CENTER
    page.bgcolor= ft.colors.BLUE_ACCENT_100

    #flet text control
    text_one= ft.Text(value="Mi primera pantalla azul con Flet")

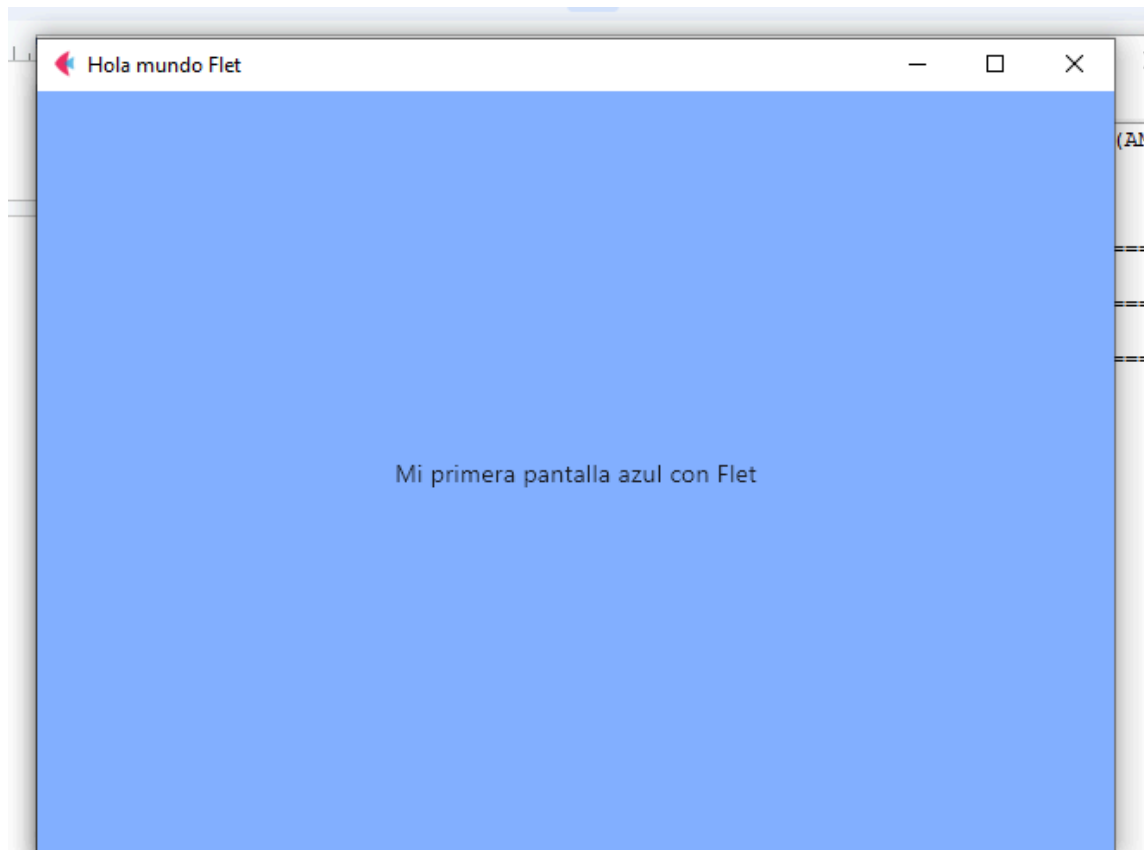
    #appending (añadir control) & actualizar la página
    page.controls.append(text_one)
    page.update()

#start app

ft.app(target=main)
```

En este simple ejemplo mostramos cómo cambiar el fondo de la aplicación creada como opción para tus futuras aplicaciones. Observa que los controles de la página empiezan siempre por la palabra **page**. Por ejemplo, **page.vertical.alignment= 'center'**.

Este es el resultado del ejemplo 26:



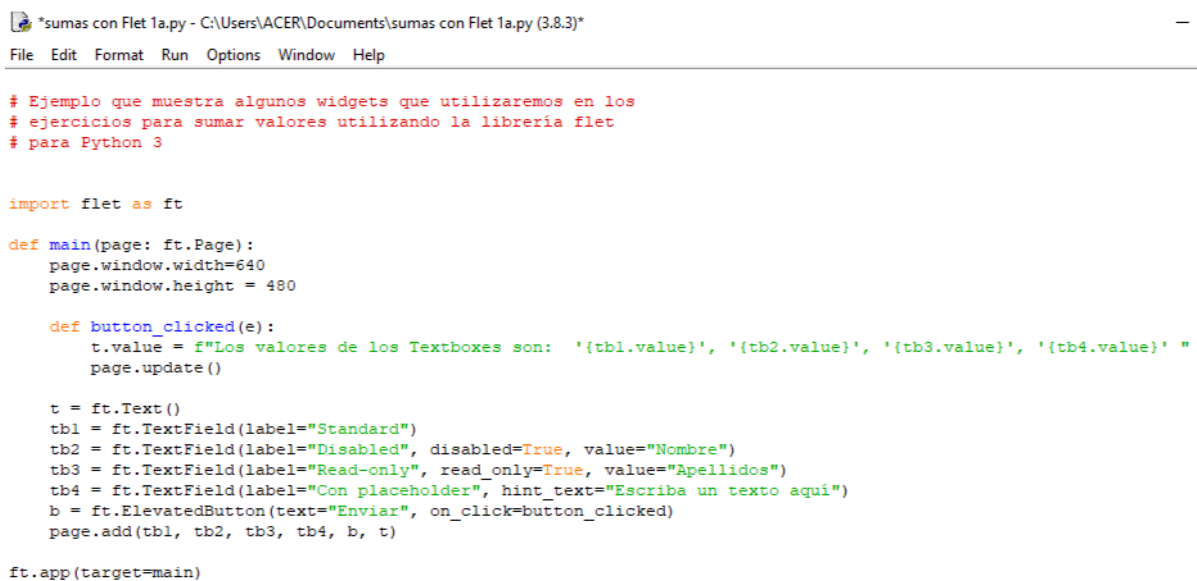
Fin del Ejemplo 26.

Ejemplo 27. Creación de una aplicación que obtenga una entrada por parte del usuario e imprima textos en la pantalla.

Objetivo: Utilizando la librería **flet**, crear una aplicación que imprima textos en la pantalla.

Cree un archivo nuevo con el nombre: “sumas con Flet 1a.py”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```

*sumas con Flet 1a.py - C:\Users\ACER\Documents\sumas con Flet 1a.py (3.8.3)*
File Edit Format Run Options Window Help

# Ejemplo que muestra algunos widgets que utilizaremos en los
# ejercicios para sumar valores utilizando la libreria flet
# para Python 3

import flet as ft

def main(page: ft.Page):
    page.window.width=640
    page.window.height = 480

    def button_clicked(e):
        t.value = f"Los valores de los Textboxes son: '{tb1.value}', '{tb2.value}', '{tb3.value}', '{tb4.value}' "
        page.update()

    t = ft.Text()
    tb1 = ft.TextField(label="Standard")
    tb2 = ft.TextField(label="Disabled", disabled=True, value="Nombre")
    tb3 = ft.TextField(label="Read-only", read_only=True, value="Apellidos")
    tb4 = ft.TextField(label="Con placeholder", hint_text="Escriba un texto aqui")
    b = ft.ElevatedButton(text="Enviar", on_click=button_clicked)
    page.add(tb1, tb2, tb3, tb4, b, t)

ft.app(target=main)

```

En este ejemplo mostramos cómo activar o desactivar algunas propiedades de los cuatro objetos **TextField**. Estos objetos serán manipulados para realizar cambios o conversiones en textos o números entrados por el usuario.

Este es el resultado del ejemplo 27:



The screenshot shows a web application window with a light blue background. It contains four textboxes stacked vertically, each with a label above it: "Standard", "Disabled", "Read-only", and "Con placeholder". The "Standard" textbox contains the text "Standard". The "Disabled" textbox contains the text "Nombre". The "Read-only" textbox contains the text "Apellidos". The "Con placeholder" textbox contains the text "El avión vuela muy alto". Below the textboxes is a blue button with the text "Enviar". At the bottom of the window, there is a message that reads: "Los valores de los Textboxes son: ", 'Nombre', 'Apellidos', 'El avión vuela muy alto'.

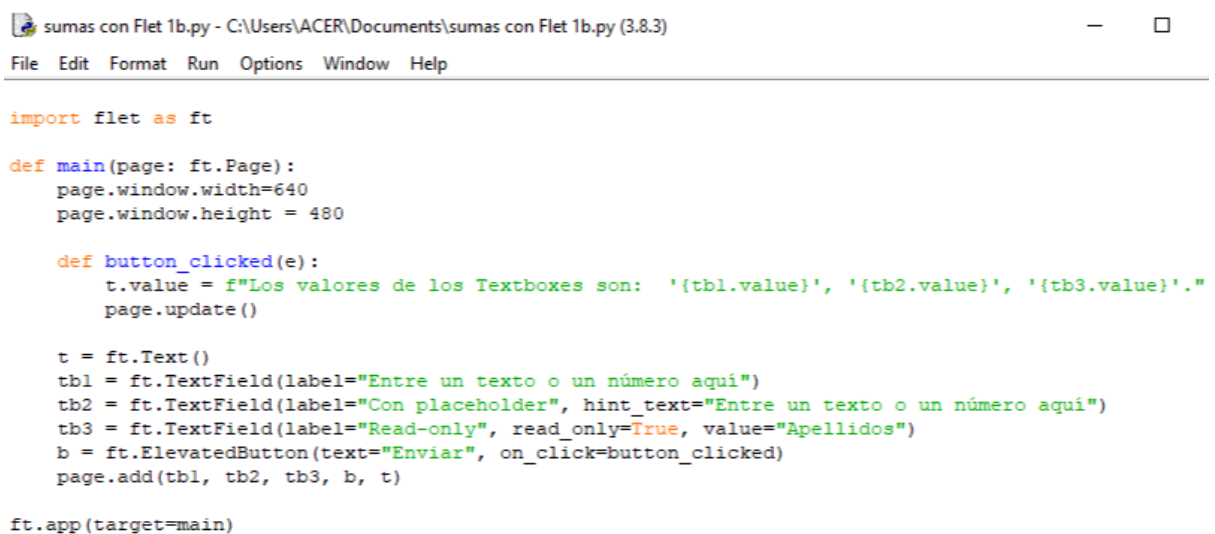
Fin del Ejemplo 27.

Ejemplo 28. Creación de una aplicación que obtenga una entrada por parte del usuario e imprima textos (bloqueados o sin bloquear) en la pantalla.

Objetivo: Utilizando la librería **flet**, crear una aplicación que imprima textos en la pantalla.

Cree un archivo nuevo con el nombre: “sumas con Flet 1b.py”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```

sumas con Flet 1b.py - C:\Users\ACER\Documents\sumas con Flet 1b.py (3.8.3)
File Edit Format Run Options Window Help

import flet as ft

def main(page: ft.Page):
    page.window.width=640
    page.window.height = 480

    def button_clicked(e):
        t.value = f"Los valores de los Textboxes son: '{tb1.value}', '{tb2.value}', '{tb3.value}'."
        page.update()

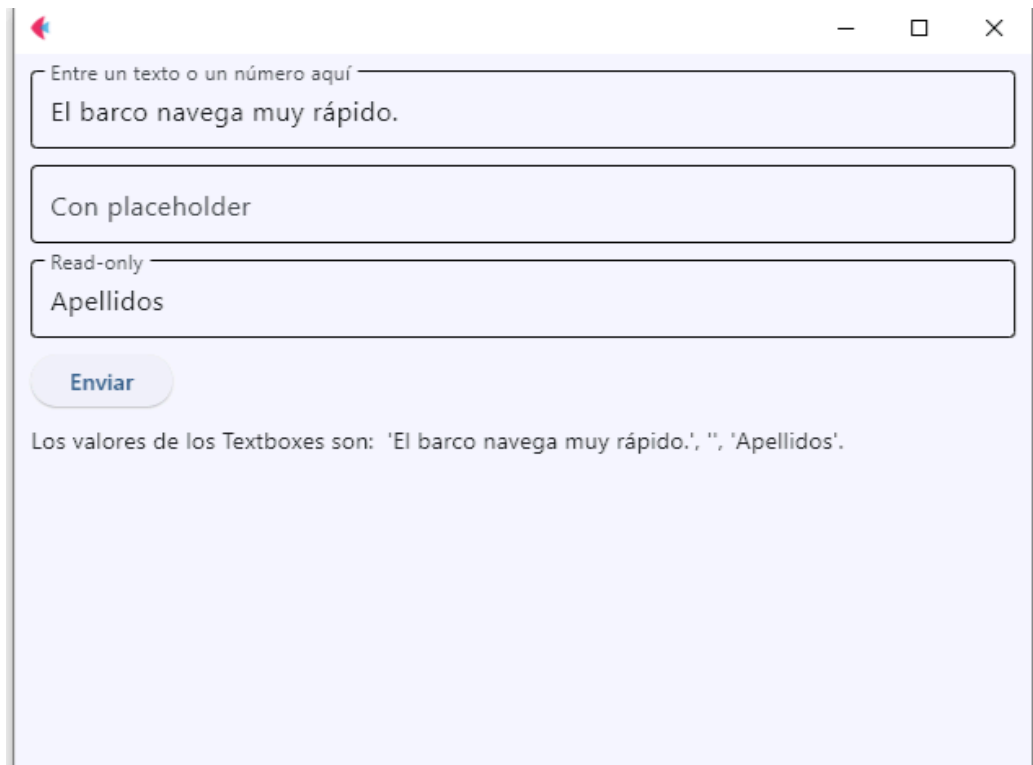
    t = ft.Text()
    tb1 = ft.TextField(label="Entre un texto o un número aquí")
    tb2 = ft.TextField(label="Con placeholder", hint_text="Entre un texto o un número aquí")
    tb3 = ft.TextField(label="Read-only", read_only=True, value="Apellidos")
    b = ft.ElevatedButton(text="Enviar", on_click=button_clicked)
    page.add(tb1, tb2, tb3, b, t)

ft.app(target=main)

```

En este ejemplo mostramos cómo activar o desactivar algunas propiedades de los cuatro objetos **TextField**. Observe que hemos quitado el objeto **tb4** del ejemplo anterior.

Este es el resultado del ejemplo 28:



The screenshot shows a web application window with a light blue background. At the top, there is a header bar with a small logo on the left and window control buttons (minimize, maximize, close) on the right. Below the header, there are three text input fields stacked vertically. The first field has a placeholder text "Entre un texto o un número aquí" and contains the text "El barco navega muy rápido.". The second field has a placeholder text "Con placeholder" and is empty. The third field has a placeholder text "Read-only" and contains the text "Apellidos". Below the input fields, there is a blue button with the text "Enviar". At the bottom of the window, there is a text label that reads "Los valores de los Textboxes son: 'El barco navega muy rápido.', '', 'Apellidos'."

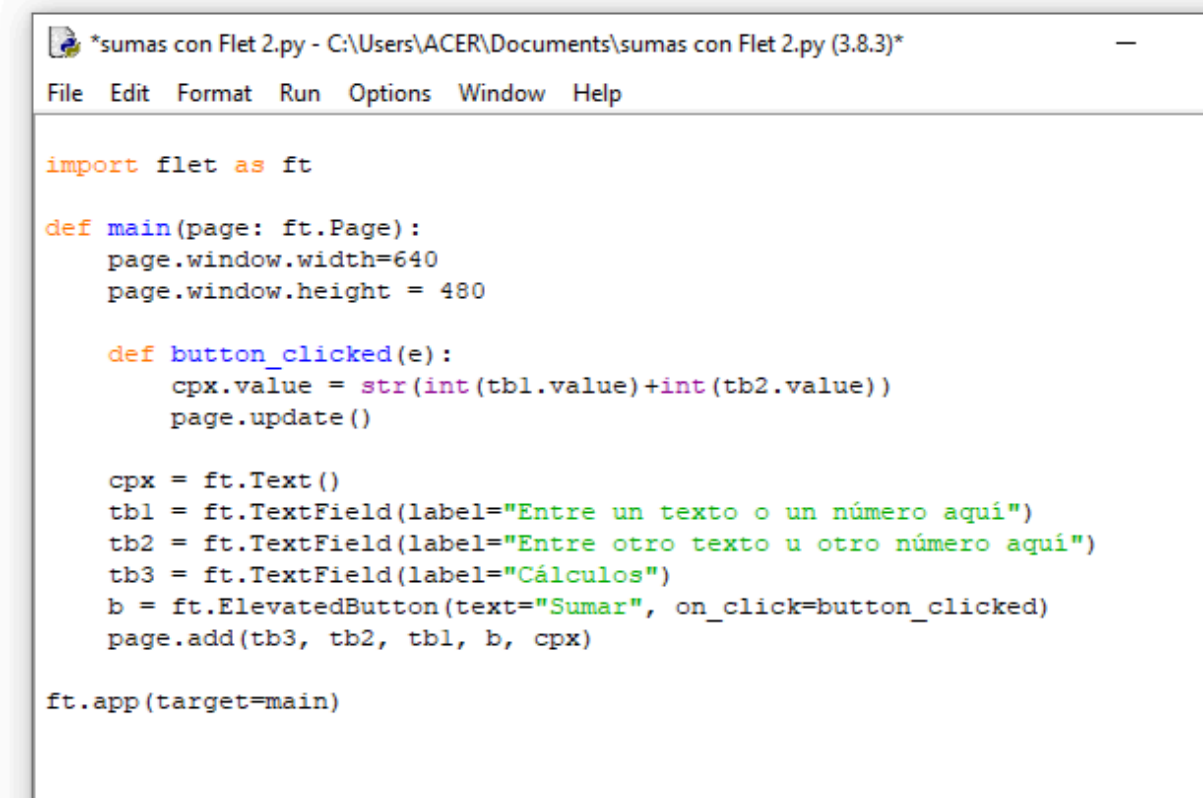
Fin del Ejemplo 28.

Ejemplo 29. Creación de un programa que imprima el resultado de sumar dos números.

Objetivo: Utilizando la librería **flet**, crear un programa que imprima el resultado de sumar dos números.

Cree un archivo nuevo con el nombre: “sumas con Flet 2.py”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.



```
*sumas con Flet 2.py - C:\Users\ACER\Documents\sumas con Flet 2.py (3.8.3)*
File Edit Format Run Options Window Help

import flet as ft

def main(page: ft.Page):
    page.window.width=640
    page.window.height = 480

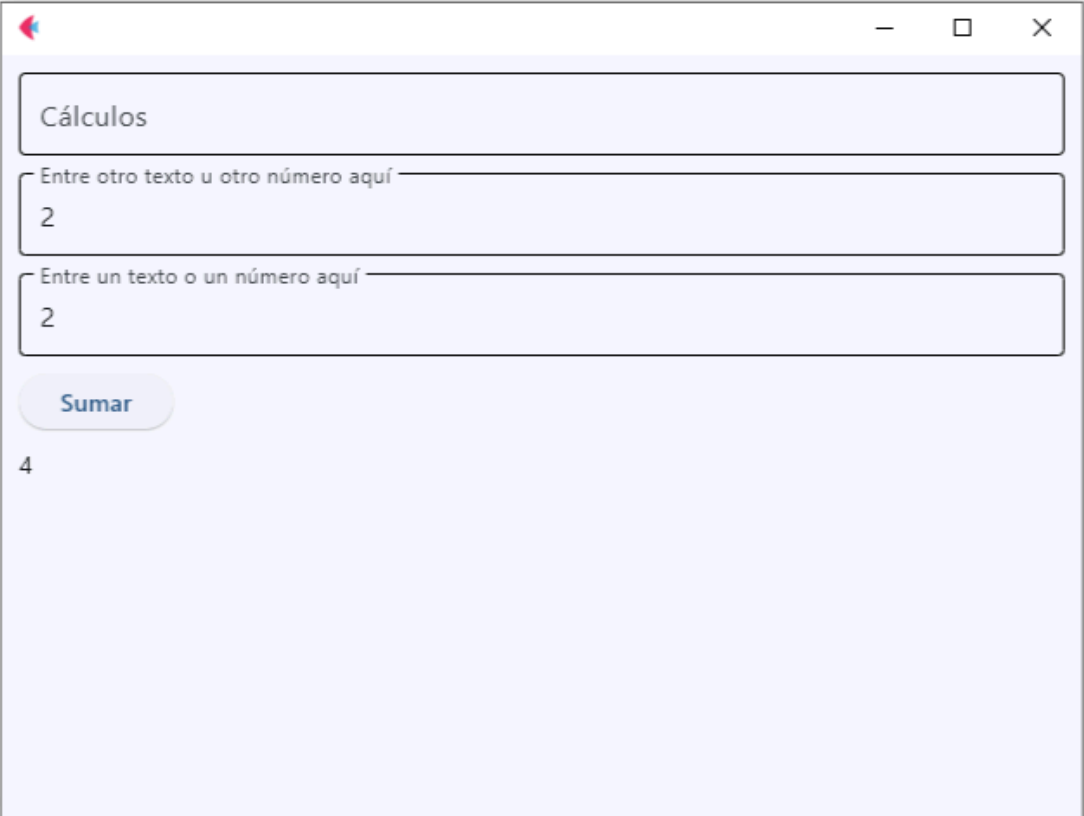
    def button_clicked(e):
        cpx.value = str(int(tb1.value)+int(tb2.value))
        page.update()

    cpx = ft.Text()
    tb1 = ft.TextField(label="Entre un texto o un número aquí")
    tb2 = ft.TextField(label="Entre otro texto u otro número aquí")
    tb3 = ft.TextField(label="Cálculos")
    b = ft.ElevatedButton(text="Sumar", on_click=button_clicked)
    page.add(tb3, tb2, tb1, b, cpx)

ft.app(target=main)
```

En este ejemplo mostramos cómo manipular los objetos **TextField**. Observe que la variable **cpx** almacena la suma de los dos valores **tb1** y **tb2**. Estos valores almacenan textos que fueron introducidos por el usuario. Para convertirlos en valores numéricos, utilizamos la función **int()** para después, convertir el resultado en una cadena (string) con la función **str()**.

Este es el resultado del ejemplo 29:



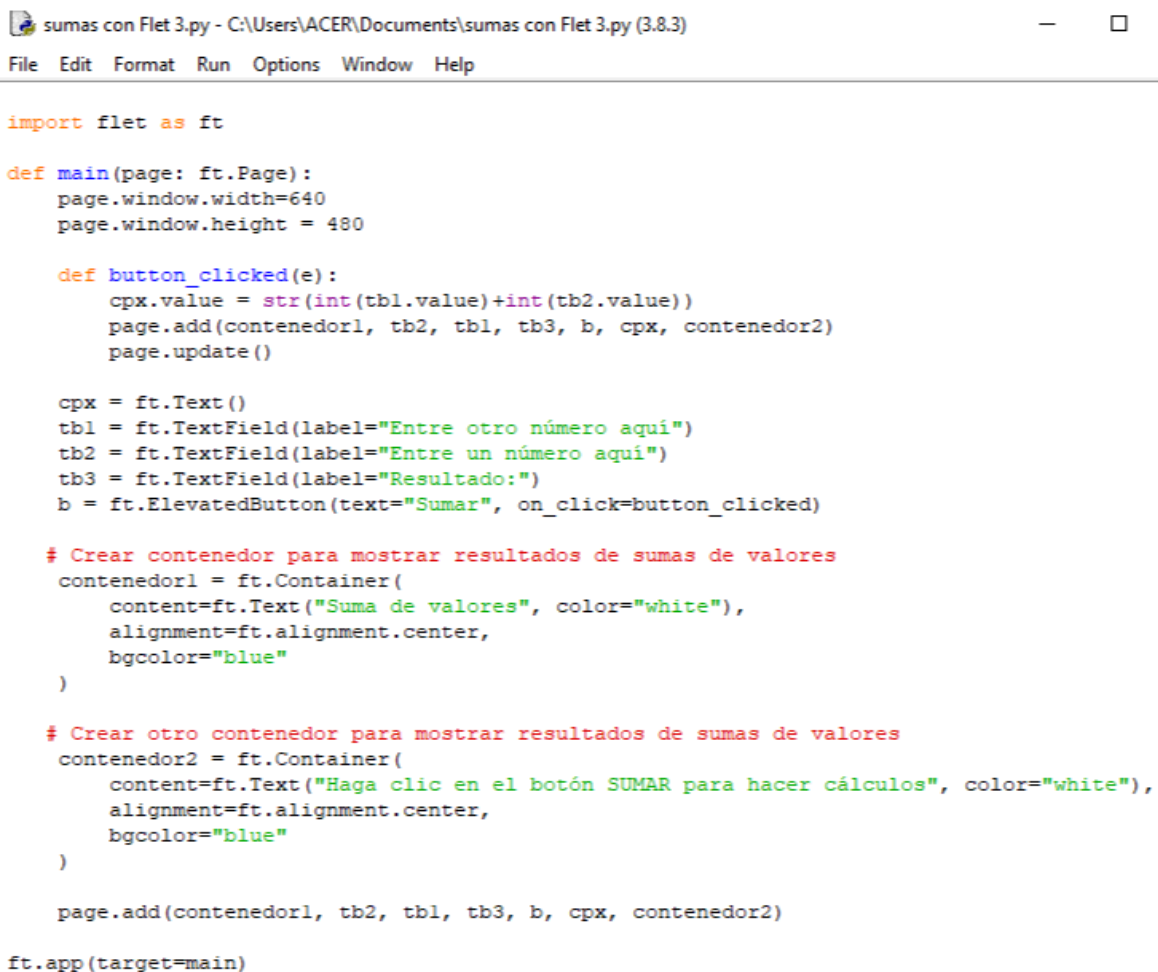
The image shows a web browser window with a title bar containing standard minimize, maximize, and close buttons. The page content is titled "Cálculos" in a light blue header. Below the header, there are two input fields. The first field is labeled "Entre otro texto u otro número aquí" and contains the number "2". The second field is labeled "Entre un texto o un número aquí" and also contains the number "2". Below these fields is a light blue button labeled "Sumar". At the bottom left of the page, the number "4" is displayed, representing the sum of the two input values.

Fin del Ejemplo 29.

Ejemplo 30. Creación de una aplicación que imprima el resultado de sumar dos números y añada dos líneas para dar más información al usuario.

Objetivo: Utilizando la librería **flet**, crear una aplicación que imprima el resultado de sumar dos números.

Cree un archivo nuevo con el nombre: “sumas con Flet 3.py” y añada las siguientes órdenes.



```

import flet as ft

def main(page: ft.Page):
    page.window.width=640
    page.window.height = 480

    def button_clicked(e):
        cpx.value = str(int(tb1.value)+int(tb2.value))
        page.add(contenedor1, tb2, tb1, tb3, b, cpx, contenedor2)
        page.update()

    cpx = ft.Text()
    tb1 = ft.TextField(label="Entre otro número aquí")
    tb2 = ft.TextField(label="Entre un número aquí")
    tb3 = ft.TextField(label="Resultado:")
    b = ft.ElevatedButton(text="Sumar", on_click=button_clicked)

    # Crear contenedor para mostrar resultados de sumas de valores
    contenedor1 = ft.Container(
        content=ft.Text("Suma de valores", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    # Crear otro contenedor para mostrar resultados de sumas de valores
    contenedor2 = ft.Container(
        content=ft.Text("Haga clic en el botón SUMAR para hacer cálculos", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

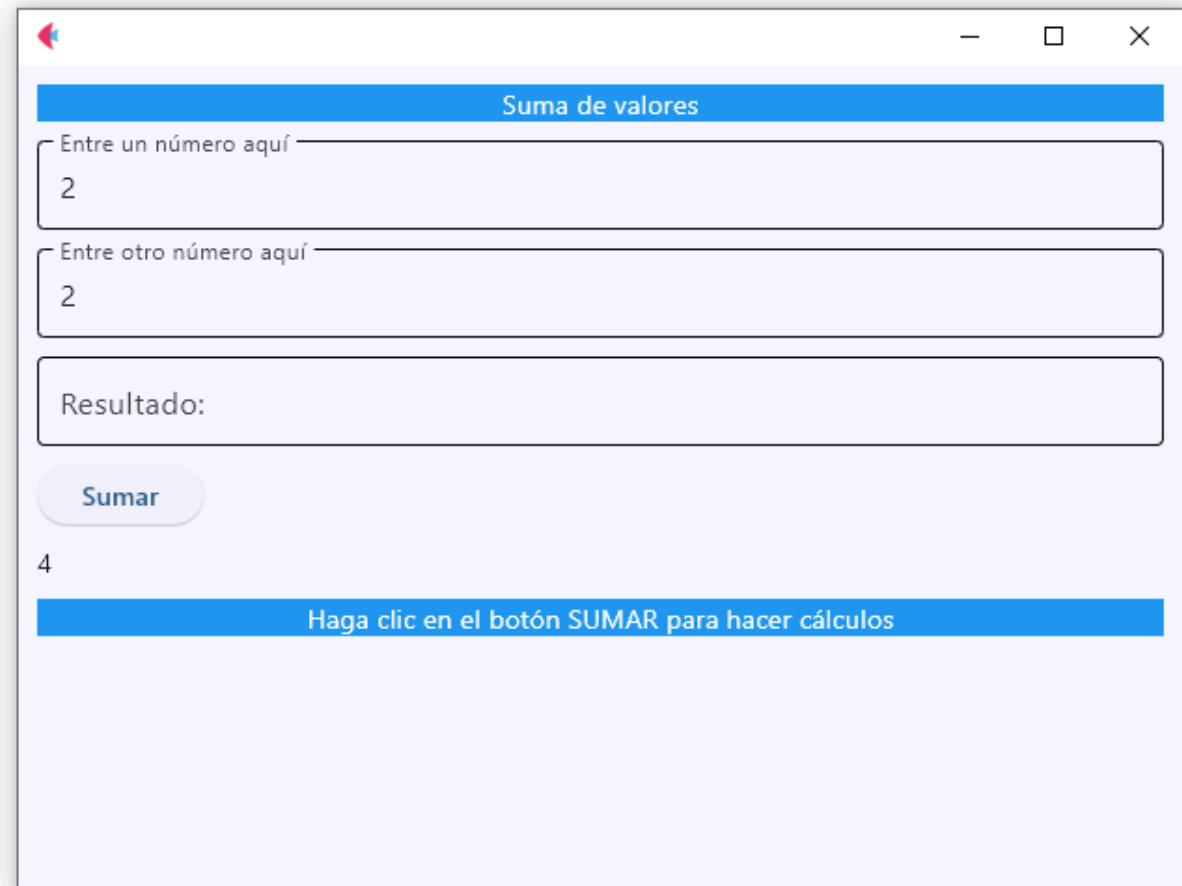
    page.add(contenedor1, tb2, tb1, tb3, b, cpx, contenedor2)

ft.app(target=main)

```

En este ejemplo mostramos cómo manipular los objetos **TextField**. Observe que la variable **cpx** almacena la suma de los dos valores **tb1** y **tb2**. Para convertirlos en valores numéricos, utilizamos la función **int()** para después, convertir el resultado en una cadena (string) con la función **str()**. Observe también que se añadieron dos líneas de texto con fondo azul (variables **contenedor1** y **contenedor2**).

Este es el resultado del ejemplo 30:



The image shows a web application window titled "Suma de valores". It contains two input fields, each with the placeholder text "Entre un número aquí" and the value "2". Below these is a "Resultado:" label and a "Sumar" button. The result "4" is displayed below the button. A blue banner at the bottom of the window contains the text "Haga clic en el botón SUMAR para hacer cálculos".

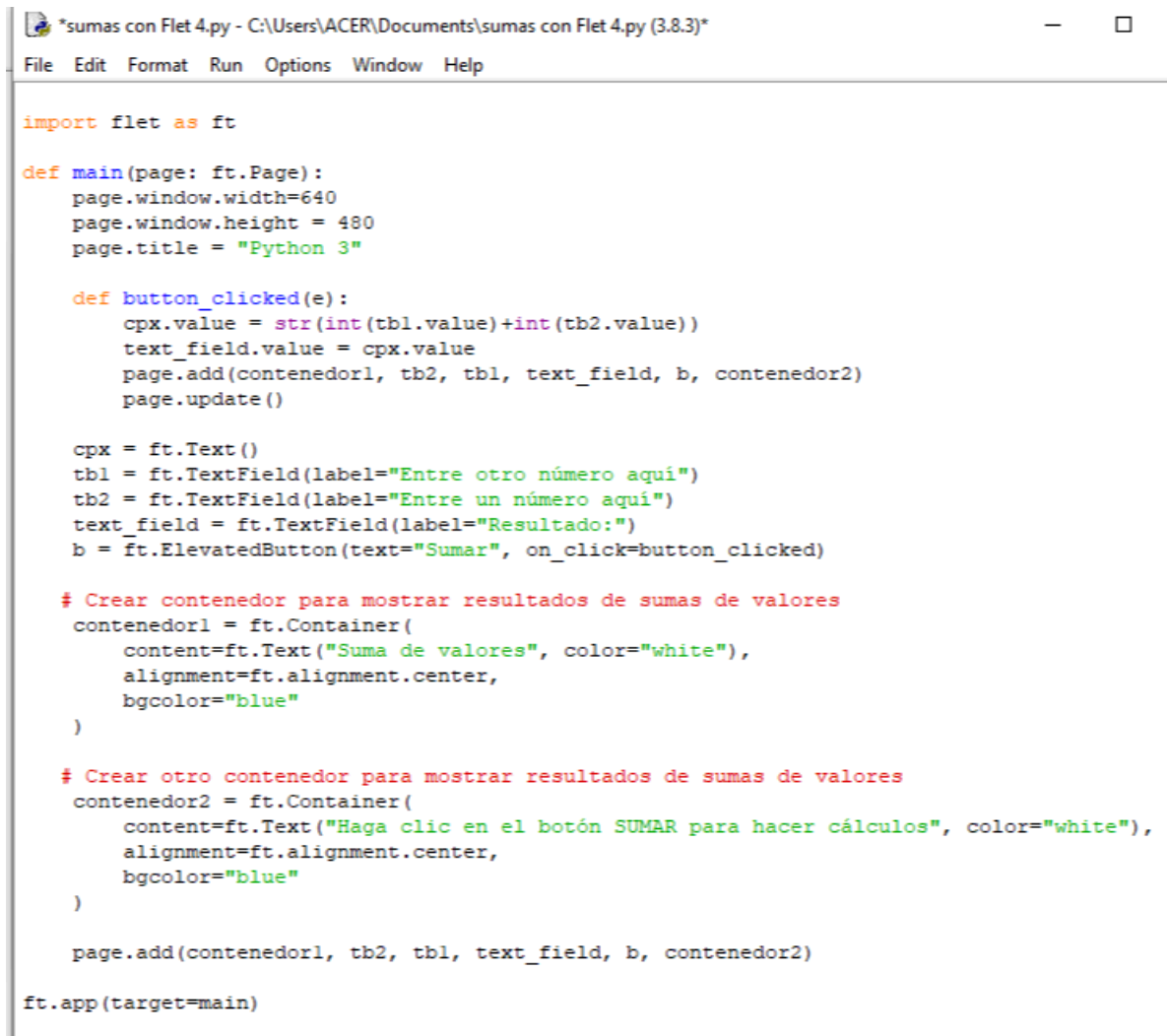
Suma de valores	
Entre un número aquí	2
Entre otro número aquí	2
Resultado:	
<button>Sumar</button>	
4	
Haga clic en el botón SUMAR para hacer cálculos	

Fin del Ejemplo 30.

Ejemplo 31. Creación de una aplicación que imprima el resultado de sumar dos números e introduzca el resultado en un objeto **TextField**.

Objetivo: Utilizando la librería **flet**, crear una aplicación que imprima el resultado de sumar dos números actualizando objetos tipo **TextField**.

Cree el siguiente archivo: “sumas con Flet 4.py” y añada las siguientes órdenes.



```

*sumas con Flet 4.py - C:\Users\ACER\Documents\sumas con Flet 4.py (3.8.3)*
File Edit Format Run Options Window Help

import flet as ft

def main(page: ft.Page):
    page.window.width=640
    page.window.height = 480
    page.title = "Python 3"

    def button_clicked(e):
        cpx.value = str(int(tb1.value)+int(tb2.value))
        text_field.value = cpx.value
        page.add(contenedor1, tb2, tb1, text_field, b, contenedor2)
        page.update()

    cpx = ft.Text()
    tb1 = ft.TextField(label="Entre otro número aquí")
    tb2 = ft.TextField(label="Entre un número aquí")
    text_field = ft.TextField(label="Resultado:")
    b = ft.ElevatedButton(text="Sumar", on_click=button_clicked)

    # Crear contenedor para mostrar resultados de sumas de valores
    contenedor1 = ft.Container(
        content=ft.Text("Suma de valores", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    # Crear otro contenedor para mostrar resultados de sumas de valores
    contenedor2 = ft.Container(
        content=ft.Text("Haga clic en el botón SUMAR para hacer cálculos", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

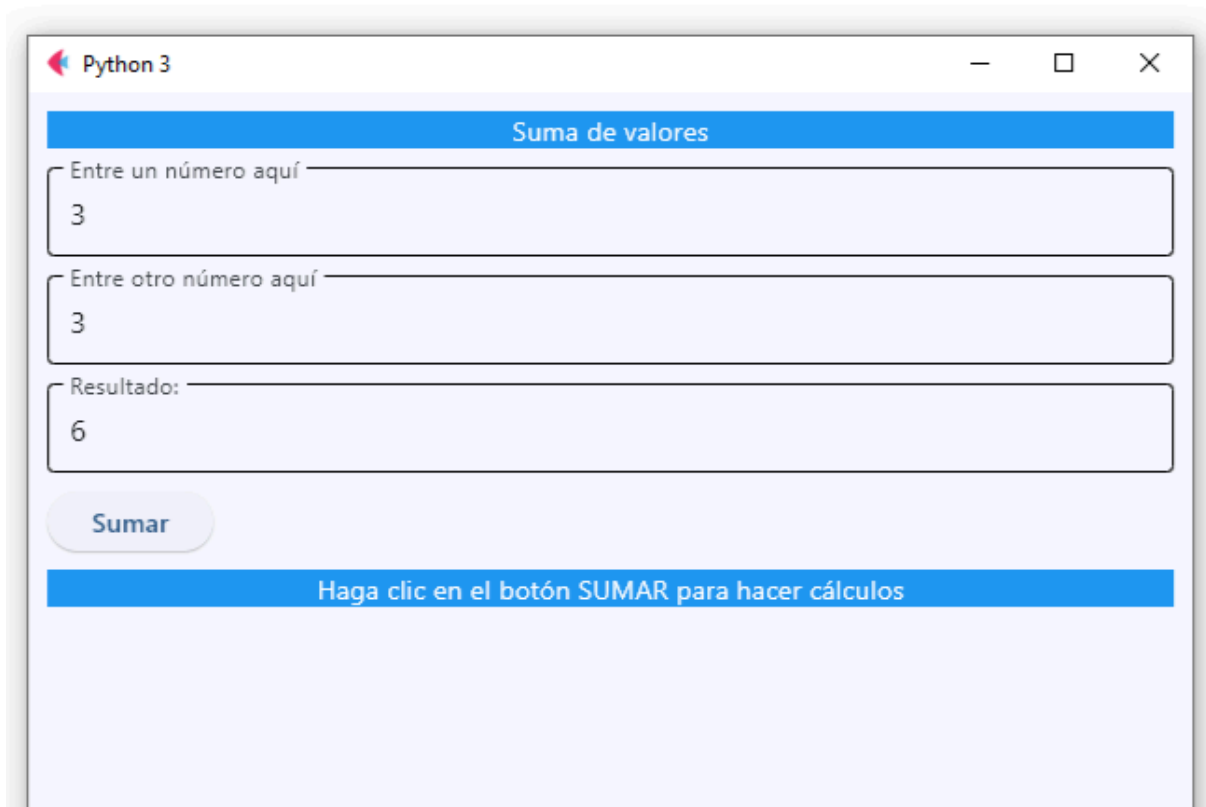
    page.add(contenedor1, tb2, tb1, text_field, b, contenedor2)

ft.app(target=main)

```

En este ejemplo mostramos cómo manipular los objetos **TextField**. Observe que la variable **cpx** almacena la suma de los dos valores **tb1** y **tb2**. Para convertirlos en valores numéricos, utilizamos la función **int()** para después, convertir el resultado en una cadena (string) con la función **str()**. Observe que la variable **text_field** toma el resultado de la suma de la variable **cpx** para luego actualizar la página (update) con el cálculo realizado.

Este es el resultado del ejemplo 31:



En este ejemplo mostramos cómo manipular los objetos **TextField** para incorporar el resultado de la suma de los dos valores introducidos. Observe que la variable **cp_x** almacena la suma de los dos valores **tb₁** y **tb₂**. Para convertirlos en valores numéricos, utilizamos la función **int()** para después, convertir el resultado en una cadena (string) con la función **str()**. Observe que la variable **text_field** toma el resultado de la suma de la variable **cp_x** para luego actualizar la página (update) con el cálculo realizado.

Fin del Ejemplo 31.

Ejemplo 32. Creación de una aplicación que imprima el resultado de sumar o restar dos números e introduzca el resultado en un objeto **TextField**.

Objetivo: Utilizando la librería **flet**, crear una aplicación que imprima el resultado de sumar o restar dos números.

Cree el siguiente archivo: “**sumas con Flet 4r.py**” y añada las siguientes órdenes.

```
import flet as ft

def main(page: ft.Page):
    page.title = "Python 3"

    def button1_clicked(e):
        cpx.value = str(int(tbl1.value)+int(tb2.value))
        text_field.value = cpx.value
        page.add(contenedor1, tb2, tbl1, text_field, b, bb, contenedor2)
        page.update()

    def button2_clicked(e):
        cpx.value = str(-1*(int(tbl1.value))+int(tb2.value))
        text_field.value = cpx.value
        page.add(contenedor1, tb2, tbl1, text_field, b, bb, contenedor2)
        page.update()

    cpx = ft.Text()
    tbl1 = ft.TextField(label="Entre otro número aquí")
    tb2 = ft.TextField(label="Entre un número aquí")
    text_field = ft.TextField(label="Resultado:")
    b = ft.ElevatedButton(text="Sumar", on_click=button1_clicked)
    bb = ft.ElevatedButton(text="Restar", on_click=button2_clicked)

    # Crear contenedor para mostrar resultados de sumas de valores
    contenedor1 = ft.Container(
        content=ft.Text("Suma y resta de valores", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

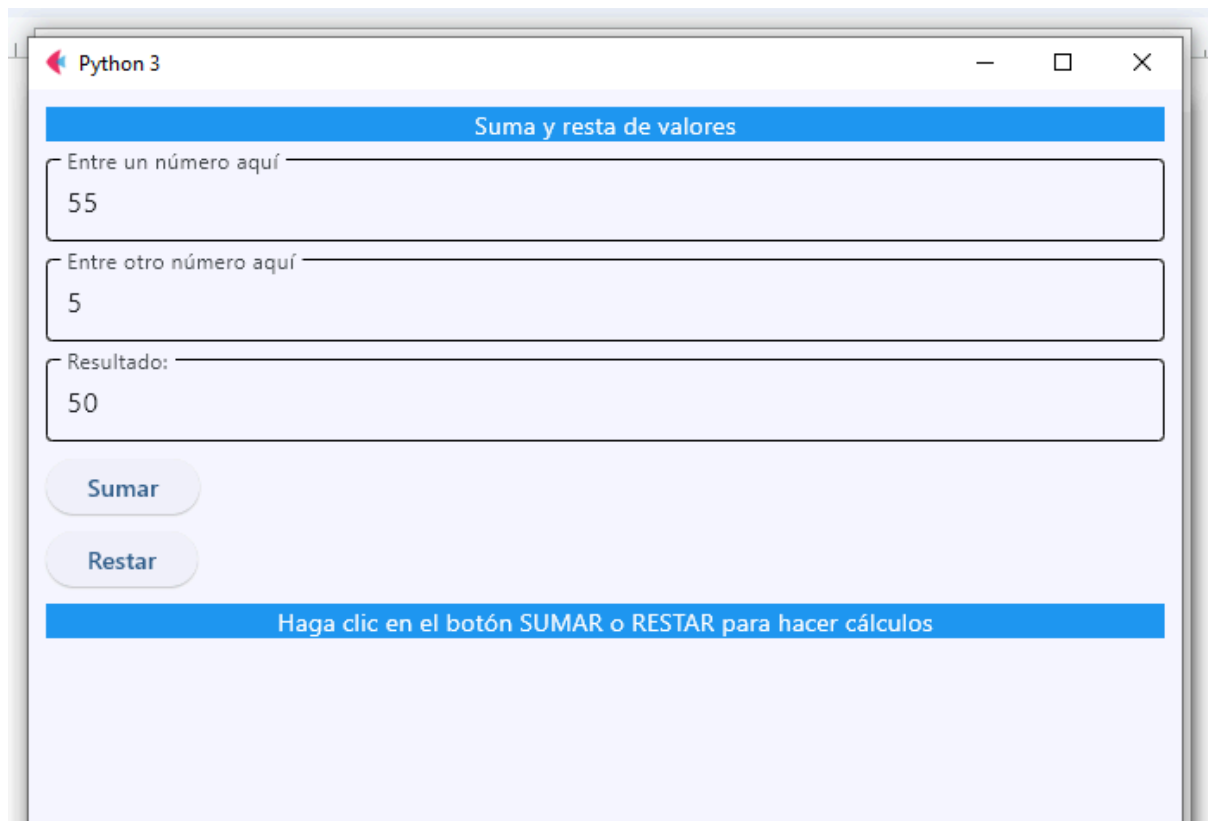
    # Crear otro contenedor para mostrar resultados de sumas de valores
    contenedor2 = ft.Container(
        content=ft.Text("Haga clic en el botón SUMAR o RESTAR para hacer cálculos", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor1, tb2, tbl1, text_field, b, bb, contenedor2)

ft.app(target=main)
```

En este ejemplo mostramos cómo manipular los objetos **TextField**. Observe la función **button2_clicked()** que se ha implementado para restar los valores. Para convertirlos en valores numéricos, utilizamos la función **int()** para después, convertir el resultado en una cadena (string) con la función **str()**. La variable **text_field** toma el resultado del cálculo de la variable **cpx** para luego actualizar la página (update) con el cálculo realizado.

Este es el resultado del ejemplo 32:



Observe que la variable **cp_x** almacena la suma o resta de los dos valores **tb₁** y **tb₂**. Para convertirlos en valores numéricos, utilizamos la función **int()** para después, convertir el resultado en una cadena (string) con la función **str()**. Observe que la variable **text_field** toma el resultado del cálculo de la variable **cp_x** para luego actualizar la página (update).

Reto: ¿Cómo implementarías un tercer botón que calcule y muestre la multiplicación de los dos valores?

Fin del Ejemplo 32.

Ejemplo 33. Creación de una aplicación que genere una clave (password) combinando dos palabras introducidas por el usuario.

Objetivo: Utilizando la librería **flet**, crear una aplicación que genere una clave (password) e imprima el resultado de concatenar dos cadenas alfanuméricas introducidas por el usuario.

Cree el siguiente archivo: “su encriptacion.py” y añada las siguientes órdenes.

```
*su encriptacion.py - C:\Users\ACER\Documents\sus encriptacion.py (3.8.3)*
File Edit Format Run Options Window Help

import flet as ft

def main(page: ft.Page):
    page.title = "Python 3"

    def button1_clicked(e):
        cpx.value = (tb1.value)+(tb2.value)
        text_field.value = cpx.value
        page.add(contenedor1, tb2, tb1, text_field, b, bb, contenedor2)
        page.update()

    def button2_clicked(e):
        cpx.value = (tb2.value)+(tb1.value)
        text_field.value = cpx.value
        page.add(contenedor1, tb2, tb1, text_field, b, bb, contenedor2)
        page.update()

    cpx = ft.Text()
    tb1 = ft.TextField(label="Entre una palabra aquí")
    tb2 = ft.TextField(label="Entre otra palabra aquí")
    text_field = ft.TextField(label="Resultado:")
    b = ft.ElevatedButton(text="Code", on_click=button1_clicked)
    bb = ft.ElevatedButton(text="Decode", on_click=button2_clicked)

    # Crear contenedor para mostrar resultados de sumas de valores
    contenedor1 = ft.Container(
        content=ft.Text("Generación de claves de seguridad.", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

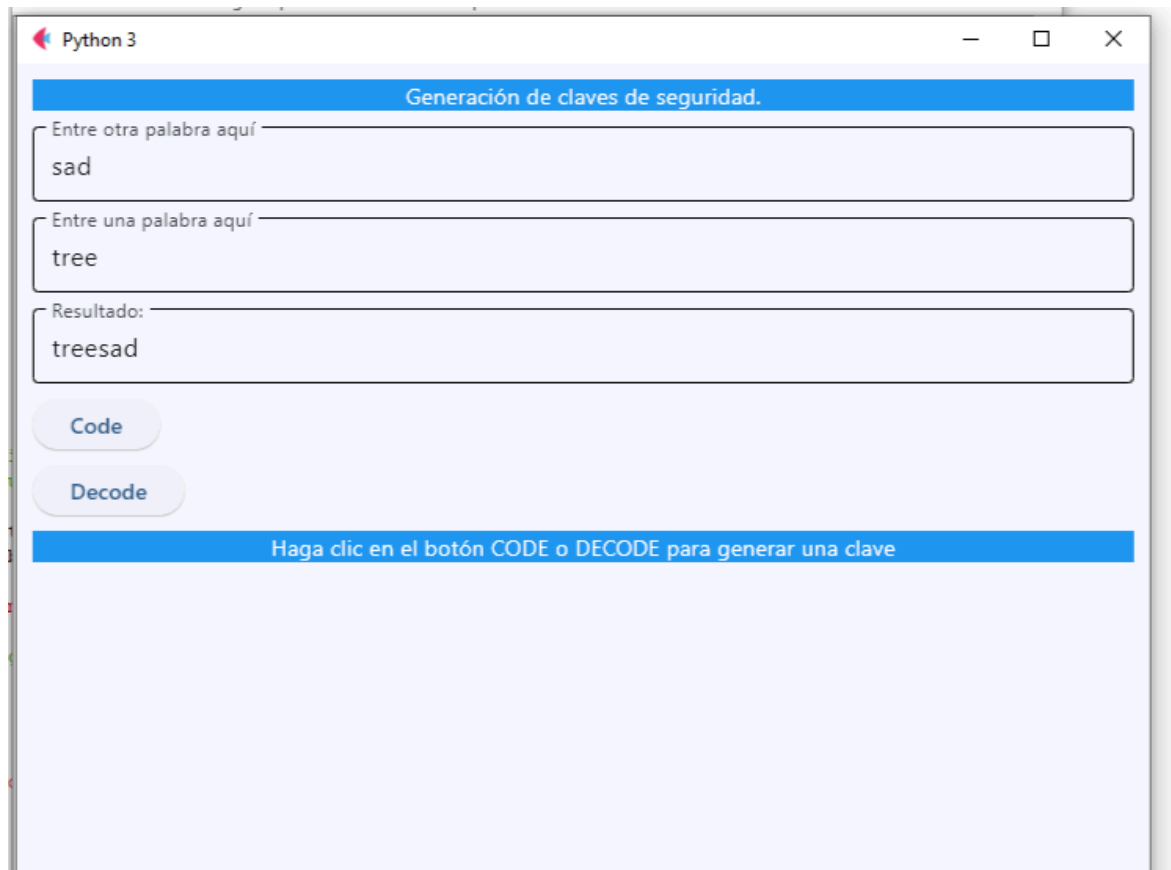
    # Crear otro contenedor para mostrar resultados de sumas de valores
    contenedor2 = ft.Container(
        content=ft.Text("Haga clic en el botón CODE o DECODE para generar una clave", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor1, tb2, tb1, text_field, b, bb, contenedor2)

ft.app(target=main)
```

En este ejemplo mostramos cómo manipular los objetos **TextField** para crear una clave de seguridad. Observe las funciones **button1_clicked()** y **button2_clicked()** que se han escrito para crear una cadena nueva a partir de los valores **tb1** y **tb2**. Al pulsar el botón **b**, se crea la clave concatenando los valores **tb1** y **tb2**, y al pulsar el botón **bb**, se decodifica la clave (password) tomando los valores **tb2** y **tb1**.

Este es el resultado del ejemplo 33:



Observe que la variable **cpx** almacena la concatenación de los dos valores **tb1** y **tb2**. La variable **text_field** toma el resultado de la variable **cpx** para luego actualizar la página (update).

Fin del Ejemplo 33.

Ejemplo 34. Creación de una aplicación que genere una clave (password) combinando números, símbolos y dos palabras introducidas por el usuario.

Objetivo: Utilizando la librería **flet**, crear una aplicación que genere una clave combinando números, símbolos y dos palabras introducidas por el usuario.

Cree el archivo: “su encriptacion plus.py” y añada las siguientes órdenes.

```
import flet as ft
def main(page: ft.Page):
    page.title = "Python 3"

    def button1_clicked(e):
        cpx.value = (tb1.value)+(tb2.value)+"$"+"XX"+"123456789"
        text_field.value = cpx.value
        page.add(contenedor1, tb2, tb1, text_field, b, bb, contenedor2)
        page.update()

    def button2_clicked(e):
        cpx.value = (tb2.value)+(tb1.value)
        text_field.value = cpx.value
        page.add(contenedor1, tb2, tb1, text_field, b, bb, contenedor2)
        page.update()

    cpx = ft.Text()
    tb1 = ft.TextField(label="Entre una palabra aquí")
    tb2 = ft.TextField(label="Entre otra palabra aquí")
    text_field = ft.TextField(label="Resultado:")
    b = ft.ElevatedButton(text="Code", on_click=button1_clicked)
    bb = ft.ElevatedButton(text="Decode", on_click=button2_clicked)

    # Crear contenedor para mostrar resultados de sumas de valores
    segu=" Para aumentar la seguridad, introduzca palabras de más de cinco letras"
    contenedor1 = ft.Container(
        content=ft.Text("Generación de claves de seguridad."+segu, color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    # Crear otro contenedor para mostrar resultados de sumas de valores
    contenedor2 = ft.Container(
        content=ft.Text("Haga clic en el botón CODE o DECODE para generar una clave", color="white"),|
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor1, tb2, tb1, text_field, b, bb, contenedor2)

ft.app(target=main)
```

En el ejemplo mostramos cómo manipular los objetos **TextField** para crear una clave de seguridad. Observe las funciones **button1_clicked()** y **button2_clicked()** que se han escrito para crear una cadena nueva a partir de sumar los valores **tb1** y **tb2**. Al pulsar el botón **b**, se crea la clave concatenando los valores **tb1**, **tb2** y la cadena “\$XX123456789”. Al pulsar el botón **bb**, se decodifica la clave tomando únicamente los valores **tb2** y **tb1**.

Este es el resultado del ejemplo 34:

Python 3

Generación de claves de seguridad. Para aumentar la seguridad, introduzca palabras de más de cinco letras

Entre otra palabra aquí
delta

Entre una palabra aquí
force

Resultado:
forcedelta\$XX123456789

Code

Decode

Haga clic en el botón CODE o DECODE para generar una clave

Observe que la variable **cpx** almacena la suma de los valores **tb1**, **tb2** y la cadena **"\$XX123456789"**. La variable **text_field** toma el resultado de la variable **cpx** para luego actualizar la página (update).

Fin del Ejemplo 34.

Ejemplo 35. Creación de una aplicación que genere una clave (password) con una longitud mínima de trece caracteres (combinando números, símbolos y dos palabras introducidas por el usuario).

Objetivo: Utilizando la librería **flet**, crear una aplicación que genere una clave con una longitud mínima de trece caracteres (combinando números, símbolos y dos palabras introducidas por el usuario).

Cree el archivo: “su encriptacion ultra.py” y añada las siguientes órdenes.

```
import flet as ft
def main(page: ft.Page):
    page.title = "Python 3"

    def button1_clicked(e):
        cpx.value = (tb1.value)+(tb2.value)+"$$"+"XX"+"123456789"
        text_field.value = cpx.value
        page.add(contenedor1, tb1, tb2, text_field, b, bb, contenedor2)
        page.update()

    def button2_clicked(e):
        cpx.value = (tb1.value)+(tb2.value)
        text_field.value = cpx.value
        page.add(contenedor1, tb1, tb2, text_field, b, bb, contenedor2)
        page.update()

    cpx = ft.Text()
    tb1 = ft.TextField(label="Entre una palabra aquí", value="hola", color="grey")
    tb2 = ft.TextField(label="Entre otra palabra aquí", value="mundo", color="grey")
    text_field = ft.TextField(label="Resultado:")
    b = ft.ElevatedButton(text="Code", on_click=button1_clicked)
    bb = ft.ElevatedButton(text="Decode", on_click=button2_clicked)

    # Crear contenedor para mostrar resultados de sumas de valores
    segu=" Para aumentar la seguridad, introduzca palabras de más de cinco letras"
    contenedor1 = ft.Container(
        content=ft.Text("Generación de claves de seguridad."+segu, color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    # Crear otro contenedor para mostrar resultados de sumas de valores
    contenedor2 = ft.Container(
        content=ft.Text("Haga clic en el botón CODE o DECODE para generar una clave", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor1, tb1, tb2, text_field, b, bb, contenedor2)

ft.app(target=main)
```

En el ejemplo mostramos cómo manipular los objetos **TextField** para crear una clave de seguridad. Observe las funciones **button1_clicked()** y **button2_clicked()** que se han escrito para crear una cadena nueva a partir de concatenar los valores **tb1** y **tb2**. Al pulsar el botón **b**, se crea la clave con los valores **tb1**, **tb2** y la cadena “\$\$XX123456789”. Al pulsar el botón **bb**, se decodifica la clave tomando únicamente los valores **tb2** y **tb1**.

Este es el resultado del ejemplo 35:

Python 3

Generación de claves de seguridad. Para aumentar la seguridad, introduzca palabras de más de cinco letras

Entre una palabra aquí
hola

Entre otra palabra aquí
mundo

Resultado:
holamundo\$\$XX123456789

Code

Decode

Haga clic en el botón CODE o DECODE para generar una clave

Observe que la variable **cpx** almacena los valores **tb1**, **tb2** y la siguiente cadena de 13 caracteres: `$$xx123456789`. Además, se ha cambiado el orden en que se concatenan los valores de **tb1** y **tb2**. La variable **text_field** toma el resultado de la variable **cpx** para luego actualizar la página (`update`). Ahora, si borramos el contenido de las dos primeras palabras (“hola” y “mundo”), la clave construida seguirá teniendo una longitud de 13 caracteres, lo que la convierte en una clave segura.

Fin del Ejemplo 35.

Aspectos a tener en cuenta al escribir programas para construir claves de seguridad:

Nunca utilice datos personales para construir claves de seguridad

Una clave o “password” de 13 caracteres puede ser bastante segura si incluye una combinación de letras mayúsculas y minúsculas, números y símbolos. Por ejemplo, una clave como **\$X123456789x** es mucho más segura que una clave simple, como **“StarWars1977”**, que solo contiene letras o números.

Generalmente, se recomienda utilizar claves más largas para una mayor seguridad, especialmente si se está intentando proteger información muy sensible.

La longitud de una clave es un factor muy importante para su seguridad. Generalmente, se considera que una clave es segura si tiene al menos 12 caracteres. Sin embargo, cuanto más larga sea la clave, mejor, ya que esto aumenta el tiempo y los recursos necesarios para descifrarla mediante ataques de fuerza bruta.

En el siguiente ejemplo añadiremos una complicación más a la generación de la clave: Al final de la cadena añadiremos la palabra “uno” o la palabra “dos” seguido del símbolo de exclamación. Para ello crearemos la función **fun1(n)**, donde **n** puede ser 1 o 2, que representa la palabra que queremos añadir a la clave generada: Si el parámetro pasado es 1, se añadirá la palabra “uno”. Si el parámetro es 2, se añadirá la palabra “dos”.

Ejemplo 36. Creación de una aplicación que genere una clave (password) con una longitud mínima de 17 caracteres (combinando números, símbolos y dos palabras introducidas por el usuario).

Objetivo: Utilizando la librería **flet**, crear una aplicación que genere una clave con una longitud mínima de 17 caracteres (combinando números, símbolos y dos palabras introducidas por el usuario).

Cree el archivo: “su encriptacion ultra II.py” y añada las siguientes órdenes.

```
import flet as ft
def main(page: ft.Page):
    def fun1(n):
        if n==1:
            return("uno!")
        if n==2:
            return("dos!")
    def button1_clicked(e):
        cpx.value = (tb1.value)+(tb2.value)+"$$"+"XX"+"123456789"+fun1(1)
        text_field.value = cpx.value
        page.add(contenedor1, tb1, tb2, text_field, b, bb, contenedor2)
        page.update()
    def button2_clicked(e):
        cpx.value = (tb1.value)+(tb2.value)
        text_field.value = cpx.value
        page.add(contenedor1, tb1, tb2, text_field, b, bb, contenedor2)
        page.update()

    cpx = ft.Text()
    tb1 = ft.TextField(label="Entre una palabra aquí", value="hola", color="grey")
    tb2 = ft.TextField(label="Entre otra palabra aquí", value="mundo", color="grey")
    text_field = ft.TextField(label="Resultado:")
    b = ft.ElevatedButton(text="Code", on_click=button1_clicked)
    bb = ft.ElevatedButton(text="Decode", on_click=button2_clicked)

    # Crear contenedores para mostrar resultados de sumas de valores
    segu=" Para aumentar la seguridad, introduzca palabras de más de cinco letras"
    contenedor1 = ft.Container(
        content=ft.Text("Generación de claves de seguridad."+segu, color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    contenedor2 = ft.Container(
        content=ft.Text("Haga clic en el botón CODE o DECODE para generar una clave", color="white"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    page.add(contenedor1, tb1, tb2, text_field, b, bb, contenedor2)
ft.app(target=main)
```

En el ejemplo mostramos cómo manipular los objetos **TextField** para crear una clave de seguridad. Observe las funciones **button1_clicked()** y **button2_clicked()** que se han escrito para crear una cadena nueva a partir de concatenar los valores **tb1** y **tb2**. Al pulsar el botón **b**, se crea la clave con los valores **tb1**, **tb2** y la cadena “\$\$XX123456789uno\$”. Al pulsar el botón **bb**, se decodifica la clave tomando únicamente los valores **tb2** y **tb1**.

Este es el resultado del ejemplo 36:

Generación de claves de seguridad. Para aumentar la seguridad, introduzca palabras de más de cinco letras

Entre una palabra aquí
hola

Entre otra palabra aquí
mundo

Resultado:
holamundo\$\$XX123456789uno!

Code

Decode

Haga clic en el botón CODE o DECODE para generar una clave

Observe que la variable **cpx** almacena los valores **tb1**, **tb2** y la siguiente cadena de 17 caracteres: `$$XX123456789uno$`. Además, se ha cambiado el orden en que se concatenan los valores de **tb1** y **tb2**. La variable **text_field** toma el resultado de la variable **cpx** para luego actualizar la página (update). Ahora, si borramos el contenido de las dos primeras palabras (“hola” y “mundo”), la clave construida seguirá teniendo una longitud de 17 caracteres, lo que la convierte en una clave más segura.

Fin del Ejemplo 36.

La función `fun1(n)` del ejemplo anterior debería elegir la palabra de forma aleatoria de una lista de 5 palabras o más.

Observe en el siguiente ejemplo que para generar números aleatorios, debemos utilizar la librería `random`.

Podríamos utilizar las siguientes instrucciones para generar 5 números aleatorios que estén entre el 1 y el 10.

```
import random

def genera_random_num(n):
    random_nums = [random.randint(1, 10) for _ in range(n)]
    return random_nums

print(genera_random_num(5))
```

Si únicamente necesitamos un número aleatorio, utilizaremos las siguientes instrucciones:

```
import random

def genera_random_numbers(n):
    random_nums = [random.randint(1, 10) for _ in range(n)]
    return random_nums

print(genera_random_numbers(1))
```

Ejemplo 37. Creación de una aplicación que genere un número aleatorio.

Objetivo: Utilizando la librería **flet**, genere un número aleatorio entre el 1 y el 6 utilizando la librería **random**.

Cree el archivo: “solomundo1randomnumsFLET.py” y añada las siguientes órdenes:

```
import flet as ft
import random

def genera_random_nums(n):
    #genera numeros aleatorios entre el 1 y el 6
    random_num = [random.randint(1, 6) for _ in range(n)]
    return random_num

def main(page):
    page.title = "Números aleatorios"
    page.window.width = 400
    page.window.height = 300

    # Crear un contenedor con alineación centrada y fondo azul
    contenedor1 = ft.Container(
        content=ft.Text("Generar un número aleatorio entre el 1 y el 6", color="yellow"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    zx=genera_random_nums(1)
    ## imprime en la terminal 1 numero generado para comprobaciones
    print(zx)

    # Crear un contenedor para ver contenido de variable zx
    contenedor2 = ft.Container(
        content=ft.Text(zx, color="blue"),
        alignment=ft.alignment.center,
    )

    page.add(contenedor1,contenedor2)

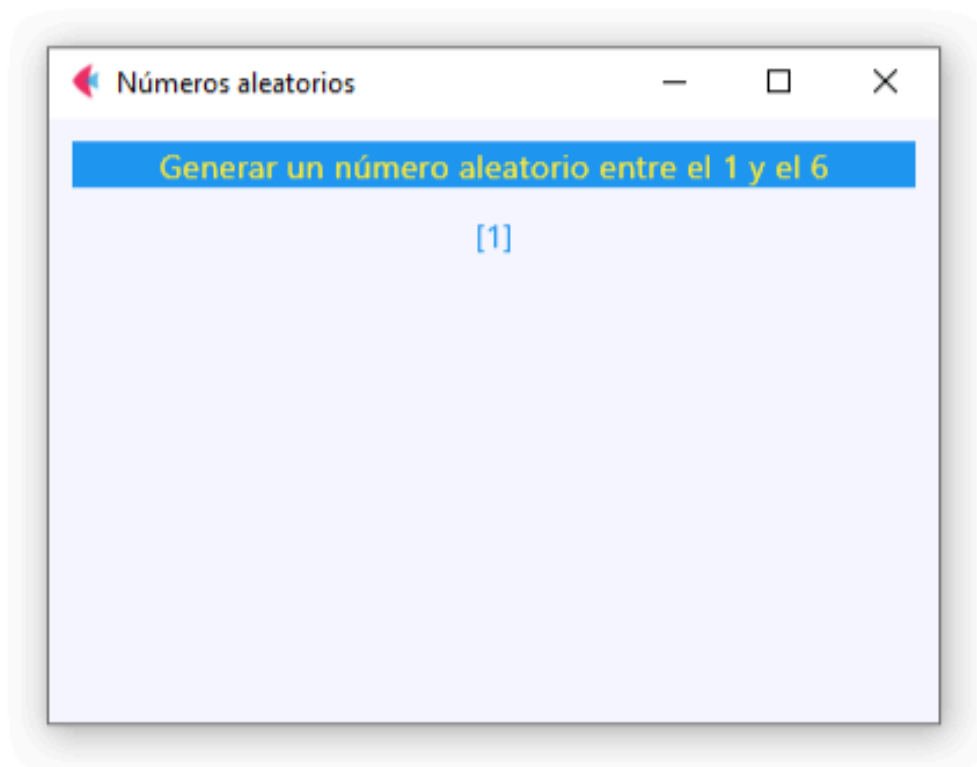
ft.app(target=main)
```

En el ejemplo mostramos cómo generar solo un número aleatorio entre el 1 y el 6. El número de valores generados que se quieren obtener de manera aleatoria se pasa como parámetro en la función **genera_random_nums()**.

Si se quiere obtener solo un número aleatorio, debemos escribir:

<pre>zx = genera_random_nums(1)</pre>

Este es el resultado del ejemplo 37:



El número generado de manera aleatoria es el **1**. Observe que la variable **zx** no es necesario que se imprima en la ventana de la terminal. Únicamente se ha hecho para comprobar que se pasan los parámetros correctamente. Esa línea se puede omitir.

Fin del Ejemplo 37.

Ejemplo 38. Creación de una aplicación que genere 5 números aleatorios.

Objetivo: Utilizando la librería **flet**, genere números aleatorios entre el 1 y el 10 utilizando la librería **random**.

Cree el archivo: “solomundo5randomnumsFLET.py” y añada las siguientes órdenes:

```
import flet as ft
import random

def genera_random_nums(n):
    #genera numeros aleatorios entre el 1 y el 10
    random_num = [random.randint(1, 10) for _ in range(n)]
    return random_num

def main(page):
    page.title = "Números aleatorios"
    page.window.width = 400
    page.window.height = 300

    # Crear un contenedor con alineación centrada y fondo azul
    contenedor1 = ft.Container(
        content=ft.Text("Generar números aleatorios entre el 1 y el 10", color="yellow"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    zx=genera_random_nums(5)
    ## imprime en la terminal los 5 numeros generados para comprobaciones
    print(zx)

    # Crear un contenedor para ver contenido de variable zx
    contenedor2 = ft.Container(
        content=ft.Text(zx, color="blue"),
        alignment=ft.alignment.center,
    )

    page.add(contenedor1, contenedor2)

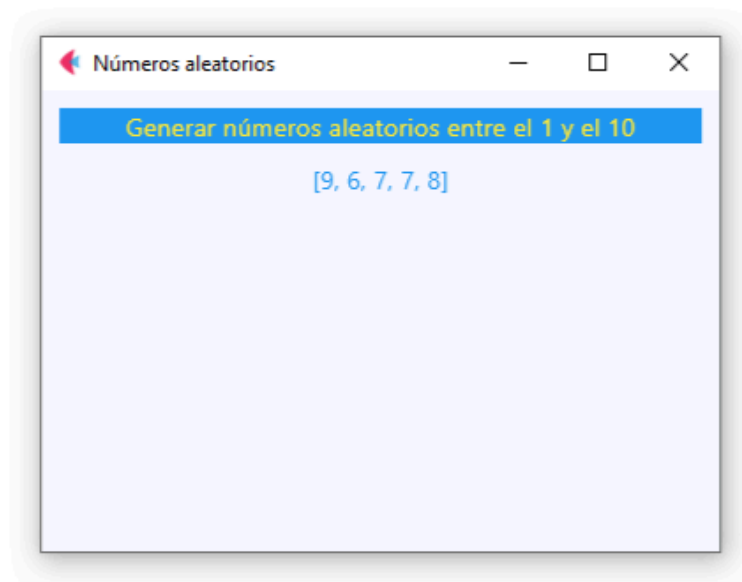
ft.app(target=main)
```

En el ejemplo mostramos cómo generar números aleatorios entre el 1 y el 10. El número de valores generados que se quieren obtener de manera aleatoria se pasa como parámetro en la función **genera_random_nums()**.

Si se desea obtener más de un número aleatorio, deberemos cambiar el número 1 en la siguiente línea:

```
zx = genera_random_nums(1)
```

Este es el resultado del ejemplo 38:



Se han generado 5 números entre el 1 y el 10 de manera aleatoria. Observe que la variable **zx** no es necesario que se imprima en la ventana de la terminal. Únicamente se ha hecho para comprobar que se pasan los parámetros correctamente. Esa línea se puede omitir.

Si se desea obtener más de 5 números aleatorios, deberemos cambiar el número 5 en la siguiente línea del código:

```
zx = genera_random_nums(5)
```

Fin del Ejemplo 38.

Ejemplo 39. Juego de los dados.

Objetivo: Utilizando la librería **flet**, generar dos números aleatorios entre el 1 y el 6 utilizando la librería **random**.

Cree el archivo: “solomundo2randomnumsFLET.py” y añada las siguientes órdenes:

```
import flet as ft
import random

def genera_random_nums(n):
    #genera 2 numeros aleatorios entre el 1 y el 6 (por ejemplo: juego de tirar dados)
    random_num = [random.randint(1, 6) for _ in range(n)]
    return random_num

def main(page):
    page.title = "Números aleatorios"
    page.window.width = 400
    page.window.height = 300

    # Crear un contenedor con alineación centrada y fondo azul
    contenedor1 = ft.Container(
        content=ft.Text("Generar dos números aleatorio entre el 1 y el 6", color="yellow"),
        alignment=ft.alignment.center,
        bgcolor="blue"
    )

    zx=genera_random_nums(2)
    ## imprime en la terminal 1 numero generado para comprobaciones
    print(zx)

    # Crear un contenedor para ver contenido de variable zx
    contenedor2 = ft.Container(
        content=ft.Text(zx, color="blue"),
        alignment=ft.alignment.center,
    )

    page.add(contenedor1, contenedor2)

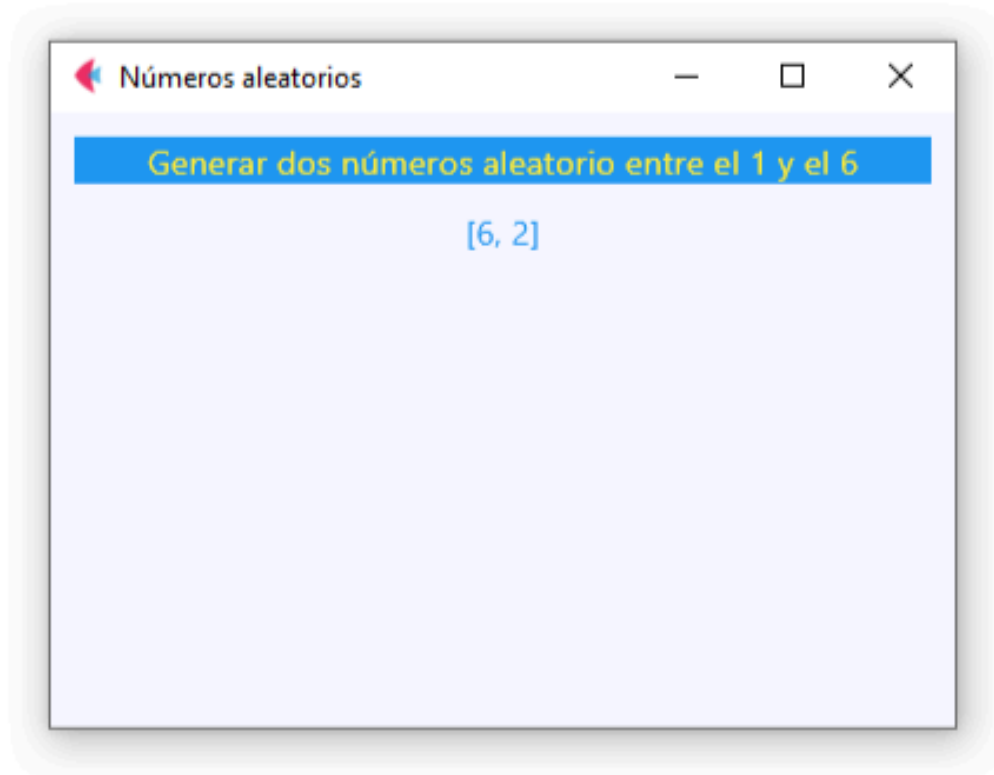
ft.app(target=main)
```

En el ejemplo mostramos cómo generar números aleatorios entre el 1 y el 6. El número de valores generados que se quieren obtener de manera aleatoria se pasa como parámetro en la función **genera_random_nums()**.

Para obtener 2 números aleatorios, hemos cambiado el número que está entre paréntesis en la siguiente línea y hemos incluido el número 2:

```
zx = genera_random_nums(2)
```

Este es el resultado del ejemplo 39:



Se han generado 2 números entre el 1 y el 6 de manera aleatoria. Observe que la variable **zx** no es necesario que se imprima en la ventana de la terminal. Únicamente se ha hecho para comprobar que se pasan los parámetros correctamente. Esa línea se puede omitir.

Fin del Ejemplo 39.

Reto 1:

Corrija los errores en el siguiente bloque de código. El ejemplo debe imprimir la historia almacenada en la siguiente lista (array):

```
# Este es un ejercicio de repaso de listas, matrices y arrays
# No es necesario utilizar la libreria flet.

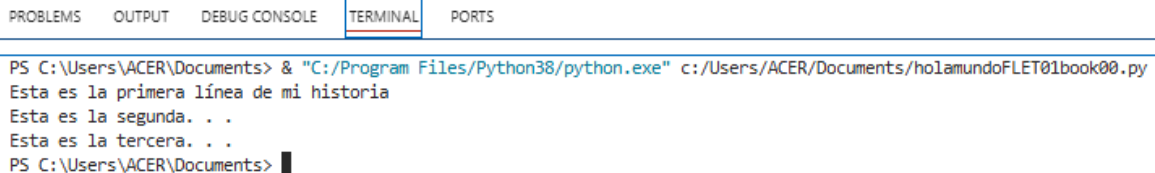
# La lista contiene 3 elementos:

lineas = ["Esta es la primera línea de mi historia",
          "Esta es la segunda. . .",
          "Esta es la tercera. . ."]

i = 0

while i < len(lineas):
    print(lineas[i])
    i=i+
```

Resultado esperado después de ejecutar el ejemplo anterior:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ACER\Documents> & "C:/Program Files/Python38/python.exe" c:/Users/ACER/Documents/holamundoFLET01book00.py
Esta es la primera línea de mi historia
Esta es la segunda. . .
Esta es la tercera. . .
PS C:\Users\ACER\Documents> █
```

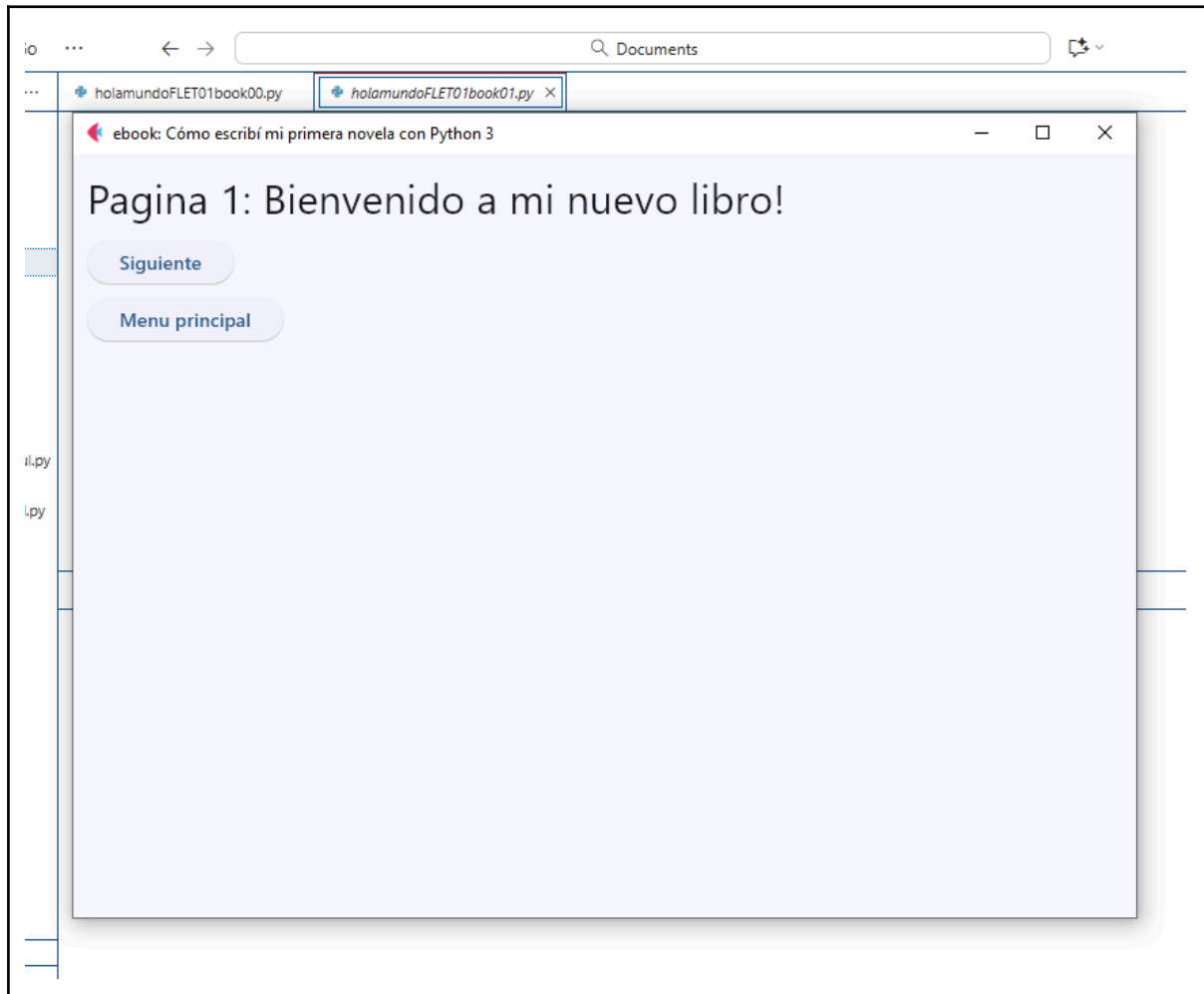
Reto 2:

La estructura del bloque de código siguiente debe crear una pequeña aplicación con la librería Flet que simula un libro digital con una historia de dos páginas con diferentes textos:

- Debe definir tres vistas: una para el menú principal y dos para las páginas de la historia.
- Cada página (slide) debe tener botones para navegar entre ellas y volver al menú principal.
- Cuando el usuario haga clic en un botón, la función correspondiente debe cambiar la vista mostrando la página seleccionada.
- El sistema debe usar rutas (/slide1, /slide2) para saber qué página mostrar.
- Al ejecutar el programa por primera vez, se debe mostrar la primera diapositiva y el usuario debe poder avanzar o retroceder usando los botones.

Corrija el siguiente bloque de código para visualizar la historia:

Resultado esperado después de ejecutar el ejemplo anterior utilizando la librería flet para visualizar textos:



¿Ha encontrado algún error?

Reto 3:

Corrija la siguiente versión del relato almacenado en la siguiente lista:

```
import flet as ft

lineas= ["La idea de escribir una novela surgió después de leer.",
        "el relato <Palmeritas a cambio de un paseo> en",]

def main(page: ft.Page):
    page.window.width = 800
    page.window.height = 600
    page.title = "ebook: Cómo escribí mi primera novela con Python 3"

    def navigate_to_slidel(e):
        page.go("/slide1")

    def navigate_to_slide2(e):
        page.go("/slide2")

    # Slide/Pagina 1 with Next button
    slide1 = ft.Column(
        [
            ft.Text("Pagina 1: Bienvenido a mi nuevo libro!", size=30),
            ft.Text(lineas[0], size=18),
            ft.Text(lineas[1], size=18),
            ft.Text("En el año 2010.", size=18),
            ft.ElevatedButton(text="Siguiente", on_click=navigate_to_slide2),
            ft.ElevatedButton(text="Menu principal", on_click=lambda _: page.go("/")),
        ],
    )

    # Slide/Pagina 2 with Previous button
    slide2 = ft.Column(
        [
            ft.Text("Pagina 2: Esta es la pagina 2!", size=30),
            ft.ElevatedButton(text="Anterior", on_click=navigate_to_slidel),
            ft.ElevatedButton(text="Menu principal", on_click=lambda _: page.go("/")),
        ],
    )

    # Main menu / Main view
    main_view = ft.Column(
        [
            ft.ElevatedButton(text="Ir a Pagina 1", on_click=navigate_to_slidel),
            ft.ElevatedButton(text="Ir a Pagina 2", on_click=navigate_to_slide2),
        ],
    )

    ### Continúa en la siguiente página. . .
```

```
page.views.clear()
if page.route == "/slide1":
    page.views.append(slide1)
elif page.route == "/slide2":
    page.views.append(slide2)
else:
    page.views.append(main_view)
page.update()

page.route = "/slide1"
page.on_route_change = route_change
route_change(page.route)

ft.app(target=main)

### Fin del Reto 3.

### ¿Ha encontrado algún error? ¿Cómo se logra incluir un párrafo en el diseño de la
página?
```

Reto 4: Corrija los errores de indentación del siguiente script:

En este juego, el personaje sigue avanzando sin que, aparentemente, nunca termine la partida. Debe implementar una condición (regla de juego) para que la partida acabe cuando el jugador llegue al final del tablero (grid).

```
# Juego en el que el jugador X avanza por el grid cada vez que pulsamos <Enter>
# Cuando el jugador llega al final de una línea, salta a la siguiente línea
# Juego termina cuando jugador llega al final del grid en la última pos de la última línea
from tabulate import tabulate
# Definir los datos de la tabla (grid del juego) con casillas vacías (6x6 en este caso)
table_data = [
    [" ", " ", " ", " ", " ", " "],
    [" ", " ", " ", " ", " ", " "],
    [" ", " ", " ", " ", " ", " "],
    [" ", " ", " ", " ", " ", " "],
    [" ", " ", " ", " ", " ", " "],
    [" ", " ", " ", " ", " ", " "],
    [" ", " ", " ", " ", " ", " "]
]
# Definir los encabezados de la tabla (6 columnas)
headers = ["column 1", "column 2", "column 3", "column 4", "column 5", "column 6"]

# Inicializar la posición de la "X"
x_pos = [0, 0] # Comienza en la primera fila y primera columna

def move_x_line():
    global x_pos
    # Limpiar la posición anterior de la "X"
    table_data[x_pos[0]][x_pos[1]] = " "

    # Mover la "X" de izquierda a derecha y de arriba hacia abajo
    if x_pos[1] < len(table_data[0]) - 1: # Si no estamos en la última columna
        x_pos[1] += 1
    else: # Si estamos en la última columna, mover a la siguiente fila
        x_pos[1] = 0
        if x_pos[0] < len(table_data) - 1: # Si no estamos en la última fila
            x_pos[0] += 1
        else: # Si estamos en la última fila, reiniciar a la primera fila
            x_pos[0] = 0

    table_data[x_pos[0]][x_pos[1]] = "X" # Colocar "X" en la nueva posición

def print_table():
    table = tabulate(table_data, headers, tablefmt="fancy_grid"); print(table)

# Main program. Programa principal
empezamos=0
# Repetir el proceso de mover la "X" de manera secuencial cada vez que presionen Enter
while True:
    if empezamos==0: # Esta parte se ejecuta al iniciar el juego, cuando empezamos=0
        x_pos = [0, 0] # Comienza en la primera fila y primera columna
        ## Colocar la "X" en la nueva posición
        table_data[x_pos[0]][x_pos[0]] = "X"
        print_table()
        print("Pulsa <Enter> para mover la X..."); input() # Esperar q user presione
        Enter

        ## Movemos la "X" y generamos la tabla:
        move_x_line(); print_table()
        empezamos=empezamos+1 ### ya hemos empezado a jugar
    else: ### Esta parte se ejecuta cuando var. empezamos ya no es 0
        print("Pulsa <Enter> para mover la X..."); input()

        ## Movemos la "X" y generamos la tabla:
        move_x_line(); print_table() ###Recuerde: la indentación es importante
```

Ejemplo 40. Creación de un archivo compatible con Microsoft PowerPoint.

Objetivo: Utilizar la librería **python-pptx** y generar una presentación PowerPoint con un texto centrado en una diapositiva.

Una presentación de PowerPoint es un archivo digital creado con el software Microsoft PowerPoint. Este programa, que es muy utilizado en el mundo empresarial para hacer presentaciones de productos, permite diseñar presentaciones combinando gráficos, imágenes, videos, texto y animaciones en diapositivas.

Cree el archivo: “**powerpoint1.py**” y añada las siguientes órdenes:

```
#
# No olvide instalar la librería pptx en su PC:
#
# pip install python-pptx
#
#

from pptx import Presentation

# Crear una nueva presentacion
prs = Presentation()

# Añadir una diapositiva (slide)
slide = prs.slides.add_slide(prs.slide_layouts[0])

# Añadir el título a la diapositiva
title = slide.shapes.title
title.text = "Hacer una presentación es fácil con Python"

# Grabar el archivo pptx
prs.save("PowerPoint1.pptx")

print("")
print(" El archivo < PowerPoint1.pptx > ha sido creado.")
print(" ")
```

En el ejemplo mostramos cómo generar una presentación compatible con Microsoft PowerPoint con sólo una diapositiva.

El archivo creado en su directorio de trabajo se generó con la función:

<pre>prs.save("nombre-de-archivo.pptx")</pre>

Este es el resultado del ejemplo 40:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\powerpoint1.py =====

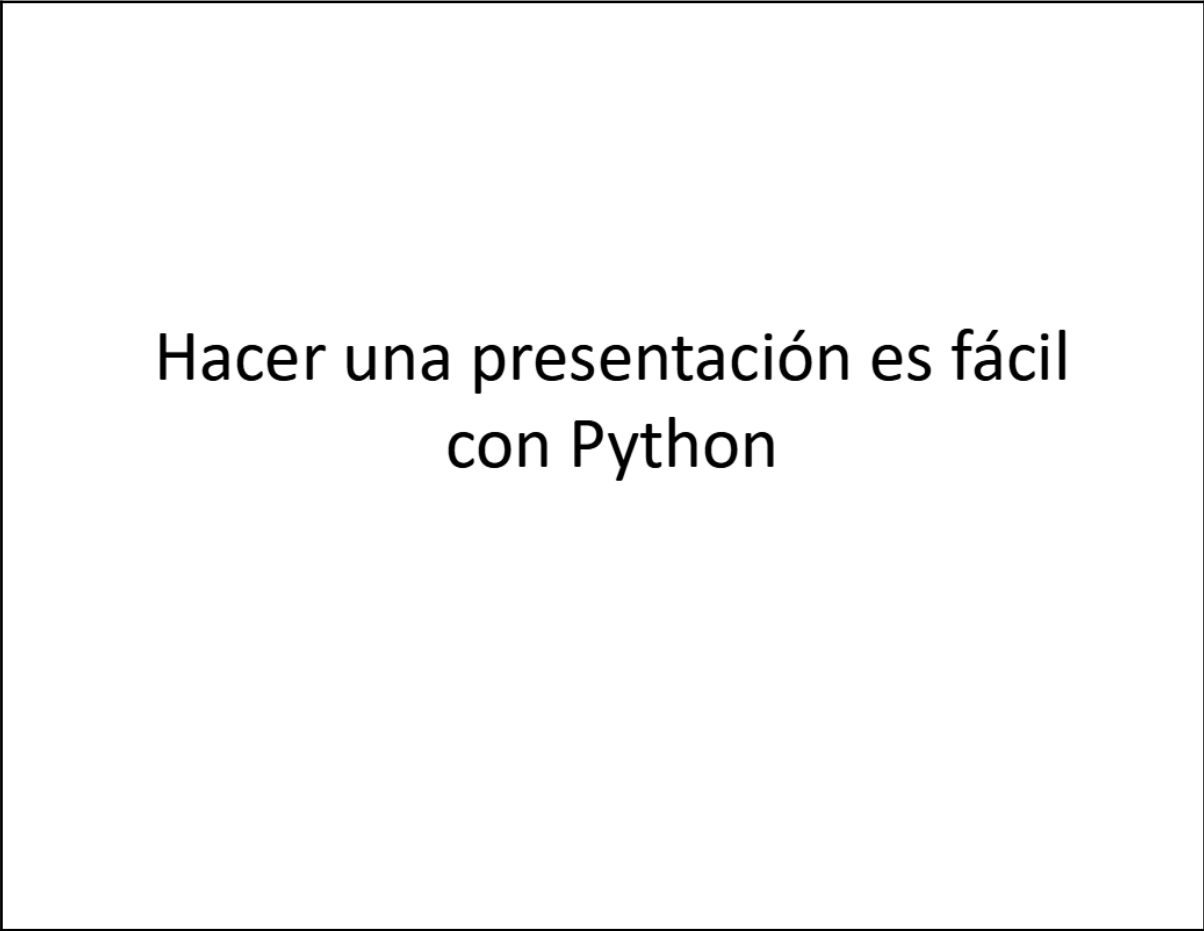
  El archivo < PowerPoint1.pptx > ha sido creado.

>>>
```

Como puede observar, únicamente se ha impreso en la terminal un mensaje (en azul) indicando que el archivo PowerPoint.pptx ha sido creado. Para comprobar que se ha creado correctamente el archivo, vaya a su directorio de trabajo y abra el archivo generado con extensión .pptx con el programa gratuito LibreOffice o descargue el visor de archivos .pptx de la web de Microsoft.

Otra opción para ver el archivo creado, llamado “PowerPoint.pptx”, es utilizar Google Presentación (*Google Slides* en inglés). Si tiene una cuenta Gmail, puede subir y ver sus archivos .pptx directamente en la web de Google con su PC o con su tablet. Es una opción muy fácil de usar si prefiere ver el resultado de su trabajo en línea.

Este es el resultado de visualizar el ejemplo 40 con Google Slides:

A screenshot of a Google Slides presentation slide. The slide has a white background and a black border. The text is centered and reads: "Hacer una presentación es fácil con Python".

Hacer una presentación es fácil
con Python

Fin del Ejemplo 40.

Ejemplo 41. Creación de un archivo compatible con Microsoft PowerPoint con dos diapositivas.

Objetivo: Utilizar la librería **python-pptx** y generar una presentación PowerPoint con dos diapositivas y los textos centrados en cada diapositiva.

Cree el archivo: “powerpoint11.py” y añada las siguientes órdenes:

```
from pptx import Presentation

# Crear una nueva presentacion
prs = Presentation()

# Añadir una diapositiva (slide)
slide = prs.slides.add_slide(prs.slide_layouts[0])

# Añadir el título a la diapositiva
title = slide.shapes.title
title.text = "Hacer una presentación es fácil con Python"

## Añadir la segunda diapositiva (slide)
slide = prs.slides.add_slide(prs.slide_layouts[0])

## Añadir el título a la segunda diapositiva
title = slide.shapes.title
title.text = "Hacer una presentación es divertido con Python"

## Grabar el archivo pptx
prs.save("PowerPoint11.pptx")

print("")
print(" El archivo < PowerPoint11.pptx > ha sido creado.")
print(" ")
```

(la explicación continúa en la siguiente página)

En el ejemplo mostramos cómo generar una presentación compatible con Microsoft PowerPoint con 2 diapositivas. Únicamente se han añadido las siguientes líneas para generar la segunda diapositiva:

```
## Añadir la segunda diapositiva (slide)
slide = prs.slides.add_slide(prs.slide_layouts[0])

## Añadir el título a la segunda diapositiva
title = slide.shapes.title
title.text = "Hacer una presentación es divertido con
Python"
```

El programador podría seguir añadiendo una tercera, cuarta, quinta, etc. diapositiva a la presentación siguiendo esta estructura para crear una historia, un cuento, o la presentación de un producto.

Este es el resultado del ejemplo 41:



```
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ACER/Documents/powerpoint11.py =====

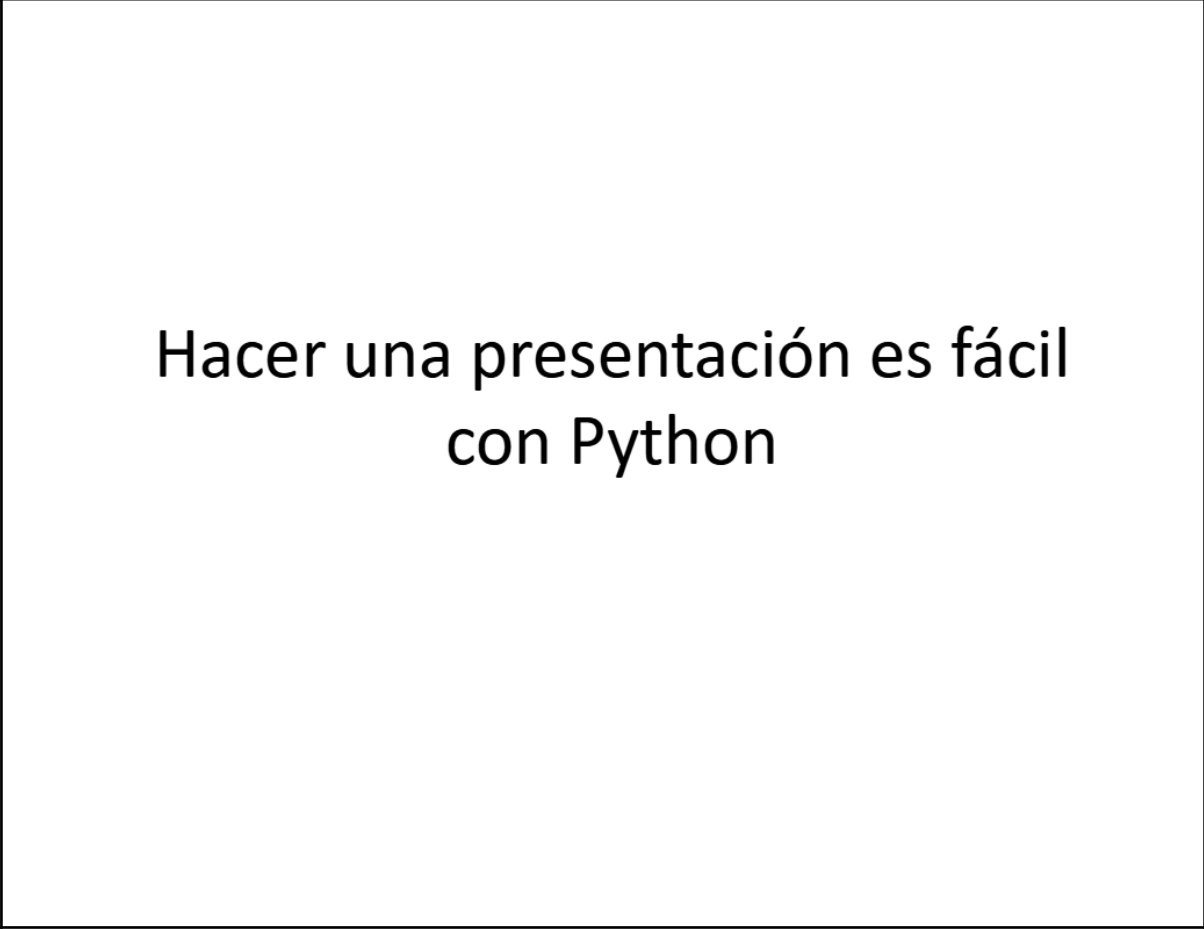
El archivo < PowerPoint11.pptx > ha sido creado.

>>> |
```

Únicamente se ha impreso en la terminal (en azul) que el archivo PowerPoint.pptx ha sido creado. Para comprobar que se ha creado correctamente el archivo, vaya a su directorio de trabajo y abra el archivo generado con extensión .pptx con el programa gratuito LibreOffice.

Este es el resultado de visualizar el ejemplo 41 con Google Slides:

Primera diapositiva:



Hacer una presentación es fácil
con Python

Segunda diapositiva:



Hacer una presentación es
divertido con Python

Recuerde que para pasar a la segunda diapositiva utilizando el programa Microsoft PowerPoint es necesario pulsar la tecla <flecha derecha> y para retroceder a la diapositiva anterior es necesario pulsar <flecha izquierda>.

Reto 1:

Cree una historia con 6 diapositivas. Incluya, como mínimo, 2 frases en cada diapositiva. Grabe el archivo generado como **historia2025.pptx**.

```
from pptx import Presentation
prs = Presentation()          ### Crear presentation

# Define the sentencesE (sentences in English)
sentencesE = [
    "Microsoft sells millions of Office packages every year. Millions of people use Microsoft Office every day. Millions of users complain about the price of Microsoft Office.",
    "Microsoft Office is expensive. Apple sells a competing product called iWork.",
    "Apple sells millions of iWork packages every year",
    "Would you pay 3 dollars per month for iWork?",
    "Would you pay 5 dollars per month for Microsoft Office online?",
    "Consider switching to Google Docs, LibreOffice or OpenOffice"
]

# Define the sentencesS (sentences in Spanish- FRASES EN ESPAÑOL)
sentencesS= [
    "Microsoft vende millones de paquetes de Office cada año. Millones de personas usan Microsoft Office todos los días. Millones de usuarios se quejan del precio de Microsoft Office.",
    "Microsoft Office es caro. Apple vende un producto competidor llamado iWork.",
    "Apple vende millones de paquetes de iWork cada año.",
    "¿Pagarías 3 dólares al mes por el paquete iWork?",
    "¿Pagarías 5 dólares al mes por Microsoft Office en línea?",
    "Considera cambiar a Google Docs, LibreOffice o OpenOffice."
]

# Add a slide with the first sentence, element zero of the array
slide1 = prs.slides.add_slide(prs.slide_layouts[0])
# Add the first sentence as the title
title1 = slide1.shapes.title
title1.text = sentencesE[0]

# Add a slide with the second sentence, element 1
slide1 = prs.slides.add_slide(prs.slide_layouts[0])
# Add the sentence as the title
title1 = slide1.shapes.title
title1.text = sentencesE[1]

slide1 = prs.slides.add_slide(prs.slide_layouts[0])
title1 = slide1.shapes.title
title1.text = sentencesE[2]

slide1 = prs.slides.add_slide(prs.slide_layouts[0])
title1 = slide1.shapes.title; title1.text = sentencesE[3]

slide1 = prs.slides.add_slide(prs.slide_layouts[0])
title1 = slide1.shapes.title; title1.text = sentencesE[4]

slide1 = prs.slides.add_slide(prs.slide_layouts[0])
title1 = slide1.shapes.title; title1.text = sentencesE[5]

prs.save("presentation123.pptx")      ## Save the presentation

print(" "); print("Presentation created successfully!"); print(" ")
```

Reto 2:

Optimice el script anterior y añada un bucle para crear el archivo con extensión .pptx con las diapositivas de la presentación en Español.

```
from pptx import Presentation

prs = Presentation()          ### Crear presentation

# Define sentencesE (sentences in English)
sentencesE = [
    "Microsoft sells millions of Office packages every year. Millions of people use Microsoft Office every day. Millions of users complain about the price of Microsoft Office.",
    "Microsoft Office is expensive. Apple sells a competing product called iWork.",
    "Apple sells millions of iWork packages every year",
    "Would you pay 3 dollars per month for iWork?",
    "Would you pay 5 dollars per month for Microsoft Office online?",
    "Consider switching to Google Docs, LibreOffice or OpenOffice in order to save money."
]

# Define sentencesS (sentences in Spanish- FRASES EN ESPAÑOL)
sentencesS= [
    "Microsoft vende millones de paquetes de Office cada año. Millones de personas usan Microsoft Office todos los días. Millones de usuarios se quejan del precio de Microsoft Office.",
    "Microsoft Office es caro. Apple vende un producto competidor llamado iWork.",
    "Apple vende millones de paquetes de iWork cada año.",
    "¿Pagarías 3 dólares al mes por el paquete iWork?",
    "¿Pagarías 5 dólares al mes por Microsoft Office en línea?",
    "Considera cambiar a Google Docs, LibreOffice o OpenOffice."
]

i=0
while i<len(sentencesS):      ## Add slides with sentences (elements of the array)
    slidel = prs.slides.add_slide(prs.slide_layouts[0])    ## layout 0 (slide with title)
    titlel = slidel.shapes.title                            # Add sentence as title
    titlel.text = sentencesS[i]                             # Frases en Español (Spanish)
    i=i+1

prs.save("presentation123.pptx")    ## Save the presentation

print(" "); print("Presentation created successfully!"); print(" ")
```

Reto 3:

Optimice el script anterior y añada una pregunta para que el usuario decida si quiere crear el archivo con extensión .pptx con las diapositivas de la presentación en Español o en Inglés.

```
from pptx import Presentation

prs = Presentation()          ### Crear presentation

# Define the sentencesE (sentences in English)
sentencesE = [
    "Microsoft sells millions of Office packages every year. Millions of people use Microsoft Office every day. Millions of users complain about the price of Microsoft Office.",
    "Microsoft Office is expensive. Apple sells a competing product called iWork.",
    "Apple sells millions of iWork packages every year",
    "Would you pay 3 dollars per month for iWork?",
    "Would you pay 5 dollars per month for Microsoft Office online?",
    "Consider switching to Google Docs, LibreOffice or OpenOffice in order to save money."
]

# Define the sentencesS (sentences in Spanish- FRASES EN ESPAÑOL)
sentencesS = [
    "Microsoft vende millones de paquetes de Office cada año. Millones de personas usan Microsoft Office todos los días. Millones de usuarios se quejan del precio de Microsoft Office.",
    "Microsoft Office es caro. Apple vende un producto competidor llamado iWork.",
    "Apple vende millones de paquetes de iWork cada año.",
    "¿Pagarías 3 dólares al mes por el paquete iWork?",
    "¿Pagarías 5 dólares al mes por Microsoft Office en línea?",
    "Considera cambiar a Google Docs, LibreOffice o OpenOffice."
]

idioma=input("Elija el idioma que se debe utilizar para crear la presentación (E para English / S para Spanish): ")
if idioma=="S":
    sentencesE=sentencesS
#elseif

i=0
while i<len(sentencesE):
    # Add a slide with the first sentence, element zero of the array
    slidel = prs.slides.add_slide(prs.slide_layouts[0])    ## layout 0 (slide with title and subtitle)
    # Add the first sentence as the title
    title1 = slidel.shapes.title
    title1.text = sentencesE[i]
    i=i+1

prs.save("presentation123.pptx")          ## Save the presentation

print(" "); print("Presentation created successfully!"); print(" ")
```

Ejemplo 42. Creación de un archivo compatible con Microsoft Excel.

Objetivo: Utilizar la librería `openpyxl` y generar un archivo compatible con Microsoft Excel para poder realizar cálculos.

Cree el archivo: `excel111.py` y añada las siguientes órdenes:

```
#
# creación de un archivo xlsx en blanco (para
# hacer cálculos o para crear una base de datos)
# (creando una hoja de cálculo -sheet- en un libro -w

from openpyxl import Workbook

def create_sheets(path):
    workbook = Workbook()
    sheet = workbook.active
    sheet.title = "HolaDatabase1"
    ##para crear una segunda hoja de cálculo (sheet)
    ##sheet2 = workbook.create_sheet(title="World2")
    workbook.save(path)

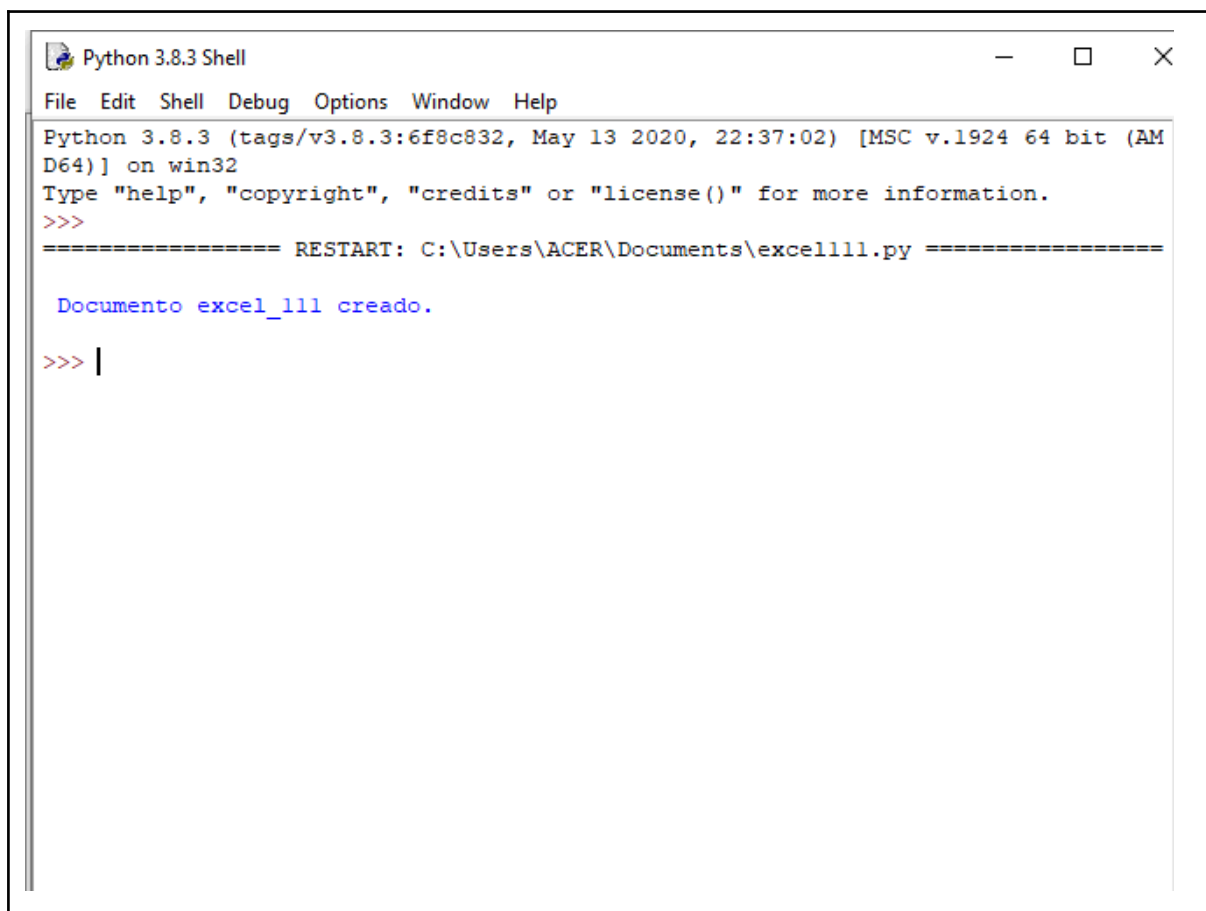
create_sheets("excel_111.xlsx")
print(" ")
print(" Documento excel_111 creado. ")
print(" ")
```

(la explicación continúa en la siguiente página)

En el ejemplo mostramos cómo generar una hoja de cálculo (spreadsheet) compatible con Microsoft Excel.

El programador podría seguir añadiendo una tercera, cuarta, quinta, etc. hoja de cálculo al documento siguiendo la estructura (marcada en rojo) para crear una hoja de cálculo más compleja.

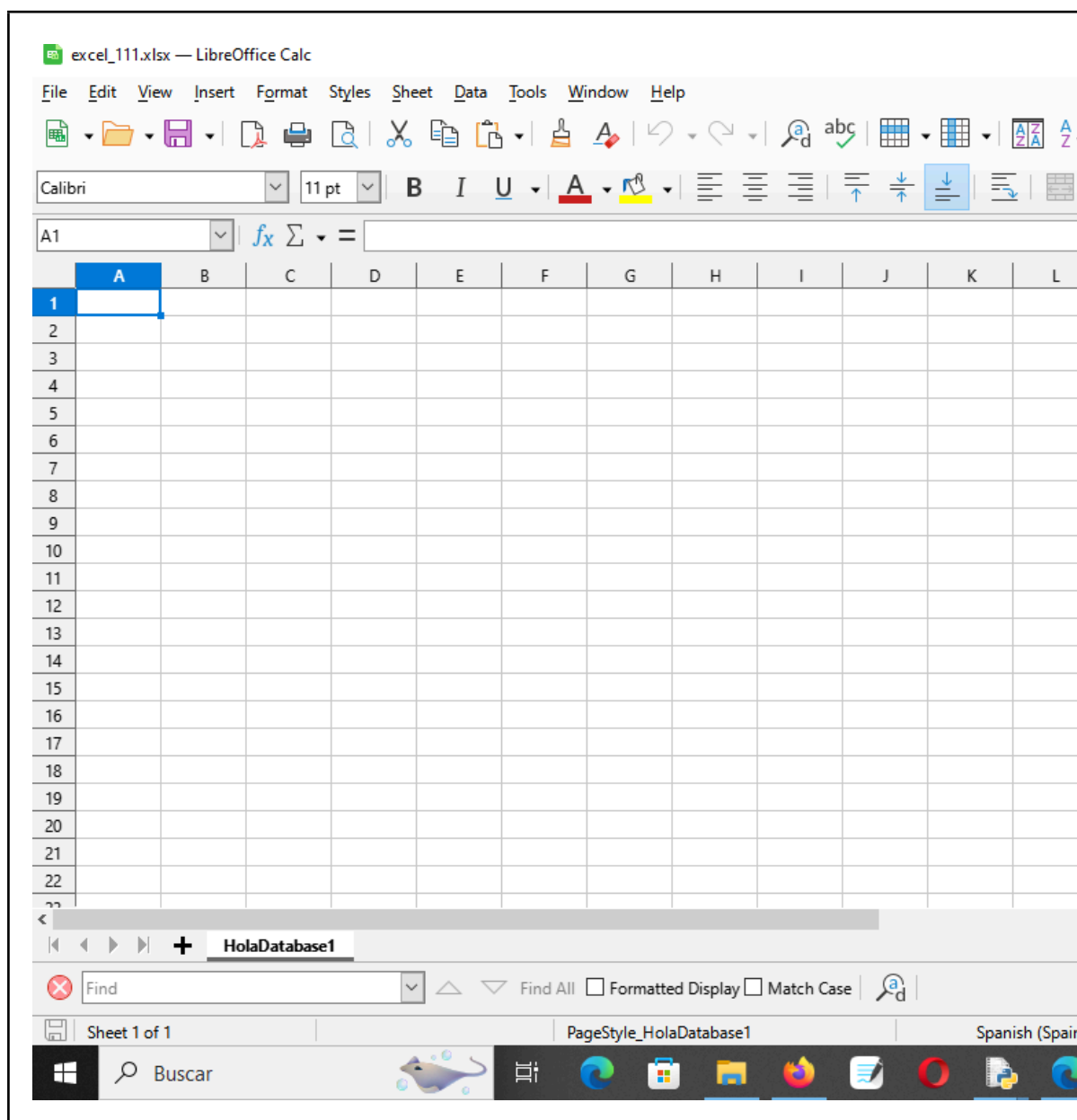
Este es el resultado del ejemplo 42:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\excel111.py =====
Documento excel_111 creado.
>>> |
```


Únicamente se ha impreso en la terminal (en azul) un mensaje indicando que el archivo **excel_111.xlsx** ha sido creado. Para comprobar que se ha creado correctamente el archivo, vaya a su directorio de trabajo y abra el archivo generado con extensión .xlsx con el programa gratuito LibreOffice.

Este es el resultado de abrir el archivo .xlsx del ejemplo 42 con LibreOffice:

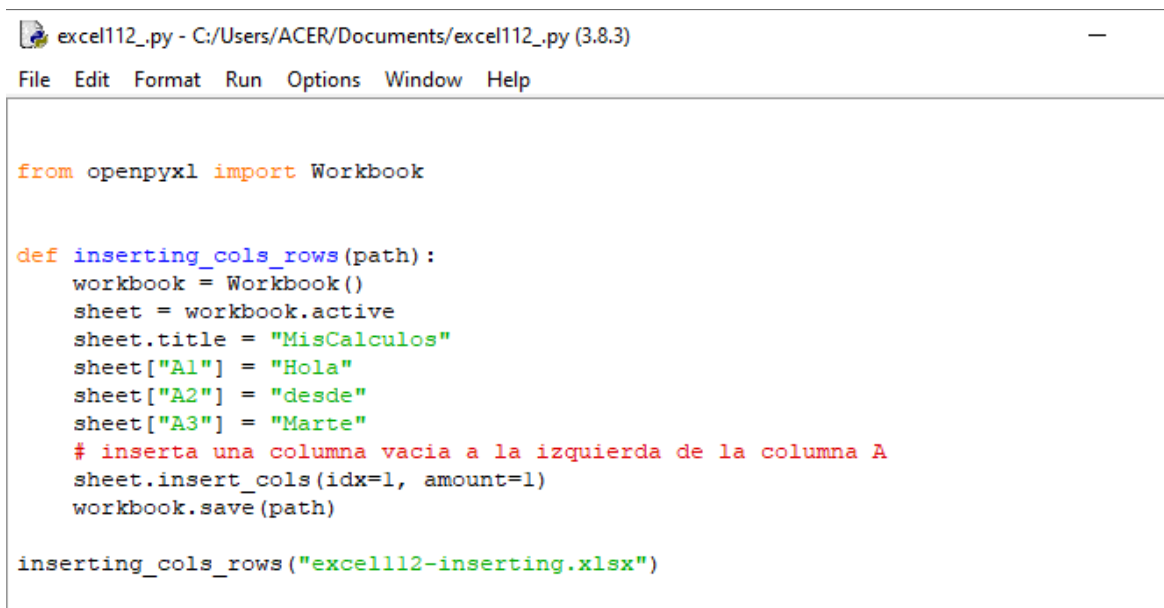


Fin del Ejemplo 42.

Ejemplo 43. Creación de un archivo con datos compatible con Microsoft Excel.

Objetivo: Utilizar la librería `openpyxl` y generar un archivo con datos compatible con Microsoft Excel.

Cree el archivo: `excel112.py` y añada las siguientes órdenes:



```
excel112.py - C:/Users/ACER/Documents/excel112_.py (3.8.3)
File Edit Format Run Options Window Help

from openpyxl import Workbook

def inserting_cols_rows(path):
    workbook = Workbook()
    sheet = workbook.active
    sheet.title = "MisCalculos"
    sheet["A1"] = "Hola"
    sheet["A2"] = "desde"
    sheet["A3"] = "Marte"
    # inserta una columna vacia a la izquierda de la columna A
    sheet.insert_cols(idx=1, amount=1)
    workbook.save(path)

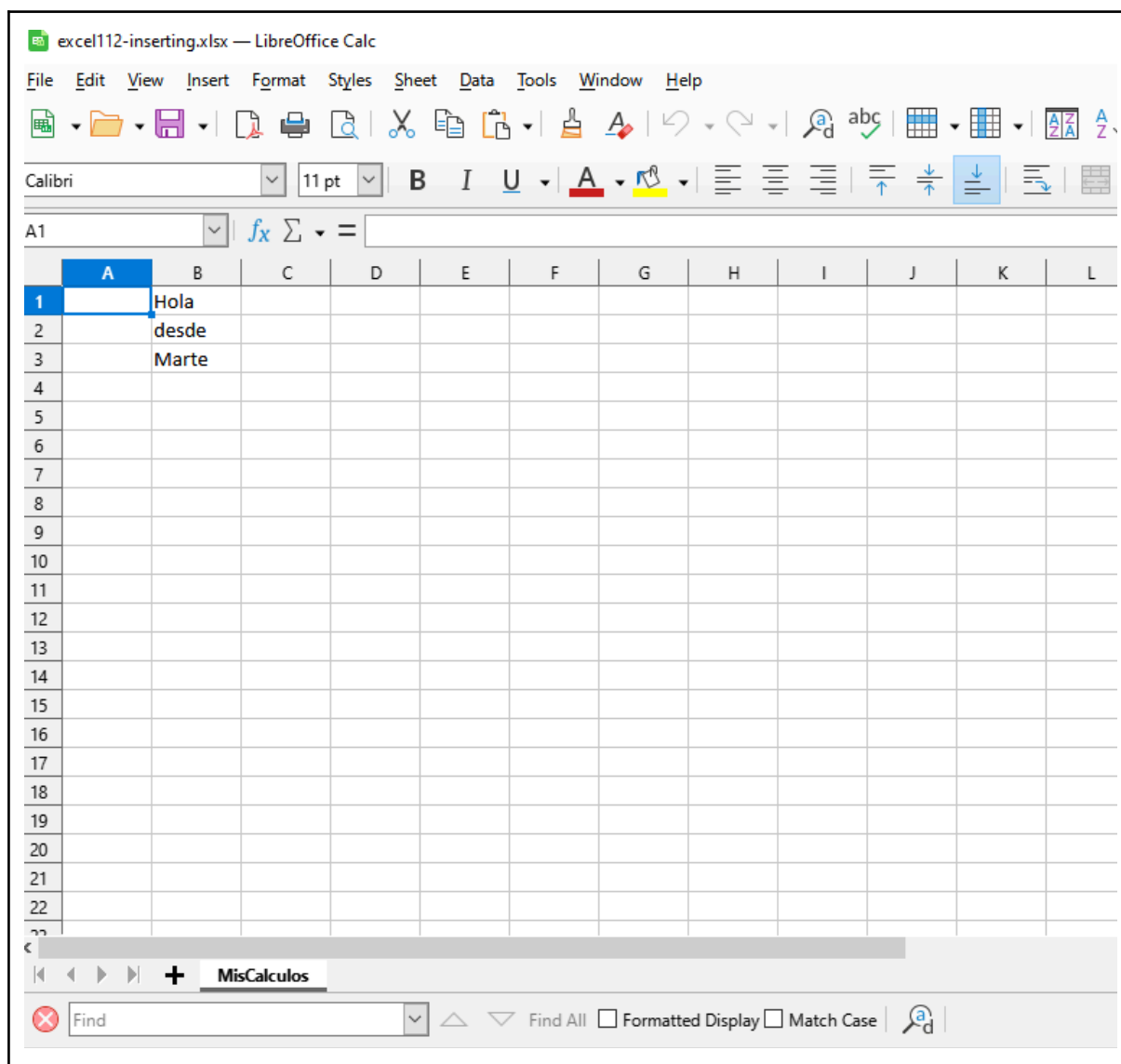
inserting_cols_rows("excel112-inserting.xlsx")
```

(la explicación continúa en la siguiente página)

En el ejemplo mostramos cómo generar una hoja de cálculo (spreadsheet) con datos compatible con Microsoft Excel.

Únicamente se ha impreso en la terminal (en azul) que el archivo **excel_112.xlsx** ha sido creado. Para comprobar que se ha creado correctamente el archivo, vaya a su directorio de trabajo y abra el archivo generado con extensión .xlsx con el programa gratuito LibreOffice.

Este es el resultado de abrir el archivo .xlsx del ejemplo 43 con LibreOffice:

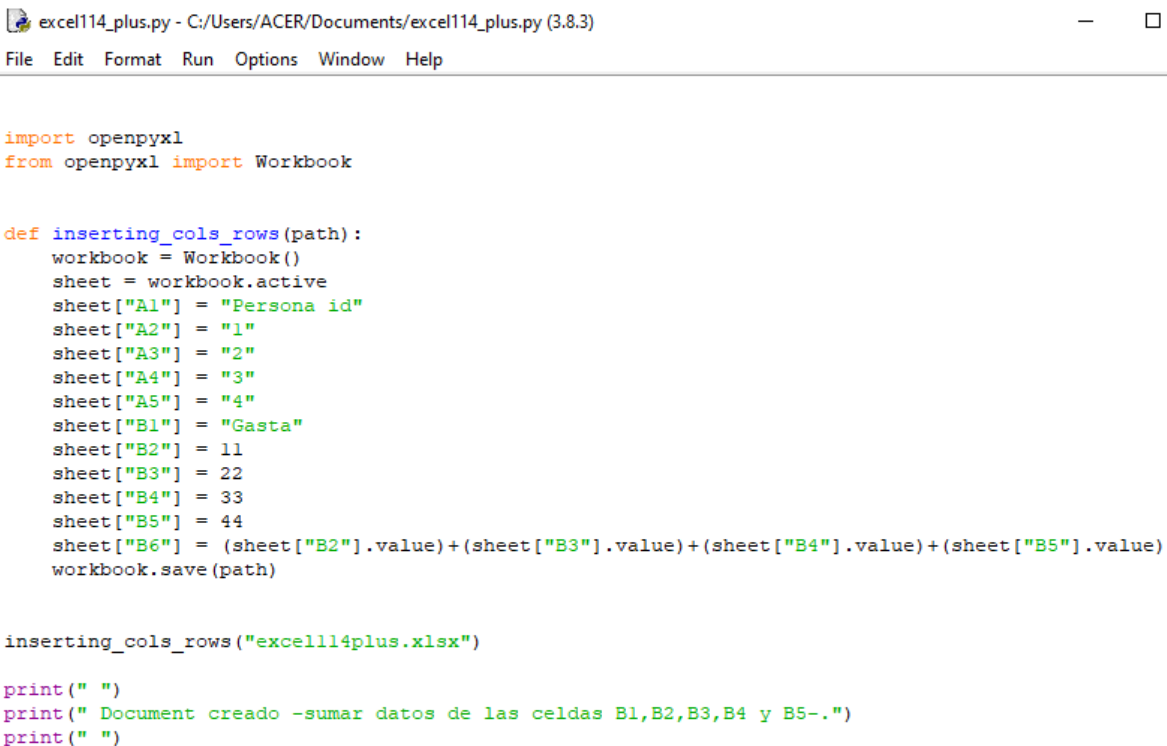


Fin del Ejemplo 43.

Ejemplo 44. Creación de un archivo con datos compatible con Microsoft Excel para realizar la suma del contenido de varias celdas.

Objetivo: Utilizar la librería **openpyxl** y generar un archivo con datos compatible con Microsoft Excel para realizar sumas de datos.

Cree el archivo: “**excel114plus.py**” y añada las siguientes órdenes:



```

import openpyxl
from openpyxl import Workbook

def inserting_cols_rows(path):
    workbook = Workbook()
    sheet = workbook.active
    sheet["A1"] = "Persona id"
    sheet["A2"] = "1"
    sheet["A3"] = "2"
    sheet["A4"] = "3"
    sheet["A5"] = "4"
    sheet["B1"] = "Gasta"
    sheet["B2"] = 11
    sheet["B3"] = 22
    sheet["B4"] = 33
    sheet["B5"] = 44
    sheet["B6"] = (sheet["B2"].value)+(sheet["B3"].value)+(sheet["B4"].value)+(sheet["B5"].value)
    workbook.save(path)

inserting_cols_rows("excel114plus.xlsx")

print(" ")
print(" Document creado -sumar datos de las celdas B1,B2,B3,B4 y B5-.")
print(" ")

```

En este ejemplo mostramos cómo generar una hoja de cálculo (spreadsheet) donde se suman en la celda B6 los valores de las celdas B2, B3, B4 y B5.

(la explicación continúa en la siguiente página)

Este es el resultado de abrir el archivo .xlsx del ejemplo 44 con LibreOffice:

	A	B	C	D	E	F	G	H	I	J	K
1	Persona id	Gasta									
2	1	11									
3	2	22									
4	3	33									
5	4	44									
6		110									
7											
8											
9											
10											
11											
12											
13											
14											
15											
16											
17											
18											
19											
20											
21											
22											
23											

Observe que el contenido de las celdas A1 y B1 son textos mientras que el contenido del resto de celdas son números.

Reto:

Modifique el script anterior: Reemplace el contenido de la celda B1 por “Gasto mensual”. Reemplace el contenido de la celda C1 por “Gasto anual”. Calcule el gasto anual para cada persona de la lista.

Ejemplo 45. Creación de un archivo con datos de una lista de contactos compatible con Microsoft Excel.

Objetivo: Utilizar la librería `openpyxl` y crear una lista de contactos en un archivo compatible con Microsoft Excel.

Cree el archivo: “`excel123.py`” y añada las siguientes órdenes:

```
from openpyxl import Workbook

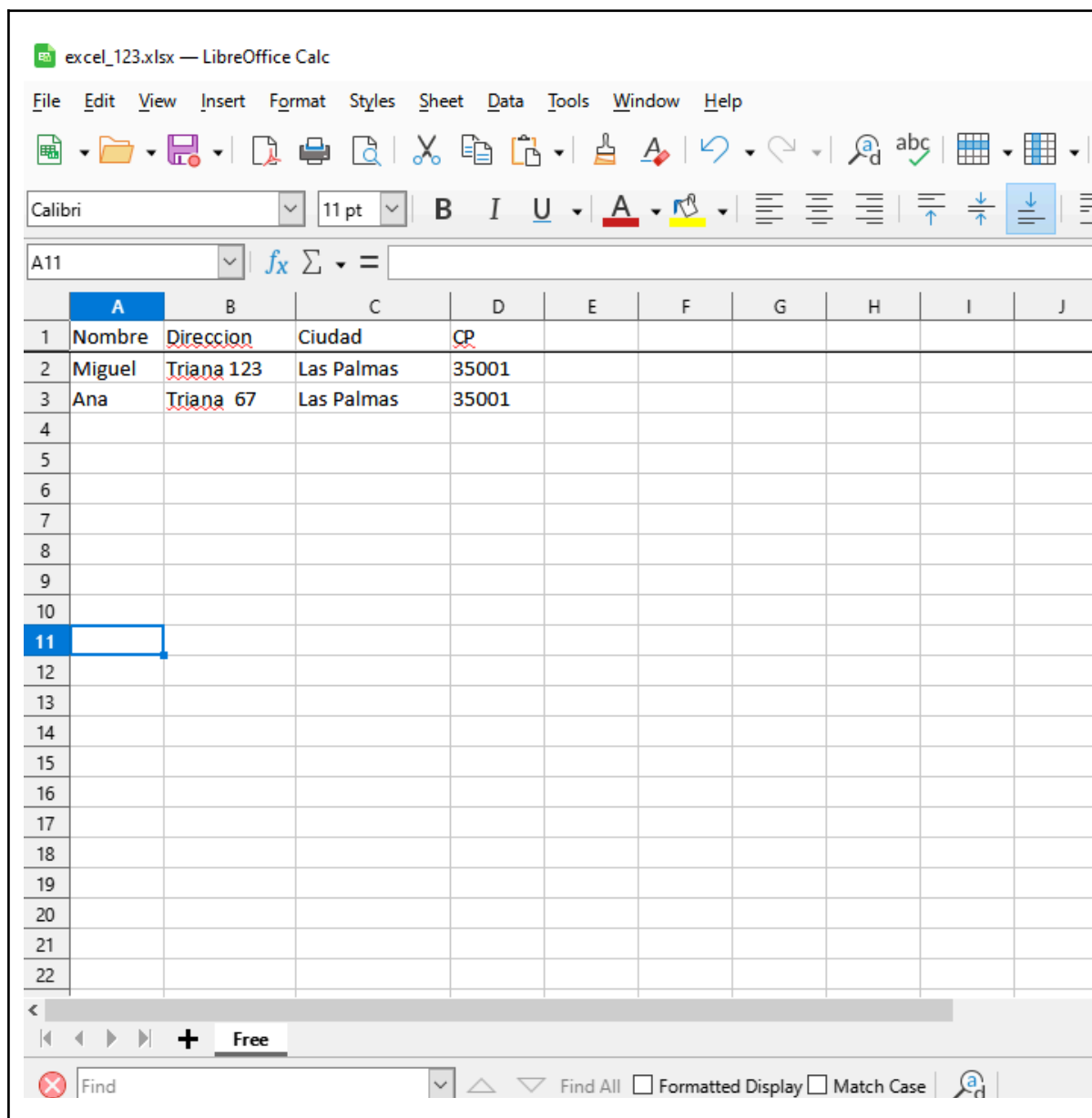
def freeze(path, row_to_freeze):
    workbook = Workbook()
    sheet = workbook.active
    sheet.title = "Free"
    sheet.freeze_panes = row_to_freeze
    headers = ["Nombre", "Direccion", "Ciudad", "CP"]
    sheet["A1"] = headers[0]
    sheet["B1"] = headers[1]
    sheet["C1"] = headers[2]
    sheet["D1"] = headers[3]
    data = [dict(zip(headers, ("Miguel", "Triana 123", "Las Palmas", "35001"))),
            dict(zip(headers, ("Ana", "Triana 67", "Las Palmas", "35001")))]
    row = 2
    for d in data:
        sheet[f'A{row}'] = d["Nombre"]
        sheet[f'B{row}'] = d["Direccion"]
        sheet[f'C{row}'] = d["Ciudad"]
        sheet[f'D{row}'] = d["CP"]
        row += 1
    workbook.save(path)

freeze("excel_123.xlsx", row_to_freeze="A2")
print(" ")
```

En el ejemplo mostramos cómo generar una hoja de cálculo (spreadsheet) con una lista de contactos.

(la explicación continúa en la siguiente página)

Este es el resultado de abrir el archivo .xlsx del ejemplo 45 con LibreOffice:



Observe que el contenido de la hoja a partir de la celda A2 queda libre mientras que se congelan los encabezados con la función freeze. Es decir, si se añaden más de 100 contactos, éstos se desplazarán hacia arriba o abajo pero el encabezado (contenido de la línea 1) quedará congelado sin moverse.

Si necesitamos añadir más contactos, lo haremos en la siguiente línea de código:

```
data = [dict(zip(headers, ("Miguel", "Triana 123", "Las Palmas",
"35001"))),
        dict(zip(headers, ("Ana ", "Triana 67", "Las Palmas", "35001")))]
```

Cada contacto se deberá incluir entre comillas en este orden: nombre, dirección, ciudad y código postal.

Ejemplo: Si queremos añadir a Juan, deberemos hacerlo de la siguiente forma:

```
data = [dict(zip(headers, ("Miguel", "Triana 123", "Las Palmas",
"35001"))),
        dict(zip(headers, ("Ana ", "Triana 67", "Las Palmas", "35001"))),
        dict(zip(headers, ("Juan ", "Triana 68", "Las Palmas", "35001")))]
```

Además, como el número de contactos ha cambiado, la siguiente línea debe incluir el número 3, pues ahora tenemos 3 contactos en la lista.

```
row= 3
```

Fin del Ejemplo 45.

Ejemplo 46. Creación de una tabla con dos filas y dos columnas con nombres de pueblos de las Islas Canarias.

Objetivo: Utilizar la librería **tabulate** y crear una lista de palabras en una tabla 2 x 2 (dos filas y dos columnas).

Cree el archivo: “**hello-tabulate1.py**” y añada las siguientes órdenes:

```
print(" ")
print(" Hello modulo tabulate!")
print(" ")
from tabulate import tabulate

# ejemplo1 con el modulo tabulate
# Definir table data (recuerda: pip install tabulate)
table_data = [
    ["Artenara", "Betancuria"],
    ["Candelaria", "Damaso"]
]

# Definir table headers (encabezados)
headers = ["Column 1", "Column 2"]

# Crear table
table = tabulate(table_data, headers, tablefmt="fancy_grid")

# Print result:
print(table)
```

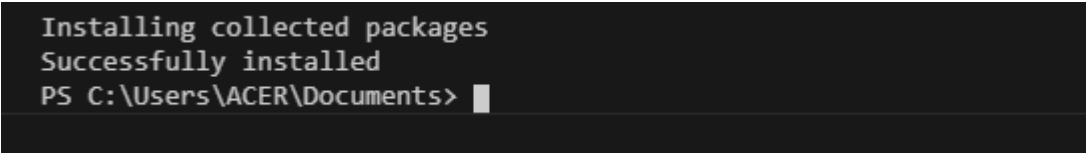
Los formatos más comúnmente utilizados son: **plain**, **simple**, **grid**, **psql** y **fancy_grid**, cada uno con características específicas en cuanto a delimitación, estructura y estética, que facilitan su integración en flujos de trabajo automatizados, documentación técnica o interfaces de usuario.

Antes de ejecutar el código deberá instalar la librería **tabulate**. Esta librería debe descargarse e instalarse en su sistema pues no es parte del paquete estándar Python 3.

En la ventana “terminal” en su ordenador teclee y ejecute la siguiente orden:

pip install tabulate

Después de ejecutar la orden, verá una pantalla con este resultado o similar. Significa que la librería **tabulate** se ha instalado con éxito:

A screenshot of a Windows command prompt terminal window. The background is dark, and the text is light gray. It shows the output of a pip install command: 'Installing collected packages' followed by 'Successfully installed' on the next line. The prompt 'PS C:\Users\ACER\Documents>' is visible at the bottom with a cursor.

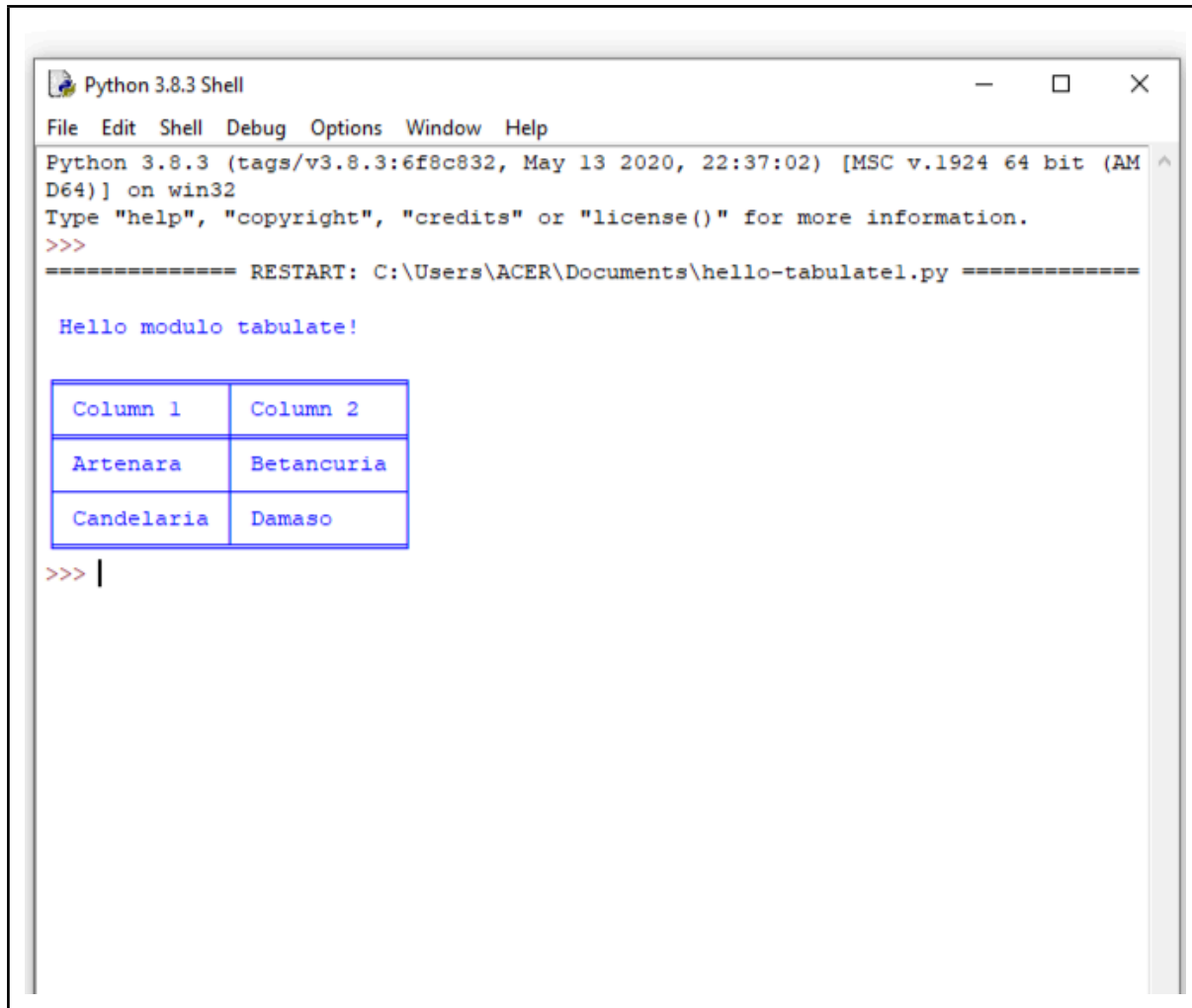
```
Installing collected packages  
Successfully installed  
PS C:\Users\ACER\Documents> █
```

Importante:

El comando **pip install** se utiliza en la terminal de Windows para instalar paquetes de Python desde el repositorio de paquetes de Python (PyPI).

Además, este comando resolverá las dependencias de otros paquetes y las descargará si es necesario para el correcto funcionamiento de la librería **tabulate**.

Este es el resultado de ejecutar el código utilizando la librería **tabulate**:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\hello-tabulatel.py =====

Hello modulo tabulate!



| Column 1   | Column 2   |
|------------|------------|
| Artenara   | Betancuria |
| Candelaria | Damaso     |


>>> |
```

Reto:

Modifique el script anterior y reemplace la lista **table_data** por **data_lake** y reemplace la variable **table** por la variable **t**. Ejecute el código nuevamente y compruebe que funciona de manera correcta.

Ejemplo 47. Creación de una tabla con dos filas y dos columnas con números y palabras.

Objetivo: Utilizar la librería **tabulate** y crear una lista de palabras y números enteros en una tabla con tres filas y dos columnas.

Cree el archivo: “**hello-tabulate2numeros.py**” y añada este código:

```
print(" ")
print(" Creemos una tabla con cadenas alfanumericas:")
print(" ")
from tabulate import tabulate

# ejemplo2: Definir table data con cadenas alfanumericas (aunque sean numeros)
table_data = [
    ["1", "2"],
    ["3", "4"]
]

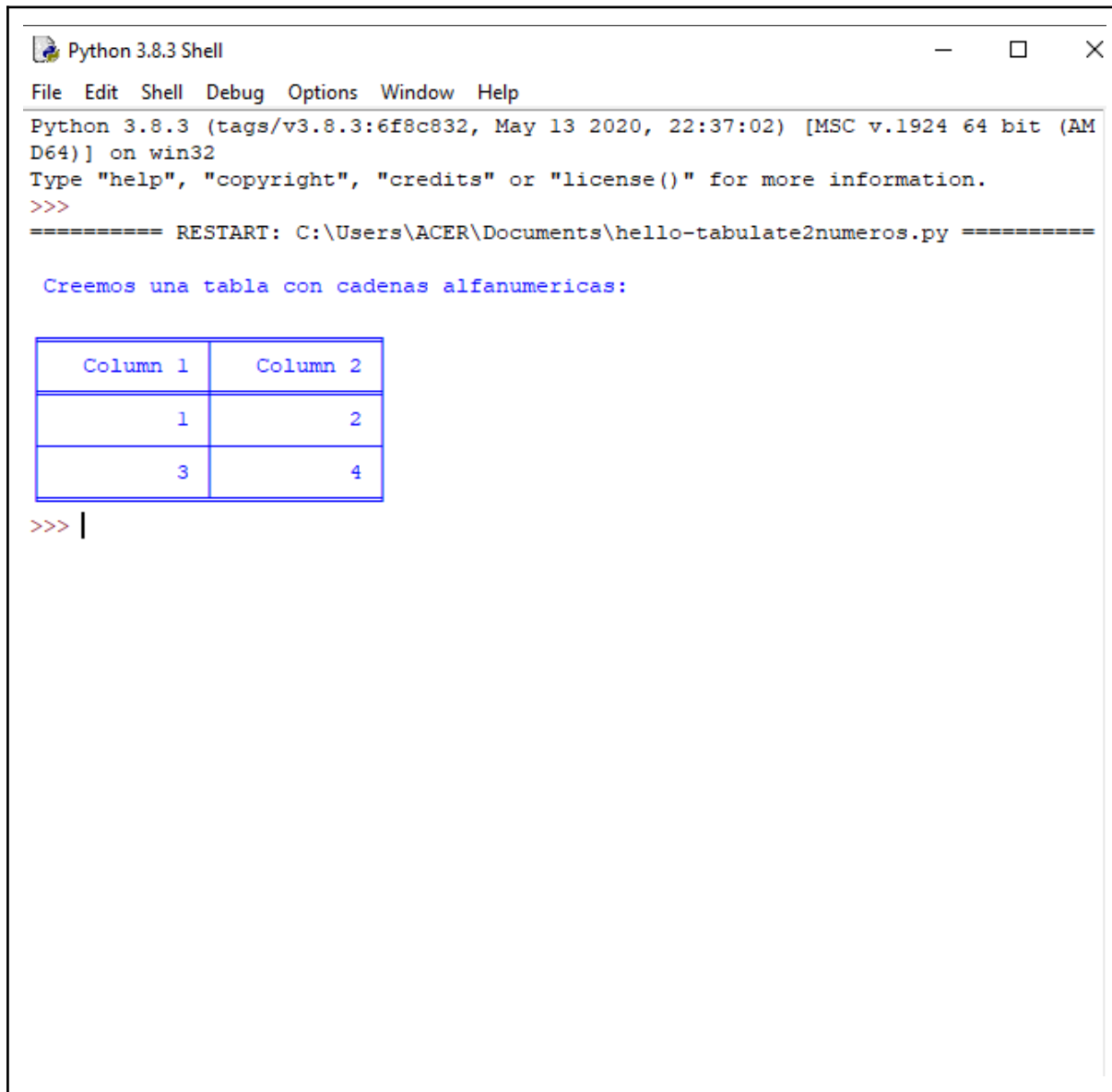
# Definir table headers (encabezados)
headers = ["Column 1", "Column 2"]

# Crear table
table = tabulate(table_data, headers, tablefmt="fancy_grid")

# Print result:
print(table)
```

Observe que la lista **table_data** contiene dos sublistas con números enteros (entre comillas). Recuerde que los números que están entre comillas se consideran cadenas alfanuméricas.

Este es el resultado de ejecutar el código utilizando cadenas alfanuméricas dentro de las sublistas creadas.



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\hello-tabulate2numeros.py =====

Creemos una tabla con cadenas alfanumericas:



| Column 1 | Column 2 |
|----------|----------|
| 1        | 2        |
| 3        | 4        |



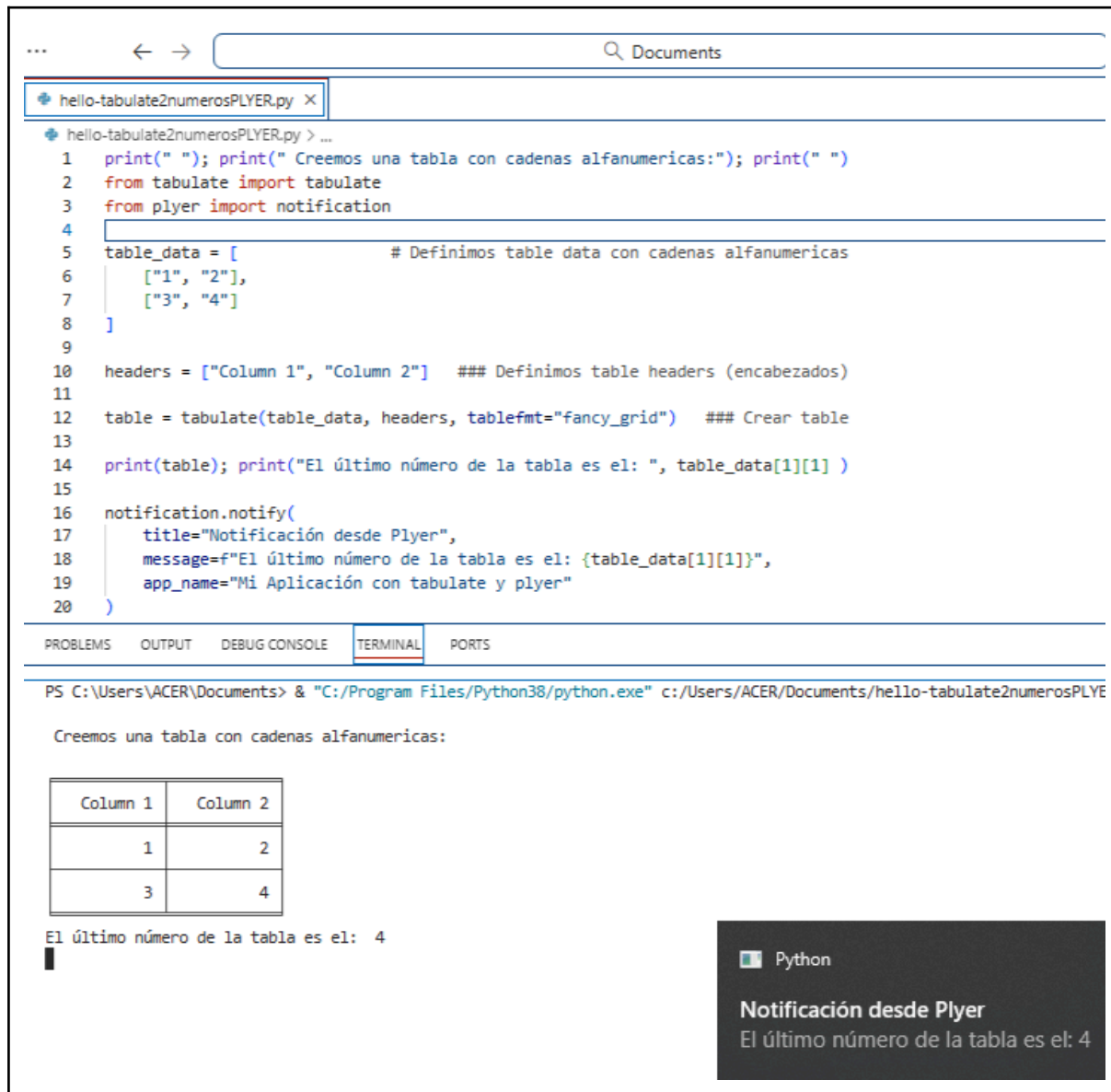
>>> |
```

Fin del Ejemplo 47.

Reto:

Cree una tabla con números o palabras. Utilice las librerías **plyer** y **tabulate** para crear la lista y una notificación en el sistema.

Cree el archivo: “**hello-tabulate2numerosPLYER.py**” y corrija los errores en el siguiente bloque de código:



```

...  ← →  Documents
hello-tabulate2numerosPLYER.py X
hello-tabulate2numerosPLYER.py > ...
1  print(" "); print(" Creemos una tabla con cadenas alfanumericas:"); print(" ")
2  from tabulate import tabulate
3  from plyer import notification
4
5  table_data = [          # Definimos table data con cadenas alfanumericas
6      ["1", "2"],
7      ["3", "4"]
8  ]
9
10 headers = ["Column 1", "Column 2"]  ### Definimos table headers (encabezados)
11
12 table = tabulate(table_data, headers, tablefmt="fancy_grid")  ### Crear table
13
14 print(table); print("El último número de la tabla es el: ", table_data[1][1] )
15
16 notification.notify(
17     title="Notificación desde Plyer",
18     message=f"El último número de la tabla es el: {table_data[1][1]}",
19     app_name="Mi Aplicación con tabulate y plyer"
20 )

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\ACER\Documents> & "C:/Program Files/Python38/python.exe" c:/Users/ACER/Documents/hello-tabulate2numerosPLYE

Creemos una tabla con cadenas alfanumericas:

Column 1	Column 2
1	2
3	4

El último número de la tabla es el: 4

Python

Notificación desde Plyer

El último número de la tabla es el: 4

¿Qué diferencias observa con el código del ejemplo 47? ¿Para qué sirve una notificación en Microsoft Windows?

Ejemplo 48. Creación de una tabla con dos filas y dos columnas con las coordenadas de cada celda de la tabla.

Objetivo: Utilizar la librería **tabulate** y conocer el sistema de coordenadas para saber localizar cada valor de cada sublista.

Cree el archivo: “**hello-tabulate3coord.py**” y añada este código:

```
print(" Creemos una tabla con cadenas alfanumericas:")
print(" ")
from tabulate import tabulate

# ejemplo3: Definir table data con cadenas alfanumericas (aunque sean numeros)
table_data = [
    ["coord(0,0)", "coord(0,1)"],
    ["coord(1,0)", "coord(1,1)"]
]

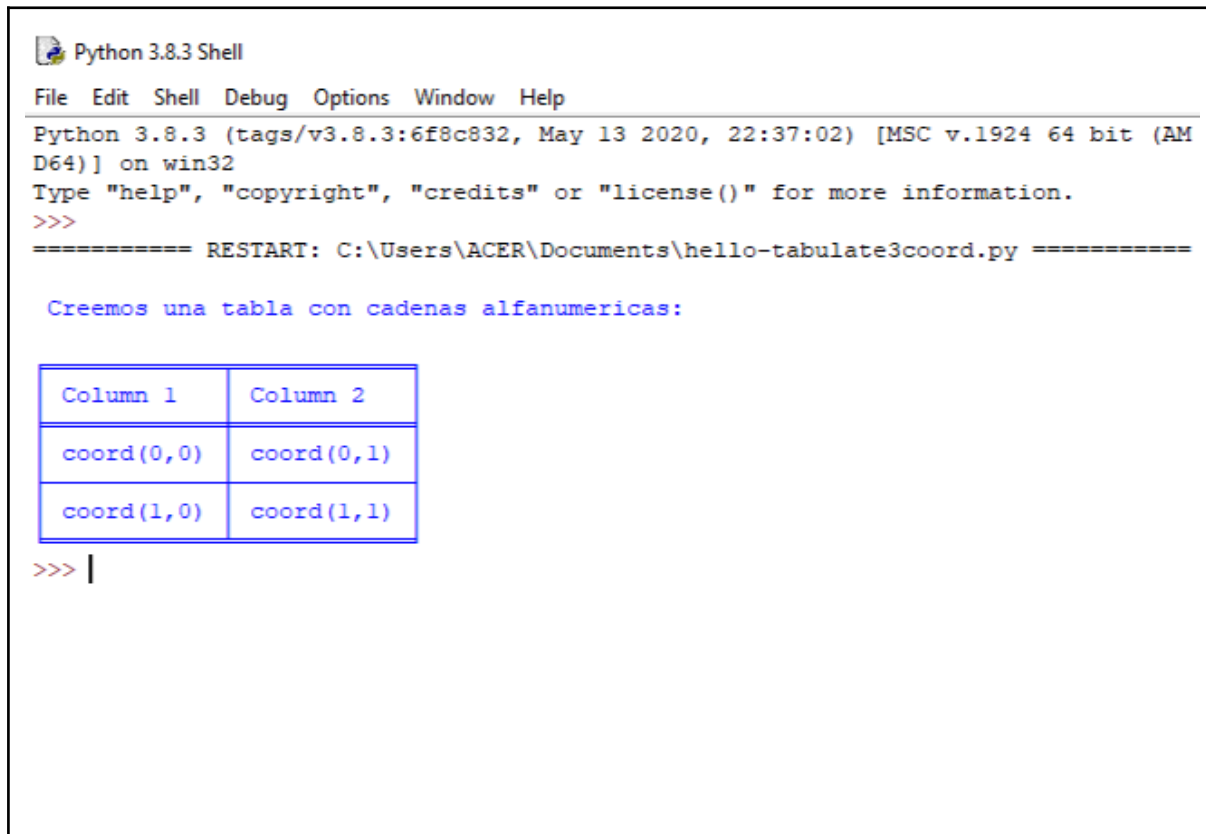
# Definir table headers (encabezados)
headers = ["Column 1", "Column 2"]

# Crear table
table = tabulate(table_data, headers, tablefmt="fancy_grid")

# Print result:
print(table)
```

Observe que la lista **table_data** contiene dos sublistas con las coordenadas de sus respectivas celdas (entre comillas).

Este es el resultado de ejecutar el código utilizando cadenas alfanuméricas dentro de las sublistas creadas.



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\hello-tabulate3coord.py =====

Creemos una tabla con cadenas alfanumericas:



| Column 1   | Column 2   |
|------------|------------|
| coord(0,0) | coord(0,1) |
| coord(1,0) | coord(1,1) |

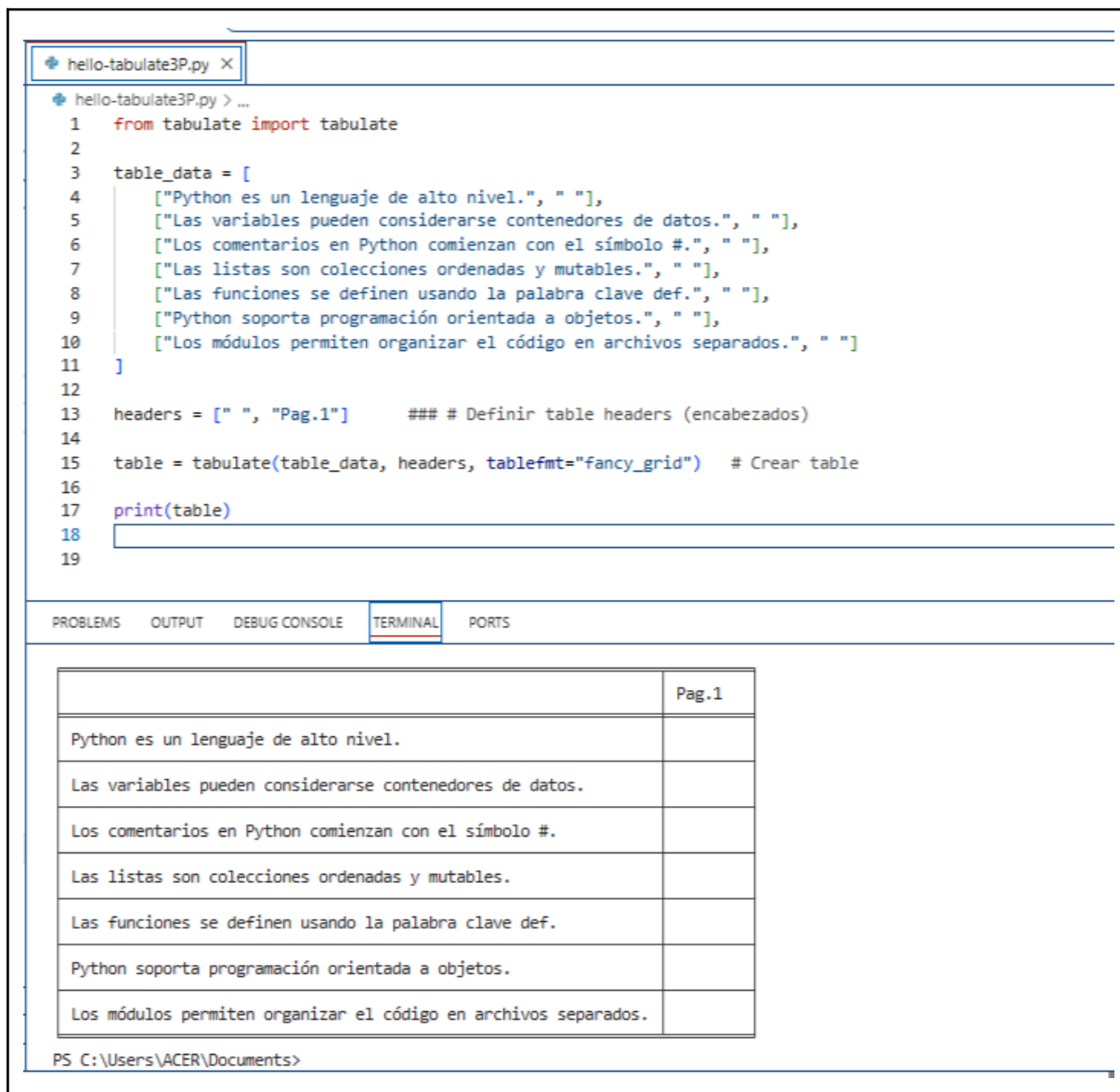

>>> |
```

Fin del Ejemplo 48.

Reto 1:

Cree una tabla con un texto (que contenga un párrafo de ocho líneas). Utilice la librería **tabulate** para crear la lista.

Cree el archivo: **“hello-tabulate3P.py”** y corrija los errores en el siguiente bloque de código:



The screenshot shows a code editor with a file named `hello-tabulate3P.py`. The code defines a list of text lines, sets headers, and uses the `tabulate` library to create a table with a 'fancy_grid' format. Below the code, the terminal window displays the resulting table.

```

1  from tabulate import tabulate
2
3  table_data = [
4      ["Python es un lenguaje de alto nivel.", " "],
5      ["Las variables pueden considerarse contenedores de datos.", " "],
6      ["Los comentarios en Python comienzan con el símbolo #.", " "],
7      ["Las listas son colecciones ordenadas y mutables.", " "],
8      ["Las funciones se definen usando la palabra clave def.", " "],
9      ["Python soporta programación orientada a objetos.", " "],
10     ["Los módulos permiten organizar el código en archivos separados.", " "]
11 ]
12
13 headers = [" ", "Pag.1"]      ### # Definir table headers (encabezados)
14
15 table = tabulate(table_data, headers, tablefmt="fancy_grid") # Crear table
16
17 print(table)
18
19

```

	Pag.1
Python es un lenguaje de alto nivel.	
Las variables pueden considerarse contenedores de datos.	
Los comentarios en Python comienzan con el símbolo #.	
Las listas son colecciones ordenadas y mutables.	
Las funciones se definen usando la palabra clave def.	
Python soporta programación orientada a objetos.	
Los módulos permiten organizar el código en archivos separados.	

PS C:\Users\ACER\Documents>

Reto 2:

Modifique el código anterior para que no aparezcan líneas en el diseño.

Ejemplo 49. Creación de una tabla con dos filas y dos columnas con números enteros. Realización de la suma de números enteros con los valores de la tabla.

Objetivo: Utilizar la librería `tabulate` y conocer el sistema de coordenadas para saber localizar cada valor de cada sublista.

Cree el archivo: `“hello-tabulate4suma.py”` y añada este código:

```
print(" ")
print(" Sumemos los valores de la tabla: ")
print(" ")
from tabulate import tabulate

# Definir table data con numeros enteros (ejemplo 4)
table_data = [
    [1, 2],
    [3, 4]
]

# Definir table headers (encabezados)
headers = ["Column 1", "Column 2", "Sumas"]

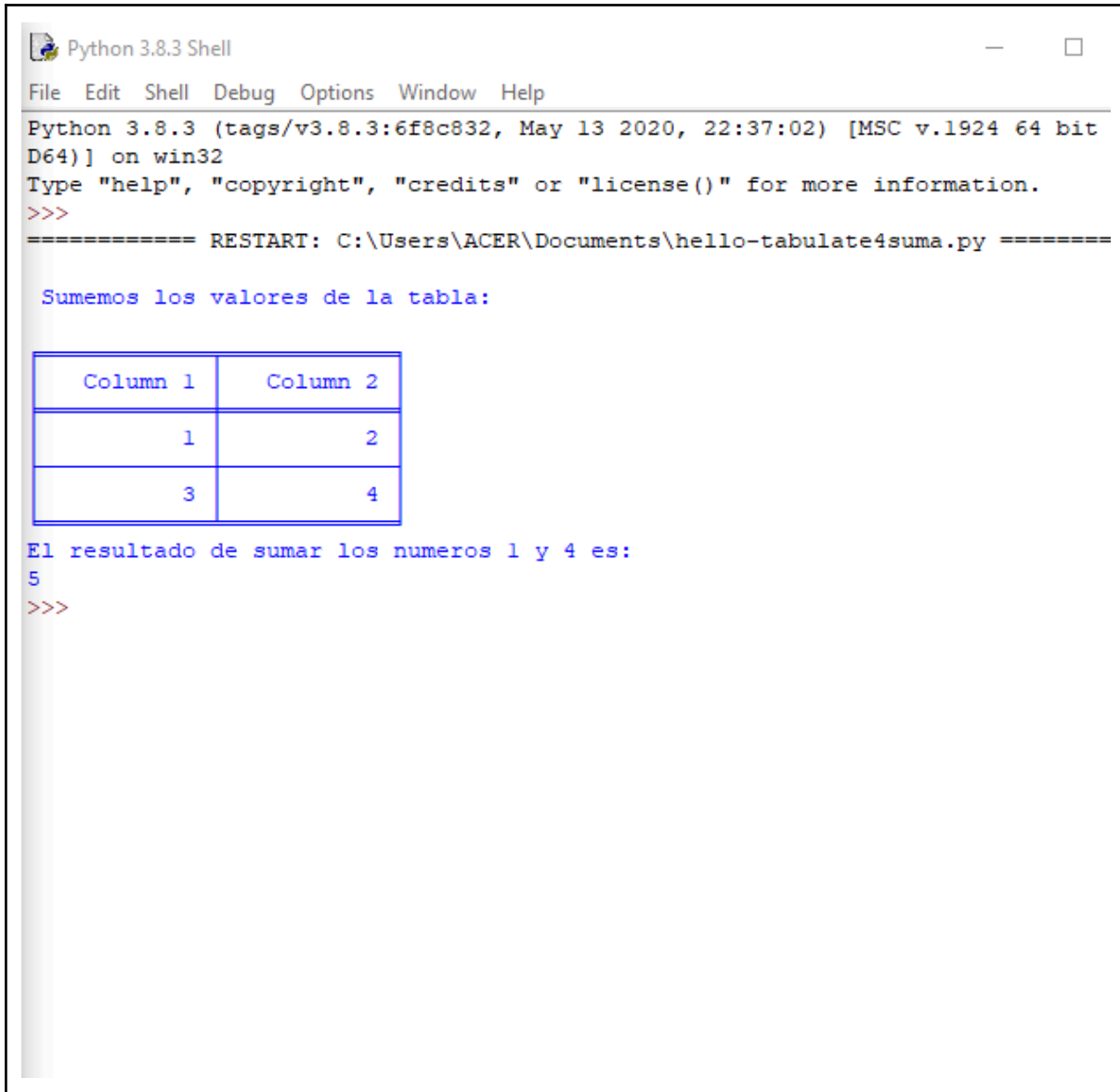
# Crear table
table = tabulate(table_data, headers, tablefmt="fancy_grid")

# Imprimir tabla (table):
print(table)

# Sumar el primer elemento de la primera sublista y el segundo
# elemento de la segunda sublista
# Resultado: 1 + 4 = 5
suma = table_data[0][0] + table_data[1][1]
print("El resultado de sumar los numeros 1 y 4 es:")
print(suma)
```

Observe que la lista `table_data` contiene dos sublistas con números enteros.

Este es el resultado de ejecutar el código utilizando números enteros dentro de las sublistas creadas para después crear la variable suma con los sumandos de las coordenadas (0,0) y (1,1).



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit
D64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\hello-tabulate4suma.py =====

Sumemos los valores de la tabla:



| Column 1 | Column 2 |
|----------|----------|
| 1        | 2        |
| 3        | 4        |



El resultado de sumar los numeros 1 y 4 es:
5
>>>
```

Fin del Ejemplo 49.

Ejemplo 50. Creación de una tabla con dos filas y tres columnas a partir de los datos de una tabla (con dos filas y dos columnas) previamente creada.

Objetivo: Utilizar la librería `tabulate` y realizar cálculos con las celdas de una tabla para crear una nueva tabla con los resultados obtenidos.

Cree el archivo: “`hello-tabulate5sumaplus.py`” y añada este código:

```
from tabulate import tabulate

# Definir table data (lista principal) con numeros enteros (con dos sublistas)
table_data = [
    [1, 2],
    [3, 4]
]

# Definir table headers (encabezados)
headers = ["Column 1", "Column 2"]

# Crear table
table = tabulate(table_data, headers, tablefmt="fancy_grid")

print(table)

# Sumar primer elemento de la primera sublista y segundo elem de la primera sublista
# Calcular: 1 + 2 = 3
sumarfila1 = table_data[0][0] + table_data[0][1]
print("El resultado de sumar los numeros 1 y 2 es:")
print(sumarfila1)

# Sumar primer elemento de la segunda sublista y segundo elem de la segunda sublista
# Resultado: 3 + 4 = 7
sumarfila2 = table_data[1][0] + table_data[1][1]
print("El resultado de sumar los numeros 3 y 4 es:")
print(sumarfila2)

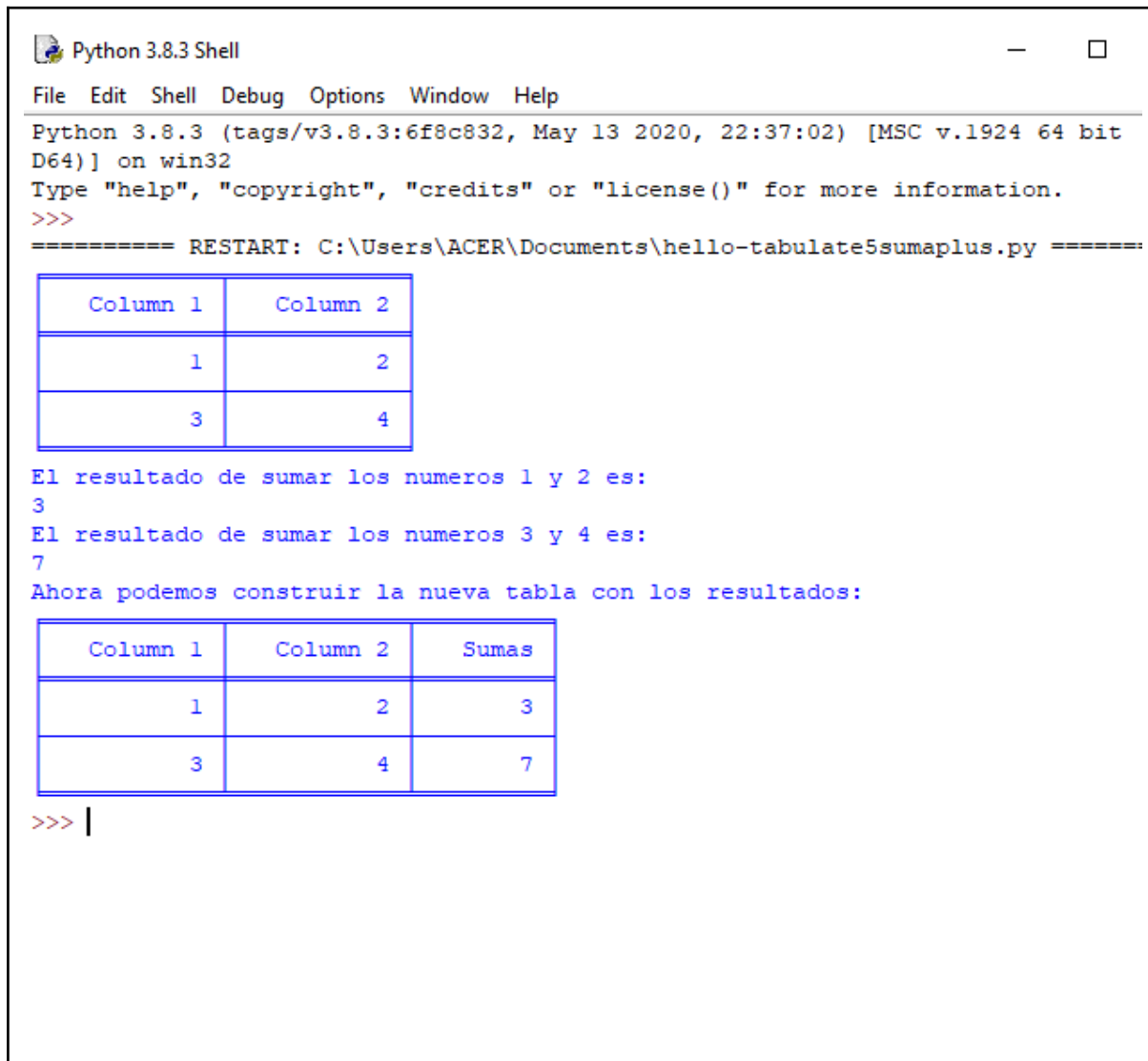
print("Ahora podemos construir la nueva tabla con los resultados:")
# Ahora construir la nueva tabla (newtable_data) con las sumas calculadas
newtable_data = [
    [table_data[0][0], table_data[0][1], sumarfila1],
    [table_data[1][0], table_data[1][1], sumarfila2]
]

# Definir table headers (encabezados)
headers = ["Column 1", "Column 2", "Sumas"]

# Crear nueva tabla (newtable)
newtable = tabulate(newtable_data, headers, tablefmt="fancy_grid")
print(newtable)
```

Observe que la lista `newtable_data` contiene dos sublistas con números enteros y el resultado de sumar ambas filas (`sumarfila1` y `sumarfila2`).

Este es el resultado de ejecutar el código utilizando los datos de la primera lista (**table_data**) para después crear la segunda lista (**newtable_data**) con los datos de la primera lista y el resultado de sumar cada fila.



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit
D64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\hello-tabulate5sumapplus.py =====
```

Column 1	Column 2
1	2
3	4

```
El resultado de sumar los numeros 1 y 2 es:
3
El resultado de sumar los numeros 3 y 4 es:
7
Ahora podemos construir la nueva tabla con los resultados:
```

Column 1	Column 2	Sumas
1	2	3
3	4	7

```
>>> |
```

Fin del Ejemplo 50.

Ejemplo 51. Creación de un juego basado en una tabla con cinco filas y dos columnas. En la columna de la izquierda aparecerán una serie de palabras y en la segunda columna aparecerán letras. El jugador debe responder a las preguntas en base a las palabras que aparecen en la tabla.

Ejemplo: ¿La primera palabra, “Android”, contiene la letra “A”?

Objetivo: Utilizar la librería `tabulate` y realizar comparaciones para comprobar que el jugador responde correctamente a la pregunta realizada.

Cree el archivo: “`hello-tabulate6.py`” y añada este código:

```
print(" ")
from tabulate import tabulate

# Definir table data (recuerda: pip install tabulate)
table_data = [
    ["Android" , "A"],
    ["Cobol"   , "C"],
    ["Basic"   , "B"],
    ["Compiler", "I"],
    ["Code"    , "E"]
]

# Definir table headers (encabezados)
headers = ["Palabras", "Letras"]

# Crear table
table = tabulate(table_data, headers, tablefmt="fancy_grid")

print(table)

print(" ")
print("Hola!")
print("Vamos a hacer un test sobre términos técnicos de programación.")

puntos=0

v=". Verdadero o Falso? (V/F)"
frase="La palabra " + table_data[0][0] + " contiene la letra " + table_data[0][1] + v
xxx = input(frase)

#Pregunta 1
if xxx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

print("Tu nota final en este test es:")
print(puntos)
```

Este es el resultado de ejecutar el código utilizando los datos de la lista (**table_data**). Observe que el test debería tener cinco preguntas y el código sólo muestra una. Debe completar el algoritmo con el resto de preguntas.

```
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\hello-tabulate6.py =====
```

Palabras	Letras
Android	A
Cobol	C
Basic	B
Compiler	I
Code	E

```

Hola!
Vamos a hacer un test sobre términos técnicos de programación.
La palabra Android contiene la letra A. Verdadero o Falso? (V/F)V
Respuesta correcta. Tienes 1 punto.
Tu nota final en este test es:
1
>>> |

```

Reto 1: (a) Añada cinco nuevos términos (palabras nuevas) a la lista original. (b) Añada a la lista una tercera columna para almacenar los puntos obtenidos en cada respuesta correcta. (c) Imprima la lista de listas completa con los puntos obtenidos en cada pregunta. (d) Cambie el encabezado de la tercera columna y llámelo "superpuntos". (e) Imprima el total de superpuntos obtenidos por el usuario.

Reto 2:

La Unión Europea fue creada el 1 de noviembre de 1993 con la entrada en vigor del Tratado de Maastricht. Sin embargo, sus inicios datan de 1951, con la Comunidad Europea del Carbón y del Acero (CECA) para más tarde formar la Comunidad Económica Europea (CEE) en 1957.

Los países que firmaron el Tratado de Maastricht en 1993 fueron:

- | | |
|----|--------------|
| 1 | Alemania |
| 2 | Bélgica |
| 3 | Francia |
| 4 | Italia |
| 5 | Luxemburgo |
| 6 | Países Bajos |
| 7 | España |
| 8 | Portugal |
| 9 | Reino Unido |
| 10 | Irlanda |
| 11 | Grecia |
| 12 | Dinamarca |

Utilizando la librería **tabulate**, cree una tabla con los países que formaban la Unión Europea en el año 1993 (cree una función para ordenar la lista alfabéticamente).

Reto 3:

Utilizando la librería **tabulate**, cree una tabla con los países que forman la Unión Europea en el año 2025, junto con la población de cada país miembro. A continuación, sume todas las cantidades y muestre el resultado.

Ejemplo 52. Creación de una aplicación que cree una ventana en modo gráfico utilizando la librería tkinter.

Objetivo: Entender e implementar el funcionamiento de diferentes tipos de objetos de la librería tkinter.

Grabe su programa con el nombre “**hello-world-tk-1.py**” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
#
# remember to install in your windows terminal:
#
# pip install tk
#
import tkinter as tk

def main():
    root = tk.Tk()
    root.title("tk module")

    # Set the window size to 600x400
    root.geometry("600x400")

    label = tk.Label(root, text="Hello, Windows!")
    label.pack(padx=20, pady=20)

    root.mainloop()

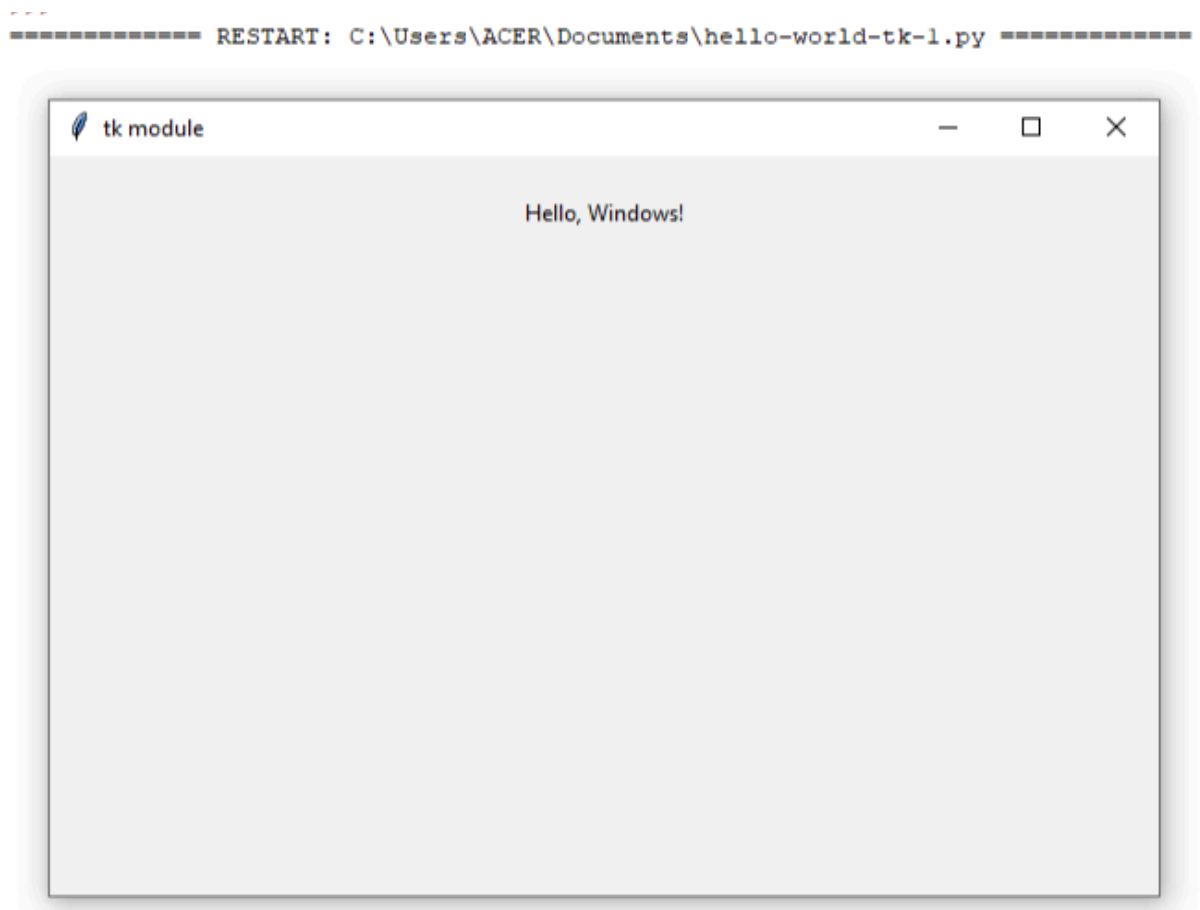
if __name__ == "__main__":
    main()
```

(continúa)

En este ejemplo se han implementado dos acciones principales:

- a) **Creación de la ventana principal (en modo gráfico)**
- b) **Inserción de una línea de texto en la ventana creada**

Podemos diseñar ventanas con diferentes resoluciones e insertar texto en las ventanas creadas. De este modo podemos crear, por ejemplo, una aplicación que mostrara instrucciones sobre cómo montar una mesa. También podríamos crear una aplicación que mostrase diferentes páginas de un cuento para niños.



Fin del Ejemplo 52

Ejemplo 53. Creación de un test sobre Microsoft Windows y Python 3 (implementando funciones definidas por el usuario).

Objetivos: Entender el funcionamiento de una función definida por el usuario. Entender cómo se definen las diferentes funciones implementadas y el orden en que deben ser escritas.

Grabe su programa (vacío, por ahora) con el nombre “test101_.py” en su ordenador en la carpeta “Mis Documentos” y añada las siguientes órdenes:

```
def test1():
    # Definición de la función test1
    puntos=0
    # VAMOS A HACER UN TEST SOBRE PYTHON Y WINDOWS.
    #PREGUNTA 1
    xxx = input("El comando DIR en Windows sirve para ver los archivos de una carpeta (V/F)? : ")
    if xxx=="V":
        # TODAS LAS cinco RESPUESTAS SON VERDADERAS
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 2
    xxx = input("El comando DEL sirve para borrar archivos. (V/F)? : ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
    #endif
    #PREGUNTA 3
    xxx=input("Las variables en Python pueden considerarse contenedores de datos. (V/F)? : ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
    #endif
    #PREGUNTA 4
    xxx = input("En Python, debemos asignar un nombre diferente a cada variable. (V/F)? : ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
    #endif
    #PREGUNTA 5
    xxx=input("Para imprimir el valor de X, utilizamos la instrucción print(X). (V/F)? : ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
    #endif
    print("Tu nota en este test es:"); print(puntos)

def felicitacion():
    # Definición de la función felicitacion
    print("Felicitaciones, siga estudiando así."); print("Estamos orgullosos de usted.")

##### Programa principal (main program)
xname = input("Dime tu nombre, por favor: "); print("Hola, " + xname + " !")
a="Vamos a hacer un test de 5 preguntas sobre"; b=" Microsoft Windows y Python 3."
print(a+b)
test1(); felicitacion(); print("Fin de la prueba.")
```

Explicación:

El programa está dividido en dos partes:

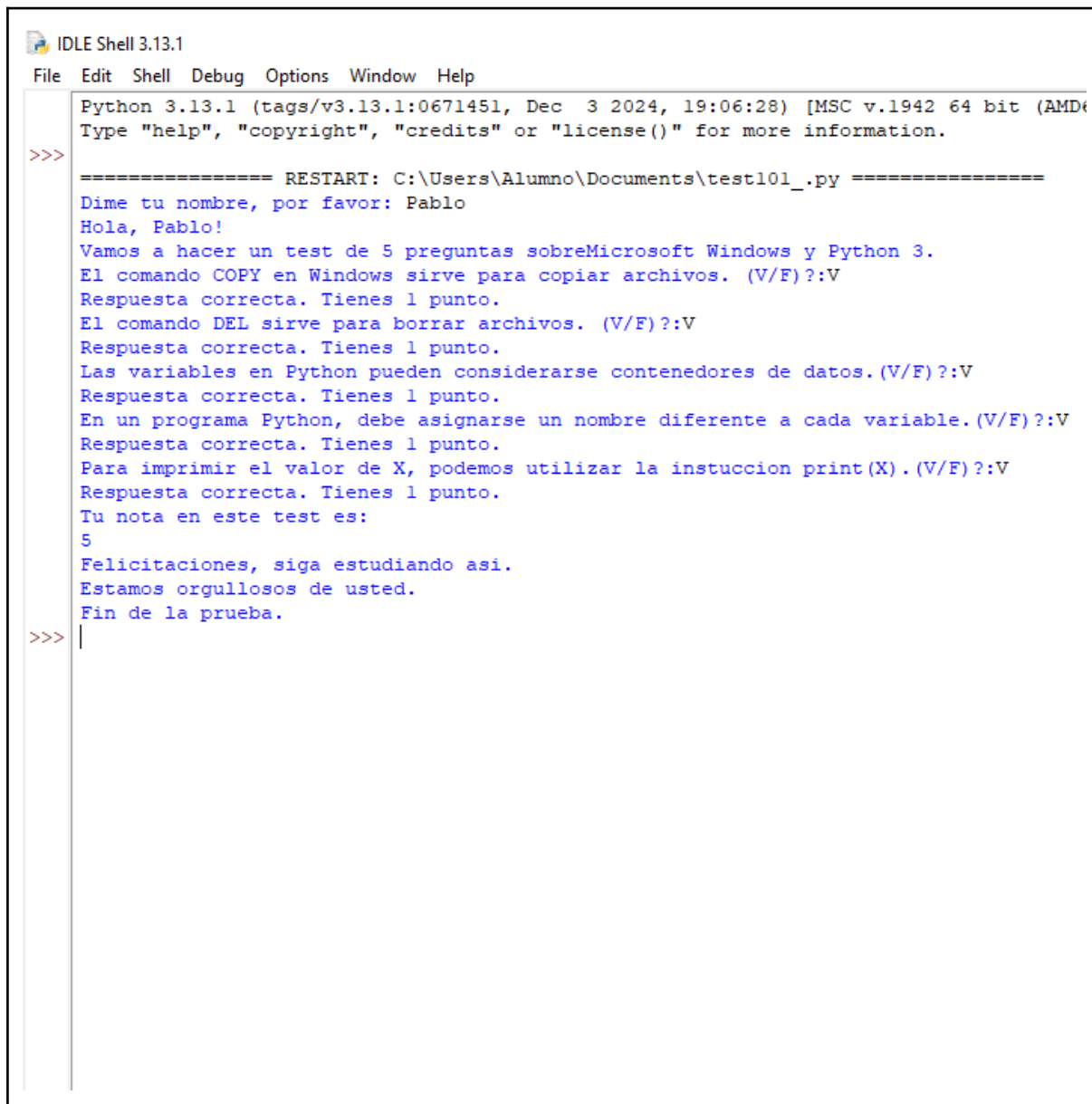
- a) Definición de las **dos** funciones: **test1()** y **felicitation()**
- b) El programa principal. Las funciones siempre deben ser escritas antes del programa principal.

Importante:

Podemos diseñar funciones para ejecutar una tarea o varias. Dentro de una función pueden existir una o varias órdenes.

(continúa)

Este es el resultado al ejecutar el código del archivo “test101_.py”:



```
IDLE Shell 3.13.1
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\Alumno\Documents\test101_.py =====
Dime tu nombre, por favor: Pablo
Hola, Pablo!
Vamos a hacer un test de 5 preguntas sobre Microsoft Windows y Python 3.
El comando COPY en Windows sirve para copiar archivos. (V/F)? :V
Respuesta correcta. Tienes 1 punto.
El comando DEL sirve para borrar archivos. (V/F)? :V
Respuesta correcta. Tienes 1 punto.
Las variables en Python pueden considerarse contenedores de datos. (V/F)? :V
Respuesta correcta. Tienes 1 punto.
En un programa Python, debe asignarse un nombre diferente a cada variable. (V/F)? :V
Respuesta correcta. Tienes 1 punto.
Para imprimir el valor de X, podemos utilizar la instrucción print(X). (V/F)? :V
Respuesta correcta. Tienes 1 punto.
Tu nota en este test es:
5
Felicitaciones, siga estudiando así.
Estamos orgullosos de usted.
Fin de la prueba.
>>> |
```

Fin del Ejemplo 53.

Ejemplo 54. Creación de un test sobre Microsoft Windows y Python 3 (implementando funciones definidas por el usuario devolviendo resultados).

Objetivo: Entender el funcionamiento de una función definida por el usuario. Entender cómo se definen las diferentes funciones implementadas. En este ejemplo implementamos la función `puntos_pregunta5()` para mostrar otra forma de devolver resultados cuando llamamos a una función.

Grabe su programa (vacío, por ahora) con el nombre “test102_funcion.py” y añada las siguientes órdenes.

```
# definicion de funciones

def test1():
    # TODAS LAS cinco RESPUESTAS SON VERDADERAS
    puntos=0
    #PREGUNTA 1
    xxx = input("El comando COPY en Windows sirve para copiar archivos. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 2
    xxx = input("El comando DEL sirve para borrar archivos. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 3
    xxx = input("Las variables en Python pueden considerarse contenedores de datos. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 4
    xxx = input("En un programa Python, debe asignarse un nombre diferente a cada variable. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    total=puntos+puntos_pregunta5()
    print("Tu nota en este test es:")
    print(total)

def puntos_pregunta5():
    #PREGUNTA 5. Esta función devolverá 1 si la respuesta es correcta y devolverá 0 si es incorrecta.
    x=input("Para ver el valor de la variable X en Python, podemos usar la instrucción print(X). (V/F)? ")
    if x=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        return 1
    else:
        return 0
```

(continúa)

```
def felicitacion():
    print("Felicitaciones, siga estudiando así.")
    print("Estamos orgullosos de usted.")

#### programa principal (main program)

xname = input("Dime tu nombre, por favor: ")
print("Hola, " + xname + "!")
a="Vamos a hacer un test de 5 preguntas sobre"
b=" Microsoft Windows y Python 3."
print(a+b)
test1()
felicitacion()
print("Fin de la prueba.")
```

El programa del ejemplo (número 54) está dividido en dos partes:

a) Definición de las **tres** funciones utilizadas:

<code>felicitacion()</code>	<code>test1()</code>	<code>puntos_pregunta5()</code>
-----------------------------	----------------------	---------------------------------

b) El programa principal.

Como hemos visto en ejercicios anteriores, dentro de una función pueden existir una o varias órdenes. Además una función puede llamar a otra función. En este ejemplo, dentro de la función `test1()` podemos observar que para calcular los puntos totales, hemos llamado a la función `puntos_pregunta5()`. Esta función hace la pregunta 5 al usuario y devuelve un punto si la respuesta fue correctamente respondida. El total se ha calculado con la siguiente fórmula:

$\text{total} = \text{puntos} + \text{puntos_pregunta5}()$

Este es el resultado que obtenemos al ejecutar el código:

```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\test102_funcion.py =====
Dime tu nombre, por favor: Antonio
Hola, Antonio!
Vamos a hacer un test sobre de 5 preguntas sobre Microsoft Windows y Python 3.
El comando COPY en Windows sirve para copiar archivos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
El comando DEL sirve para borrar archivos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
Las variables en Python pueden considerarse contenedores de datos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
En un programa Python, debe asignarse un nombre diferente a cada variable. (V/F)? V
Respuesta correcta. Tienes 1 punto.
Para imprimir el valor de la variable X en Python, podemos usar la instrucción print(X). (V/F)? V
Respuesta correcta. Tienes 1 punto.
Tu nota en este test es:
5
Fin de la prueba.
>>>
```

Fin del Ejemplo 54.

Ejemplo 55. Creación de un test sobre Microsoft Windows y Python 3 (implementando funciones definidas por el usuario devolviendo resultados diferentes según el parámetro enviado).

Objetivo: Entender el funcionamiento de una función definida por el usuario. Entender cómo se definen las diferentes funciones implementadas. En este ejemplo implementamos la función `puntos_pregunta(n)` para mostrar otra forma de devolver resultados cuando llamamos a una función.

Grabe su fichero (vacío, por ahora) con el siguiente nombre de archivo: “test103_funcion_param.py”. A continuación, añada las siguientes órdenes:

```
*test103_funcion_param.py - C:\Users\ACER\Documents\test103_funcion_param.py (3.8.3)*
File Edit Format Run Options Window Help

# definicion de funciones
def felicitacion():
    print("Felicitaciones, siga estudiando así.")
    print("Estamos orgullosos de usted.")

def test1():
    # TODAS LAS cinco RESPUESTAS SON VERDADERAS
    puntos=0
    #PREGUNTA 1
    xxx = input("El comando COPY en Windows sirve para copiar archivos. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 2
    xxx = input("El comando DEL sirve para borrar archivos. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 3
    xxx = input("Las variables en Python pueden considerarse contenedores de datos. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 4
    xxx = input("En un programa Python, debe asignarse un nombre diferente a cada variable. (V/F)? ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 5
    vf=" (V/F): "
    x=input("Para ver el valor de la variable X en Python, podemos utilizar la instrucción print(X)."+vf)
    if x=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    print("Tu nota en esta parte del test es:")
    print(puntos)
```

```

def puntos_pregunta(num):
    """ Esta función es llamada por la función test2() para hacer las preguntas 4 y 5
    vf=" (V/F): "
    if num==5:
        #PREGUNTA 5. Esta función devolverá 1 si la respuesta es correcta y devolverá 0 si es incorrect
        p_string="La app (para Windows) Sublime Text no puede editar programas escritos en Python."+vf
    if num==4:
        #PREGUNTA 4. Esta función devolverá 1 si la respuesta es correcta y devolverá 0 si es incorrect
        p_string="La aplicación de Windows Notepad no puede editar programas escritos en Python."+vf
    xxx = input(p_string)
    if xxx=="F":
        print("Respuesta correcta. Tienes 1 punto más.")
        return 1
    else:
        return 0

```

En este ejemplo, dentro de la función `test2()` podemos observar que para calcular los puntos totales, hemos llamado a la función `puntos_pregunta(n)`. Esta función hace la preguntas 4 y 5 al usuario y devuelve un punto si la respuesta fue correctamente respondida. Como puede verse en la siguiente página, el total se ha calculado con la siguiente fórmula:

$$\text{total} = \text{puntos} + \text{puntos_pregunta}(4) + \text{puntos_pregunta}(5)$$

(Nota: No olvide copiar el código de la siguiente página)

```

def test2():
    # TODAS LAS cinco RESPUESTAS SON FALSAS
    puntos=0
    #PREGUNTA 1
    xxx = input("El comando TREE en Windows sirve para copiar archivos. (V/F)? : ")
    if xxx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 2
    xxx = input("El comando DEL sirve para mover o renombrar archivos. (V/F)? : ")
    if xxx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    #PREGUNTA 3
    xxx = input("Los programas creados en lenguaje Python no se almacenan en ficheros de texto. (V/F)? : ")
    if xxx=="F":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #Calculamos totales y hacemos llamada a las PREGUNTAS 4 y 5 con la funcion puntos_pregunta(num de pregunta)
    total=puntos+puntos_pregunta(4)+puntos_pregunta(5)
    print("Tu nota en este test es:")
    print(total)

```

```

#### programa principal (main program)

xname = input("Dime tu nombre, por favor: ")
print("Hola, " + xname + " !")
a="Vamos a hacer un test de 5 preguntas sobre"
b=" Microsoft Windows y Python 3."
print(a+b)
test1()
test2()
felicitacion()
print("Fin de la prueba.")

```

El programa está dividido en dos partes:

a) Definición de las **cuatro** funciones utilizadas:

<code>felicitacion()</code>	<code>test1()</code>	<code>test2()</code>	<code>puntos_pregunta(n)</code>
-----------------------------	----------------------	----------------------	---------------------------------

b) El programa principal.

Como hemos visto en ejercicios anteriores, dentro de una función pueden existir una o varias órdenes. Además una función puede llamar a otra función. En este ejemplo, dentro de la función `test2()` podemos observar que para calcular los puntos totales, hemos llamado dos veces a la función `puntos_pregunta(n)`.

(continúa)

Este es el resultado al ejecutar el código:

```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\test103_funcion_param.py =====
Dime tu nombre, por favor: Maria
Hola, Maria!
Vamos a hacer un test sobre de 10 preguntas sobre Microsoft Windows y Python 3.
El comando COPY en Windows sirve para copiar archivos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
El comando DEL sirve para borrar archivos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
Las variables en Python pueden considerarse contenedores de datos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
En un programa Python, debe asignarse un nombre diferente a cada variable. (V/F)? V
Respuesta correcta. Tienes 1 punto.
Para ver el valor de la variable X en Python, podemos utilizar la instrucción print(X). (V/F): V
Respuesta correcta. Tienes 1 punto.
Tu nota en esta parte del test es:
5
El comando TREE en Windows sirve para copiar archivos. (V/F)? F
Respuesta correcta. Tienes 1 punto.
El comando DEL sirve para mover o renombrar archivos. (V/F)? F
Respuesta correcta. Tienes 1 punto.
Los programas creados en lenguaje Python no se almacenan en ficheros de texto. (V/F)? F
Respuesta correcta. Tienes 1 punto.
La aplicación de Windows Notepad no puede editar programas escritos en Python. (V/F): F
Respuesta correcta. Tienes 1 punto más.
La app (para Windows) Sublime Text no puede editar programas escritos en Python. (V/F): F
Respuesta correcta. Tienes 1 punto más.
Tu nota en este test es:
5
Felicitaciones, siga estudiando así.
Estamos orgullosos de usted.
Fin de la prueba.
>>> |
```

Fin del Ejemplo 55.

Ejemplo 56. Creación de un test de 3 preguntas sobre ofimática utilizando una matriz simple y los botones de la librería flet mostrando la respuesta correcta.

Objetivo: Entender e implementar el funcionamiento de una matriz simple con los diferentes tipos de botones y el objeto **listview** de la librería flet.

Grabe su programa (vacío) con el nombre “test105_flet_test3q3matrix_.py” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
test105_flet_test3q3matrix.py - C:\Users\Alumno\Documents\test105_flet_test3q3matrix.py (3.13.1)
File Edit Format Run Options Window Help
import flet as ft    ### Test de tres preguntas utilizando la librería flet en Python 3.8

leng=["LiveCode", "Cobol", "Python"]    ### opciones de respuesta para la question 1

def main(page):
    page.title = "Python 3"
    page.window_width = 640
    page.window_height = 480

    # Creamos contenedor para preguntas con ListView (list de opciones con scroll a la der de la pant.
    content = ft.ListView(expand=True, spacing=10, padding=10)

    # Creamos la Function para comprobar las respuestas
    def check_answer(cg, correct_answer, tz, tx):
        tx.value = ""
        tz.value = f"Su respuesta: {cg.value}"
        if cg.value == correct_answer:
            tx.value = "Es correcta."
        else:
            tx.value = "Es incorrecta."
        page.update()

    # Question 1
    tx = ft.Text()
    tz = ft.Text()
    b = ft.ElevatedButton(text='Comprobar', on_click=lambda e: check_answer(cg, "Python", tz, tx))
    cg = ft.RadioGroup(content=ft.Column([
        ft.Radio(value= leng[0], label= leng[0]),
        ft.Radio(value= leng[1], label= leng[1]),
        ft.Radio(value= leng[2], label= leng[2])
    ]))
    content.controls.append(ft.Column([
        ft.Text("1. ¿Que lenguaje de programacion invento Guido Van Rossum?"), cg, b, tz, tx
    ]))
```

(continúa)

Continuación del código del ejemplo:

```
# Question 2
tx2 = ft.Text()
tz2 = ft.Text()
b2 = ft.ElevatedButton(text='Comprobar', on_click=lambda e: check_answer(cg2, "Dennis Ritchie", tz2, tx2))
cg2 = ft.RadioGroup(content=ft.Column([
    ft.Radio(value="Ken Thompson", label="Ken Thompson"),
    ft.Radio(value="Bjarne Stroustrup", label="Bjarne Stroustrup"),
    ft.Radio(value="Dennis Ritchie", label="Dennis Ritchie")
]))
content.controls.append(ft.Column([
    ft.Text("2. ¿Quien invento el lenguaje de programacion C?"), cg2, b2, tz2, tx2
]))

# Question 3
tx3 = ft.Text()
tz3 = ft.Text()
b3 = ft.ElevatedButton(text='Comprobar', on_click=lambda e: check_answer(cg3, "Tim Paterson", tz3, tx3))
cg3 = ft.RadioGroup(content=ft.Column([
    ft.Radio(value="Tim Paterson", label="Tim Paterson"),
    ft.Radio(value="Bjarne Stroustrup", label="Bjarne Stroustrup"),
    ft.Radio(value="Dennis Ritchie", label="Dennis Ritchie")
]))
content.controls.append(ft.Column([
    ft.Text("3. ¿Quien invento el QDOS?"), cg3, b3, tz3, tx3
]))

# Ahora ya tenemos el contenido (content) que queremos poner en la pagina (main window)
page.add(content)

ft.app(target=main)
```

Como puede observarse, el código de la pregunta 1 (question 1) es ligeramente distinto al código de la pregunta 2 (question 2). Este último bloque de código es idéntico al del ejemplo anterior. Sin embargo en el bloque de la pregunta 1 (question 1), se han utilizado los elementos de la matriz **leng[]** declarada al principio del “script”. La matriz **leng[]** contiene 3 elementos: “Livecode”, “Cobol” y “Python”. Estas cadenas alfanuméricas son las posibles respuestas para la pregunta 1.

Veamos cómo funcionan las **matrices más simples** (también llamadas “listas”) en **Python 3** de manera sencilla.

Ejemplo 57d. Creación de un bucle utilizando una matriz simple

Objetivo: Repasar el funcionamiento de las matrices. En este ejemplo, utilizaremos una matriz simple con 10 elementos (cada color es un elemento de la matriz creada).

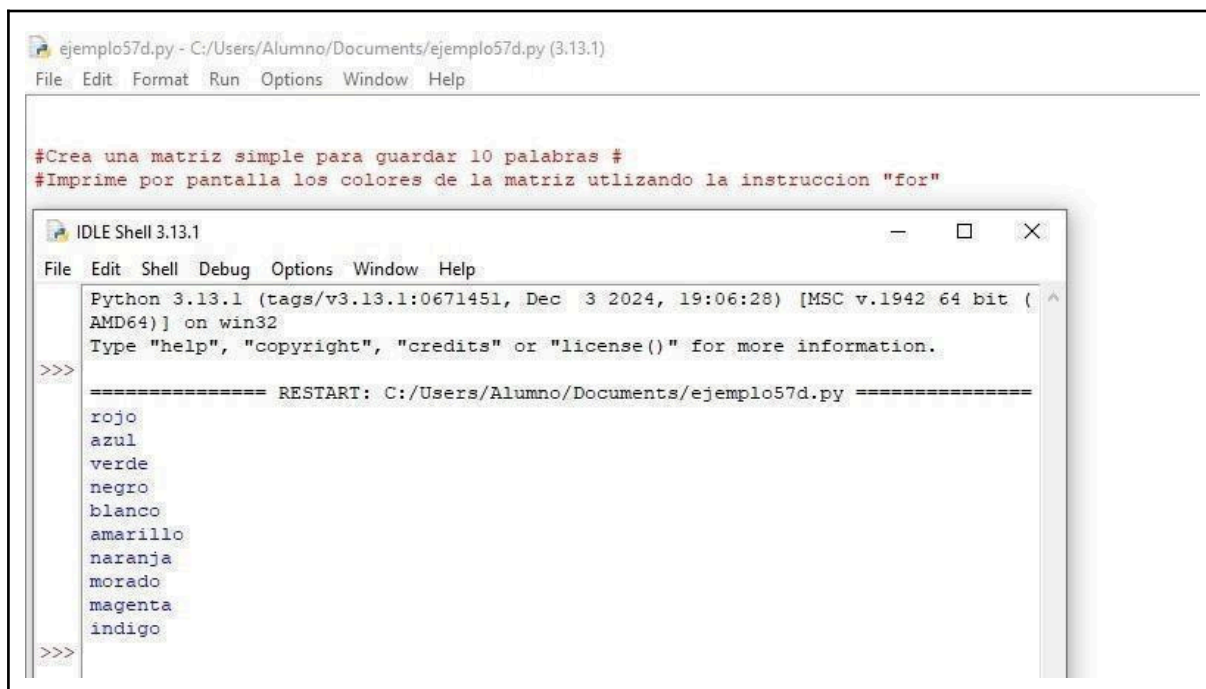
Grabe su programa (vacío) con el nombre “ejemplo57d.py” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
palabras= [ "rojo", "azul", "verde",
            "negro", "blanco", "amarillo",
            "naranja", "morado", "magenta", "indigo" ]

for c in palabras:
    print(c)
```

Este es el resultado de ejecutar el código del ejemplo 57d:



The screenshot shows a Python IDE window titled "ejemplo57d.py - C:/Users/Alumno/Documents/ejemplo57d.py (3.13.1)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The main editor area contains the following code:

```
#Crea una matriz simple para guardar 10 palabras #
#Imprime por pantalla los colores de la matriz utilizando la instruccion "for"

palabras= [ "rojo", "azul", "verde",
            "negro", "blanco", "amarillo",
            "naranja", "morado", "magenta", "indigo" ]

for c in palabras:
    print(c)
```

Below the main editor, there is an "IDLE Shell 3.13.1" window. It shows the output of the program:

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/Users/Alumno/Documents/ejemplo57d.py =====
rojo
azul
verde
negro
blanco
amarillo
naranja
morado
magenta
indigo
>>>
```

Repasemos detenidamente qué es una lista (o matriz) en Python 3:

Una matriz o lista en Python es como un contenedor (que está en la memoria del ordenador) donde podemos guardar un grupo de elementos. Estos elementos pueden ser palabras o números (incluso una lista puede contener otra lista).

Para poder trabajar con los elementos de una lista, ésta debe ser declarada previamente.

Para declarar los valores de la lista, emplearemos el siguiente bloque de código:

```
palabras= [ "rojo", "azul", "verde",  
            "negro", "blanco", "amarillo",  
            "naranja", "morado", "magenta", "indigo" ]
```

Para poder ver o imprimir los valores de una lista, debemos recorrer la lista. Para ello utilizamos un bucle que recorra la lista de principio a fin, es decir desde el elemento 0 hasta el último elemento.

En el código del ejemplo57d, el bucle (iniciado con la instrucción **for**) recorre cada elemento de la lista “**palabras**”. En cada iteración, la variable “**c**” toma el valor de un color de la lista (matriz). Luego, la función **print()** imprime cada elemento en la pantalla.

Esto quiere decir que en Python podemos considerar un “array” como los bloques que constituyen una “matriz”. Sin embargo, existen algunas diferencias:

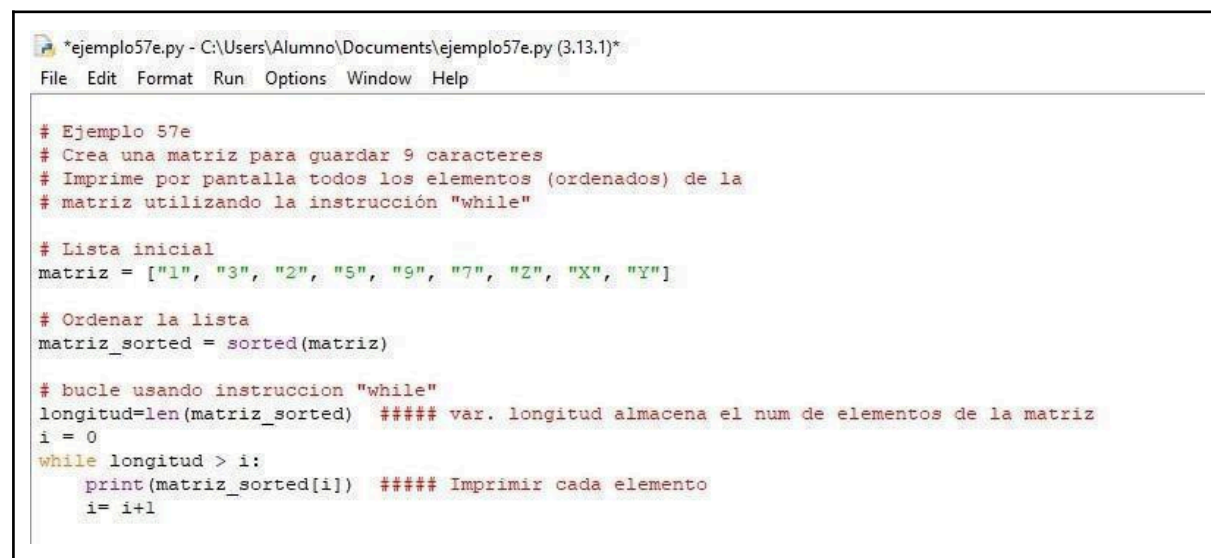
Un array (por ejemplo, de la librería **numpy**) es una colección de valores y puede ser unidimensional (como una lista), bidimensional (como una cuadrícula, también llamada “grid”) o tener más dimensiones. Cuando creamos un “array” bidimensional (“2D array”), funciona de manera muy similar a una matriz, que es básicamente una disposición rectangular de números o elementos.

Ejemplo 57e. Crear un bucle utilizando una matriz simple de caracteres

Objetivo: Repasar e implementar el funcionamiento de una matriz simple con nueve elementos (cada elemento de la matriz es una cadena alfanumérica).

Grabe su programa (vacío) con el nombre “ejemplo57e.py” en su ordenador en la carpeta “Mis Documentos”

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:



```
*ejemplo57e.py - C:\Users\Alumno\Documents\ejemplo57e.py (3.13.1)*
File Edit Format Run Options Window Help

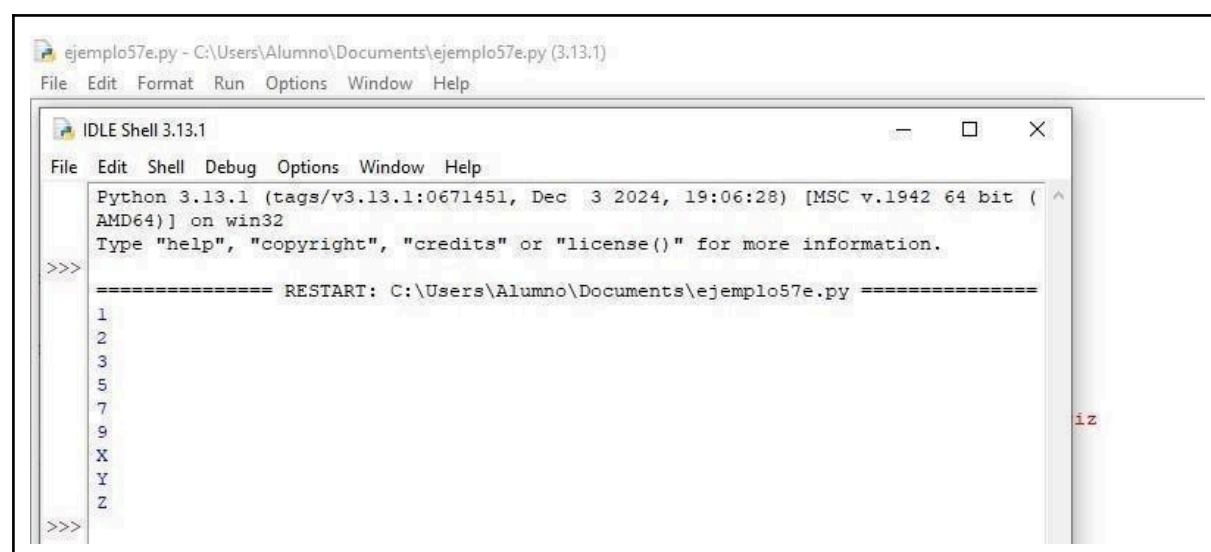
# Ejemplo 57e
# Crea una matriz para guardar 9 caracteres
# Imprime por pantalla todos los elementos (ordenados) de la
# matriz utilizando la instrucción "while"

# Lista inicial
matriz = ["1", "3", "2", "5", "9", "7", "Z", "X", "Y"]

# Ordenar la lista
matriz_sorted = sorted(matriz)

# bucle usando instruccion "while"
longitud=len(matriz_sorted) ##### var. longitud almacena el num de elementos de la matriz
i = 0
while longitud > i:
    print(matriz_sorted[i]) ##### Imprimir cada elemento
    i= i+1
```

Este es el resultado de ejecutar el código del ejemplo:



```
ejemplo57e.py - C:\Users\Alumno\Documents\ejemplo57e.py (3.13.1)
File Edit Format Run Options Window Help

IDLE Shell 3.13.1
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit ( ^
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\Alumno\Documents\ejemplo57e.py =====
1
2
3
5
7
9
X
Y
Z
>>>
```

Para poder ver o imprimir los valores de la lista, debemos recorrerla. Para ello utilizaremos un bucle (con la instrucción **while**) que recorrerá la lista de principio a fin, es decir desde el elemento 0 hasta el último elemento. Para saber cuál es el último elemento, calcularemos la longitud de la matriz que almacena los valores de la lista. Para ello, utilizaremos la siguiente función:

```
longitud= len(matriz_sorted)
```

En el ejemplo, el bucle (iniciado con la instrucción **while**) recorre cada elemento de la lista ordenada, llamada “**matriz_sorted**”. En cada iteración, la variable “**i**” toma el valor de un elemento de la lista (matriz). El bucle acaba cuando la variable **i** es igual al valor almacenado en la variable **longitud**. Luego, la función **print()** imprime cada elemento en la pantalla.

Ejemplo 57f. Crear un bucle utilizando una matriz simple de palabras

Objetivo: Repasar e implementar el funcionamiento de una matriz simple con seis elementos (cada elemento de la matriz es una cadena alfanumérica).

Grabe su programa (vacío) con el nombre “ejemplo57f.py” en su ordenador en la carpeta “Mis Documentos”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes:

```
# Ejemplo 57f

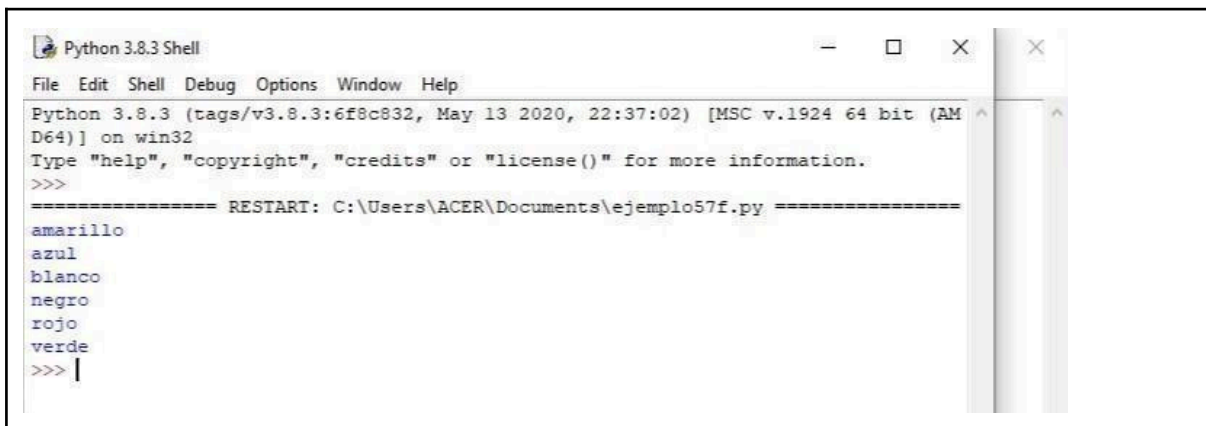
# Crea una matriz para guardar 6 palabras #
# Imprime por pantalla todos los colores (ordenados alfabeticamente) de la
# matriz utilizando la instrucción "while"

# Lista inicial
palabras = ["rojo", "azul", "verde", "negro", "blanco", "amarillo"]

# Ordenar la lista
palabras_sorted = sorted(palabras)

# do-while loop usando while True
i = 0
while True:
    print(palabras_sorted[i]) # Imprimir cada palabra
    i = i + 1
    if i >= len(palabras_sorted): # Condición para salir del bucle
        break
```

Este es el resultado de ejecutar el código del ejemplo:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ACER\Documents\ejemplo57f.py =====
amarillo
azul
blanco
negro
rojo
verde
>>> |
```

Ejemplo 57fx. Convertir una lista de temperaturas Celsius a temperaturas Fahrenheit utilizando un bucle.

Objetivo: Realizar operaciones matemáticas para convertir temperaturas.

Grabe su programa (vacío) con el nombre “ejemplo57fx.py” en su ordenador en la carpeta “Mis Documentos”. Ahora que ya tiene creado el fichero, añada las siguientes órdenes a su archivo:

```
temperaturas_celsius = [0, 10, 20, 30, 40]
temperaturas_fahrenheit = []           # Lista vacía para almacenar resultados
for temp_c in temperaturas_celsius:
    temp_f = (temp_c * 9/5) + 32        # Aplicamos la fórmula para cada temperatura
    temperaturas_fahrenheit.append(temp_f) # Añadimos el resultado a la nueva lista
print(temperaturas_fahrenheit)
```

Reto: Utilice la librería **NumPy** (para aplicar la operación matemática a toda la matriz de una sola vez, lo cual es más eficiente). Reemplace el código anterior por estas líneas (recuerde que antes necesita instalar en su sistema la librería mediante la orden: **pip install numpy**):

```
import numpy as np
# Primero crearemos un array de NumPy con las temperaturas en Celsius
temperaturas_celsius = np.array([0, 10, 20, 30, 40])
# NumPy es más eficiente pq aplica la fórmula a todos los elem. del array automáticamente
temperaturas_fahrenheit = (temperaturas_celsius * 9/5) + 32
print(temperaturas_fahrenheit)
```

Importante

`np.array()` es la función que utilizamos para crear un **ndarray**.
ndarray es el tipo de objeto o la clase que resulta de usar esa función.

La librería NumPy es esencial para la ciencia de datos,
 el aprendizaje automático y la computación científica.
 Recuerde que en un contexto Big Data, este conjunto de datos es el que
 proporciona los datos y la ciencia de datos los analiza y les da sentido.

Consideraciones

No hay una regla fija para definir qué es "Big Data" en función de la cantidad de datos procesados. La definición no se basa únicamente en el volumen. Sin embargo, los científicos de datos, analistas, técnicos informáticos y las empresas a menudo se refieren a cantidades que superan la capacidad de las herramientas de procesamiento de datos tradicionales.

La definición más aceptada de Big Data se basa en las "3 V" (en algunos textos científicos pueden ser "5 V"): Volumen (la cantidad de datos), Velocidad (el ritmo al que se generan y procesan) y Variedad (los diferentes tipos de datos, estructurados y no estructurados). Por ello, un conjunto de datos puede ser considerado "Big Data" incluso si no se mide en terabytes, siempre y cuando la velocidad y la variedad sean un desafío para las herramientas convencionales.

Aunque no hay una cifra estándar, muchos científicos de datos consideran que los conjuntos de datos que comienzan a ser un desafío para procesar con software y hardware tradicionales, a menudo superando los **40** terabytes, pueden considerarse Big Data. Sin embargo, las grandes empresas y los gigantes tecnológicos manejan volúmenes de datos que se miden en petabytes y exabytes.

Contexto y escala: Lo que se considera Big Data en el mundo científico también es relativo a cada empresa. Para una pequeña organización, procesar cientos de gigabytes podría ser un desafío y, por lo tanto, considerarse "Big Data". Para una empresa como Microsoft, el Big Data se mide en petabytes y exabytes.

Un anuncio de Microsoft de mayo de 2025 reveló que Visual Studio y Visual Studio Code tienen conjuntamente 50 millones de desarrolladores activos cada mes. La mayoría de los análisis indican que una gran parte de esta cifra corresponde a Visual Studio Code. Cada vez que Microsoft actualiza VS Code, actualiza millones de dispositivos. El tamaño promedio de una actualización de Visual Studio Code suele estar entre 50 MB y 100 MB.

Ejemplo 57gt2. Convertir una lista de temperaturas Celsius a temperaturas Fahrenheit y visualizar los resultados con un gráfico.

Objetivo: Crear gráficos con la librería **Matplotlib** después de convertir temperaturas.

Grabe su programa (vacío) con el nombre “ejemplo57gt2.py” en su ordenador en la carpeta “Mis Documentos”.

Para visualizar gráficos de temperaturas en Celsius y Fahrenheit, podemos usar la biblioteca **Matplotlib**, que funciona correctamente en paralelo con **NumPy**. Utilice el siguiente ejemplo para ver cómo funcionan las librerías mencionadas (recuerde que antes necesita instalar en su sistema la librería gráfica mediante la orden: **pip install matplotlib**):

```
import numpy as np
import matplotlib.pyplot as plt

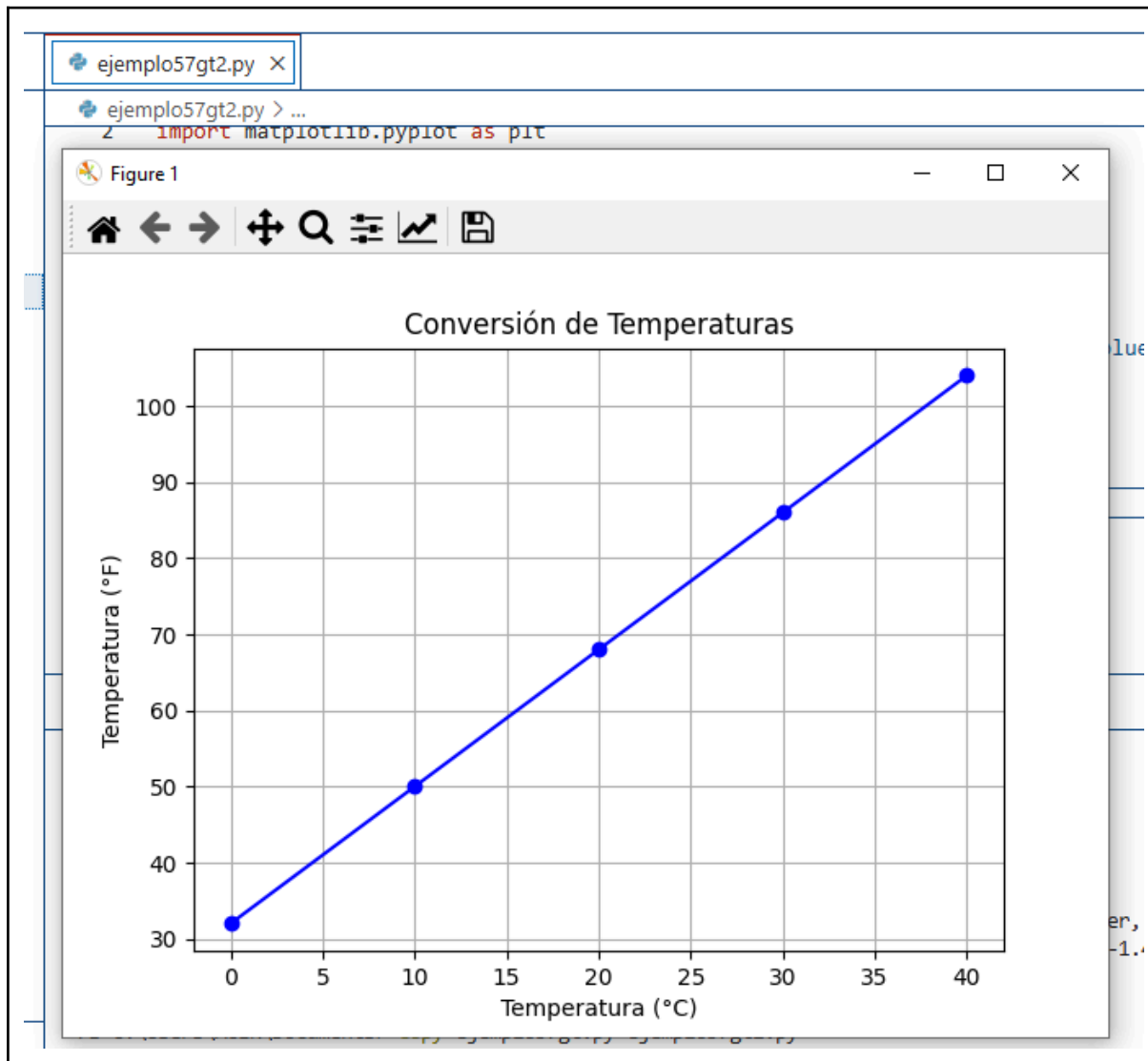
# Datos
temperaturas_celsius = np.array([0, 10, 20, 30, 40])
temperaturas_fahrenheit = (temperaturas_celsius * 9/5) + 32

# Crear gráfico
plt.plot(temperaturas_celsius, temperaturas_fahrenheit, marker='o', color='blue')
plt.title('Conversión de Temperaturas')
plt.xlabel('Temperatura (°C)')
plt.ylabel('Temperatura (°F)')
plt.grid(True)
plt.show()
```

Importante

<code>plt.plot(...)</code>	Crea una línea conectando los puntos.
<code>marker='o'</code>	Dibuja un círculo en cada punto.
<code>plt.grid(True)</code>	Añade una cuadrícula para mejor visualización.
<code>plt.show()</code>	Muestra el gráfico en pantalla.

Al ejecutar el programa anterior, podremos ver el siguiente gráfico:



Ejemplo 57gt3. Crear un gráfico de barras a partir de los datos de las ventas de motos en España entre 2021 y 2025.

Objetivo: Crear gráficos con la librería **Matplotlib** después de verificar los datos en Internet..

Grabe su programa (vacío) con el nombre “ejemplo57gt3.py” en su ordenador en la carpeta “Mis Documentos”.

Para visualizar gráficos podemos usar la biblioteca **Matplotlib**. Utilice el siguiente ejemplo para ver cómo funciona:

```
import matplotlib.pyplot as plt

años = ['2021', '2022', '2023', '2024', '2025']
ventas = [197000, 218000, 230000, 247927, 250700]

fig, ax = plt.subplots(figsize=(8, 5))
barras = ax.bar(años, ventas, color='#58D68D', edgecolor='#145A32')

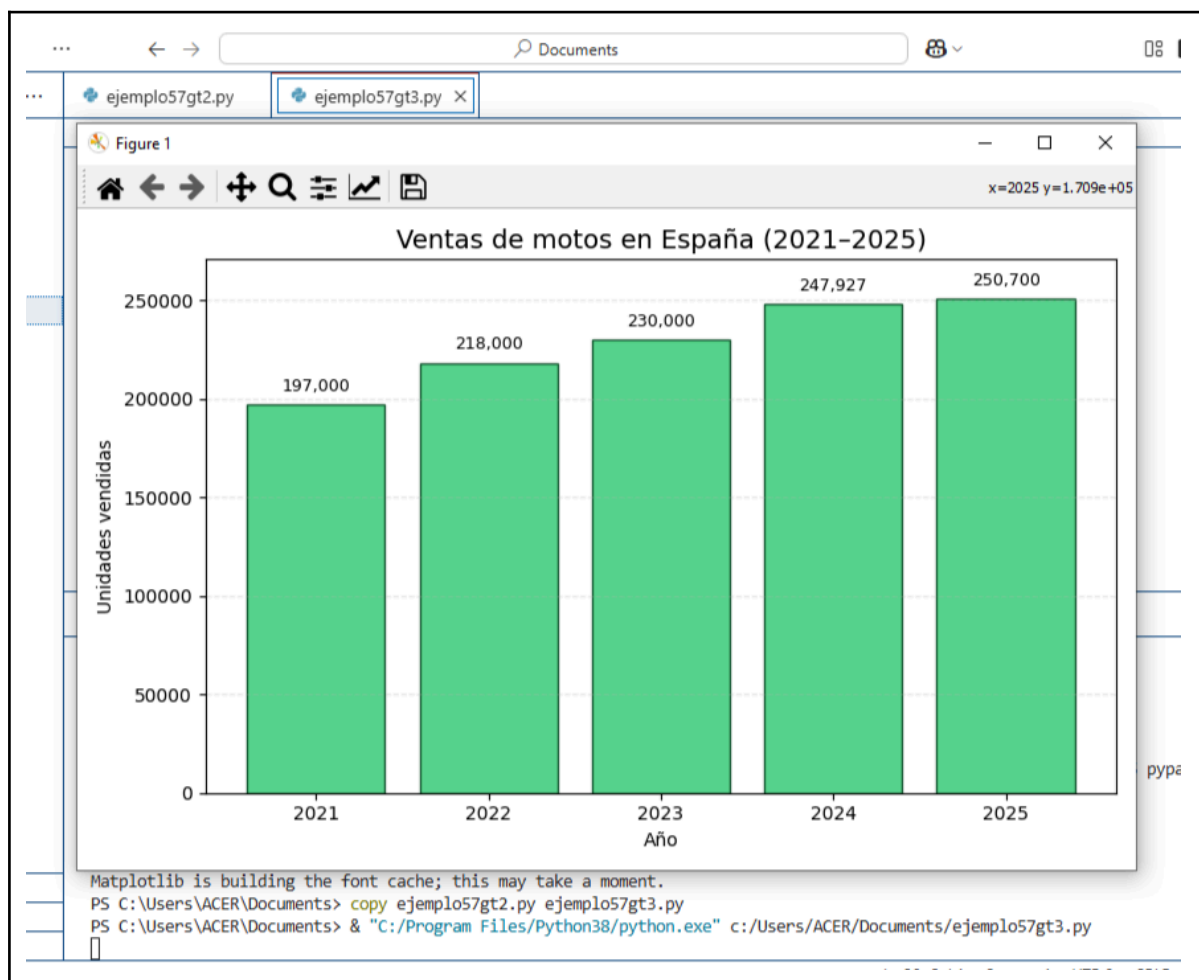
# Títulos y ejes
ax.set_title('Ventas de motos en España (2021-2025)', fontsize=14)
ax.set_xlabel('Año')
ax.set_ylabel('Unidades vendidas')

ax.set_ylim(0, max(ventas) + 20000)          # Límites y rejilla
ax.grid(axis='y', linestyle='--', alpha=0.3)

# Etiquetas de valor encima de cada barra
for barra in barras:
    altura = barra.get_height()
    ax.text(
        barra.get_x() + barra.get_width() / 2,
        altura + 5000,
        f'{altura:,}',
        ha='center', va='bottom', fontsize=9
    )

plt.tight_layout()
plt.show()
```

Al ejecutar el programa anterior, podremos ver el siguiente gráfico:



Ejemplo 58a. Creación de un test sobre criptomonedas utilizando botones de la librería flet mostrando la respuesta correcta.

Objetivo: Entender e implementar el funcionamiento de los diferentes tipos de botones de la librería flet añadiendo sentencias **if**.

Introducción:

El nuevo dinero digital, bitcoin, nació en 2008. En esa fecha, una persona o grupo de personas bajo el seudónimo de Satoshi Nakamoto publicó el famoso “white paper” (libro blanco) que describía su funcionamiento.

Este revolucionario dinero digital no está controlado por ningún gobierno o banco. Es como una moneda virtual que se puede usar para comprar cosas en línea o enviar este dinero digital a otras personas. Su registro se realiza a través de una base de datos cifrada que está en Internet. Esta base de datos se basa en una tecnología especial llamada “blockchain” (cadena de bloques) que registra todas las transacciones de forma segura y transparente.

En febrero de 2011 el bitcoin alcanzó el valor de 1 dólar. En noviembre de 2013 ese valor llegó a los 1,000 dólares y en diciembre de 2024 se pudo ver cómo llegaba a los 100,000 dólares. Días después alcanzó 106,000 dólares. Las criptomonedas se han vuelto populares en los últimos años. Se estima que alrededor de 500 millones de personas en el mundo ya utilizan alguna criptomoneda.

Grabe su programa (vacío, por ahora) con el nombre “test158_flet.py” en su ordenador en la carpeta “Mis Documentos”.

Ahora que ya tiene creado el archivo, añade las siguientes órdenes:

```
import flet as ft

def main(page):
    page.title = "Python 3"
    page.window.width=640
    page.window.height = 480

    def button_clicked(e):
        tx.value=""
        tz.value = f"Su respuesta fue: {cg.value}"
        if cg.value=="bitcoin":
            tx.value= f"Es correcta."
        else:
            tx.value= f"Es incorrecta."
        #end if
        page.update()

    tx= ft.Text()
    tz =ft.Text()
    b = ft.ElevatedButton(text='Comprobar', on_click=button_clicked)
    cg = ft.RadioGroup(content=ft.Column([
        ft.Radio(value="bitcoin", label="bitcoin"),
        ft.Radio(value="botcoin", label="botcoin"),
        ft.Radio(value="batman", label="batman")]))

    page.add(ft.Text("¿Qué criptomoneda es utilizada en el mundo por más de 500 millones de
personas?"),cg,b,tz,tx)

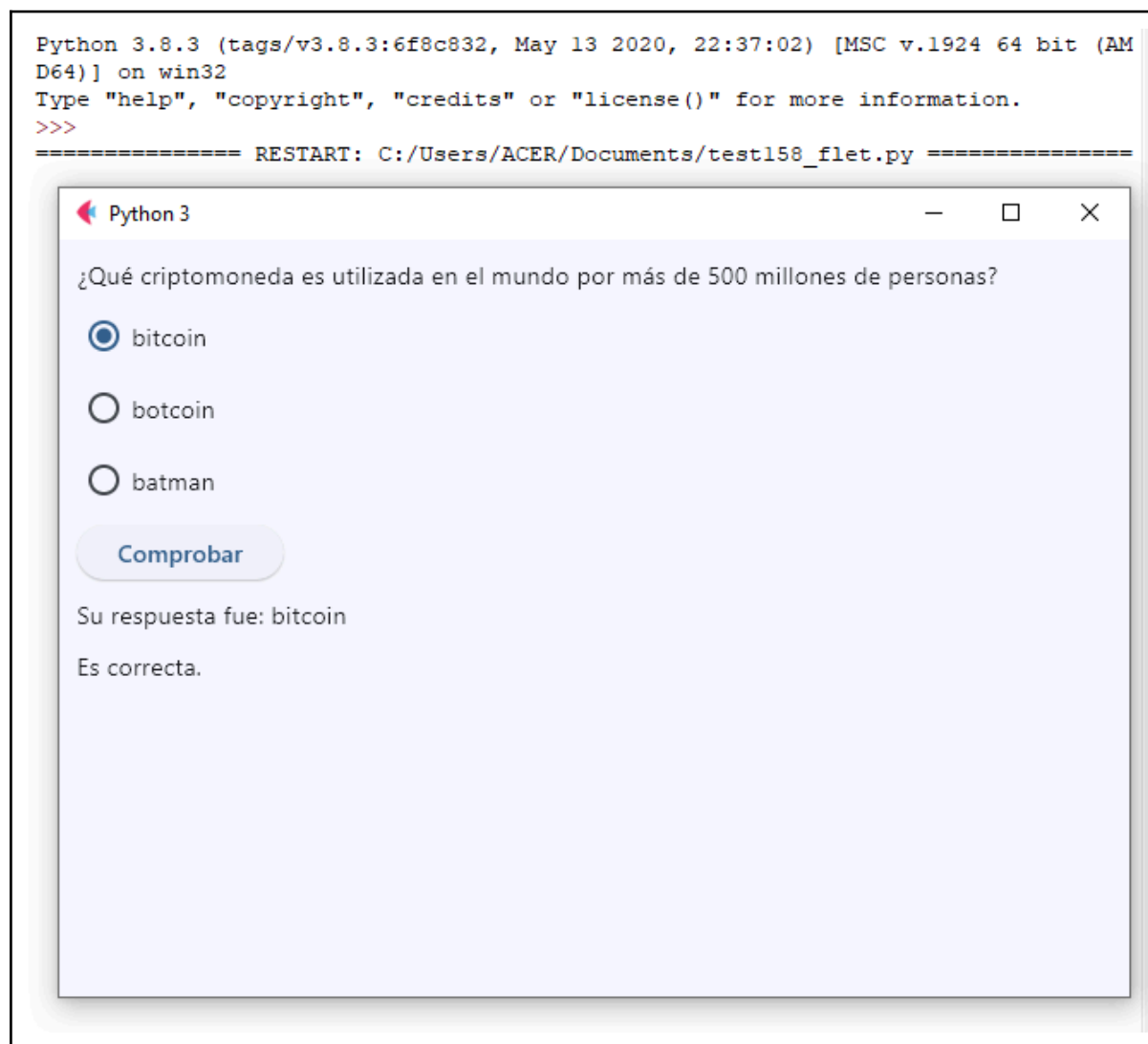
ft.app(main)
```

Como puede observar, debe completar la penúltima línea del código (también puede dividir la cadena en dos, tres o más partes como se muestra en el ejemplo de la página siguiente):

Solución:

```
page.add(ft.Text("¿Qué criptomoneda es utilizada en el mundo por más de 500
millones de personas?"),cg,b,tz,tx)
```

Este es el resultado al ejecutar el código corregido del ejemplo “test158_flet.py”:



Nota:

No invierta más de lo que pueda permitirse perder. Las criptomonedas son muy volátiles. Asegúrese de que su inversión no afecte su estabilidad financiera. Este ejemplo de programación en Python 3.8 no es un consejo financiero, únicamente pretende ilustrar los diferentes usos del lenguaje de programación Python.

Reto:

Cree un test sobre rentabilidad financiera utilizando botones de la librería flet mostrando la respuesta correcta.

Ejemplo 58b. Creación de un test sobre Big Data, criptomonedas y la tecnología “blockchain” (cadena de bloques).

Objetivo: Entender e implementar un test relacionando el mundo Big Data y las criptomonedas.

```
xname = input("Dime tu nombre, por favor: ")
print("Hola, " + xname + "!")

print("Vamos a hacer un test sobre Microsoft Windows, Big data y tecnología Blockchain.")

puntos=0
x = input("La criptomoneda más usada en el mundo en 2025 fue Bitcoin (V/F)? ")

if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

xx = input("La criptomoneda más usada en el mundo en 2025 fue Pepe (V/F)? ")

if xx=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

xxx = input("La cadena de bloques (blockchain) se puede considerar un libro de contabilidad digital y compartido. (V/F)? ")

if xxx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

xxx = input("La cadena de bloques proporciona un marco seguro para gestionar grandes volúmenes de datos (Big Data). (V/F)? ")

if xxx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

xxx = input("La cadena de bloques proporciona un marco seguro para almacenar grandes volúmenes de datos. (V/F)? ")

if xxx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

xxx = input("El volumen de transacciones máximo de Bitcoin alcanzado en 2025 fue de 90,900 millones de dólares. (V/F)? ")

if xxx=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

eth = input("Ethereum fue la segunda criptomoneda más grande por capitalización de mercado en 2025. (V/F)? ")

if eth=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

### Continúa en la página siguiente. . .
```

```

### continuación. . .

eth = input("Ethereum es además de ser una moneda, una plataforma que permite la creación
de contratos inteligentes. (V/F)? ")

if eth=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

eth = input("Ethereum es además de ser una moneda, una plataforma que permite la creación
de aplicaciones descentralizadas. (V/F)? ")

if eth=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

eth = input("JP Morgan Chase ha desarrollado su propia cadena de bloques llamada Quorum,
que se basa en Ethereum. (V/F)? ")

if eth=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+0.5
#endif

eth = input("HSBC utiliza la tecnología de Ethereum para facilitar transacciones y mejorar
la eficiencia en sus operaciones. (V/F)? ")

if eth=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+0.5
#endif

print("Tu nota final en este test es:")
print(puntos)

## Recuerde: No invierta más de lo que pueda permitirse perder.
## Las criptomonedas son muy volátiles.
## Asegúrese de que su inversión no afecte su estabilidad financiera.
## Este ejemplo de programación en Python 3.8 no es un consejo financiero,
## únicamente pretende ilustrar los diferentes usos del lenguaje de programación Python.

```

Ejemplo 58c. Crear un gráfico de barras a partir de los datos del valor de un Bitcoin entre 2017 y 2021.

Objetivo: Crear gráficos con la librería **Matplotlib** después de verificar los datos en Internet.

Grabe su programa (vacío) con el nombre “ejemplo58c.py” en su ordenador en la carpeta “Mis Documentos”.

Para visualizar gráficos podemos usar la biblioteca **Matplotlib**. Utilice el siguiente ejemplo para ver cómo funciona:

```
import matplotlib.pyplot as plt

años = ['2017', '2018', '2019', '2020', '2021']
precios = [19783, 3194, 7200, 29000, 47000] #Precios aprox de Bitcoin a final de cada año

fig, ax = plt.subplots(figsize=(8, 5))
barras = ax.bar(años, precios, color='#FFC300', edgecolor='#9A6100') ### color llamativo

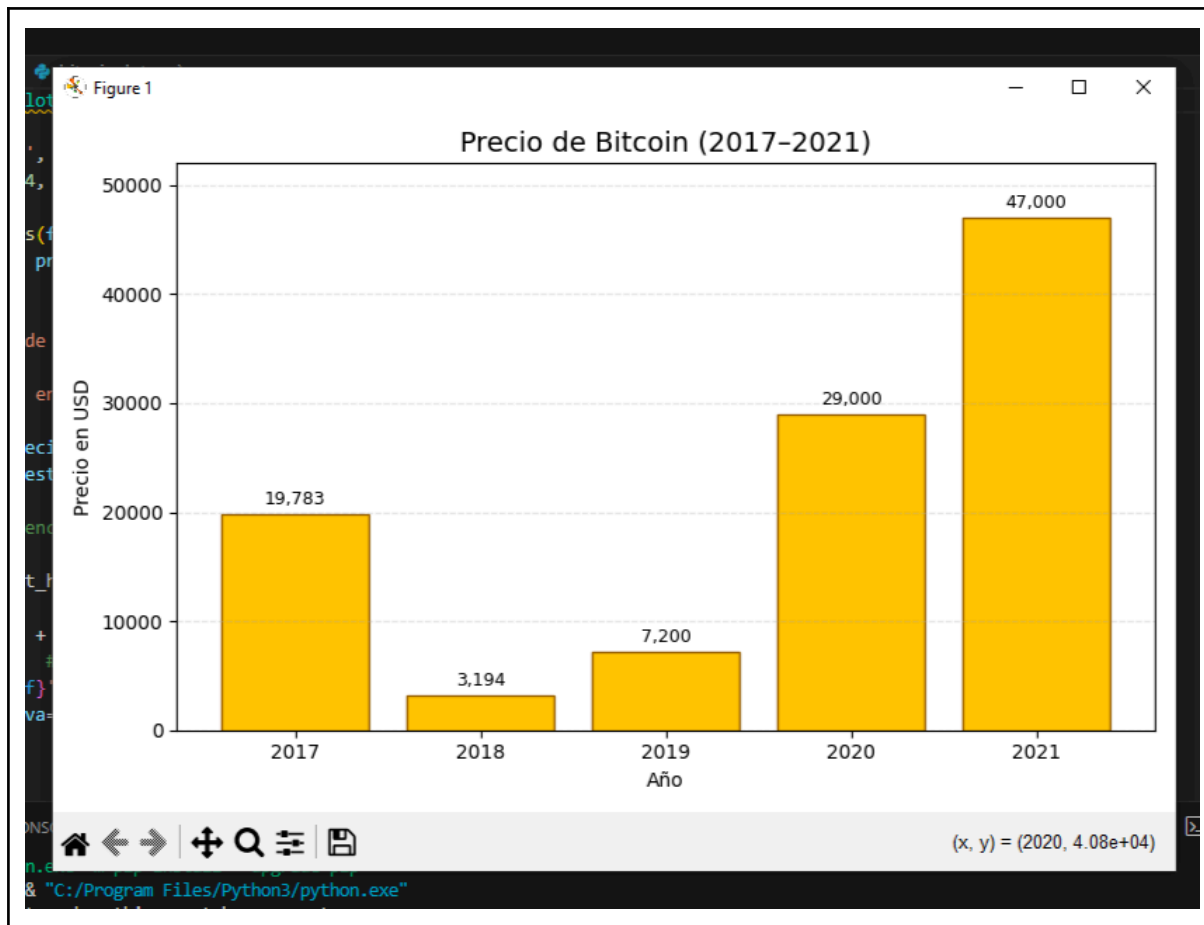
# Títulos y ejes
ax.set_title('Precio de Bitcoin (2017-2021)', fontsize=14)
ax.set_xlabel('Año')
ax.set_ylabel('Precio en USD')

ax.set_ylim(0, max(precios) + 5000) # Ajustamos el límite del eje y
ax.grid(axis='y', linestyle='--', alpha=0.3)

# Etiquetas de valor encima de cada barra
for barra in barras:
    altura = barra.get_height()
    ax.text(
        barra.get_x() + barra.get_width() / 2,
        altura + 500, # Ajustamos la posición de la etiqueta
        f'{altura:,.0f}',
        ha='center', va='bottom', fontsize=9
    )

plt.tight_layout()
plt.show()
```


Al ejecutar el programa anterior, podremos ver el siguiente gráfico:



```
## Recuerde: No invierta más de lo que pueda permitirse perder.
## Las criptomonedas son muy volátiles.
## Asegúrese de que su inversión no afecte su estabilidad financiera.
## Este ejemplo de programación en Python 3.8 no es un consejo financiero,
## únicamente pretende ilustrar los diferentes usos del lenguaje de programación Python.
```

Reto:

Crear un gráfico de barras con la librería **Matplotlib** a partir de los datos del valor de la criptomoneda ether (tecnología Ethereum) entre 2017 y 2021.

Ejemplo 58d. Creación de un test sobre la plataforma Ethereum utilizando botones de la librería flet mostrando la respuesta correcta.

Objetivo: Implementar el funcionamiento de los diferentes tipos de botones de la librería flet añadiendo sentencias **if**.

Introducción:

Ethereum, se lanzó en julio de 2015. En 2013, un joven programador llamado Vitalik Buterin propuso la idea de una plataforma descentralizada que fuera más allá de una simple criptomoneda, publicando el famoso "white paper" (libro blanco) que describía su funcionamiento.

Esta revolucionaria plataforma digital no está controlada por ningún gobierno o banco. Ethereum es más que una moneda virtual; es una plataforma descentralizada que permite la creación de contratos inteligentes (smart contracts) y aplicaciones descentralizadas (dApps). Su registro se realiza a través de una base de datos cifrada que está en Internet. Esta base de datos se basa en una tecnología especial llamada "blockchain" (cadena de bloques), que registra todas las transacciones y la ejecución de los contratos de forma segura y transparente.

En octubre de 2015, la criptomoneda nativa de la red Ethereum, el Ether (ETH), alcanzó un valor aproximado de 1 dólar. En enero de 2018 ese valor llegó a alcanzar los 1,400 dólares y, en noviembre de 2021, se pudo ver cómo llegaba a cerca de 4,800 dólares, su máximo histórico.

Las criptomonedas y las plataformas como Ethereum se han vuelto populares en los últimos años.

Grabe su programa (vacío, por ahora) con el nombre “test158eth_flet.py” en su ordenador en la carpeta “Mis Documentos”.

Ahora que ya tiene creado el archivo, añade las siguientes órdenes:

```
import flet as ft

def main(page):
    page.title = "Python 3"
    page.window.width=640
    page.window.height = 480

    def button_clicked(e):
        tx.value=""
        tz.value = f"Su respuesta fue: {cg.value}"
        if cg.value=="ether":
            tx.value= f"Es correcta."
        else:
            tx.value= f"Es incorrecta."
        #end if
        page.update()

    tx= ft.Text()
    tz =ft.Text()
    b = ft.ElevatedButton(text='Comprobar', on_click=button_clicked)
    cg = ft.RadioGroup(content=ft.Column([
        ft.Radio(value="bitcoin", label="bitcoin"),
        ft.Radio(value="ethereum", label="botcoin"),
        ft.Radio(value="ether", label="batman")]))

    page.add(ft.Text("¿Qué criptomoneda utiliza la red Ethereum?"),cg,b,tz,tx)

ft.app(main)
```

Ejemplo 58e. Crear un test en Python sobre la normativa de blanqueamiento de capitales en España en 2025.

Objetivo: Estudiar la normativa sobre blanqueamiento de capitales en España en 2025 después de verificar los datos en Internet.

```
def test_verdadero_false():
    ## ejemplo58e.py

    preguntas = [
        {
            "pregunta": "Los bancos en España podrán notificar al Servicio Ejecutivo de la Comisión de Prevención del Blanqueo de Capitales (SEPBLAC) si un ciudadano ingresa o retira 3000 euros en efectivo porque la entidad considera que la operación es sospechosa. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Los bancos en España están obligados a notificar a Hacienda si un ciudadano ingresa un billete de 500 euros (Ley 10/2010 de prevención del blanqueo de capitales y financiación del terrorismo). (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Los bancos en España están obligados a notificar a Hacienda si un empresario autónomo acepta un pago en efectivo de más de 1000 euros. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Los bancos en España están obligados a notificar a Hacienda si un ciudadano hace transferencias de más de 10000 euros. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Si un préstamo es entre particulares (por ejemplo, entre familiares o amigos), debe declararse ante Hacienda mediante el modelo 600, ya que puede considerarse una donación si no se documenta correctamente (V/F)",
            "respuesta": "V"
        }
    ]

    puntos = 0
    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print("¡Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuación final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_false()

if __name__ == "__main__":
    test_verdadero_false()
    ##### Ver apéndice H para más información
```

Ejemplo 58f. Crear un test en Python sobre la normativa de blanqueamiento de capitales en España en 2025.

Objetivo: Estudiar la normativa sobre blanqueamiento de capitales en España.

```
def test_verdadero_false():                                     ## ejemplo58f.py

    xx="podrán notificar al Servicio Ejecutivo de la Comisión de Prevención del Blanqueo
de Capitales"
    yy=" (SEPBLAC) si un ciudadano ingresa o retira 3000 euros en efectivo porque la
entidad considera que"
    zz="es sospechosa. (V/F)?"
    aa="modelo 600"
    bb=" puede considerarse una donación si no se documenta correctamente"

    preguntas = [
        {
            "pregunta": "Los bancos en España "+xx+yy+zz,
            "respuesta": "V"
        },
        {
            "pregunta": "Si un préstamo es entre particulares (por ejemplo, entre
familiares o amigos), debe declararse ante Hacienda mediante el "+aa+" debido a que"+bb+"
(V/F)?",
            "respuesta": "V"
        }
    ]

    puntos = 0
    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print(";Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuación final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_false()

if __name__ == "__main__":
    test_verdadero_false()                                     ##### Ver apéndice H para más información
```

Ejemplo 58g. Crear un test en Python sobre la normativa de blanqueamiento de capitales en España en 2025.

Objetivo: Estudiar la normativa sobre blanqueamiento de capitales.

```
def test_verdadero_false():                                     ## ejemplo58g.py
    modelo="modelo 600"
    razon=" puede considerarse una donación si no se documenta correctamente"
    mil="1000 euros"

    preguntas = [
        {
            "pregunta": "Los bancos en España podrán notificar al Servicio Ejecutivo de
la Comisión de Prevención del Blanqueo de Capitales (SEPBLAC) si un ciudadano ingresa o
retira 3000 euros en efectivo porque la entidad considera que la operación es sospechosa.
(V/F)?",
            "respuesta": "V"
        },
        {
            "pregunta": "Los bancos en España están obligados a notificar a Hacienda si
un ciudadano ingresa un billete de 500 euros (Ley 10/2010 de prevención del blanqueo de
capitales y financiación del terrorismo). (V/F)?",
            "respuesta": "V"
        },
        {
            "pregunta": "Los bancos en España están obligados a notificar a Hacienda si
un empresario autónomo acepta un pago en efectivo de más de " + mil + " (V/F)?",
            "respuesta": "V"
        },
        {
            "pregunta": "Si un préstamo es entre particulares (por ejemplo, entre
familiares o amigos), debe declararse ante Hacienda mediante el "+modelo+" debido a
que"+razon+" (V/F)?",
            "respuesta": "V"
        }
    ]

    puntos = 0
    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print("¡Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuación final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_false()
if __name__ == "__main__":
    test_verdadero_false()                                     ##### Ver apéndice H para más información
```

Ejemplo 59. Juego de los dados con super premios.

Objetivo: Utilizando la librería **random**, generar números aleatorios entre el 1 y el 6 para simular un juego de dados. Según la puntuación obtenida, simular la entrega de regalos extraordinarios.

Cree el archivo: “**superpremios.py**” y añada las siguientes órdenes:

```
##### Programa superpremios.py
import random
dice = random.Random()

roll1 = dice.randint(1, 6)
roll2 = dice.randint(1, 6)
roll3 = dice.randint(1, 6)

total = roll1 + roll2 + roll3
print("Juego de Loteria y dados.")
print(f"Al tirar el dado 1, obtuve: +{roll1}")
print(f"Al tirar el dado 2, obtuve: +{roll2}")
print(f"Al tirar el dado 3, obtuve: +{roll3}")
print(f"Puntos totales: {roll1} + {roll2} + {roll3} = {total}")
print(" ")

if (roll1 == roll2) or (roll2 == roll3) or (roll1 == roll3):
    if (roll1 == roll2) and (roll2 == roll3):
        print("Has hecho un triple! +6 bonus se suma al total!")
        total += 6
    else:
        print("Has hecho un doble! +2 bonus se suma al total!")
        total += 2

    print(f"Puntuación final (bonus incluido): {total}")

if total >= 16:
    print("Has ganado un super coche Honda Civic Type R.")
elif total >= 10:
    print("Has ganado una tablet!")
elif total == 7:
    print("Has ganado un viaje a Alaska para dos personas!")
else:
    print("Has ganado un ratón nuevo para tu PC!")
```

Reto 1:

Modifica el script para que si el jugador gana un coche, el sistema elija de manera aleatoria un elemento de un array llamado **ArrayPremios**. Los elementos del array serán: Ferrari F40, Porsche 911 y Maserati Bora.

Solución:

```
import random
# Define matriz (array) para supercars
ArrayPremios = ["Tesla", "Porsche", "Maserati"]

dice = random.Random()
roll1 = dice.randint(1, 6); roll2 = dice.randint(1, 6); roll3 = dice.randint(1, 6)
total = roll1 + roll2 + roll3

print("Juego de Loteria con tres dados.")
print(f"Al tirar el dado 1, obtuve: +{roll1}")
print(f"Al tirar el dado 2, obtuve: +{roll2}")
print(f"Al tirar el dado 3, obtuve: +{roll3}")
print(f"Puntos totales: {roll1} + {roll2} + {roll3} = {total}")
print(" ")

if (roll1 == roll2) or (roll2 == roll3) or (roll1 == roll3):
    if (roll1 == roll2) and (roll2 == roll3):
        print("Has hecho un triple! +6 bonus se suma al total!")
        total += 6
    else:
        print("Has hecho un doble! +2 bonus se suma al total!")
        total += 2

    print(f"Puntuación final (bonus incluido): {total}")

if total >= 16:
    # Genera un num random entre 0 and 2
    car_index = random.randint(0, len(ArrayPremios) - 1)
    # Encuentra el premio del car using car_index
    won_car = ArrayPremios[car_index]
    print(f"¡Felicitaciones! ¡Has ganado un super coche marca {won_car}!")
elif total >= 10:
    print("Has ganado una tablet!")
elif total == 7:
    print("Has ganado un viaje a Alaska para dos personas!")
else:
    print("Has ganado un ratón nuevo para tu PC!")
```

Reto 2:

Modifica el script anterior: Si el jugador gana un coche, el sistema elegirá de manera aleatoria un elemento de un array llamado **ArrayPremios**. Los elementos del array serán: Ferrari F40, Porsche 911 y Renault 5 Copa Turbo.

Además, de manera aleatoria, se escogerá un extra para el premio entre los siguientes 13 elementos: Climatizador (1), Asientos de cuero (2), Asientos en tela vaquera (3), volante de cuero estándar (4), volante deportivo (5), volante con mandos para la radio (6), navegador GPS Garmin (7), navegador GPS estándar (8), llantas aleación ligera (9), discos de freno delanteros Brembo (10), discos de freno traseros Brembo (11), cambiador de CDs en maletero (12), toma USB delantera y trasera (13).

Para analizar los extras que eligen los compradores potenciales de una marca como **BMW**, **Honda** o **Porsche**, en un sistema de **Big Data**, se pueden seguir varios pasos clave (este enfoque no solo ayuda a entender las preferencias de los clientes, sino que también permite a las empresas innovar y adaptar su oferta):

1. **Recolección de Datos:**

Datos de ventas: Recopilar información sobre las ventas de vehículos, incluyendo los extras seleccionados por los compradores.

Datos de comportamiento: Analizar datos de navegación en el sitio web de Honda o Porsche, donde los clientes potenciales configuran sus vehículos.

Encuestas y feedback: Realizar encuestas a clientes para entender sus preferencias y motivaciones.

2. **Almacenamiento de Datos:**

Utilizar un **lago de datos** (data lake) para almacenar grandes volúmenes de datos estructurados y no estructurados. BMW, por ejemplo, utiliza **AWS (Amazon Web Services)** para procesar hasta **10 TB** de datos diariamente.

3. **Análisis de Datos:**

Análisis descriptivo: Identificar patrones en los datos de ventas y preferencias de extras.

Análisis predictivo: Utilizar **inteligencia artificial** y **aprendizaje automático** para predecir qué extras son más propensos a ser elegidos por diferentes segmentos de clientes.

4. Visualización de Datos:

Crear gráficos interactivos (dashboards) que muestren las tendencias de elección de extras, permitiendo a los analistas y a la dirección tomar decisiones informadas.

5. Optimización de Ofertas:

Basado en el análisis, ajustar las ofertas de extras y personalizar las campañas de marketing para atraer a los compradores potenciales.

Ejemplo 60. Juego de los dados con premios extraordinarios.

Objetivo: Utilizando la librería **random**, generar números aleatorios entre el 1 y el 6 para simular un juego de dados. Según la puntuación obtenida, simular la entrega de regalos extraordinarios.

Si el jugador gana un ratón para PC, el sistema debe elegir de manera aleatoria un elemento de un array llamado **ArrayPremios**. Los 3 elementos del array serán: Ratón Logitech, Ratón Acer y Ratón IBM.

Cree el archivo: “**superMice.py**” y añada las siguientes órdenes:

```
import random
dice = random.Random()

ArrayPremios = ["Ratón Logitech", "Ratón Acer", "Ratón IBM"]

roll1 = dice.randint(1, 6); roll2 = dice.randint(1, 6); roll3 = dice.randint(1, 6)

total = roll1 + roll2 + roll3
print("Juego de Loteria y dados.")
print(f"Al tirar el dado 1, obtuve: +{roll1}")
print(f"Al tirar el dado 2, obtuve: +{roll2}")
print(f"Al tirar el dado 3, obtuve: +{roll3}")
print(f"Puntos totales: {roll1} + {roll2} + {roll3} = {total}")

if (roll1 == roll2) or (roll2 == roll3) or (roll1 == roll3):
    if (roll1 == roll2) and (roll2 == roll3):
        print("Has hecho un triple! +6 bonus se suma al total!")
        total += 6
    else:
        print("Has hecho un doble! +2 bonus se suma al total!"); total += 2

    print(f"Puntuación final (bonus incluido): {total}")

if total >= 16:
    print("Has ganado un super coche Honda Civic Type R.")
elif total >= 10:
    print("Has ganado una tablet!")
elif total == 7:
    print("Has ganado un viaje a Alaska para dos personas!")
else:
    ndx = random.randint(0, len(ArrayPremios) - 1)
    txt = ArrayPremios[ndx]          ### Encuentra el premio using ndx (index)
    print(f"¡Felicidades! ¡Has ganado este dispositivo:{txt}!")
```

Reto:

Modifica el script anterior para que el programa elija aleatoriamente entre tres ciudades de Alaska: Anchorage, Fairbanks o Sitka.

PARTE 2

BASES DE DATOS con PYTHON en
ENTORNOS MULTIUSUARIO

¿Qué es una base de datos?

Una base de datos es un conjunto organizado de datos que se almacena en un servidor, generalmente en una ubicación remota, y se maneja electrónicamente, usualmente con un ordenador. Si puedes imaginar una gran hoja de cálculo donde puedes guardar y acceder fácilmente a información importante, como nombres, fechas o números, ya puedes tener una idea básica de lo que es una base de datos.

Las bases de datos se utilizan en diferentes aplicaciones en redes de ordenadores de millones de empresas en el mundo para mantener la información de los clientes ordenada y accesible. Pequeñas empresas como el supermercado donde realizamos la compra cada semana o grandes empresas como Amazon, Microsoft o Google serían difícilmente exitosas sin una buena gestión y organización de sus bases de datos.

Después de más de diez años programando bases de datos para numerosas empresas privadas y organismos públicos en diversos lenguajes como C#, Microsoft FoxPro, Microsoft Visual Basic para Access o Python, descubrí LiveCode. Este lenguaje de programación de alto nivel, que ha evolucionado desde su versión inicial llamada HyperCard lanzada en 2001, me sorprendió gratamente. LiveCode permite compilar código y generar archivos ejecutables, además de conectarse con bases de datos SQL. Los resultados de mi trabajo con LiveCode y SQL los presentaré en un próximo libro.

Encontré en los lenguajes SQL, C# y Python una forma más rápida y sencilla de programar bases de datos, gracias a sus numerosas librerías gratuitas. Crear una guía con ejemplos se hizo realidad con este libro.

Descripción del lenguaje SQL

En el entorno de las redes locales, el lenguaje SQL facilita que numerosos usuarios accedan de forma simultánea a la información almacenada en bases de datos relacionales.

En la actualidad, LibreOffice, Microsoft Office, Euro-Office y la mayoría de los paquetes o aplicaciones que manejan sistemas relacionales de bases de datos en red, incorporan ya este lenguaje.

Los diferentes mandatos de SQL, se pueden utilizar junto al lenguaje de programación de un gestor de base de datos (como Microsoft Access o LibreOffice Base) para definir las diferentes tablas que forman una base de datos y acceder a los datos previamente almacenados.

La terminología usada por el lenguaje SQL puede variar ligeramente de la usada en Microsoft Access 2000, Microsoft Access 2007, Microsoft SQL Server, SQLite, PostgreSQL o LibreOffice Base.

Para SQL una fila es el equivalente a un registro de Microsoft Access 97 (Microsoft Access 2000 o Microsoft Access 2007).

Por otro lado, una columna, es un campo del registro.

Y, por último, un dato es la intersección de una fila y una columna.

Tipos de datos

Existen 3 grandes tipos de datos: numéricos, alfanuméricos y de tiempo.

Los datos numéricos representan cantidades numéricas que pueden participar en cálculos aritméticos. Los datos alfanuméricos representan combinaciones de caracteres.

Los datos de tiempo representan magnitudes temporales; años, días, meses, etc.

SQL tiene una serie de palabras reservadas que definen los diferentes tipos de datos, a continuación, se puede observar una lista con una descripción de los mismos:

1. SMALLINT

Se utiliza para datos numéricos enteros de 6 dígitos.

2. INTEGER

Se utiliza para datos numéricos enteros de 11 dígitos.

3. DECIMAL (p,s)

Se utiliza para datos numéricos decimales: donde p es el número total de dígitos (parte entera y parte fraccionaria) que pueden llenar un determinado valor, s es el número de dígitos que puede tener la parte fraccionaria.

4. FLOAT o REAL

Se utiliza para datos numéricos en coma flotante.

5. CHAR (n)

Alfanumérico de longitud fija; donde n es la longitud de la hilera de caracteres. Puede contener desde 1 hasta 254 caracteres.

6. VARCHAR (n)

Alfanumérico de longitud variable, donde n es la longitud.

7. DATE

Para datos de tiempo. Contiene una fecha. El formato por omisión es DDMMAA.

8. LOGICAL ó BOOLEAN

Para datos lógicos. Contienen valores lógicos; verdadero o falso, representado por los valores .T. o .F.

Operaciones con tablas

En un sistema de base de datos relacional como LibreOffice Base, Microsoft Access, Microsoft Visual Foxpro, Microsoft SQL o SQLite, la estructura de datos principal que manejan los programadores y los usuarios son las tablas. Estas tablas están formadas por una serie de filas y columnas donde se almacenan diferentes datos.

El significado de cada dato en una tabla depende de la fila y la columna que este dato ocupe y el valor que tome.

No existe una información implícita, como el lugar relativo que ocupe un campo o un registro con respecto a otros, como ocurre en un fichero tradicional.

El número de tablas varía según las necesidades de información y el diseño realizado en cada paso.

Pueden existir tablas con pocas columnas y tablas con miles.

SQL utiliza el mandato **CREATE** para definir una tabla. Veamos cómo funciona:

```
CREATE TABLE <nombre tabla>

(<nombre columna.> <tipo de datos>,

[<nombre columna.> <tipo de datos>] );
```

Como se puede ver en la sintaxis del mandato, es necesario dar un nombre a cada columna y decir qué tipo de datos se le van a asignar.

Microsoft Access 2000 y LibreOffice Base disponen de más tipos de campos que el lenguaje estándar SQL. Si define una tabla por medio de SQL, tendrá que adaptar el tipo de los campos.

Por ejemplo, el tipo de campo "memo" no está disponible en SQL, pero puede definirse como un campo de tipo carácter de una longitud elevada.

Cuando se crea una tabla en SQL, tanto LibreOffice como Microsoft Access construyen el correspondiente objeto (tabla) dentro de la base de datos. Es decir, el archivo de base de datos con extensión MDB contendrá una tabla nueva, que será un objeto nuevo. Así, cada tabla nueva será un objeto nuevo con sus correspondientes campos en la base de datos.

Para poder introducir filas en una tabla, el mandato que se debe utilizar es **INSERT**.

La sintaxis de este mandato se describe a continuación:

```
INSERT INTO <nombre tabla>  
[<columnas>] VALUES <valores>
```

Del mismo modo que se pueden insertar filas en una tabla, existe el mandato **DELETE** para borrarlas. De este modo, podremos borrar registros de una tabla. Veamos la sintaxis:

```
DELETE FROM <nombre tabla>  
  
WHERE <condición>;
```

La cláusula **WHERE** determina una condición, que si se cumple, el registro o grupo de registros quedarán borrados de la tabla especificada.

Para destruir una tabla (desaparecerá en la base de datos) se utiliza la sentencia **DROP**. Su sintaxis es:

```
DROP TABLE <nombre tabla>
```

Las consultas sobre tablas se realizan mediante la sentencia SELECT. La sentencia SELECT permite realizar consultas sobre una o varias tablas. Es una de las instrucciones más potentes y más utilizadas de SQL. Su sintaxis en el uso más sencillo es:

```
SELECT <nombre columna> FROM <tabla>
```

ó también:

```
SELECT <nombre columna> FROM <tabla>  
  
[WHERE <condición>]
```

Además, para hacer consultas más complicadas, su sintaxis es:

```
SELECT <nombre columna> FROM <tabla>

[[INTO <destino>] | [TO FILE <.arch nom>] | TO PRINTER

[PREFERENCE <nombre>]

[NO CONSOLE]

[PLAIN]

[NOWAIT]

[PROMPT] | [TO SCREEN]

[WHERE <condición>]

[GROUP BY <nombre columna-agrupamientos>]

[HAVING <filtro-condición>]

[UNION <comando SELECT>]

[ORDER BY <nombre columna>]

[SAVE TO TEMP <cláusula>];
```

El resultado de hacer una consulta realizada con SELECT es siempre otra tabla. Esta nueva tabla contiene los resultados de la búsqueda.

La tabla que se obtiene como resultado de ejecutar el mandato SELECT, se construye siguiendo los siguientes pasos:

1. De entre todas las tablas que existen en la base de datos, se seleccionan las nombradas en la cláusula FROM.
2. A la fila de la tabla seleccionada se les aplica la cláusula WHERE, dejando que satisfagan la condición y suprimiendo todas las demás.
3. De la tabla que resulta se eliminan todas las columnas no mencionadas en la cláusula SELECT. Las demás columnas se ponen en la misma posición en que se hayan escrito en la cláusula SELECT. Pueden repetirse columnas, si su nombre aparece repetido en la cláusula SELECT.
4. La tabla resultante estará ordenada clasificada por los valores de las columnas mencionadas en la cláusula ORDER BY.

El uso de la cláusula WHERE permite establecer una serie de condiciones relativas a las filas que se van a seleccionar.

En la tabla siguiente se muestran los operadores de comparación que se pueden usar:

DESCRIPCIÓN	OPERADOR
Igual a	=
Mayor que	>
Menor que	<
Mayor o igual que	>=
Menor o igual que	<=
Distinto que	<> ó #
Negación	!

Ejemplo 61. Creación de un archivo de datos para almacenar textos.

Objetivo: Utilizar Python para generar un archivo de texto (en formato txt) con datos.

Recordemos en primer lugar algunos aspectos importantes en la lectura o escritura de archivos de texto con Python 3:

- **Flujo de Trabajo:** El proceso completo para escribir en un archivo incluye:
 1. Abrir el archivo con `open()` en modo escritura ('w').
 2. Escribir contenido usando la función `write()`.
 3. Cerrar el archivo con `close()`.

- **Lectura de Archivos:** Podemos leer el contenido completo de un archivo en una variable o imprimirlo directamente usando la función `read()`.

Antes de hacer el ejercicio, recordemos cómo leer y escribir archivos de texto usando las funciones integradas de Python:

1. Abrir un Archivo

Para trabajar con un archivo, primero necesitaremos abrirlo. La función `open()` se usa para este propósito. Los parámetros que le damos a la función `open()` dependen del sistema operativo y del objetivo (leer o escribir).

- Abrir un archivo en Windows:

Las rutas de Windows usan barras invertidas (`\`). Necesitamos usar barras invertidas dobles (`\\`) o una cadena de texto sin formato (raw string) para representar la ruta correctamente.

Usando barras invertidas dobles

```
test_file = open('c:\\myFile.txt')
```

O, usando una cadena de texto sin formato (preferido)

```
test_file = open(r'c:\myFile.txt')
```


2. Escribir en un Archivo

Para escribir en un archivo, debemos abrirlo con un segundo parámetro, la cadena 'w', que significa "write" (escribir). Esto creará un archivo nuevo y vacío o sobrescribirá uno existente.

- Abrir un archivo en modo escritura:

```
test_file = open('c:\\myFile.txt', 'w')
```

El parámetro 'w' le dice a Python que queremos escribir en el archivo, no leerlo.

- Escribir información en el archivo:

Usa la función write() en el objeto del archivo para añadir contenido.

```
test_file = open('c:\\myFile.txt', 'w')
test_file.write('este es mi archivo de prueba')
```

Puedes añadir varias líneas de texto llamando a write() varias veces.

- Cerrar el archivo:

Es crucial cerrar el archivo una vez que hayamos terminado de escribir para guardar los cambios y liberar recursos del sistema. Usaremos la función close().

```
test_file = open('c:\\myFile.txt', 'w')
test_file.write('Qué es rojo, azul y blanco? Una Honda Africa Twin!')
test_file.close()
```

3. Leer un Archivo

Una vez que un archivo es creado y tiene contenido, podremos leerlo. Para hacer esto, abriremos el archivo en el modo "lectura" por defecto (o explícitamente con el parámetro 'r') y usaremos la función read().

- Leer el contenido de un archivo:

```
test_file = open('c:\\myFile.txt') # Sin un segundo parámetro, el modo  
predeterminado es 'r' (read)  
print(test_file.read())
```

La salida será todo el contenido del archivo.

- Cerrar el archivo después de leerlo:
Al igual que al escribir, es una buena práctica cerrar el archivo después de terminar de leerlo.

```
test_file.close()
```

```

import os

def read_line(file_path, line_number):
    """
    ¿Qué vamos a hacer? Leer líneas (con num enteros) de un file txt y devolver el
    contenido

    Args:
        file_path (str): La ruta completa al archivo de texto.
        line_number (int): El número de línea del file txt que se debe leer (empezando en
    0).

    Returns:
        str: El contenido de la línea solicitada, o None si el file no existe

    """
    try:
        # 'with open' se asegura de que el file se cierre automáticamente
        with open(file_path, 'r', encoding='utf-8') as file:
            lines = file.readlines()
            # Comprobar si el número de línea es válido
            if 0 <= line_number < len(lines):
                # .strip() elimina espacios en blanco y saltos de línea (\n) al final
                return lines[line_number].strip()
            else:
                print(f"Error: El número de línea {line_number} está fuera de rango.")
                return None
    except FileNotFoundError:
        print(f"Error: No se encontró el archivo en la ruta: {file_path}")
        return None

# --- Bloque principal . Main program
if __name__ == "__main__":
    print(";Hola! Vamos a leer un fichero txt!")

    file_name="names.txt"    ## El file names.txt debe ser creado previamente a la lectura

    # Construye la ruta al archivo
    # Se asume que 'names.txt' está en la misma carpeta que este script
    # os.path.dirname(__file__) obtiene la ruta del directorio del script actual.
    script_directory = os.path.dirname(__file__)
    my_file_path = os.path.join(script_directory, file_name)

    print(f"Intentando leer el archivo: {my_file_path}")

    # Llama a la función para leer la primera y segunda pregunta (líneas 0 y 1)
    q1 = read_line(my_file_path, 0)
    q2 = read_line(my_file_path, 1)

    # Imprimir
    if q1 is not None:
        print(f"Pregunta 1: {q1}")

    if q2 is not None:
        print(f"Pregunta 2: {q2}")

    #¿Qué correcciones debe completar en el script para hacer la suma de los datos de q1+q2?

    #¿Qué diferencia existe entre un fichero de datos .txt y un fichero de datos .sqlite?

```

Ejemplo 62a. Creación de un archivo de datos utilizando instrucciones SQL con Python 3.

Objetivo: Utilizar Python para generar un archivo con formato SQLite con datos para realizar la lectura y escritura de datos en bases de datos con este formato.

Recordemos en primer lugar algunos aspectos importantes en la lectura o escritura de bases de datos SQLite con Python 3:

Leer y Escribir Datos con SQLite

¿Cómo crearemos, conectaremos y manipularemos una base de datos usando el módulo `sqlite3` de Python 3?

SQLite es un motor de base de datos muy popular que utiliza instrucciones del lenguaje de base de datos SQL estándar.

SQLite guarda la información de una o más tablas con datos en un solo archivo, lo que lo hace perfecto para aplicaciones multiusuario pequeñas y medianas sin necesidad de un servidor dedicado como Microsoft SQL Server.

El límite teórico de SQLite es de 140 terabytes (TB). En la práctica, la cantidad de registros que podemos manejar de forma eficiente depende de los recursos de la máquina, como la memoria RAM y el espacio en disco, así como del diseño de la base de datos y las consultas (queries).

El primer paso es conectarse a la base de datos. Esto se logra usando la función `connect()` del módulo `sqlite3`. Si el archivo de la base de datos no existe, Python lo creará automáticamente.

1. Importar el módulo `sqlite3` y conectar la base de datos `sqlite`:

```
import sqlite3

# Conectar a una base de datos llamada 'mi_base_de_datosXX.db'
# Si el archivo no existe, será creado.
conexion = sqlite3.connect('mi_base_de_datosXX.db')
```

2. Crear un Cursor

Una vez conectado, necesitas un objeto "cursor" para ejecutar comandos SQL. Un cursor es como un puntero que te permite interactuar con la base de datos.

```
cursor = conexion.cursor()
```

3. Escribir Datos (Insertar Registros)

Para escribir datos, usaremos el comando INSERT. Primero, crearemos una tabla para almacenar los datos. Luego, insertaremos registros en la tabla y guardaremos los cambios.

Crear una tabla:

```
#
# Usamos execute() para ejecutar un comando SQL
# La instrucción CREATE TABLE define la estructura de la tabla

cursor.execute('''
    CREATE TABLE IF NOT EXISTS usuarios (
        id INTEGER PRIMARY KEY,
        nombre TEXT NOT NULL,
        edad INTEGER
    )
''')
```

Nota: La cláusula IF NOT EXISTS provoca que no se produzca un error si la tabla no existe.

Insertar un registro en la tabla creada:

```
cursor.execute("INSERT INTO usuarios (nombre, edad) VALUES (?, ?)", ('Ana',
33))

conexion.commit()      ## Para que los cambios sean permanentes, usaremos
                        ## el método commit().
```

Nota: Utilizamos los signos de interrogación (?) y pasamos los valores como una tupla. Es la forma más utilizada para evitar inyecciones de SQL.

4. Leer Datos (Seleccionar Registros)

Para que los usuarios de la red local puedan leer datos, utilizaremos un comando SQL de SELECT. Después de ejecutar la consulta, el cursor contendrá los resultados.

```
cursor.execute("SELECT nombre, edad FROM usuarios")

# Obtener todos los resultados de la consulta
registros = cursor.fetchall()

# Imprimir los resultados
print("Registros de la tabla de usuarios:")
for registro in registros:
    print(registro) # Esto imprimirá una tupla como ('Juan', 30)
```

5. Cerrar la Conexión

Una vez que hayamos terminado de trabajar con la base de datos, es una buena práctica cerrar la conexión para liberar recursos.

```
conexion.close()
```

Importante: Si utilizamos la declaración **with**, automáticamente cerrará la conexión por nosotros, incluso si ocurre un error.

```
import sqlite3
with sqlite3.connect('mi_base_de_datos.db') as conexion:
    cursor = conexion.cursor()
    ## Aquí podemos ejecutar todas las operaciones SQL
    ## La conexión se cerrará automáticamente al salir del bloque 'with'
```

Solución del Ejemplo 62a:

```
import sqlite3                                     ## Ejemplo62a.py

# Paso 1: Conectar a la base de datos
try:
    with sqlite3.connect('mi_base_de_datos.db') as conexion:
        # Paso 2: Crear un cursor
        cursor = conexion.cursor()

        # Paso 3: Crear una tabla (si no existe) e insertar datos
        print(" ")
        print("Creando tabla e insertando un nuevo registro...")
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS usuarios (
                id INTEGER PRIMARY KEY,
                nombre TEXT NOT NULL,
                edad INTEGER
            )
        ''')
        cursor.execute("INSERT INTO usuarios (nombre, edad) VALUES (?, ?)", ('Ana', 30))
        print("Registro insertado exitosamente.")

        # Ojo: No necesitas conexion.commit() dentro del 'with',
        # ya que la transacción se confirma automáticamente si todo sale bien.

        # Paso 4: Leer los datos
        print("\nLeyendo todos los registros de la tabla...")
        cursor.execute("SELECT id, nombre, edad FROM usuarios")
        registros = cursor.fetchall()

        for registro in registros:
            print(f"ID: {registro[0]}, Nombre: {registro[1]}, Edad: {registro[2]}")

except sqlite3.Error as e:
    print(f"Ocurrió un error con la base de datos: {e}")
finally:
    print("\nProceso terminado.")
```


Al ejecutar el programa anterior, podremos ver el siguiente resultado:

```
24         # OJO: NO NECESITAS CONEXION.COMMIT() DENTRO DEL WITH ,
25         # ya que la transacción se confirma automáticamente si todo sale bien.
26
27         # Paso 4: Leer los datos
28         print("\nLeyendo todos los registros de la tabla...")
29         cursor.execute("SELECT id, nombre, edad FROM usuarios")
30         registros = cursor.fetchall()
31
32         for registro in registros:
33             print(f"ID: {registro[0]}, Nombre: {registro[1]}, Edad: {registro[2]}")
34
35     except sqlite3.Error as e:
36         print(f"Ocurrió un error con la base de datos: {e}")
37     finally:
38         print("\nProceso terminado.")
39
40
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
PS C:\Users\ACER\Documents> & "C:/Program Files/Python38/python.exe" c:/Users/ACER/Documents/ejemplo62.py Creando tabla e insertando un nuevo registro... Registro insertado exitosamente. Leyendo todos los registros de la tabla... ID: 1, Nombre: Ana, Edad: 30 Proceso terminado.				

Observe que se ha leído el registro insertado en la tabla e impreso en pantalla el nombre (Ana) y la edad (30) del registro cuyo ID es 1.

Reto 1:

Modifique el código del ejemplo 62a para crear las tablas de Autores y Libros en su base de datos. Cree un registro en cada tabla.

	Field Name	Field Type	
¶	DNI	Text [VARCHAR]	DNI del autor
	Nombre	Text [VARCHAR]	Nombre del autor
	Apellidos	Text [VARCHAR]	Apellidos del autor
	Direccion	Text [VARCHAR]	Dirección
	Telefono	Text [VARCHAR]	Teléfono
▶	Ciudad	Text [VARCHAR]	Ciudad donde reside el autor

	Field Name	Field Type	
▶¶	ISBN	Text [VARCHAR]	INTERNATIONAL SYSTEM BOOK NUMBER
	Titulo	Text [VARCHAR]	TITULO DEL LIBRO
	Genero	Text [VARCHAR]	GENERO
	Paginas	Text [VARCHAR]	NUMERO DE PAGINAS
	Nombre del Autor	Text [VARCHAR]	NOMBRE DEL AUTOR DEL LIBRO
	Apellidos del Autor	Text [VARCHAR]	APELLIDOS DEL AUTOR DEL LIBRO
	DNI del autor	Text [VARCHAR]	DNI DEL AUTOR

Reto 2:

Añada un campo llamado Salario (Numérico) a la tabla Autores. Introduzca dos autoras, Juana y Antonia, con un salario de 999 (eur) cada una. A continuación, cree un programa con Python 3 y la librería SQLite3 para calcular el salario total (suma de los salarios de todos los autores de la tabla).

Corrija las órdenes que crea necesarias en el siguiente bloque de código:

```
SELECT DNI, Nombre, SUM(VALUE1), SUM(VALUE2)
FROM tableName
GROUP by DNI
```

Ejemplo 62b. Visualización de registros de una base de datos SQLite.

Objetivo: Utilizar Python para leer un archivo con formato SQLite con datos para realizar la lectura de bases de datos con este formato.

```
#
# Programa xsumal.py desarrollado en Python 3 para Microsoft Windows 10
#
# El script visualiza el primer registro de la tabla page1 y
#
# permite su modificación mediante la entrada de una cadena nueva:
#
# newline1
#
# El primer registro de la tabla page1 contiene un párrafo largo
#
# de la primera página del libro o un nombre de una persona
#
# Cómo construir la cadena para pasarla como orden SQLite?
#
# La base de datos se llama test11.db. Puede descargar el archivo .db visitando:
#
# https://github.com/javier928/xxupdate
#
# La tabla 1 se llama page1. La tabla contiene párrafos de un libro o
# preguntas de un test
#
# La tabla 1 (y cada tabla de la bd solo contiene 1 registro. Cada registro
# contiene los párrafos de esa pag)
#
# La tabla 2 se llama page2. La tabla contiene más párrafos del libro o más
# preguntas de un test.
#
# La tabla 9 se llama page9. La tabla contiene más párrafos del libro. El libro
# contiene 10 páginas (10 tablas).
#
# La tabla tiene los siguientes campos (todos son tipo texto excepto
# booknn y pagenn, que son numéricos):
#
# line1, line2...line20
#
# ¿Qué campo estamos utilizando para filtrar registros? pagenn
#
#
#####

import os
import sqlite3
import tabulate

def cls():
    os.system("cls")

## El código continúa en la siguiente página
```

```
def superprint(ppp,qqq):  
    ##si ppp es '1' se imprime una línea en blanco antes de imprimir el valor de qqq  
    if ppp=="1":  
        print(" ")  
    print(qqq)
```

```
def printlongline():  
    print("-----")  
    print("-----")
```

El código continúa en la siguiente página

```

## Preguntamos qué database (ebook) queremos abrir:
## Esta parte se puede modificar para abrir diferentes libros (databases)
cls()
superprint("0"," Bienvenido a SQL database editor lite")
print(" ")
zx= input(" Qué database quiere abrir? <Enter 0 para salir, entre 1 para abrir test11.db>
")

conn= sqlite3.connect("test11.db")
print(" ")
print(" SQLite database <test11.db> se ha abierto correctamente. ")

## preguntamos que TABLE (page of the book) queremos ACTUALIZAR
print(" ")
Xplineyyy= input(" Qué (TABLE or PAGE) number quiere ver? <Entre 0 para salir> ")
Xpyyy=int(Xplineyyy)

## construimos orden SQLite para contar registros en la tabla 1. SELECT field1,field2,etc.
from table1 (or pagel)
cursor1= "SELECT LINE1,"
cursor2= "LINE2, LINE3, LINE4, LINE5,LINE6, LINE7, LINE8, LINE9,LINE10,LINE11, LINE12,
LINE13,LINE14,LINE15,LINE16, LINE17, LINE18, LINE19,LINE20, PAGENN, "
cursor3= "LINE1N, LINE2N, LINE3N FROM PAGE"+Xplineyyy
cursor= conn.execute(cursor1+cursor2+cursor3)

cursor1= "SELECT COUNT(*)"
cursor2= " "
cursor3= "FROM PAGE"+Xplineyyy
#pagel is table1 in sqlite database test.db
cursor= conn.execute(cursor1+cursor2+cursor3)

print(" ")
plineyyy= input(" Qué (record) number quiere editar? <Entre 0 para salir> ")
pyyy=int(plineyyy)

### falta cambiar el filtro en la orden SELECT para mostrar pagenn= plineyyy
cursor1= "SELECT LINE1,"
cursor2= "LINE2, LINE3, LINE4, LINE5, LINE6, LINE7, LINE8, LINE9,LINE10,LINE11, LINE12,
LINE13,LINE14,LINE15,LINE16, LINE17, LINE18, LINE19,LINE20,PAGENN, "
cursor3= "LINE1N, LINE2N, LINE3N FROM PAGE"+Xplineyyy+" "
cursor4= "WHERE PAGENN="+plineyyy+" "
cursor= conn.execute(cursor1+cursor2+cursor3+cursor4)

### Continúa en la página siguiente...

```

```

cls()
for row in cursor:

print("-----\n")
print(" Fields:      Content:")

print("-----\n")
print(" line1:      ", row[0].strip(), "gasta "+str(row[21])+" eur")
print(" line2:      ", row[1].strip(), "gasta "+str(row[22])+" eur")
print(" line3:      ", row[2].strip(), "gasta "+str(row[23])+" eur")
print(" line4:      ", row[3])
print(" line5:      ", row[4])
print(" line6:      ", row[5])
print(" line7:      ", row[6])
print(" line8:      ", row[7])
print(" line9:      ", row[8])
print(" line10:     ", row[9])
print(" line11:     ", row[10])
print(" line12:     ", row[11])
print(" line13:     ", row[12])
print(" line14:     ", row[13])
print(" line15:     ", row[14])
print(" line16:     ", row[15])
print(" line17:     ", row[16])
print(" line18:     ", row[17])
print(" line19:     ", row[18])
print(" line20:     ", row[19], "Total: "+str(row[21]+row[22]+row[23])+" eur")
print(" pagenn:    ", row[20])

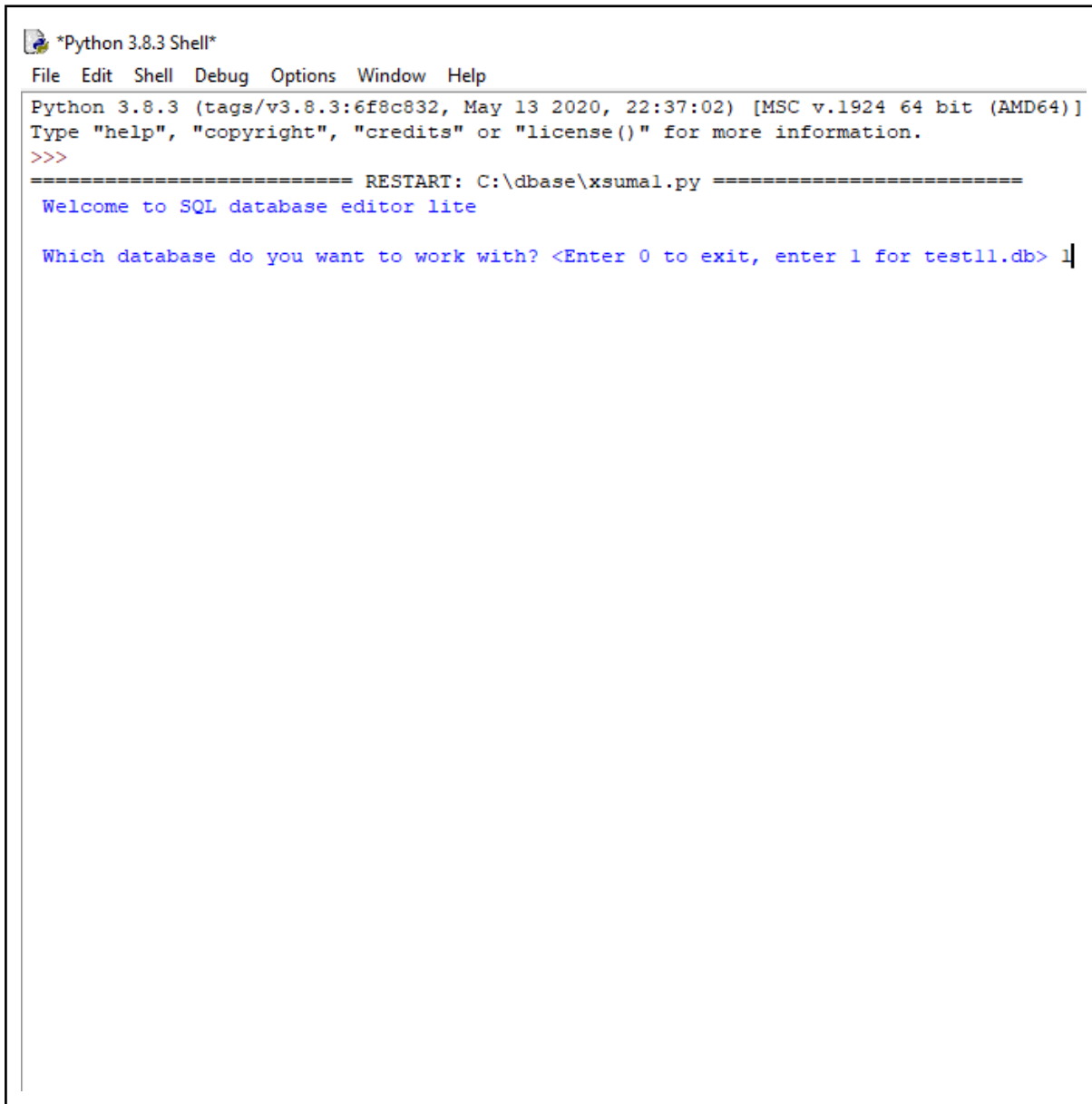
print("\n-----")

conn.close()

## Fin del código ## Puede descargar el código fuente (y la base de datos SQLite) de este ejemplo visitando:
## https://github.com/javier928/xxupdate

```

Al ejecutar el código podemos ver que tenemos que responder la primera pregunta.



```
*Python 3.8.3 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite
Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1
```

Si tecleamos 1 y pulsamos <Enter>, estaremos de acuerdo en abrir el archivo test11.db y veremos la siguiente pregunta:

A continuación podemos ver la siguiente pregunta (donde se filtra la tabla que queremos ver-la cual contiene una página de nuestro hipotético libro-):

```
*Python 3.8.3 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite

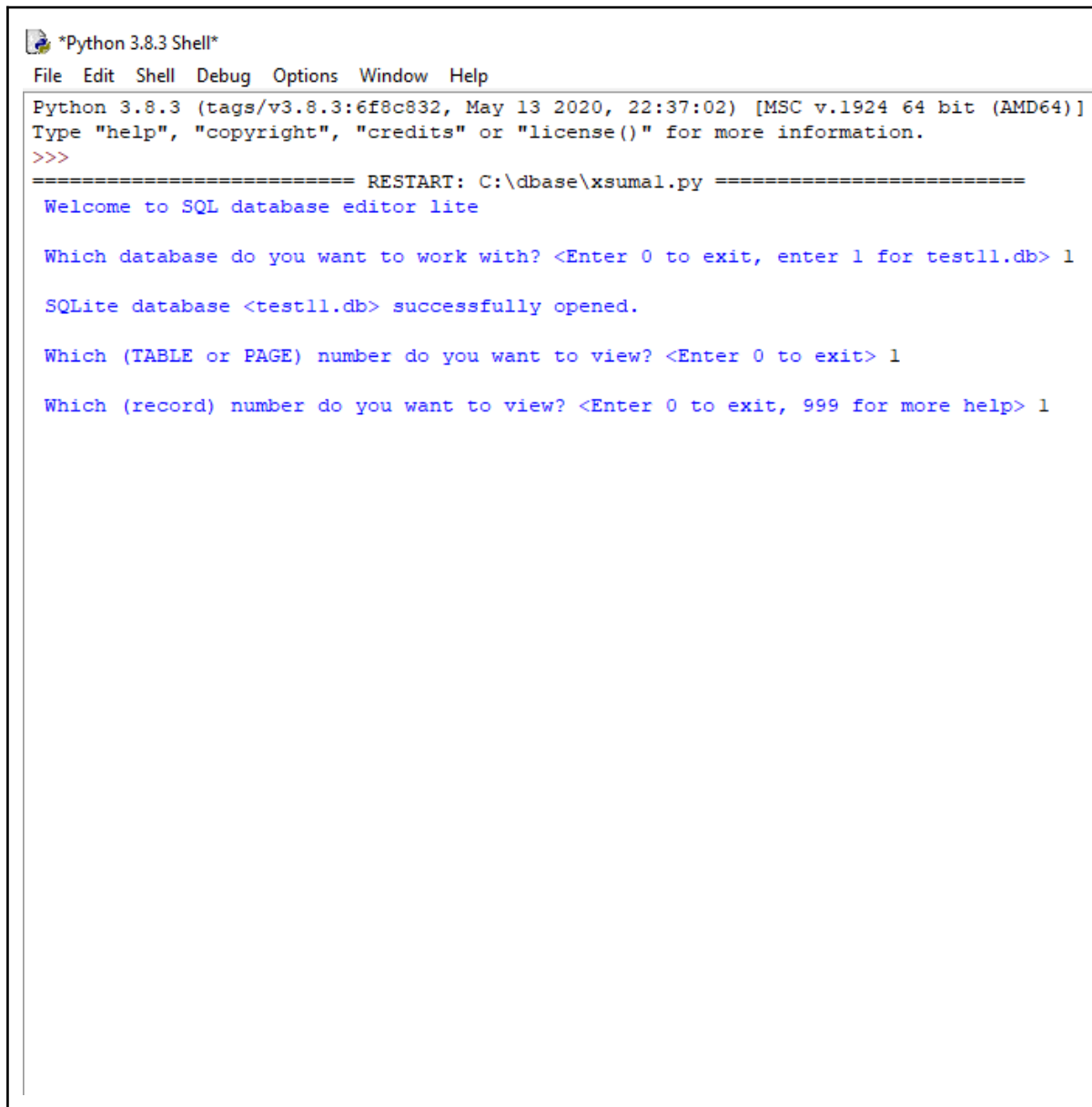
Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1

SQLite database <test11.db> successfully opened.

Which (TABLE or PAGE) number do you want to view? <Enter 0 to exit> 1
```

Si tecleamos 1 y pulsamos <Enter>, veremos la siguiente pregunta:

A continuación podemos ver la siguiente pregunta (donde se filtra qué registro -record- visualizaremos (recuerde que cada tabla únicamente contiene un registro):

A screenshot of a Python 3.8.3 Shell window titled '*Python 3.8.3 Shell*'. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area shows the following content:

```
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite

Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1

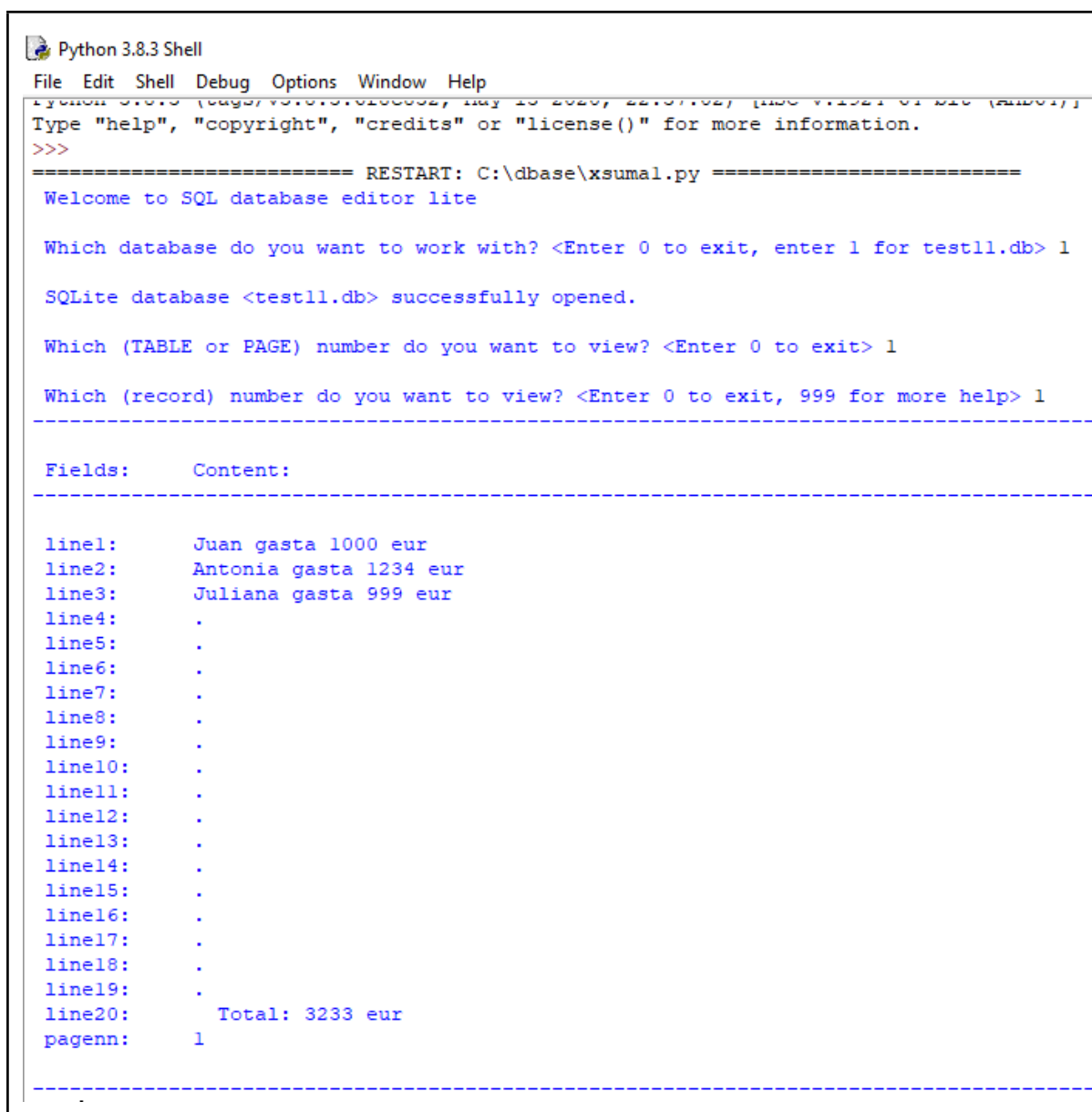
SQLite database <test11.db> successfully opened.

Which (TABLE or PAGE) number do you want to view? <Enter 0 to exit> 1

Which (record) number do you want to view? <Enter 0 to exit, 999 for more help> 1
```

Si tecleamos 1 y pulsamos <Enter>, veremos el contenido del registro seleccionado:

Contenido del registro:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:100002, May 13 2020, 22:07:02) [AMD64]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite

Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1

SQLite database <test11.db> successfully opened.

Which (TABLE or PAGE) number do you want to view? <Enter 0 to exit> 1

Which (record) number do you want to view? <Enter 0 to exit, 999 for more help> 1
-----
Fields:      Content:
-----
line1:      Juan gasta 1000 eur
line2:      Antonia gasta 1234 eur
line3:      Juliana gasta 999 eur
line4:      .
line5:      .
line6:      .
line7:      .
line8:      .
line9:      .
line10:     .
line11:     .
line12:     .
line13:     .
line14:     .
line15:     .
line16:     .
line17:     .
line18:     .
line19:     .
line20:     Total: 3233 eur
pagenn:     1
-----
```

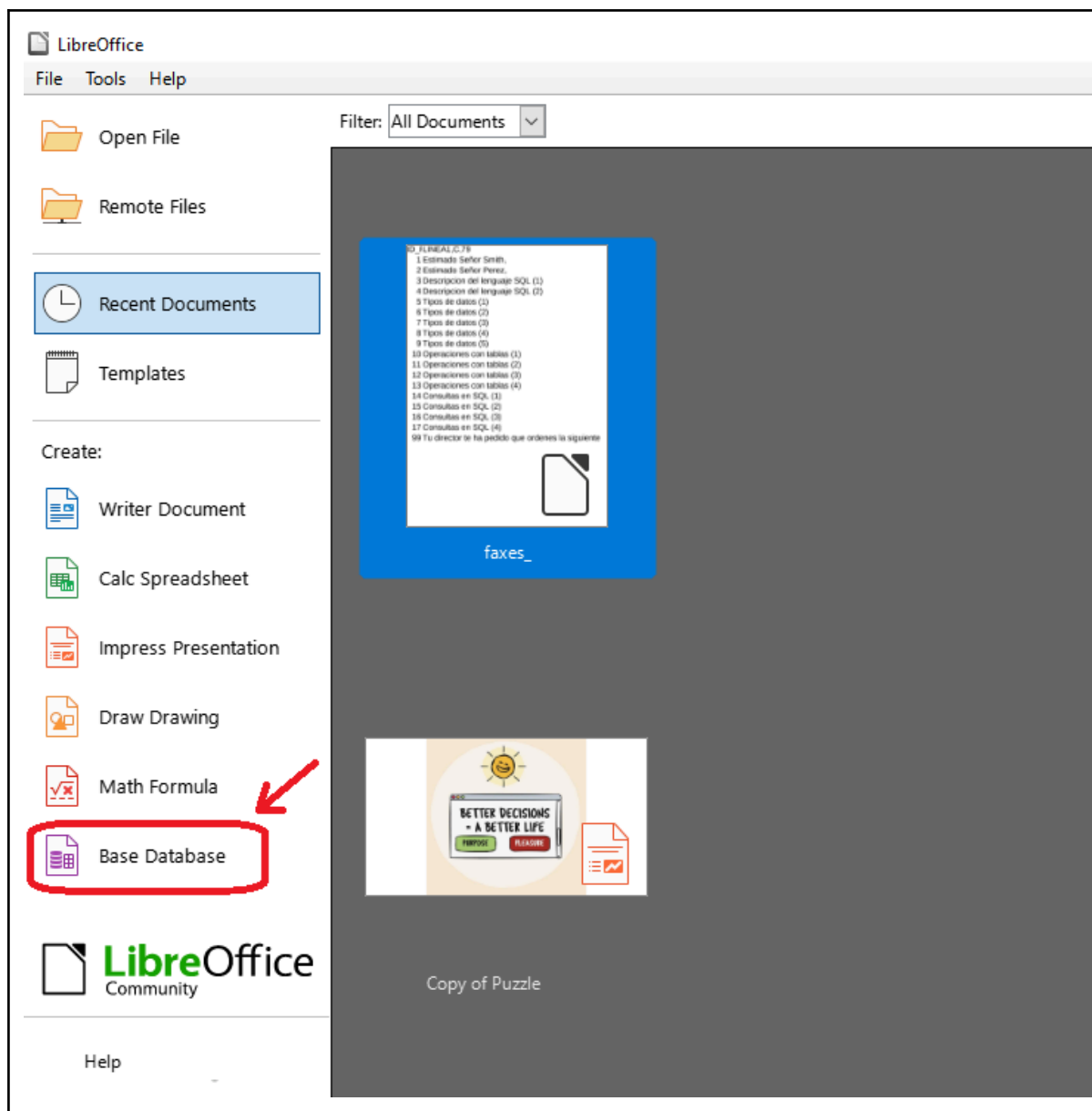
Nota:

Observe que en el campo `LINE20` se ha sumado el contenido de los campos `LINE1N`, `LINE2N` y `LINE3N` (no se ha sumado el contenido de `LINE1`, `LINE2` y `LINE3`).

Si necesita sumar otros valores, tendrá que cambiar el contenido de los campos `LINE1N`, `LINE2N` y `LINE3N` en el primer registro de la tabla seleccionada, que en este caso es la número 1.

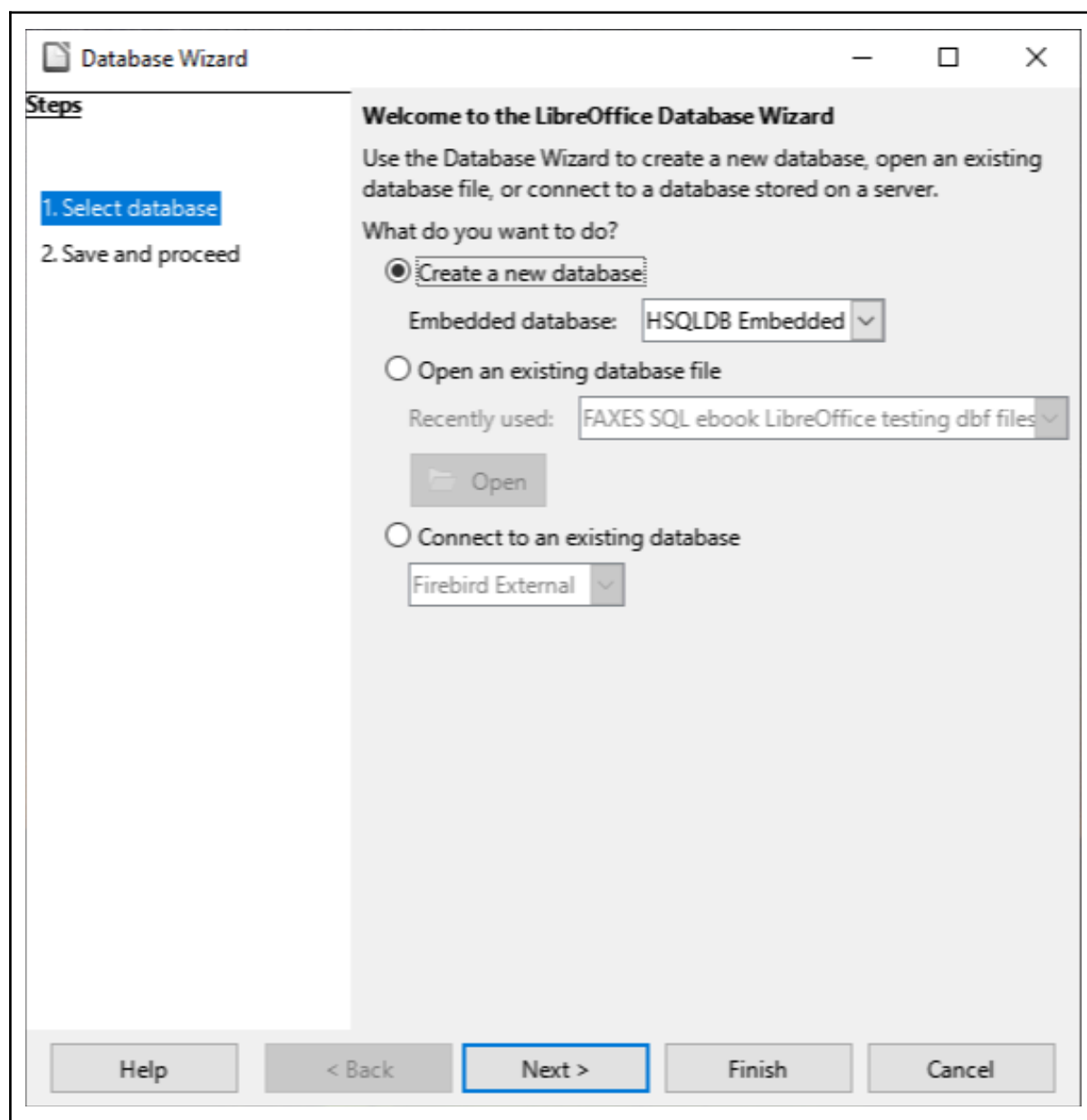
Creación de una base de datos con LibreOffice Base

Una vez abierto el programa LibreOffice, verá esta imagen:



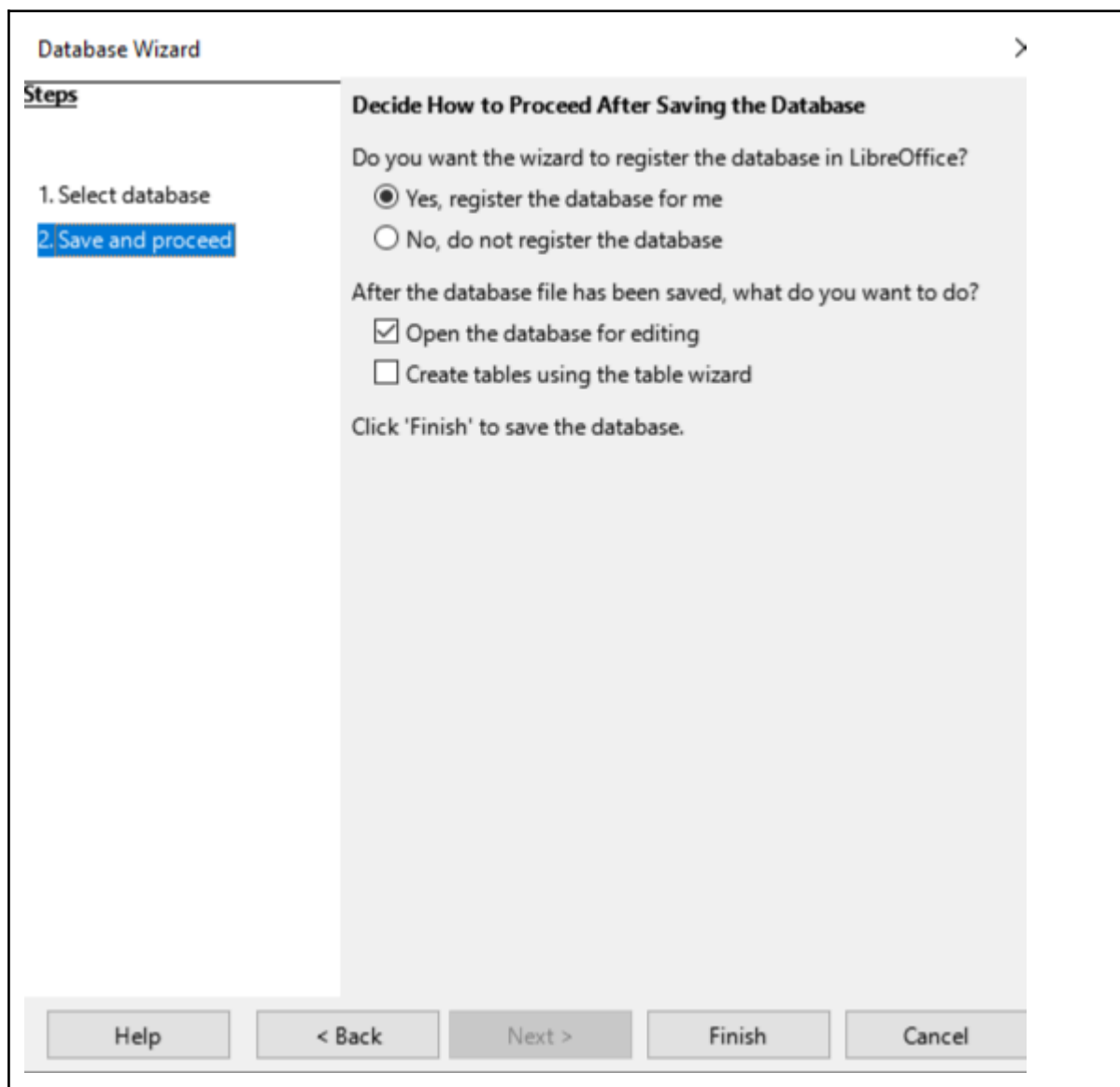
Elija la opción **Base Database**.

Verá esta imagen a continuación:



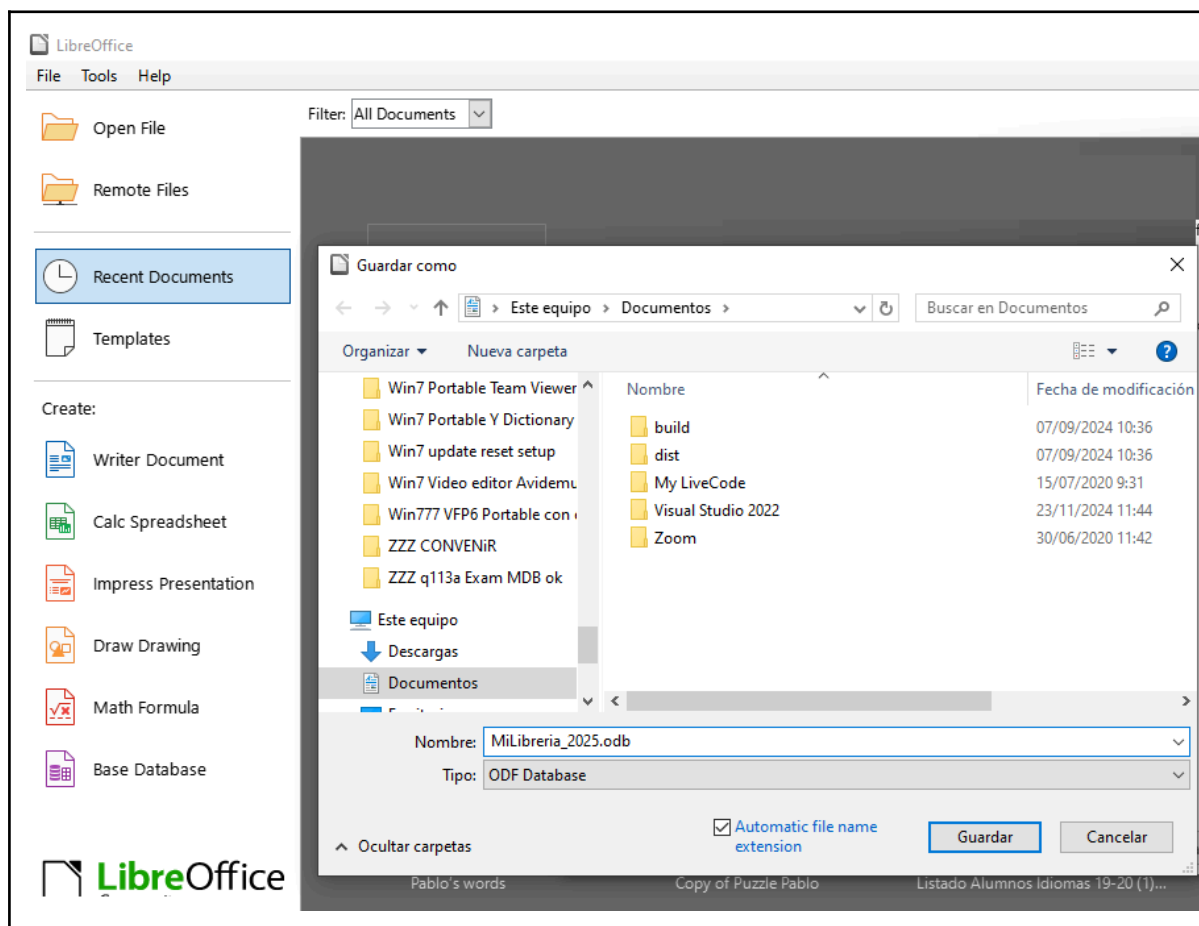
Seleccione **Crear nueva base de datos** (create) y haga clic en siguiente (next).

Verá esta imagen a continuación:



Seleccione **Sí. Registrar la base de datos** (Register the database) y también seleccione **Abrir base de datos para editar** (Open database for editing).

Verá esta imagen a continuación:



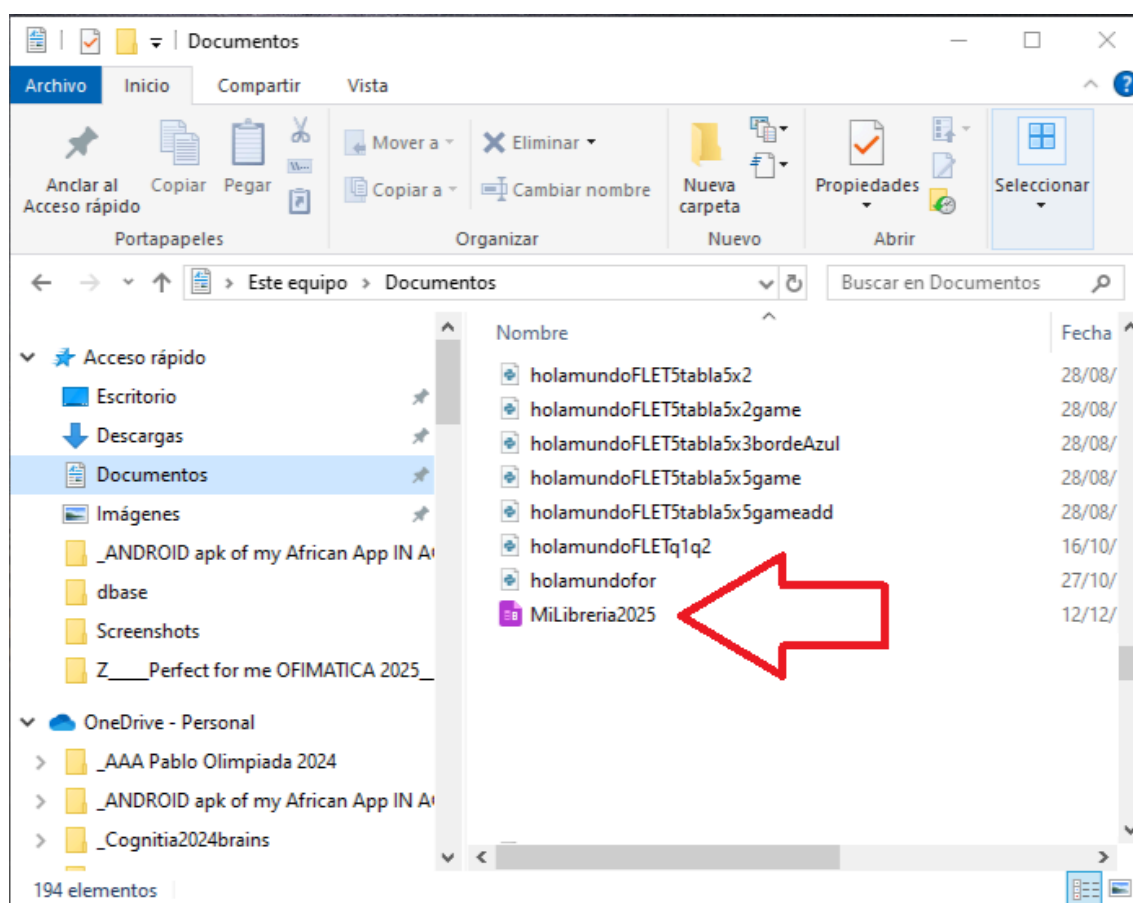
Introduzca el nombre “MiLibrería_2025”. Su base de datos quedará grabada en su ordenador con ese nombre. Además tendrá la extensión .odb.

A continuación, cierre LibreOffice. Volveremos a abrirlo más tarde para comprobar que el archivo se grabó con éxito y continuar con nuestro ejercicio.

Nota: Recuerde hacer una copia de seguridad en un disco USB externo. Es probable que su archivo se grabase originalmente en la carpeta “Documentos”.

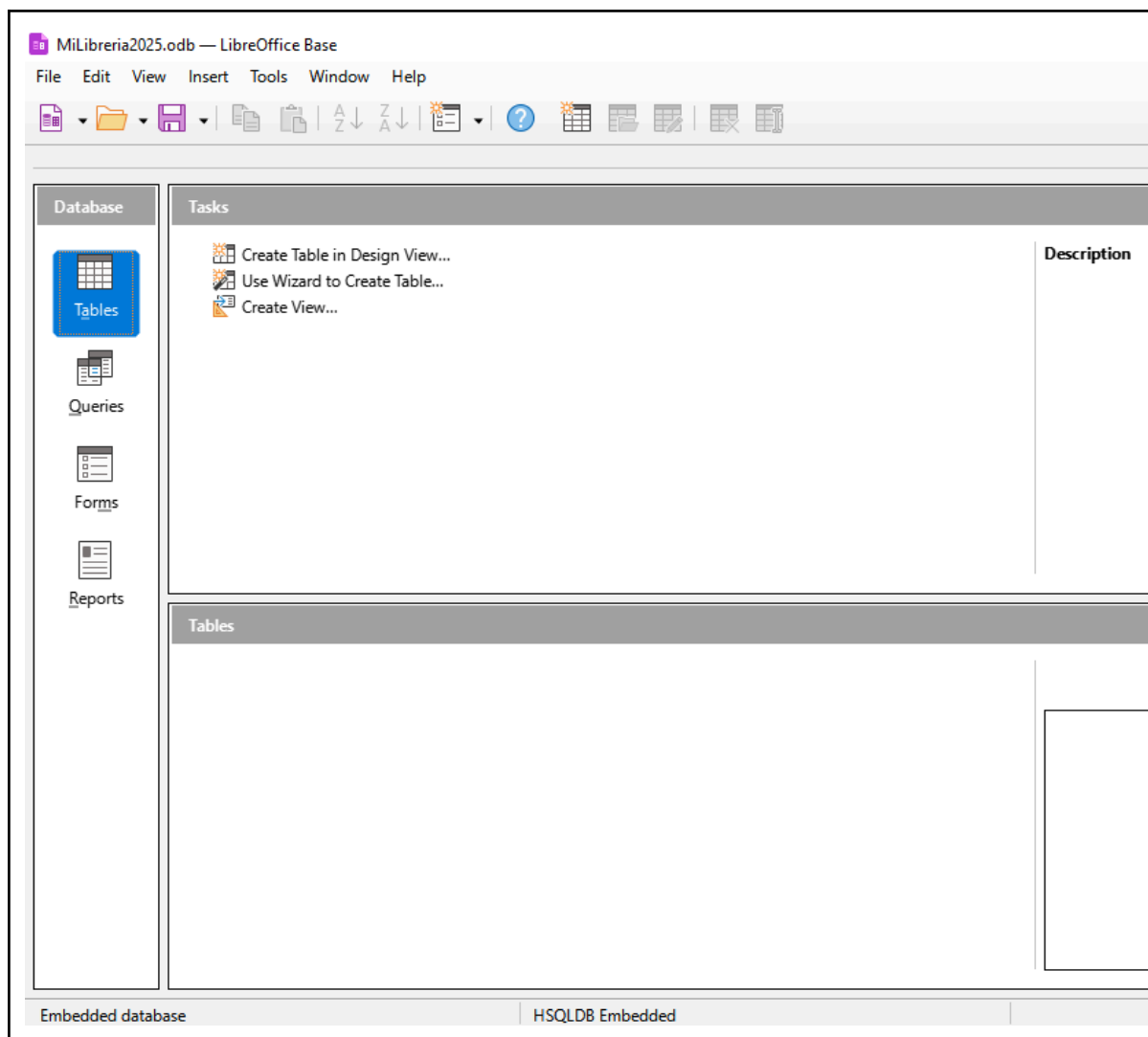
Abrir la base de datos

Una vez grabado su archivo en el paso anterior, busque en la carpeta “Documentos” de su ordenador el fichero “MiLibreria2025” y haga doble clic en él para abrirlo:



A continuación, verá esta imagen:

Una vez abierto su archivo, verá esta imagen:



A continuación, crearemos una tabla, que es el objeto básico que almacena los datos de los autores de la librería de nuestro proyecto. Aunque nuestro proyecto necesitará más de una tabla, de momento, nos centraremos en crear la primera, que se llamará “Autores”.

Para crear la estructura de nuestra primera tabla, llamada “Autores”, necesitamos introducir los tres elementos básicos que componen todas las tablas:

Nombre de campo (Field):

Una tabla puede contener más de un campo. En nuestro caso, los campos para la tabla “Autores” serán DNI, Nombre, Apellidos, Dirección, Teléfono y Ciudad. Es aconsejable no acentuar estas palabras ni incluir caracteres especiales.

Tipo de datos (Data type):

Los tipos de datos indican entre otras cosas la naturaleza de los datos que se almacenarán en el campo al que nos referimos. Por ejemplo: Si serán sólo texto, números, fechas, etc.

Además LibreOffice reservará espacio para almacenar el contenido de dicho campo. Es decir, el número de bytes que necesitará por registro (cada ficha de cada autor).

Por último, LibreOffice, almacenará si se pueden realizar operaciones con ese campo y también almacenará si se debe utilizar el campo como clave principal, dato útil si se trata de número de Pasaporte, DNI, etc..

Los diferentes tipos de datos en LibreOffice son: Texto (VARCHAR), Nota, Numérico (INTEGER y REAL), Fecha/Hora (DATE), Sí/No (boolean), Objeto (VARBINARY) e Imagen (LONGVARBINARY).

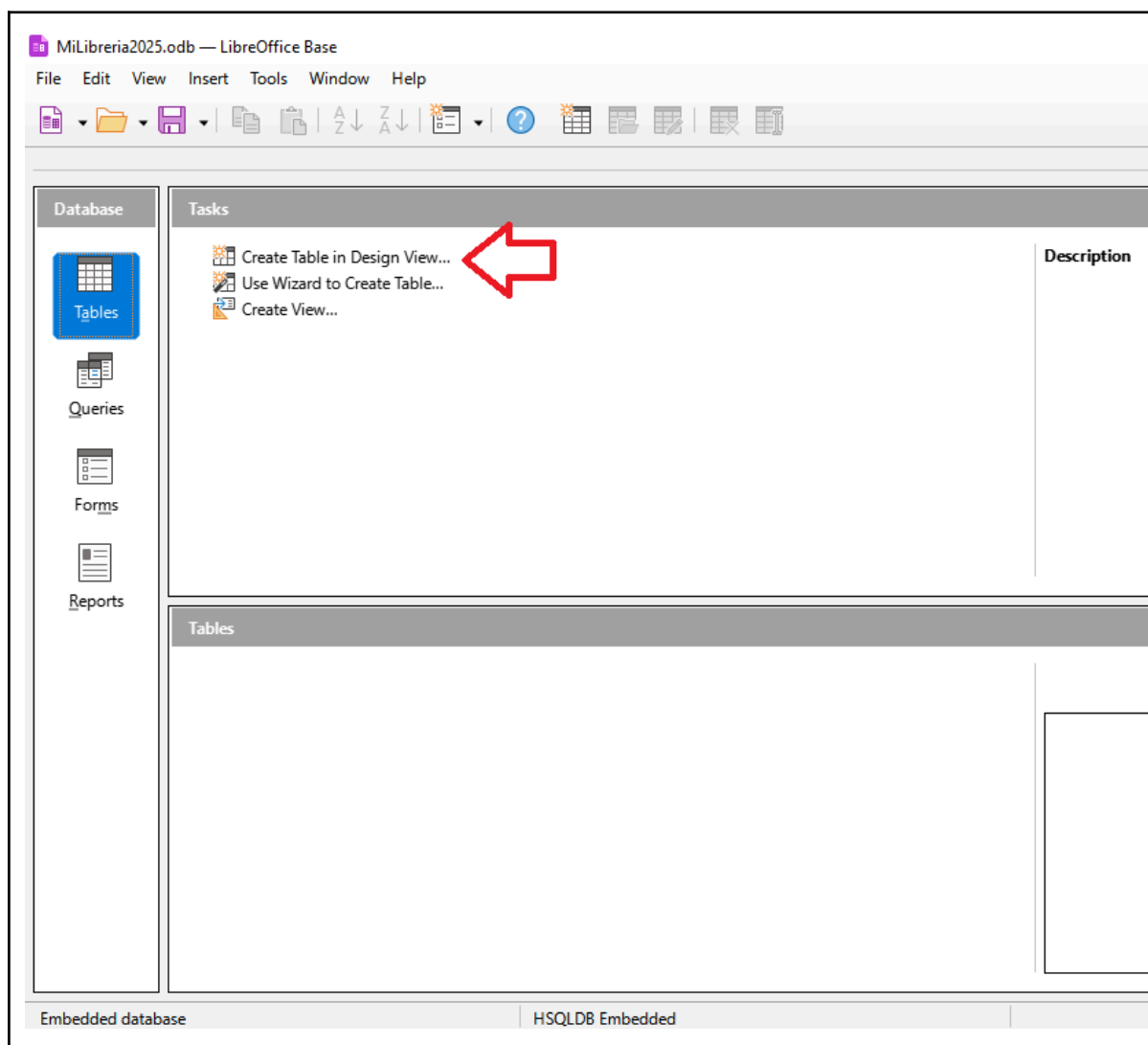
Propiedades del campo (Field properties):

Las propiedades de campo que existen en LibreOffice son:

Valor automático, entrada requerida, tamaño, decimales, valor predeterminado y ejemplo de formato.

A medida que nuestro proyecto avance, veremos cómo se implementan cada una de las características mencionadas para el diseño de nuestras tablas.

Una vez abierto su archivo, para crear la primera tabla en vista diseño, haga clic en “Create Table in Design View”:



A continuación, añadiremos los campos necesarios para nuestra “Autores”.

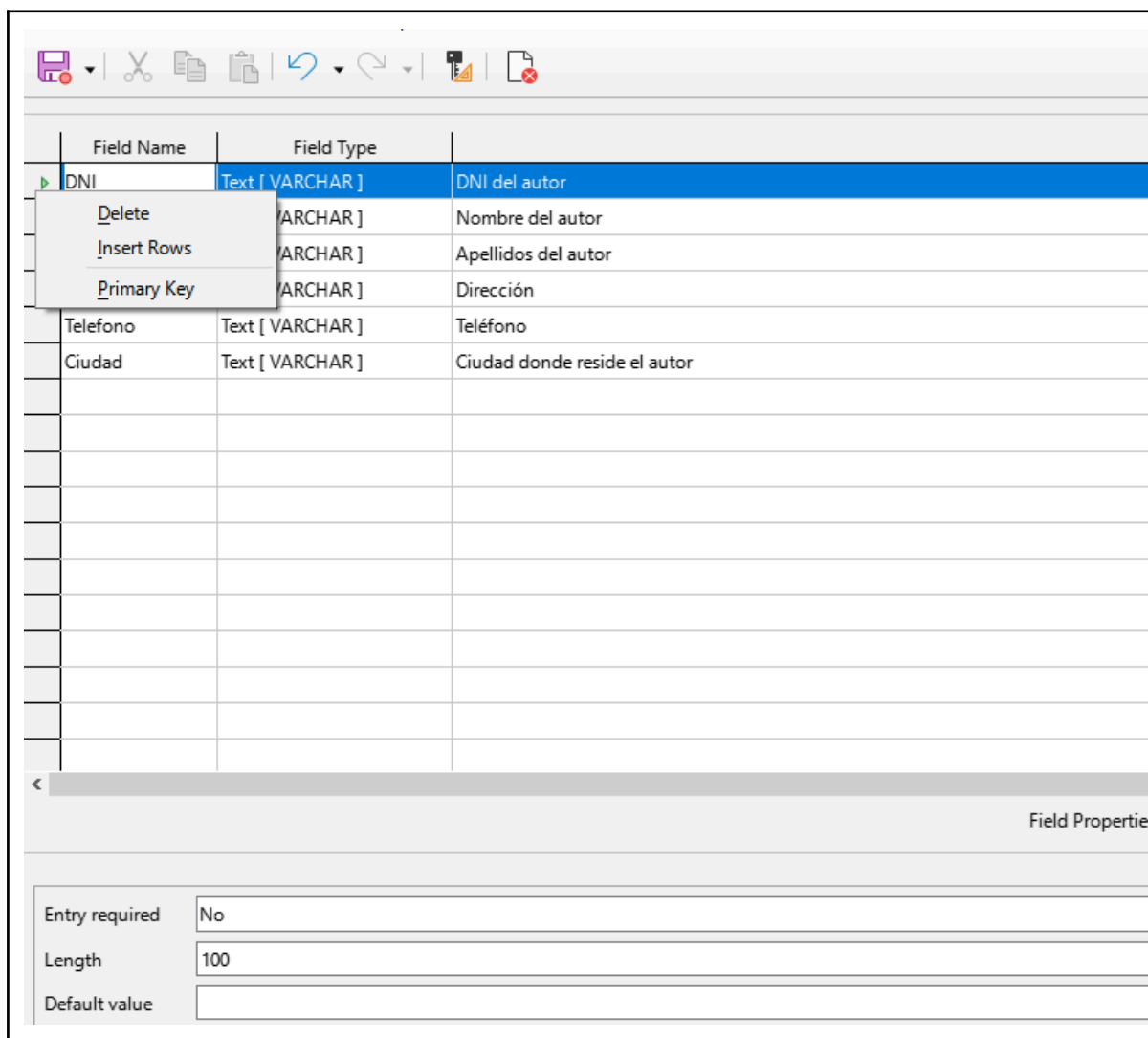
En esta pantalla, haga clic en la primera caja vacía (o primera celda):

Field Name	Field Type	Description

Field Properties

A continuación, añadiremos los campos necesarios para nuestra tabla “Autores”. Haga clic en cada celda vacía y teclee los siguientes campos hasta que quede exactamente igual que la imagen siguiente:

Haga clic con el botón derecho del ratón en la zona donde está la llave (a la izquierda de la palabra “DNI”) y elija “Primary Key”.



La llave debe haber aparecido junto a la palabra “DNI”.

En LibreOffice Base, una clave primaria (primary key) es una columna (o combinación de columnas) que identifica de forma única cada fila de una tabla. Esto significa que ninguna fila puede tener el mismo valor en la columna de la clave primaria. Es decir, que ningún DNI en la tabla “Autores” puede repetirse porque debe ser único y el campo DNI no puede estar vacío.

Recuerde que una tabla sólo puede tener una clave primaria. Además, el tipo de datos de la columna de la clave primaria debe ser apropiado para la identificación única (por ejemplo, texto, número o fecha).

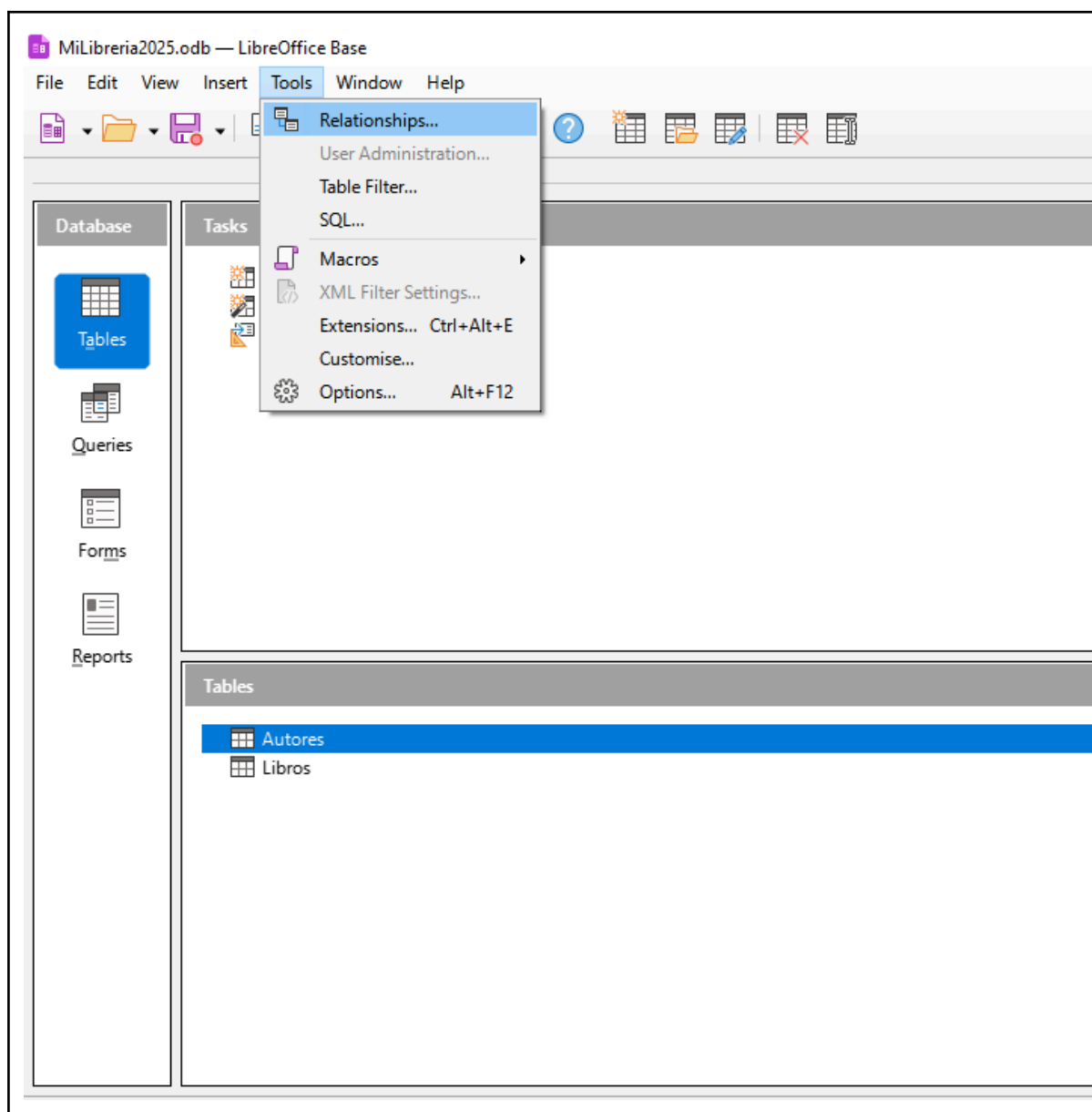
A medida que nuestro proyecto avance, veremos cómo implementamos la relación en SQL que debe existir entre un autor y todos los libros escritos por él.

De esta manera, nos aseguramos de que podamos, por ejemplo, dar las instrucciones necesarias en SQL para imprimir un informe de todos los libros escritos por un determinado autor. Esto quiere decir que un autor puede tener un número “n” libros. Osea que ese autor puede tener 1, 2 ó 10 libros en nuestra base de datos.

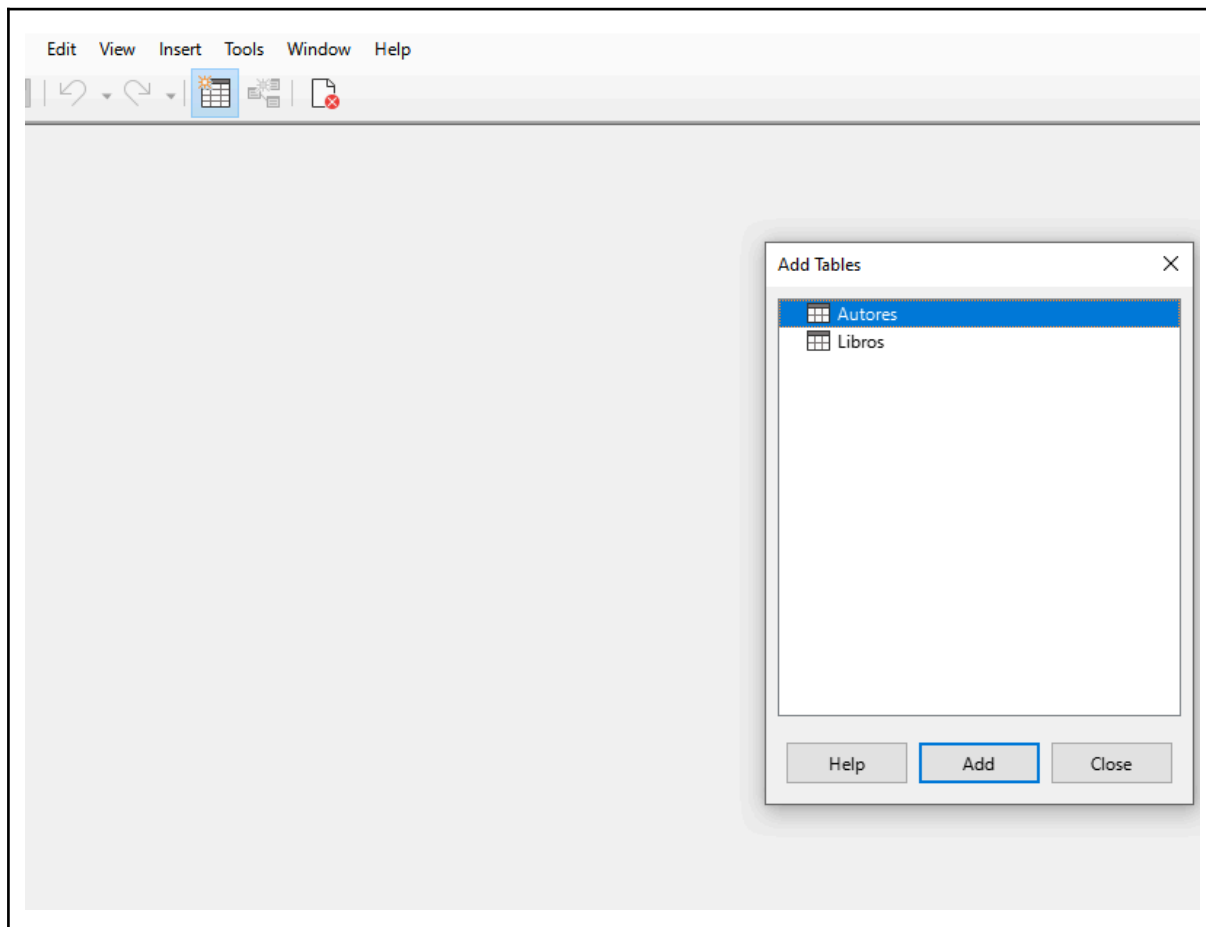
Un diseño correcto de bases de datos depende en gran medida de establecer relaciones entre tablas. Un buen diseño de relaciones garantiza que las aplicaciones que se construyan sobre la base de datos sean rápidas, estables y eficientes.

Creación de la relación entre las tablas de nuestra base de datos

Ahora que ya tenemos creadas las dos tablas, “Autores” y “Libros” de nuestra base de datos, debemos crear la relación antes mencionada. En esta ocasión, no la crearemos con instrucciones SQL, como haremos más adelante, sino que lo haremos de forma visual. Para ello, abriremos nuestra base de datos y haremos clic en “Tools” (Herramientas), y a continuación, haremos clic en “Relationships” (Relaciones).

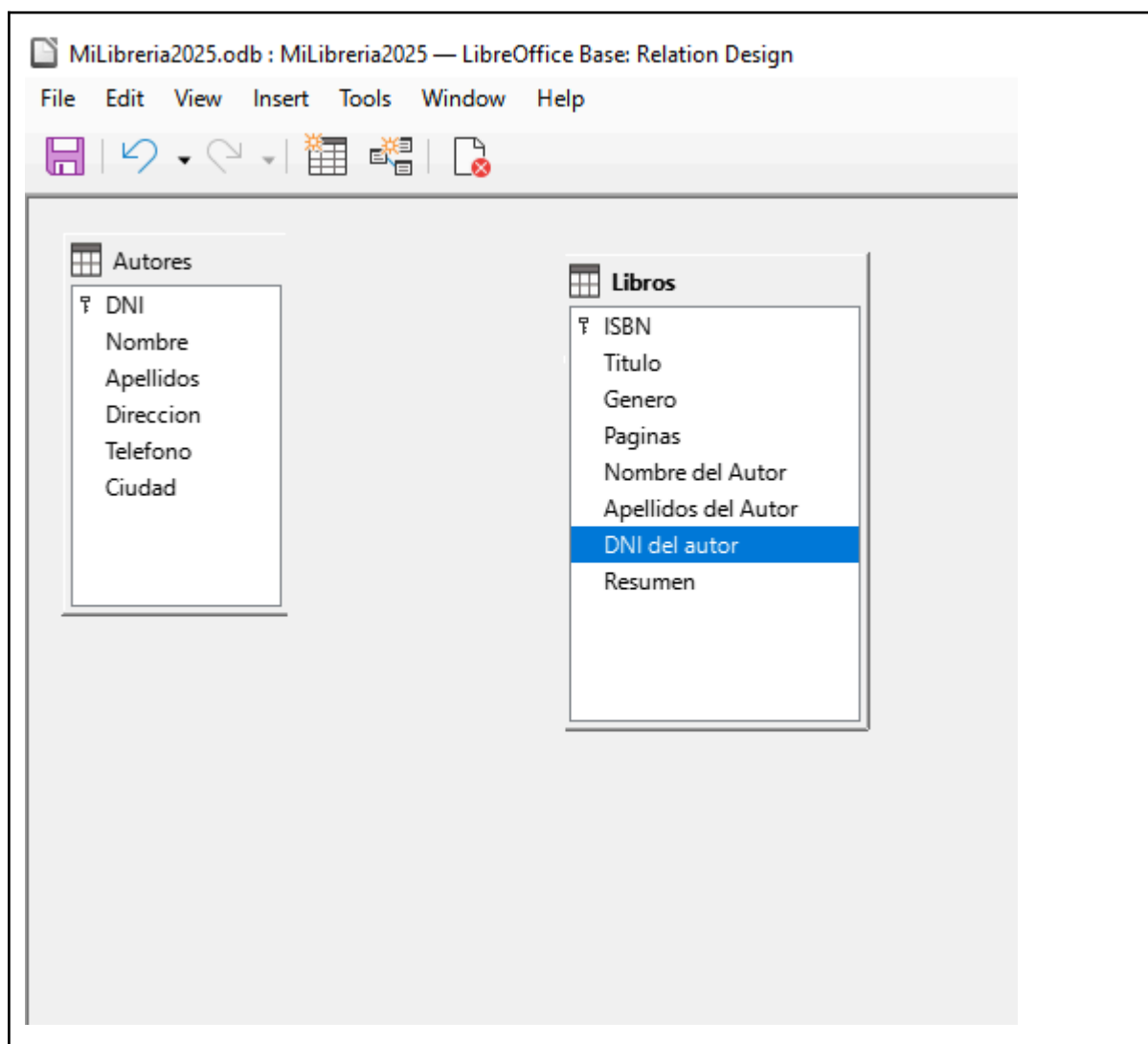


Aparecerá esta pantalla:



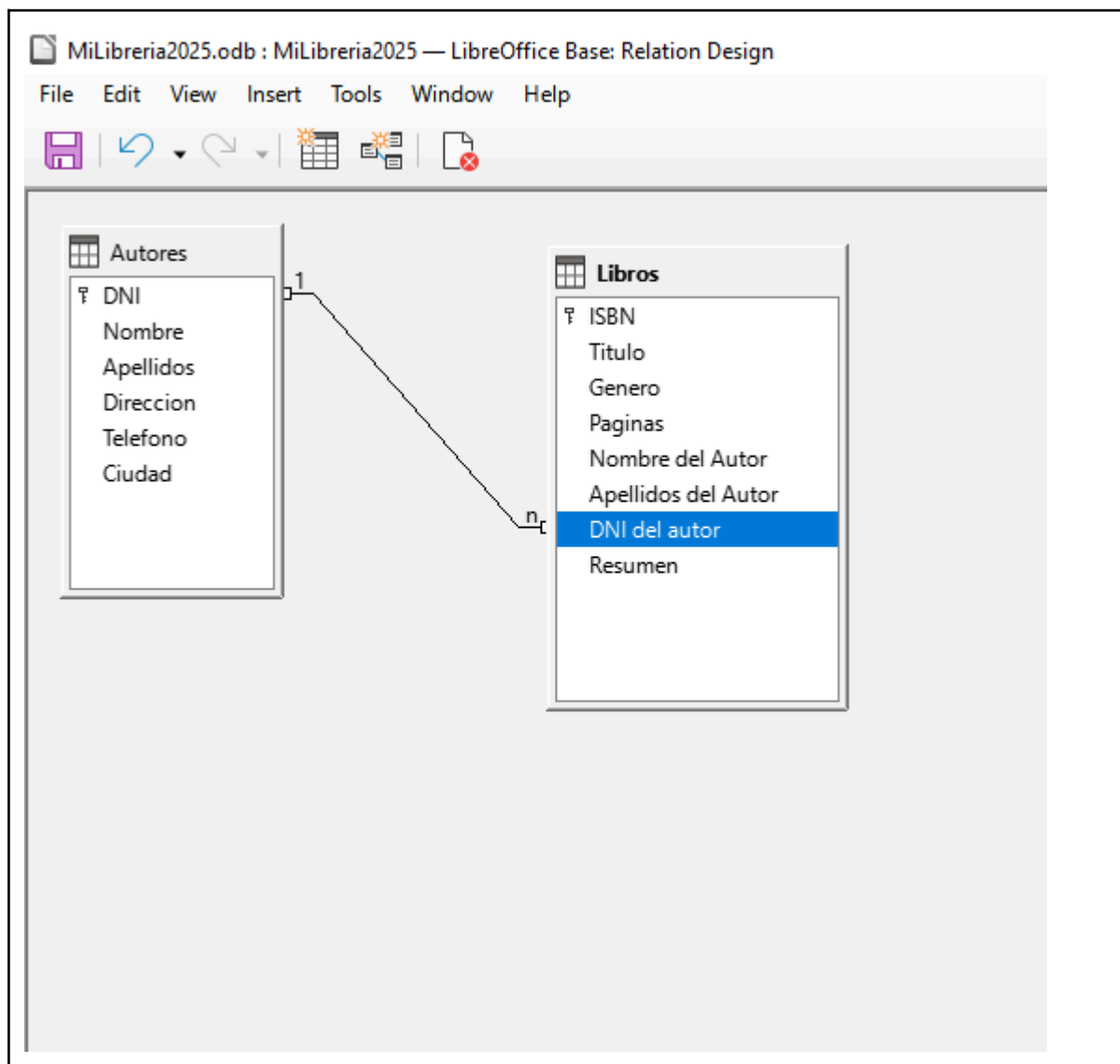
Haga clic en “Autores” y a continuación, haga clic en “Añadir” (Add). Después haga clic en “Libros” y haga también clic en “Añadir”.

Aparecerá esta pantalla:



Haga clic en “DNI” en la tabla de la izquierda (Autores) y arrastre la palabra “DNI” hasta que toque las palabras “DNI del autor” (en la tabla de la derecha). Cuando suelte el ratón, debe haberse creado una línea entre las dos tablas como se muestra en la siguiente imagen:

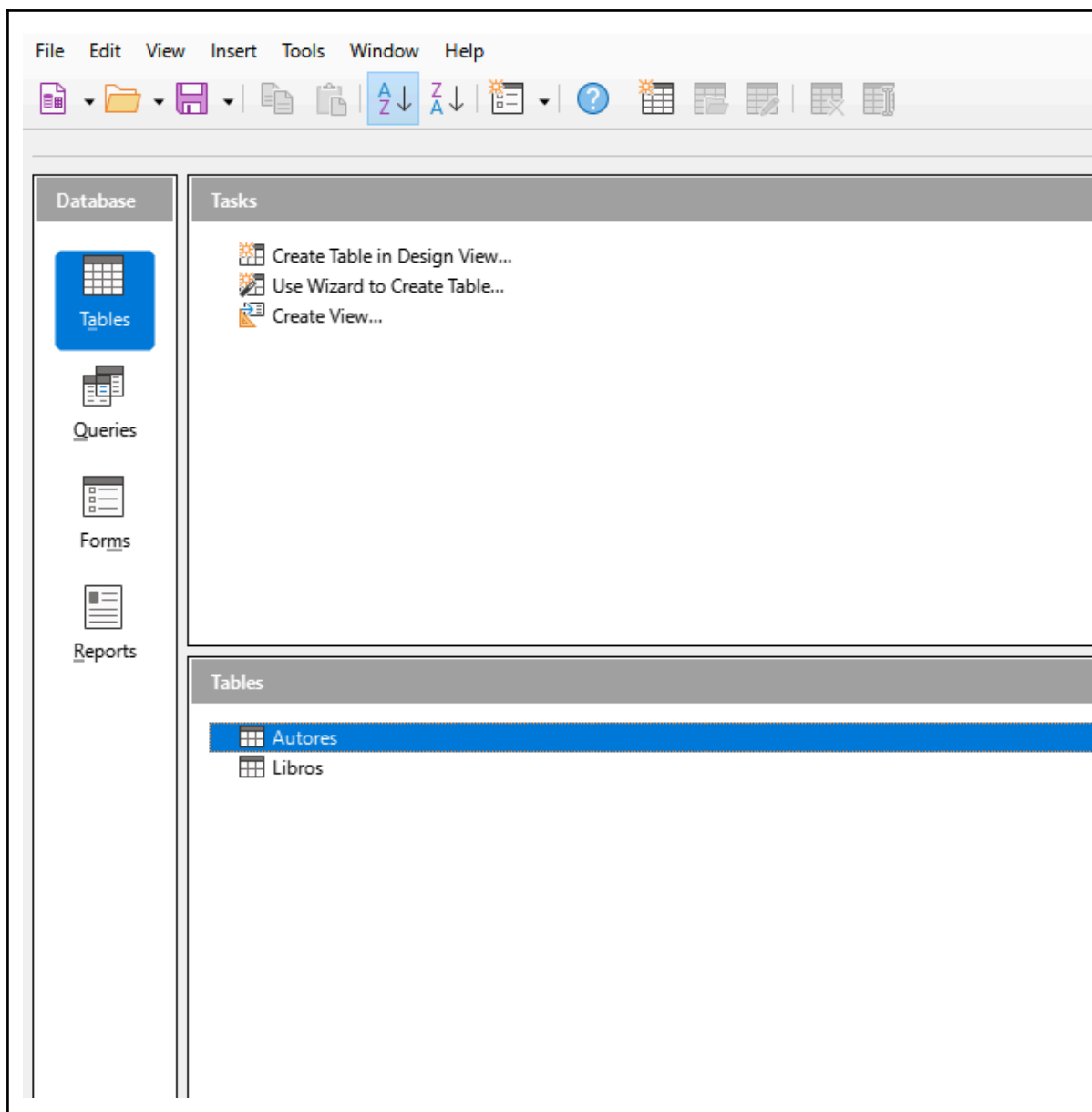
La relación se habrá creado con éxito si ve esta imagen;



Guarde la relación creada: Haga clic en “File”, y luego clic en “Save” para grabar los cambios. Cierre la base de datos pulsando CTRL+Q. Asegúrese de guardar los cambios.

Introducción de datos de autores en nuestra base de datos

Abra la base de datos, para ello, busque en la carpeta “Documentos” de su ordenador el fichero “MiLibreria2025” y haga doble clic en él para abrirlo. Verá esta imagen:



Para abrir la tabla “Autores”, haga clic en “Tables” (Tablas) en la parte izquierda de la pantalla y luego haga clic en “Autores” en la parte derecha de la pantalla.

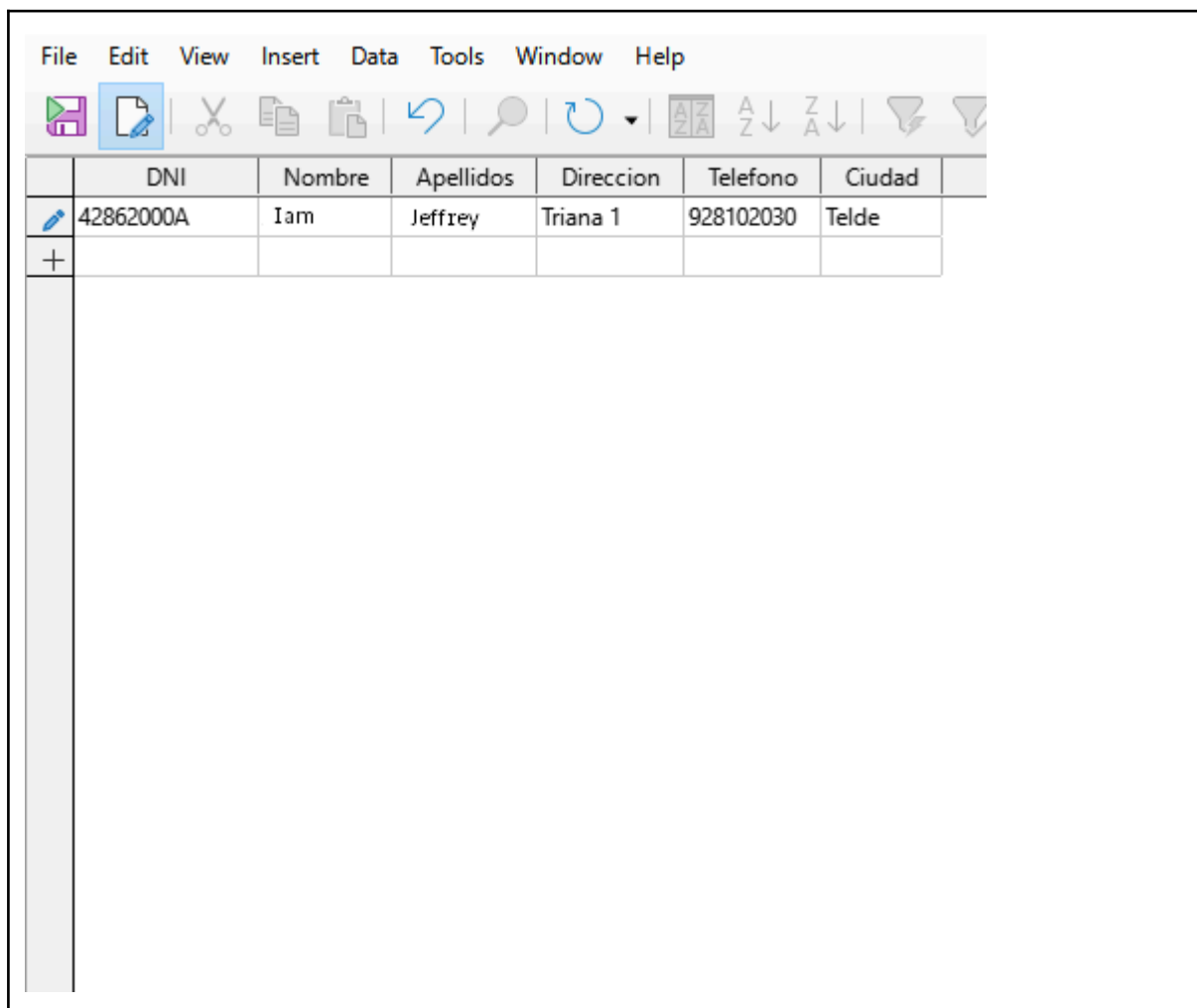
Verá esta pantalla:

The screenshot shows a database application window with a menu bar (File, Edit, View, Insert, Data, Tools, Window, Help) and a toolbar. The 'Edit' menu item is highlighted. Below the toolbar is a table with the following columns: DNI, Nombre, Apellidos, Direccion, Telefono, Ciudad. The first row of the table is empty, and a cursor is positioned in the 'DNI' column. A vertical scrollbar is visible on the left side of the table.

DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad

Introduzca el DNI del autor justo debajo de la palabra “DNI” y complete el resto de datos del autor hasta que quede como el ejemplo de la siguiente página:

Ejemplo 1:



The screenshot shows a database application window with a menu bar (File, Edit, View, Insert, Data, Tools, Window, Help) and a toolbar. The toolbar includes icons for saving, editing, deleting, copying, pasting, undo, redo, and sorting. Below the toolbar is a table with the following data:

	DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
	42862000A	Ian	Jeffrey	Triana 1	928102030	Telde
+						

The table has a vertical scrollbar on the left side.

Recuerde pulsar <Enter> cuando introduzca la palabra “Telde”, que es la ciudad del autor “Ian Jeffrey”.

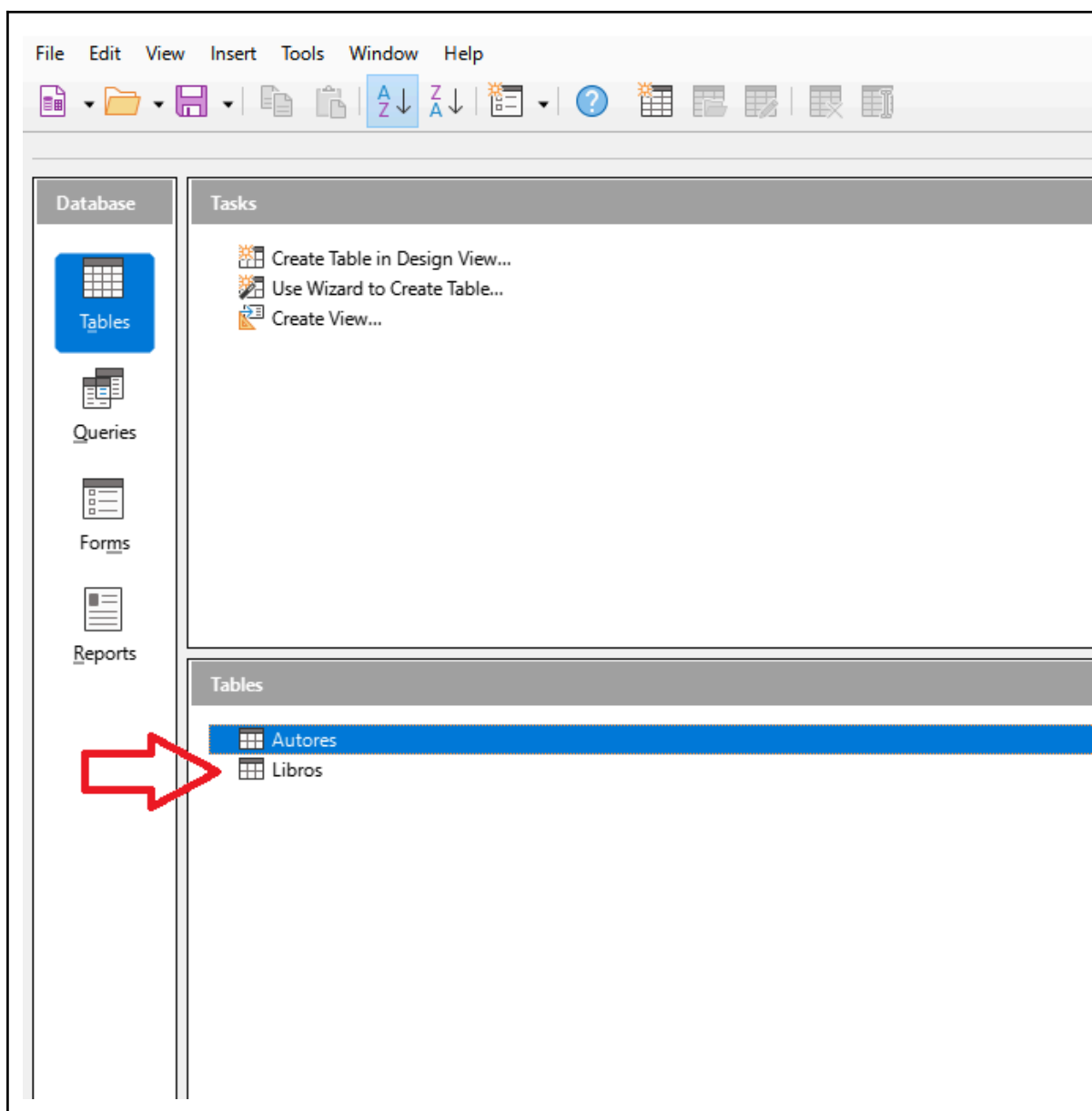
Ahora ya ha introducido el primer registro de la tabla “Autores”. Puede seguir introduciendo cuantos autores quiera.

Cierre la base de datos pulsando CTRL+Q. Asegúrese de guardar los cambios.

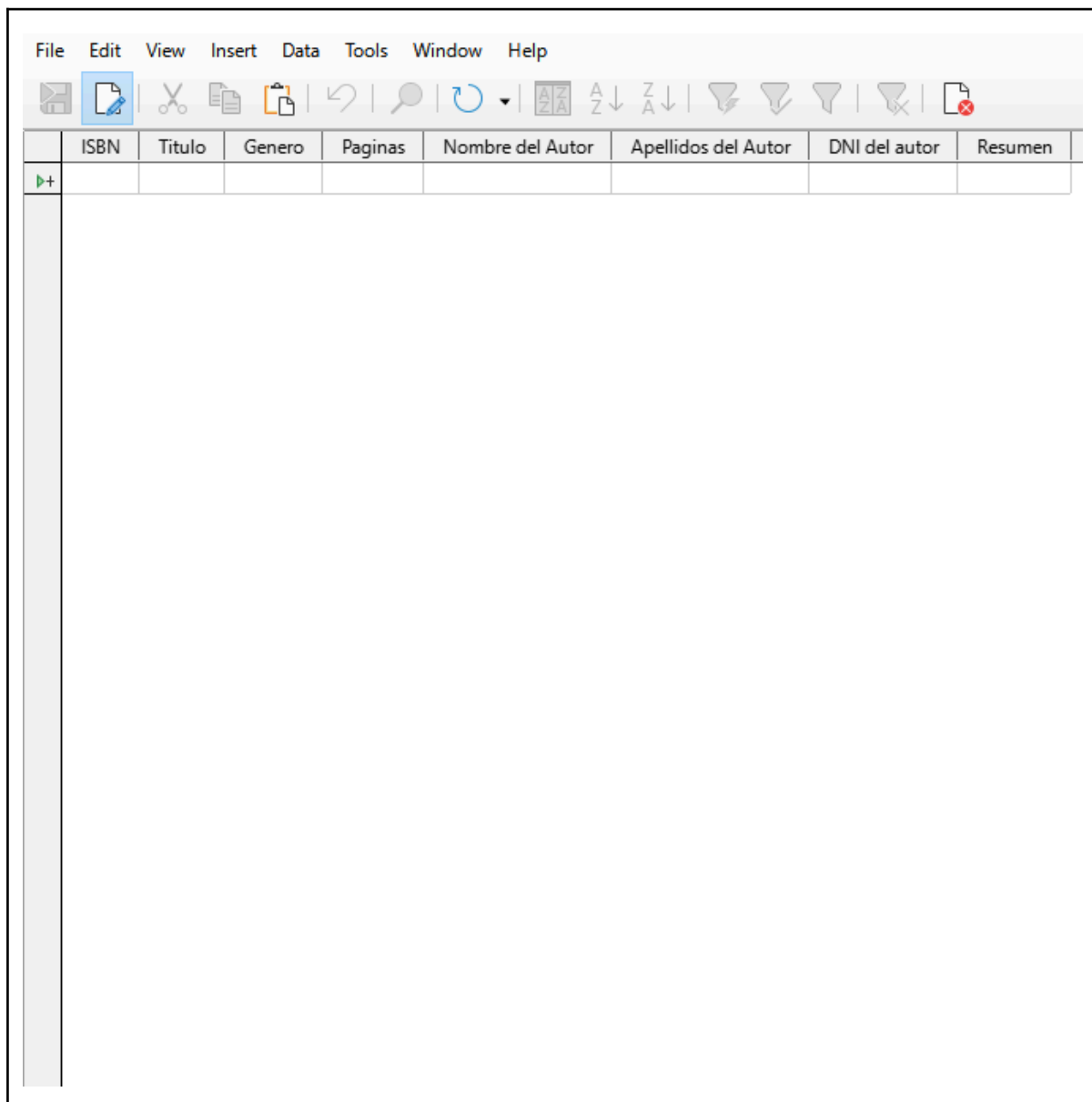
Veamos un ejemplo similar introduciendo datos de dos libros del primer autor, John Smith.

Abra la base de datos, para ello, busque en la carpeta “Documentos” de su ordenador el fichero “MiLibreria2025” y haga doble clic en él para abrirlo.

Para abrir la tabla “Libros”, haga clic en “Tables” (Tablas) en la parte izquierda de la pantalla y luego haga clic en “Libros” en la parte derecha de la pantalla.



Verá la siguiente pantalla:

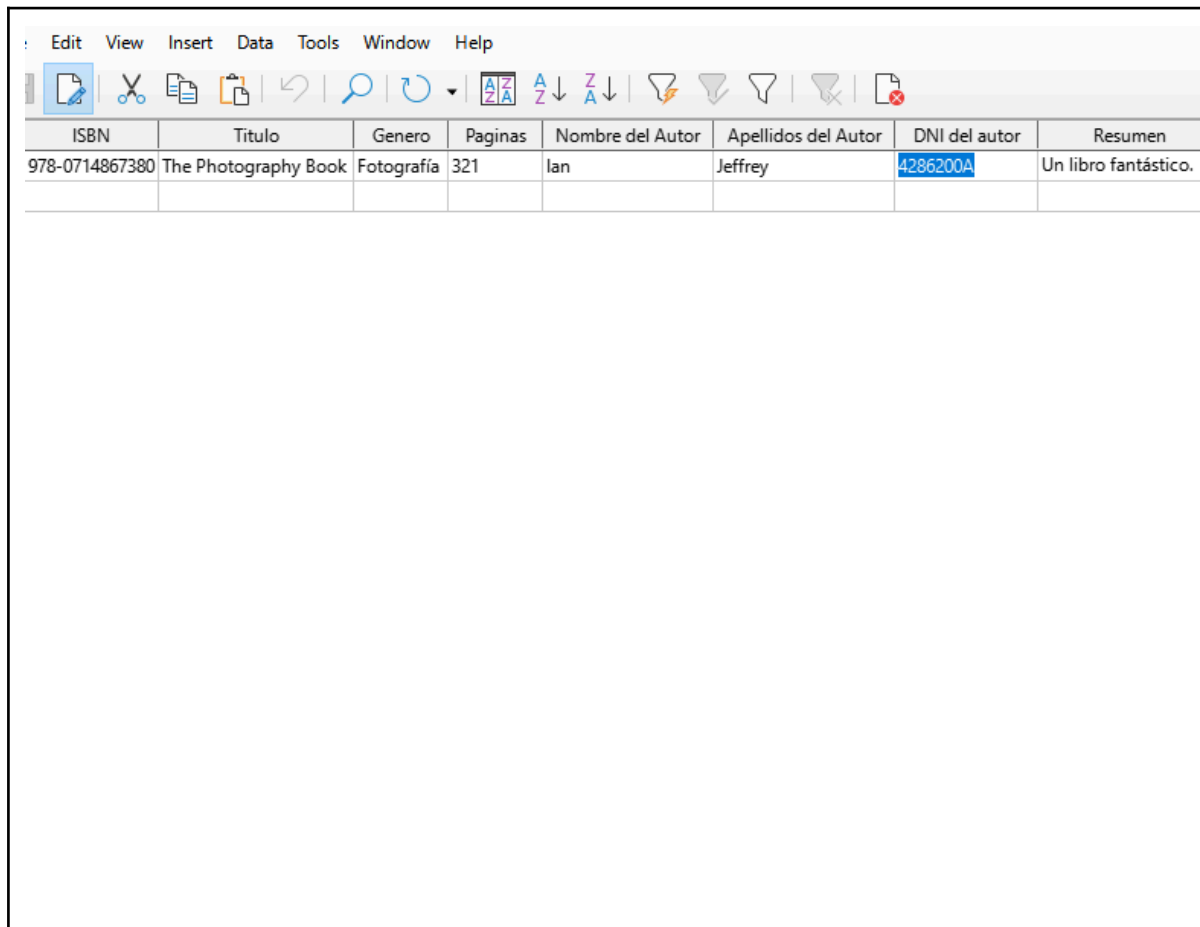


The screenshot shows a spreadsheet application interface. At the top is a menu bar with the following options: File, Edit, View, Insert, Data, Tools, Window, and Help. Below the menu bar is a toolbar containing various icons for file operations (like Save, Open, Print), editing (like Cut, Copy, Paste, Undo, Redo), and data manipulation (like Sort, Filter, AutoFill). The main area of the application is a table with the following columns: ISBN, Titulo, Genero, Paginas, Nombre del Autor, Apellidos del Autor, DNI del autor, and Resumen. The first row of the table is highlighted, and a green plus sign is visible in the first cell of this row, indicating that a new row can be added.

	ISBN	Titulo	Genero	Paginas	Nombre del Autor	Apellidos del Autor	DNI del autor	Resumen
▶+								

Introduzca el DNI del autor justo debajo de la palabra “DNI del autor” y complete el resto de datos hasta que quede como el ejemplo de la siguiente página:

Ejemplo 2.1:



ISBN	Titulo	Genero	Paginas	Nombre del Autor	Apellidos del Autor	DNI del autor	Resumen
978-0714867380	The Photography Book	Fotografía	321	Ian	Jeffrey	4286200A	Un libro fantástico.

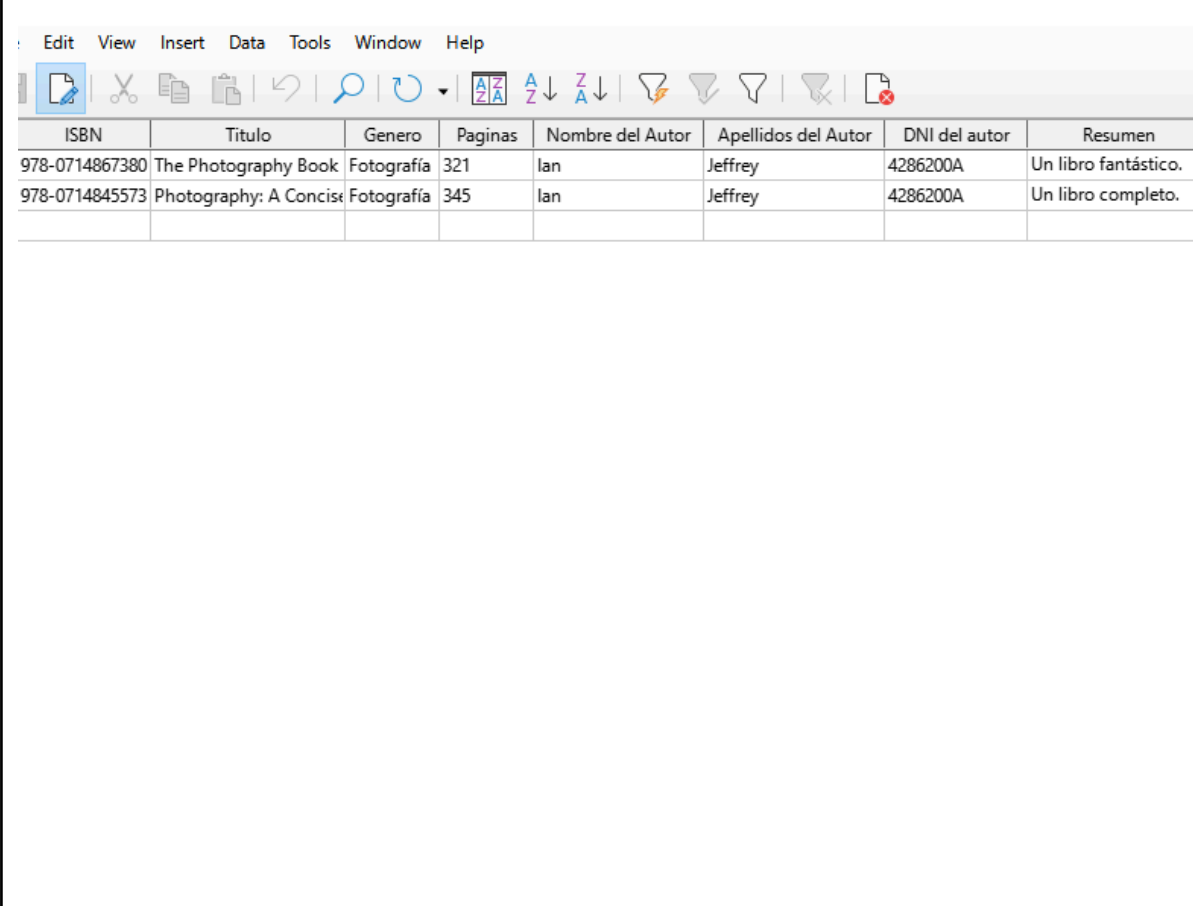
Recuerde pulsar <Enter> cuando introduzca la palabra “fantástico.”, que es la última palabra del resumen del libro.

Ahora ya ha introducido el **primer** registro de la tabla “Libros”. Puede seguir introduciendo cuantos libros desee.

Cierre la base de datos pulsando CTRL+Q. Asegúrese de guardar los cambios.

Ejemplo 2.2:

Abra la tabla “Libros” de su base de datos para introducir el siguiente libro del mismo autor:



ISBN	Título	Genero	Paginas	Nombre del Autor	Apellidos del Autor	DNI del autor	Resumen
978-0714867380	The Photography Book	Fotografía	321	Ian	Jeffrey	4286200A	Un libro fantástico.
978-0714845573	Photography: A Concise	Fotografía	345	Ian	Jeffrey	4286200A	Un libro completo.

Recuerde pulsar <Enter> cuando introduzca la palabra “completo.”, que es la última palabra del resumen del libro.

Ahora ya ha introducido el **segundo** registro de la tabla “Libros”. Puede seguir introduciendo cuantos libros desee.

Cierre la base de datos pulsando CTRL+Q. Asegúrese de guardar los cambios.

Nota: Recuerde que si el DNI del autor no coincide con ningún DNI de la tabla “Autores”, LibreOffice no le dejará avanzar. Por esta razón es mejor copiar en una nota el DNI del autor del cual quiere introducir libros.

Creación de consultas (queries) en nuestra base de datos

En LibreOffice Base, una consulta (query) es una pregunta que hacemos a la base de datos. Decimos qué información queremos obtener y cómo queremos que la presente en pantalla (o en papel).

En palabras sencillas podríamos preguntar a la base de datos lo siguiente: "Muéstrame todos los autores que viven en Telde" o "Calcula el total de ventas de libros de fotografía de este mes".

La consulta (query) muestra la información que necesitamos de una forma organizada.

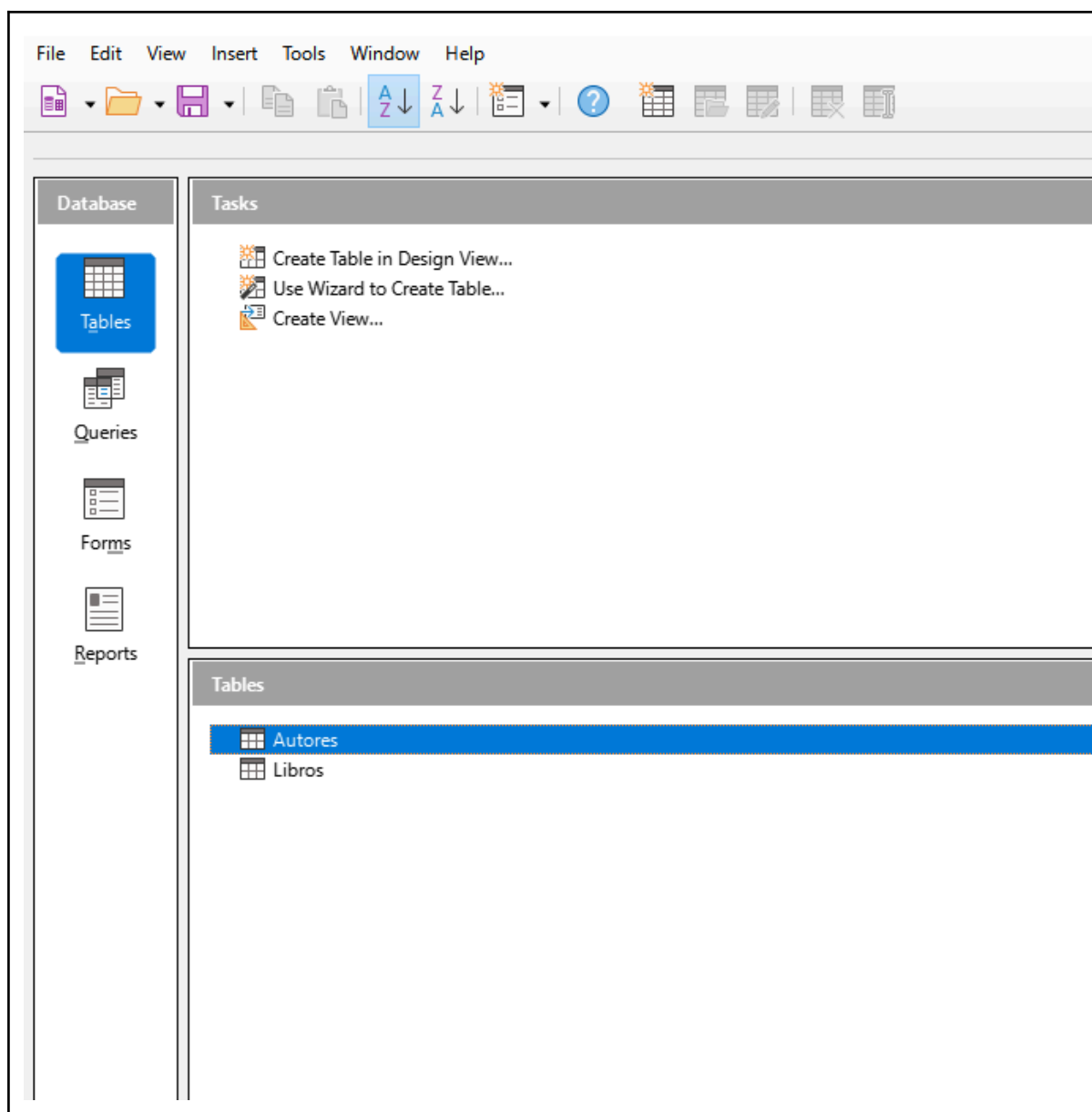
Existen dos modos de crear una consulta:

1. Método sencillo: Usando el ratón, de forma similar a cómo hemos creado las tablas de nuestra base de datos.
2. Método SQL: Escribiendo instrucciones en inglés, siguiendo las reglas del estándar internacional.

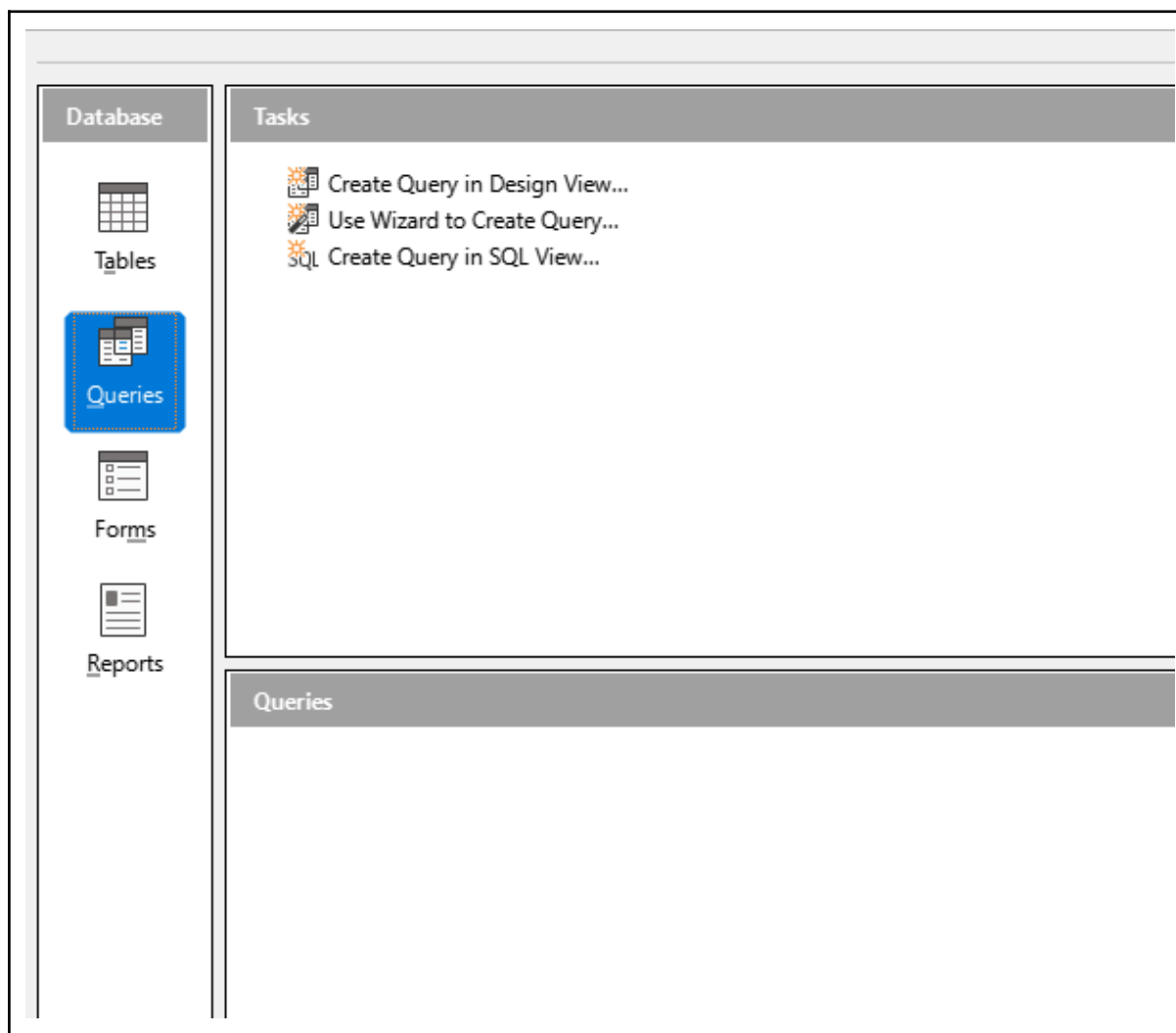
Empecemos por el método 1:

Crearemos una consulta para visualizar a los **autores que viven en la ciudad de Telde**.

Abra la base de datos, para ello, busque en la carpeta “Documentos” de su ordenador el fichero “MiLibreria2025” y haga doble clic en él para abrirlo. Verá esta imagen:

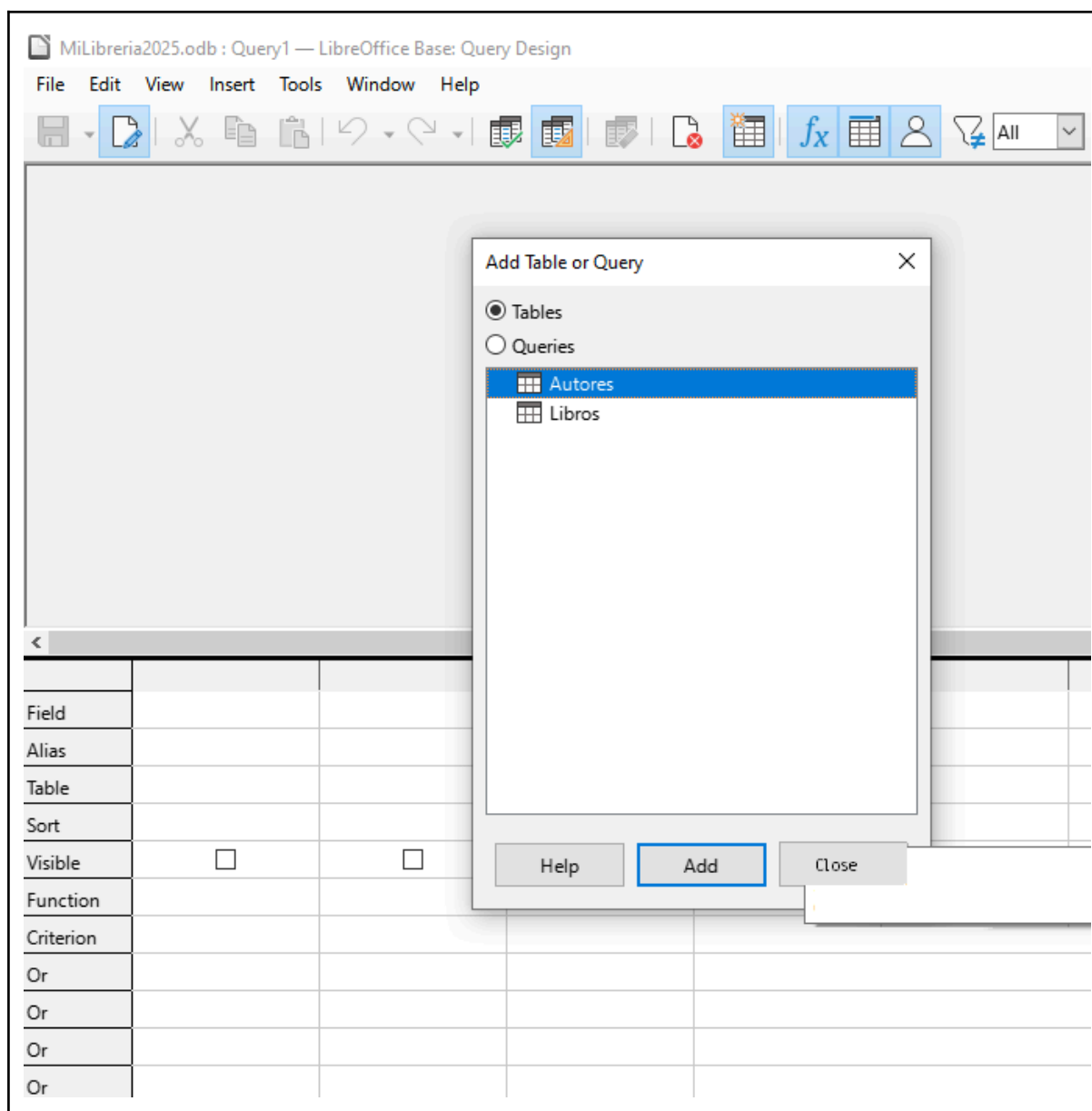


A continuación haga clic en “Queries” (consultas) en la parte izquierda de la pantalla. Verá esta pantalla:



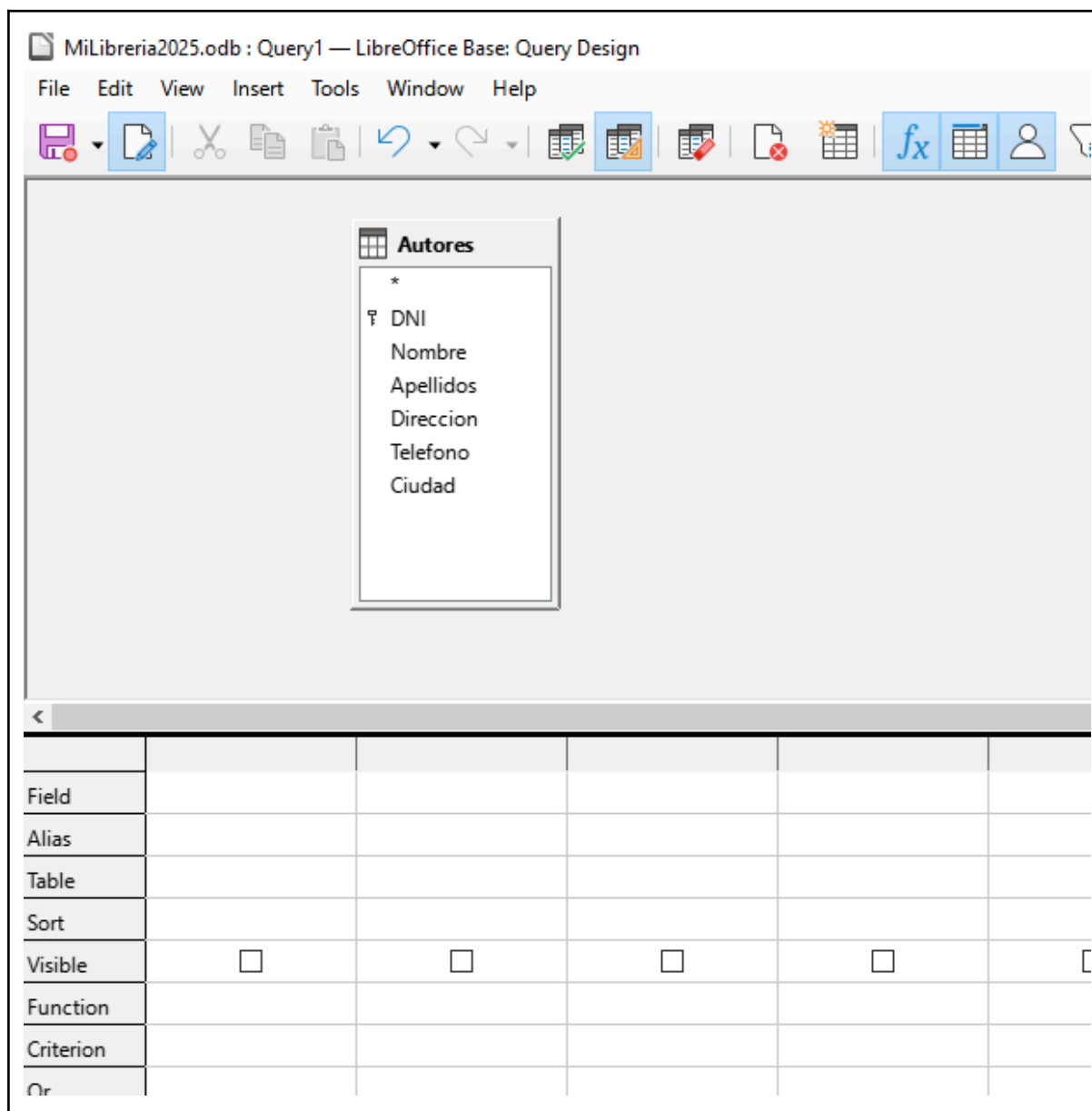
Haga clic en la primera opción: Create query in design view (Crear consulta en la vista diseño).

A continuación, verá esta pantalla:



Haga clic en “Autores” en la ventana central. A continuación haga clic en “Add” (Añadir).

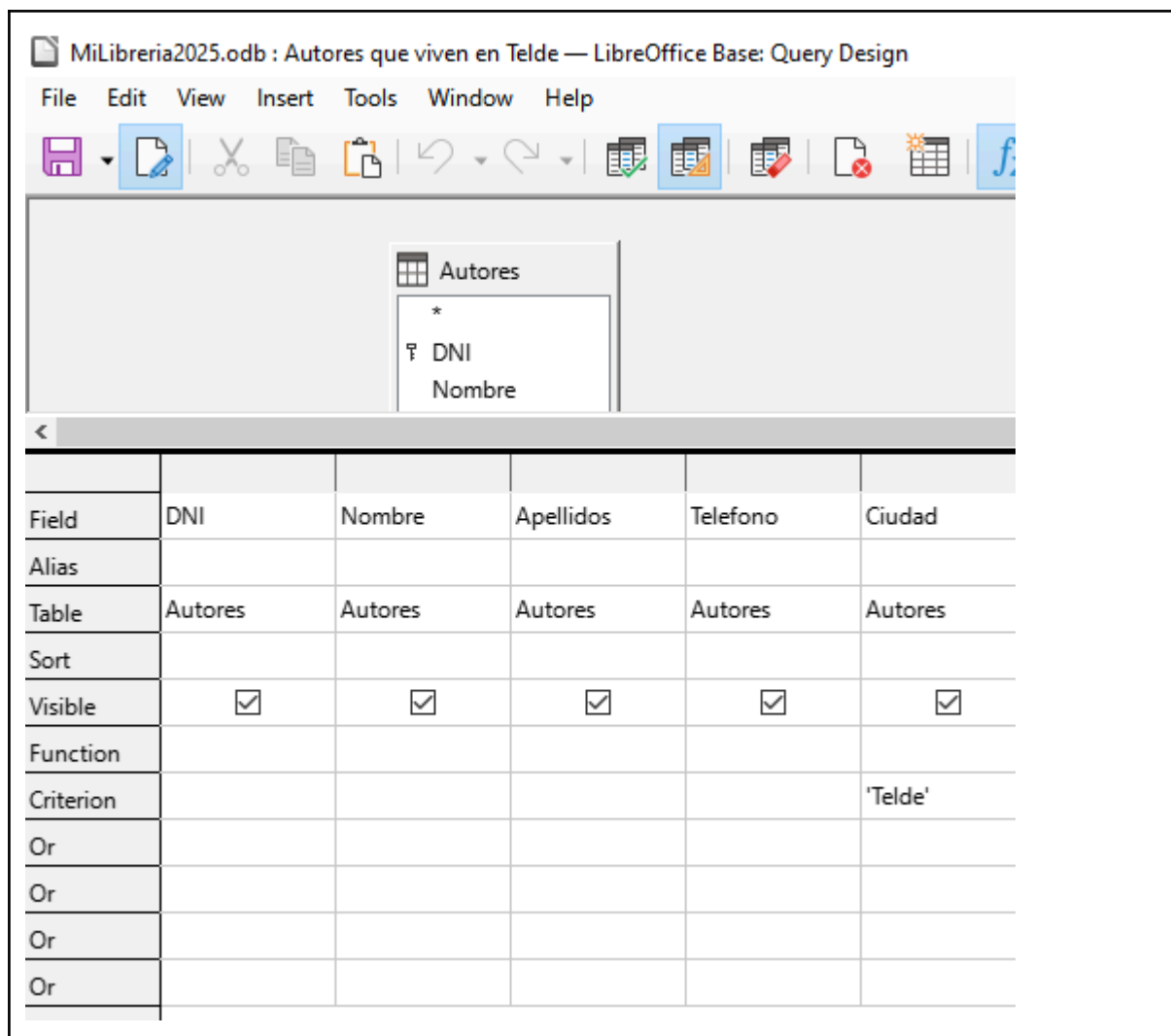
En la pantalla habrá aparecido un cuadro con los campos de la tabla “Autores”:



Ahora haremos doble clic en cada uno de los siguientes campos del cuadro que acaba de aparecer: DNI, Nombre, Apellidos, Teléfono y Ciudad.

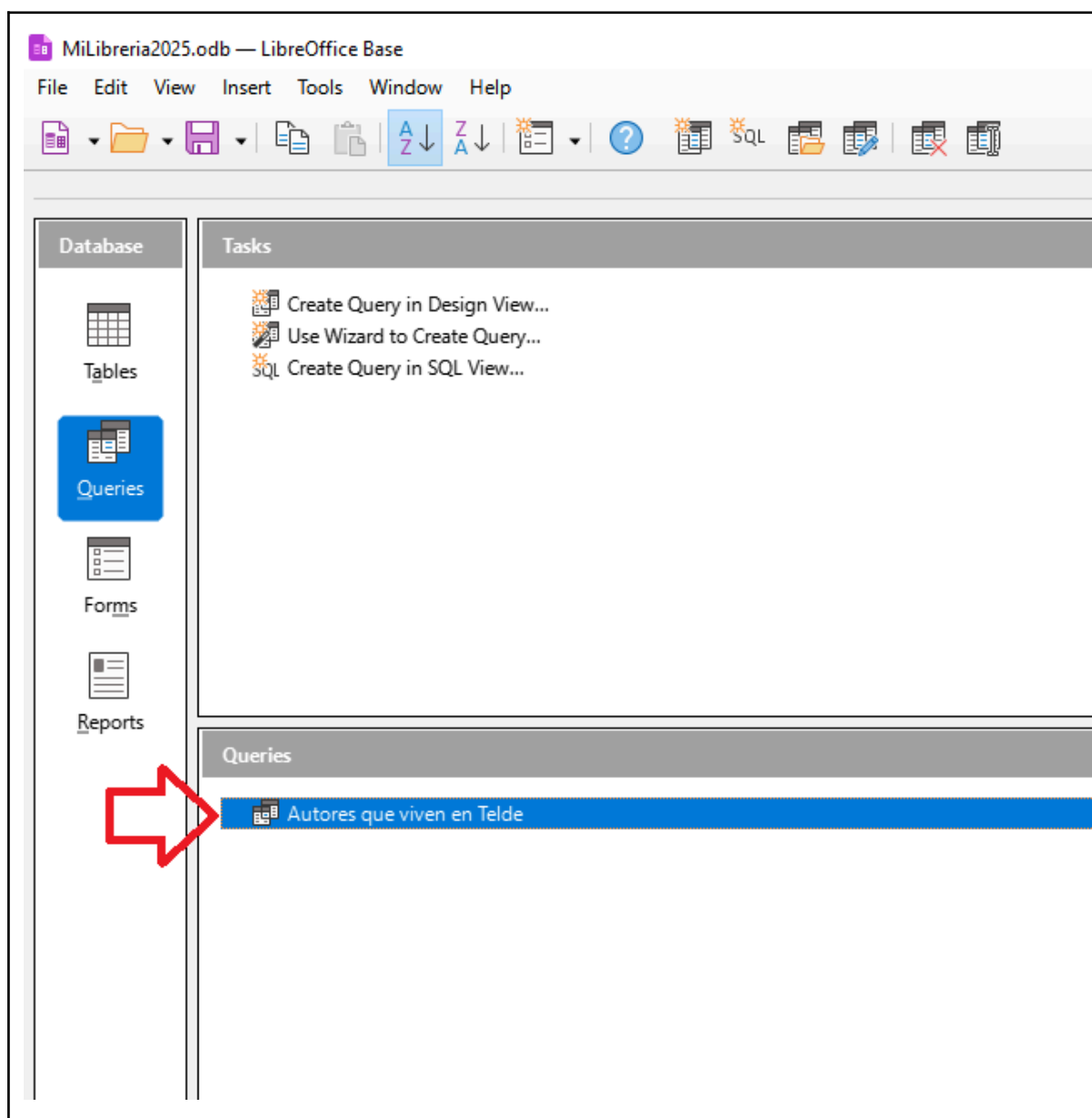
Al hacer doble clic en estos campos, hemos seleccionado los campos que queremos que aparezcan en la consulta (query).

Para que LibreOffice nos muestre los autores de Telde, debemos escribir la palabra “Telde” (que es nuestro criterio de búsqueda) en la casilla de intersección de “Criterion” con “Ciudad” (vea la página siguiente).



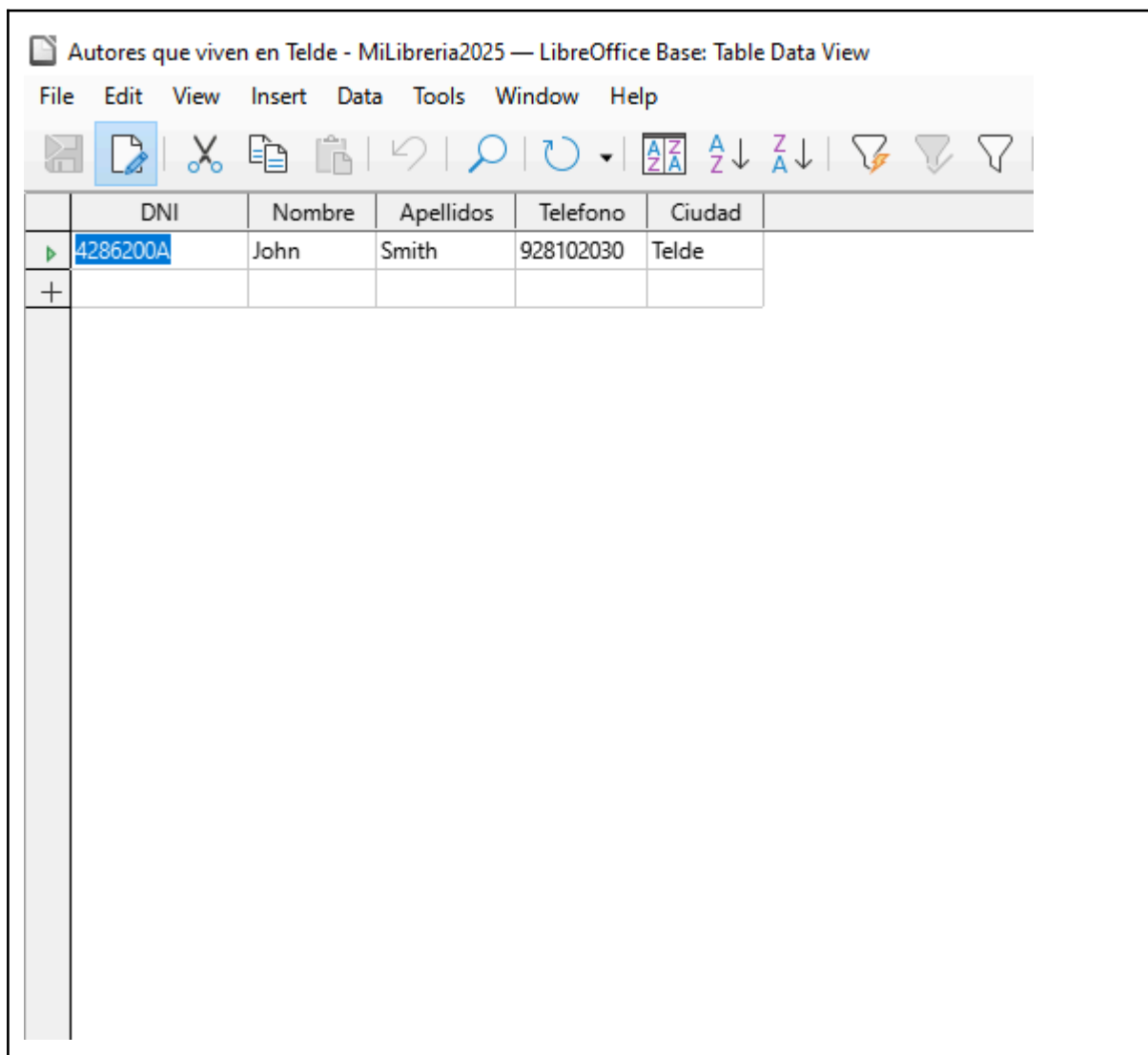
Haremos clic en “File” y después clic en “Close”. la grabaremos con el nombre “Autores que viven en la ciudad de Telde” (No la llamaremos “Query 1”).

La consulta que acaba de crear se puede localizar en la ventana principal de LibreOffice:



Al hacer doble clic sobre la consulta “Autores que viven en Telde” se mostrará el resultado:

Resultado de la consulta (query):



The screenshot shows the 'Autores que viven en Telde - MiLibreria2025' window in LibreOffice Base. The window has a menu bar (File, Edit, View, Insert, Data, Tools, Window, Help) and a toolbar with various icons. Below the toolbar is a table with the following data:

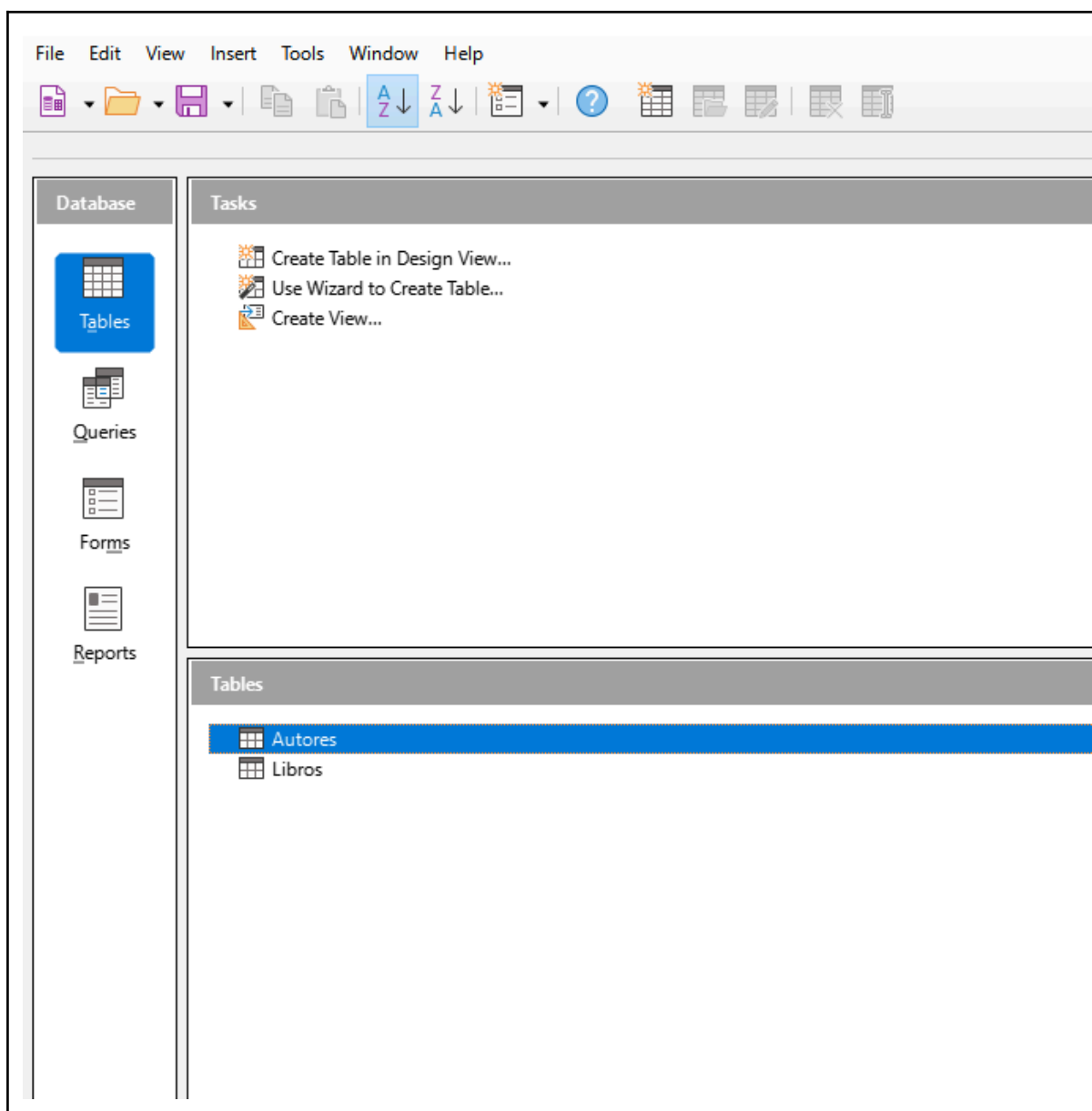
	DNI	Nombre	Apellidos	Telefono	Ciudad
▶	4286200A	John	Smith	928102030	Telde
+					

Haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q.

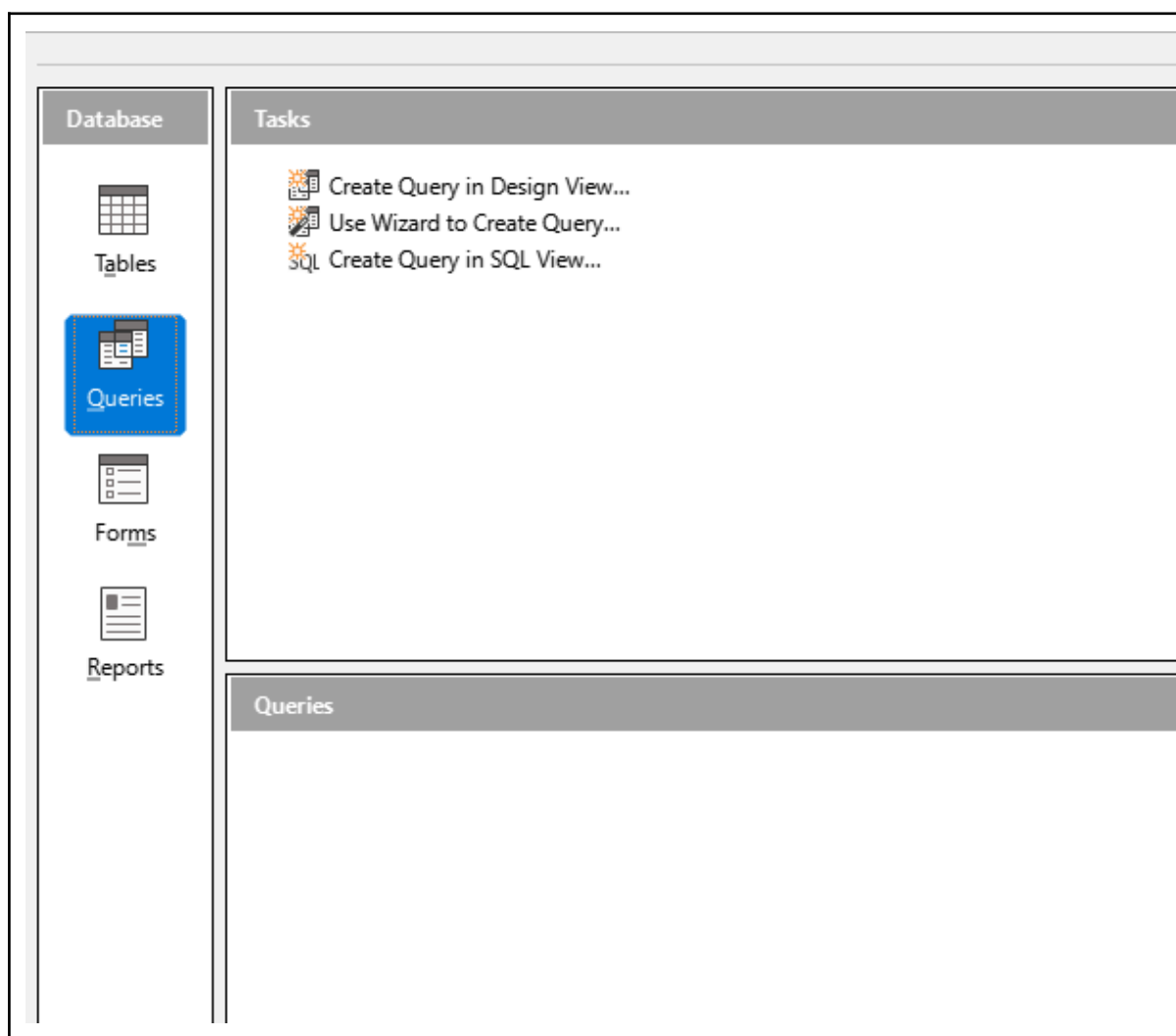
A continuación crearemos una consulta utilizando el método 2: **SQL**.

Crearemos una consulta para visualizar los **libros que pertenecen al género de fotografía**.

Abra la base de datos, para ello, busque en la carpeta “Documentos” de su ordenador el fichero “MiLibreria2025” y haga doble clic en él para abrirlo. Verá esta imagen:

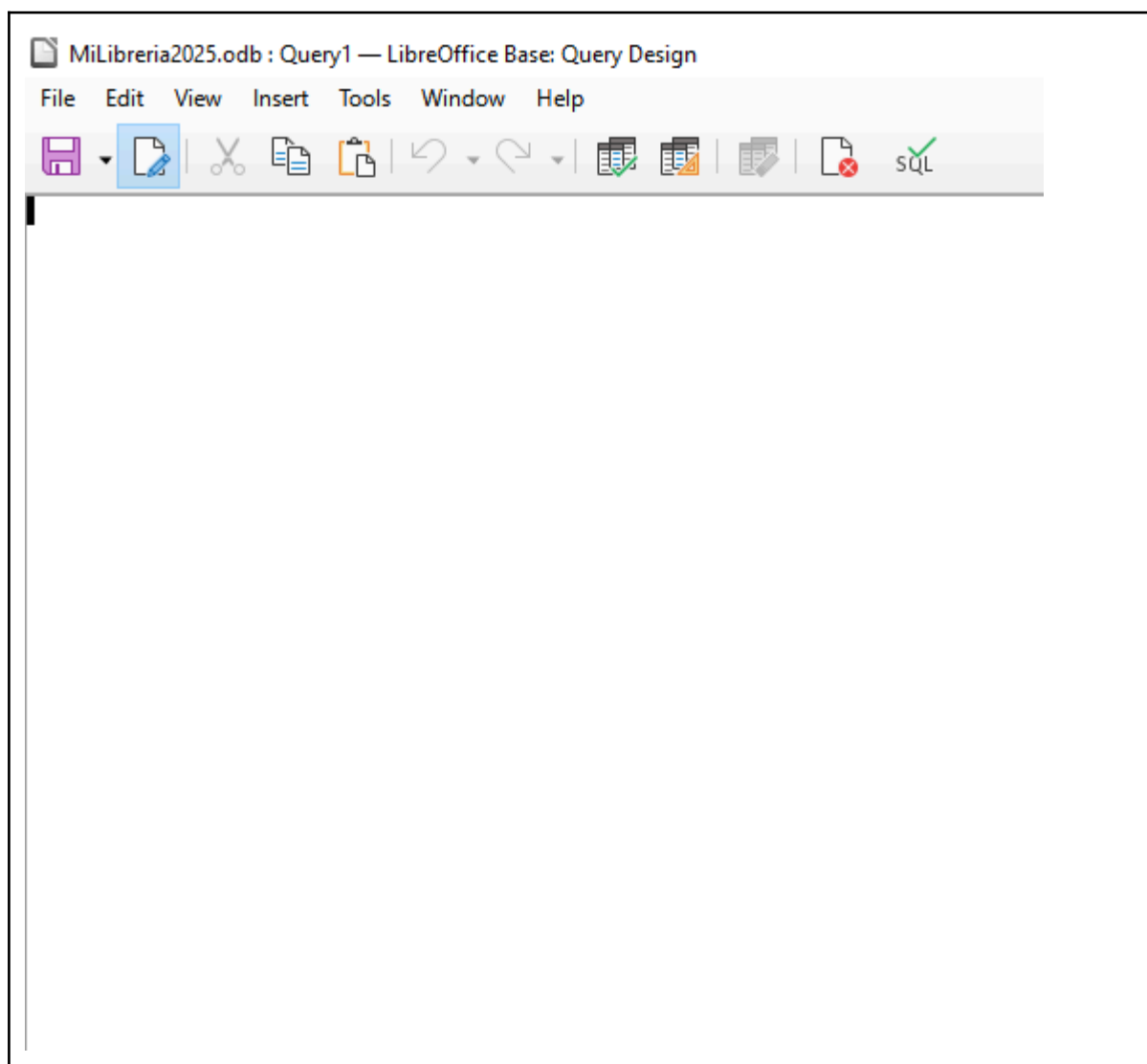


A continuación haga clic en “Queries” (consultas) en la parte izquierda de la pantalla. Verá esta pantalla:



Haga clic en la tercera opción: Create query in SQL view (Crear consulta en la vista SQL).

Aparecerá esta pantalla en blanco:

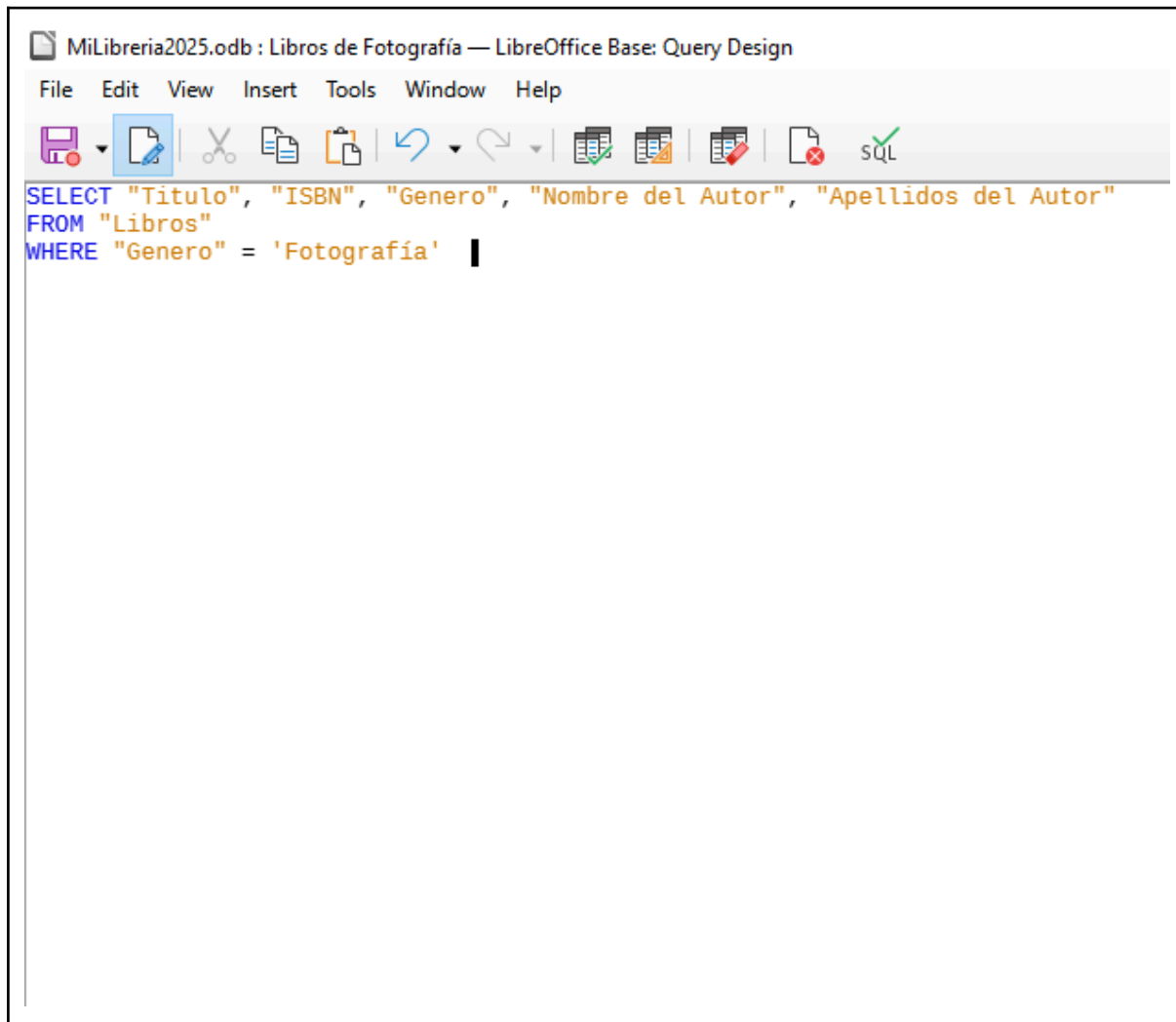


En esta pantalla en blanco deberá introducir las órdenes necesarias escritas en SQL para la consulta que necesita crear.

Es importante resaltar que al definir nuestras tablas, no acentuamos las palabras. Sin embargo, al introducir datos, sí acentuamos las palabras.

Copie cada palabra (e instrucción) de la siguiente página tal cual se muestra en el ejemplo:

Ejemplo:



Nota: Compruebe los acentos antes de grabar su consulta. Recuerde que LibreOffice diferencia entre comillas simples y comillas dobles.

Haremos clic en “File” y después clic en “Close”. la grabaremos con el nombre “Libros de fotografía” (No la llamaremos “Query 1”).

Haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q.

Compruebe que el Query (consulta) muestra los libros de fotografía:

Libros de Fotografía - MiLibreria2025 — LibreOffice Base: Table Data View

File Edit View Insert Data Tools Window Help
















	Titulo	ISBN	Genero	Nombre del Autor	Apellidos del Autor
▶	Photography: A Concise History	978-0714845573	Fotografía	Ian	Jeffrey
	The Photography Book	978-0714867380	Fotografía	Ian	Jeffrey
+					

Ahora que hemos introducido algunos libros más en la tabla de “Libros”, vamos a diseñar una segunda consulta SQL para visualizar los libros del género “Linux”.

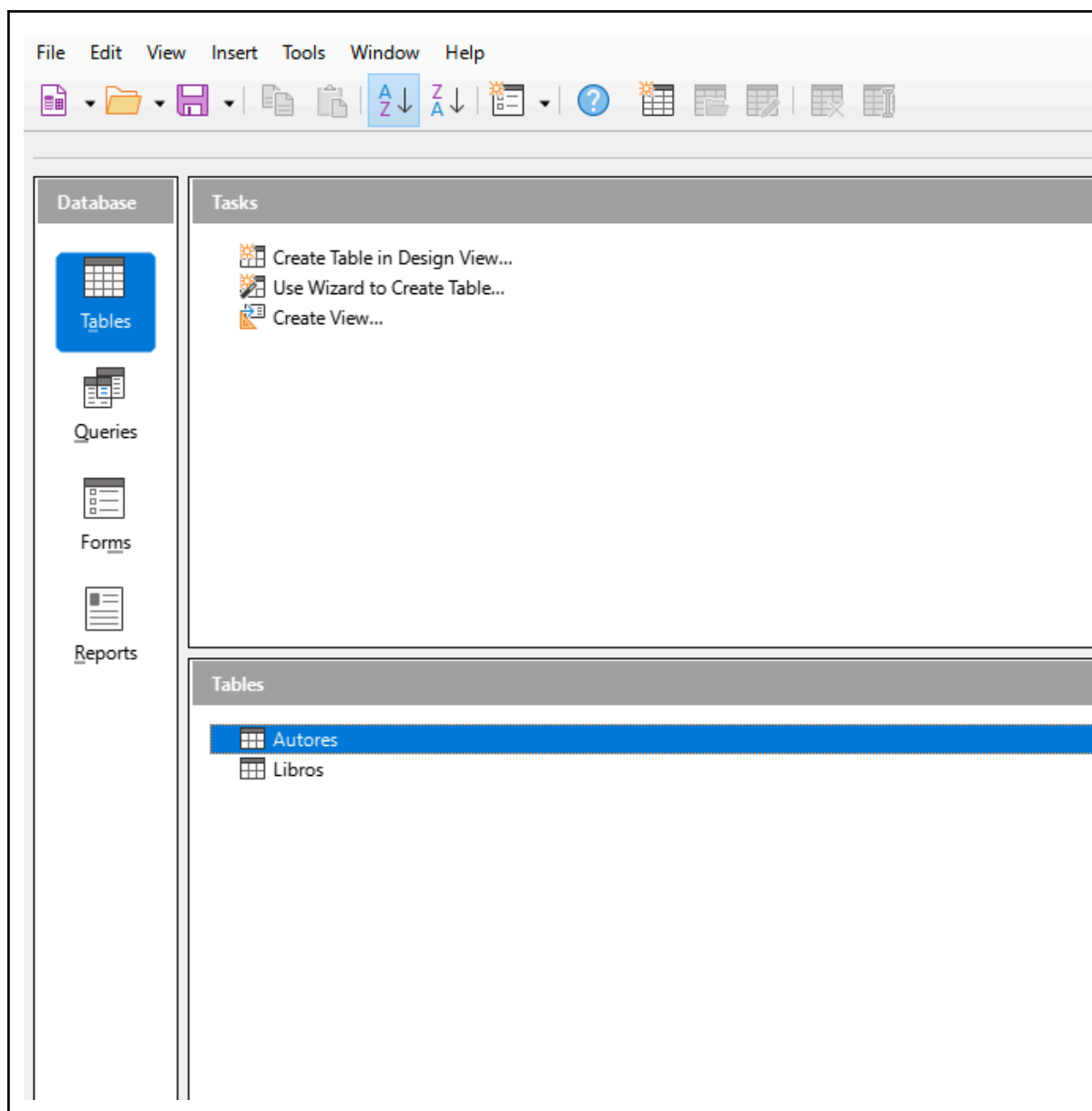
Esta es la tabla “Libros” que contiene los libros de nuestra base de datos:

ISBN	Titulo	Genero	Paginas	Nombre del Autor	Apellidos del Autor
84-267-1311-4	Red Hat Linux 7	Linux	158	Juan	Espinosa
84-415-1043-1	Red Hat Linux 6	Linux	290	Santiago	Eibe-Garcia
84-481-3112-6	Corel Linux	Linux	560	Joe	Merlino
84-7614-290-0	Guia completa de Dbase IV	Bases de datos	659	Douglas	Hergert
978-0714845573	Photography: A Concise History	Fotografía	345	Ian	Jeffrey
978-0714867380	The Photography Book	Fotografía	321	Ian	Jeffrey

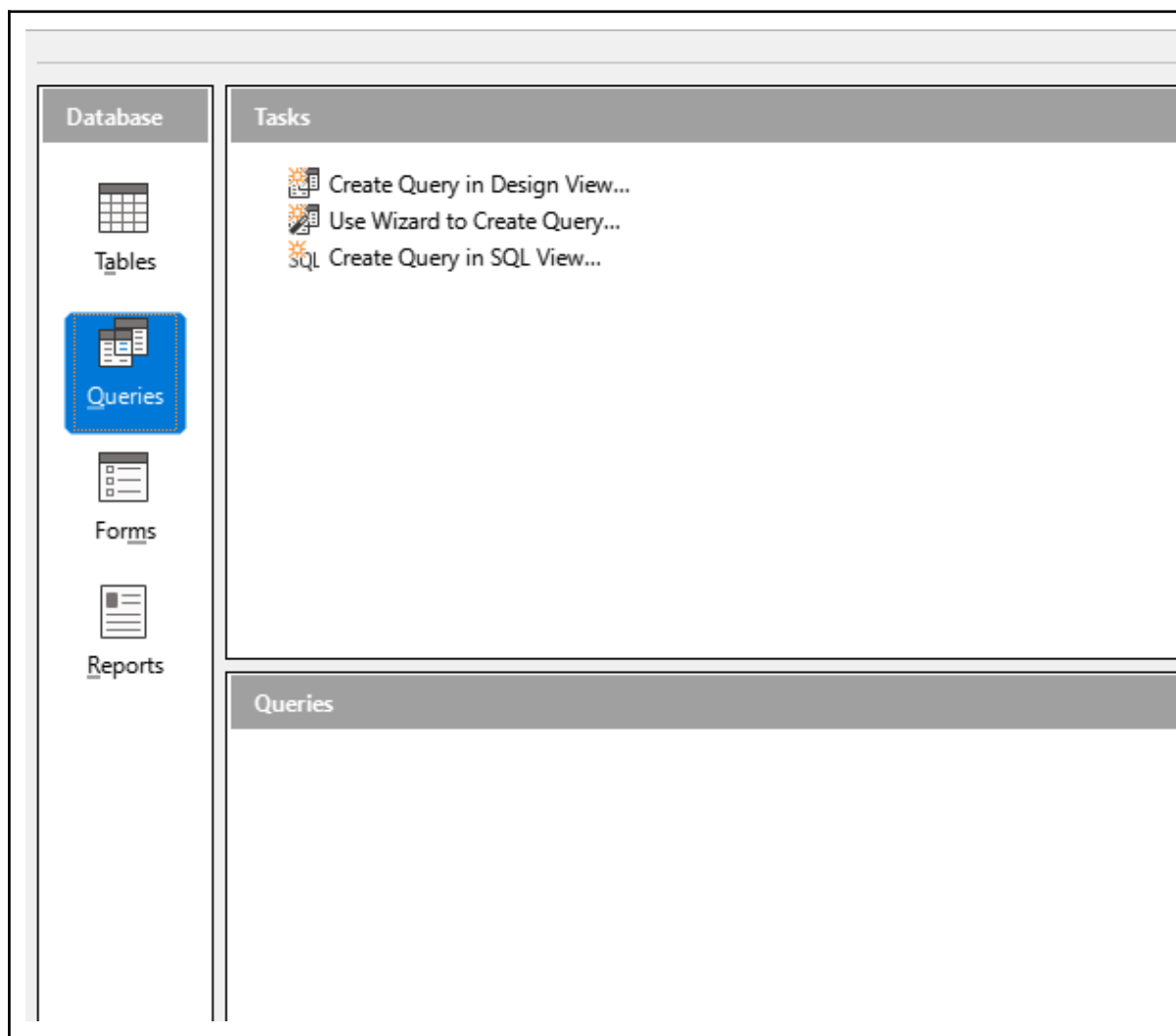
Podemos ver que tenemos libros sobre Fotografía y libros sobre el sistema operativo Linux en nuestra base de datos.

Crearemos una consulta para visualizar los **libros que pertenecen al género Linux**.

Abra la base de datos, para ello, busque en la carpeta “Documentos” de su ordenador el fichero “MiLibreria2025” y haga doble clic en él para abrirlo. Verá esta imagen:

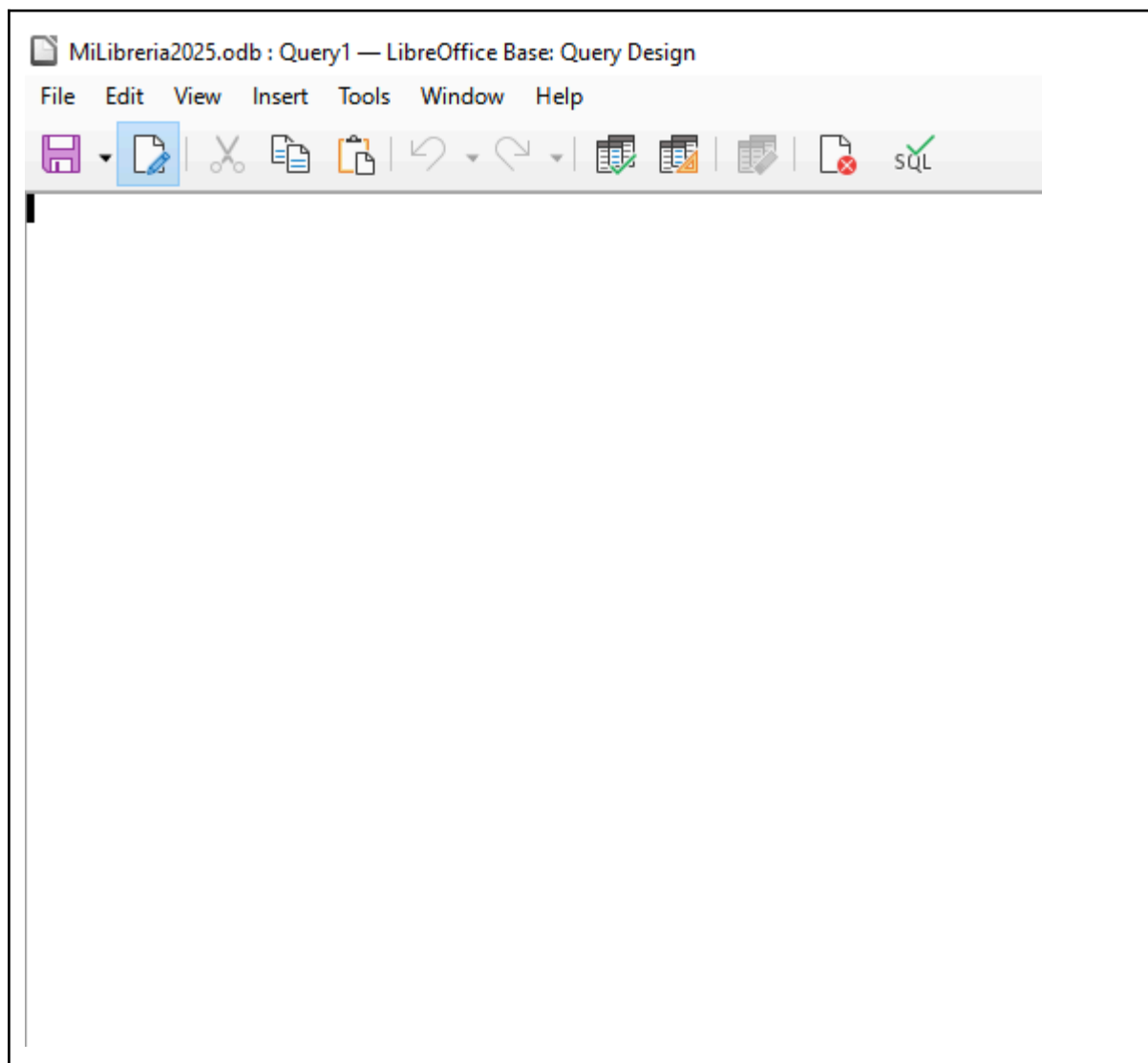


A continuación haga clic en “Queries” (consultas) en la parte izquierda de la pantalla. Verá esta pantalla:



Haga clic en la tercera opción: Create query in SQL view (Crear consulta en la vista SQL).

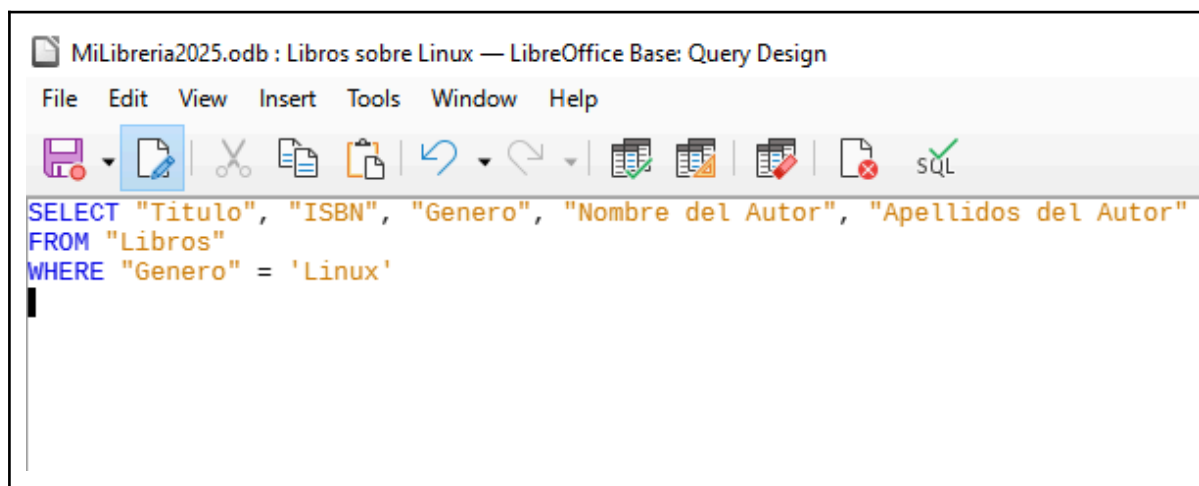
Aparecerá esta pantalla en blanco:



En esta pantalla en blanco deberá introducir las órdenes necesarias escritas en SQL para la consulta que necesita crear.

Copie cada palabra (e instrucción) de la siguiente página tal cual se muestra en el ejemplo:

Ejemplo:



Nota: Compruebe los acentos antes de grabar su consulta.

Haremos clic en “File” y después clic en “Close”. la grabaremos con el nombre “Libros sobre Linux” (No la llamaremos “Query 1”).

Haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q.

Nota: Si no desea incluir en la consulta el nombre y los apellidos del autor, la orden SQL quedaría de la siguiente forma:

```
SELECT "Titulo", "ISBN", "Genero" FROM "Libros" WHERE "Genero" = 'Linux'
```

Observe que los campos de la tabla utilizada (Libros) van entre comillas y la palabra (Linux) que queremos usar como filtro va entre comillas simples. La palabra “Genero” no va acentuada. Es un campo de la tabla.

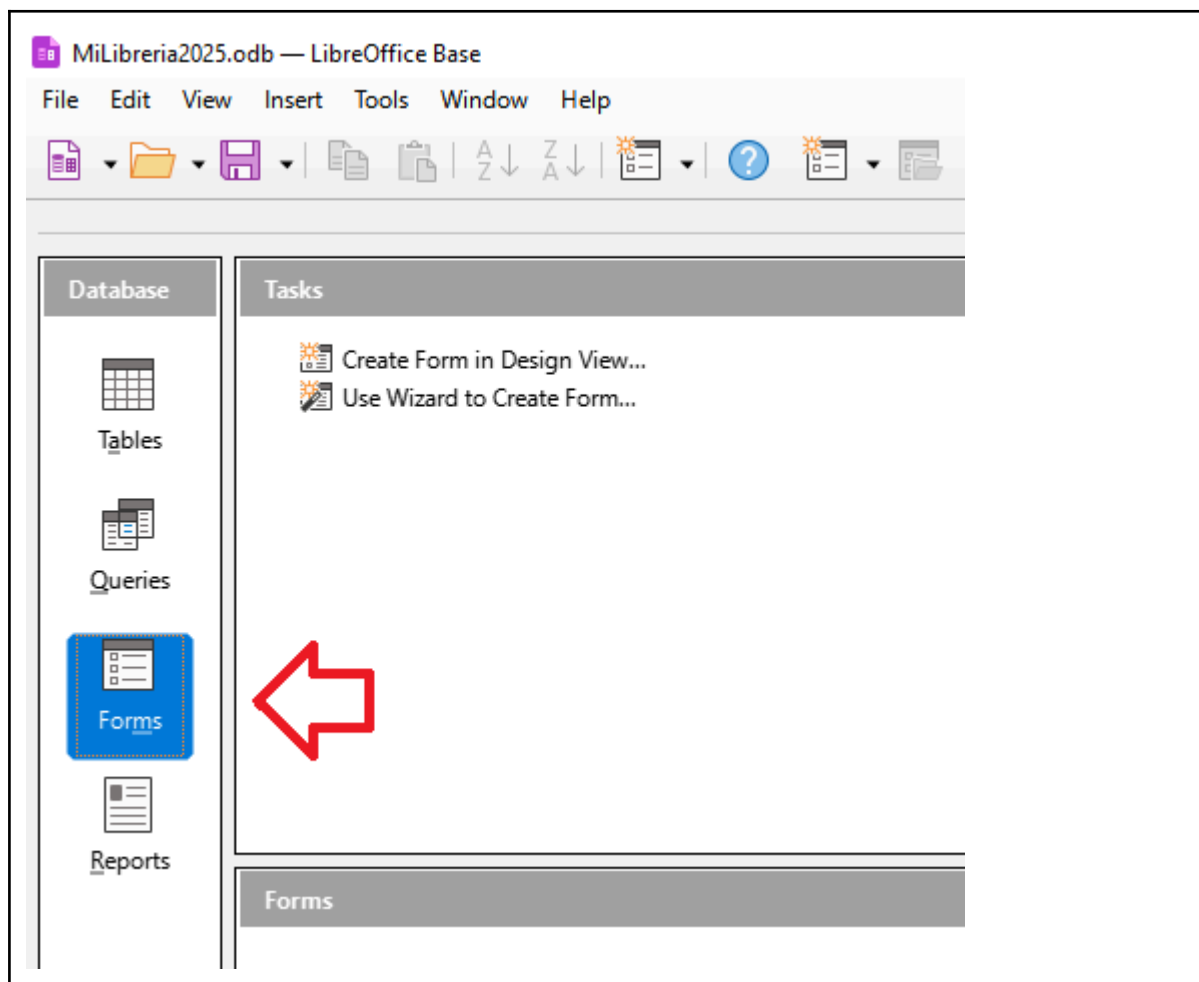
Para hacer estos cambios necesita editar la consulta SQL. Para ello, haga clic con el botón derecho del ratón en la consulta a editar y elija **Editar en Vista SQL** (no olvide grabar los cambios).

Creación de formularios (forms)

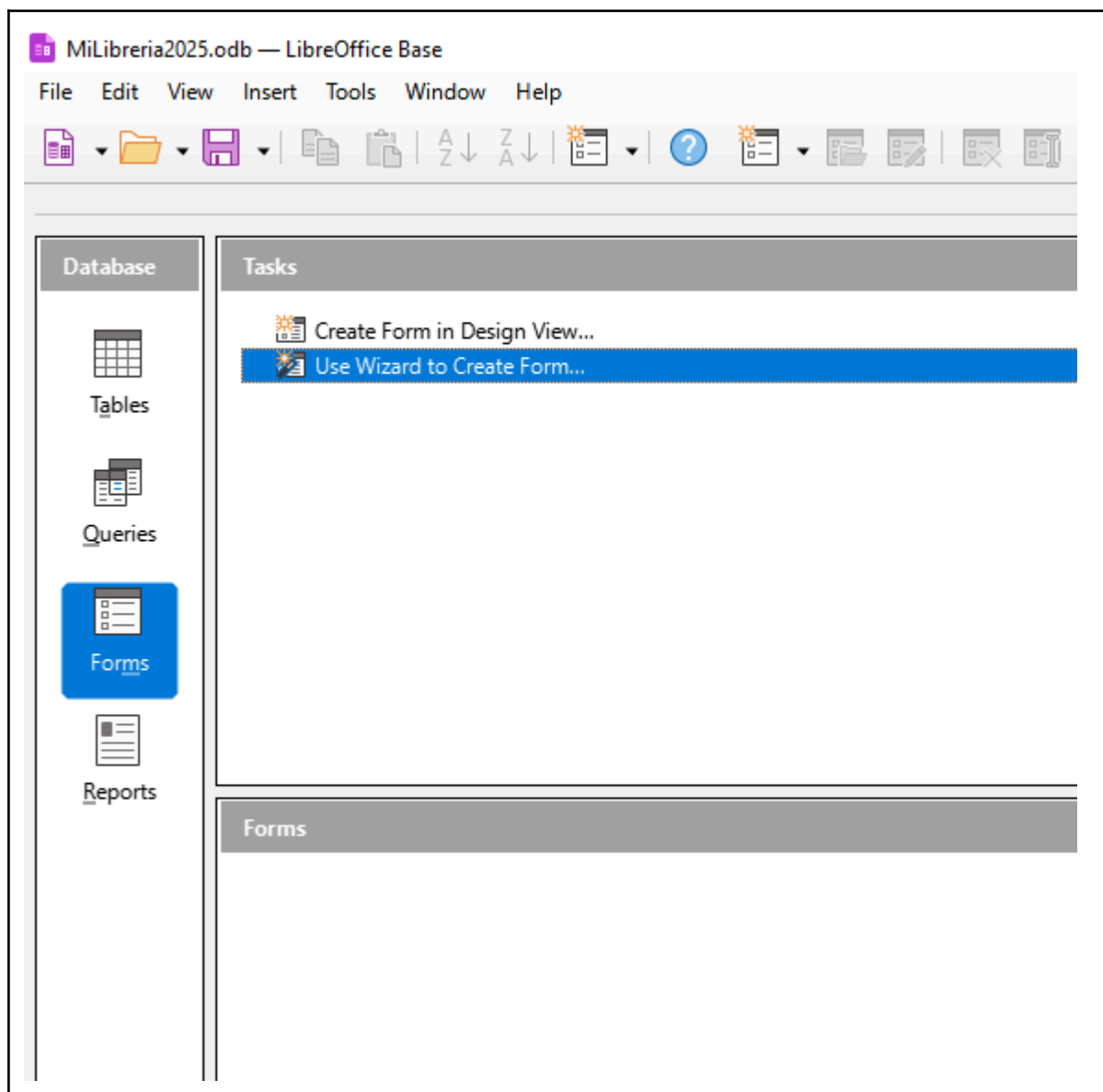
A la hora recopilar información de los autores, como sus nombres, direcciones o teléfonos utilizamos los formularios. Un formulario permite organizar esa información en un formato fácil de usar.

Podríamos decir, entonces, que el formulario de clientes permite ver la ficha de cada cliente de nuestra base de datos y el formulario de libros permite ver la ficha de cada libro de nuestra librería.

Para crear un formulario debemos abrir nuestra base de datos y hacer clic en Forms (formularios) en la parte izquierda de la ventana principal.

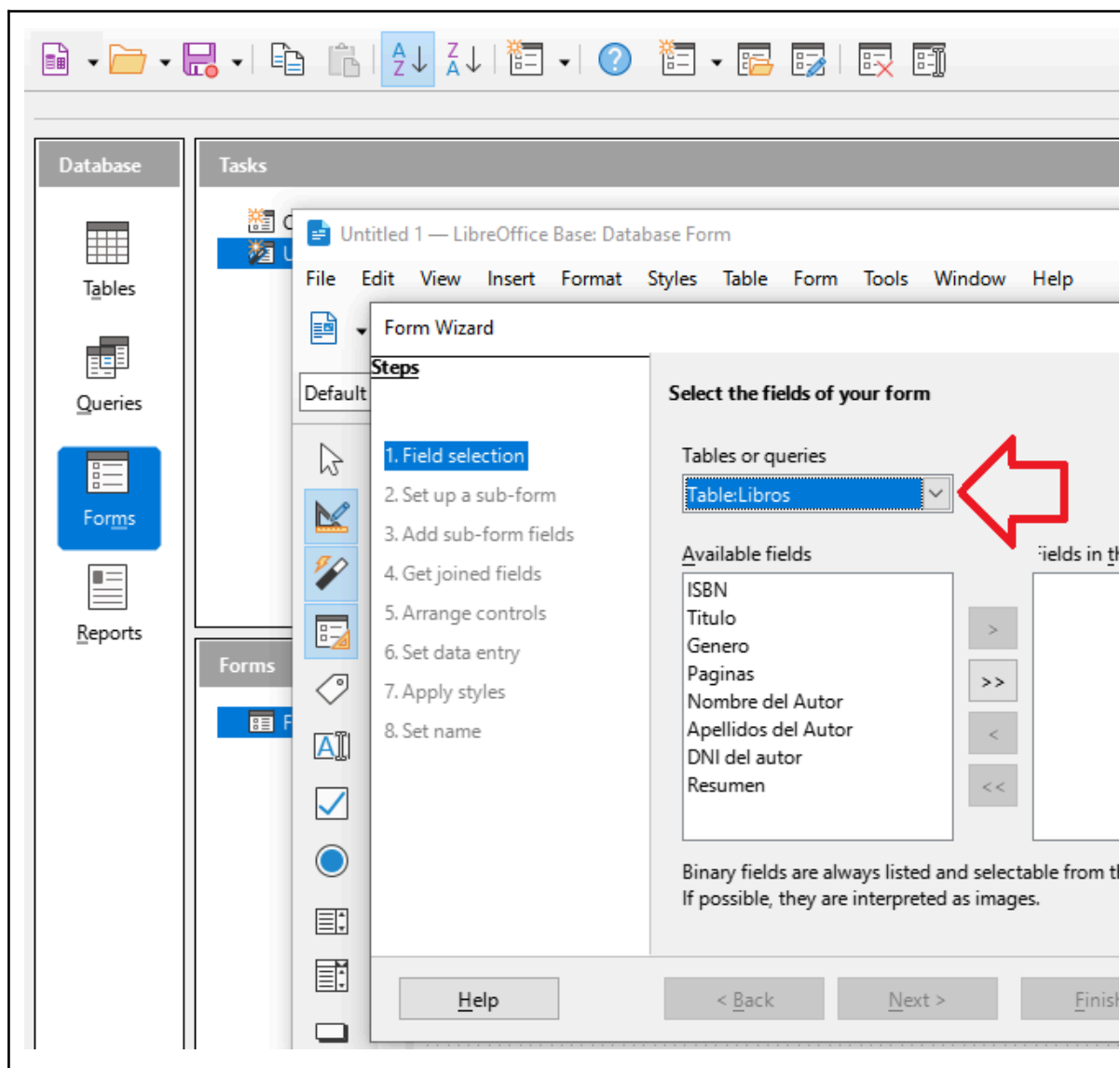


A continuación, haga clic en la opción resaltada “Use wizard to create form” (usar el asistente para crear un formulario).



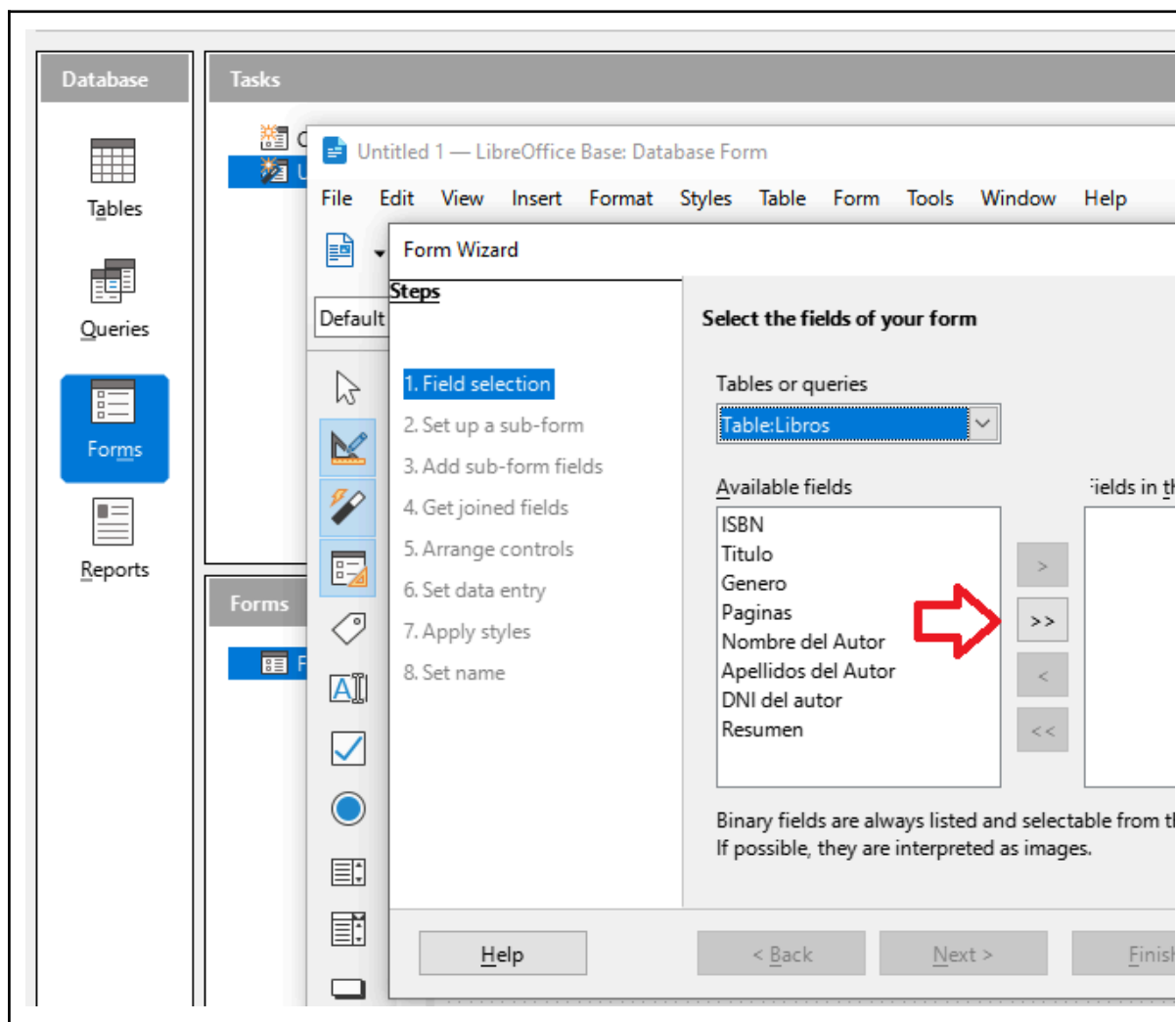
Seguidamente, verá esta pantalla:

Elija la tabla “Libros” del menú desplegable:



Al seleccionar la tabla “Libros” verá todos los campos disponibles (available fields) de la tabla elegida.

Haga clic en el símbolo >> para seleccionar todos los campos de la tabla libros y que así puedan aparecer en el formulario (form).



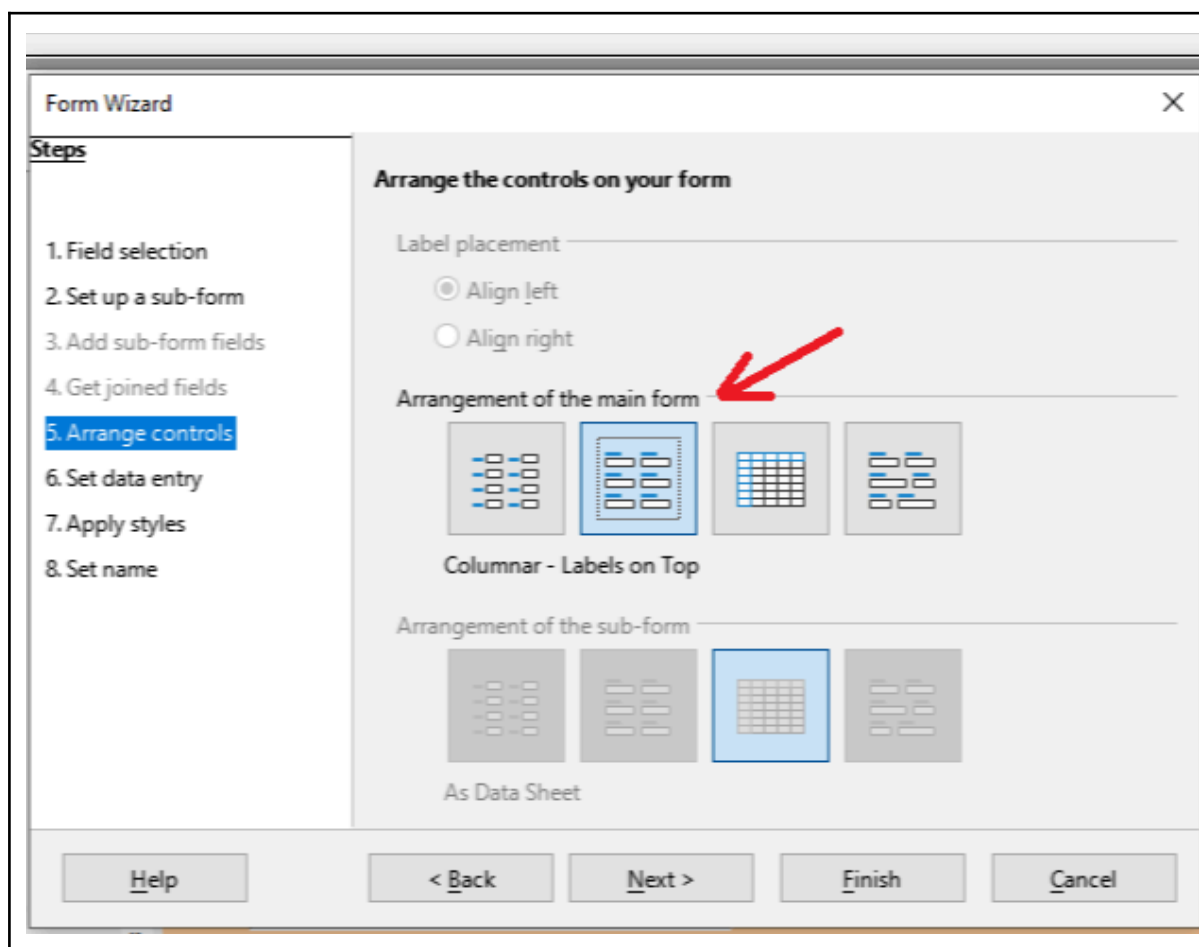
Todos los campos disponibles habrán pasado al cuadro de la derecha.

A continuación, decidiremos si necesitamos un subformulario. De momento, no lo utilizaremos.

The screenshot shows the 'Form Wizard' dialog box in Microsoft Access, specifically the 'Set up a sub-form' step. The 'Steps' list on the left includes: 1. Field selection, 2. Set up a sub-form (highlighted), 3. Add sub-form fields, 4. Get joined fields, 5. Arrange controls, 6. Set data entry, 7. Apply styles, and 8. Set name. The main area is titled 'Decide if you want to set up a sub-form' and contains three options: 'Add Sub-form' (unchecked), 'Sub-form based on existing relation' (radio button), and 'Sub-form based on manual selection of fields' (radio button, selected). The 'Sub-form based on existing relation' option has a text box labeled 'Which relation do you want to add?'. A help icon and text explain that a sub-form is a form inserted into another form to show data from tables or queries with a one-to-many relationship. At the bottom, there are buttons for 'Help', '< Back', 'Next >' (highlighted), 'Finish', and 'Cancel'.

Haga clic en “Next” para continuar.

Haga clic en la segunda opción en “Arrangement of the main form” (colocación del formulario principal):



A continuación, haga clic en “Next”.

Verá la siguiente pantalla:

Form Wizard

Steps

1. Field selection
2. Set up a sub-form
3. Add sub-form fields
4. Get joined fields
5. Arrange controls
- 6. Set data entry**
7. Apply styles
8. Set name

Select the data entry mode

☐ The form is to be used for entering new data only.
Existing data will not be displayed

☒ The form is to display all data

☐ Do not allow modification of existing data

☐ Do not allow deletion of existing data

☐ Do not allow addition of new data

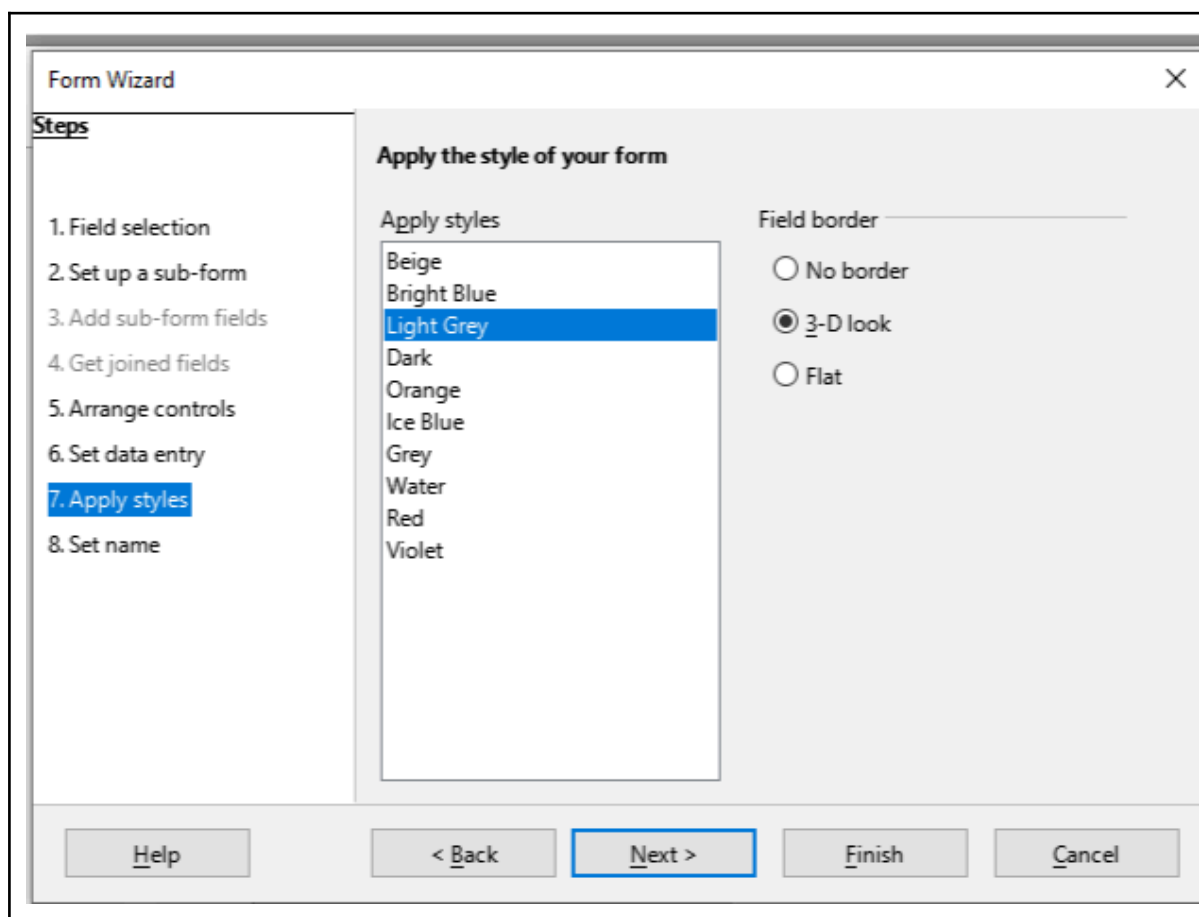
Help < Back Next > Finish Cancel

Puede marcar cualquiera de las tres casillas disponibles si no desea:

- Permitir la modificación de datos existentes.
- Permitir el borrado de datos existentes.
- Permitir añadir nuevos datos.

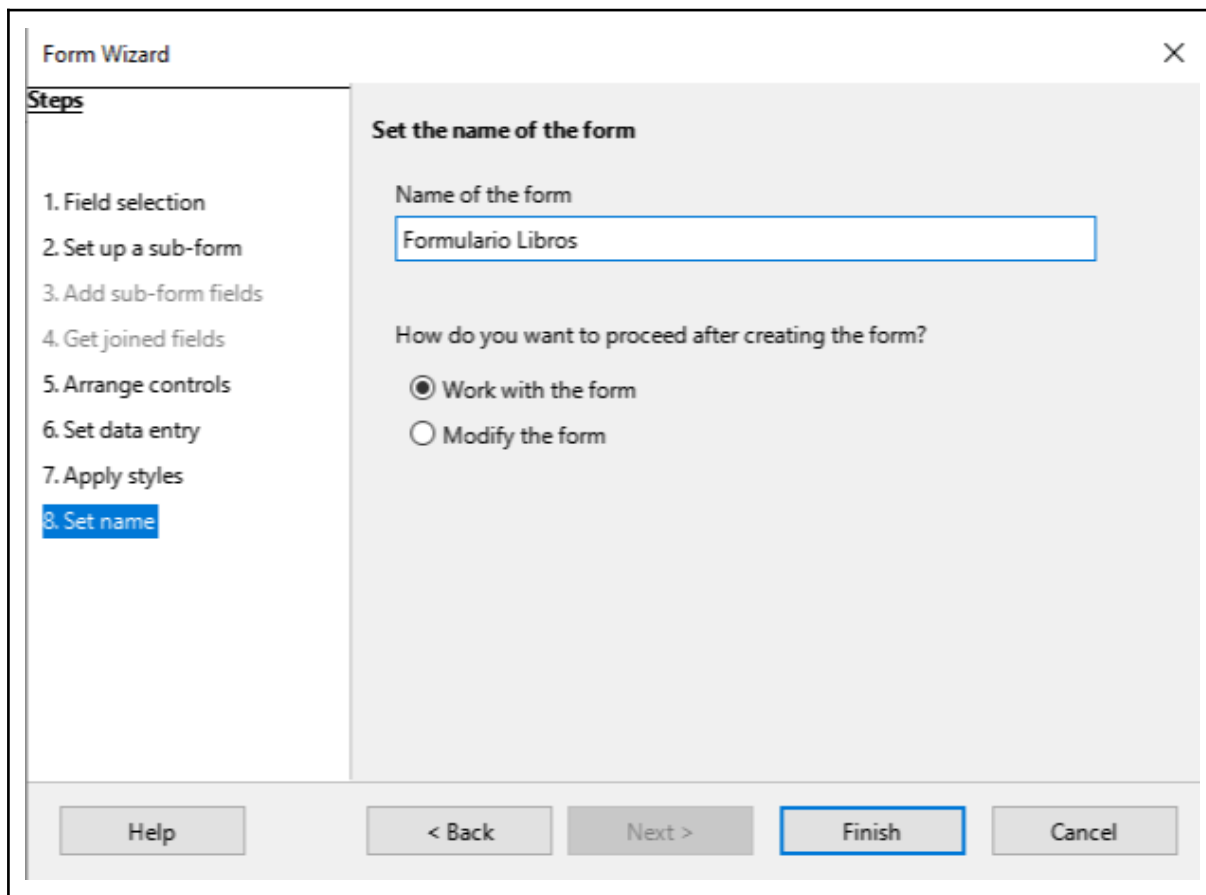
No marcaremos ninguna casilla de momento. Haga clic en “Next”.

En este paso, elegiremos el estilo de nuestro formulario y de los bordes de los campos de la tabla:



Haga clic en “Next” para continuar:

En este paso, escriba el nombre del formulario: “Formulario libros”.



The image shows a 'Form Wizard' dialog box with a close button (X) in the top right corner. On the left, a 'Steps' list contains eight items: '1. Field selection', '2. Set up a sub-form', '3. Add sub-form fields', '4. Get joined fields', '5. Arrange controls', '6. Set data entry', '7. Apply styles', and '8. Set name'. The '8. Set name' step is highlighted with a blue background. The main area of the dialog is titled 'Set the name of the form'. It contains a text input field labeled 'Name of the form' with the text 'Formulario Libros' entered. Below this, there is a question 'How do you want to proceed after creating the form?' with two radio button options: 'Work with the form' (which is selected) and 'Modify the form'. At the bottom of the dialog, there are five buttons: 'Help', '< Back', 'Next >', 'Finish' (which is highlighted with a blue border), and 'Cancel'.

A continuación, haga clic en “Finish” para ver el resultado de su diseño:

Este es el resultado de su diseño:

The screenshot shows a LibreOffice Forms design window with a blue background. On the left is a vertical toolbar with various icons. The main area displays a form with the following fields:

- ISBN**: 84-267-1311-4
- Título**: Red Hat Linux 7
- Genero**: Linux
- Paginas**: 158
- Nombre del Autor**: Juan
- Apellidos del Autor**: Espinosa
- DNI del autor**: 42888000L
- Resumen**: Un libro completo sobre Linux.

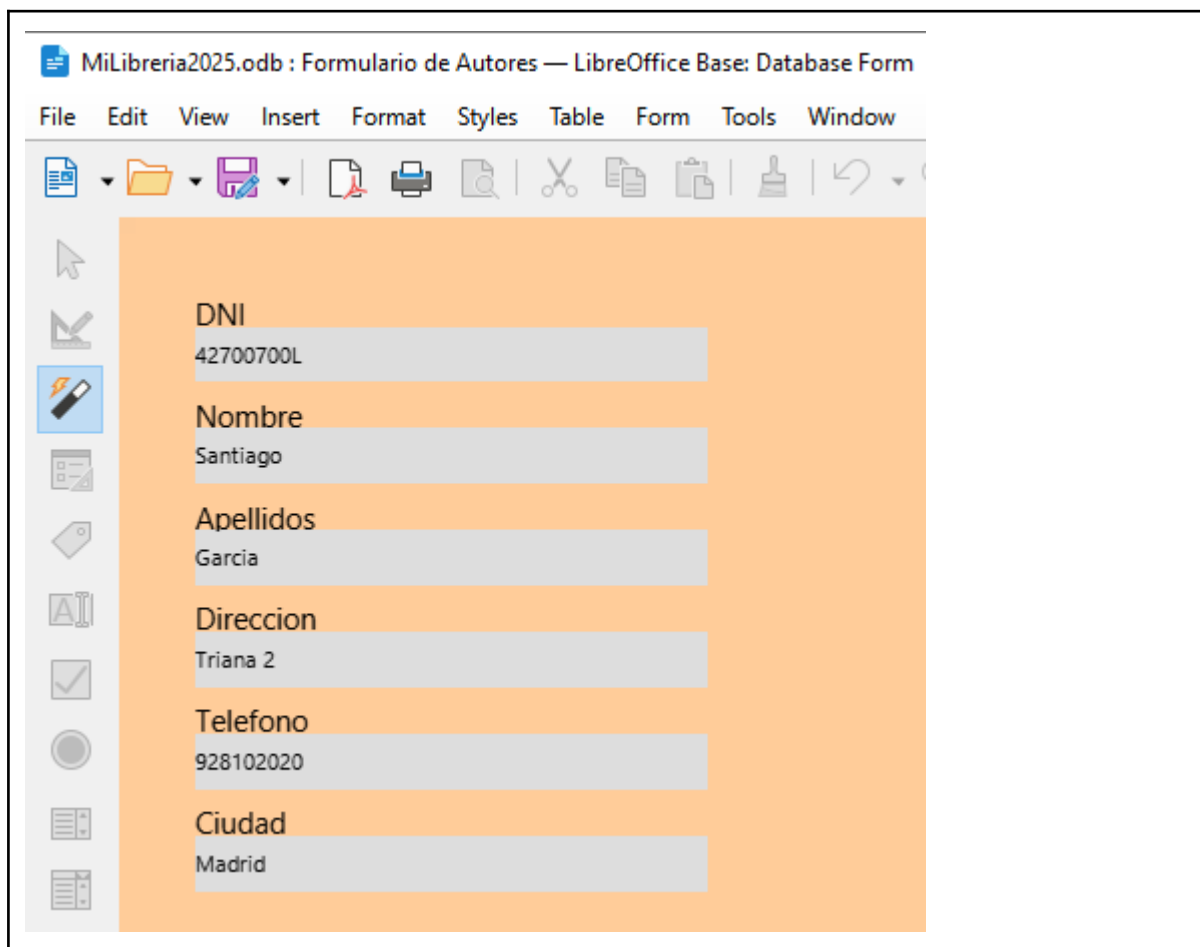
At the bottom of the window, there is a status bar showing "Record 1 of 7" and navigation icons.

Haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q.

Para crear el formulario de “Autores” deberemos abrir nuestra base de datos y hacer clic en Forms (formularios) en la parte izquierda de la ventana principal.

A continuación, haga clic en la opción resaltada “Use wizard to create form” (usar el asistente para crear un formulario) y repita todos los pasos anteriores para crear el formulario de la tabla “Autores” desde la página 203 hasta la página 210. Repita este proceso con todas las tablas de su base de datos.

Ejemplo de formulario para la tabla “Autores”:

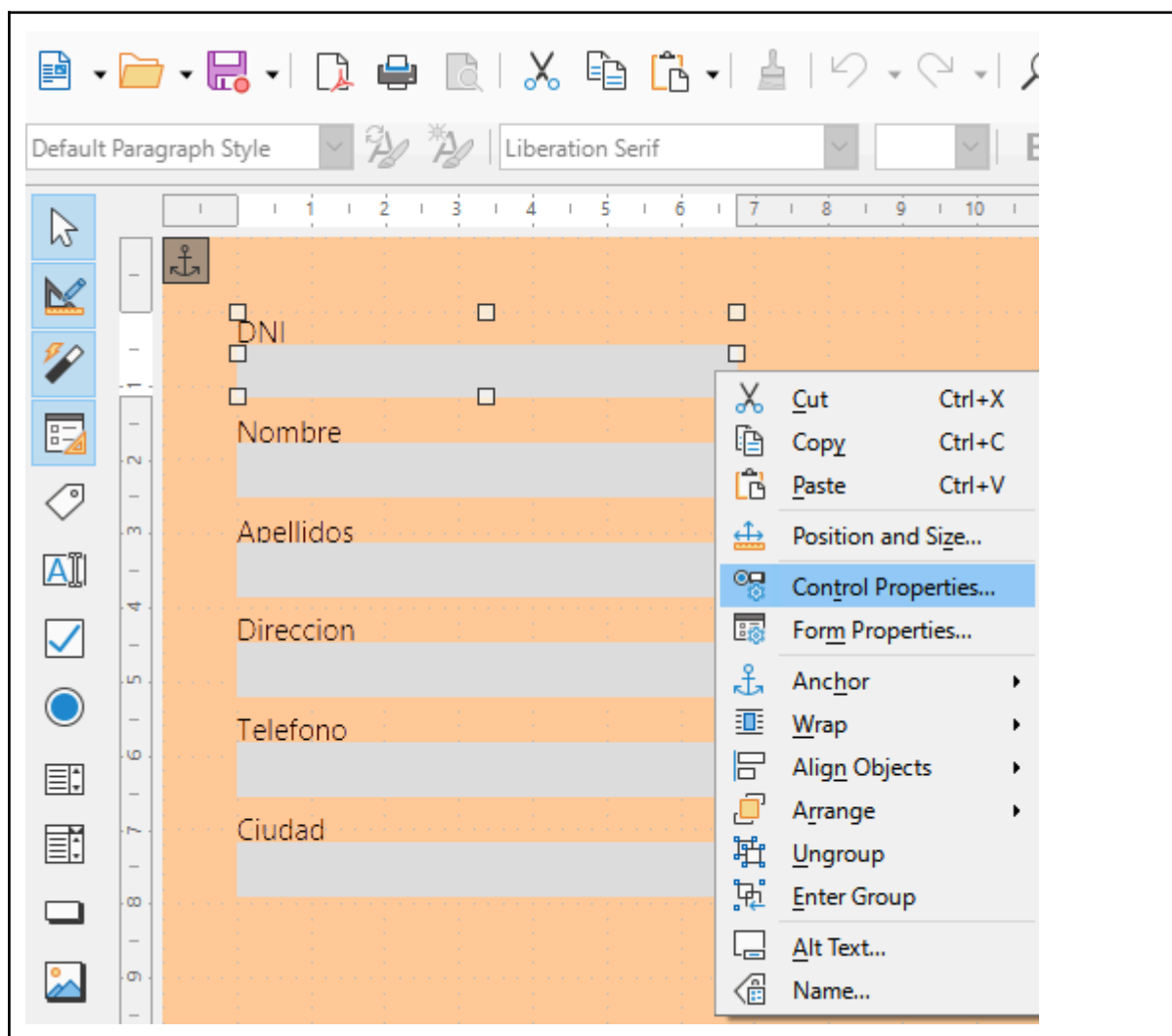


The screenshot shows a LibreOffice Base Database Form titled "MiLibreria2025.odb : Formulario de Autores — LibreOffice Base: Database Form". The form has a menu bar (File, Edit, View, Insert, Format, Styles, Table, Form, Tools, Window) and a toolbar with various icons. The form itself is a light orange rectangle containing several text input fields. On the left side of the form, there is a vertical toolbar with icons for different tools. The fields are labeled and filled with the following data:

Field Label	Value
DNI	42700700L
Nombre	Santiago
Apellidos	Garcia
Direccion	Triana 2
Telefono	928102020
Ciudad	Madrid

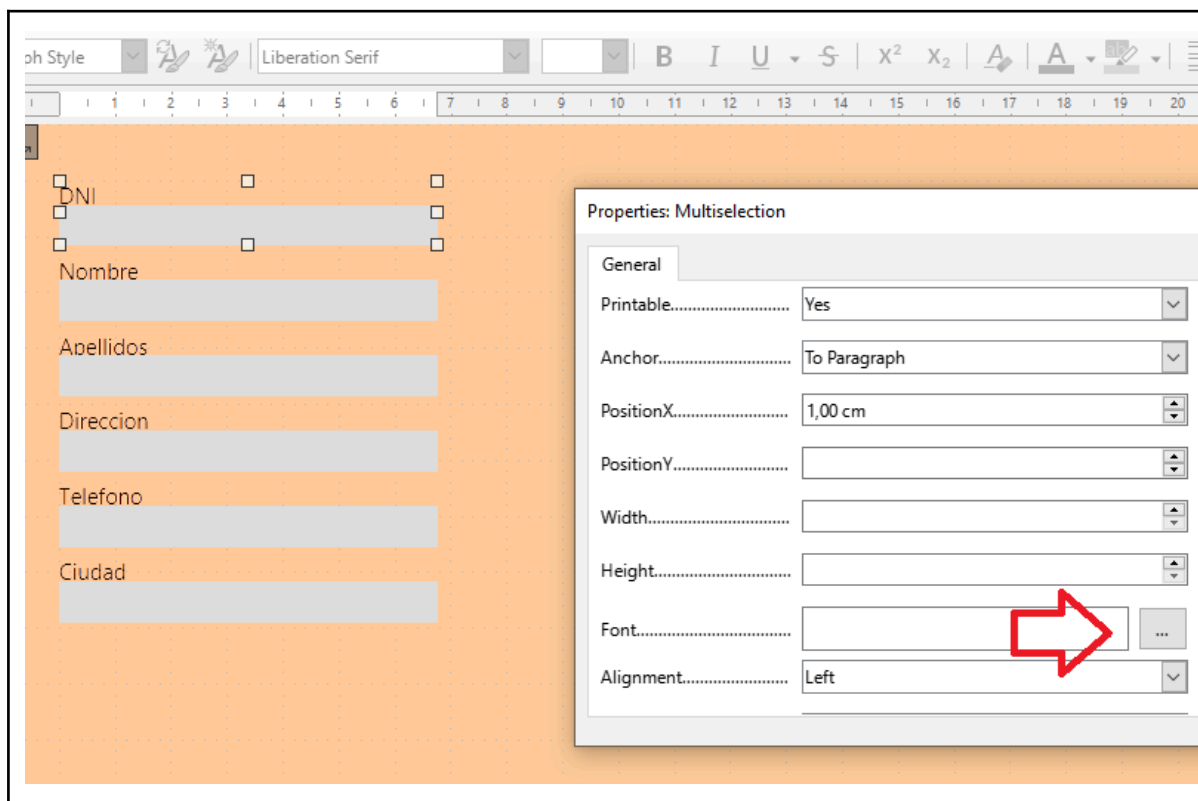
Como puede observar, se ha aumentado el tamaño de la fuente de los campos del formulario.

Para ello, en modo diseño, hemos hecho clic con la tecla CTRL pulsada y el botón derecho del ratón en el campo que queremos modificar y elegiremos “Control properties” (propiedades del control).



A continuación, hemos buscado la propiedad Fuente. Comprobamos que podemos cambiar el tamaño de la fuente haciendo clic en el símbolo “...”.

Para cambiar el tamaño de la fuente, podemos hacer clic en el símbolo “...” y subir el tamaño de 8 a 11.



Podemos repetir este paso con todos los campos para los que queremos reducir o aumentar el tamaño de la fuente.

Haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q.

Creación de informes (reports)

A la hora recopilar información de los autores, como sus nombres, direcciones o teléfonos utilizamos los formularios. Un informe permite organizar esa información en un formato en papel o pdf fácil de usar.

Podríamos decir, entonces, que el informe de clientes permite ver un listado de los clientes de nuestra base de datos y el informe de libros permite ver un listado de libros de nuestra librería.

Para crear un informe debemos abrir nuestra base de datos y hacer clic en Reports (informes) en la parte izquierda de la ventana principal.

Básicamente, existen dos tipos de informes:

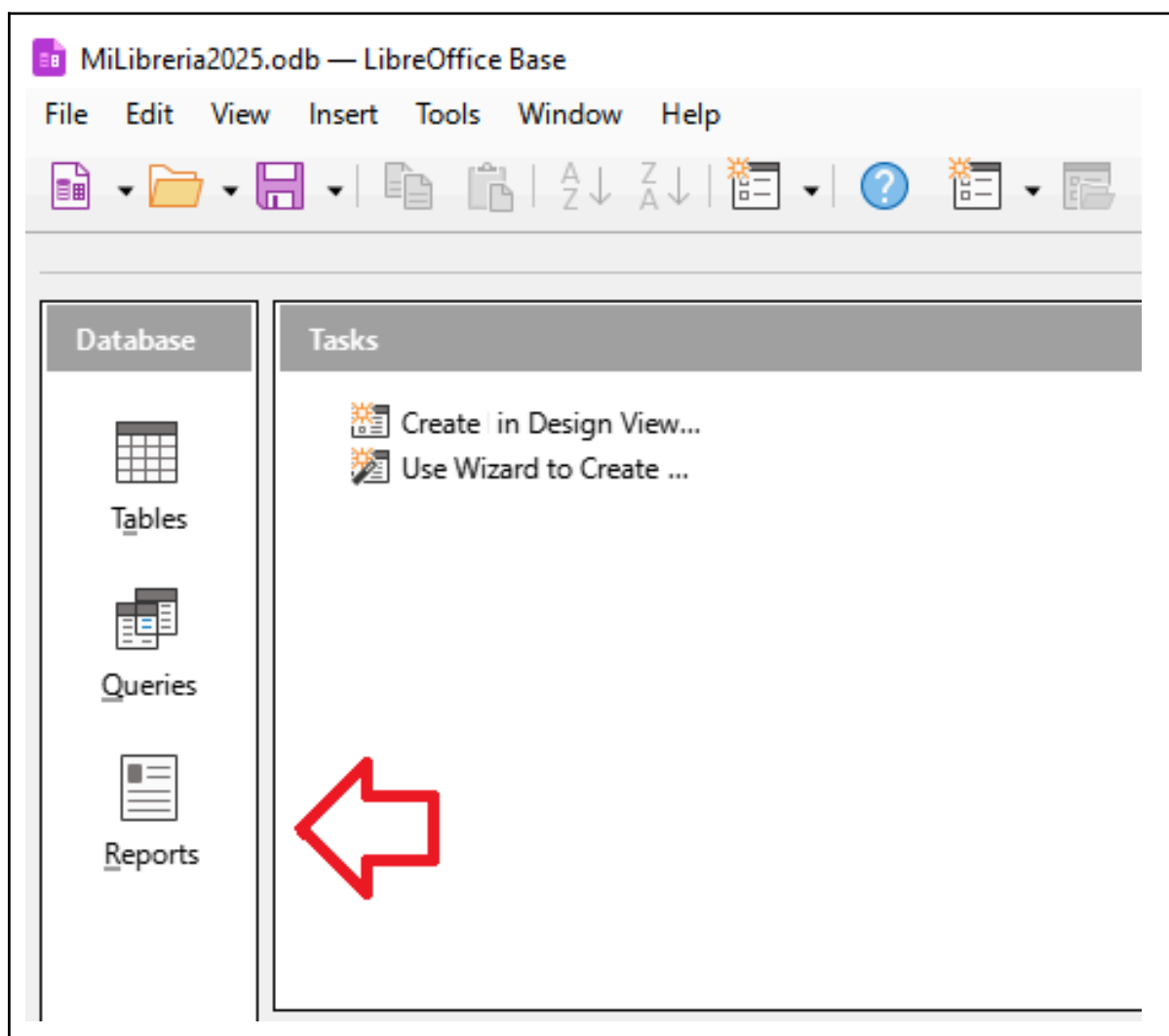
- Los informes simples, basados en una única tabla.
- Los informes complejos o informes de unión, basados en dos tablas o más.

Empezaremos por los informes simples:

Creación de informes simples (reports)

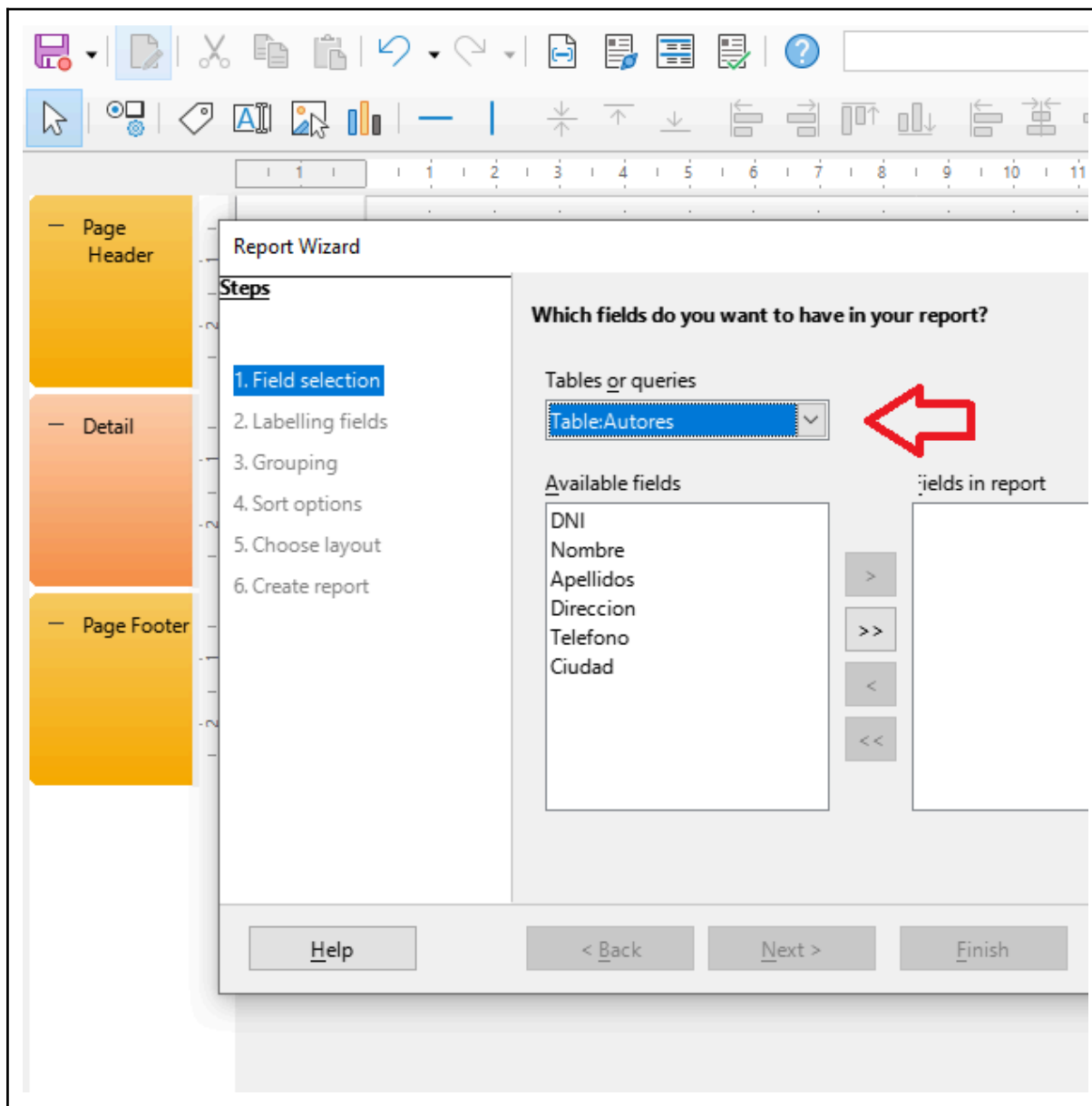
Para crear el informe simple de “Autores” deberemos abrir nuestra base de datos y hacer clic en Reports (informes) en la parte izquierda de la ventana principal.

A continuación, haga clic en la opción resaltada “Use wizard to create report” (usar el asistente para crear un informe).

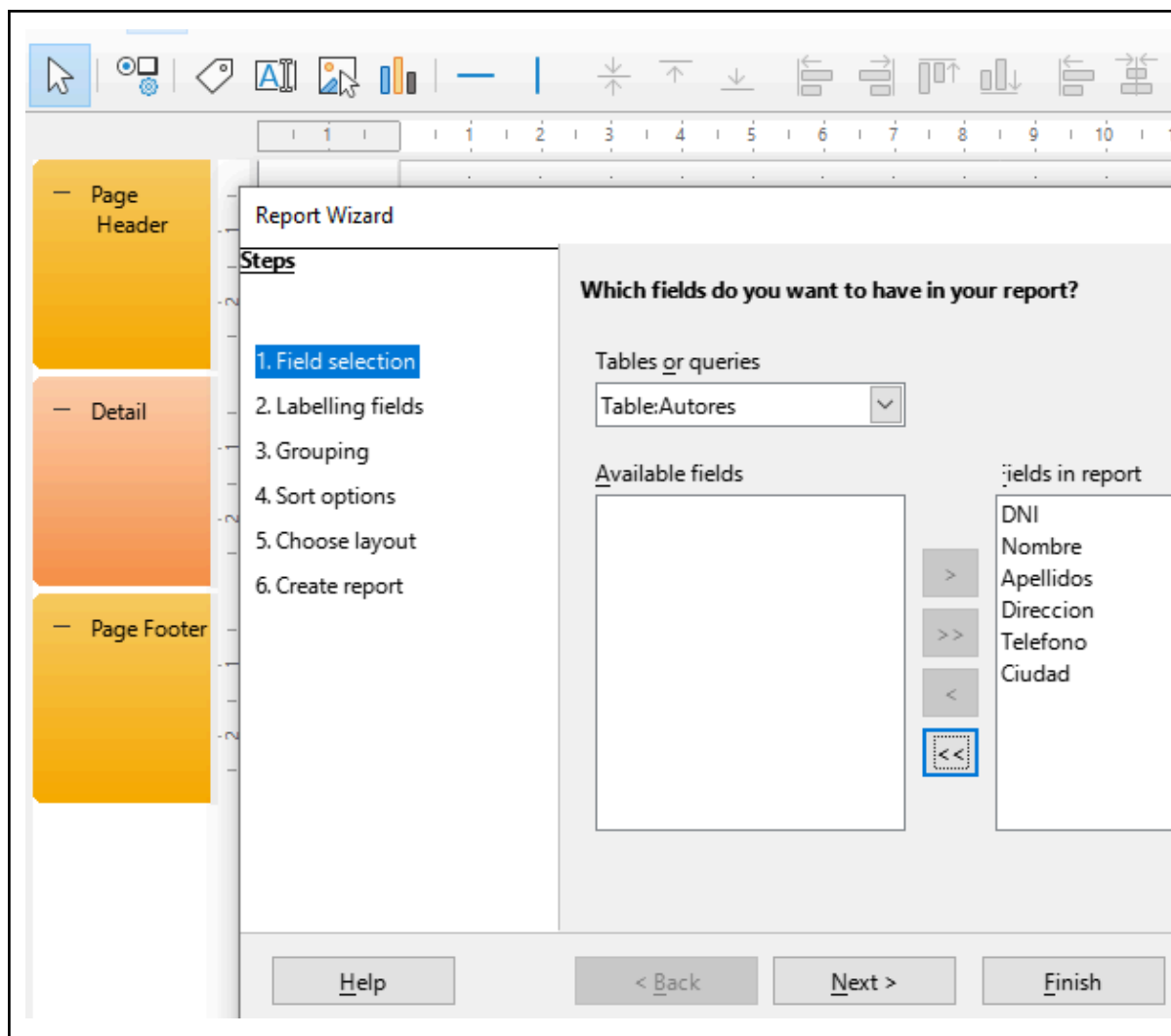


Seguidamente, verá esta pantalla:

Seleccione la tabla “Autores”:



Haga clic en el símbolo >> para seleccionar todos los campos de la tabla “Autores” y que así puedan aparecer en el informe. Todos los campos disponibles habrán pasado al cuadro de la derecha como puede verse en esta imagen:



A continuación, deberá elegir los encabezados del informe:

¿Cómo quiere etiquetar (label) los campos (fields) del informe?

Recuerde que estas etiquetas son los encabezados de cada columna del informe.

Report Wizard

Steps

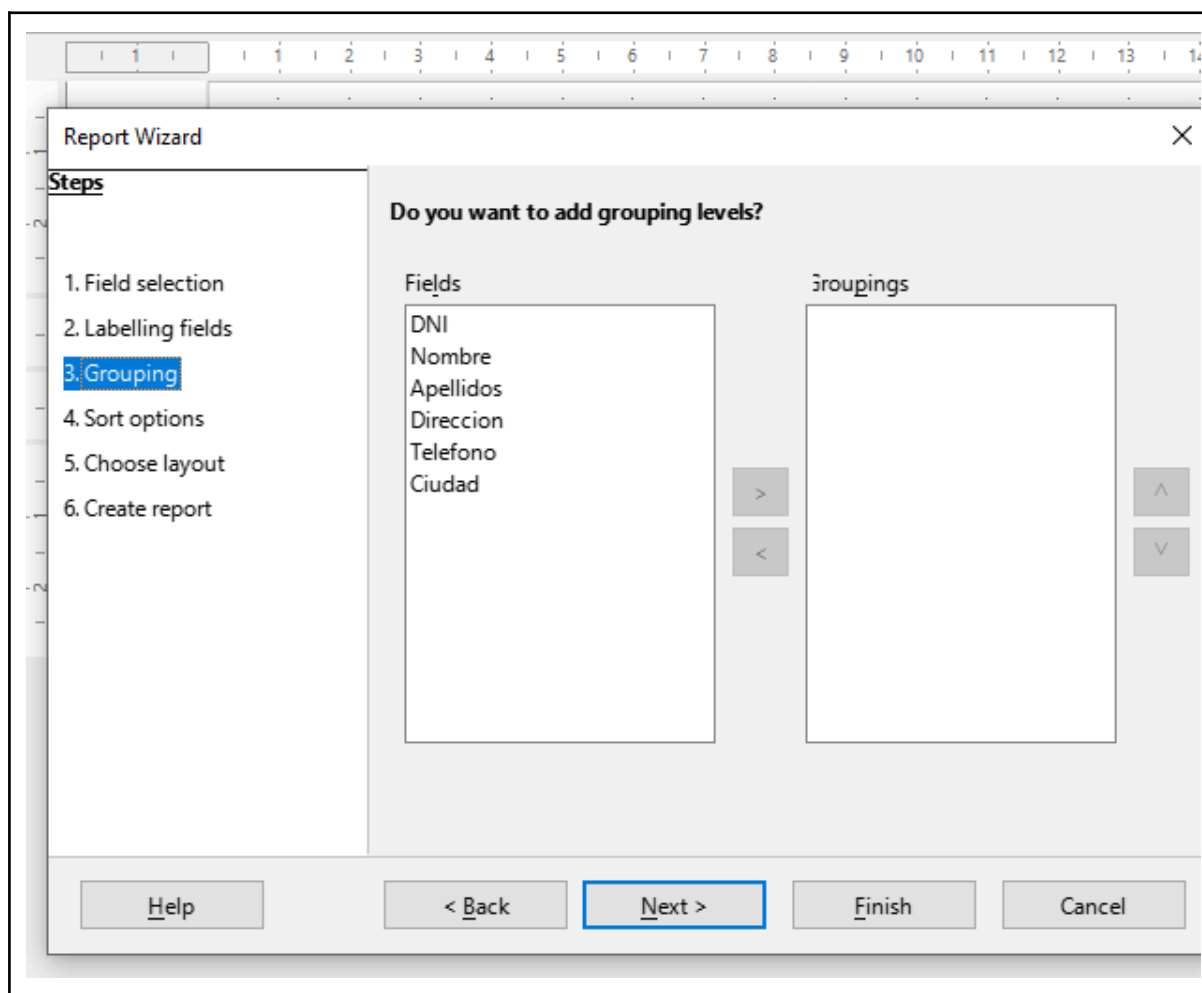
1. Field selection
2. Labelling fields
3. Grouping
4. Sort options
5. Choose layout
6. Create report

How do you want to label the fields?

Field	Label
DNI	DNI
Nombre	Nombre
Apellidos	Apellidos
Direccion	Direccion
Telefono	Telefono
Ciudad	Ciudad

Haga clic en “Next” para continuar.

¿Desea añadir niveles de agrupación? Podríamos elegir “Ciudad” y así aparecerán en nuestro informe los autores por ciudades. Esto quiere decir que primero aparecerán los autores de Dublín, después aparecerán los de Lisboa, a continuación, aparecerán los autores de Madrid, etc..



En nuestro caso, haremos clic en “Next” puesto que utilizaremos SQL para agrupar los registros de nuestros informes más adelante.

¿Qué campos desea utilizar para ordenar el informe de autores? Elegiremos Apellidos y Nombre:

The screenshot shows the 'Report Wizard' dialog box, specifically the 'Sort options' step. The 'Steps' list on the left includes: 1. Field selection, 2. Labelling fields, 3. Grouping, 4. Sort options (highlighted), 5. Choose layout, and 6. Create report. The main area is titled 'Which fields do you want to sort the data by?'. It contains four sorting rows. The first row is for 'Sort by' with 'Apellidos' selected and 'Ascending' chosen. The second row is for 'Then by' with 'Nombre' selected and 'Ascending' chosen. The third and fourth rows are for 'Then by' with '- undefined -' selected and 'Ascending' chosen. At the bottom, there are buttons for 'Help', '< Back', 'Next >' (highlighted), 'Finish', and 'Cancel'.

Report Wizard

Steps

1. Field selection
2. Labelling fields
3. Grouping
4. Sort options
5. Choose layout
6. Create report

Which fields do you want to sort the data by?

Sort by: Apellidos ☒ Ascending ☐ Descending

Then by: Nombre ☒ Ascending ☐ Descending

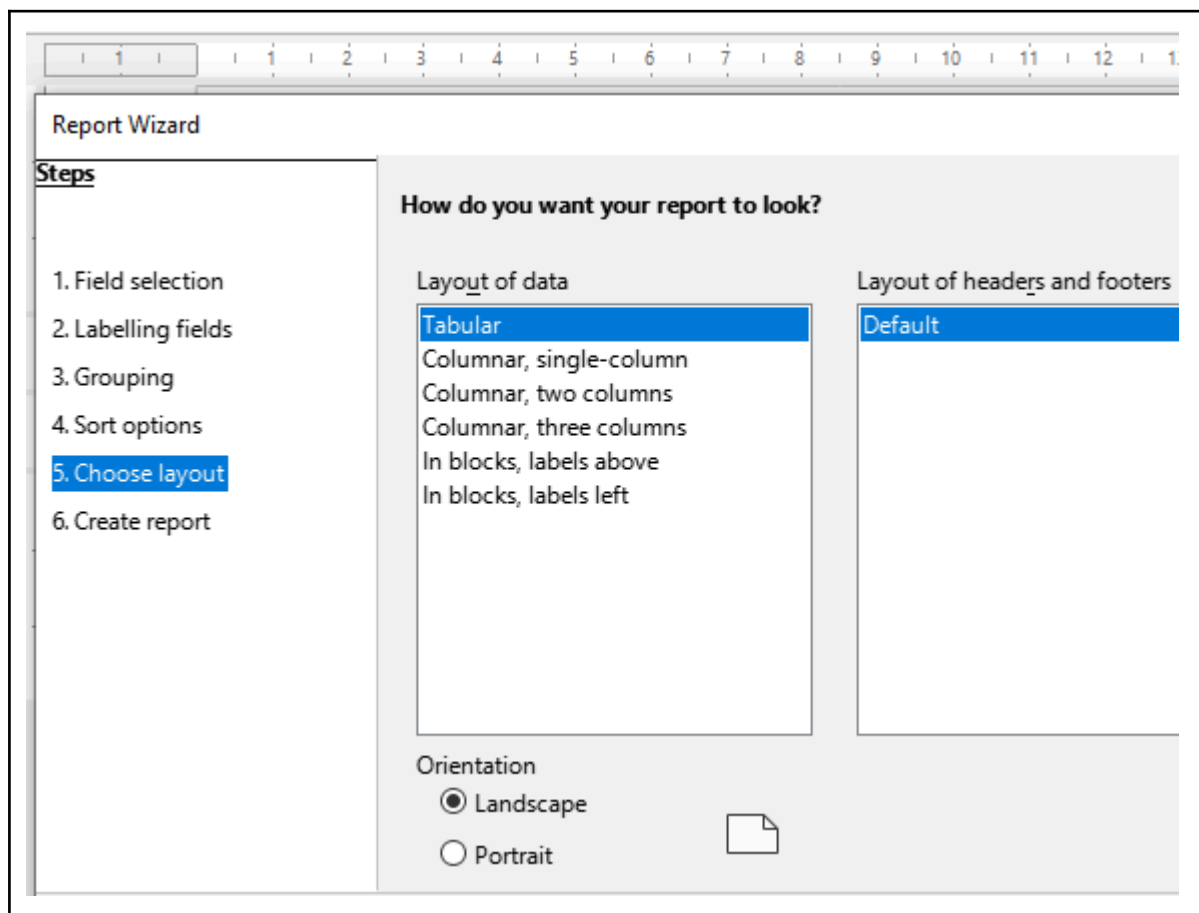
Then by: - undefined - ☒ Ascending ☐ Descending

Then by: - undefined - ☒ Ascending ☐ Descending

Buttons: Help, < Back, Next >, Finish, Cancel

A continuación, haremos clic en “Next”.

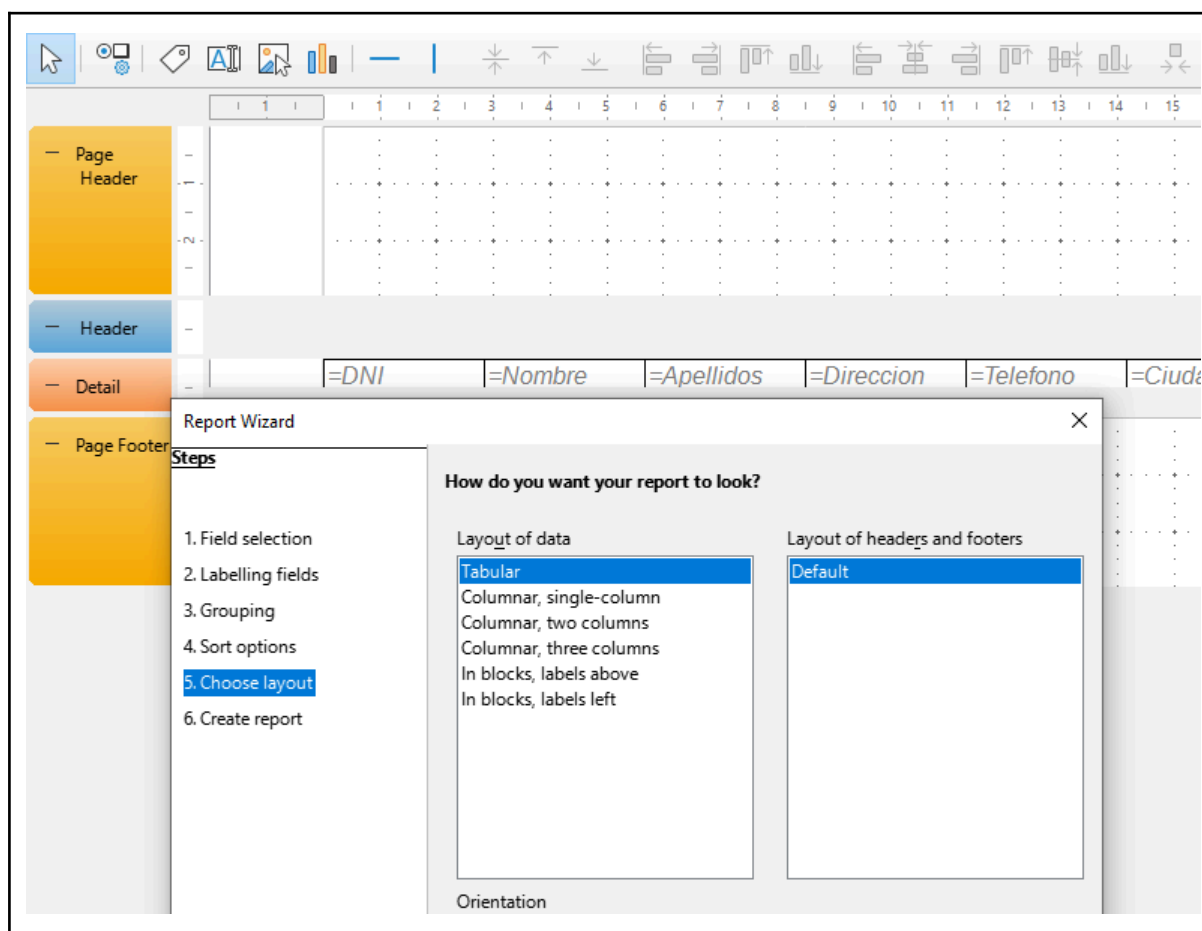
¿Cómo quiere el informe? Elegiremos “Tabular” que es el informe por columnas más común en las empresas.



A continuación, puede elegir la orientación del informe. “Landscape” se refiere a los informes con orientación apaisada y “Portrait” a la orientación vertical o tipo retrato.

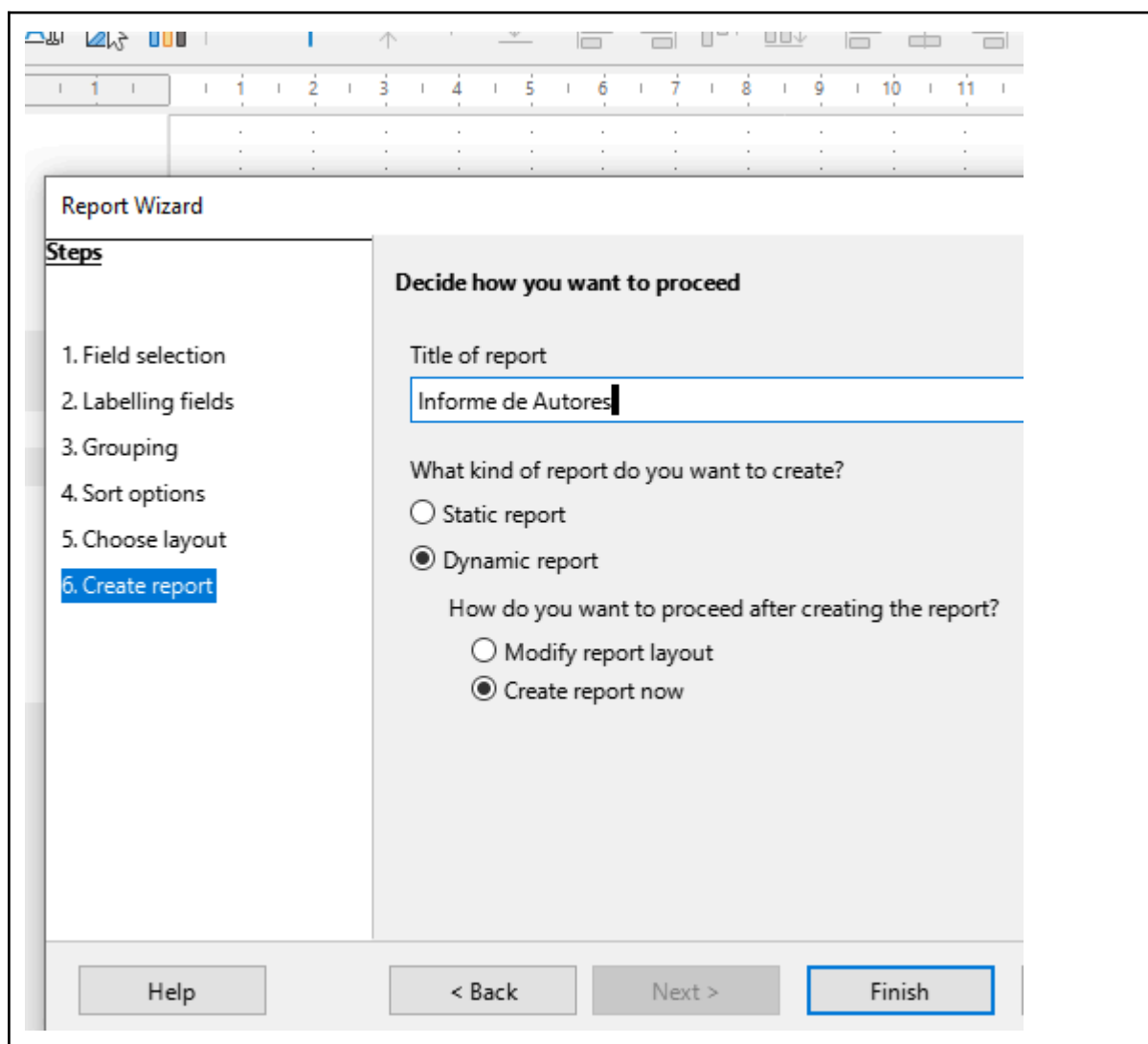
Elegiremos “Portrait” y haremos clic en “Next”.

Podemos comprobar que LibreOffice ha colocado los campos para nuestro informe en la zona “Detail” (detalle):



A continuación, haremos clic en “Next”.

En este paso, introduciremos el título del informe, “Informe de Autores”.



A continuación, haremos clic en “Finish” para ver cómo queda finalmente el diseño de nuestro informe:

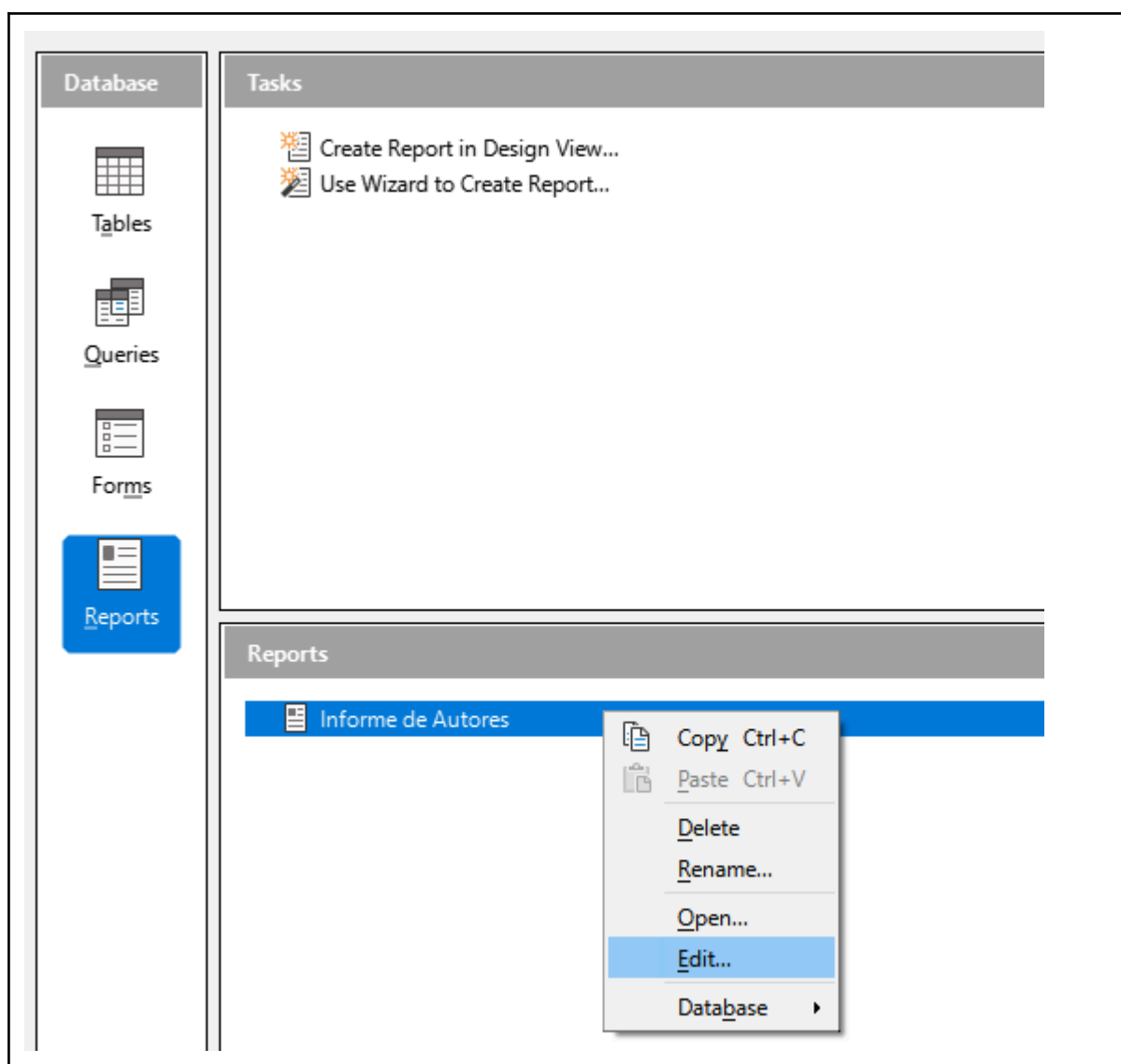
Este es el resultado final del Informe de Autores:

DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
50123000L	Gary	Bennet	Gomera 1	928999000	Sevilla
DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
42888000L	Juan	Espinosa	Triana 5	928900900	Telde
DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
42800800K	Javier	Garcia	Gomera 2	928950095	Lisboa
DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
42700700L	Santiago	Garcia	Triana 2	928102020	Madrid
DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
42777555L	Douglas	Hergert	Triana 4	928000999	Dublin
DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
42700800L	Joe	Merlino	Triana 3	928101022	Paris
DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
4286200A	John	Smith	Triana 1	928102030	Telde

Find All ☐ Match Case 

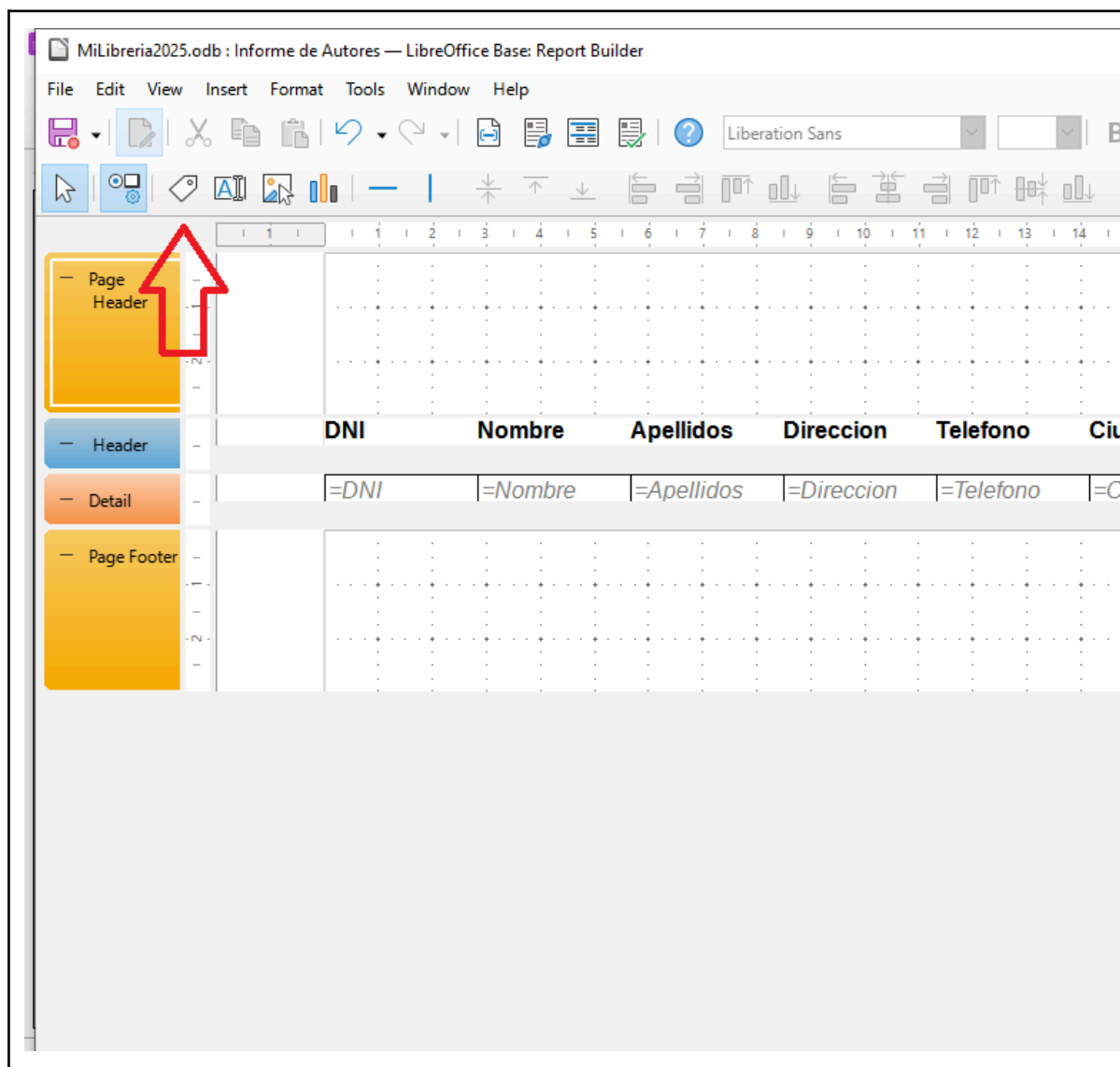
Como puede ver, el informe no tiene título.

Para ponerle título a nuestro informe (título que aparecerá en la primera hoja del archivo pdf resultante), iremos a la ventana principal de LibreOffice y haremos clic con el botón derecho del ratón en “Edit” (sobre el informe que queremos editar).

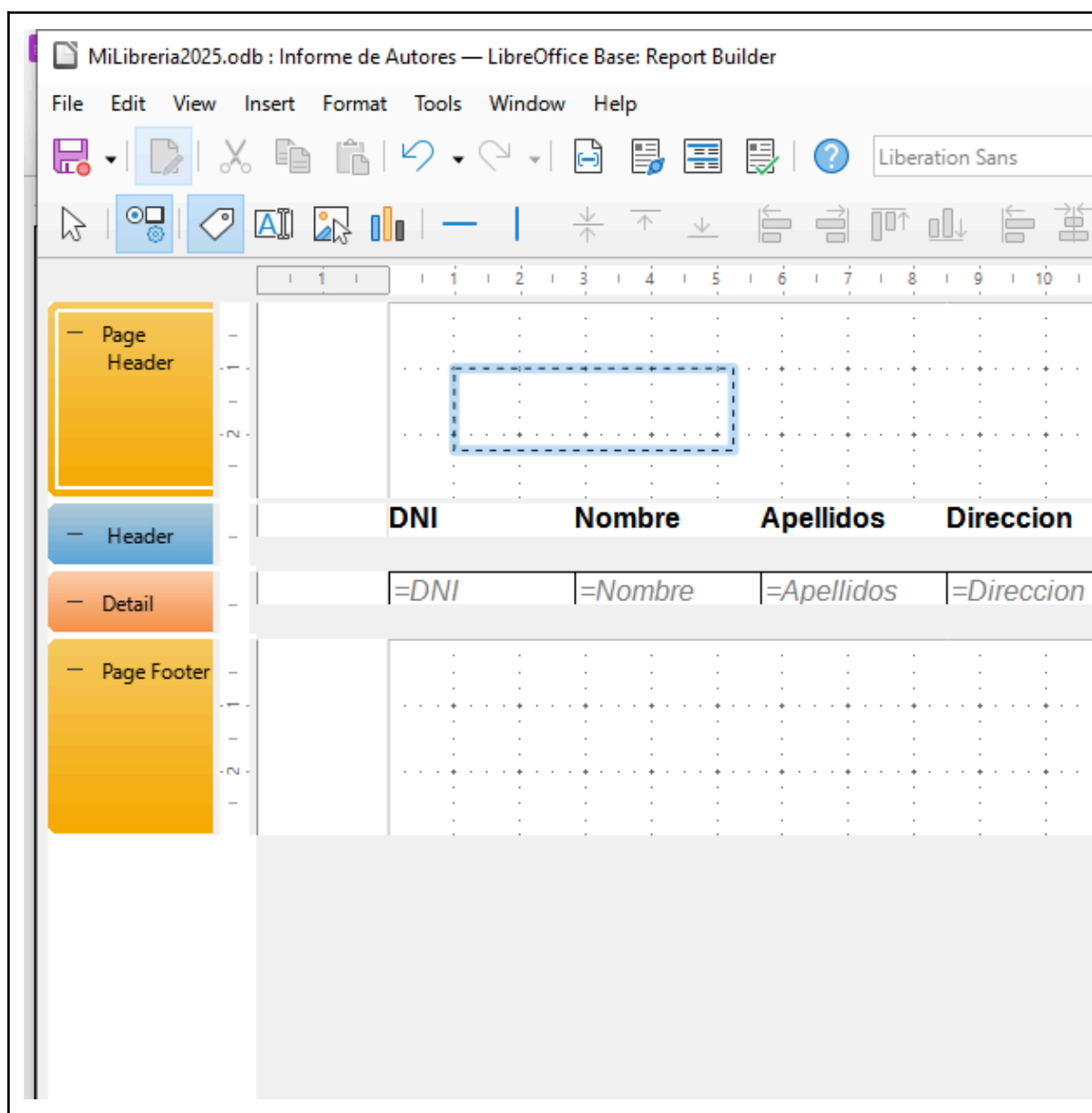


A continuación veremos el informe en modo diseño:

A continuación veremos el informe en modo diseño y haremos clic en “Label” (etiqueta) para ponerle un título a nuestro informe:

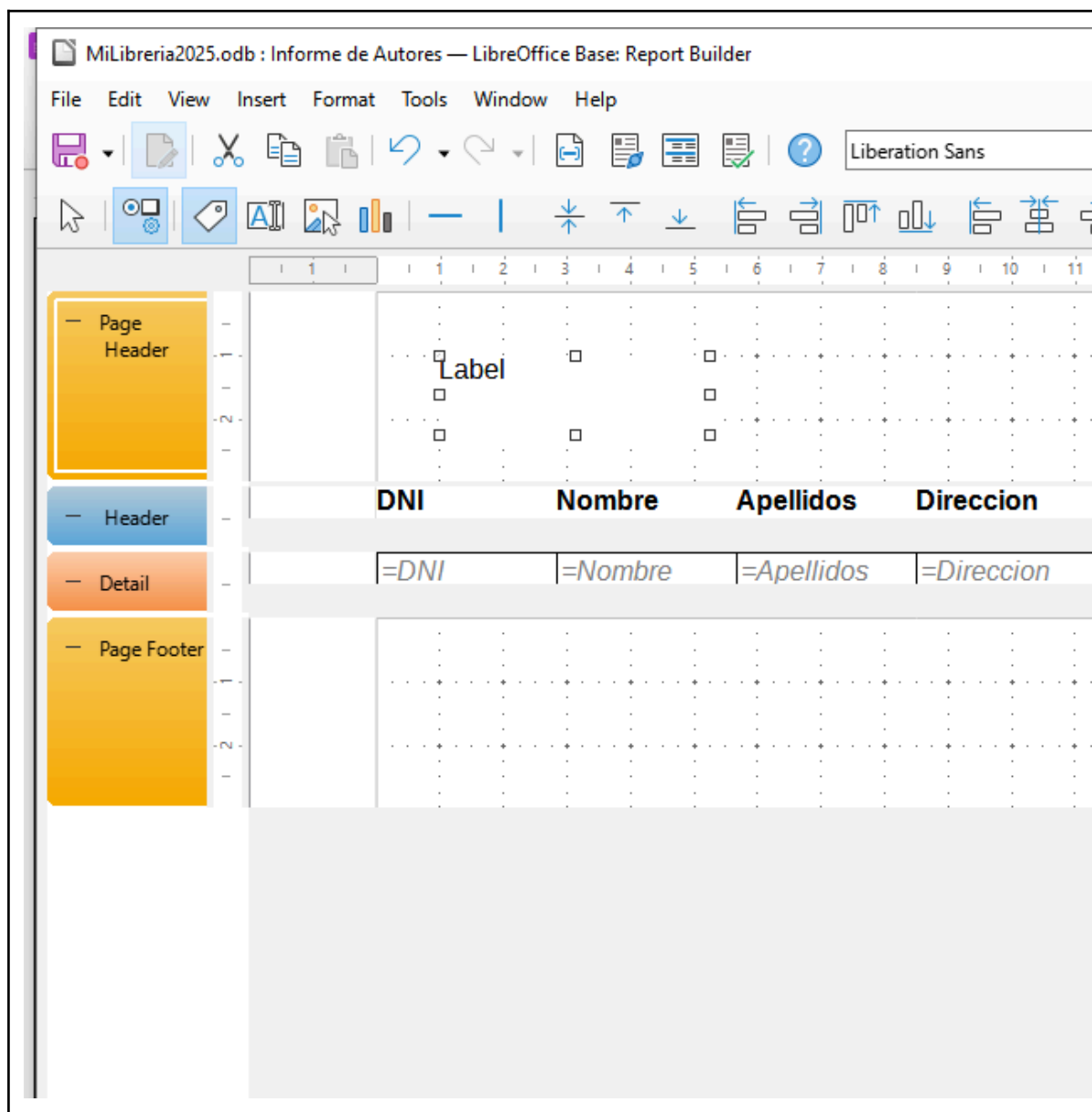


Al hacer clic sobre “Label” (etiqueta), deberemos dibujar un rectángulo con el ratón en la zona donde queremos situar el título de nuestro informe:



Podemos ver que mientras hacemos el rectángulo con el ratón, la zona donde irá el título se ilumina con un borde azul.

Al soltar el ratón, LibreOffice habrá creado el objeto “Label” (etiqueta):



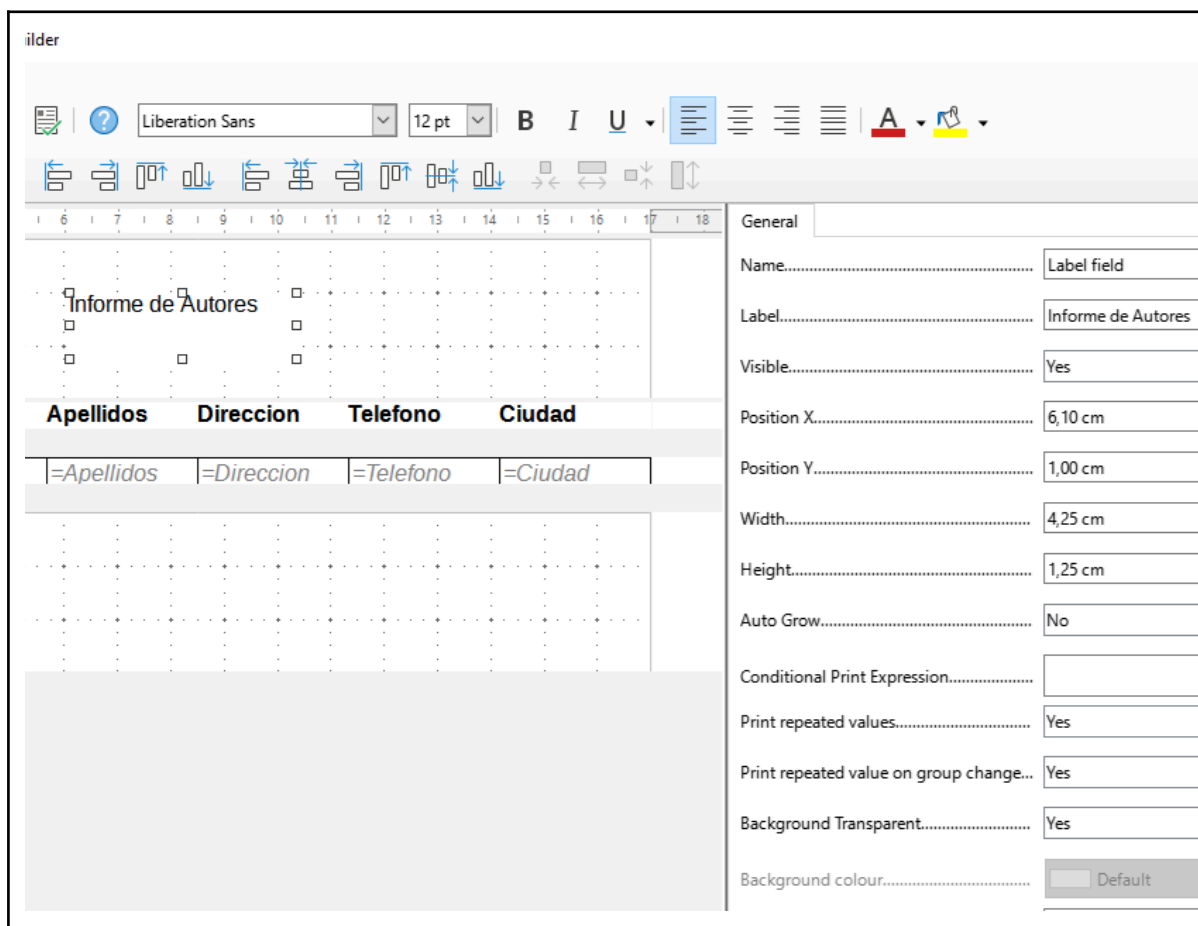
Para cambiar la palabra “Label” por el título que deseamos, haremos clic en “Label” con el botón derecho del ratón.

Y veremos la siguiente ventana en la parte derecha de la pantalla:

The screenshot shows a software window with a report design interface. On the left is a preview of a report with a table containing the word "Ciudad". On the right is a "General" properties panel for a "Label field". A red arrow points to the "Label" property, which is set to "Label".

General	
Name.....	Label field
Label.....	Label
Visible.....	Yes
Position X.....	1,00 cm
Position Y.....	1,00 cm
Width.....	4,25 cm
Height.....	1,25 cm
Auto Grow.....	No
Conditional Print Expression.....	
Print repeated values.....	Yes
Print repeated value on group change...	Yes
Background Transparent.....	Yes
Background colour.....	Default

Cambiaremos la palabra “Label” por “Informe de Autores” y pulsaremos la tecla “Tab” (tabulador) para que los cambios se vean en nuestro diseño final.



Haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q (recuerde grabar los cambios).

Nota:

Si no puede ver la ventana de la derecha, haga clic en “View” el menú principal, y después haga clic en “Properties” (propiedades).

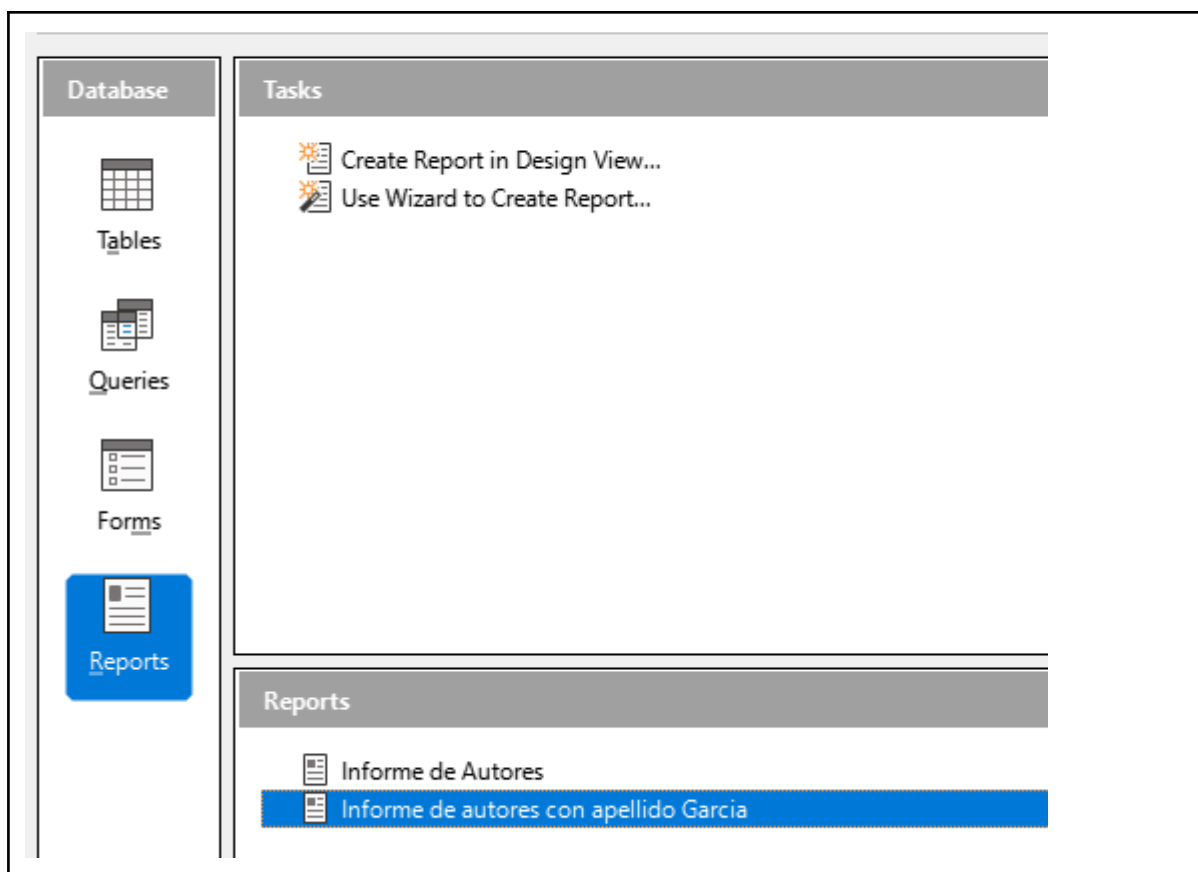
Creación de informes complejos con filtros

Para crear un informe con un filtro deberemos abrir nuestra base de datos y hacer clic en Reports (informes) en la parte izquierda de la ventana principal.

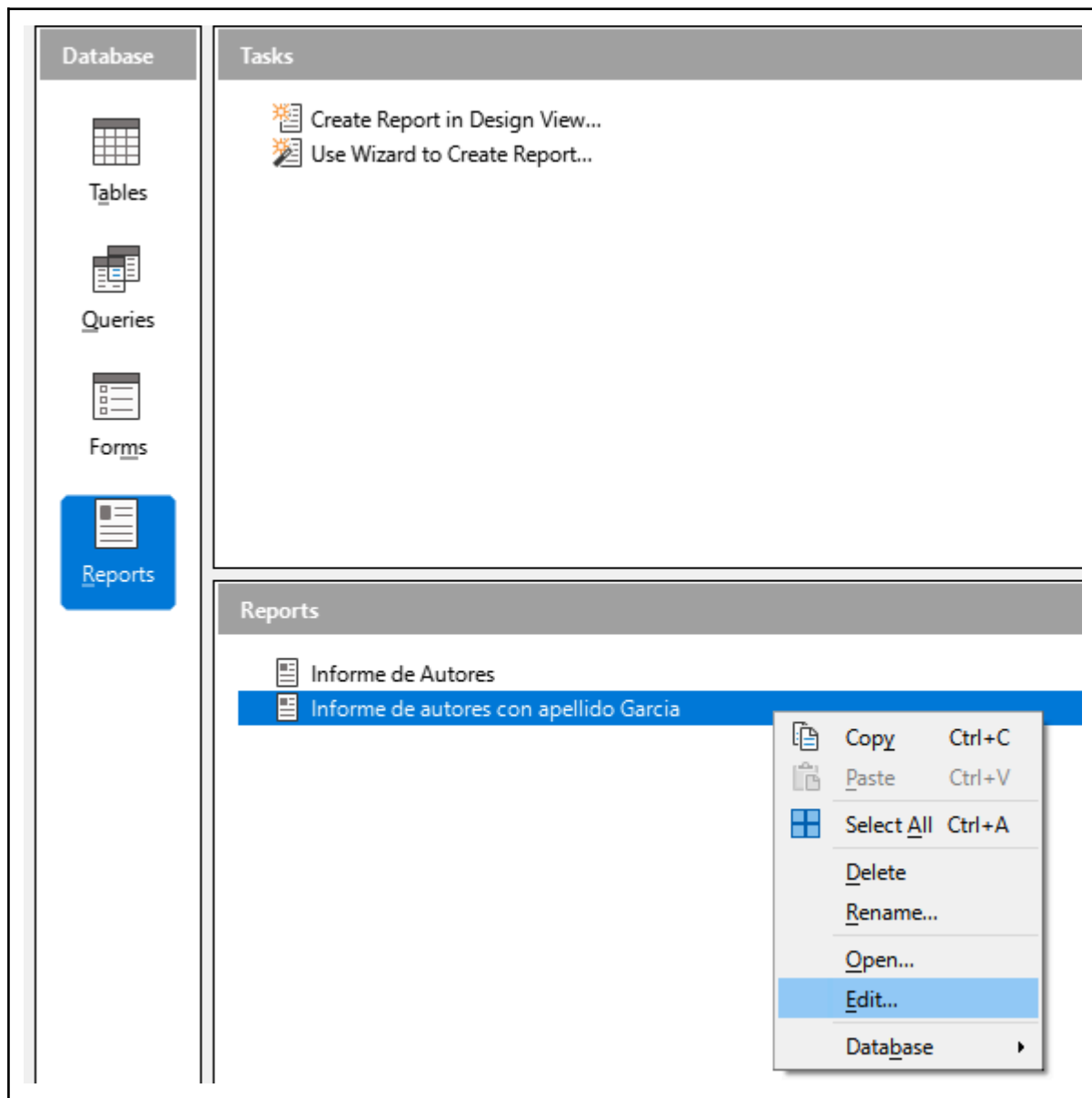
A continuación crearemos un informe para listar los autores con el apellido “García”.

Para ello, haremos clic en el “Informe de Autores” y pulsaremos CTRL+C y a continuación CTRL+V.

En la ventana emergente, cambiaremos el nombre del informe por “Informe de autores con el apellido García” y aparecerá un nuevo informe con la misma estructura y el mismo diseño que hemos seleccionado:

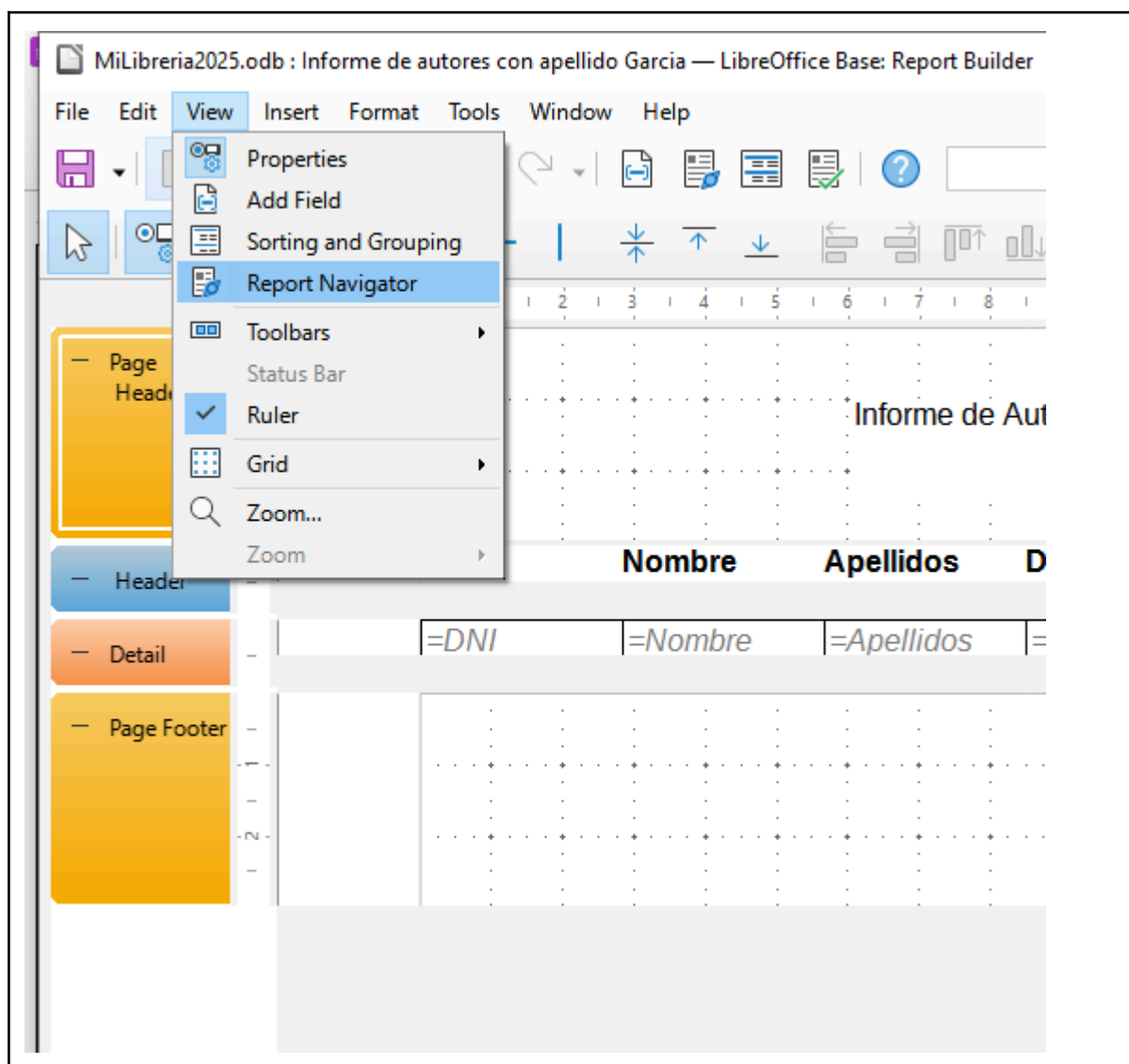


Ahora que tenemos el informe duplicado, debemos aplicar un filtro para que únicamente muestre los autores con apellido “García”.



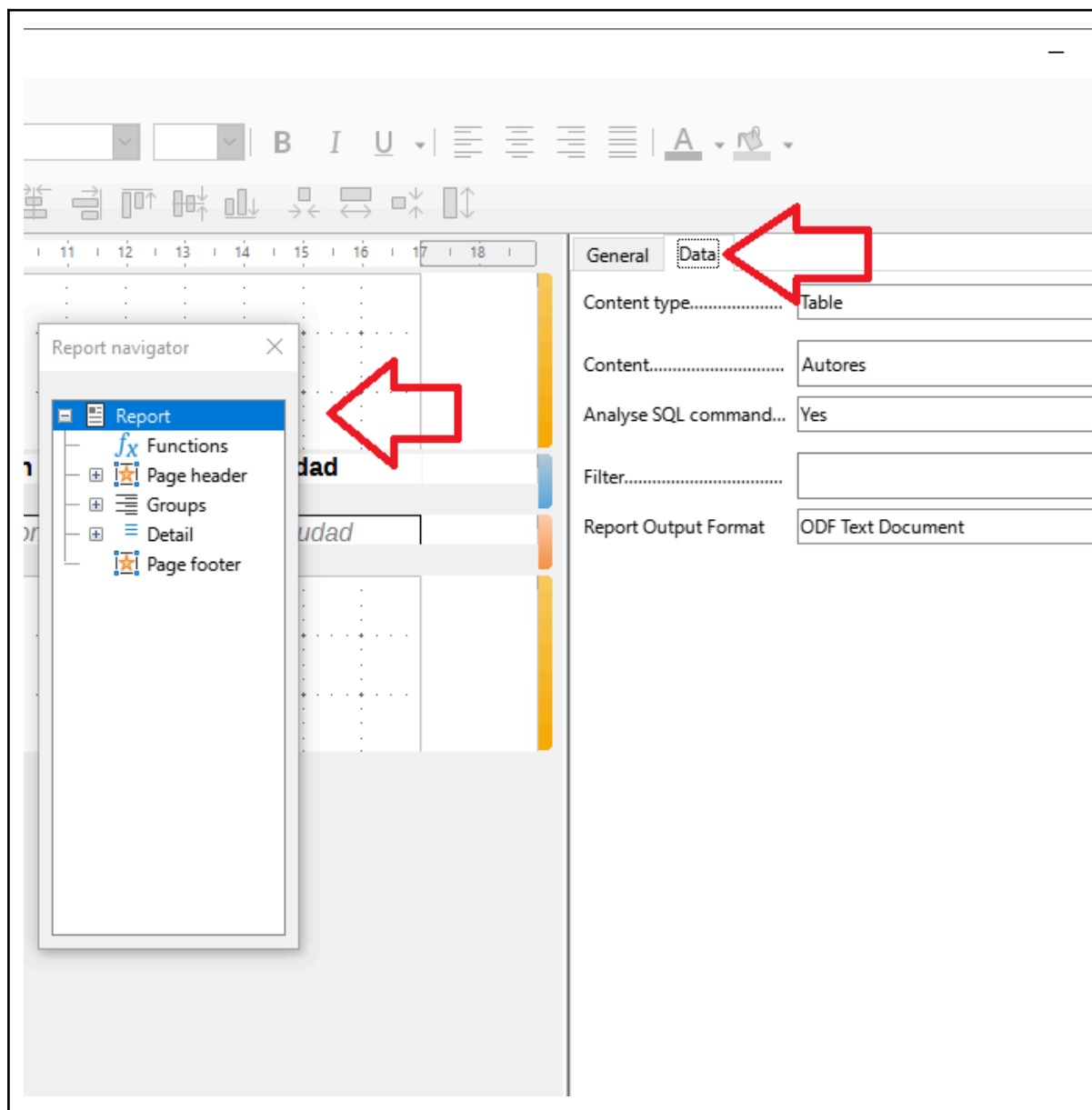
Para editar las propiedades del informe duplicado, haremos clic con el botón derecho en el “Informe de autores con apellido García” y elegiremos “Edit”.

A continuación, haremos clic en “View” y después en “Report Navigator”:



Y veremos la siguiente pantalla:

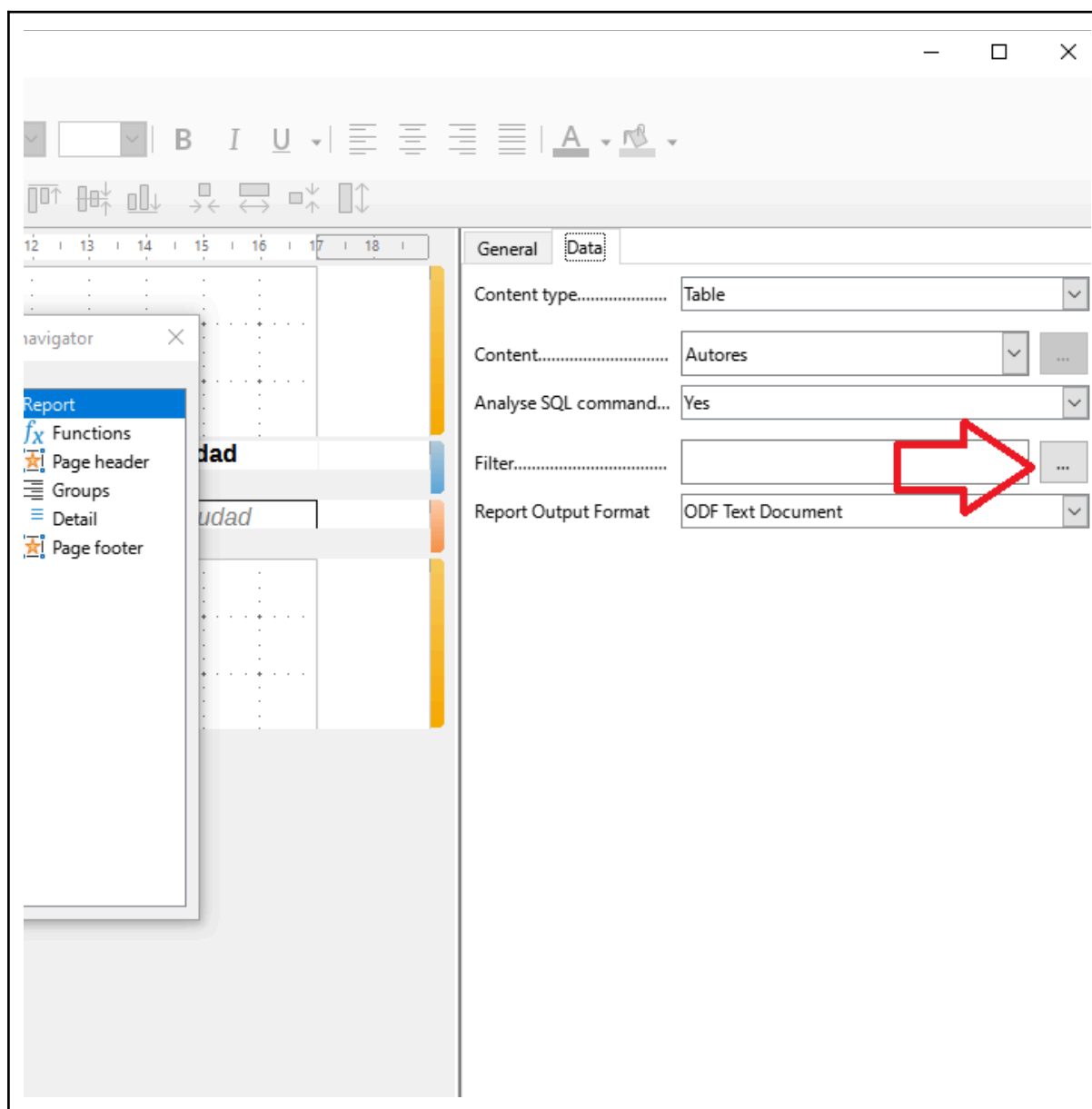
A continuación, haremos clic primero en “Report” (informe) y después en “Data” (datos):



A continuación veremos la siguiente pantalla:

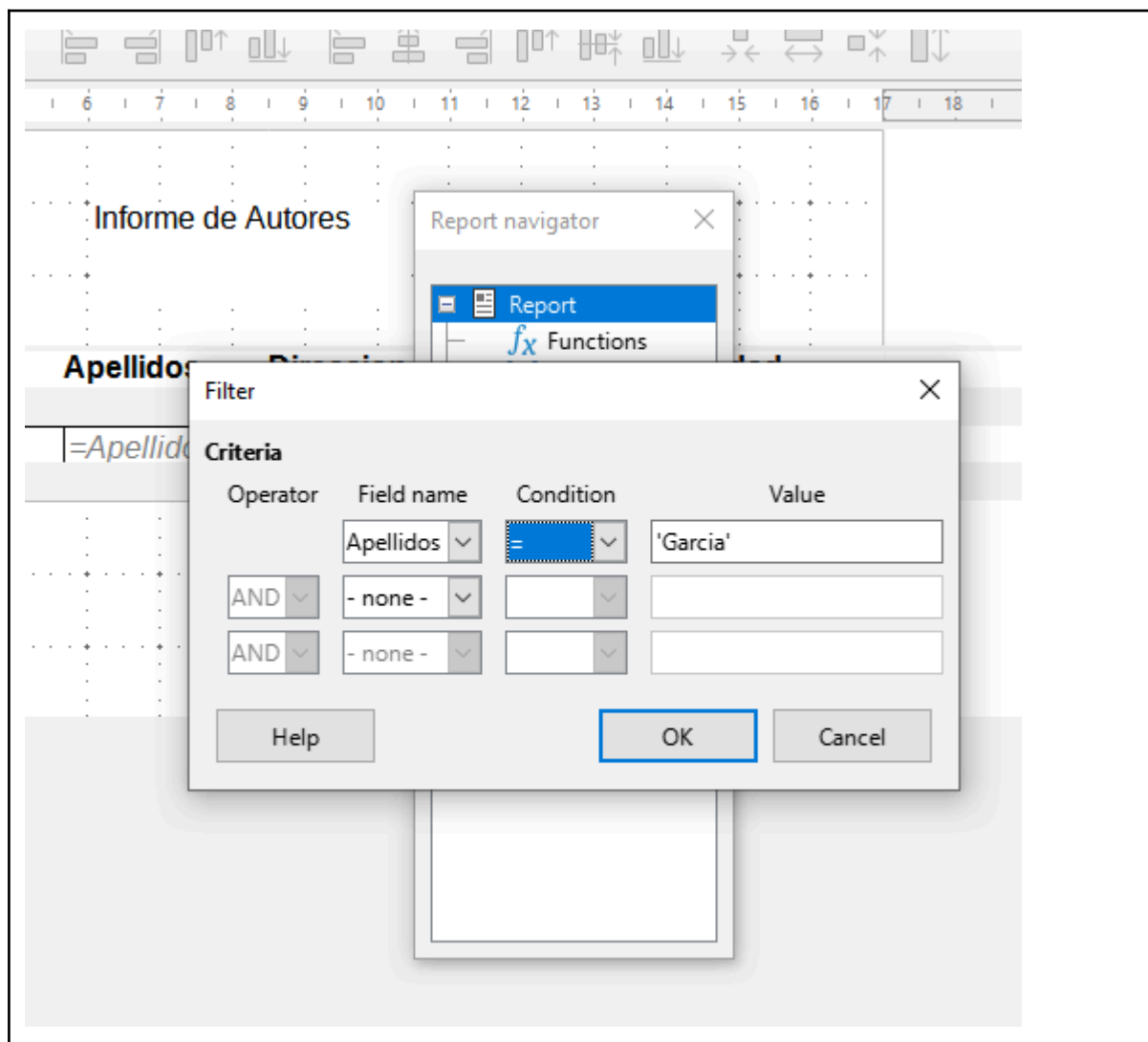
Nota: Si no puede ver la pestaña “Data” (Datos), haga clic en el menú principal en “Ver”, y a continuación, haga clic en “Propiedades”.

Para crear el filtro a nuestro informe, haremos clic en los tres puntos (...) de la propiedad “Filter” (filtro):



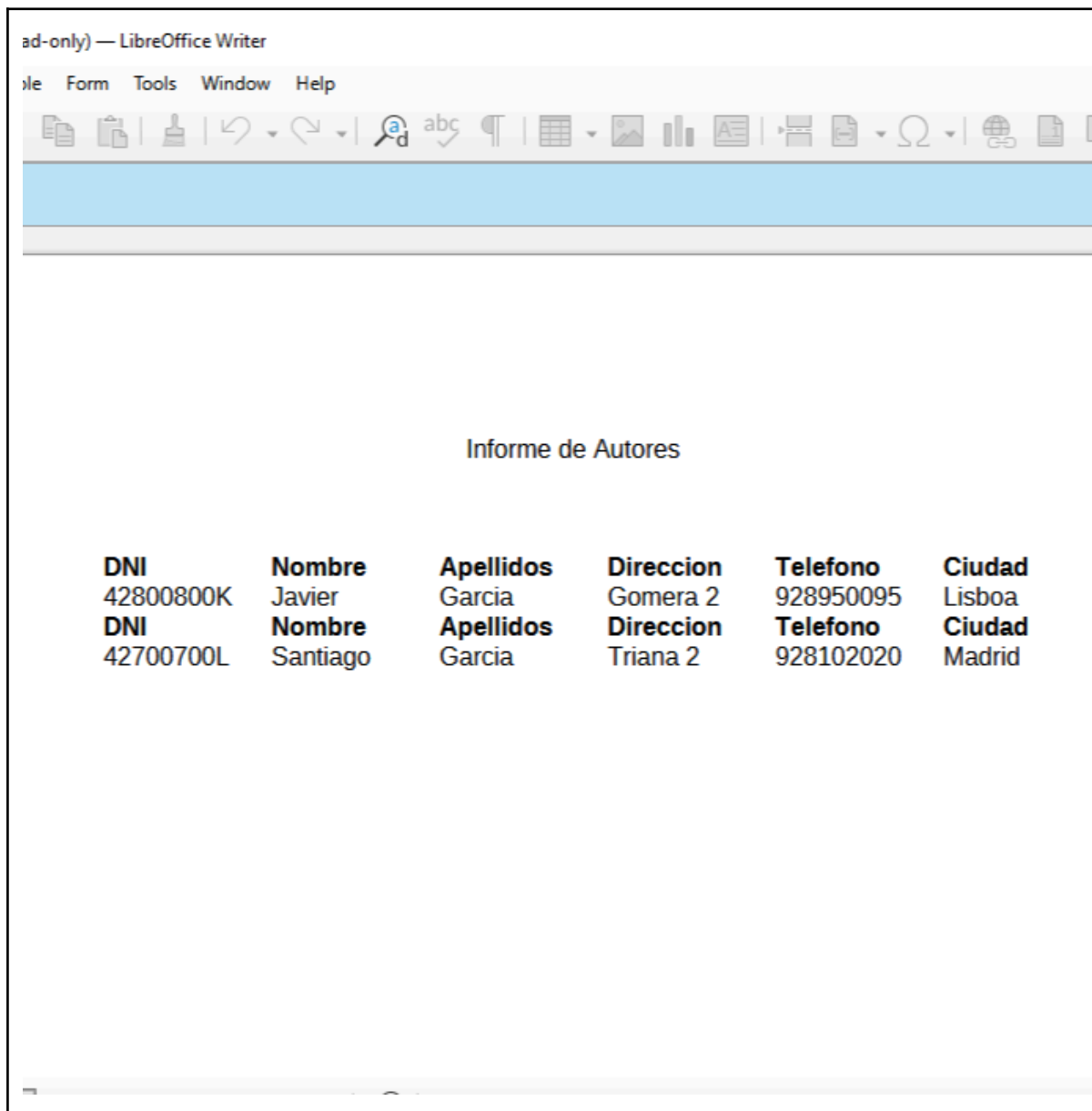
A continuación veremos la siguiente pantalla:

En el menú desplegable “Field name”, elegiremos “Apellidos” y teclearemos “García” en el campo “Value”.



Haremos clic en “Ok” para aplicar el filtro.

Este será el resultado de nuestro nuevo informe de “Autores con apellido García”:



The screenshot shows the LibreOffice Writer interface. The title bar reads 'ad-only) — LibreOffice Writer'. The menu bar includes 'File', 'Form', 'Tools', 'Window', and 'Help'. The toolbar contains various icons for document operations. The main content area displays a report titled 'Informe de Autores' which contains a table with author data.

DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
42800800K	Javier	Garcia	Gomera 2	928950095	Lisboa
42700700L	Santiago	Garcia	Triana 2	928102020	Madrid

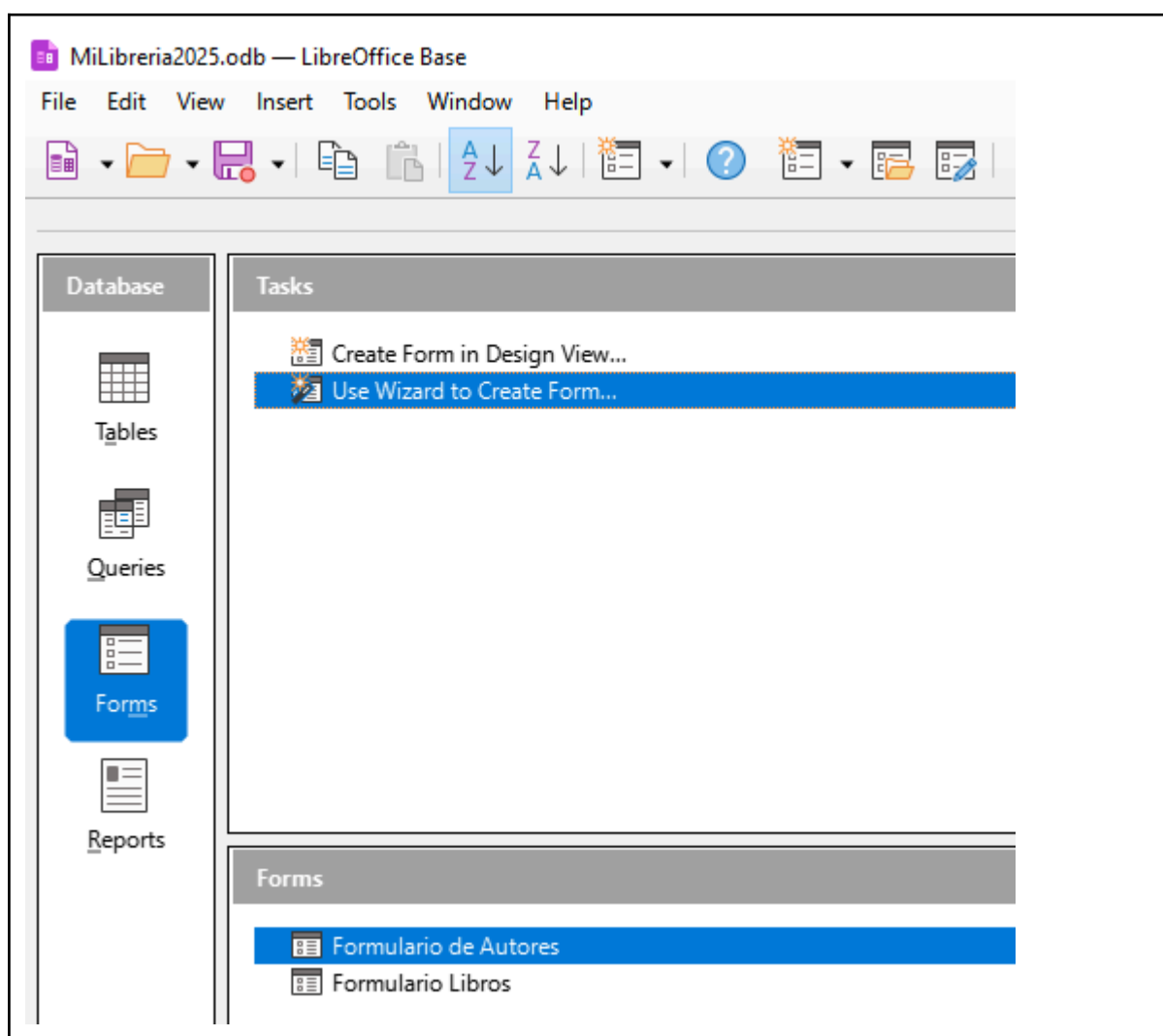
De la misma manera podrá duplicar cualquier informe y crear filtros para las buscar cualquier dato en cualquier tabla.

A continuación, haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q (recuerde grabar los cambios).

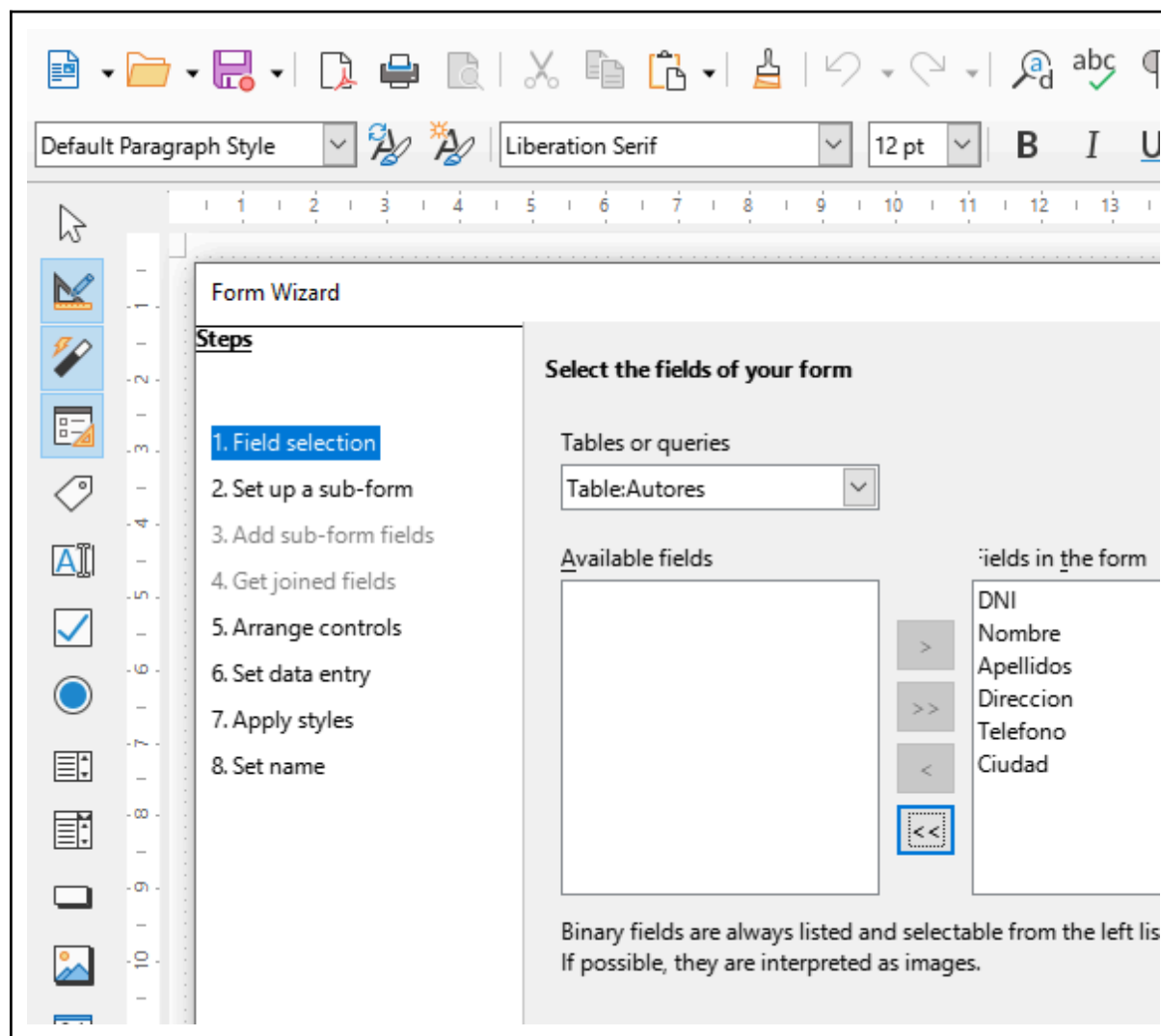
¿Podemos visualizar todos los libros de un autor en la misma pantalla?

Sí. Para ello crearemos un formulario con el asistente (Use Wizard to Create Form).

Para crear este tipo de formulario deberemos abrir nuestra base de datos y hacer clic en “Forms” (formularios) en la parte izquierda de la ventana principal.

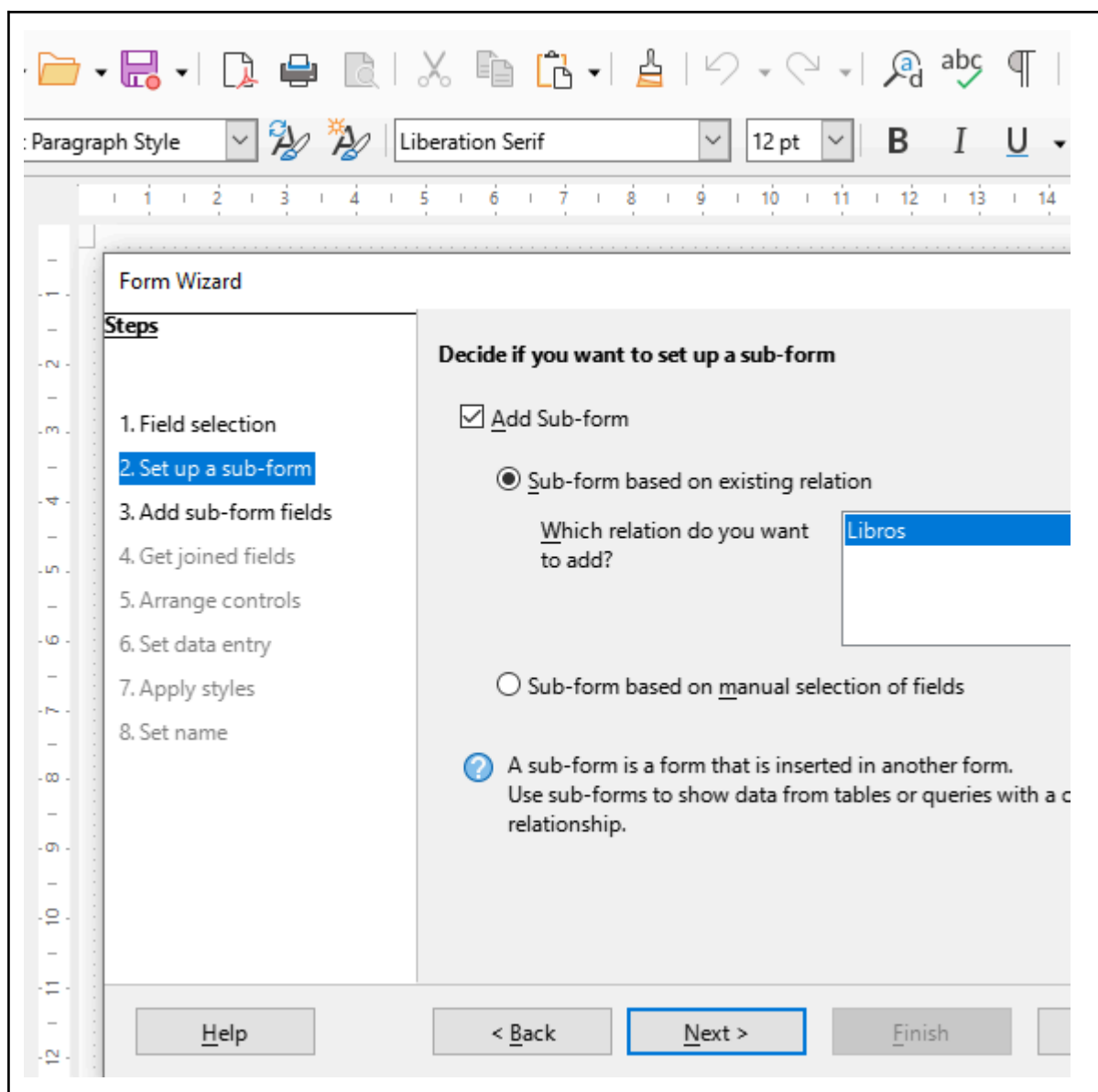


A continuación elegiremos todos los campos de la tabla “Autores”:



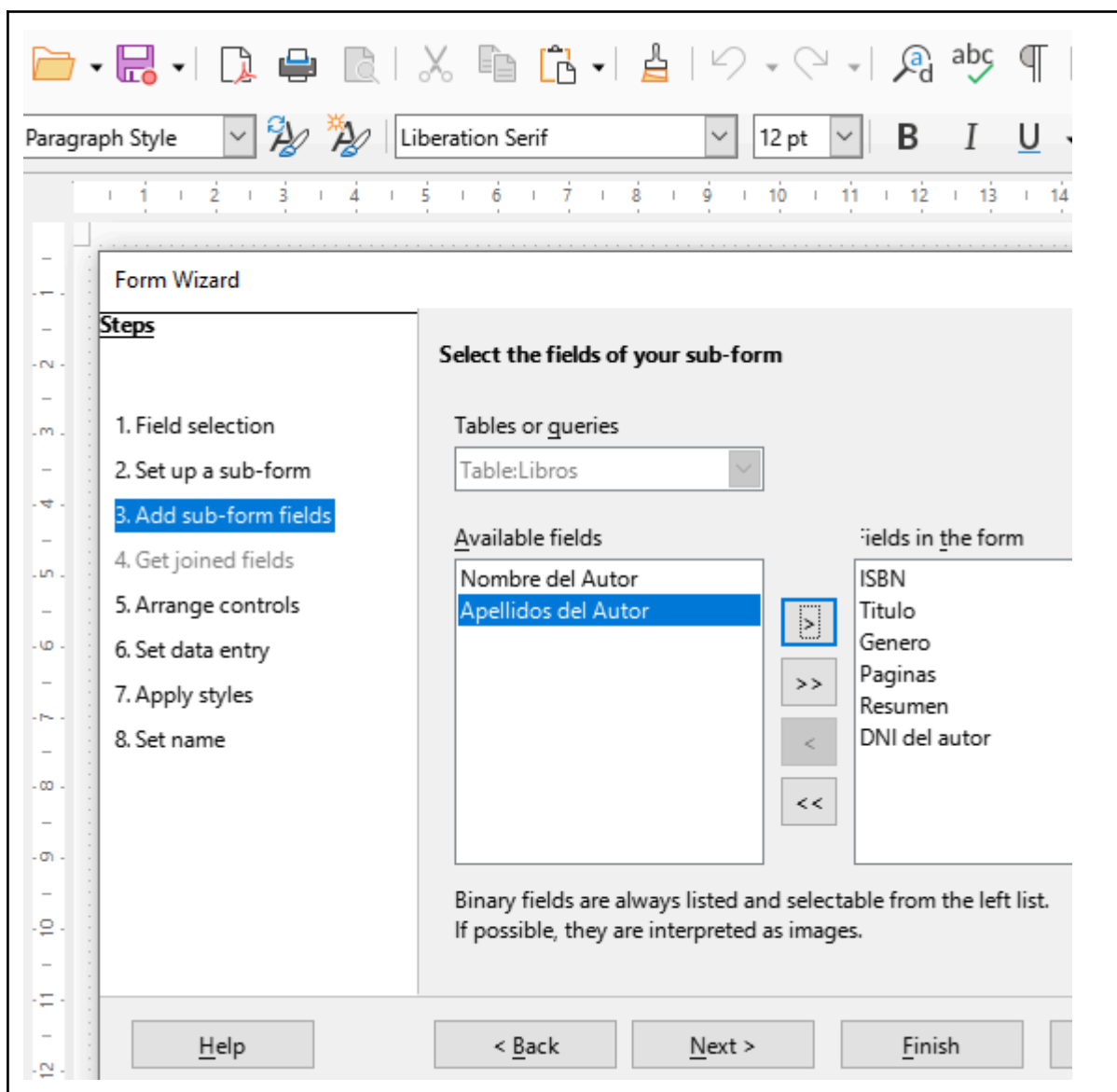
Todos los campos de la tabla “Autores” pasarán al cuadro de la derecha.

A continuación elegiremos “Sub-form based on existing relation” (sub-formulario basado en una relación existente) y marcaremos la relación “Libros”.



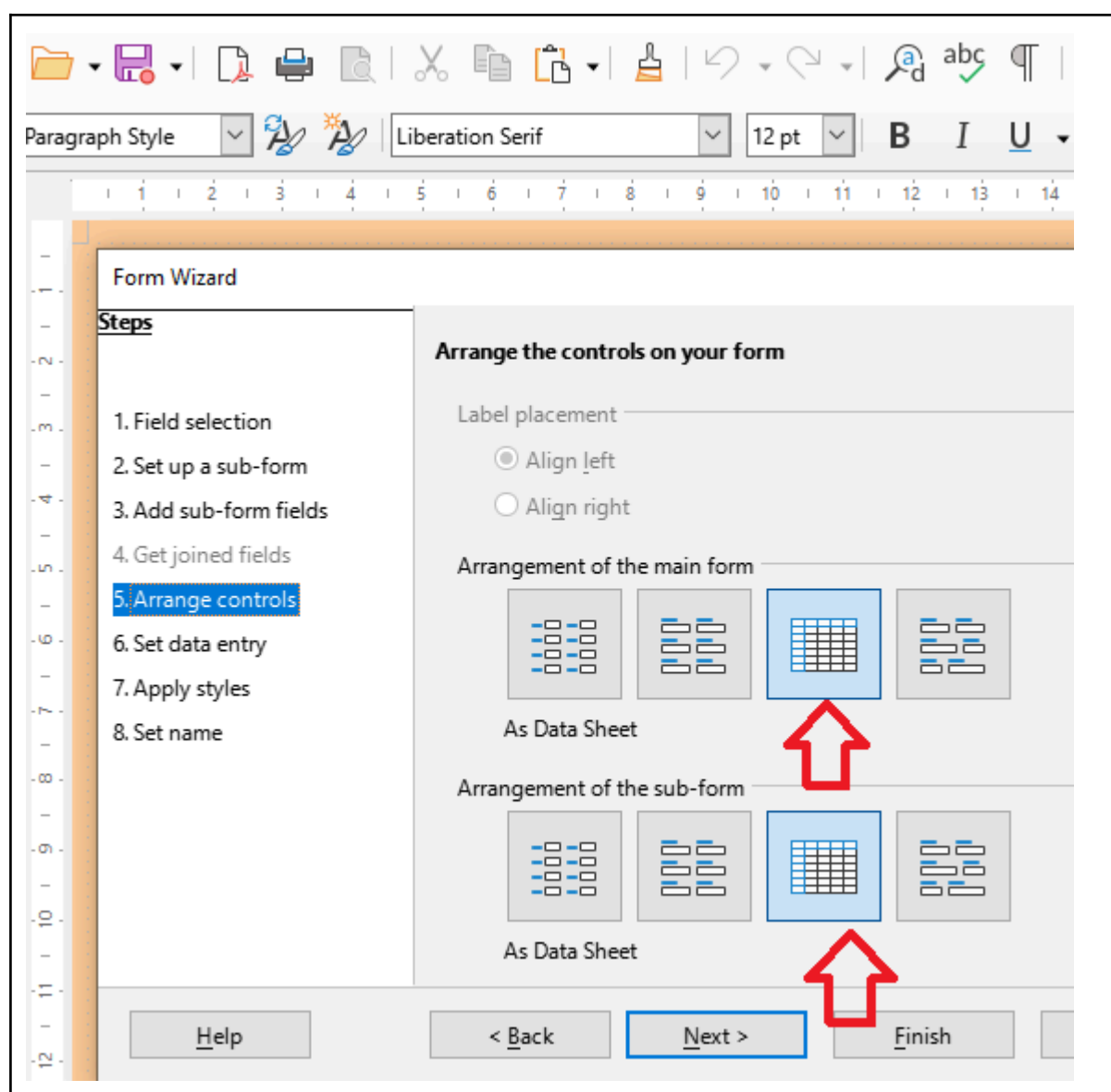
Después de comprobar que hemos marcado la relación “Libros”, haremos clic en “Next”.

En este paso seleccionaremos entre los campos disponibles (“Available fields”) los campos que deseamos incluir en el subformulario: ISBN, Título, Género, Páginas, Resumen y DNI del autor (hemos incluido DNI del autor para comprobar que el formulario principal y el subformulario tienen una coincidencia de DNI. Es decir que este campo es el nexo de unión entre los dos formularios).



A continuación, haremos clic en “Next” para continuar.

En este paso, elegiremos la disposición del formulario principal (“Main form”) y la disposición del subformulario (Subform).



A continuación, haremos clic en “Next” para continuar.

En este paso elegiremos “The form is to display all data” (el formulario visualizará todos los datos).

Form Wizard

Steps

1. Field selection
2. Set up a sub-form
3. Add sub-form fields
4. Get joined fields
5. Arrange controls
- 6. Set data entry**
7. Apply styles
8. Set name

Select the data entry mode

☐ The form is to be used for entering new data only.
Existing data will not be displayed

☒ The form is to display all data

☐ Do not allow modification of existing data

☐ Do not allow deletion of existing data

☐ Do not allow addition of new data

Help < Back Next > Finish

A continuación, haremos clic en “Next” para continuar.

En este paso elegiremos el estilo que queremos para nuestro diseño:

The image shows a screenshot of the Microsoft Word interface. At the top, the ribbon is visible with the 'Paragraph Style' dropdown set to 'Liberation Serif' and the font size set to '12 pt'. Below the ribbon, the 'Form Wizard' dialog box is open. The 'Steps' list on the left includes: 1. Field selection, 2. Set up a sub-form, 3. Add sub-form fields, 4. Get joined fields, 5. Arrange controls, 6. Set data entry (highlighted in blue), 7. Apply styles, and 8. Set name. The 'Select the data entry mode' section on the right contains four options: 'The form is to be used for entering new data only. Existing data will not be displayed' (unselected), 'The form is to display all data' (selected with a radio button), 'Do not allow modification of existing data' (unselected), 'Do not allow deletion of existing data' (unselected), and 'Do not allow addition of new data' (unselected). At the bottom of the dialog, there are buttons for 'Help', '< Back', 'Next >' (highlighted in blue), and 'Finish'.

A continuación, haremos clic en “Next” para continuar.

En este paso teclearemos el nombre de nuestro nuevo formulario: “Autores con sus libros”:

The screenshot shows the Microsoft Word interface with the Form Wizard dialog box open. The dialog box is titled "Form Wizard" and has a "Steps" list on the left. The steps are: 1. Field selection, 2. Set up a sub-form, 3. Add sub-form fields, 4. Get joined fields, 5. Arrange controls, 6. Set data entry, 7. Apply styles, and 8. Set name. Step 8 is currently selected. The main area of the dialog box is titled "Set the name of the form". It contains a text box labeled "Name of the form" with the text "Autores con sus libros" entered. A red arrow points to this text box. Below the text box is a section titled "How do you want to proceed after creating the form?" with two radio button options: "Work with the form" (selected) and "Modify the form". At the bottom of the dialog box are four buttons: "Help", "< Back", "Next >", and "Finish".

A continuación, haremos clic en “Finish” para grabar nuestro diseño.

Este es el resultado del diseño de nuestro formulario donde se muestran todos los libros de cada autor. Al hacer clic en cualquier autor en el formulario principal, se mostrarán los libros de ese autor en el subformulario.

.libreria2025.odb : Autores con sus libros — LibreOffice Base: Database Form

Edit View Insert Format Styles Table Form Tools Window Help

	DNI	Nombre	Apellidos	Direccion	Telefono	Ciudad
▶	42700700L	Santiago	Garcia	Triana 2	928102020	Madrid
	42700800L	Joe	Merlino	Triana 3	928101022	Paris
	42777555L	Douglas	Hergert	Triana 4	928000999	Dublin
	42800800K	Javier	Garcia	Gomera 2	928950095	Lisboa
	4286200A	John	Smith	Triana 1	928102030	Telde
	42888000L	Juan	Espinosa	Triana 5	928900900	Telde
	50123000L	Gary	Bennet	Gomera 1	928999000	Sevilla

Record 1 of 7

	ISBN	Titulo	Genero	Paginas	Resumen
▶	84-415-1043-1	Red Hat Linux 6	Linux	290	Un libro sobre Linux genial.
+					

Record 1 of 1

Haremos clic en “File” y después clic en “Close” para cerrar la ventana. A continuación, cierre LibreOffice pulsando CTRL+Q (recuerde grabar los cambios).

Puesto que ahora sabemos que un autor puede tener más de un libro en nuestra base de datos, deberíamos poder imprimir un informe con todos los libros de cada autor agrupado por DNI, ya que este campo es el campo común a las dos tablas: Autores y Libros.

Le planteamos el reto de crear este informe y añadir al informe en el encabezado el número de página y el título del informe. El informe quedaría así:

Listado de Libros de cada autor					Pag. 1 de 2
DNI del autor 42700700L					
ISBN 84-415-1043-1	Título Red Hat Linux 6	Genero Linux	Nombre del Autor Santiago	Apellidos del Autor Eibe-Garcia	
DNI del autor 42700800L					
ISBN 84-481-3112-6	Título Corel Linux	Genero Linux	Nombre del Autor Joe	Apellidos del Autor Merlino	
DNI del autor 42777555L					
ISBN 84-7614-290-0	Título Guia completa de Dbase IV	Genero Bases de datos	Nombre del Autor Douglas	Apellidos del Autor Hergert	
DNI del autor 42800800K					
ISBN 978	Título Fundamentos de la ofimática	Genero Ofimática	Nombre del Autor Javier	Apellidos del Autor Garcia	

El informe continúa en la siguiente página:

Segunda página del informe:

Listado de Libros de cada autor					Pag. 2 de 2
DNI del autor 4286200A					
ISBN 978-0714867380	Título The Photography Book	Genero Fotografía	Nombre del Autor Ian	Apellidos del Autor Jeffrey	
978-0714845573	Photography: History	Fotografía	Ian	Jeffrey	
DNI del autor 42888000L					
ISBN 84-267-1311-4	Título Red Hat Linux 7	Genero Linux	Nombre del Autor Juan	Apellidos del Autor Espinosa	
DNI del autor 50123000L					
ISBN 978-1-4302-2357-3	Título iPhone cool projects	Genero iPhone	Nombre del Autor Gary	Apellidos del Autor Bennet	

Podemos comprobar que el primer autor de la segunda página del informe, Ian Jeffrey, tiene dos libros en nuestra base de datos. Esto quiere decir que nuestro informe agrupa correctamente los libros de cada autor (la agrupación se hace de acuerdo al DNI del autor, que es el campo clave).

Reto 1:

Construya una base de datos con las funciones de Excel y su correspondiente explicación. Utilice las funciones incluidas en el apéndice C.

Reto 2:

Construya una base de datos con los sistemas operativos de Microsoft y los distintos marcos de desarrollo .NET utilizados por cada S.O.

Importante: LibreOffice Base en entornos multiusuario:

La versión de SQLite que utilice LibreOffice Base en su ordenador depende de la biblioteca o controlador instalado en el sistema operativo de red. Aunque la última versión de LibreOffice es la 26.8.0 (a fecha de agosto de 2026), la versión recomendada para entornos de red corporativos que priorizan la estabilidad es la versión 26.2.6.

Dos o más usuarios de la base de datos compartida que estén utilizando dos ordenadores diferentes para acceder al archivo SQLite que contiene los datos deben tener en cuenta las siguientes consideraciones:

1. Si se intenta compartir la base de datos colocándolo en una carpeta de compartida de red como Samba, NAS o Dropbox, se producirá un error por corrupción de datos o bloqueo de archivos (recuerde que el motor HSQLDB o Firebird incluido en LibreOffice es **monopuesto**).
2. Para acceder de forma simultánea a los datos, la base de datos debe alojarse en un servidor centralizado. LibreOffice Base solo funciona como “*front-end*” para hacer consultas, informes o utilizar formularios. Los dos usuarios que quieren conectarse al servidor deberán hacerlo a través de conectores ODBC o JDBC. En vez de utilizar el motor SQLite incluido en LibreOffice Base, deberemos instalar el motor **PostgreSQL** o MariaDB en el servidor.
3. Si necesitamos que dos o más usuarios editen datos en tiempo real simultáneamente, un único usuario deberá abrir y modificar información en la base de datos mientras los otros tengan cerrada el archivo de datos. Para ello se establecerá un sistema de turnos o un calendario de uso.

Principales similitudes y diferencias de sintaxis

Estándar ANSI SQL (PostgreSQL): Tanto SQLite como PostgreSQL priorizan seguir el estándar internacional. Usan delimitadores de texto estándar (comillas simples ' para cadenas), operadores lógicos estándar y estructuras SQL tradicionales.

Concatenación de cadenas: Ambos utilizan el operador estándar de doble tubería (||) para concatenar texto ('Hola ' || 'Mundo'). MariaDB utiliza la función CONCAT() o requiere configurar un modo SQL especial para habilitar ||.

Identificadores (Tablas y Columnas): Tanto SQLite como PostgreSQL aceptan comillas dobles ("nombre_tabla") para escapar nombres de objetos. MariaDB utiliza por defecto comillas invertidas (backticks: `nombre\_tabla`).

Funciones avanzadas:

En versiones recientes, SQLite ha implementado sintaxis muy parecida a PostgreSQL para Common Table Expressions (CTE con WITH), funciones de ventana (Window Functions) y operadores JSON.

Creación y configuración de bases de datos con Microsoft Visual FoxPro.

Microsoft Visual FoxPro es un gestor de bases de datos con un asistente integrado muy potente que puede manejar archivos con extensión .txt, .xls y .dbf entre otros. También es conocido por su capacidad para crear aplicaciones de bases de datos robustas en entornos tanto mono como multiusuario.

Aunque el uso del formato de archivo .dbf ha disminuido con el tiempo, fue muy popular en los años 80 y 90, y todavía se encuentra funcionando correctamente en miles de servidores y sistemas modernos como Microsoft Azure, Microsoft Access o LibreOffice Base.

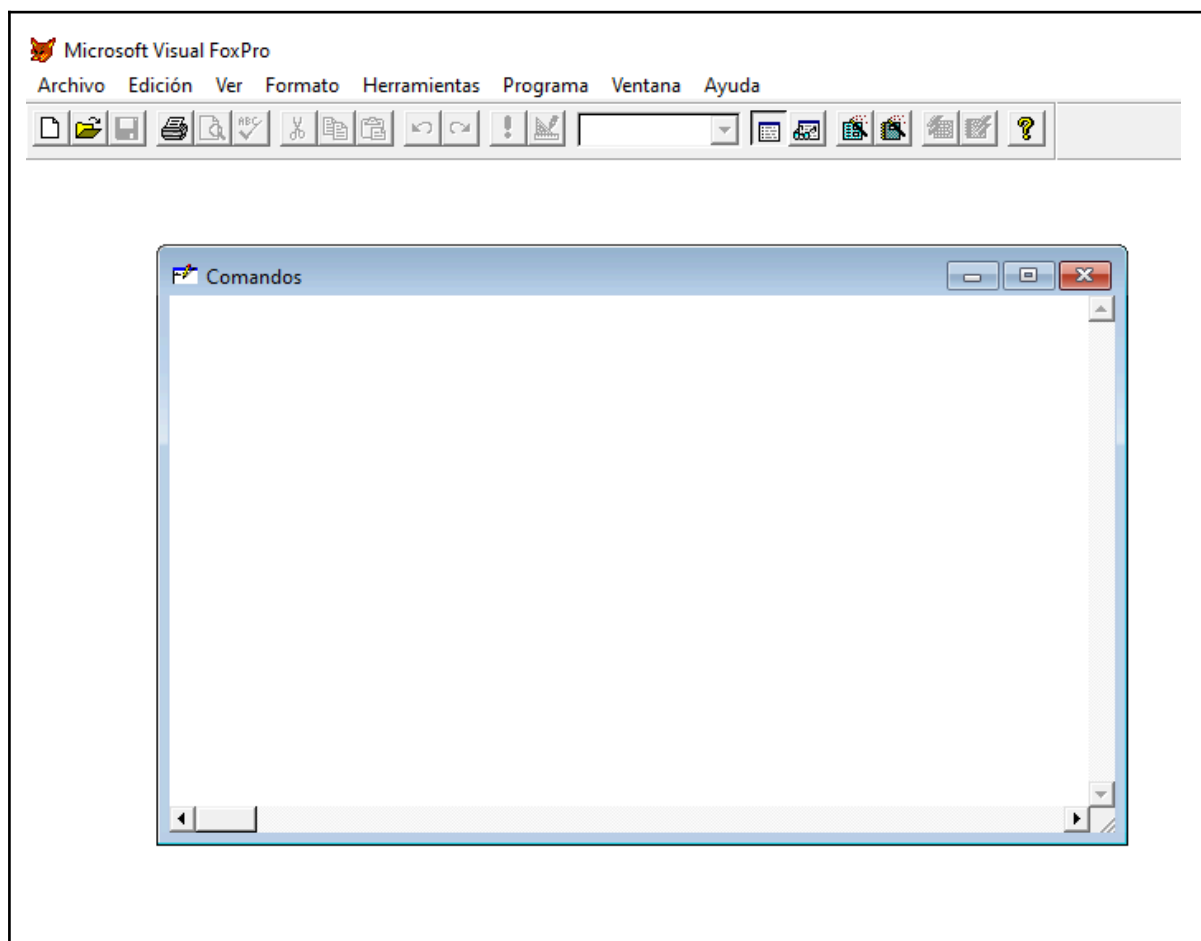
Muchos usuarios corporativos utilizan el formato .dbf en el día a día pues es un sistema intuitivo basado en comandos e instrucciones sencillas, especialmente en entornos de red donde se requiere un manejo básico de bases de datos.

El formato .dbf actualmente se utiliza en algunos sistemas gubernamentales en los Estados Unidos, especialmente en programas relacionados con la gestión de datos censales. Un ejemplo es el Programa de Actualización Local de Direcciones Censales (LUCA), que se implementó para el Censo del 2020. Este programa permite localizar y actualizar la información almacenada en ficheros con formato .dbf. No obstante, FoxPro alcanzó el estado de fin de vida útil (EOL) en 2015.

Es importante recordar que Microsoft Power BI permite conectar con diversas fuentes de datos, desde archivos de textos, hojas de cálculo simples, bases de datos SQL o incluso archivos con formato .dbf. Los ejemplos proporcionados en este libro se presentan únicamente con fines educativos para la creación de bases de datos e informes simples.

A continuación, crearemos una base de datos de prueba muy simple con un archivo con extensión .dbf que contendrá una lista con nombres, apellidos y teléfonos:

Al abrir Microsoft Visual FoxPro, en adelante, VFP, veremos el centro de control, llamado “Comandos” o “Command Prompt”.



En la ventana inferior, llamada “Comandos”, se introducen los comandos para ejecutar distintas acciones, como por ejemplo, crear un archivo para grabar los nombres y teléfonos de nuestra agenda.

Llamaremos a nuestro archivo “Contactos”.

El archivo “Contactos” tendrá por defecto la extensión .dbf y en él se almacenará la información de nuestros contactos.

Veamos cómo utilizar la creación interactiva de archivos DBF:

1. Escriba “**CREATE**” en la Ventana de Comandos de Visual FoxPro y presione Enter.
2. Aparecerá un cuadro de diálogo pidiéndole que le dé un nombre al nuevo archivo DBF (ej., “Contactos.dbf”).
3. Se abrirá la pantalla del diseñador de tablas (“TABLE DESIGNER”).
4. Aquí, usted debe introducir manualmente el Nombre del Campo (“Field Name”), selecciona el Tipo (“Type”) del menú desplegable y establece el Ancho (“Width”) y los Lugares Decimales (“Decimals”) (para campos Numéricos).

Los campos necesarios para nuestro ejemplo son: nombre, apellidos y teléfono. Todos ellos son tipo carácter con un ancho de 30 caracteres.

5. Haga clic en “OK” para guardar la estructura y crear el archivo DBF vacío (donde introducirá los datos de cada registro).

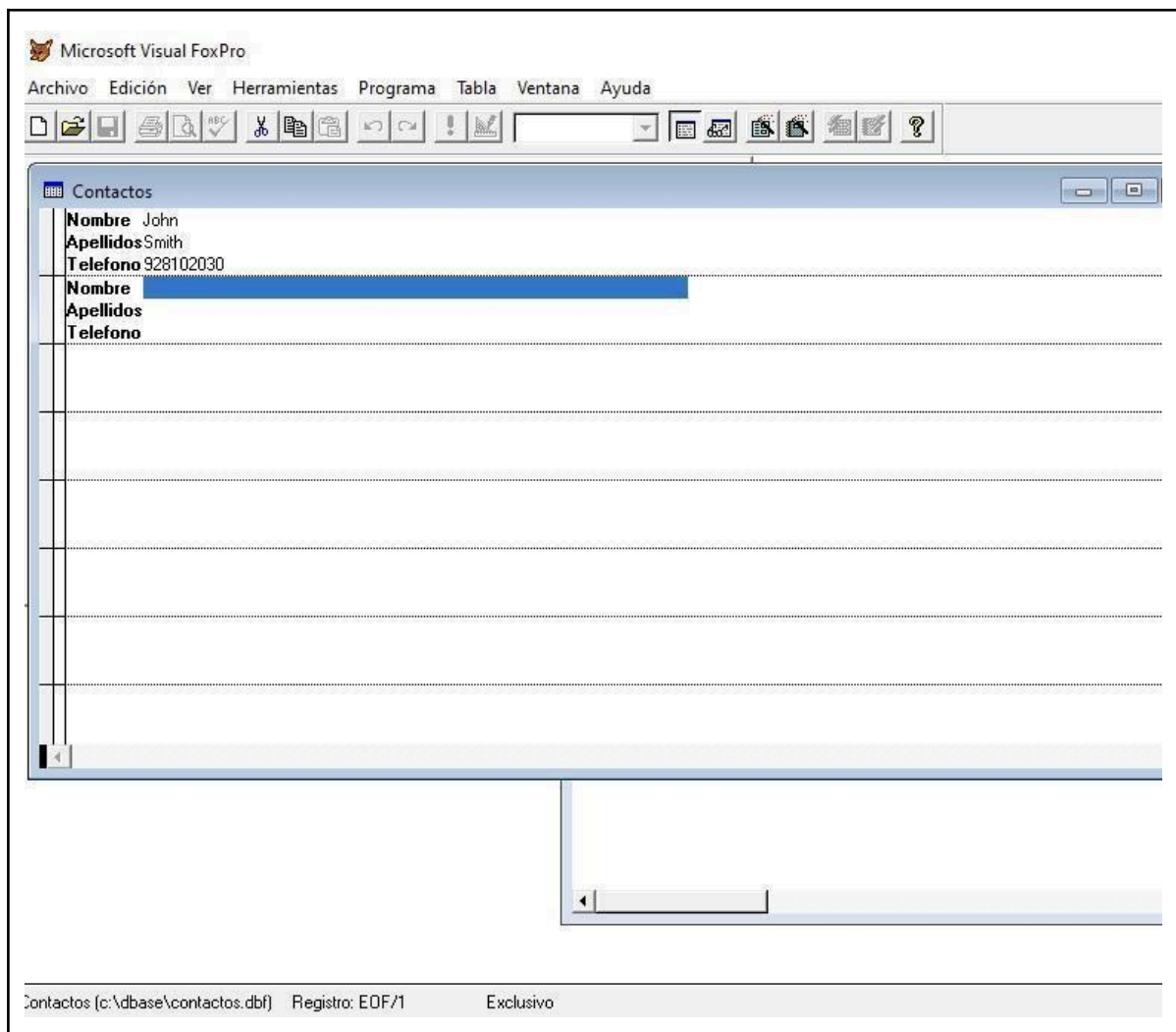
Antes de comenzar, cree una carpeta en su disco llamada **C:\DBASE**. En esta carpeta almacenaremos los archivos de nuestra base de datos.

Introduzca la siguiente orden para crear el archivo Contactos.dbf con el comando:

```
CREATE TABLE C:\DBASE\CONTACTOS.DBF (NOMBRE C(33), APELLIDOS C(33), TELEFONO C(9))
```

A continuación introduzca el siguiente comando para introducir los datos del primer contacto, John:

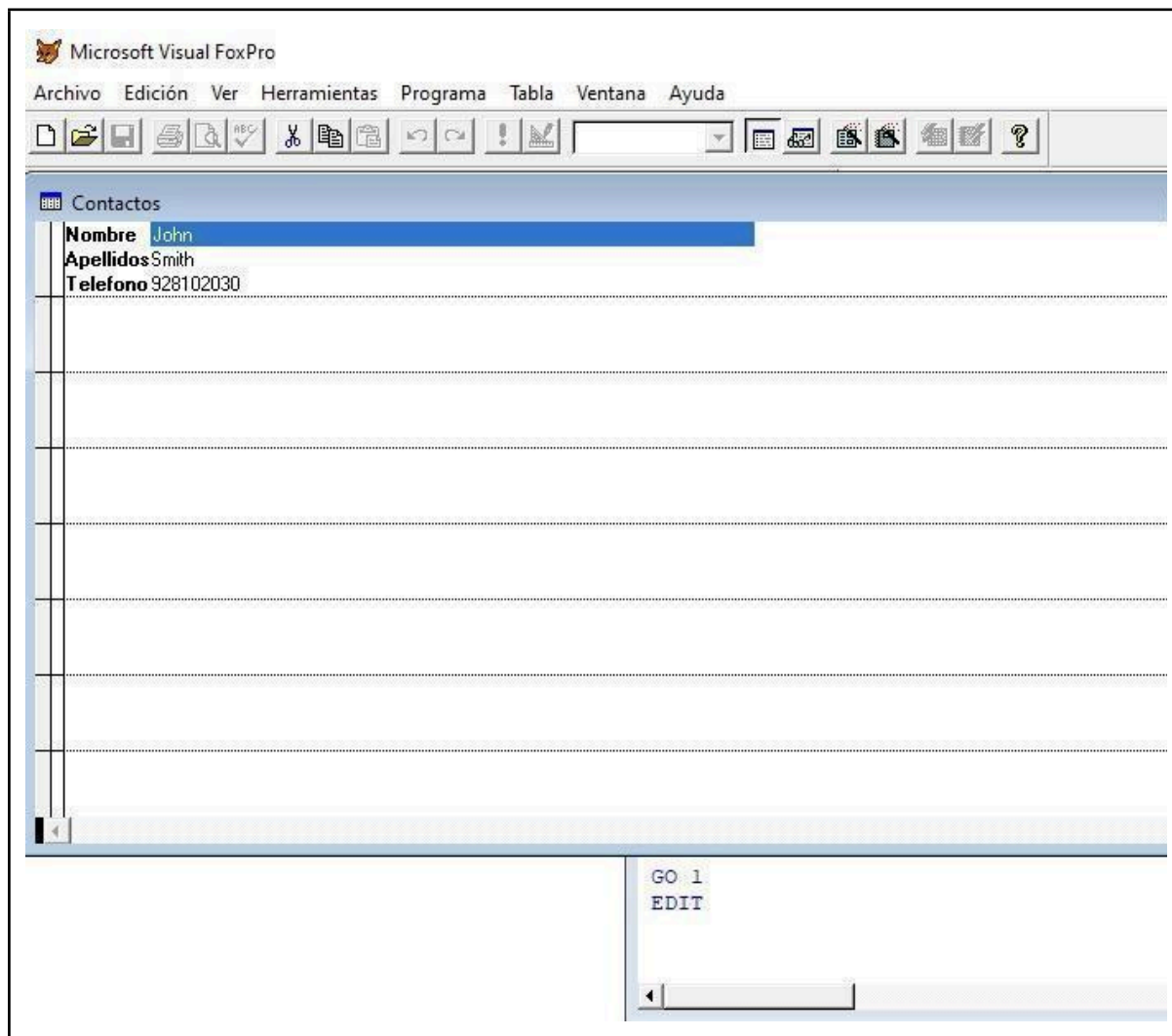
APPEND



A continuación introduzca el siguiente comando para editar los datos del primer contacto, John:

GO 1

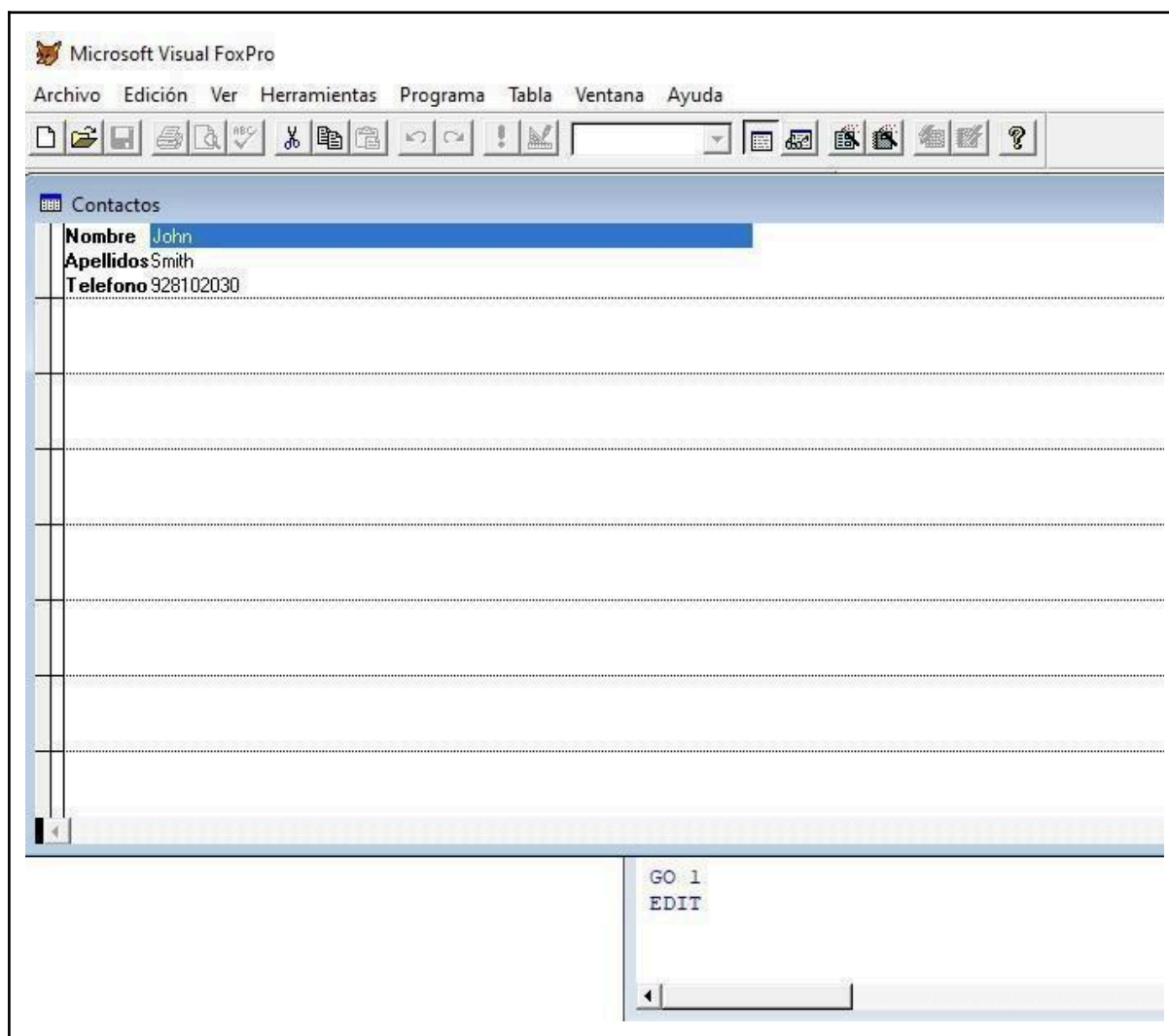
EDIT



Importante:

El comando **GO 1** prepara el fichero, seleccionado anteriormente con la orden **USE**, para acceder al registro número 1. Puede acceder al registro número 2, cambiando la orden por: **GO 2**.

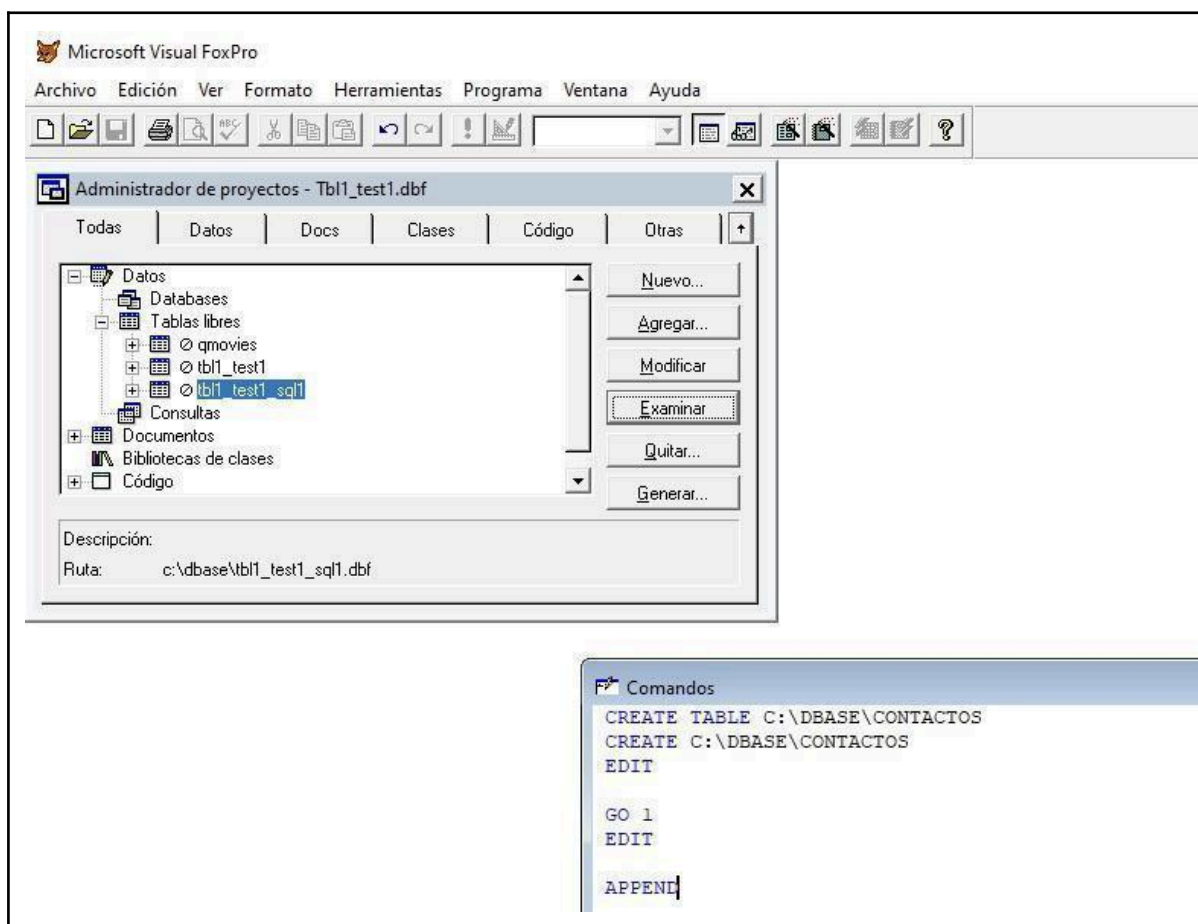
Como puede observar, se ha abierto la ficha del primer contacto, John, para poder editarla.



En la ventana “Contactos” podrá editar cualquier campo, como por ejemplo, el nombre o los apellidos.

A continuación introduzca el siguiente comando para introducir los datos del segundo contacto, Pablo:

APPEND



Introduzca los datos del segundo contacto, Pablo, junto a los campos Nombre, Apellidos y Teléfono:

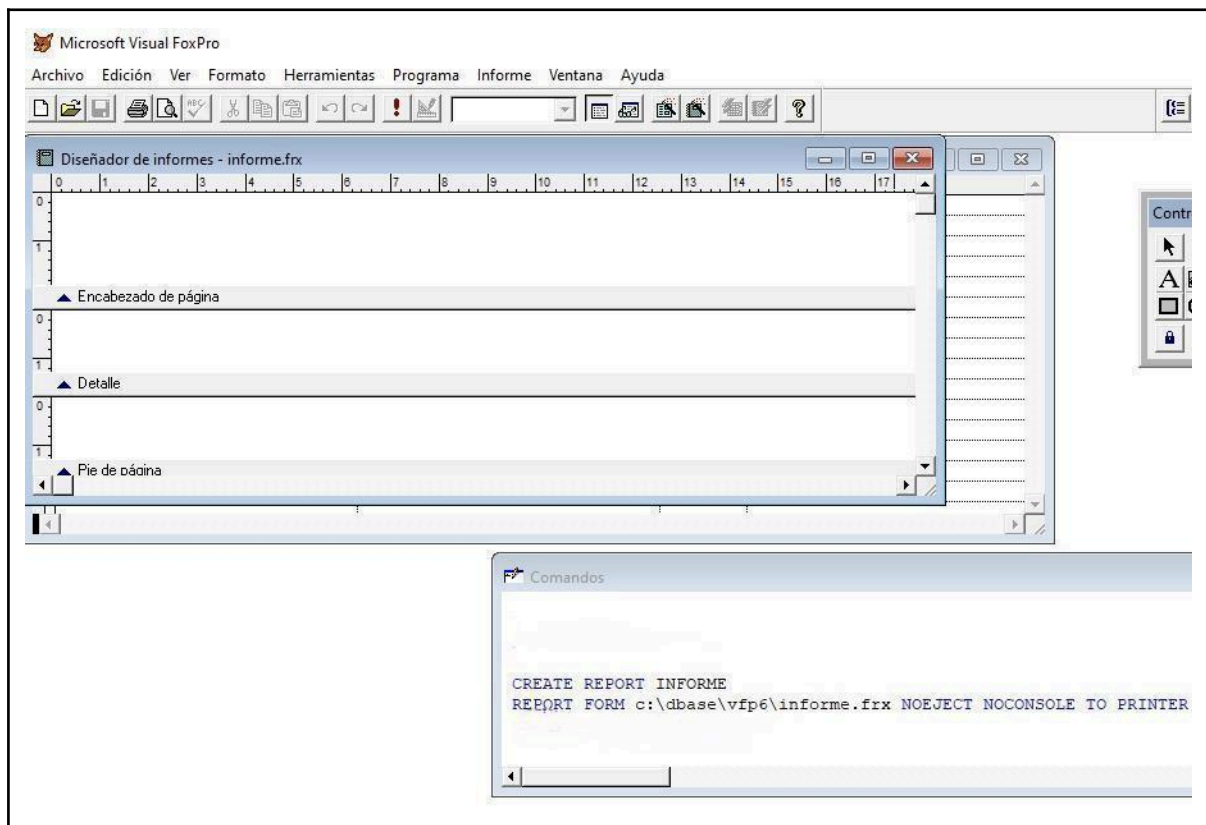
The screenshot shows the Microsoft Visual FoxPro interface. The title bar reads 'Microsoft Visual FoxPro'. The menu bar includes 'Archivo', 'Edición', 'Ver', 'Herramientas', 'Programa', 'Tabla', 'Ventana', and 'Ayuda'. The toolbar contains various icons for file operations, editing, and viewing. The main window displays a table named 'Contactos' with the following structure:

Nombre	Apellidos	Telefono

At the bottom right of the window, there are buttons for 'EDIT' and 'APPEND', and a small text input field.

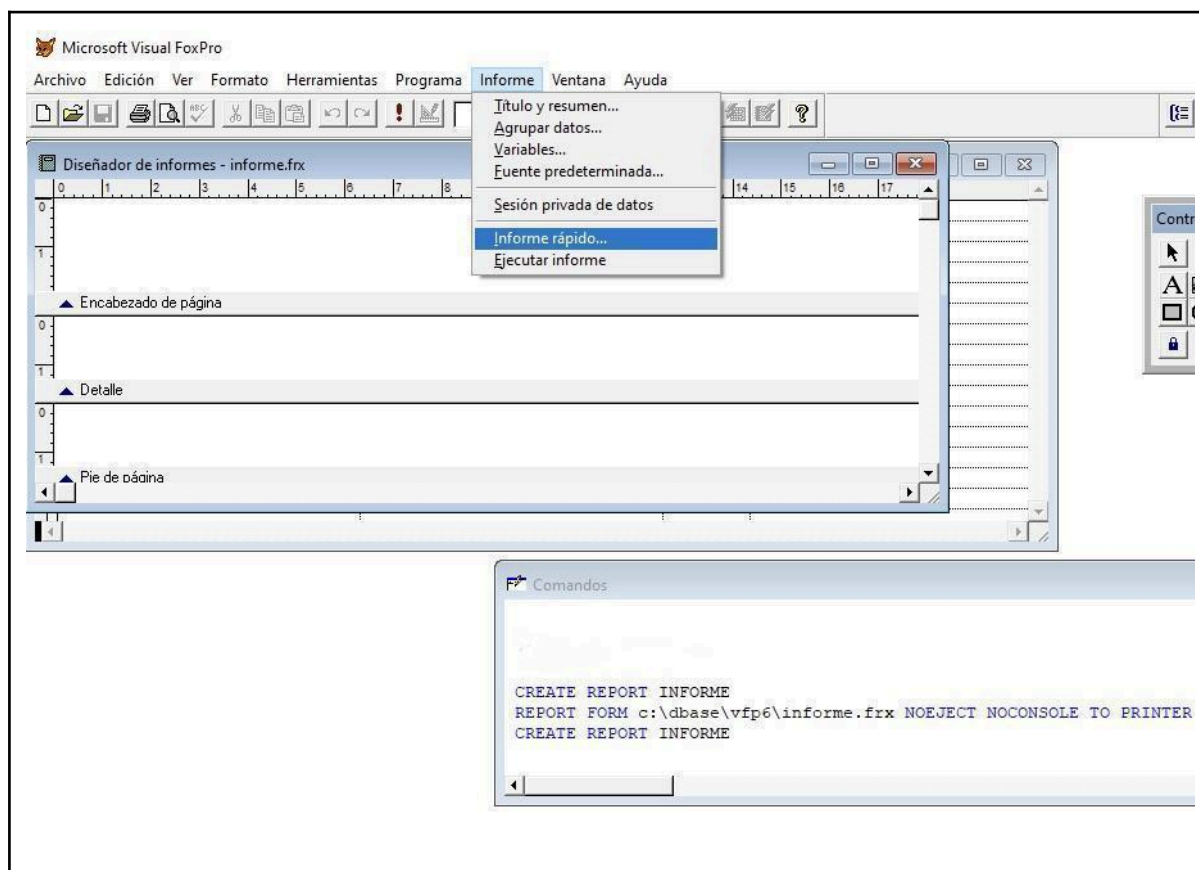
A continuación introduzca el siguiente comando para preparar el **informe** de personas almacenadas en el archivo CONTACTOS.dbf.

CREATE REPORT C:\DBASE\INFORME

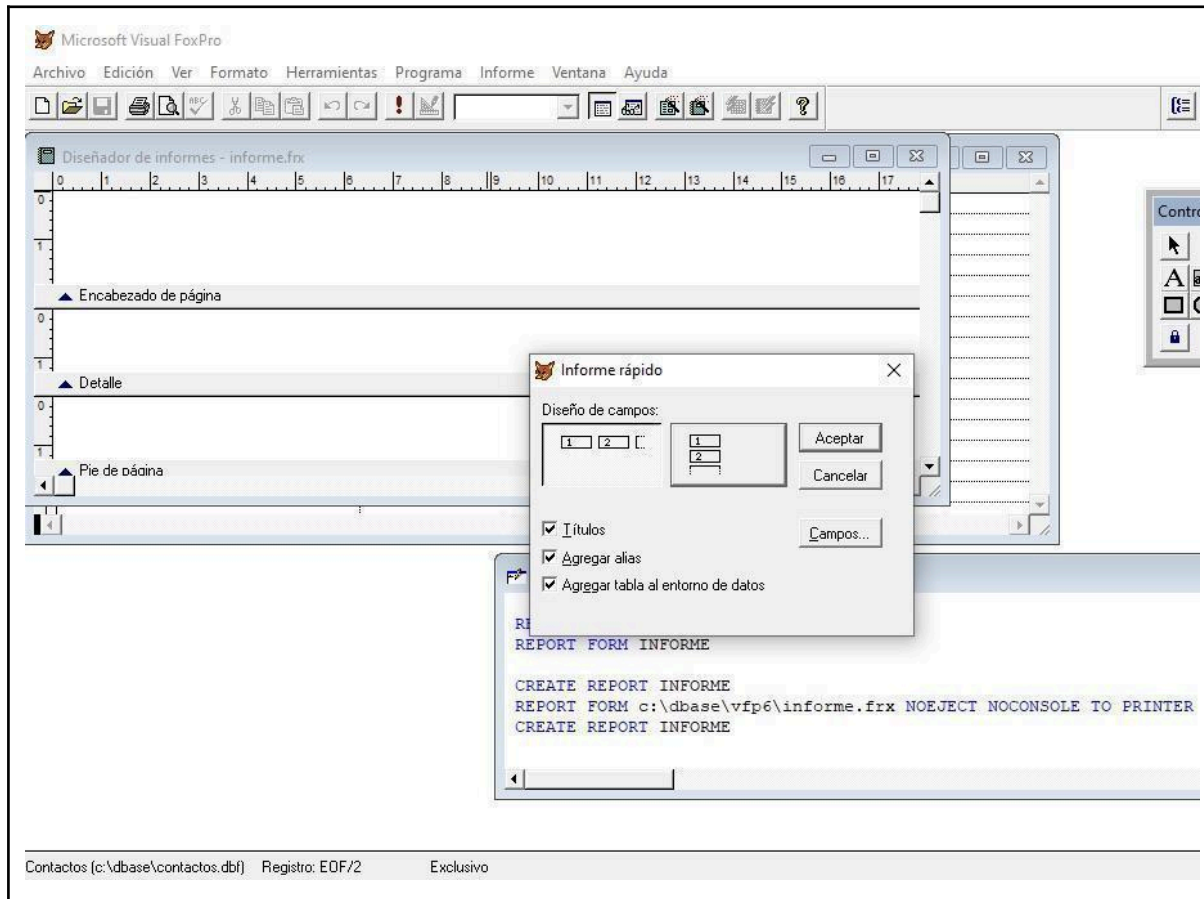


VFP creará un informe, vacío por ahora, para incluir los campos del archivo CONTACTOS.DBF.

Haga clic en **Informe** y a continuación, haga clic en **Informe rápido**.

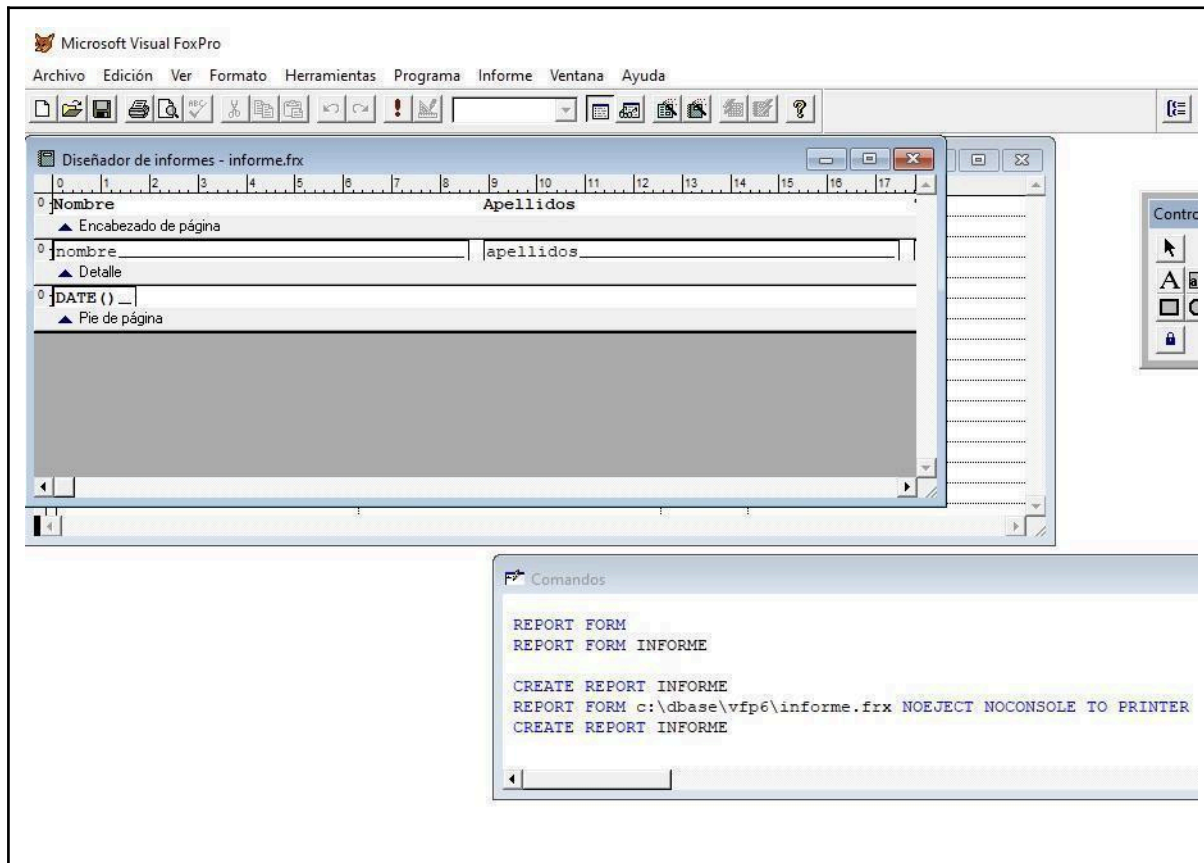


A continuación, haga clic en el **Diseño de campos** que desee para su informe final.



Haga clic en **Aceptar** para continuar.

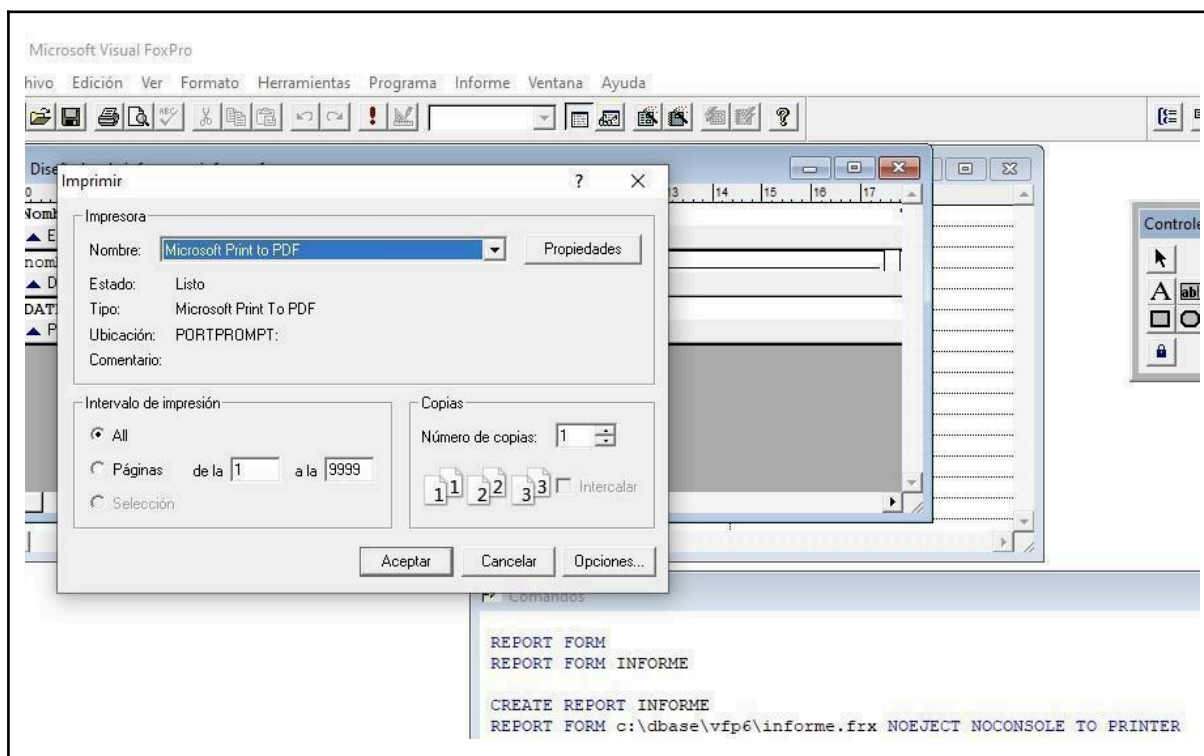
VFP añadirá los campos al **Diseñador de informes**. Observe que este informe será almacenado como **informe.frx**.



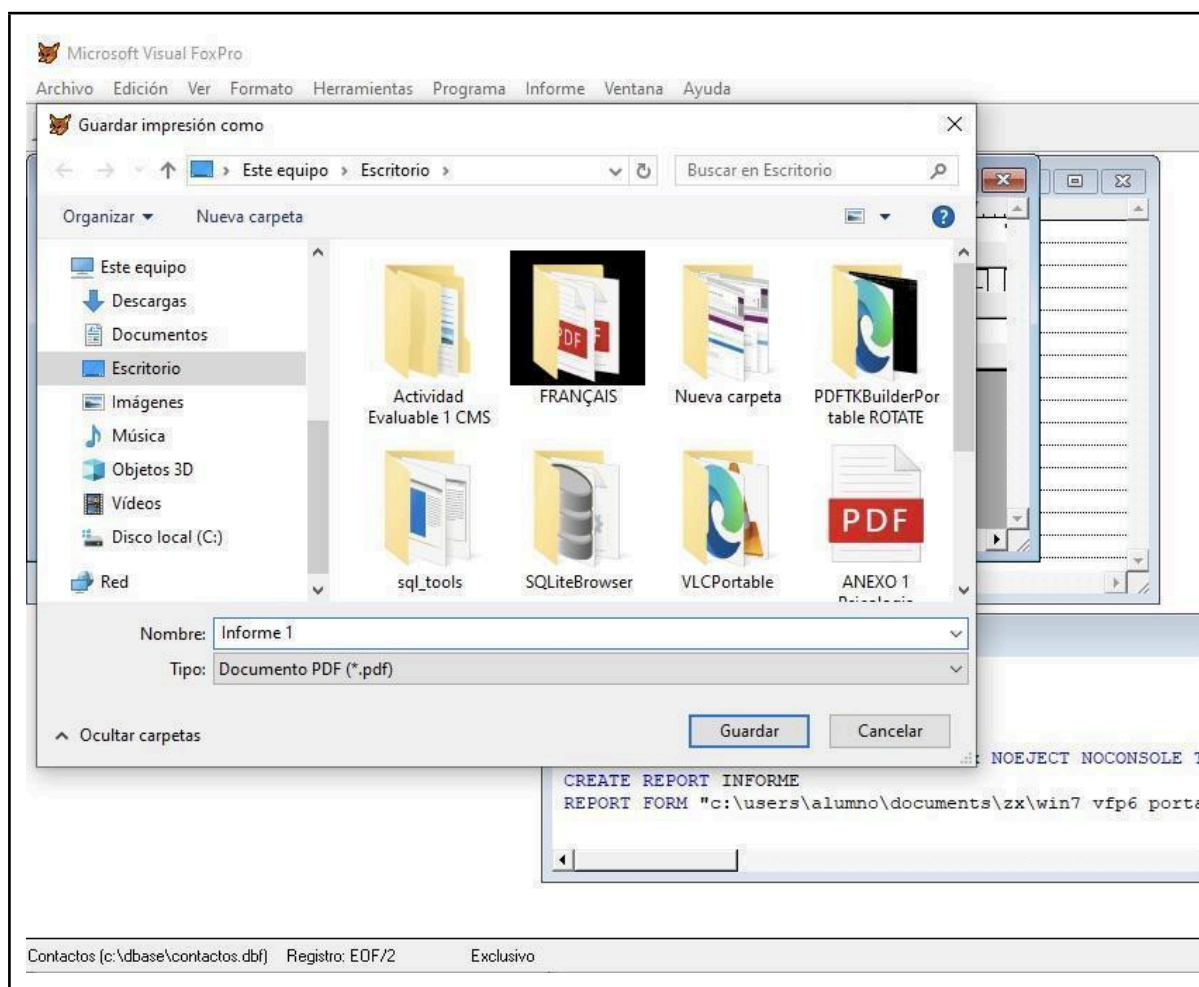
A continuación introduzca el siguiente comando para preparar el informe de registros almacenado en el archivo CONTACTOS.dbf:

```
REPORT FORM C:\DBASE\INFORME.FRX NOEJECT NOCONSOLE TO PRINTER
```

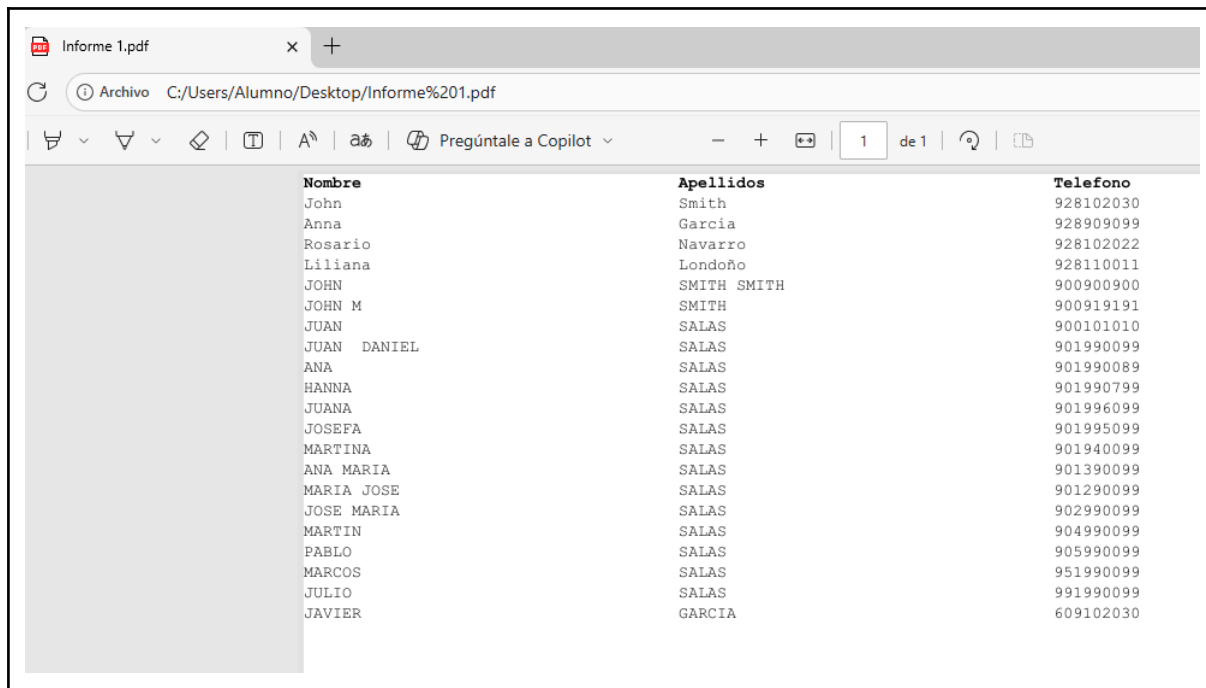
Elija la impresora “Microsoft Print to PDF”. Esto hará que se genere un archivo en formato PDF con el informe de la tabla CONTACTOS.dbf.



Guarde el archivo con el nombre “Informe 1.PDF” en su escritorio. Este archivo contendrá un listado con todos los contactos almacenados en la tabla CONTACTOS.DBF.



Para comprobar que el informe ha sido creado con éxito, cierre Microsoft Visual FoxPro y busque en su ordenador el archivo PDF creado anteriormente. Podrá abrirlo con la aplicación Microsoft Edge o con la aplicación Adobe Acrobat Reader:



The screenshot shows a PDF document titled 'Informe 1.pdf' open in a browser. The document contains a table with three columns: 'Nombre', 'Apellidos', and 'Telefono'. The table lists 20 contacts. The first contact is John Smith with phone number 928102030. The last contact is JAVIER GARCIA with phone number 609102030.

Nombre	Apellidos	Telefono
John	Smith	928102030
Anna	Garcia	928909099
Rosario	Navarro	928102022
Liliana	Londoño	928110011
JOHN	SMITH SMITH	900900900
JOHN M	SMITH	900919191
JUAN	SALAS	900101010
JUAN DANIEL	SALAS	901990099
ANA	SALAS	901990089
HANNA	SALAS	901990799
JUANA	SALAS	901996099
JOSEFA	SALAS	901995099
MARTINA	SALAS	901940099
ANA MARIA	SALAS	901390099
MARIA JOSE	SALAS	901290099
JOSE MARIA	SALAS	902990099
MARTIN	SALAS	904990099
PABLO	SALAS	905990099
MARCOS	SALAS	951990099
JULIO	SALAS	991990099
JAVIER	GARCIA	609102030

Reto:

Añada el campo “Sueldo”, de tipo numérico, a la tabla CONTACTOS.dbf. Añada al primer contacto de la lista un sueldo de 1500 euros y añade al último contacto de la lista un sueldo de 2500 euros. A continuación cree un informe donde se vea cada contacto con su respectivo sueldo.

Importante:

Al utilizar archivos con formato dbf en una red local multiusuario, debemos grabar los archivos a compartir con los permisos correctos para cada usuario. Esto implica activar bloqueos de registros y ficheros mediante programación, además de hacer numerosas pruebas de inserción de registros y búsquedas simultáneas. Recuerde que por defecto, los archivos están bloqueados para acceso compartido.

INTRODUCCIÓN A LA PROGRAMACIÓN DE APLICACIONES PARA
REDES LOCALES CON MICROSOFT VISUAL FOXPRO

Ejemplo 1f. Hola Mundo.

Objetivo: Entender la estructura de un programa escrito en Microsoft Visual FoxPro mediante el desarrollo del ejemplo más básico: “Hola Mundo”.

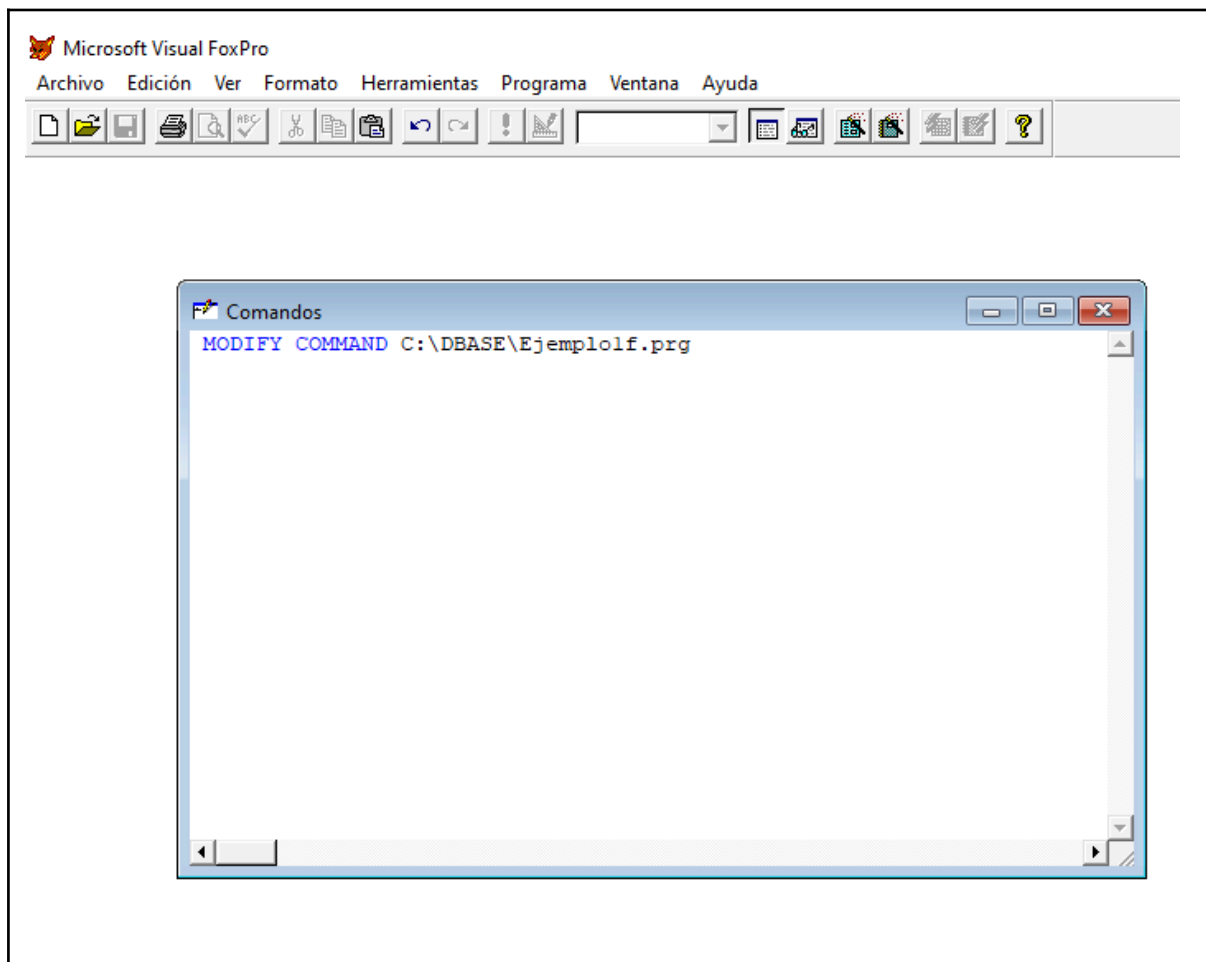
Para empezar, abra el Shell (el Shell es un intérprete interactivo de comandos).

Haga clic en:

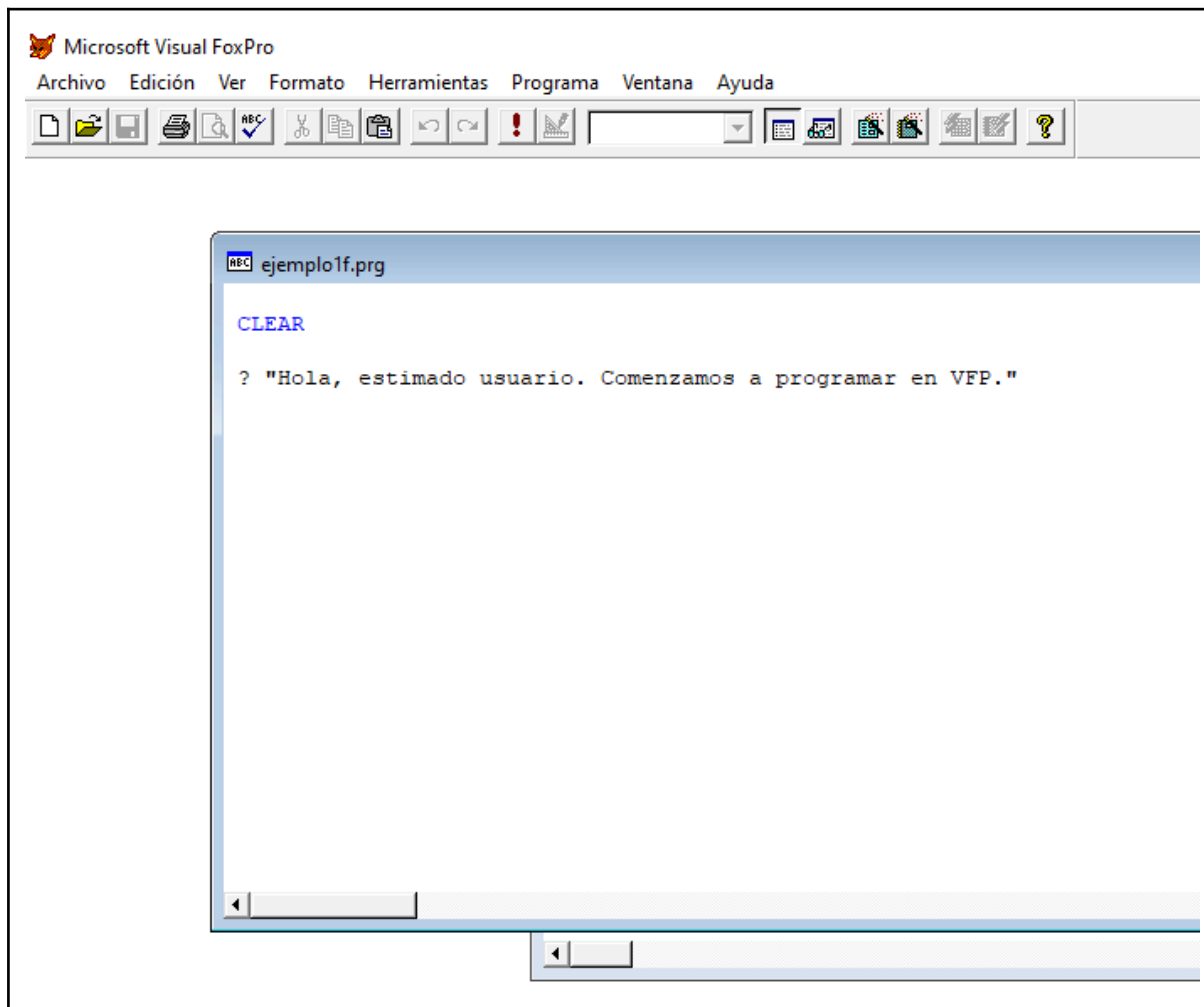
Start > Programs > Visual FoxPro

Para crear un nuevo programa, escriba en la ventana “Comandos” la siguiente instrucción:

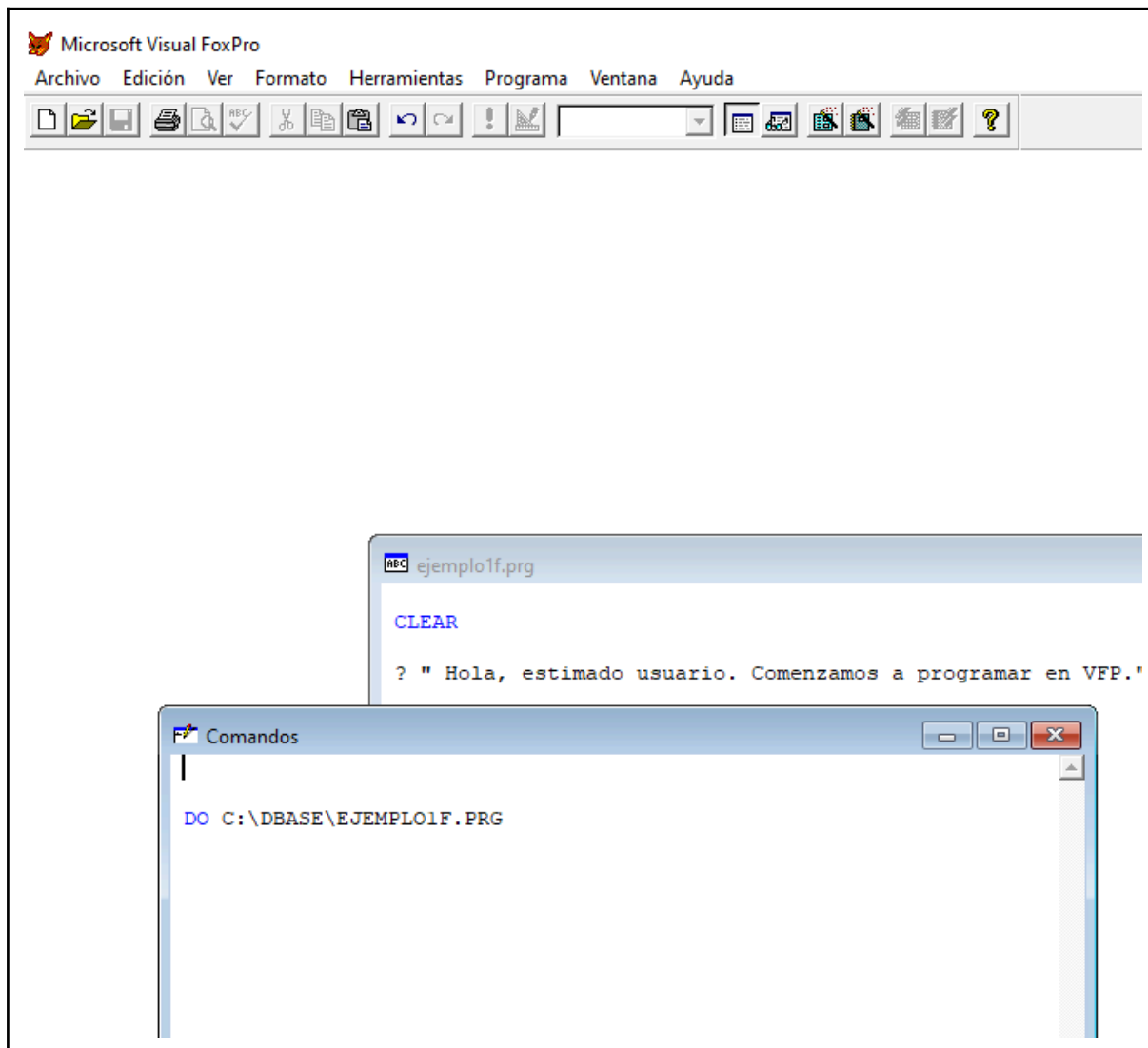
MODIFY COMMAND C:\DBASE\Ejemplo1f.prg



Ahora copiaremos el código del siguiente cuadro dentro de la ventana `ejemplo1f.prg`:



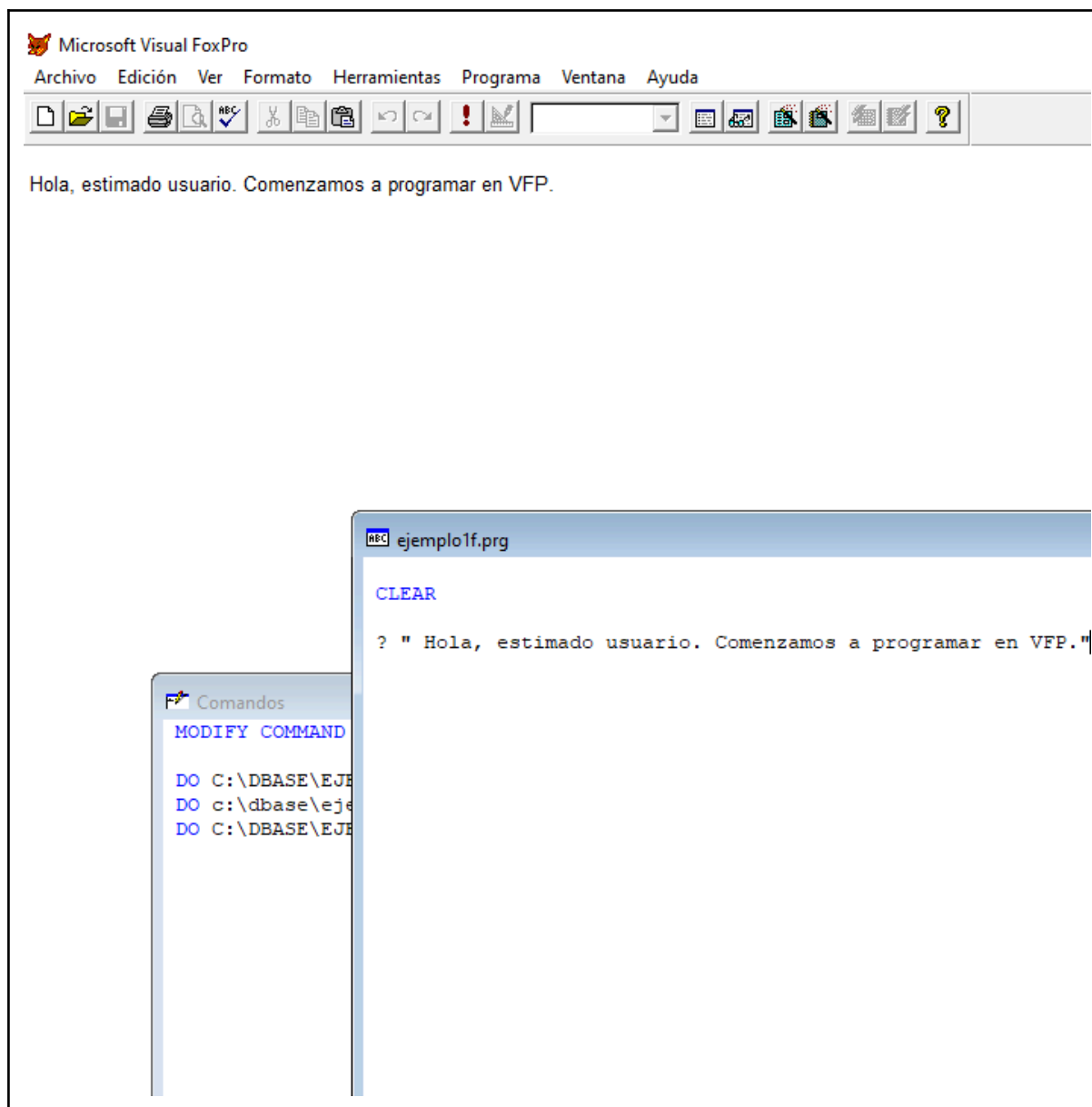
Ahora ejecutaremos el código del programa `ejemplo1f.prg`:



Para ejecutar nuestro programa, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo1f.prg
```

Al ejecutar el programa `ejemplo1f.prg`, veremos en la parte superior izquierda de la pantalla la frase: “Hola, estimado usuario. Comenzamos a programar en VFP.”



Existen 3 métodos principales para mostrar mensajes en la pantalla con VFP:

Método 1.

Impresión de mensajes sin utilizar el sistema de coordenadas de la pantalla:

```
*** Ejemplo1f.prg

CLEAR

? "Hola, estimado usuario. Comenzamos a programar en VFP."
```

Método 2.

Impresión de mensajes utilizando el sistema de coordenadas de la pantalla:

```
*** Ejemplo1f.prg

CLEAR
SET TALK OFF

Nombre = SPACE(30)

@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET Nombre
READ

@ 4, 5 SAY "Hola, " + TRIM(Nombre)

@ 6, 5 SAY "Comenzamos a programar en VFP."
```

El método 3, que utiliza la programación orientada a objetos, lo dejaremos para más adelante.

Ejemplo 2f2. Hola Mundo de Pablo.

Objetivo: Entender la estructura de un programa escrito en Microsoft Visual FoxPro mediante el desarrollo de un programa básico: “Hola Mundo de Pablo”.

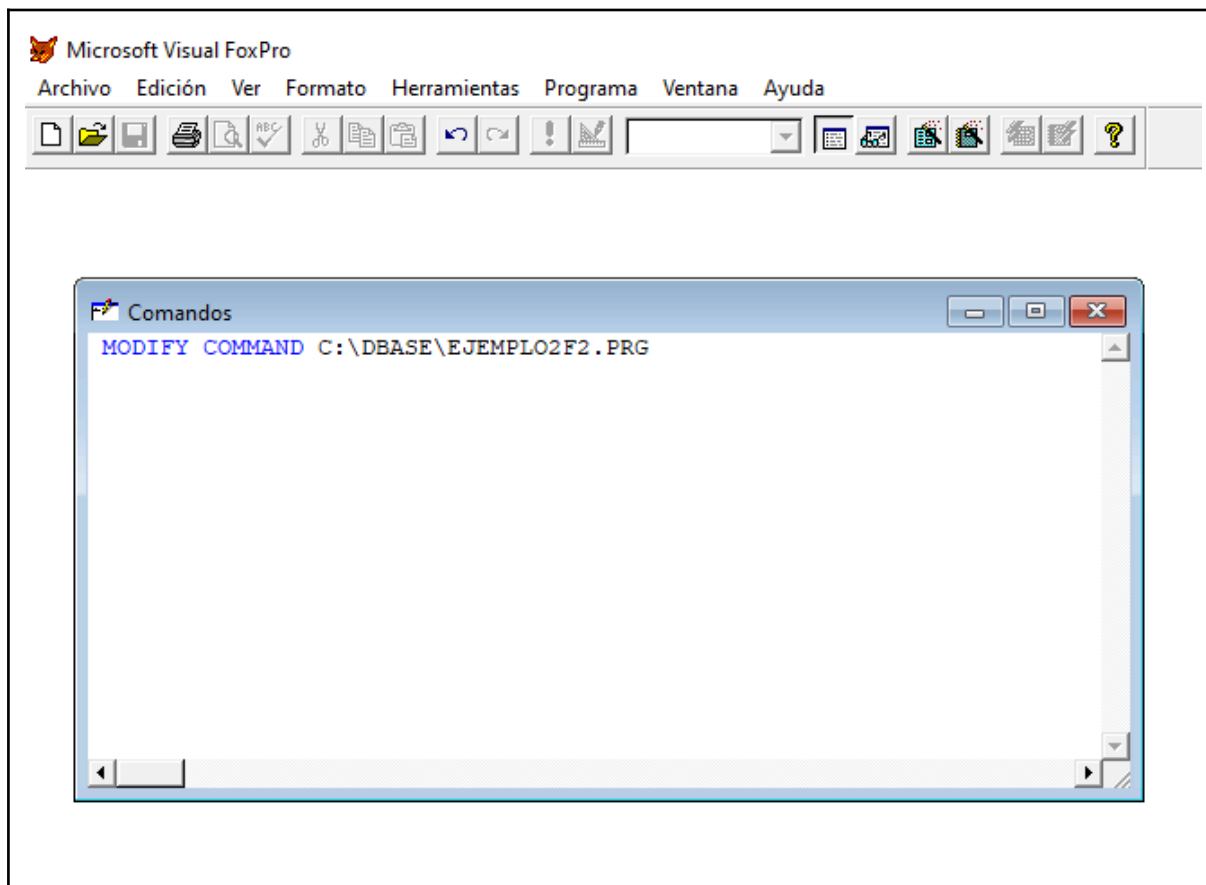
Para empezar, abra el Shell (el Shell es un intérprete interactivo de comandos).

Haga clic en:

Start > Programs > Visual FoxPro

Para crear un nuevo programa, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo2f2.prg



A continuación, copie el siguiente script dentro de la ventana **Ejemplo2f2.prg**:

```
* -- Ejemplo2f2.PRG -- Mensajes con Msft Visual FoxPro
LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer una prueba. Dime tu nombre, por favor:" GET wNombre
READ

@ 5, 5 SAY "Hola, " + TRIM(wNombre) + ". Espero que tengas un buen día!"

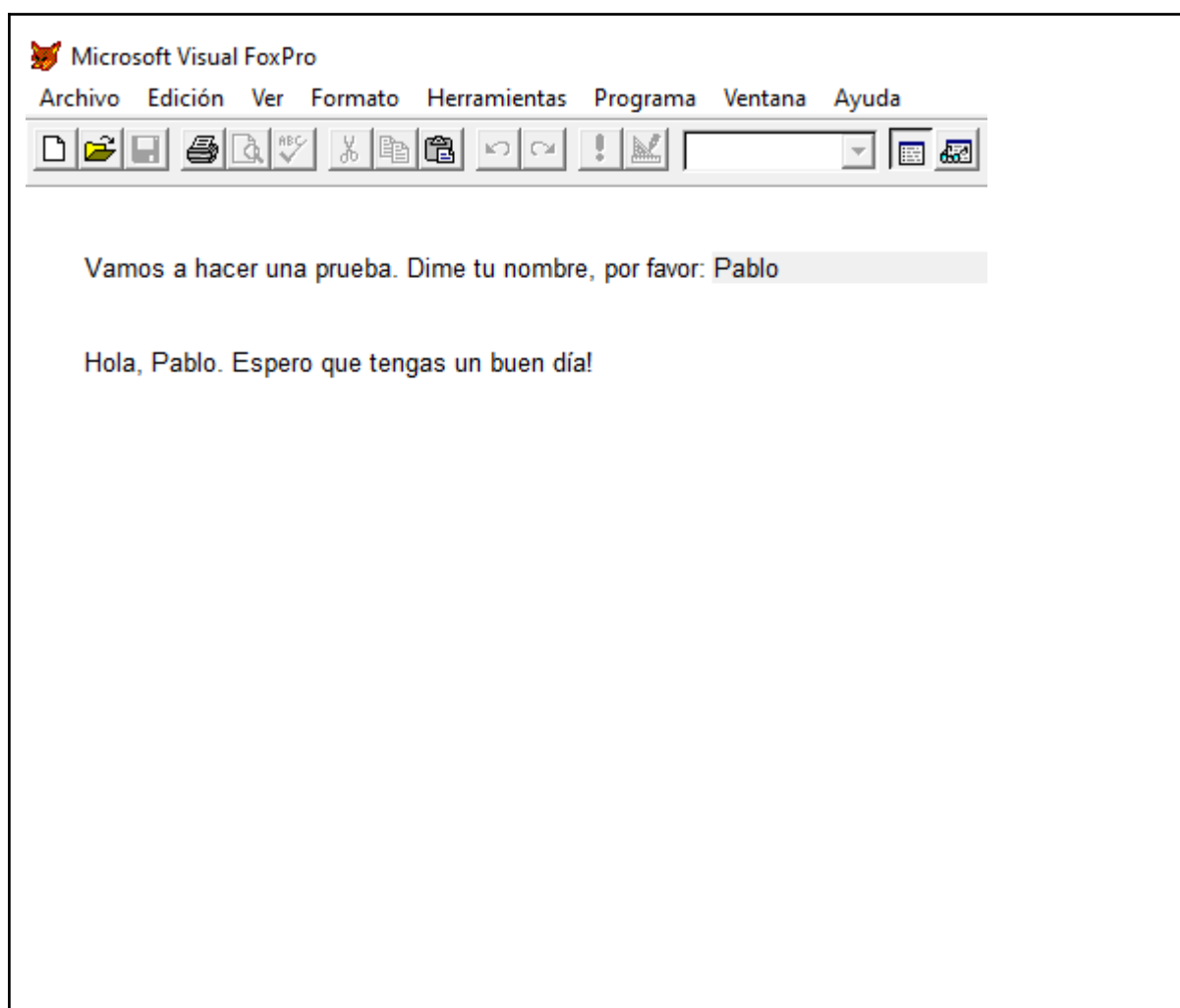
SET TALK ON
```

Continúa en la siguiente página. . .

Para ejecutar nuestro programa, teclearemos en la ventana “Comandos”, la siguiente instrucción:

DO C:\DBASE\Ejemplo2f2.prg

Después de ejecutar el programa **Ejemplo2f2.prg**, veremos la siguiente pantalla donde se mostrará el mensaje “Vamos a hacer una prueba”:



Ejemplo 2f. Hola Mundo.

Objetivo: Entender la estructura de un programa escrito en Microsoft Visual FoxPro mediante el desarrollo de un test básico con dos preguntas sobre Microsoft Windows.

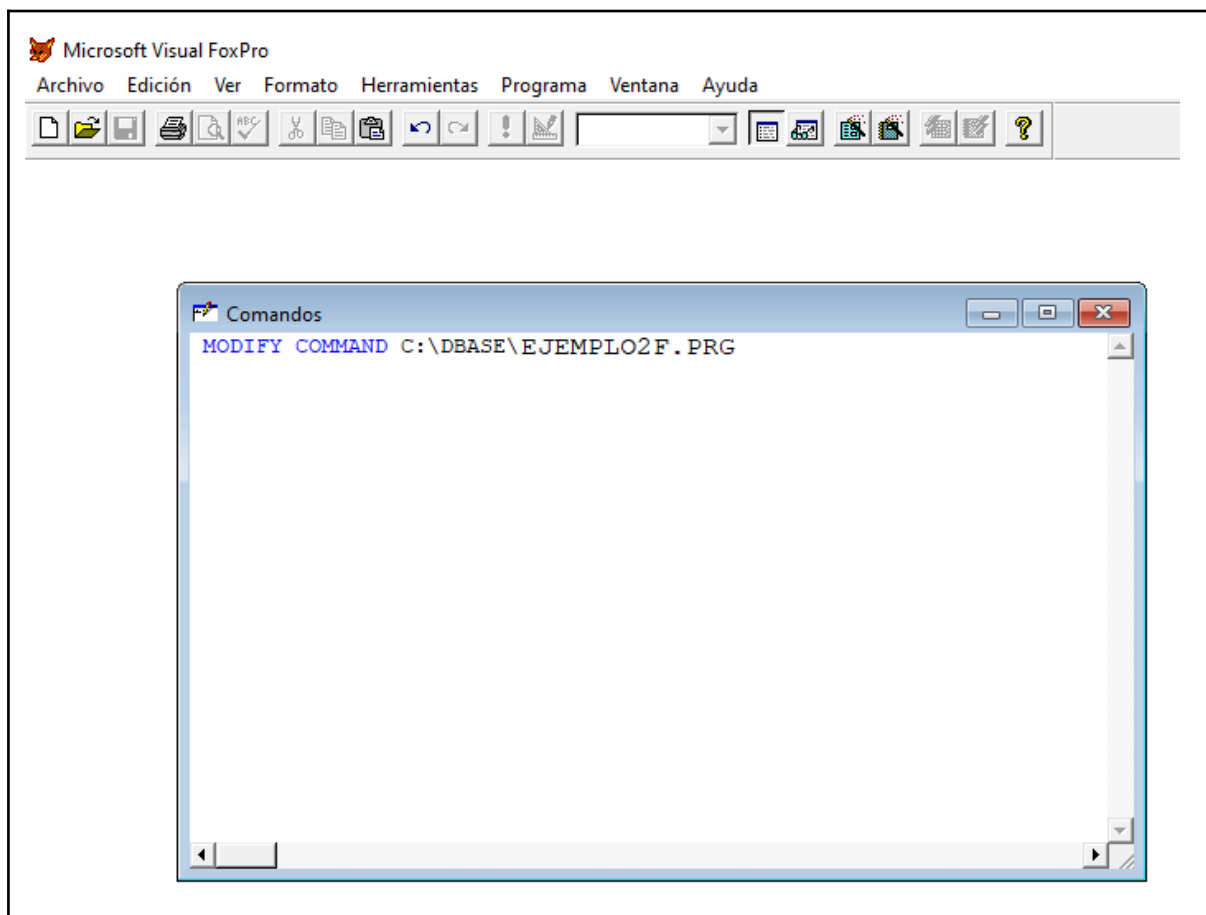
Para empezar, abra el Shell (el Shell es un intérprete interactivo de comandos).

Haga clic en:

Start > Programs > Visual FoxPro

Para crear un nuevo programa, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo2f.prg



A continuación, copie el siguiente script dentro de la ventana **Ejemplo2f.prg**:

```
* -- Ejemplo2f.PRG -- Vamos a hacer un test sobre Msft Windows y Msft Visual FoxPro

LOCAL wNombre, Preguntal, Pregunta2, lnPuntos
CLEAR
SET TALK OFF

* Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

* === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "Q1. El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF

* === PREGUNTA 2 ===
Pregunta2 = SPACE(1)
@ 8,5 SAY "Q2.El comando DIR sirve para renombrar archivos.(V/F)?:" GET Pregunta2 PICT
"@!"
READ
Pregunta2 = UPPER(TRIM(Pregunta2))

IF Pregunta2 = "F"
    @ 10, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 10, 5 SAY "Respuesta incorrecta."
ENDIF

@ 18, 5 SAY "Tu nota final en este test es: " + STR(lnPuntos, 1)

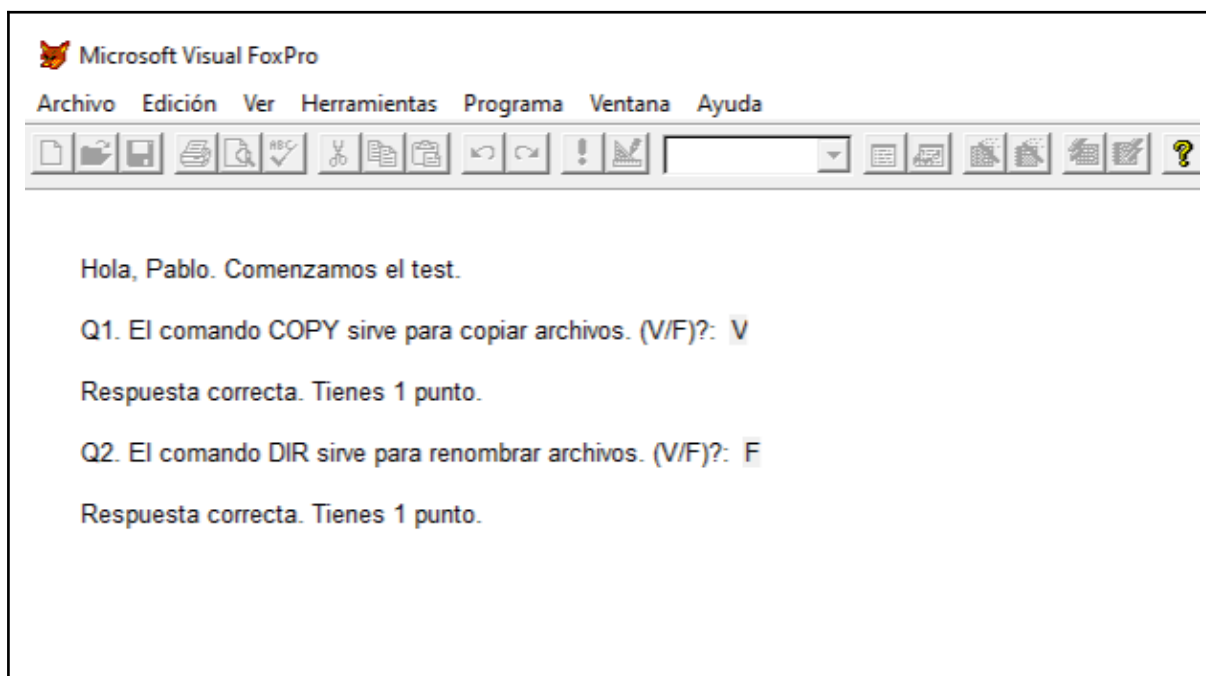
SET TALK ON
```

Continúa en la siguiente página. . .

Para ejecutar nuestro programa, teclearemos en la ventana “Comandos”, la siguiente instrucción:

DO C:\DBASE\Ejemplo2f.prg

Después de ejecutar el programa **Ejemplo2f.prg**, veremos la siguiente pantalla donde el usuario debe introducir su nombre y responder “V” o “F” a dos preguntas sobre Microsoft Windows:



Ejemplo 3f. Sumas y restas.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Para empezar, abra el Shell (el Shell es un intérprete interactivo de comandos).

Haga clic en:

Start > Programs> Visual FoxPro

Para crear un nuevo programa, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo3f.prg

A continuación, copie el siguiente script dentro de la ventana **Ejemplo3f.prg**:

```
* -- Ejemplo3f.PRG -- Vamos a hacer un ejemplo con sumas y restas

LOCAL a,b
CLEAR
SET TALK OFF

a=12

b=12

?a
?b

?a+b

SET TALK ON
```

Ejecute el programa y observe el resultado de la suma. Preguntas: ¿Qué tipo de variables cree que hemos utilizado? ¿Integer (entero) o String (cadena)? ¿Has ejecutado el programa? ¿Cuál ha sido el resultado?

Ejemplo 4f. Cálculo del área de un rectángulo.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Para empezar, abra el Shell (el Shell es un intérprete interactivo de comandos).

Haga clic en:

Start > Programs > Visual FoxPro

Para crear un nuevo programa, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo4f.prg

A continuación, copie el siguiente script dentro de la ventana **Ejemplo4f.prg**:

```
* -- Ejemplo4f.PRG -- Vamos a hacer un ejemplo con multiplicaciones.

LOCAL area,base,altura
base=10.0
altura=12.0
area=1.0

CLEAR
SET TALK OFF

@ 3,5 SAY "Vamos a calcular el área de un rectángulo."

@ 6,5 SAY "Entre en metros, cuál es la longitud de la base: " GET base
READ

@ 9,5 SAY "Entre en metros, cuál es la longitud de la ALTURA: " GET altura
READ

area= (base)*(altura)
@ 12,5 SAY "El área del rectángulo es: "
@ 13,5 SAY area PICTURE "@999.999"
@ 14,5 SAY "metros cuadrados."

SET TALK ON
```

Preguntas: ¿Has ejecutado el programa? ¿Qué tipo de variables crees que hemos utilizado? ¿Integer (entero) o String (cadena)?

Ejemplo 5f. Cálculo del área de un triángulo.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Para empezar, abra el Shell (el Shell es un intérprete interactivo de comandos).

Haga clic en:

Start > Programs > Visual FoxPro

Para crear un nuevo programa, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo5f.prg

A continuación, copie el siguiente script dentro de la ventana **Ejemplo5f.prg**:

```
* -- Ejemplo5f.PRG -- Vamos a hacer un ejemplo con un triángulo.

LOCAL area,base,altura
base=12.0
altura=2.0
area=1.0

CLEAR
SET TALK OFF

@ 3,5 SAY "Vamos a calcular el área de un triángulo."

@ 6,5 SAY "Entre en metros, cuál es la longitud de la base: " GET base
READ

@ 9,5 SAY "Entre en metros, cuál es la longitud de la ALTURA: " GET altura
READ

area= (base)*(altura)
@ 12,5 SAY "El área del triángulo es: "
@ 13,5 SAY area/2 PICTURE "@999.999"
@ 14,5 SAY "metros cuadrados."

SET TALK ON
```

Preguntas: ¿Qué tipo de variables crees que hemos utilizado? ¿Cuál es la principal diferencia que existe con el ejemplo de la página anterior? ¿Cómo modificaría el programa para calcular el área de un hexágono?

Ejemplo 6f. Creación de un programa que hace una pregunta a los alumnos de una clase de informática en una escuela de programación (creación de un test básico sobre Microsoft Windows).

Objetivo: Entender la sentencia **if** del lenguaje Microsoft Visual FoxPro.

Una sentencia **if** esencialmente dice:

"Si la expresión evaluada, resulta ser verdadera (True), entonces ejecuta una vez el código en la expresión."

Haga clic en:

Start > Programs> Visual FoxPro

Para crear un nuevo programa, escriba en la ventana "Comandos" la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo6f.prg

A continuación, copie el siguiente script dentro de la ventana **Ejemplo6f.prg**:

Ahora que ya tiene creado el archivo creado, añade las siguientes órdenes:

```
* -- Ejemplo6f.PRG -- Vamos a hacer un test sobre Msft Windows y Msft Visual FoxPro

LOCAL wNombre, Preguntal, lnPuntos
CLEAR
SET TALK OFF

* Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

* === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF
```

Preguntas: ¿Qué variables hemos utilizado para acumular puntos en nuestro test? ¿Cómo hemos comprobado que la respuesta es correcta? ¿Ya dominas la instrucción **GET**?

Fin del Ejemplo 6f.

Ejemplo 7f. Creación de un programa que hace dos preguntas a los alumnos de una clase de informática en una escuela de programación (creación de un test básico sobre Microsoft Windows).

Objetivo: Repasar la sentencia **if** del lenguaje Microsoft Visual FoxPro.

Haga clic en:

Start > Programs> Visual FoxPro

Para crear un nuevo programa, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo7f.prg

A continuación, copie el siguiente script dentro de la ventana **Ejemplo7f.prg**:

Ahora que ya tiene creado el archivo creado, añade las siguientes órdenes:

```

*** -- Ejemplo7f.PRG -- Vamos a hacer un test sobre Msft Windows y Msft VFP

LOCAL wNombre, Preguntal, Pregunta2, lnPuntos
CLEAR
SET TALK OFF

*   Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*   === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "Q1. El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF

*   === PREGUNTA 2 ===
Pregunta2 = SPACE(1)
@ 8,5 SAY "Q2.El comando DIR sirve para renombrar archivos.(V/F)?:" GET Pregunta2 PICT
"@!"
READ
Pregunta2 = UPPER(TRIM(Pregunta2))

IF Pregunta2 = "F"
    @ 10, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 10, 5 SAY "Respuesta incorrecta."
ENDIF

@ 16, 5 SAY "INFORME FINAL DE NOTAS: "
* --- Results ---
@ 18, 5 SAY "Tu nota final en este test es: " + STR(lnPuntos, 1)

pausa = SPACE(1)
@ 20, 5 SAY "Pulse <ENTER> para terminar... " GET pausa
READ

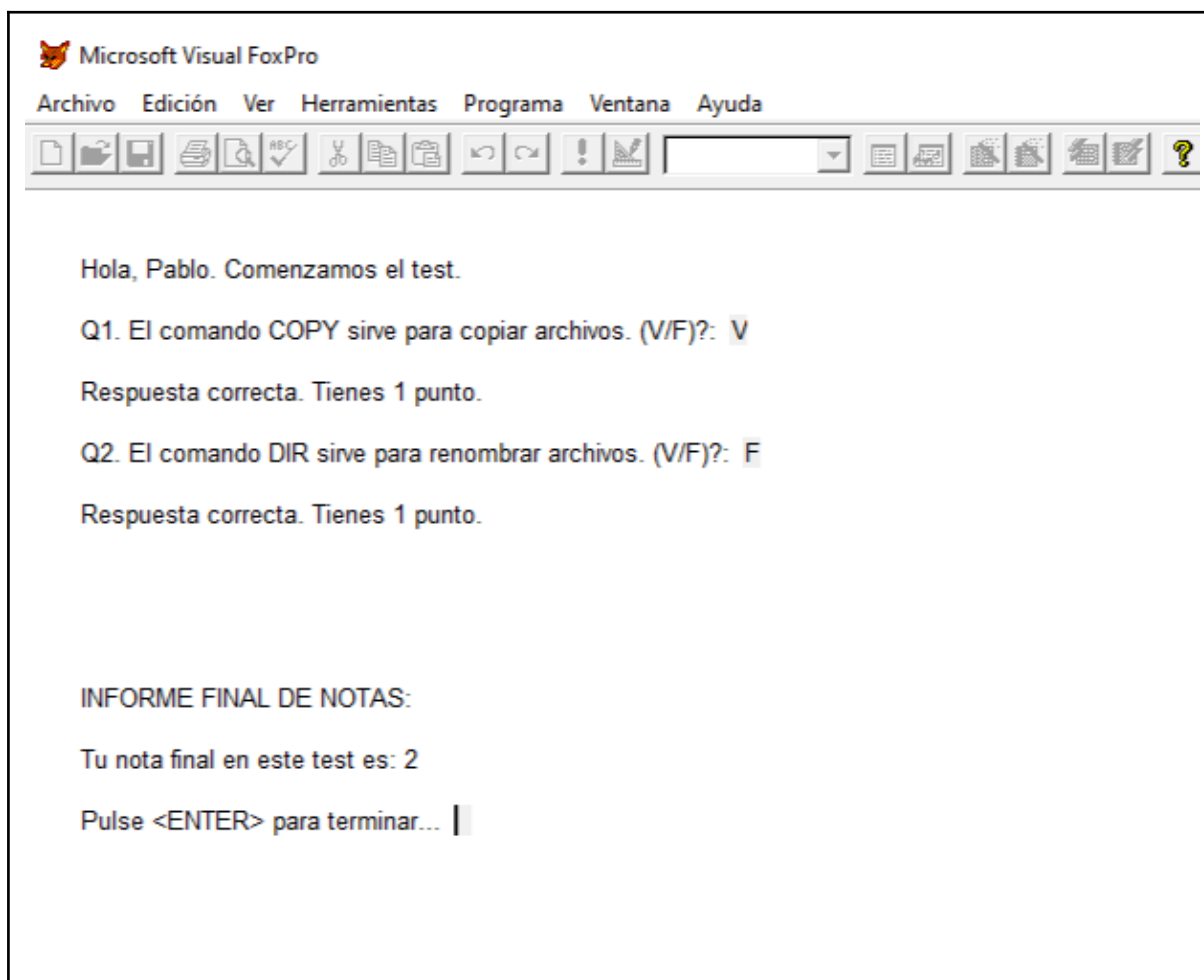
SET TALK ON

```


Para ejecutar nuestro programa, teclearemos en la ventana “Comandos”, la siguiente instrucción:

DO C:\DBASE\Ejemplo7f.prg

Después de ejecutar el programa **Ejemplo7f.prg**, veremos la siguiente pantalla donde el usuario debe introducir su nombre y responder “V” o “F” a dos preguntas sobre Microsoft Windows:



Ejemplo 8f. Suma de dos valores utilizando un procedimiento externo.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla implementando un **procedimiento** externo definido por el usuario.

Haga clic en:

Start > Programs> Visual FoxPro

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo8f.prg
```

A continuación, copie el siguiente script dentro de la ventana **Ejemplo8f.prg**:

```
***** Programa principal: Ejemplo8f.prg

CLEAR

DO C:\DBASE\SUMAR.PRG
```

Para crear el procedimiento externo llamado sumar.prg, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\sumar.prg
```

A continuación, copie el siguiente script dentro de la ventana **sumar.prg**

```
***** Procedimiento externo: sumar.prg
a = 10
b = 5
* Ahora sumamos los valores a,b para después imprimir el resultado
? a+b
```

Continúa. . .

Ahora que hemos terminado de copiar los dos archivos .prg, ejecutaremos el programa principal con la siguiente instrucción en la ventana “Comandos”:

```
DO C:\DBASE\Ejemplo8f.prg
```

Pregunta: ¿Cuántos archivos necesitamos terminar en este ejemplo 8f?

Dos archivos:

El **primer** archivo es el programa principal: `ejemplo8f.prg`

El **segundo** archivo es: `sumar.prg` (que es llamado por el programa principal).

Importante:

¿Cómo llamamos al procedimiento externo desde el programa principal?	DO sumar.prg (Procedimiento externo Simple)
Uso:	Ejecuta el código de forma secuencial.
¿Devuelve Valor?	No devuelve un valor como en otros lenguajes como Python o C#. Únicamente modifica variables que ya existen.
Acceso a Variables:	Accede directamente a variables declaradas dentro del archivo sumar.prg

Ejemplo 9f. Creación de un test básico sobre Microsoft Windows (implementando procedimientos externos definidos por el usuario).

Objetivo: Entender el funcionamiento de **procedimientos externos definidos por el usuario**. Entender cómo se define un procedimiento externo y cómo se llama a este procedimiento (externo).

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo9f.prg
```

A continuación, copie el siguiente script dentro de la ventana **Ejemplo9f.prg**:

```
***** Programa principal: Ejemplo9f.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

DO C:\DBASE\PREGUNTAS.PRG

SET TALK OFF
```

Continúa en la siguiente página. . .

Para crear el procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\PREGUNTAS.PRG

A continuación, copie el siguiente script dentro de la ventana **PREGUNTAS.prg**:

```
***** PREGUNTAS.PRG (PROCEDIMIENTO EXTERNO QUE CONTIENE LAS dos PREGUNTAS)

* Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

* === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "Q1. El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF

* === PREGUNTA 2 ===
Pregunta2 = SPACE(1)
@ 8,5 SAY "Q2.El comando DIR sirve para renombrar archivos.(V/F)?:" GET Pregunta2
READ
Pregunta2 = UPPER(TRIM(Pregunta2))

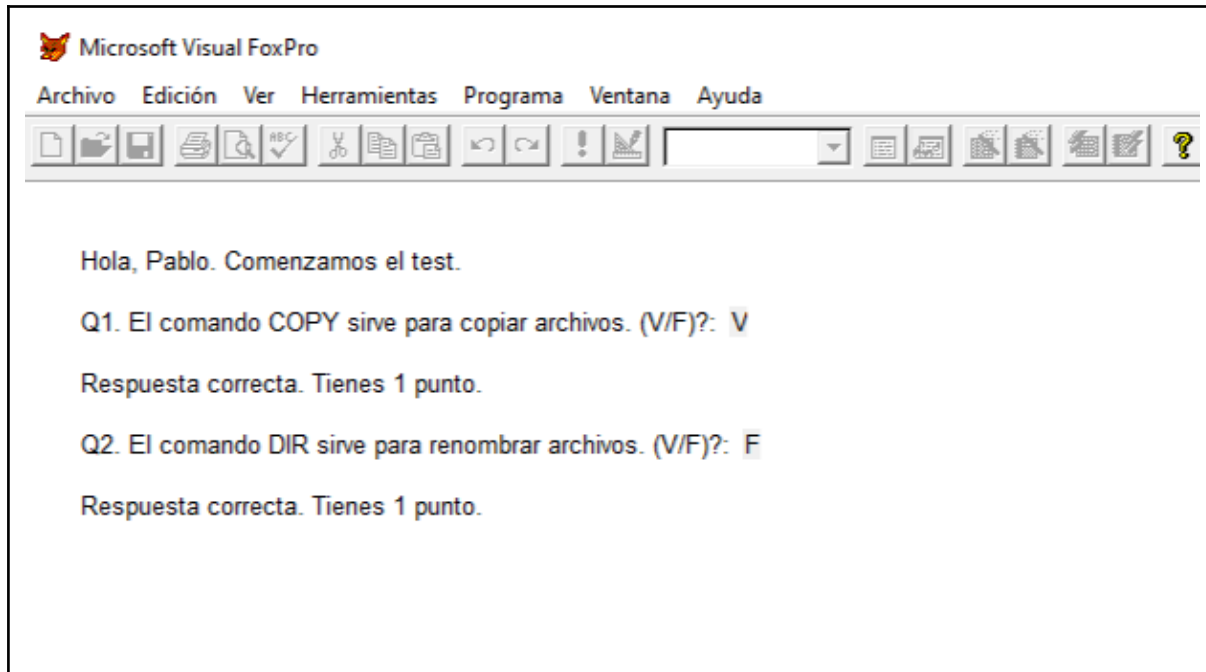
IF Pregunta2 = "F"
    @ 10, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 10, 5 SAY "Respuesta incorrecta."
ENDIF
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

DO C:\DBASE\Ejemplo9f.prg

Continúa en la siguiente página. . .

Después de ejecutar el programa principal, veremos la siguiente pantalla donde el usuario debe introducir su nombre y responder “V” o “F” a dos preguntas sobre Microsoft Windows:



Reto: ¿Cuántos archivos .prg necesitamos crear en este ejemplo 9f? ¿Qué instrucciones son necesarias incluir en la aplicación para que se muestre el total de puntos obtenidos por el usuario?

Ejemplo 10f. Creación de un test básico sobre Microsoft Windows (implementando dos procedimientos externos definidos por el usuario).

Objetivo: Entender el funcionamiento de **procedimientos externos definidos por el usuario**. Entender cómo se definen los procedimientos externos y cómo se llaman a estos procedimientos (externos).

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo10f.prg
```

A continuación, copie el siguiente script dentro de la ventana **Ejemplo10f.prg**:

```
***** Programa principal: Ejemplo10f.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a dos procedimientos externos:

DO C:\DBASE\PREGUNTAS.PRG

DO C:\DBASE\FELICITACION.PRG
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\PREGUNTAS.PRG

A continuación, copie el siguiente script dentro de la ventana **PREGUNTAS.prg**:

```
***** PREGUNTAS.PRG (PROCEDIMIENTO EXTERNO QUE CONTIENE LAS PREGUNTAS)

* Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

* === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "Q1. El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF

* === PREGUNTA 2 ===
Pregunta2 = SPACE(1)
@ 8, 5 SAY "Q2. El comando DIR sirve para renombrar archivos. (V/F)?:" GET Pregunta2
PICT "@!"
READ
Pregunta2 = UPPER(TRIM(Pregunta2))

IF Pregunta2 = "F"
    @ 10, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 10, 5 SAY "Respuesta incorrecta."
ENDIF
```


Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACION.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACION.prg**:

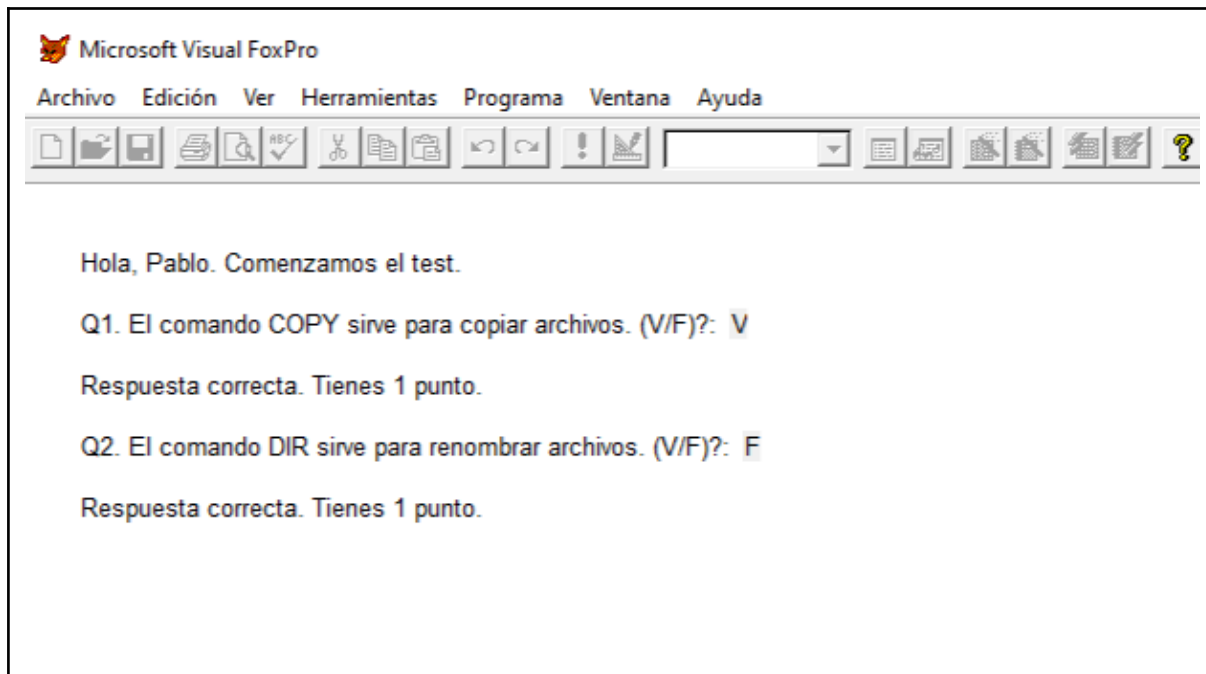
```
***** FELICITACION.PRG  
  
@ 14, 5 SAY "Felicitaciones, siga estudiando así."  
@ 16, 5 SAY "Estamos orgullosos de usted."  
  
@ 18, 5 SAY "Fin de la prueba."
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo10f.prg
```

Continúa en la siguiente página. . .

Después de ejecutar el programa principal, veremos la siguiente pantalla donde el usuario debe introducir su nombre y responder “V” o “F” a dos preguntas sobre Microsoft Windows:



El hecho de dividir los programas en diferentes archivos .prg crea, por lo general, un proyecto con un número total de líneas menor. Esto implica hacer una aplicación más limpia y legible. Si necesitáramos hacer modificaciones a nuestro programa, sería más fácil identificar la parte que necesita ser modificada.

Reto: ¿Qué instrucciones son necesarias incluir en la aplicación para que se muestre el total de puntos obtenidos por el usuario?

Ejemplo 11f. Creación de un test sobre Microsoft Windows (implementando un procedimiento externo con parámetros).

Objetivo: Entender el funcionamiento de **un procedimiento** externo definido por el usuario. Llamada al procedimiento con parámetros.

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo11f.prg
```

A continuación, copie el siguiente script dentro de la ventana **Ejemplo11f.prg**:

```
***** Programa principal: Ejemplo11f.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a los procedimientos externos:

DO C:\DBASE\PREGUNTAS.PRG

DO C:\DBASE\FELICITACION.PRG WITH wNombre
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\PREGUNTAS.PRG

A continuación, copie el siguiente script dentro de la ventana **PREGUNTAS.prg**:

```
***** PREGUNTAS.PRG (PROCEDIMIENTO EXTERNO QUE CONTIENE LAS PREGUNTAS)

* Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

* === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "Q1. El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF

* === PREGUNTA 2 ===
Pregunta2 = SPACE(1)
@ 8, 5 SAY "Q2. El comando DIR sirve para renombrar archivos. (V/F)?:" GET Pregunta2
PICT "@!"
READ
Pregunta2 = UPPER(TRIM(Pregunta2))

IF Pregunta2 = "F"
    @ 10, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
ELSE
    @ 10, 5 SAY "Respuesta incorrecta."
ENDIF
```

Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACION.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACION.prg**:

```
***** FELICITACION.PRG

PARAMETERS NN

***** El nombre del usuario se recogió como parámetro cuando el programa
principal llamó a este procedimiento utilizando la variable wNombre

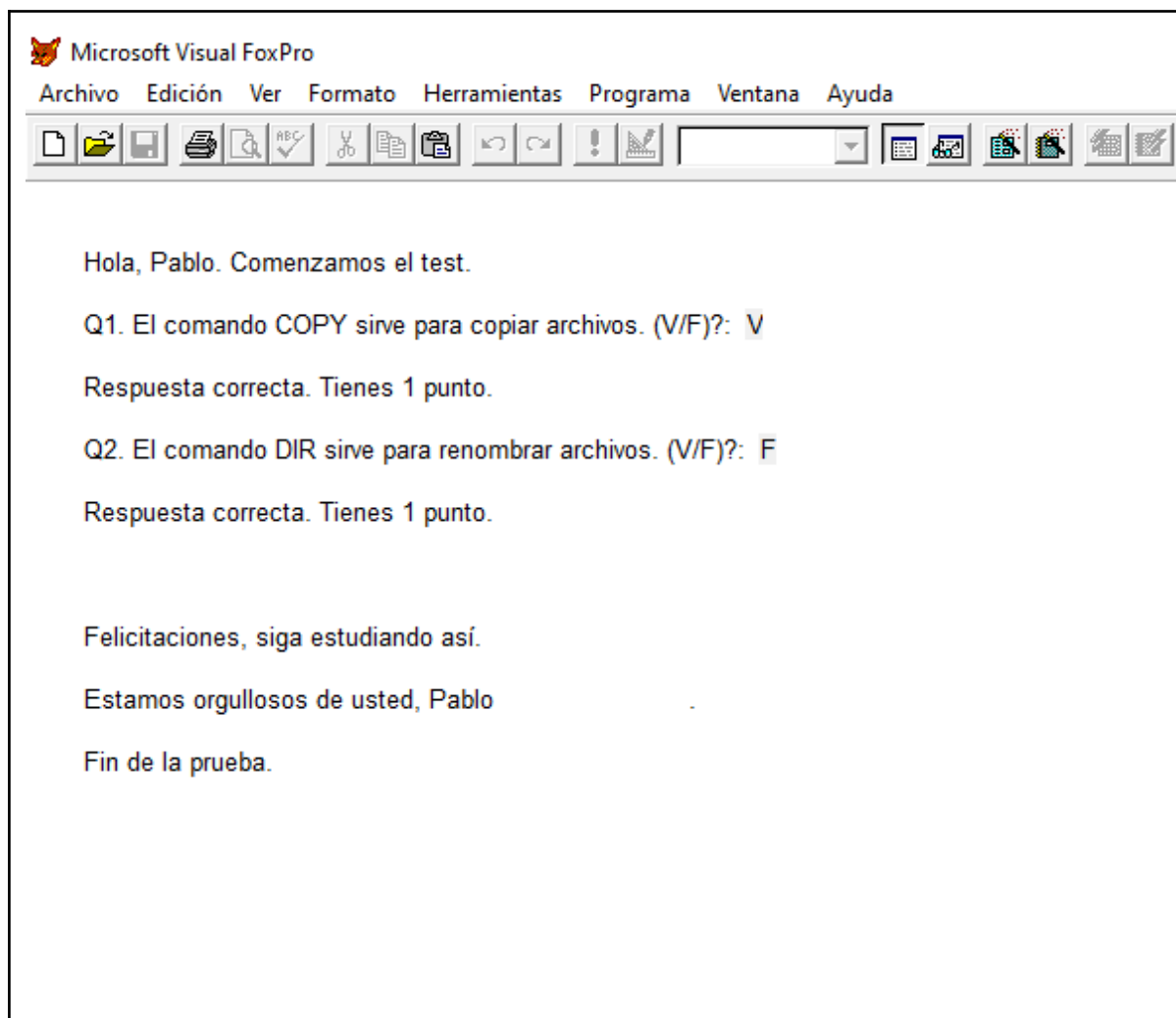
@ 14, 5 SAY "Felicitaciones, siga estudiando así."
@ 16, 5 SAY "Estamos orgullosos de usted, " + NN + "."
@ 18, 5 SAY "Fin de la prueba."
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo11f.prg
```

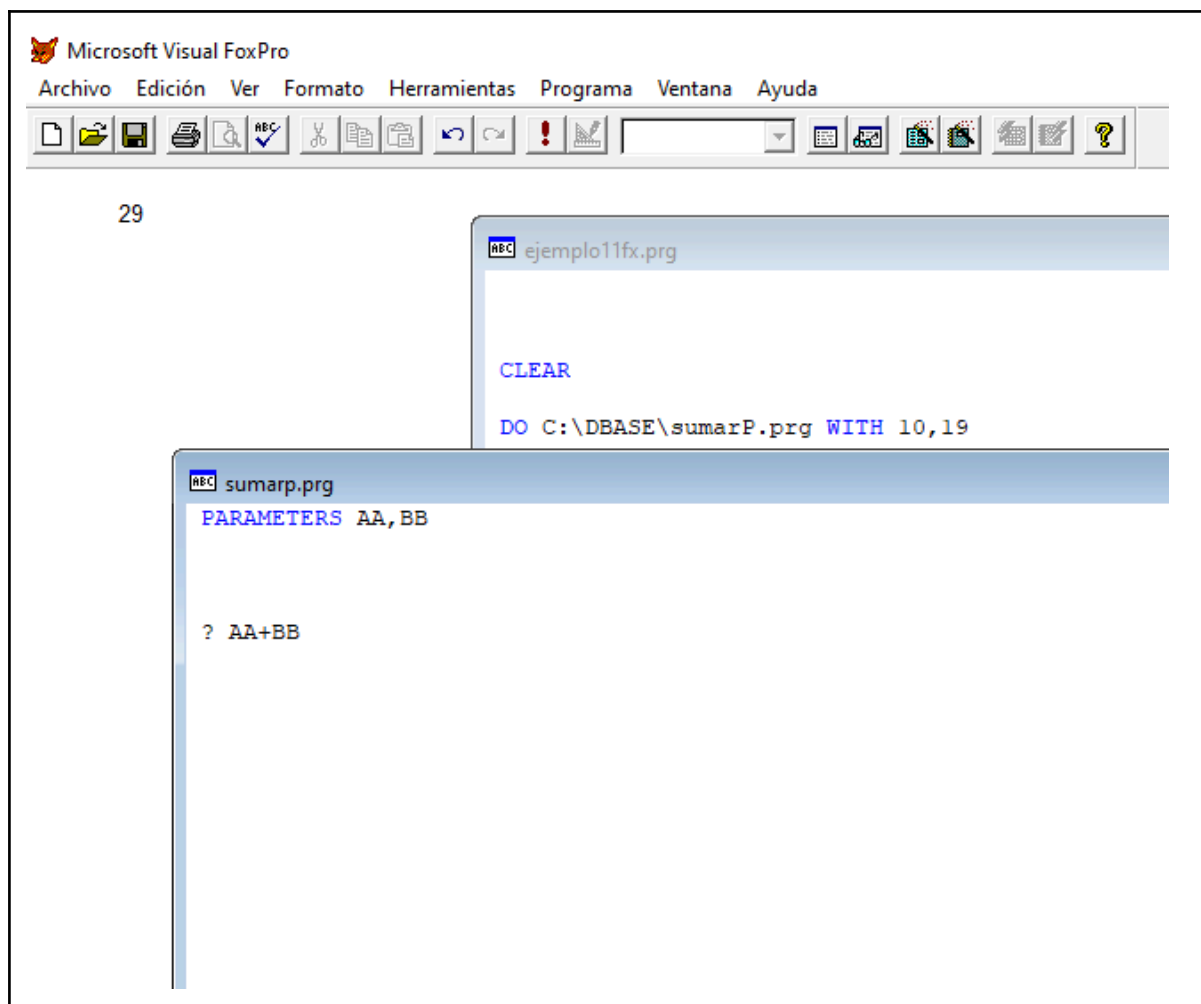
Continúa en la siguiente página. . .

Después de ejecutar el programa principal, veremos la siguiente pantalla donde el usuario debe introducir su nombre y responder “V” o “F” a dos preguntas sobre Microsoft Windows:



Reto:

Identifique en el siguiente cuadro el programa principal y el procedimiento externo:



Cree el procedimiento externo basándose en el cuadro superior. Cree el programa principal, **ejemplo11fx.prg**. Pruebe a cambiar los sumandos para comprobar que todo funciona correctamente.

Ejemplo 12f. Creación de un test sobre Microsoft Windows (implementando un procedimiento externo con parámetros).

Objetivo: Entender el funcionamiento de **un procedimiento** externo definido por el usuario. Llamada al procedimiento con parámetros.

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo12f.prg
```

A continuación, copie el siguiente script dentro de la ventana **Ejemplo12f.prg**:

```
***** Programa principal: Ejemplo12f.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a los procedimientos externos:

DO C:\DBASE\PREGUNTASX.PRG

DO C:\DBASE\FELICITACIONX.PRG WITH wNombre
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\PREGUNTASX.PRG

A continuación, copie el siguiente texto dentro de la ventana **PREGUNTASX.prg**:

```
***** PREGUNTASX.PRG (PROCEDIMIENTO EXTERNO QUE CONTIENE LAS PREGUNTAS y graba
*** LOS PUNTOS OBTENIDOS EN UN FICHEROP.DBF EXTERNO)

SELECT 1
USE C:\DBASE\FICHEROP.DBF
GO TOP
REPLACE PUNTOS WITH 0

* Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

* === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "Q1. El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
    GO TOP
    REPLACE PUNTOS WITH lnPuntos
    *** PUNTOS es un campo numérico de la tabla FICHEROP.DBF para almacenar los
    *** puntos obtenidos en cada pregunta
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF

* === PREGUNTA 2 ===
Pregunta2 = SPACE(1)
@ 8, 5 SAY "Q2. El comando DIR sirve para renombrar archivos. (V/F)?:" GET Pregunta2
PICT "@"
READ
Pregunta2 = UPPER(TRIM(Pregunta2))

IF Pregunta2 = "F"
    @ 10, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
    GO TOP
    REPLACE PUNTOS WITH lnPuntos
ELSE
    @ 10, 5 SAY "Respuesta incorrecta."
ENDIF
```

Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACIONX.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACIONX.prg**:

```
***** FELICITACIONX.PRG

PARAMETERS NN

@ 14, 5 SAY "Felicitaciones, siga estudiando así."
@ 16, 5 SAY "Estamos orgullosos de usted, " + NN + "."

SELECT 1
USE C:\DBASE\FICHEROP.DBF
GO TOP
@ 18, 5 SAY "Puntuación total: "+STR(PUNTOS,2)

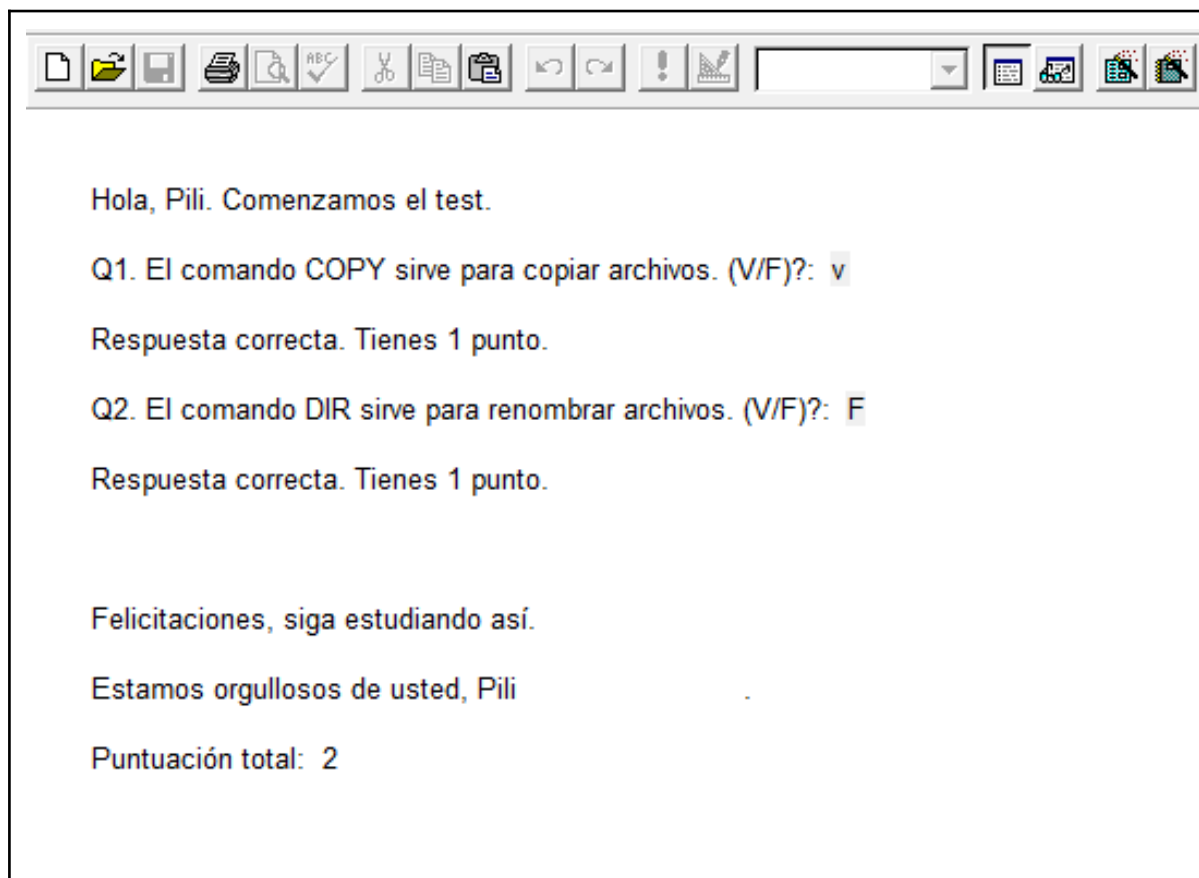
*** RECUERDE QUE PUNTOS es el campo numérico de la tabla FICHEROP.DBF donde
*** habíamos almacenado los puntos obtenidos en cada pregunta
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo12f.prg
```

Continúa en la siguiente página. . .

Después de ejecutar el programa principal, veremos la siguiente pantalla donde el usuario debe introducir su nombre y responder “V” o “F” a las preguntas sobre Microsoft Windows:



Importante:

Para que se registren los puntos en el archivo **FICHEROP.dbf** externo en la carpeta **C:\DBASE**, debe crearlo primero (antes de ejecutar el programa principal) con la siguiente instrucción:

```
CREATE TABLE C:\DBASE\FICHEROP.dbf (puntos N(2,0))
```

IMPORTANTE: **PUNTOS** es el campo numérico (de dos dígitos y cero decimales) de la tabla FICHEROP.DBF donde habíamos almacenado los puntos obtenidos en cada pregunta

Ejemplo 13f. Creación de un test sobre Microsoft Windows donde las preguntas se almacenan en un archivo externo.

Objetivo: Entender el funcionamiento de **un procedimiento** externo definido por el usuario. Llamada al procedimiento con parámetros.

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo13f.prg
```

A continuación, copie el siguiente script dentro de la ventana **Ejemplo13f.prg**:

```
***** Programa principal: Ejemplo13f.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a hacer un test. Dime tu nombre, por favor:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a los procedimientos externos:

DO C:/DBASE/PREGUNTASX.PRG

DO C:/DBASE/FELICITACIONX.PRG WITH wNombre
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\PREGUNTASX.PRG

A continuación, copie el siguiente texto dentro de la ventana **PREGUNTASX.prg**:

```
***** PREGUNTASX.PRG (PROCEDIMIENTO EXTERNO QUE CONTIENE LAS PREGUNTAS y graba
*** LOS PUNTOS OBTENIDOS EN UN FICHEROP.DBF EXTERNO)

SELECT 1
USE C:\DBASE\FICHEROP.DBF
GO TOP
REPLACE PUNTOS WITH 0

* Declarar e inicializar la variable local de puntos como numérica
lnPuntos = 0

* === PREGUNTA 1 ===
Preguntal = SPACE(1)
@ 4, 5 SAY "Q1. El comando COPY sirve para copiar archivos. (V/F)?:" GET Preguntal
READ
Preguntal = UPPER(TRIM(Preguntal))

IF Preguntal = "V"
    @ 6, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
    GO TOP
    REPLACE PUNTOS WITH lnPuntos
    *** PUNTOS es un campo numérico de la tabla FICHEROP.DBF para almacenar los
    *** puntos obtenidos en cada pregunta
ELSE
    @ 6, 5 SAY "Respuesta incorrecta."
ENDIF

* === PREGUNTA 2 ===
SELECT 2
USE C:\DBASE\PREGUNTA2.dbf
GO TOP
Pregunta2 = SPACE(1)
@ 8, 5 SAY "Q2. " + LINEA + ". (V/F)?:" GET Pregunta2
READ

SELECT 1
Pregunta2 = UPPER(TRIM(Pregunta2))
IF Pregunta2 = "F"
    @ 10, 5 SAY "Respuesta correcta. Tienes 1 punto."
    lnPuntos = lnPuntos + 1
    GO TOP
    REPLACE PUNTOS WITH lnPuntos
ELSE
    @ 10, 5 SAY "Respuesta incorrecta."
ENDIF
```

Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACIONX.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACIONX.prg**:

```
***** FELICITACIONX.PRG

PARAMETERS NN

@ 14, 5 SAY "Felicitaciones, siga estudiando así."
@ 16, 5 SAY "Estamos orgullosos de usted, " + NN + "."

SELECT 1
USE C:\DBASE\FICHEROP.DBF
GO TOP
@ 18, 5 SAY "Puntuación total: "+STR(PUNTOS,2)

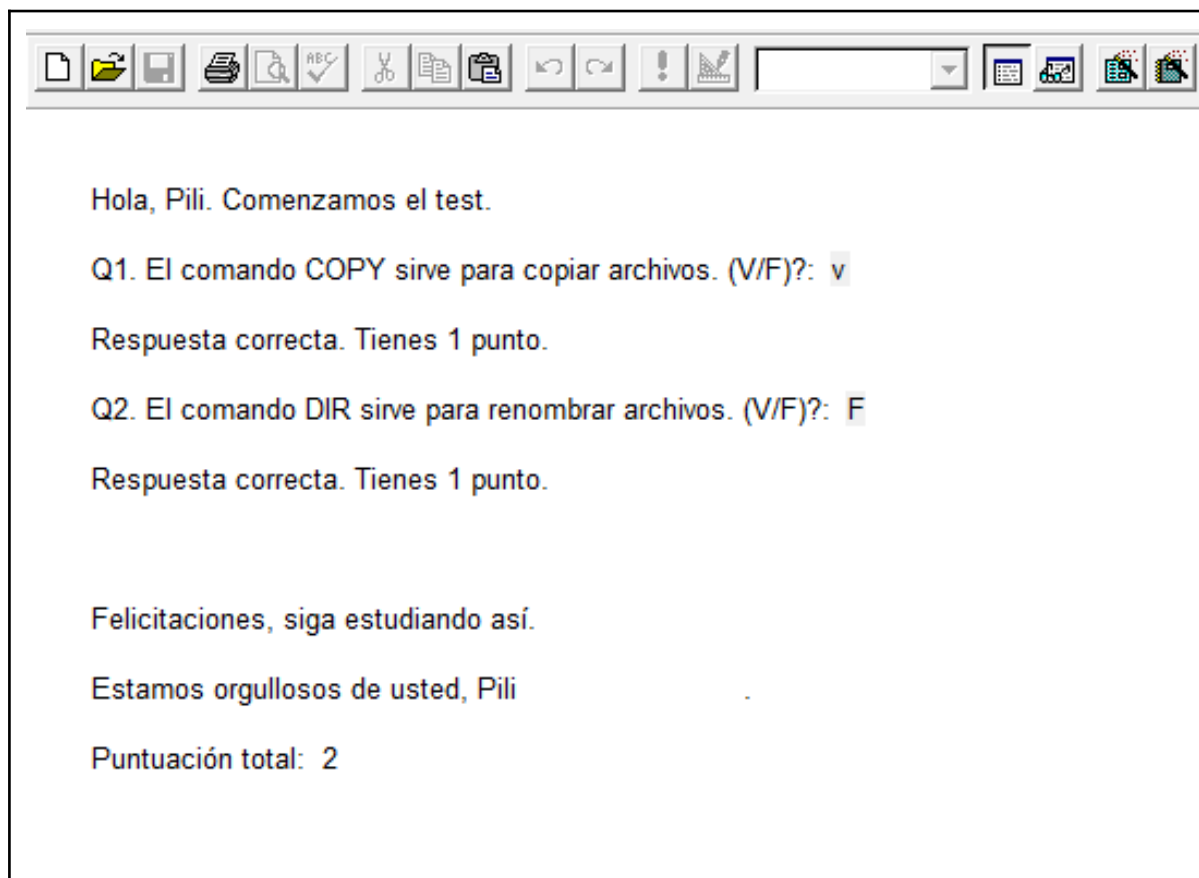
*** RECUERDE QUE PUNTOS es el campo numérico de la tabla FICHEROP.DBF donde
*** habíamos almacenado los puntos obtenidos en cada pregunta
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo13f.prg
```

Continúa en la siguiente página. . .

Después de ejecutar el programa principal, veremos la siguiente pantalla donde el usuario debe introducir su nombre y responder “V” o “F” a las preguntas sobre Microsoft Windows:



Importante:

Para que se registren los puntos en el archivo **FICHEROP.dbf** externo en la carpeta **C:\DBASE**, debe crearlo primero (antes de ejecutar el programa principal) con la siguiente instrucción:

```
CREATE TABLE C:\DBASE\FICHEROP.dbf (puntos N(2,0))
```

IMPORTANTE: **PUNTOS** es el campo numérico (de dos dígitos y cero decimales) de la tabla FICHEROP.DBF donde habíamos almacenado los puntos obtenidos en cada pregunta

La pregunta número 2 estará almacenada en un archivo externo llamado **PREGUNTA2.dbf**, que solo contiene el campo **linea** (de tipo Carácter con una longitud de 79 caracteres).

Esta tabla (archivo externo que contiene las preguntas) debe crearse con las siguientes instrucciones antes de ejecutar el programa principal: (No olvide pulsar <Enter> al final de cada línea)

```
CREATE TABLE C:\DBASE\PREGUNTA2.dbf (linea C(79))
```

```
USE C:\DBASE\PREGUNTA2.dbf
```

```
APPEND BLANK
```

```
REPLACE linea WITH "¿El comando DIR sirve para mover archivos en MS-DOS?"
```


Reto 1:

Analice el siguiente programa, identifique los archivos de datos (junto con sus estructuras necesarias), busque los errores y añada las preguntas 1 a 5 (junto a sus respuestas -en la página siguiente-) en los ficheros .dbf para que funcione correctamente.

```
* xxxPREGUNTASXXX.PRG - Optimizado con bucle
CLEAR
@ 2,5 SAY "Vamos a hacer un test de 5 preguntas:"
LOCAL lnPuntos, lnI, lcFile, lcPregunta, lcRespuestaUsuario, lcRightAnswer

lnPuntos = 0

* Bucle que busca las preguntas en 5 ficheros, cada pregunta en un fichero
FOR lnI = 1 TO 5
    lcFile = "C:\DBASE\PREGUNTA" + ALLTRIM(STR(lnI)) + ".DBF"

    IF FILE(lcFile)
        SELECT 9
        USE (lcFile)
        GO TOP

        * Obtenemos la info del fichero (DBF fields: LINEA, RESPUESTA)
        lcRightAnswer = ALLTRIM(UPPER(RESPUESTA))
        lcPregunta = ALLTRIM(LINEA)
        lcRespuestaUsuario = SPACE(1)

        * Mostramos la pregunta
        @ (lnI * 4), 5 SAY "Q" + ALLTRIM(STR(lnI)) + ". " + lcPregunta + " (V/F)?: "
        @ (lnI * 4), 77 GET lcRespuestaUsuario
        READ

        * Validamos respuesta
        IF UPPER(TRIM(lcRespuestaUsuario)) == lcRightAnswer
            @ (lnI * 4) + 2, 5 SAY "Respuesta correcta."
            lnPuntos = lnPuntos + 1
        ELSE
            @ (lnI * 4) + 2, 5 SAY "Respuesta incorrecta."
        ENDIF

        USE && Cerramos fichero en uso
    ENDIF
ENDFOR

*** Las preguntas están en la página siguiente. .
```

Preguntas y respuestas (necesarias para completar el reto 1 de la página anterior)

Campos del archivo PREGUNTA1.dbf

LINEA

El protocolo de red TCP asegura la entrega de datos mediante acuses de recibo.

Respuesta

V

Campos del archivo PREGUNTA2.dbf

LINEA

El protocolo de red TCP está diseñado para entregar los datos en el orden correcto.

Respuesta

V

Campos del archivo PREGUNTA3.dbf

LINEA

El protocolo de red TCP asegura una entrega de datos fiable.

Respuesta

V

Campos del archivo PREGUNTA4.dbf

LINEA

Utilizar una VPN (Virtual Private Network) protege la información contra posibles rastreos.

Respuesta

V

Campos del archivo PREGUNTA5.dbf

LINEA

Utilizar una VPN puede aumentar la seguridad de nuestra red.

Respuesta

V

Reto 2:

Analice el siguiente programa, identifique los archivos de datos (junto con sus estructuras necesarias), busque errores y añada las preguntas 1 a 5 (junto a sus respuestas) en los ficheros .dbf para que funcione correctamente.

```
* xxxPREGUNTASXXX.PRG - Optimizado con bucle
CLEAR
@ 2,5 SAY "Vamos a hacer un test de 5 preguntas:"
LOCAL lnPuntos, lnI, lcFile, lcPregunta, lcRespuestaUsuario, lcRightAnswer

* 1. Setup FICHEROP.DBF. Es la tabla para la puntuación, score
SELECT 1
USE C:\DBASE\FICHEROP.DBF
REPLACE PUNTOS WITH 0
lnPuntos = 0

* 2. Bucle que busca las preguntas en 5 ficheros, cada pregunta en un fichero
FOR lnI = 1 TO 5
    lcFile = "C:\DBASE\PREGUNTA" + ALLTRIM(STR(lnI)) + ".DBF"

    IF FILE(lcFile)
        SELECT 9
        USE (lcFile)
        GO TOP

        * Obtenemos la info del fichero (DBF fields: LINEA, RESPUESTA)
        lcRightAnswer = ALLTRIM(UPPER(RESPUESTA))
        lcPregunta = ALLTRIM(LINEA)
        lcRespuestaUsuario = SPACE(1)

        * Mostramos la pregunta
        @ (lnI * 4), 5 SAY "Q" + ALLTRIM(STR(lnI)) + ". " + lcPregunta + " (V/F)?:"
        @ (lnI * 4), 77 GET lcRespuestaUsuario
        READ

        * Validamos respuesta
        IF UPPER(TRIM(lcRespuestaUsuario)) == lcRightAnswer
            @ (lnI * 4) + 2, 5 SAY "Respuesta correcta."
            lnPuntos = lnPuntos + 1
        ELSE
            @ (lnI * 4) + 2, 5 SAY "Respuesta incorrecta."
        ENDIF

        USE && Cerramos fichero en uso
    ENDIF
ENDFOR

* 3. Grabamos puntuación en FICHEROP.DBF
SELECT 1
GO TOP
REPLACE PUNTOS WITH lnPuntos
lcMessage="Puntos obtenidos: "+ALLTRIM(STR(lnPuntos))
MESSAGEBOX(lcMessage, 64, "Resultado Final")

*** Las preguntas están en la página siguiente. .
.
```

Preguntas y respuestas (necesarias para completar el reto 2 de la página anterior)

Campos del archivo PREGUNTA1.dbf

LINEA

HTTP es el nombre del protocolo en el que se basa la comunicación en Internet.

Respuesta

V

Campos del archivo PREGUNTA2.dbf

LINEA

Todos los dispositivos en una red deben tener una dirección de hardware única.

Respuesta

V

Campos del archivo PREGUNTA3.dbf

LINEA

Llamamos SSID al nombre de una red WIFI.

Respuesta

V

Campos del archivo PREGUNTA4.dbf

LINEA

La clave de una red WIFI debería contener entre 12 y 63 caracteres para hacerla más segura.

Respuesta

V

Campos del archivo PREGUNTA5.dbf

LINEA

No utilizar una VPN (Virtual Private Network) puede aumentar la seguridad de nuestra red.

Respuesta

F

Reto 3:

Analice el siguiente programa, identifique los archivos de datos, busque errores y añada 10 preguntas (en el anexo solo encontrará 5). También necesitará modificar el código para leer las 10 preguntas y respuestas.

```
* xxxPREGUNTASXXX.PRG - Optimizado con bucle
CLEAR
@ 2,5 SAY "Vamos a hacer un test sobre redes locales e Internet:"
LOCAL lnPuntos, lnI, lcFile, lcPregunta, lcRespuestaUsuario, lcRightAnswer

* 1. Setup FICHEROP.DBF. Es la tabla para la puntuación, score
SELECT 1
USE C:\DBASE\FICHEROP.DBF
REPLACE PUNTOS WITH 0
lnPuntos = 0

* 2. Bucle que busca las preguntas
FOR lnI = 1 TO 5
    lcFile = "C:\DBASE\PREGUNTA" + ALLTRIM(STR(lnI)) + ".DBF"

    IF FILE(lcFile)
        SELECT 9
        USE (lcFile)
        GO TOP

        * Obtenemos la info del fichero (DBF fields: LINEA, RESPUESTA)
        lcRightAnswer = ALLTRIM(UPPER(RESPUESTA))
        lcPregunta = ALLTRIM(LINEA)
        lcRespuestaUsuario = SPACE(1)

        * Mostramos la pregunta
        @ (lnI * 4), 5 SAY "Q" + ALLTRIM(STR(lnI)) + ". " + lcPregunta + " (V/F)?: "
        @ (lnI * 4), 77 GET lcRespuestaUsuario
        READ

        * Validamos respuesta
        IF UPPER(TRIM(lcRespuestaUsuario)) == lcRightAnswer
            @ (lnI * 4) + 2, 5 SAY "Respuesta correcta."
            lnPuntos = lnPuntos + 1
        ELSE
            @ (lnI * 4) + 2, 5 SAY "Respuesta incorrecta."
        ENDIF

    USE
ENDIF
ENDFOR

* 3. Grabamos puntuación en FICHEROP.DBF
SELECT 1
GO TOP
REPLACE PUNTOS WITH lnPuntos
lcMessage="Puntos obtenidos: "+ALLTRIM(STR(lnPuntos))
MESSAGEBOX(lcMessage, 64, "Resultado Final")
*** Puede encontrar preguntas y respuestas en la página siguiente.
. .
```

Preguntas y respuestas (necesarias para completar el reto 3 de la página anterior)

Campos del archivo PREGUNTA1.dbf

LINEA

HTTPS es el nombre del protocolo en el que se basa la comunicación segura en Internet.

Respuesta

V

Campos del archivo PREGUNTA2.dbf

LINEA

Todos los dispositivos en una red deben tener una dirección de hardware única.

Respuesta

V

Campos del archivo PREGUNTA3.dbf

LINEA

Llamamos ID al nombre de una red WIFI.

Respuesta

F

Campos del archivo PREGUNTA4.dbf

LINEA

La clave de una red WIFI debería contener entre 12 y 63 caracteres para hacerla más segura.

Respuesta

V

Campos del archivo PREGUNTA5.dbf

LINEA

No utilizar una VPN (Virtual Private Network) puede aumentar la seguridad de nuestra red.

Respuesta

F

Reto 4:

Analice el siguiente programa, identifique los archivos de datos, busque errores y añada 8 preguntas (en el anexo solo encontrará 7). También necesitará modificar el código para leer las preguntas y respuestas.

```
* xxxPREGUNTASXXX.PRG - Optimizado con bucle
CLEAR
@ 2,5 SAY "Vamos a hacer un test sobre redes locales e Internet:"
LOCAL lnPuntos, lnI, lcFile, lcPregunta, lcRespuestaUsuario, lcRightAnswer

* 1. Setup FICHEROP.DBF. Es la tabla para la puntuación, score
SELECT 1
USE C:\DBASE\FICHEROP.DBF
REPLACE PUNTOS WITH 0
lnPuntos = 0

* 2. Bucle que busca las preguntas
FOR lnI = 1 TO 8
    lcFile = "C:\DBASE\PREGUNTA" + ALLTRIM(STR(lnI)) + ".DBF"

    IF FILE(lcFile)
        SELECT 9
        USE (lcFile)
        GO TOP

        * Obtenemos la info del fichero (DBF fields: LINEA, RESPUESTA)
        lcRightAnswer = ALLTRIM(UPPER(RESPUESTA))
        lcPregunta = ALLTRIM(LINEA)
        lcRespuestaUsuario = SPACE(1)

        * Mostramos la pregunta
        @ (lnI * 4), 5 SAY "Q" + ALLTRIM(STR(lnI)) + ". " + lcPregunta + " (V/F)?: "
        @ (lnI * 4), 77 GET lcRespuestaUsuario
        READ

        * Validamos respuesta
        IF UPPER(TRIM(lcRespuestaUsuario)) == lcRightAnswer
            @ (lnI * 4) + 2, 5 SAY "Respuesta correcta."
            lnPuntos = lnPuntos + 1
        ELSE
            @ (lnI * 4) + 2, 5 SAY "Respuesta incorrecta."
        ENDIF

    USE
ENDIF
ENDFOR

* 3. Grabamos puntuación en FICHEROP.DBF
SELECT 1
GO TOP
REPLACE PUNTOS WITH lnPuntos
lcMessage="Puntos obtenidos: "+ALLTRIM(STR(lnPuntos))
MESSAGEBOX(lcMessage, 64, "Resultado Final")
*** Puede encontrar preguntas y respuestas en la página siguiente.
. .
```

Preguntas y respuestas (necesarias para completar el reto 4 de la página anterior)

Campos del archivo PREGUNTA1.dbf

LINEA

HTTPS es el nombre del protocolo en el que se basa la comunicación segura en Internet.

Respuesta

V

Campos del archivo PREGUNTA2.dbf

LINEA

Todos los dispositivos en una red deben tener una dirección de hardware única.

Respuesta

V

Campos del archivo PREGUNTA3.dbf

LINEA

Llamamos ID al nombre de una red WIFI.

Respuesta

F

Campos del archivo PREGUNTA4.dbf

LINEA

La clave de una red WIFI debería contener entre 12 y 63 caracteres para hacerla más segura.

Respuesta

V

Campos del archivo PREGUNTA5.dbf

LINEA

No utilizar una VPN (Virtual Private Network) puede aumentar la seguridad de nuestra red.

Respuesta

F

Campos del archivo PREGUNTA6.dbf

LINEA

El protocolo POP3 está diseñado para buscar y descargar correos electrónicos en un servidor.

Respuesta

V

Campos del archivo PREGUNTA7.dbf

LINEA

Una WAN (Wide Area Network) puede incluir centros de datos de diferentes países.

Respuesta

V

Reto 5

Corrija el siguiente programa que contiene una lista de variables para que se imprima la lista de comandos (con su definición) en pantalla en orden alfabético. Complete las frases y corrija posibles errores de sintaxis.

```
*** ORDENAR.PRG

*** PREGUNTAS.DBF contiene tres campos:
*** pregunta (texto de 79 caracteres), respuesta (texto de 1 carácter) y
*** puntos (numérico de 1 dígito).
*** Cada registro de la tabla PREGUNTAS.DBF contiene una pregunta con su respuesta.

A="rm. El comando rm en Linux sirve para borrar un archivo."
B="ls. El comando ls sirve para. . ."
C="pwd"
D="cd"
E="cat"
F="cp"
G="mv"
H="mkdir"
I="rmdir"
JJ="head

?E
?D
?F
```

Ejemplo 13fx. Creación de una carta modelo donde la información se almacena en un archivo externo.

Objetivo: Entender el funcionamiento de **un procedimiento** externo definido por el usuario. Llamada al procedimiento con parámetros.

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo13fx.prg
```

A continuación, copie el siguiente script en la ventana **Ejemplo13fx.prg**:

```
***** Programa principal: Ejemplo13fx.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a crear una carta en un archivo DBF. Dime tu nombre:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a los procedimientos externos:

DO C:/DBASE/CREARLINEASXC.PRG

DO C:/DBASE/FELICITACIONXC.PRG WITH wNombre

*** Mostramos la carta creada para editarla:

pausa=space(1)
@ 24, 5 SAY "Pulse <ENTER> para editar la carta:" GET pausa
READ
BROWSE TITLE "PULSE <ESCAPE> PARA SALIR"
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\CREARLINEASXC.PRG

Ahora, copie el siguiente texto dentro de la ventana **CREARLINEASXC.PRG**:

```
***** CREARLINEASXC.PRG (PROCEDIMIENTO EXTERNO QUE VA A CONTENER LAS líneas DE LA CARTA
***

*** DESACTIVAMOS LA PREGUNTA DEL SISTEMA "Desea reemplazar este archivo?"
SET SAFETY OFF
CLEAR

CREATE TABLE C:\DBASE\LINEAS.dbf (linea C(79), respuesta C(1))

*** LINEA1 OR PREGUNTA 1
APPEND BLANK
REPLACE LINEA WITH "Las Palmas de Gran Canaria a 3 de Diciembre de 2025."
REPLACE RESPUESTA WITH "V"
GO 1
@ 6, 7 SAY ALLTRIM(LINEA)

*** LINEA2 OR PREGUNTA 2
APPEND BLANK
REPLACE LINEA WITH "Estimado Señor Smith,"
REPLACE RESPUESTA WITH "V"
GO 2
@ 8, 7 SAY ALLTRIM(LINEA)

*** LINEA3 OR PREGUNTA 3
APPEND BLANK
REPLACE LINEA WITH "Le escribimos para informarle de la nota final obtenida en el "
REPLACE RESPUESTA WITH "V"
GO 3
@ 10, 7 SAY ALLTRIM(LINEA)

*** LINEA4 OR PREGUNTA 4
APPEND BLANK
REPLACE LINEA WITH "módulo de SEGURIDAD EN INTERNET: Sobresaliente."
REPLACE RESPUESTA WITH "V"
GO 4
@ 12, 7 SAY ALLTRIM(LINEA)

*** LINEA5 OR PREGUNTA 5
APPEND BLANK
REPLACE LINEA WITH "Atentamente,"
REPLACE RESPUESTA WITH "V"
GO 5
@ 14, 7 SAY ALLTRIM(LINEA)

*** LINEA6 OR PREGUNTA 6
APPEND BLANK
REPLACE LINEA WITH "Javier Garcia"
REPLACE RESPUESTA WITH "V"
GO 6
@ 16, 7 SAY ALLTRIM(LINEA)

SET SAFETY ON
```

Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACIONXC.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACIONXC.prg**:

```
***** FELICITACIONXC.PRG
```

```
PARAMETERS NN
```

```
@ 20, 5 SAY "Felicitaciones, siga trabajando así."
```

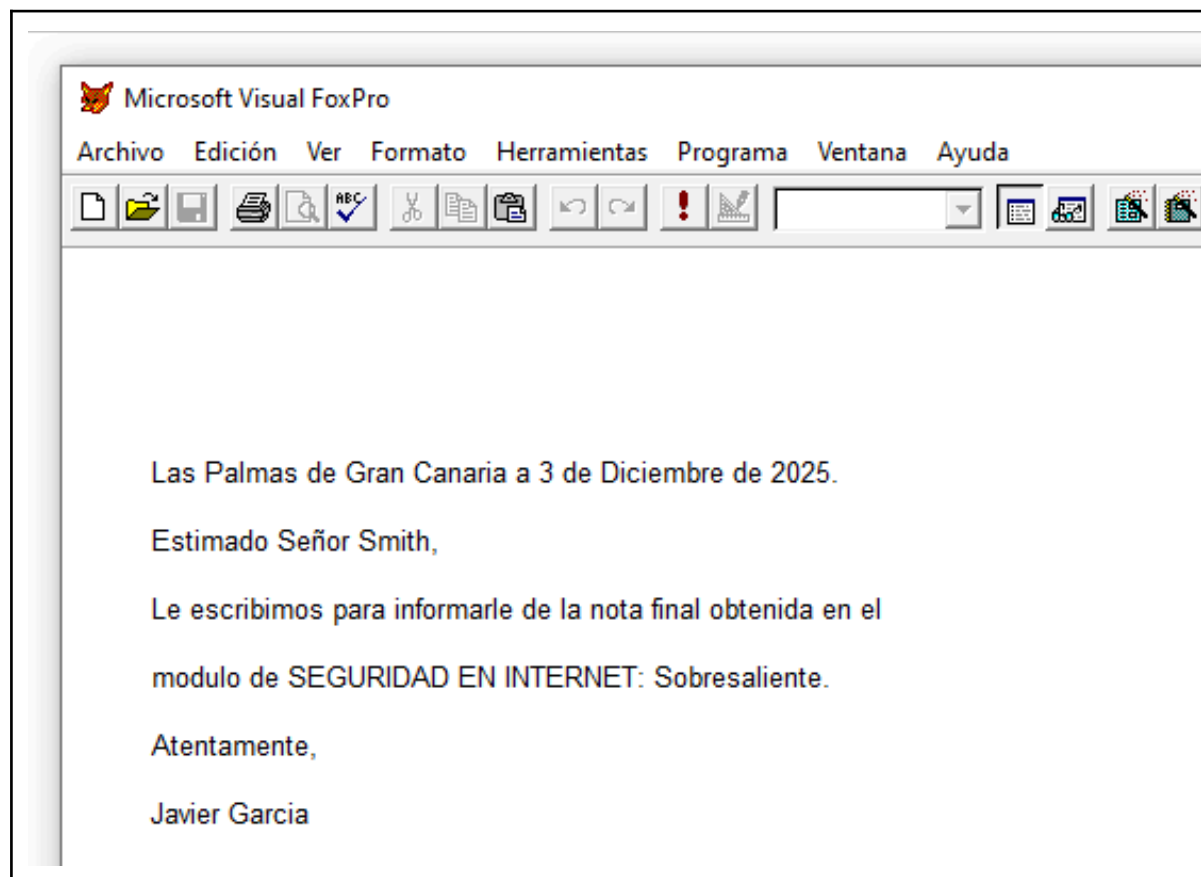
```
@ 22, 5 SAY "Estamos orgullosos de usted, " + NN + "."
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo13fx.prg
```

Continúa en la siguiente página. . .

Después de ejecutar el programa principal, veremos la siguiente pantalla donde el usuario debe introducir su nombre para, a continuación, ver la siguiente comunicación:



Las líneas de la carta están almacenadas en un archivo externo llamado **LINEAS.dbf**, que solo contiene el campo **linea** (de tipo Carácter con una longitud de 79 caracteres) y el campo **respuesta** (de tipo Carácter con una longitud de 1 carácter).

Esta tabla (archivo externo que contiene las líneas de la carta) se ha creado con las siguientes instrucciones:

```
CREATE TABLE C:\DBASE\LINEAS.dbf (linea C(79), respuesta C(1))  
  
APPEND BLANK  
  
REPLACE linea WITH "Le escribimos para informarle de la nota final obtenida en el"
```

Nota:

Observe que cada línea de la carta se ha creado con un comando “**REPLACE**” en el archivo **CREARLINEASXC.PRG**. En el cuadro anterior solo se muestra una línea de la carta creada.

Reto:

Cree un informe con formato PDF basado en el archivo que contiene la carta con la orden “**CREATE REPORT. . .**”.

Ejemplo 14f. Producto de tres valores utilizando un procedimiento externo.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla implementando un **procedimiento** externo definido por el usuario.

Haga clic en:

Start > Programs > Visual FoxPro

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo14f.prg
```

A continuación, copie el siguiente script dentro de la ventana **Ejemplo14f.prg**:

```
***** Programa principal: Ejemplo14f.prg

CLEAR

DO C:/DBASE/mar.prg with 5,5,5
```

Para crear el procedimiento externo llamado **mar.prg**, que realiza los cálculos, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\mar.prg

A continuación, copie el siguiente script dentro de la ventana **mar.prg**

```
***** Procedimiento externo: mar.prg

*** volume= 4/3 * pi * r * r * r

PARAMETERS a,b,c

* Ahora multiplicamos los valores a,b,c para calcular el volumen del
* cubo y después imprimir el resultado

CLEAR
? "Vamos a calcular el volumen de una esfera."
?
? "Si el radio de la esfera mide "
? a
? "metros,"
?
? "El volumen de la esfera será: "

pix=3.14

? 4/3*pix*a*b*c
? "metros cúbicos."
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

DO C:\DBASE\Ejemplo14f.prg

Pregunta: ¿Cuántos archivos necesitamos terminar en este ejemplo 14f?

Dos archivos:

El **primer** archivo es el programa principal: **ejemplo14f.prg**

El **segundo** archivo es: **mar.prg** (que es llamado por el programa principal).

Ejemplo 15f. Cálculo del volumen de un cilindro.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla implementando un **procedimiento** externo definido por el usuario.

Haga clic en:

Start > Programs > Visual FoxPro

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo15f.prg
```

A continuación, copie el siguiente script dentro de la ventana `Ejemplo15f.prg`:

```
***** Programa principal: Ejemplo15f.prg

CLEAR

DO C:/DBASE/xmar.prg with 5,5
```

Para crear el procedimiento externo llamado **xmar.prg**, que realiza los cálculos, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\xmar.prg

A continuación, copie el siguiente script dentro de la ventana **xmar.prg**

```
***** Procedimiento externo: xmar.prg

*** volume= pi * r * r * h

PARAMETERS radius, height

* Ahora multiplicamos los valores radius y height para calcular el volumen del
* cilindro Y Después imprimir el resultado

CLEAR
? "Vamos a calcular el volumen de un cilindro."
?
? "Si el radio del circulo mide "
? radius
? "metros,"
?
? "El volumen del cilindro será: "

pix=3.14

? pix*radius*radius*height
? "metros cubicos."
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

DO C:\DBASE\Ejemplo15f.prg

Pregunta: ¿Cuántos archivos necesitamos terminar en este ejemplo 15f?

Dos archivos:

El **primer** archivo es el programa principal: **ejemplo15f.prg**

El **segundo** archivo es: **xmar.prg** (que es llamado por el programa principal).

Ejemplo 16f. Cálculo del volumen de un cilindro.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla implementando un **procedimiento** externo definido por el usuario.

Haga clic en:

Start > Programs > Visual FoxPro

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo16f.prg
```

A continuación, copie el siguiente script dentro de la ventana `Ejemplo16f.prg`:

```
***** Programa principal: Ejemplo16f.prg

CLEAR

DO C:/DBASE/calcular_volumen.prg with 5,5
```

Para crear el procedimiento externo llamado `calcular_volumen.prg`, que realiza los cálculos, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\calcular_volumen.prg

A continuación, copie este script dentro de la ventana `calcular_volumen.prg`

```
***** Procedimiento externo: calcular_volumen.prg

PARAMETERS radius, height

* Ahora multiplicamos los valores radius y height para calcular el volumen
* del cilindro Y Después imprimir el resultado

CLEAR
? "Vamos a calcular el volumen de un cilindro."
?
? "Si el radio del circulo mide "
? radius
? "metros,"
?
? "El volumen del cilindro será: "

pix=3.14

? pix*radius*radius*height
? "metros cúbicos."
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

DO C:\DBASE\Ejemplo16f.prg

Pregunta 1: ¿Cuántos archivos necesitamos terminar en este ejemplo 16f?

Dos archivos:

El primer archivo es el programa principal: `ejemplo16f.prg`

El segundo es: `calcular_volumen.prg` (es llamado por el programa principal).

Pregunta 2: ¿Cuál es la diferencia principal entre este ejemplo 16f y el anterior?

Ejemplo 16fx. Cálculo del tiempo que se tarda en duplicar un capital con un interés compuesto del 2% anual.

Objetivo: Crear una aplicación que realiza cálculos financieros.

Para crear el programa llamado **Ejemplo16fx.prg**, que realiza los cálculos, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\Ejemplo16fx.prg

```
***** Programa principal: Ejemplo16fx.prg

*** Tasa de interés anual 2%
interes = 0.02

*** Tiempo para duplicar el dinero (capital):
tiempo = log(2) / log(1 + interes)

? " Tardarás aproximadamente"
? tiempo
? " años en duplicar un capital al"
? interes*100
? " por ciento de interés anual."
```

Ejemplo 16fxc. Cálculo del tiempo que se tarda en duplicar un capital con un interés compuesto del 9% anual.

Objetivo: Crear una aplicación que realiza cálculos financieros.

Para crear el programa llamado **Ejemplo16fxc.prg**, que realiza los cálculos, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo16fxc.prg
```

```
***** Programa principal: Ejemplo16fxc.prg

*** Tasa de interés anual 9%
inte = 0.09

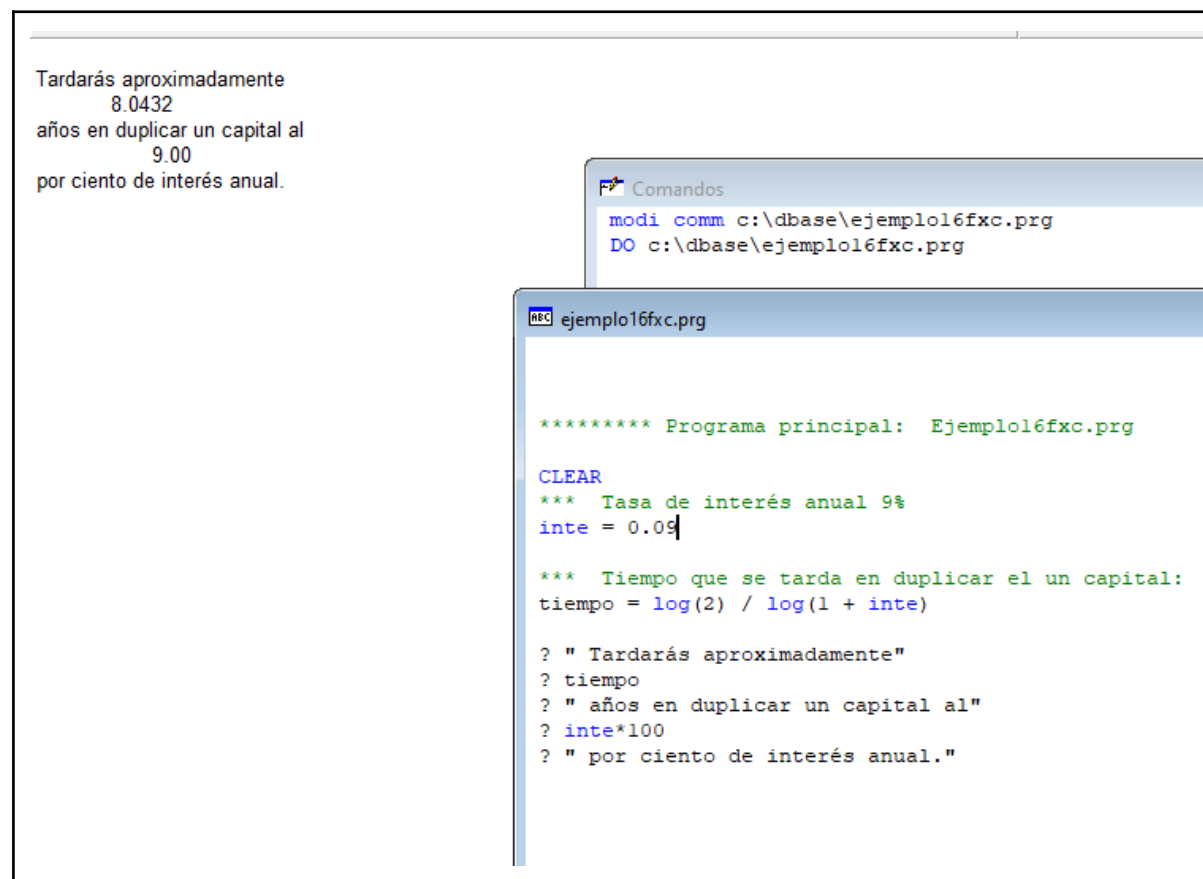
*** Tiempo que se tarda en duplicar el un capital:
tiempo = log(2) / log(1 + inte)

? " Tardará aproximadamente"
? tiempo
? " años en duplicar un capital al"
? inte*100
? " por ciento de interés anual."
```

Para ejecutar el programa, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo16fxc.prg
```

Después de ejecutar el programa **Ejemplo16fxc.prg**, veremos la siguiente pantalla donde se calcula tiempo que se tarda en duplicar un capital con un interés compuesto del 9% anual:



```
Tardarás aproximadamente
      8.0432
años en duplicar un capital al
      9.00
por ciento de interés anual.
```

```
Comandos
modi comm c:\dbase\ejemplo16fxc.prg
DO c:\dbase\ejemplo16fxc.prg
```

```
ejemplo16fxc.prg

***** Programa principal:  Ejemplo16fxc.prg

CLEAR
*** Tasa de interés anual 9%
inte = 0.09

*** Tiempo que se tarda en duplicar el un capital:
tiempo = log(2) / log(1 + inte)

? " Tardarás aproximadamente"
? tiempo
? " años en duplicar un capital al"
? inte*100
? " por ciento de interés anual."
```

Ejemplo 16fxy. Convertir una tasa de interés mensual a tasa de interés anual.

Objetivo: Crear una aplicación que realiza conversiones de tasas de interés para cálculos financieros.

Haga clic en:

File > New File

Grabe su archivo con el nombre de archivo “**Ejemplo16fxy.prg**”.

Ahora que ya tiene creado el archivo, añada las siguientes órdenes.

```
***** Programa Ejemplo16fxy.prg

# Tasa de interés mensual (2%)
interes_mensual = 0.02

# Convertir a tasa anual compuesta
interes_anual = (1 + interes_mensual) ** 12 - 1

? "Una tasa mensual del 2% equivale a un"
? interes_anual*100
? "por ciento anual de interés compuesto."

** Ojo! El interés compuesto hace que se pague mucho más a largo plazo si no
** se amortiza rápido. Veamos algunos ejemplos:

** Préstamos personales y tarjetas de crédito:
** Interés compuesto: Se calcula sobre el saldo pendiente, y si no pagas,
** los intereses se acumulan sobre los intereses anteriores.

** Hipotecas:
** Aunque el cálculo puede parecer simple, el sistema de amortización utilizado
** por la mayoría de bancos en muchos países, entre ellos, Francia y España,
** implica pagos mensuales que incluyen capital e intereses,
** y el interés se recalcula sobre el saldo restante. Es una forma
** de interés compuesto disfrazado.

** Préstamos para vehículos o consumo:
** Algunos usan interés simple, especialmente si el préstamo tiene pagos fijos
** y cortos. Pero los bancos aplican interés compuesto si hay cuotas variables
** o refinanciación.
```


Ejemplo 16fxz. Creación de una carta avisando a los prestamistas de firmar contratos de préstamos con intereses a largo plazo.

Objetivo: Entender el funcionamiento de **un procedimiento** externo definido por el usuario. Llamada a procedimientos externos con parámetros.

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo16fxz.prg
```

A continuación, copie el siguiente script en la ventana **Ejemplo16fxz.prg**:

```
***** Programa principal: Ejemplo16fxz.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a crear una carta en un archivo DBF. Dime tu nombre:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a los procedimientos externos:

DO C:/DBASE/CREARLINEASX.PRG

DO C:/DBASE/FELICITACIONX.PRG WITH wNombre
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\CREARLINEASX.PRG

Ahora, copie el siguiente texto dentro de la ventana **CREARLINEASX.PRG**:

```
*** CREARLINEASX.PRG (PROCEDIMIENTO EXTERNO QUE VA A CONTENER LAS líneas DE LA CARTA

*** DESACTIVAMOS LA PREGUNTA DEL SISTEMA "Desea reemplazar este archivo?"
SET SAFETY OFF
CLEAR

CREATE TABLE C:\DBASE\LINEASX.dbf (línea C(79), respuesta C(1))

*** LINEA1 OR PREGUNTA 1
APPEND BLANK
REPLACE LINEA WITH ";Ojo! El interés compuesto hace que se pague mucho más a largo plazo"
REPLACE RESPUESTA WITH "V"
GO 1
@ 6, 7 SAY ALLTRIM(LINEA)

*** LINEA2 OR PREGUNTA 2
APPEND BLANK
REPLACE LINEA WITH "si no se amortiza rápido. Veamos algunos ejemplos:"
REPLACE RESPUESTA WITH "V"
GO 2
@ 8, 7 SAY ALLTRIM(LINEA)

*** LINEA3 OR PREGUNTA 3
APPEND BLANK
REPLACE LINEA WITH " "
REPLACE RESPUESTA WITH "V"
GO 3
@ 10, 7 SAY ALLTRIM(LINEA)

*** LINEA4 OR PREGUNTA 4
APPEND BLANK
REPLACE LINEA WITH "Préstamos personales y tarjetas de crédito:"
REPLACE RESPUESTA WITH "V"
GO 4
@ 12, 7 SAY ALLTRIM(LINEA)

*** LINEA5 OR PREGUNTA 5
APPEND BLANK
REPLACE LINEA WITH "Interés compuesto: Se calcula sobre el saldo pendiente, y si no"
REPLACE RESPUESTA WITH "V"
GO 5
@ 14, 7 SAY ALLTRIM(LINEA)

*** LINEA6 OR PREGUNTA 6
APPEND BLANK
REPLACE LINEA WITH "pagas, los intereses se acumulan sobre los intereses anteriores."
REPLACE RESPUESTA WITH "V"
GO 6
@ 16, 7 SAY ALLTRIM(LINEA)

*** Continúa en la página siguiente . . .
```

```
*** LINEA7 OR PREGUNTA 7
p7="Hipotecas:"
APPEND BLANK
REPLACE LINEA WITH p7
REPLACE RESPUESTA WITH "v"
GO 7
@ 18, 7 SAY ALLTRIM(LINEA)

*** LINEA8 OR PREGUNTA 8
p8="Aunque el cálculo puede parecer simple, el sistema de amortización utilizado"
APPEND BLANK
REPLACE LINEA WITH p8
REPLACE RESPUESTA WITH "v"
GO 8
@ 20, 7 SAY ALLTRIM(LINEA)

*** LINEA9 OR PREGUNTA 9
p9="por la mayoría de bancos de muchos países, entre ellos, Francia y España,"
APPEND BLANK
REPLACE LINEA WITH p9
REPLACE RESPUESTA WITH "v"
GO 9
@ 22, 7 SAY ALLTRIM(LINEA)

*** LINEA10 OR PREGUNTA 10
p10="implica pagos mensuales que incluyen capital e intereses,"
APPEND BLANK
REPLACE LINEA WITH p10
REPLACE RESPUESTA WITH "v"
GO 10
@ 24, 7 SAY ALLTRIM(LINEA)

*** LINEA11 OR PREGUNTA 11
p11="y el interes se calcula sobre el saldo restante. Es una forma"
APPEND BLANK
REPLACE LINEA WITH p11
REPLACE RESPUESTA WITH "v"
GO 11
@ 26, 7 SAY ALLTRIM(LINEA)

*** LINEA12 OR PREGUNTA 12
p12="de interés compuesto disfrazado."
APPEND BLANK
REPLACE LINEA WITH p12
REPLACE RESPUESTA WITH "v"
GO 12
@ 28, 7 SAY ALLTRIM(LINEA)

*
SET SAFETY ON
*** Fin del script CREAMLINEASX.prg
```

Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACIONX.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACIONX.prg**:

```
***** FELICITACIONX.PRG

PARAMETERS NN

myRows = SROWS() / 2
myCols = SCOLS() / 2
WAIT WINDOW ALLTRIM(NN)+", pulse <ENTER> para ver el archivo de datos" AT myRows, myCols

BROWSE TITLE "ENHORABUENA, EL ARCHIVO HA SIDO CREADO CON ÉXITO. PULSE <ESCAPE> PARA SALIR"
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo16fxz.prg
```

Ejemplo 16fxz. Creación de una carta avisando a los prestamistas de firmar contratos de préstamos con intereses a largo plazo.

Objetivo: Entender el funcionamiento de **un procedimiento** externo definido por el usuario. Llamada a procedimientos externos con parámetros.

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo16fxz.prg
```

A continuación, copie el siguiente script en la ventana **Ejemplo16fxz.prg**:

```
***** Programa principal: Ejemplo16fxz.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a crear una carta en un archivo DBF. Dime tu nombre:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a los procedimientos externos:

DO C:/DBASE/CREARLINEASX.PRG

DO C:/DBASE/FELICITACIONX.PRG WITH wNombre
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\CREARLINEASX.PRG

Ahora, copie el siguiente texto dentro de la ventana **CREARLINEASX.PRG**:

```
*** CREARLINEASX.PRG (PROCEDIMIENTO EXTERNO QUE VA A CONTENER LAS líneas DE LA CARTA

*** DESACTIVAMOS LA PREGUNTA DEL SISTEMA "Desea reemplazar este archivo?"
SET SAFETY OFF
CLEAR

CREATE TABLE C:\DBASE\LINEASX.dbf (línea C(79), respuesta C(1))

*** LINEA1 OR PREGUNTA 1
APPEND BLANK
REPLACE LINEA WITH ";Ojo! El interés compuesto hace que se pague mucho más a largo plazo"
REPLACE RESPUESTA WITH "V"
GO 1
@ 6, 7 SAY ALLTRIM(LINEA)

*** LINEA2 OR PREGUNTA 2
APPEND BLANK
REPLACE LINEA WITH "si no se amortiza rápido. Veamos algunos ejemplos:"
REPLACE RESPUESTA WITH "V"
GO 2
@ 8, 7 SAY ALLTRIM(LINEA)

*** LINEA3 OR PREGUNTA 3
APPEND BLANK
REPLACE LINEA WITH " "
REPLACE RESPUESTA WITH "V"
GO 3
@ 10, 7 SAY ALLTRIM(LINEA)

*** LINEA4 OR PREGUNTA 4
APPEND BLANK
REPLACE LINEA WITH "Préstamos personales y tarjetas de crédito:"
REPLACE RESPUESTA WITH "V"
GO 4
@ 12, 7 SAY ALLTRIM(LINEA)

*** LINEA5 OR PREGUNTA 5
APPEND BLANK
REPLACE LINEA WITH "Interés compuesto: Se calcula sobre el saldo pendiente, y si no"
REPLACE RESPUESTA WITH "V"
GO 5
@ 14, 7 SAY ALLTRIM(LINEA)

*** LINEA6 OR PREGUNTA 6
APPEND BLANK
REPLACE LINEA WITH "pagas, los intereses se acumulan sobre los intereses anteriores."
REPLACE RESPUESTA WITH "V"
GO 6
@ 16, 7 SAY ALLTRIM(LINEA)

*** Continúa en la página siguiente . . .
```

```
*** LINEA7 OR PREGUNTA 7
p7="Hipotecas:"
APPEND BLANK
REPLACE LINEA WITH p7
REPLACE RESPUESTA WITH "v"
GO 7
@ 18, 7 SAY ALLTRIM(LINEA)

*** LINEA8 OR PREGUNTA 8
p8="Aunque el cálculo puede parecer simple, el sistema de amortización utilizado"
APPEND BLANK
REPLACE LINEA WITH p8
REPLACE RESPUESTA WITH "v"
GO 8
@ 20, 7 SAY ALLTRIM(LINEA)

*** LINEA9 OR PREGUNTA 9
p9="por la mayoría de bancos de muchos países, entre ellos, Francia y España,"
APPEND BLANK
REPLACE LINEA WITH p9
REPLACE RESPUESTA WITH "v"
GO 9
@ 22, 7 SAY ALLTRIM(LINEA)

*** LINEA10 OR PREGUNTA 10
p10="implica pagos mensuales que incluyen capital e intereses,"
APPEND BLANK
REPLACE LINEA WITH p10
REPLACE RESPUESTA WITH "v"
GO 10
@ 24, 7 SAY ALLTRIM(LINEA)

*** LINEA11 OR PREGUNTA 11
p11="y el interes se calcula sobre el saldo restante. Es una forma"
APPEND BLANK
REPLACE LINEA WITH p11
REPLACE RESPUESTA WITH "v"
GO 11
@ 26, 7 SAY ALLTRIM(LINEA)

*** LINEA12 OR PREGUNTA 12
p12="de interés compuesto disfrazado."
APPEND BLANK
REPLACE LINEA WITH p12
REPLACE RESPUESTA WITH "v"
GO 12
@ 28, 7 SAY ALLTRIM(LINEA)

*
SET SAFETY ON
*** Fin del script CREAMLINEASX.prg
```

Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACIONX.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACIONX.prg**:

```
***** FELICITACIONX.PRG
```

```
PARAMETERS NN
```

```
myRows = SROWS() / 2
```

```
myCols = SCOLS() / 2
```

```
WAIT WINDOW ALLTRIM(NN)+", pulse <ENTER> para ver el archivo de datos" AT myRows, myCols
```

```
BROWSE TITLE "ENHORABUENA, EL ARCHIVO HA SIDO CREADO CON ÉXITO. PULSE <ESCAPE> PARA SALIR"
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo16fxz.prg
```


Ejemplo 16fzx. Creación de una carta avisando a un prestamista de la deuda pendiente de los meses del último trimestre del año.

Objetivo: Entender el funcionamiento de **un procedimiento** externo definido por el usuario. Llamada a procedimientos externos con parámetros.

Para crear el programa principal, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\Ejemplo16fzx.prg
```

A continuación, copie el siguiente script en la ventana **Ejemplo16fzx.prg**:

```
***** Programa principal: Ejemplo16fzx.prg

LOCAL wNombre
CLEAR
SET TALK OFF

wNombre = SPACE(30)
@ 2, 5 SAY "Vamos a crear una carta en un archivo DBF. Dime tu nombre:" GET wNombre
READ
CLEAR

*   Display Hello
@ 2, 5 SAY "Hola, " + TRIM(wNombre) + ". Comenzamos el test."

*** Llamada a los procedimientos externos:

DO C:/DBASE/CREARLINEASX.PRG

DO C:/DBASE/FELICITACIONX.PRG WITH wNombre
```

Continúa en la siguiente página. . .

Para crear el **primer** procedimiento externo que contiene las preguntas del test, escriba en la ventana “Comandos” la siguiente instrucción:

MODIFY COMMAND C:\DBASE\CREARLINEASXX.PRG

Ahora, copie el siguiente texto dentro de la ventana **CREARLINEASXX.PRG**:

```
*** CREARLINEASXX.PRG (PROCEDIMIENTO EXTERNO QUE VA A CONTENER LAS líneas DE LA CARTA

*** DESACTIVAMOS LA PREGUNTA DEL SISTEMA "Desea reemplazar este archivo?"
SET SAFETY OFF
CLEAR

CREATE TABLE C:\DBASE\LINEASXX.dbf (a N(6,2), b N(6,2), c N(6,2), linea C(79), resp C(1)
)

*** Los campos numeric a, b y c guardarán el saldo pendiente de Octubre, Noviembre y
Dic.

*** LINEA1 OR PREGUNTA 1
APPEND BLANK
REPLACE LINEA WITH "¡Ojo! El interés compuesto hace que se pague mucho más a largo plazo"
GO 1
@ 6, 7 SAY ALLTRIM(LINEA)

*** LINEA2 OR PREGUNTA 2
APPEND BLANK
REPLACE LINEA WITH "si no se amortiza rápido. Veamos algunos ejemplos:"
GO 2
@ 8, 7 SAY ALLTRIM(LINEA)

*** LINEA3 OR PREGUNTA 3
APPEND BLANK
REPLACE LINEA WITH " "
GO 3
@ 10, 7 SAY ALLTRIM(LINEA)

*** LINEA4 OR PREGUNTA 4
APPEND BLANK
REPLACE LINEA WITH "Préstamos personales y tarjetas de crédito:"
GO 4
@ 12, 7 SAY ALLTRIM(LINEA)

*** LINEA5 OR PREGUNTA 5
APPEND BLANK
REPLACE LINEA WITH "Interés compuesto: Se calcula sobre el saldo pendiente, y si no"
GO 5
@ 14, 7 SAY ALLTRIM(LINEA)

*** LINEA6 OR PREGUNTA 6
APPEND BLANK
REPLACE LINEA WITH "pagas, los intereses se acumulan sobre los intereses anteriores."
GO 6
@ 16, 7 SAY ALLTRIM(LINEA)

*** Continúa en la página siguiente . . .
```

```
*** LINEA7 OR PREGUNTA 7
p7="Hipotecas:"
APPEND BLANK
REPLACE LINEA WITH p7
GO 7
@ 18, 7 SAY ALLTRIM(LINEA)

*** LINEA8 OR PREGUNTA 8
p8="Aunque el cálculo puede parecer simple, el sistema de amortización utilizado"
APPEND BLANK
REPLACE LINEA WITH p8
GO 8
@ 20, 7 SAY ALLTRIM(LINEA)

*** LINEA9 OR PREGUNTA 9
p9="por la mayoría de bancos de muchos países, entre ellos, Francia y España,"
APPEND BLANK
REPLACE LINEA WITH p9
GO 9
@ 22, 7 SAY ALLTRIM(LINEA)

*** LINEA10 OR PREGUNTA 10
p10="implica pagos mensuales que incluyen capital e intereses,"
APPEND BLANK
REPLACE LINEA WITH p10
GO 10
@ 24, 7 SAY ALLTRIM(LINEA)

*** LINEA11 OR PREGUNTA 11
p11="y el interes se calcula sobre el saldo restante. Es una forma"
APPEND BLANK
REPLACE LINEA WITH p11
GO 11
@ 26, 7 SAY ALLTRIM(LINEA)

*** LINEA12 OR PREGUNTA 12
p12="de interés compuesto disfrazado."
APPEND BLANK
REPLACE LINEA WITH p12
GO 12
@ 28, 7 SAY ALLTRIM(LINEA)

*
SET SAFETY ON
*** Fin del script CREAMLINEASXX.prg
```

Para crear el **segundo** procedimiento externo que contiene la felicitación, escriba en la ventana “Comandos” la siguiente instrucción:

```
MODIFY COMMAND C:\DBASE\FELICITACIONXX.PRG
```

A continuación, copie el siguiente script en la ventana **FELICITACIONXX.prg**:

```
***** FELICITACIONXX.PRG

PARAMETERS NN

myRows = SROWS() / 2
myCols = SCOLS() / 2
WAIT WINDOW ALLTRIM(NN)+", pulse <ENTER> para ver el archivo de datos" AT myRows, myCols

BROWSE TITLE "ENHORABUENA, EL ARCHIVO HA SIDO CREADO CON ÉXITO. PULSE <ESCAPE> PARA SALIR"
```

Para ejecutar el programa principal, teclearemos en la ventana “Comandos”, la siguiente instrucción:

```
DO C:\DBASE\Ejemplo16fzx.prg
```

Reto 1:

Cree un archivo de datos en formato dbf que contenga 5 preguntas de repaso con sus respuestas sobre el sistema operativo de red Linux. A continuación, cree el archivo PDF que contenga las preguntas y las respuestas correctas.

Preguntas y respuestas (necesarias para completar el reto)

Campos del archivo PREGUNTAS.dbf:

Pregunta (texto de 79 caracteres) y Respuesta (texto de 1 caracter) .

REGISTRO 1

PREGUNTA

La instrucción `rm` permite ver los archivos que contiene un directorio en Linux.

Respuesta

V

REGISTRO 2

PREGUNTA

El protocolo de red TCP está diseñado para entregar los datos en el orden correcto.

Respuesta

V

REGISTRO 3

PREGUNTA

El protocolo de red TCP asegura una entrega de datos fiable.

Respuesta

V

REGISTRO 4

PREGUNTA

Utilizar una VPN (Virtual Private Network) protege la información contra posibles rastreos.

Respuesta

V

Campos del archivo PREGUNTA5.dbf

PREGUNTA

La instrucción `ls` permite ver los archivos que contiene un directorio en Linux.

Respuesta

V

Reto 2:

Cree un archivo de datos en formato dbf que contenga 5 preguntas de repaso con sus respuestas sobre el sistema operativo Linux. A continuación, cree el archivo PDF que contenga las preguntas y las respuestas correctas y los puntos totales obtenidos por el alumno.

Preguntas y respuestas (necesarias para completar el reto)

Campos del archivo PREGUNTAS.dbf:

Pregunta (texto de 79 caracteres), Respuesta (texto de 1 caracter), puntos (numérico de 1 dígito), Alumno (texto de 79 caracteres).

REGISTRO 1

PREGUNTA

La instrucción `rm` permite ver los archivos que contiene un directorio en Linux.

Respuesta

V

REGISTRO 2

PREGUNTA

El software de código abierto fomenta la colaboración entre equipos de desarrolladores.

Respuesta

V

REGISTRO 3

PREGUNTA

El lenguaje Python viene incluido en casi todas las distribuciones Linux..

Respuesta

V

REGISTRO 4

PREGUNTA

Utilizar una VPN (Virtual Private Network) protege la información contra posibles rastreos.

Respuesta

V

Campos del archivo PREGUNTA5.dbf

PREGUNTA

La instrucción `ls` permite ver los archivos que contiene un directorio en Linux.

Respuesta

V

Importante: Visual FoxPro en entornos multiusuario:

La forma más fiable, fácil y directa de conectar dos clientes a una base de datos Visual FoxPro 6 (VFP 6) es mediante una **Carpeta Compartida en Red Local (SMB)** con cables Ethernet. Evite las conexiones WIFI ya que los archivos .cdx o .tmp pueden corromperse debido a una red inalámbrica inestable.

A diferencia de los sistemas cliente-servidor modernos (como PostgreSQL, SQL Server o MariaDB), VFP 6 es una base de datos basada en archivos (.dbc, .dbf, .cdx). No requiere instalar un servicio o servidor de base de datos en el servidor con Windows 10, sino dar acceso directo a los archivos compartidos.

El proceso a seguir es el siguiente:

1. Cómo compartir la carpeta de la base de datos en Windows 10 del servidor:

- En el PC servidor (Windows 10), haz clic derecho sobre la carpeta donde están los archivos .dbf y .dbc de VFP 6 y selecciona **Propiedades**.
- En la pestaña **Compartir**, haz clic en **Uso compartido avanzado**.
- Marca **Compartir esta carpeta** y haz clic en **Permisos**.
- Otorga permiso de **Control total** o **Cambiar y Leer** al usuario o grupo que accederá (necesitan permiso de escritura para crear los archivos de bloqueo .cdx y .tmp).

2. Cómo conectar cada cliente Windows 11 a la unidad de red del servidor:

- Abre el Explorador de archivos en una de las PC con Windows 11.
- Haz clic en **Este equipo** > tres puntos (...) > **Mapear unidad de red** (o Conectar a unidad de red).
- Asigna una letra fija (por ejemplo, **Z:**) y escribe la ruta de red: \\NombreServidor\NombreCarpeta.
- Marca la casilla **Volver a conectar al iniciar sesión**.
- Comprueba que puedes abrir la unidad Z: y ver los archivos .dbf. Repite este proceso en el segundo equipo con la **misma letra de unidad**.

3. Configurar el acceso en la aplicación o mediante ODBC/OLEDB en los clientes:

- Para la aplicación ejecutable (.exe de VFP): Configura la ruta de la base de datos dentro del programa o archivo .ini para que apunte a la unidad Z:\ (ejemplo: Z:\DATOS\MI_BASE.DBC).

Nota importante:

Aunque VFP 8 y VFP 9 utilizan la misma arquitectura de base de datos basada en archivos .dbf, entre otros, hay diferencias notables si utilizamos VFP 8 o VFP 9. Se ha mejorado la estabilidad de la red y actualmente existe un mejor soporte OLEDB.

Los ejecutables compilados en VFP 8 o VFP 9 pueden abrir y trabajar con las bases de datos creadas en VFP 6. No hay necesidad de convertirlas o importarlas aunque es muy importante **no alterar la estructura** de las tablas originales ni añadir registros con versiones superiores a VFP 6.

PARTE 3

PROGRAMACIÓN DE APLICACIONES EN C#

Introducción

C es un lenguaje de programación de propósito general. El lenguaje C ha inspirado a otros desarrolladores a crear otros lenguajes como Python o Java y también se utilizó para crear el sistema operativo Unix, que es la base del popular Linux que hoy conocemos.

C fue ideado por Dennis Ritchie en 1972. Ya en los años 80, el lenguaje C era muy popular en universidades y escuelas de informática.

C++ (C plus plus), que fue creado en los años 80 por Bjarne Stroustrup, es una evolución de C que incorpora programación orientada a objetos y otras características como clases, herencia y polimorfismo. Este lenguaje es muy utilizado en la creación de videojuegos, programación de sistemas e ingeniería.

El entorno de desarrollo .NET 1.0 se creó en 2002 para Microsoft Windows XP y era compatible con Windows 2000, Windows NT 4.0, Windows ME y Windows 98.

Microsoft lanzó oficialmente ese mismo año el lenguaje C# pensando en facilitar la programación de aplicaciones para entornos .NET (recuerde que aunque C# es similar a C++ en muchos aspectos, no son compatibles).

Si está pensando en crear videojuegos o aplicaciones con el motor de desarrollo de videojuegos Unity, necesitará conocer el lenguaje C# , la plataforma .NET. y un editor de código, como por ejemplo, Microsoft Visual Studio Code.

Los ejemplos de este libro han sido creados con Microsoft Windows 10, C# versión 12, Microsoft Visual Studio Code versión 1.99.3 y la plataforma (framework) .NET 8.

Nota: La plataforma .NET 8 ya no soporta oficialmente soporte para Windows 7 o Windows 8.1.

Veamos algunos ejemplos desarrollados con C#:

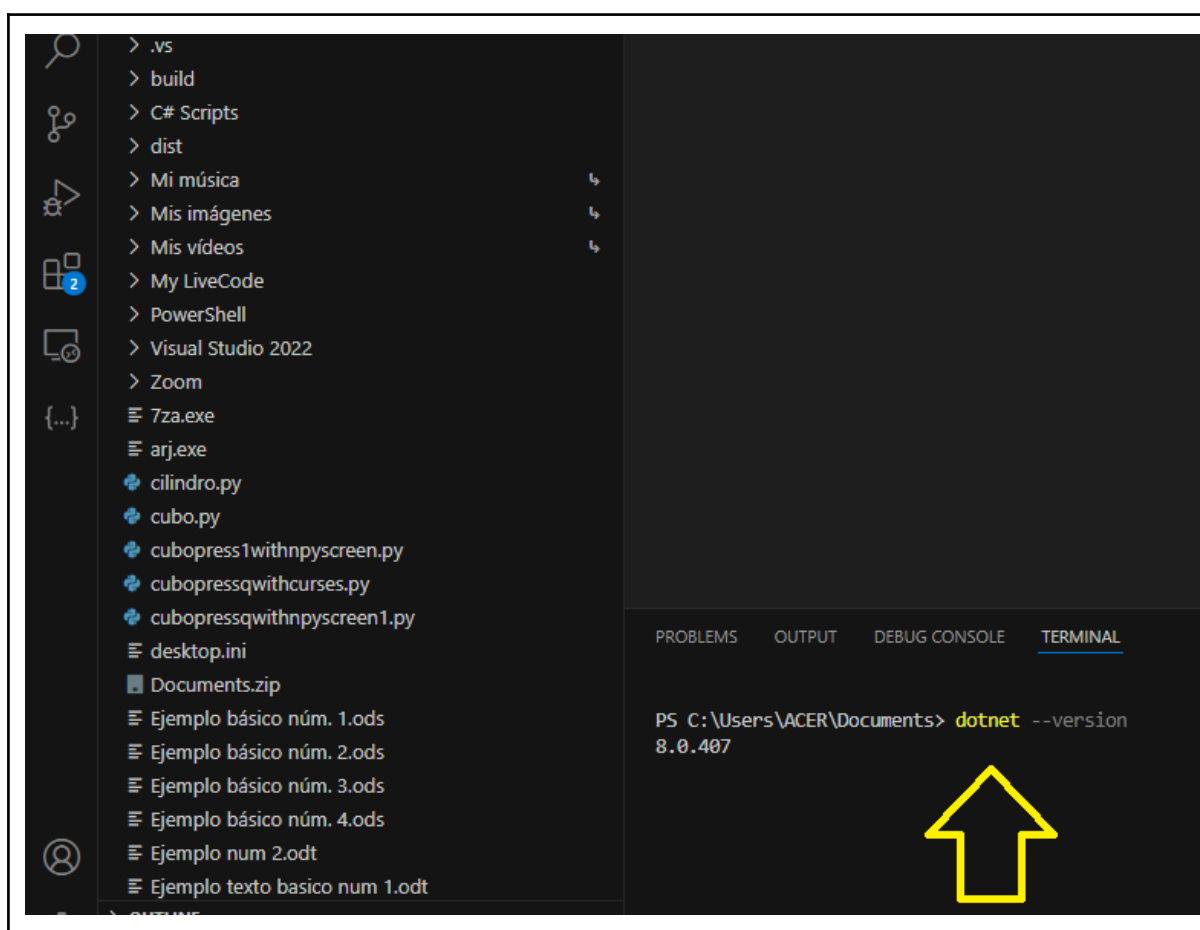
Ejemplo C1. Hola Mundo.

Objetivo: Entender la estructura de un programa en C# mediante el desarrollo del ejemplo más básico: “Hola Mundo”.

Antes de empezar a desarrollar un programa en C# en Visual Studio Code, necesitamos comprobar que nuestro sistema tiene instalado el Kit de desarrollo .NET (también llamado .NET System Development Kit).

Para ello abriremos el Terminal en Visual Studio Code y teclearemos:

dotnet --version

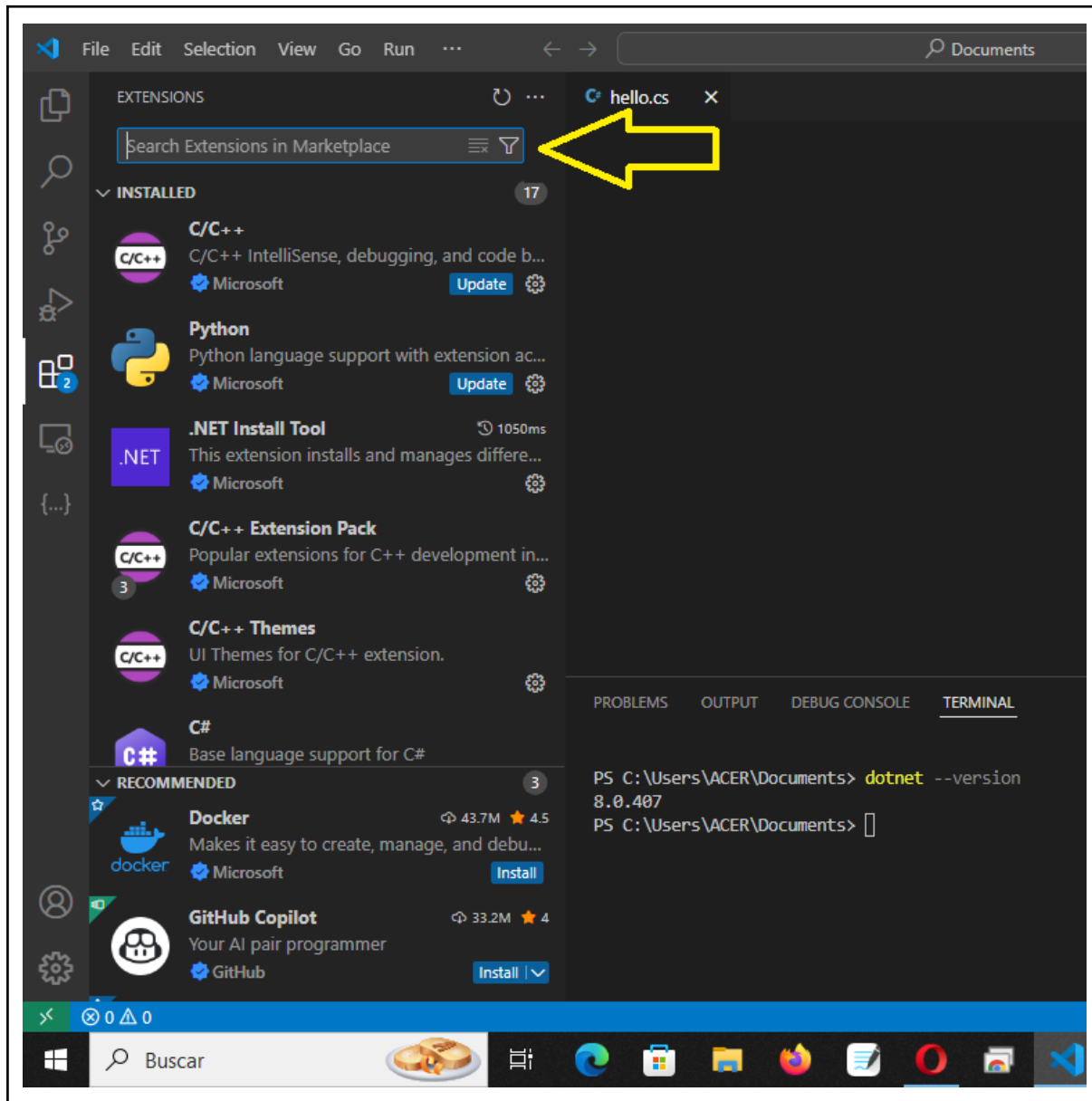


Como podemos observar, la versión instalada es .NET 8.

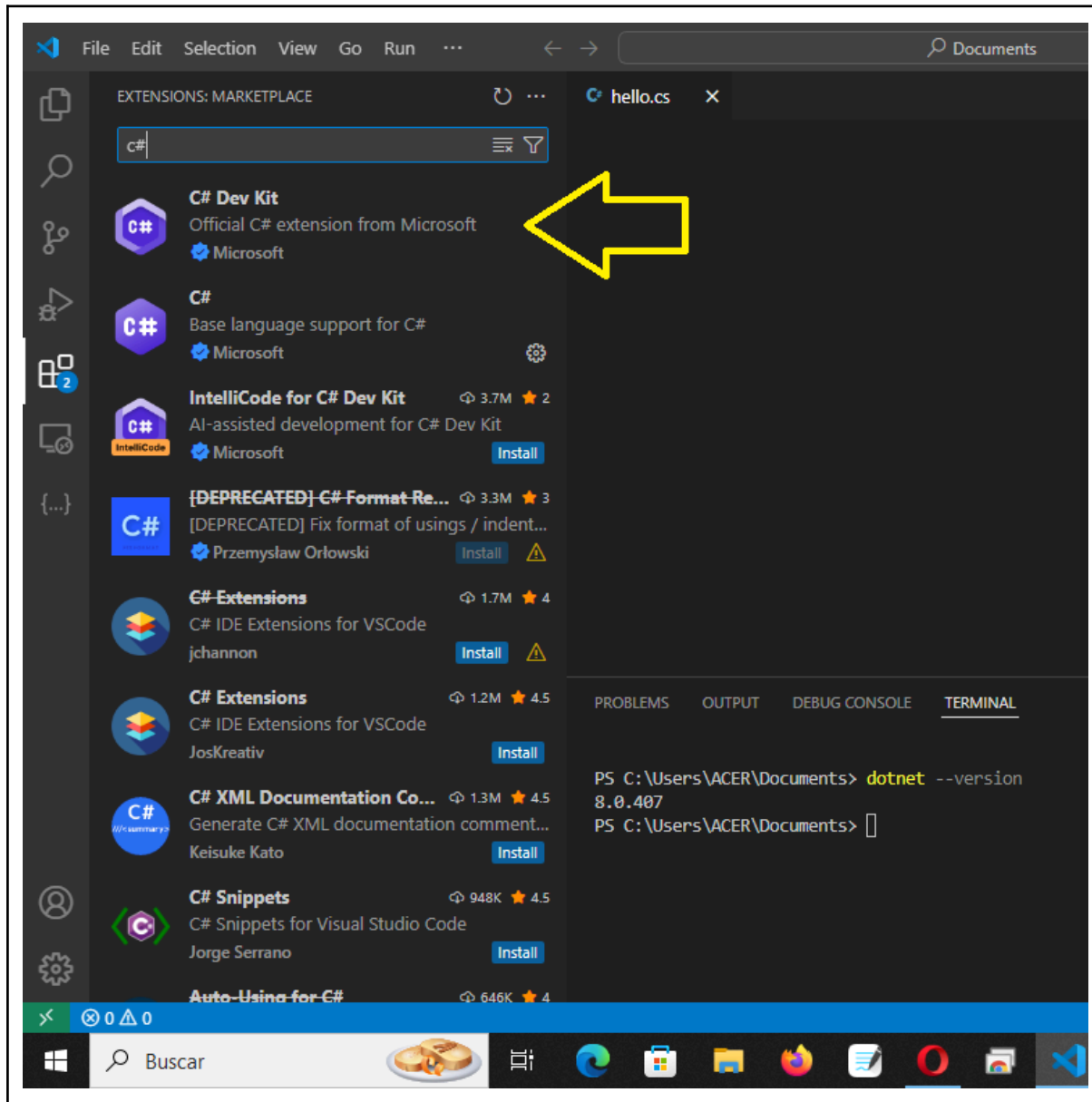
Concretamente, la versión instalada es la 8.0.407, que tiene soporte a largo plazo (LTS- Long Term Support) hasta noviembre de 2026.

A continuación, necesitamos comprobar si tenemos instalada la extensión C# en Visual Studio Code.

Para ello, teclearemos Ctrl+Shift+X y aparecerá el panel de extensiones en la parte izquierda de la pantalla (donde podemos ver la extensiones instaladas en nuestro ordenador):



Al teclear C#, podemos observar que aparece instalada la extensión oficial de Microsoft “C# Dev Kit”.

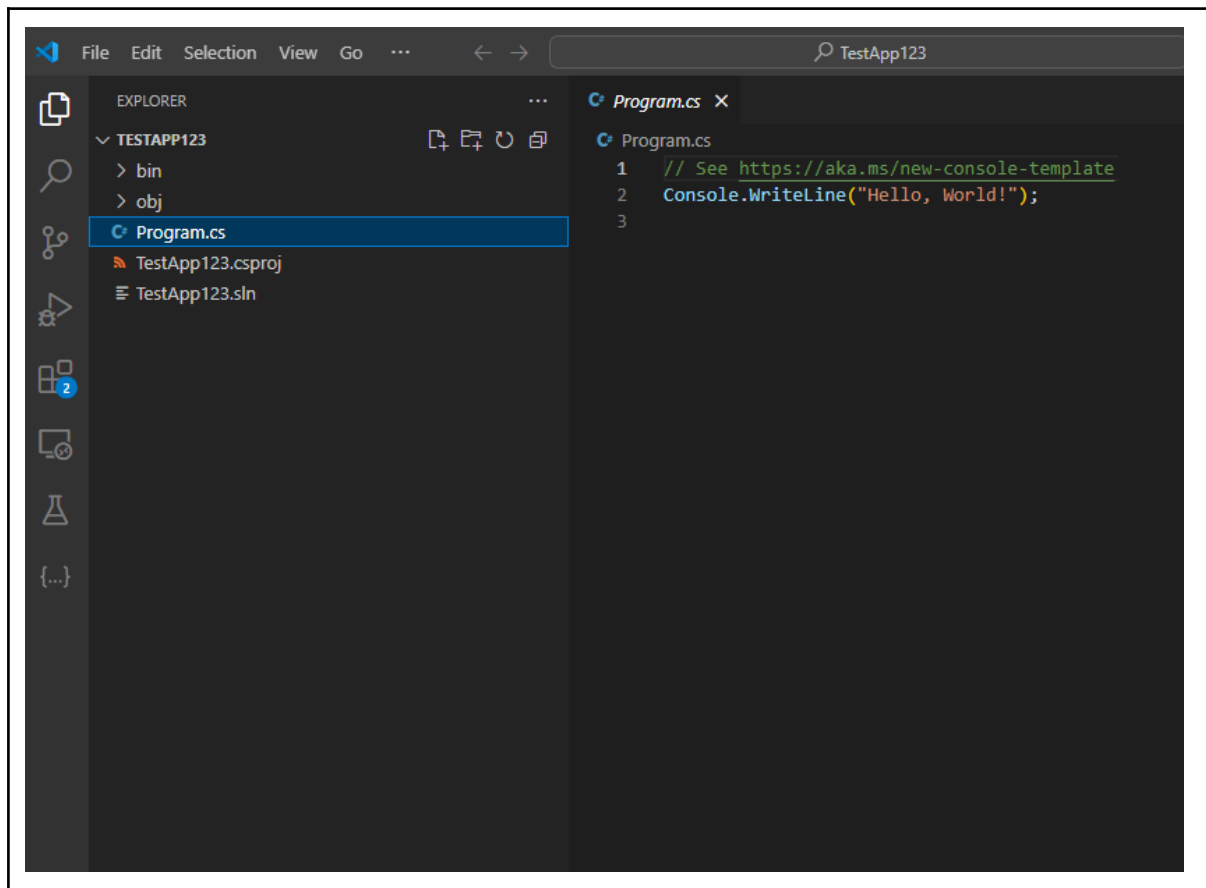


Si en este punto apareciera el botón “Install” (instalar) o el botón “Update” (actualizar) junto a la extensión “C# Dev Kit”, púlselo y espere a que se instalen todos los archivos necesarios.

Ahora que ya tenemos todos los archivos necesarios instalados, teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “TestApp123”:

```
cd documents
dotnet new console -o TestApp123
cd TestApp123
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

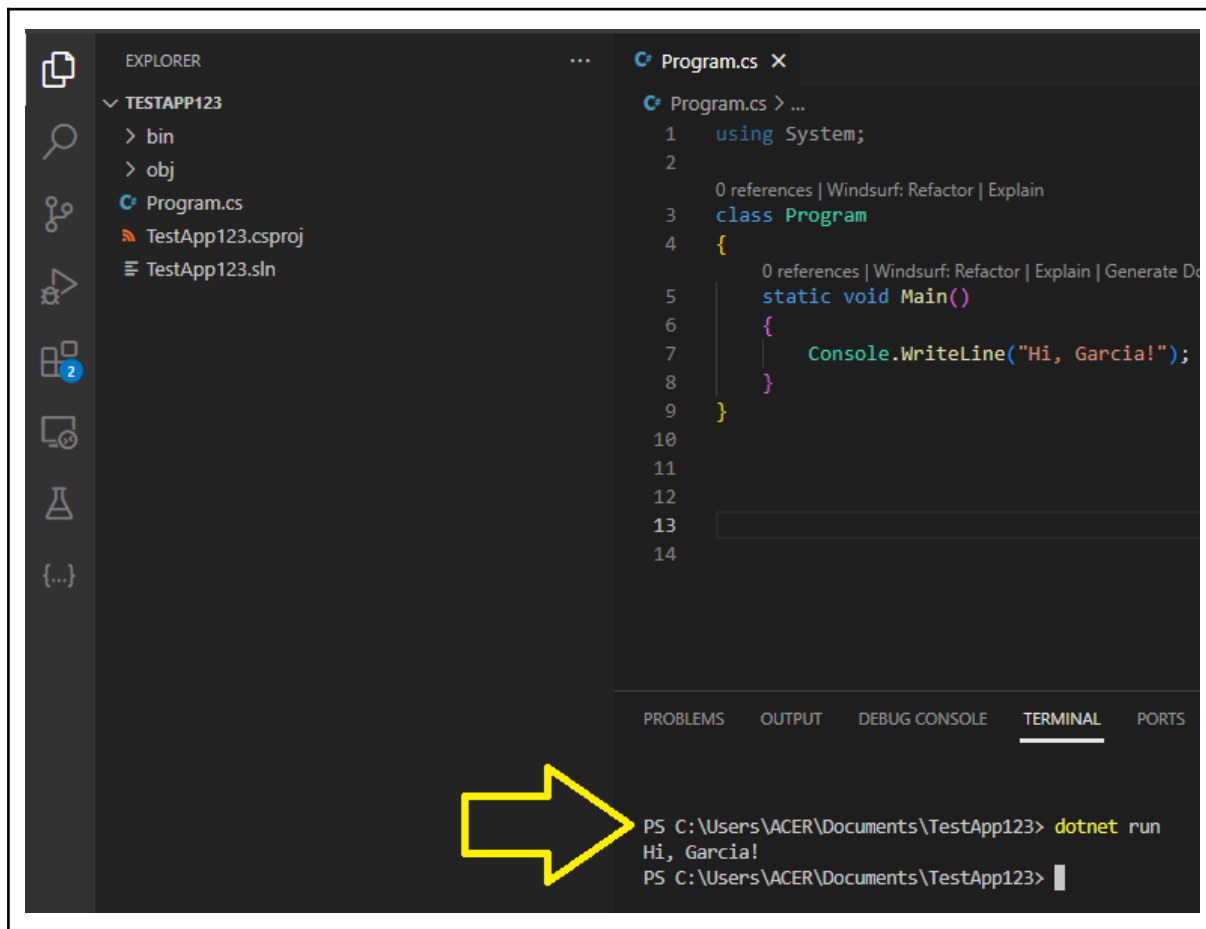
```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Hi, García!");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



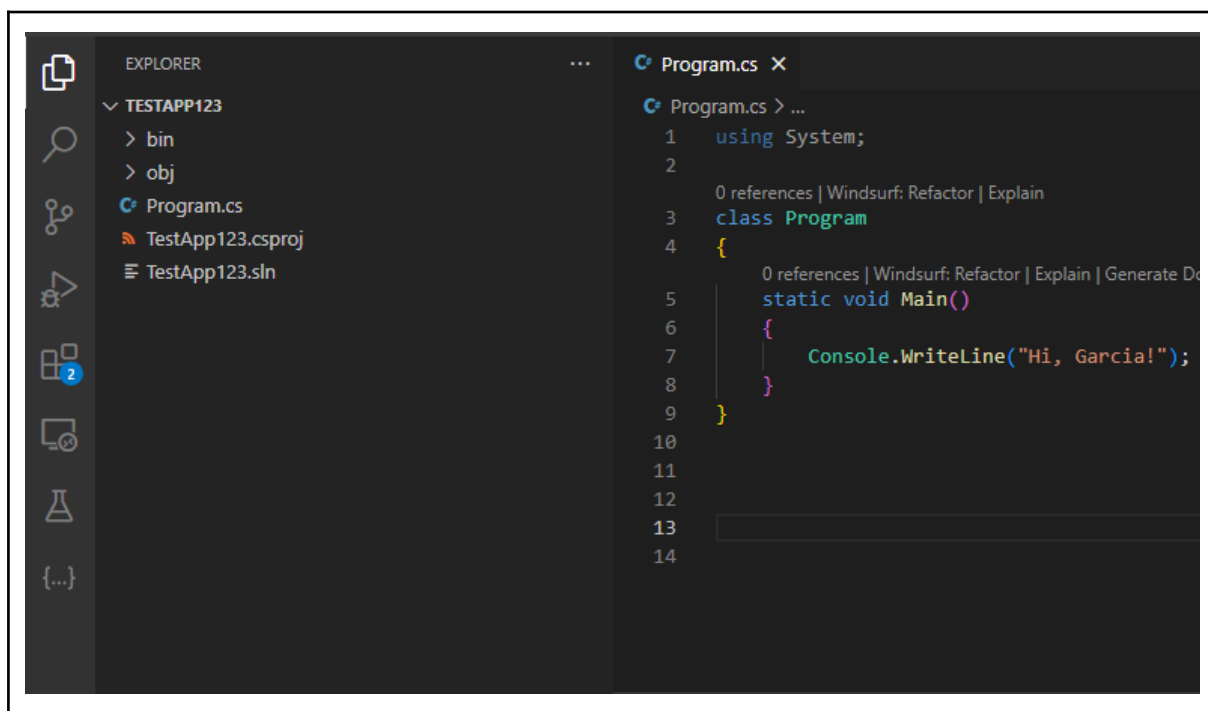
Como se puede observar, ha aparecido un mensaje justo debajo de la orden tecleada anteriormente. El mensaje es “Hi, García”.

El código ha sido ejecutado correctamente y Visual Studio Code ha sido configurado correctamente.

Si desea **modificar** el proyecto, deberá abrir la consola de comandos y teclear las siguientes órdenes:

```
cd documents
cd TestApp123
code .
```

Visual Studio Code se abrirá para que pueda editar su código y realizar modificaciones:



Algunos conceptos básicos

¿Por qué escribimos “using System;” y “class Program” al principio del programa?

using System;

- **using:** Esta palabra clave se traduce como “usando” o, en un contexto de importar funcionalidades, a veces se entiende como “importando”. Indica que quieres utilizar el espacio de nombres especificado.
- **System:** Este es el nombre de un espacio de nombres (system namespace) fundamental en .NET que contiene muchas clases y tipos base esenciales. Se mantiene igual cuando programamos en español.
- **;** (punto y coma) marca el final de la instrucción.

El “System namespace” contiene clases fundamentales y tipos bases que son esenciales para numerosos programas en C#, como:

- **Console:** Clase para interactuar con la consola (“input” y “output”).
- **String:** Para trabajar con texto (text).
- **Int32, Double, etc.:** Para trabajar con tipos numéricos..
- **Object:** La clase base para todos los demás tipos.

class Program

- **class:** Esta palabra clave se traduce directamente como clase. Indica la declaración de una clase.
- **Program:** Este es el nombre que se le ha dado a la clase, que en este caso es "Programa". Los nombres de las clases suelen mantenerse en inglés por convención, pero su significado en español es "Programa".

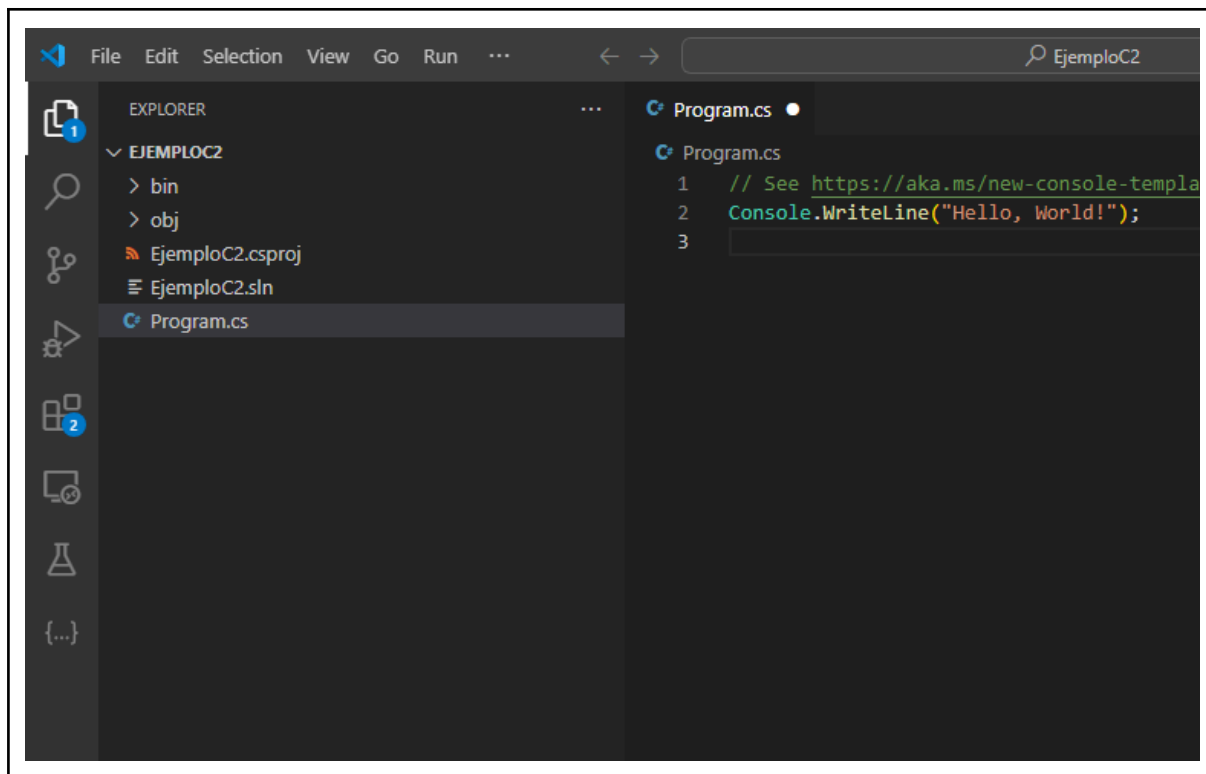
Ejemplo C2. Textos y preguntas.

Objetivo: Aplicación que realiza preguntas al usuario y muestra las respuestas en pantalla.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC2”:

```
cd documents
dotnet new console -o EjemploC2
cd EjemploC2
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

```
using System;                                     // EjemploC2
class Program
{
    static void Main(string[] args)
    {
        string name = Input("Entre su nombre: ");
        string age = Input("Entre su edad: ");

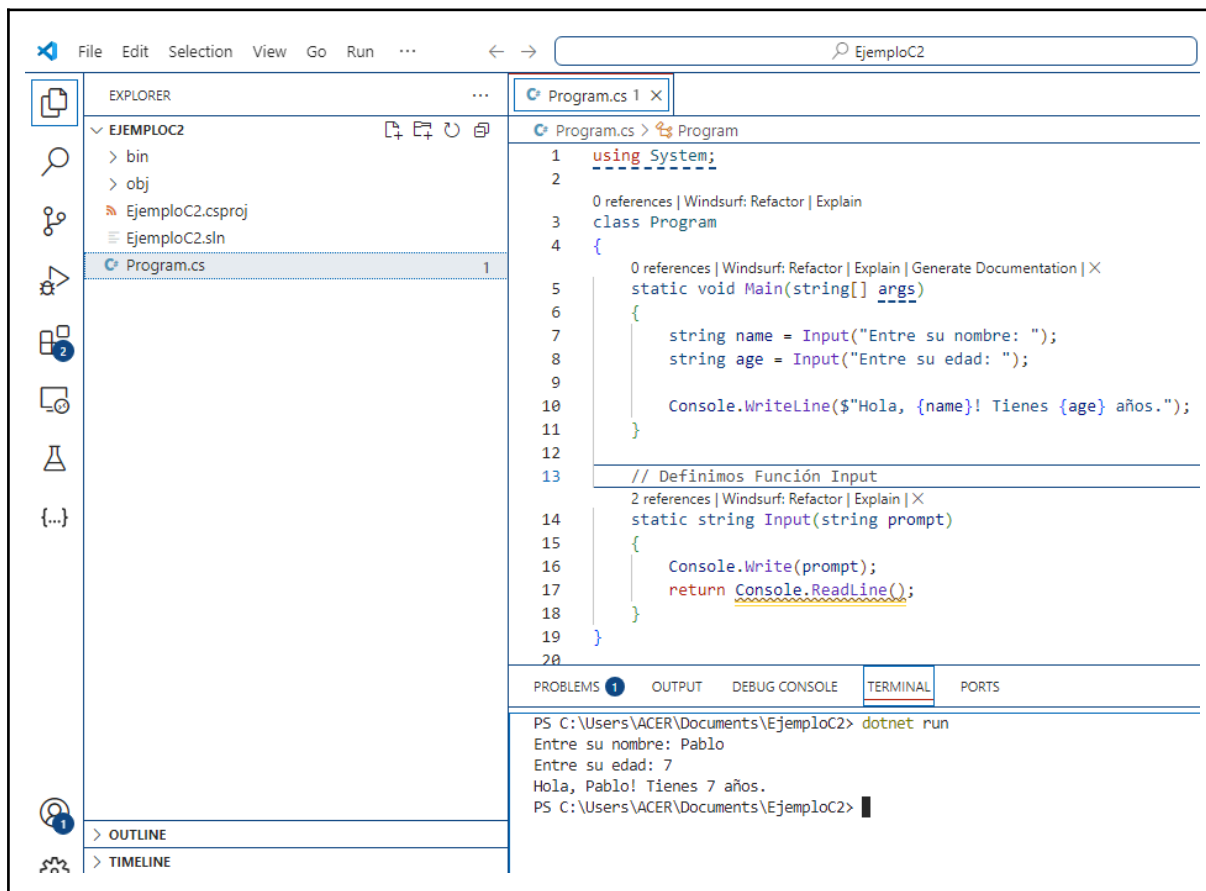
        Console.WriteLine($"Hola, {name}! Tienes {age} años.");
    }

    // Definimos Función Input
    static string Input(string prompt)
    {
        Console.Write(prompt);
        return Console.ReadLine();
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



Observe en la parte inferior de la imagen (terminal) cómo se ha ejecutado el programa. Vemos que, aunque existe un error leve, que arreglaremos más tarde. No obstante, el código se ejecuta como esperábamos.

Importante:

¿Por qué ha aparecido un error leve?

Hemos utilizado variables tipo **String**. En C# existe un grupo de estructuras básicas de datos: **Boolean**, **Integer**, **Float** y **String**. En el ejemplo 2 hemos utilizado una variable tipo **String** llamada **name**. Esta variable puede almacenar cadenas de caracteres alfanuméricas. Más adelante trabajaremos con el resto de estructuras básicas de datos.

El error aparece porque no hemos validado las entradas del usuario por teclado. Es decir, si entra un valor no válido o un valor vacío, el programa no se ejecutaría como esperamos.

El usuario podría haber tecleado un carácter no válido donde se espera una letra o un número y eso podría producir un error.

Reto 1: Cree un proyecto nuevo llamado **EjemploC2B** y escriba un programa que declare una variable llamada **firstName** y después dé un valor a esa variable declarada. Imprima en la consola el valor de la variable declarada. Busque y corrija el error en el siguiente bloque de código:

```
string firstName;  
firstName= "John";  
Console.WriteLine(firstName)
```

Reto 2: Cree un proyecto nuevo llamado **EjemploC2C** y escriba un programa que declare una variable llamada **vdecimalQ** para calcular una división y después dé un valor a esa variable declarada. Imprima en la consola el valor de la variable declarada.

```
decimal vdecimalQ = 5.0m/2;  
Console.WriteLine($"División con decimales: {vdecimalQ}")
```

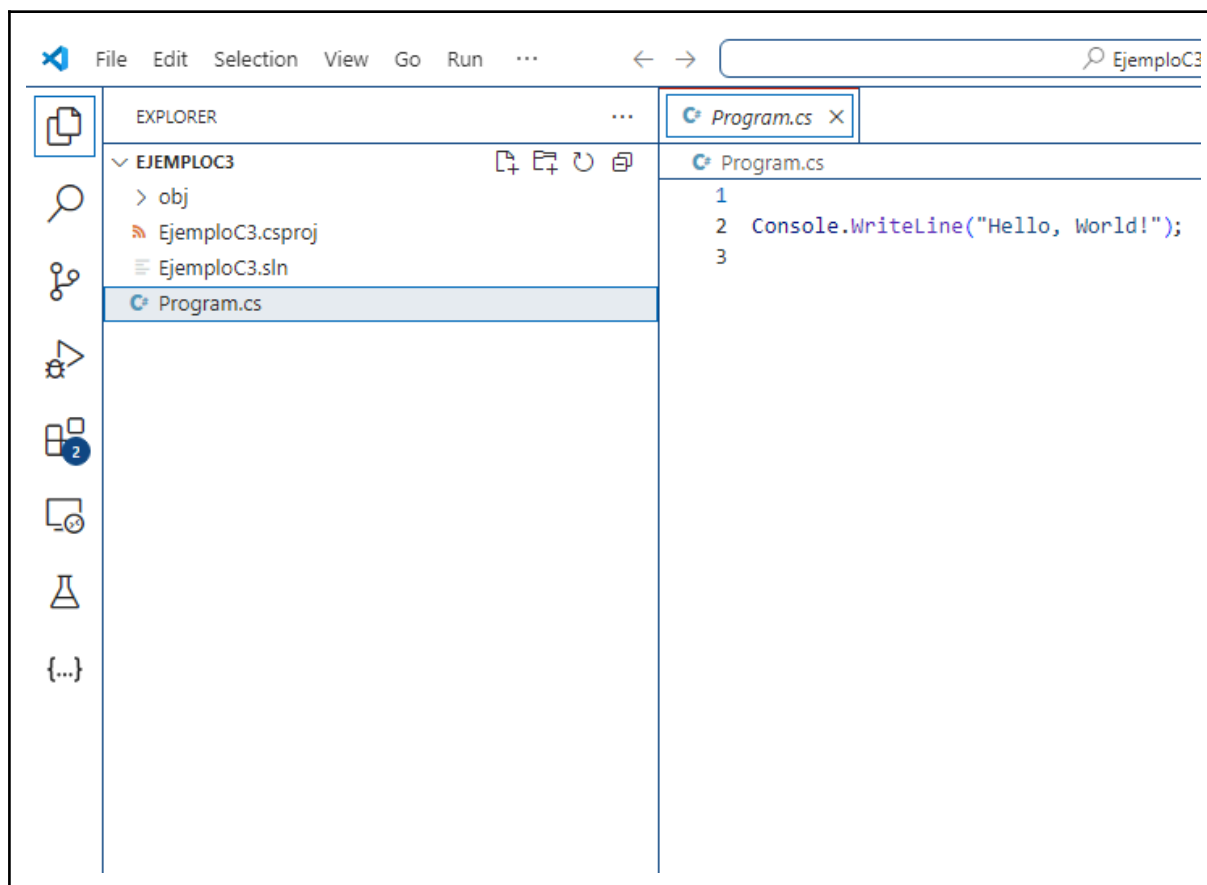
Ejemplo C3. Textos y preguntas.

Objetivo: Aplicación que realiza preguntas al usuario y muestra las respuestas en pantalla.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC3”:

```
cd documents
dotnet new console -o EjemploC3
cd EjemploC3
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

```
using System;                                     // EjemploC3.

class Program
{
    static string? SuperInput(string prompt)
    {
        Console.Write(prompt + " ");
        string? input = Console.ReadLine()?.Trim();
        return string.IsNullOrEmpty(input) ? null : input;
    }

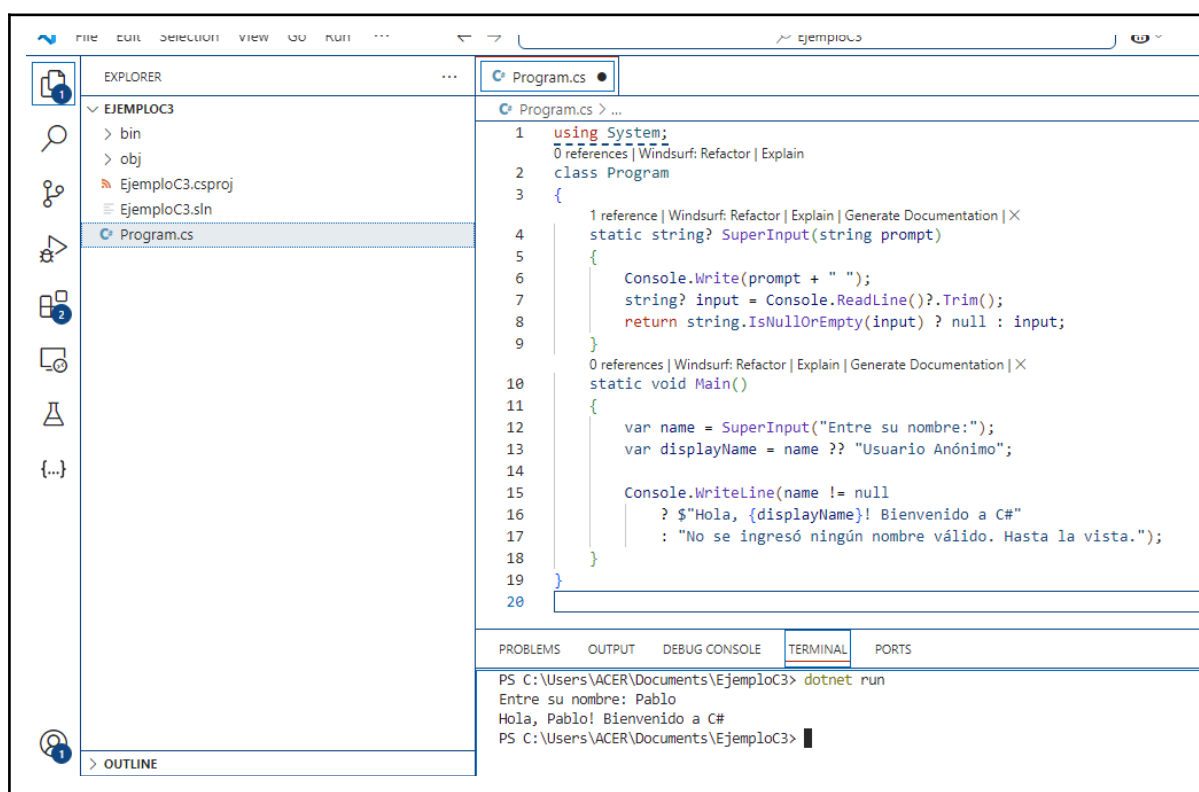
    static void Main()
    {
        var name = SuperInput("Entre su nombre:");
        var displayName = name ?? "Usuario Anónimo";

        Console.WriteLine(name != null
            ? $"Hola, {displayName}! Bienvenido a C#"
            : "No se ingresó ningún nombre válido. Hasta la vista.");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



En esta ocasión, podemos ver en el terminal que hemos arreglado el error leve aparecido anteriormente. Si el usuario ahora no entra ningún nombre, no aparece ningún error. Ya hemos validado la entrada. Si no se proporciona ningún nombre, nuestro código informará al usuario de ello.

Fin del Ejemplo C3.

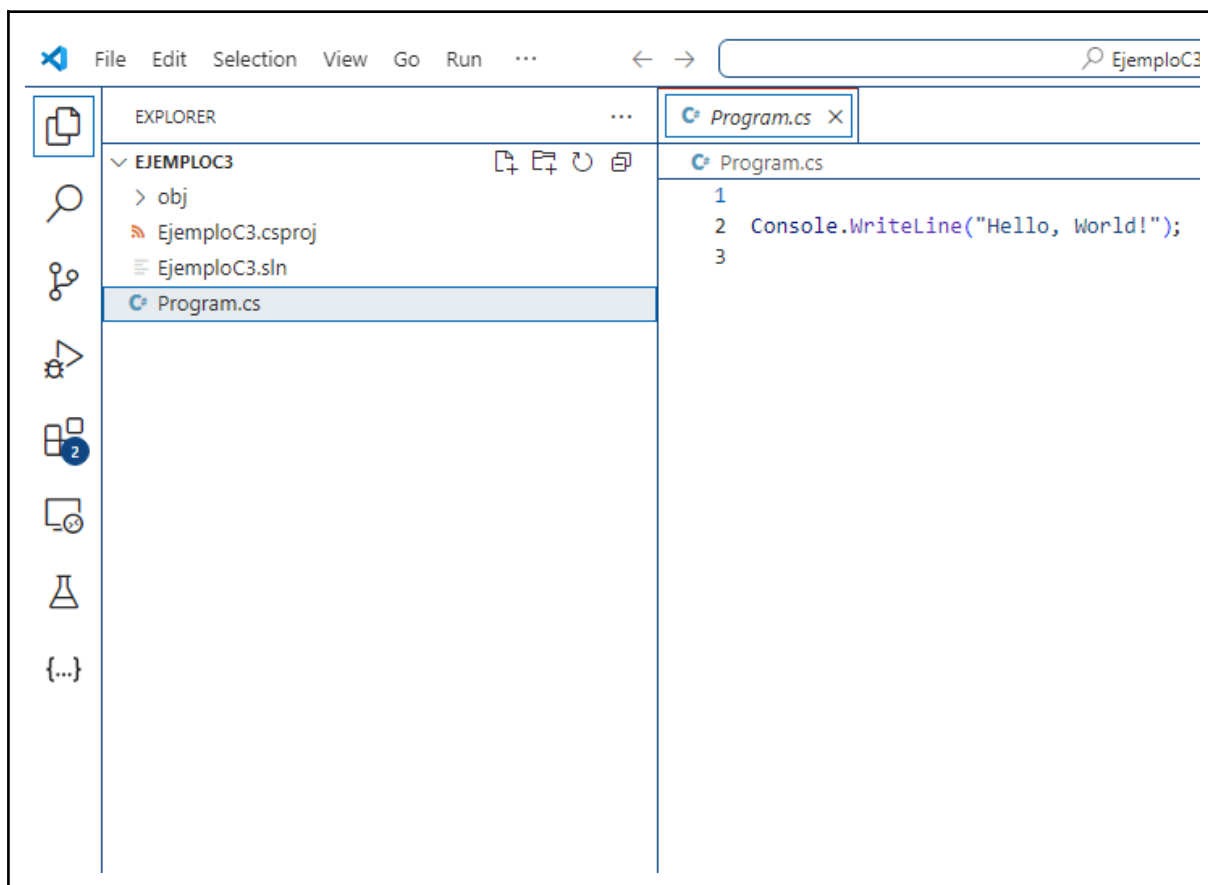
Ejemplo C4. Sumas y restas.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC4”:

```
cd documents  
dotnet new console -o EjemploC4  
cd EjemploC4  
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

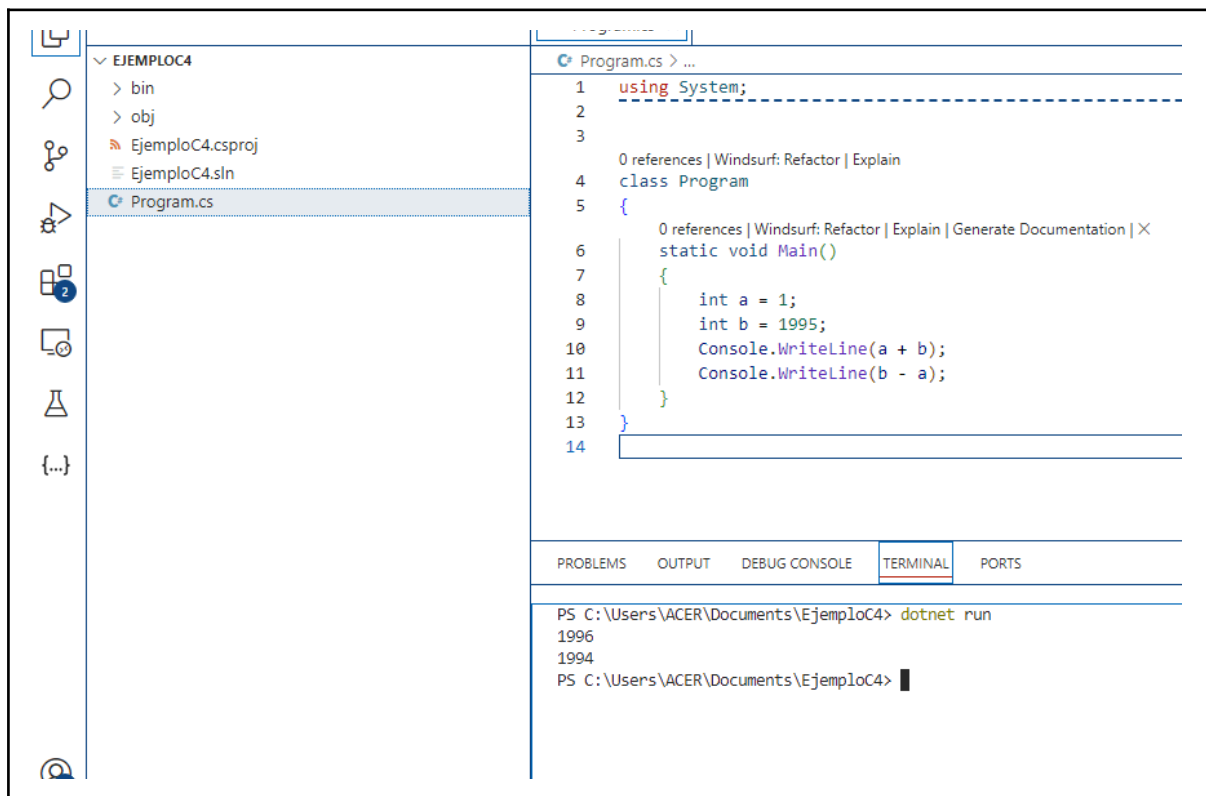
```
using System;                                     // EjemploC4.

class Program
{
    static void Main()
    {
        int a = 1;          // declaramos a como integer e inicializamos a=1
        int b = 0;          // declaramos b como integer e inicializamos b=0
        b=1995;              // cambiamos valor de b (no hace falta declarar
                             // que es integer porque ya lo hicimos en
                             // el paso anterior)
        Console.WriteLine(a + b);
        Console.WriteLine(b - a);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



En esta ocasión, podemos ver en el terminal que hemos sumado dos valores enteros almacenados en las variables **a** y **b**. Posteriormente hemos restado 1 (valor de **a**) al valor de **b**.

Preguntas: ¿Qué tipo de variables hemos utilizado? ¿Integer (entero) o String (cadena)? ¿Has ejecutado el programa? ¿Cuál ha sido el resultado?

Fin del Ejemplo C4.

Ejemplo C4pri. Sumas de dos valores con una función.

Objetivo: Aplicación que realiza cálculos y muestra los resultados en pantalla.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC4pri”:

```
cd documents
dotnet new console -o EjemploC4pri
cd EjemploC4pri
code .
```

Ahora reemplazaremos las líneas 1,2 y 3 del archivo [Program.cs](#) por el siguiente contenido:

```
                                                                    // EjemploC4pri.

using System;

class Program
{
    static int AddValues(int x, int y)
    {
        return x + y;
    }

    static void Main()
    {
        int a = 1;
        int b = 1995;

        Console.WriteLine("Suma: " + AddValues(a, b));
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal (CTRL+ñ) y teclearemos:

```
dotnet run
```

Ejemplo C4pub. Sumas de dos valores con una función (método) pública.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla con un método público.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC4pub”:

```
cd documents
dotnet new console -o EjemploC4pub
cd EjemploC4pub
code .
```

Ahora reemplazaremos el código del archivo [Program.cs](#) por lo siguiente:

```
using System;                                                    // EjemploC4pub.

// Este método (función) está dentro de la clase Helper
// Podemos llamarla Helper, Ayudame, Cálculos, etc..
// Nosotros elegimos el nombre en función del propósito.
// Al ser pública, se puede llamar desde cualquier
// parte del programa.
// El método debe estar siempre dentro de una clase.
public class Helper
{
    public static int AddValues(int x, int y)
    {
        return x + y;
    }
}

class Program
{
    static void Main()
    {
        int a = 1;
        int b = 1995;

        Console.WriteLine("Suma: " + Helper.AddValues(a, b));
        Console.WriteLine("Diferencia: " + (b - a));
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo y ejecutaremos el comando **dotnet run** en el terminal.

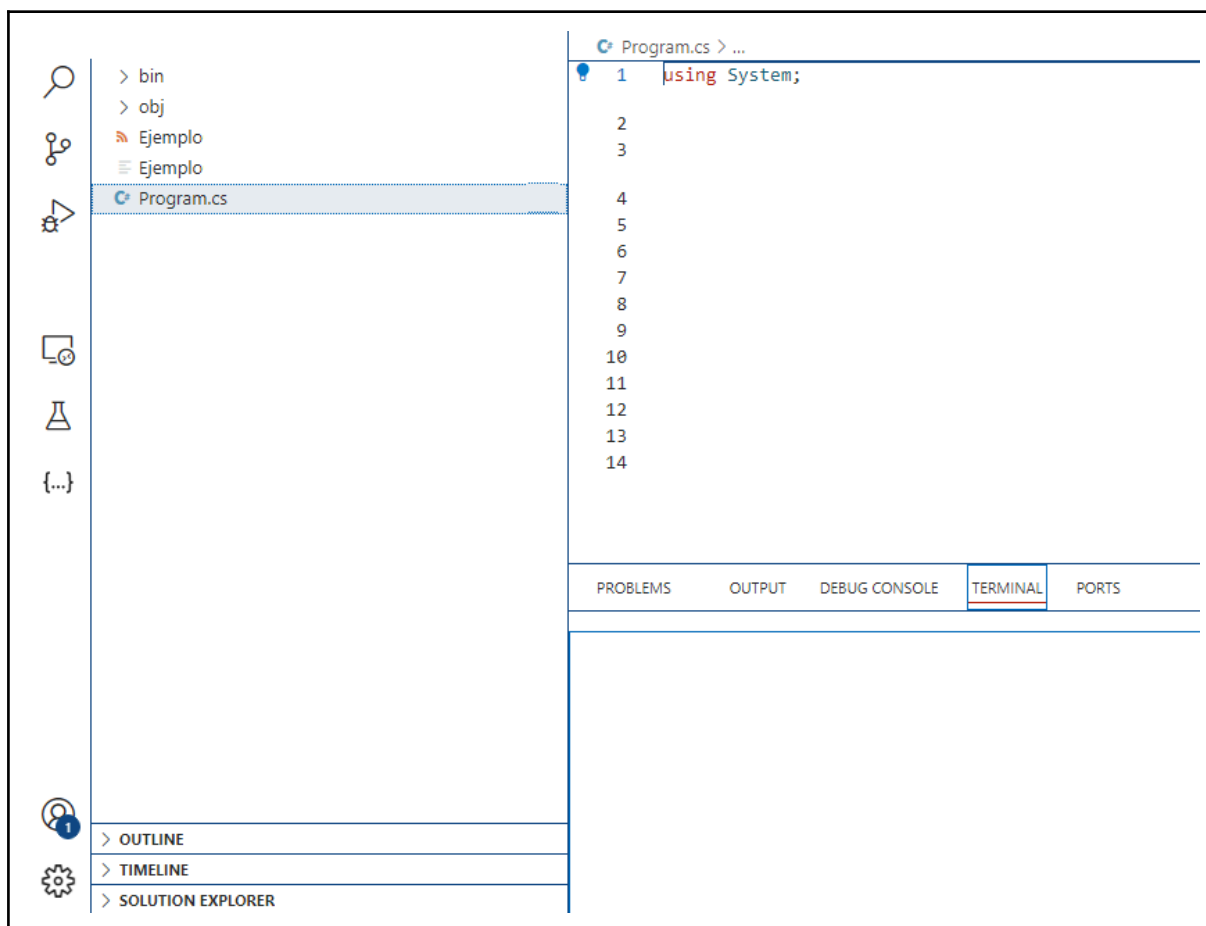
Ejemplo C4b. Cálculo del área de un rectángulo.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC4b”:

```
cd documents
dotnet new console -o EjemploC4b
cd EjemploC4b
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

```
using System;                                     // EjemploC4b.

class Program
{
    static void Main()
    {
        int area=1; string alturaInput="1"; string baseInput="1";
        Console.WriteLine(' ');

        Console.WriteLine("Vamos a calcular el área de un rectángulo.");

        Console.Write("Entre en metros, cuál es la longitud de la base: ");
        baseInput = Console.ReadLine();

        Console.Write("Entre en metros, cuál es la longitud de la altura: ");
        alturaInput = Console.ReadLine();

        Console.WriteLine("El área del rectángulo es:");

        area = int.Parse(baseInput) * int.Parse(alturaInput);

        Console.WriteLine(area);

        Console.WriteLine("metros cuadrados.");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

```

1  using System;
2  class Program
3  {
4      static void Main()
5      {
6          int area=1; string alturaInput="1"; string baseInput="1";
7          Console.WriteLine("");
8
9          Console.WriteLine("Vamos a calcular el área de un rectángulo.");
10
11         Console.Write("Entre en metros, cual es la longitud de la base: ");
12         baseInput = Console.ReadLine();
13         Console.Write("Entre en metros, cual es la longitud de la altura: ");
14         alturaInput = Console.ReadLine();
15         Console.WriteLine("El área del rectángulo es:");
16         area = int.Parse(baseInput) * int.Parse(alturaInput);
17         Console.WriteLine(area);

```

PROBLEMS 4 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

Vamos a calcular el área de un rectángulo.
Entre en metros, cual es la longitud de la base: 9
Entre en metros, cual es la longitud de la altura: 9
El área del rectángulo es:
81
metros cuadrados.
PS C:\Users\ACER\Documents\EjemploC4b>

```

En esta ocasión, podemos ver en el terminal que hemos multiplicado dos valores almacenados en las variables **baseInput** y **alturaInput**. Posteriormente hemos calculado el área del rectángulo multiplicando los dos valores anteriores.

También podemos ver en el terminal que, aunque el programa se ejecuta correctamente, tenemos cuatro errores leves.

Preguntas: ¿Cómo arreglarías los errores? ¿Qué tipo de variables hemos utilizado? ¿Integer (entero) o String (cadena)?

Fin del Ejemplo C4b.

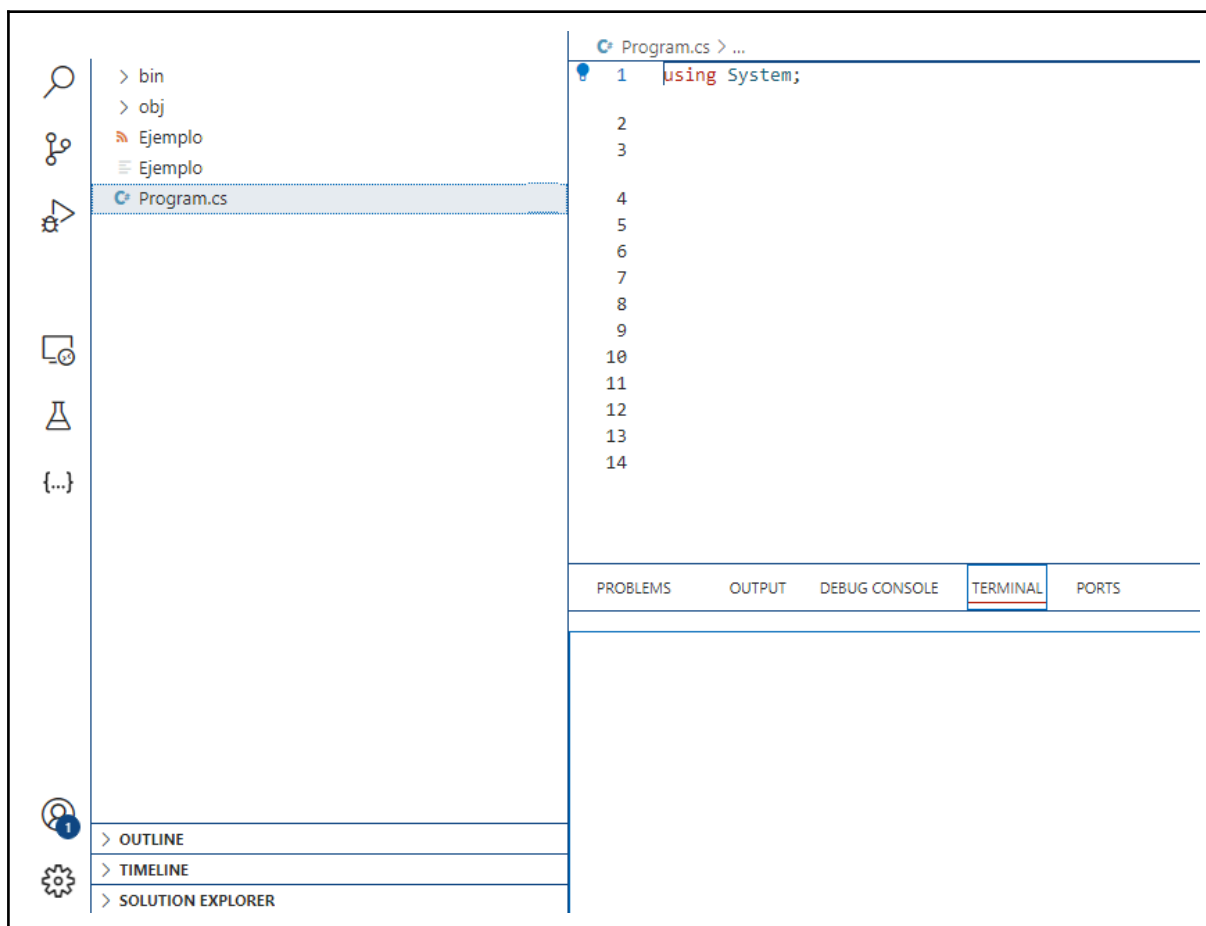
Ejemplo C5. Cálculo del área de un rectángulo.

Objetivo: Aplicación que realiza cálculos básicos y muestra los resultados en pantalla.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC5”:

```
cd documents
dotnet new console -o EjemploC5
cd EjemploC5
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System;                                     // EjemploC5
class Program
{
    static void Main()
    {
        Console.WriteLine("Triangle Area Calculator");

        // Entrar base
        Console.Write("Entre la base del triángulo: ");
        double baseLength = Convert.ToDouble(Console.ReadLine());

        // Entrar altura
        Console.Write("Entre la altura del triángulo: ");
        double height = Convert.ToDouble(Console.ReadLine());

        // Calcular area
        double area = 0.5 * baseLength * height;
        Console.WriteLine($"El área del triangulo es: {area}");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

The screenshot shows the Visual Studio Code interface with a C# file named `Program.cs` open. The Explorer pane on the left shows the project structure for `EJEMPLO5`, including `bin`, `obj`, `EjemploC5.csproj`, `EjemploC5.sln`, and `Program.cs`. The main editor displays the following code:

```

1 using System; // ejemploC5
2 class Program
3 {
4     static void Main()
5     {
6         Console.WriteLine("Triangle Area Calculator");
7         Console.Write("Entre la base del triangulo: ");
8         double baseLength = Convert.ToDouble(Console.ReadLine());
9
10        Console.Write("Entre la altura del triangulo: ");
11        double height = Convert.ToDouble(Console.ReadLine());
12
13        // Calcular area y mostrar resultado
14        double area = 0.5 * baseLength * height;
15        Console.WriteLine($"El area del triangulo es: {area}");
16    }
17 }

```

At the bottom, the TERMINAL pane shows the command `dotnet run` being executed, with the following output:

```

PS C:\Users\ACER\Documents\EjemploC5> dotnet run
Triangle Area Calculator
Entre la base del triangulo: 9
Entre la altura del triangulo: 8
El area del triangulo es: 36
PS C:\Users\ACER\Documents\EjemploC5>

```

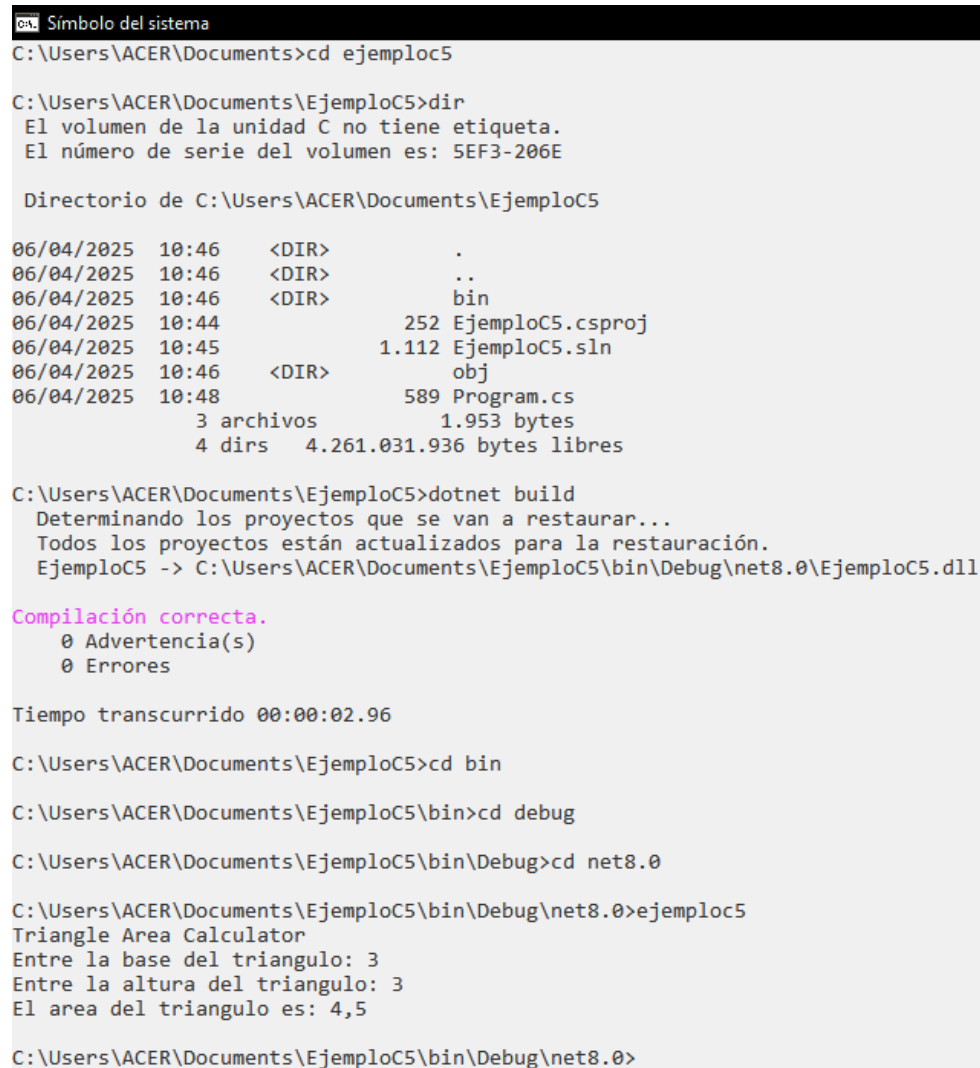
En esta ocasión, podemos ver en el terminal que hemos multiplicado los valores almacenados en las variables **baseLength** y **height**. Posteriormente hemos calculado el área del triángulo multiplicando los dos valores anteriores por **0.5**.

Preguntas: ¿Qué tipo de variables hemos utilizado? ¿Double (doble), Integer (entero) o String (cadena)?

A continuación compilaremos el archivo **EjemploC5.cs** para generar un archivo ejecutable con extensión **.exe** para ejecutarlo en cualquier ordenador equipado con Microsoft Windows 10 y VS Code con **net8.0**.

Ejecute la aplicación cmd (símbolo de sistema) y entre las siguientes órdenes:

```
cd Documents
cd EjemploC5
echo Ahora generaremos el archivo .exe con la orden build:
dotnet build
echo Ahora buscaremos el archivo .exe generado
cd bin
cd debug
cd net8.0
echo Ahora ejecutaremos el archivo .exe
ejemploC5.exe
```



```

C:\Users\ACER\Documents>cd ejemploC5

C:\Users\ACER\Documents\EjemploC5>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 5EF3-206E

Directorio de C:\Users\ACER\Documents\EjemploC5

06/04/2025  10:46    <DIR>          .
06/04/2025  10:46    <DIR>          ..
06/04/2025  10:46    <DIR>          bin
06/04/2025  10:44                252 EjemploC5.csproj
06/04/2025  10:45                1.112 EjemploC5.sln
06/04/2025  10:46    <DIR>          obj
06/04/2025  10:48                589 Program.cs
               3 archivos             1.953 bytes
               4 dirs  4.261.031.936 bytes libres

C:\Users\ACER\Documents\EjemploC5>dotnet build
Determinando los proyectos que se van a restaurar...
Todos los proyectos están actualizados para la restauración.
EjemploC5 -> C:\Users\ACER\Documents\EjemploC5\bin\Debug\net8.0\EjemploC5.dll

Compilación correcta.
    0 Advertencia(s)
    0 Errores

Tiempo transcurrido 00:00:02.96

C:\Users\ACER\Documents\EjemploC5>cd bin

C:\Users\ACER\Documents\EjemploC5\bin>cd debug

C:\Users\ACER\Documents\EjemploC5\bin\Debug>cd net8.0

C:\Users\ACER\Documents\EjemploC5\bin\Debug\net8.0>ejemploC5
Triangle Area Calculator
Entre la base del triangulo: 3
Entre la altura del triangulo: 3
El area del triangulo es: 4,5

C:\Users\ACER\Documents\EjemploC5\bin\Debug\net8.0>
  
```

Si queremos compilar el código del programa llamado **EjemploC5.cs** para crear un archivo ejecutable (con extensión .exe) con todas las dependencias y ficheros **.dll** dentro del propio fichero .exe generado, ejecute la aplicación cmd (símbolo de sistema) y entre las siguientes órdenes:

```
cd Documents

cd EjemploC5

echo Ahora generaremos el archivo .exe con la orden build:
dotnet publish -c Release -r win-x64 --self-contained true
/p:PublishSingleFile=true

echo Ahora buscaremos el archivo .exe generado
cd bin
cd release
cd net8.0
cd win-x64
cd publish

echo Ahora podemos ver y ejecutar el archivo .exe generado
ejemploC5.exe
```

Nota:

Este fichero .exe ya podremos ejecutarlo en cualquier ordenador sin necesidad de copiar los archivos **.dll** utilizados para su creación. El archivo **ejemploC5.exe** es ahora portable e independiente.

Ejemplo C6. Creación de un programa que hace una pregunta a los alumnos de una clase de informática en una escuela de programación (creación de un test básico sobre Microsoft Windows).

Objetivo: Entender la sentencia **if** del lenguaje C#.

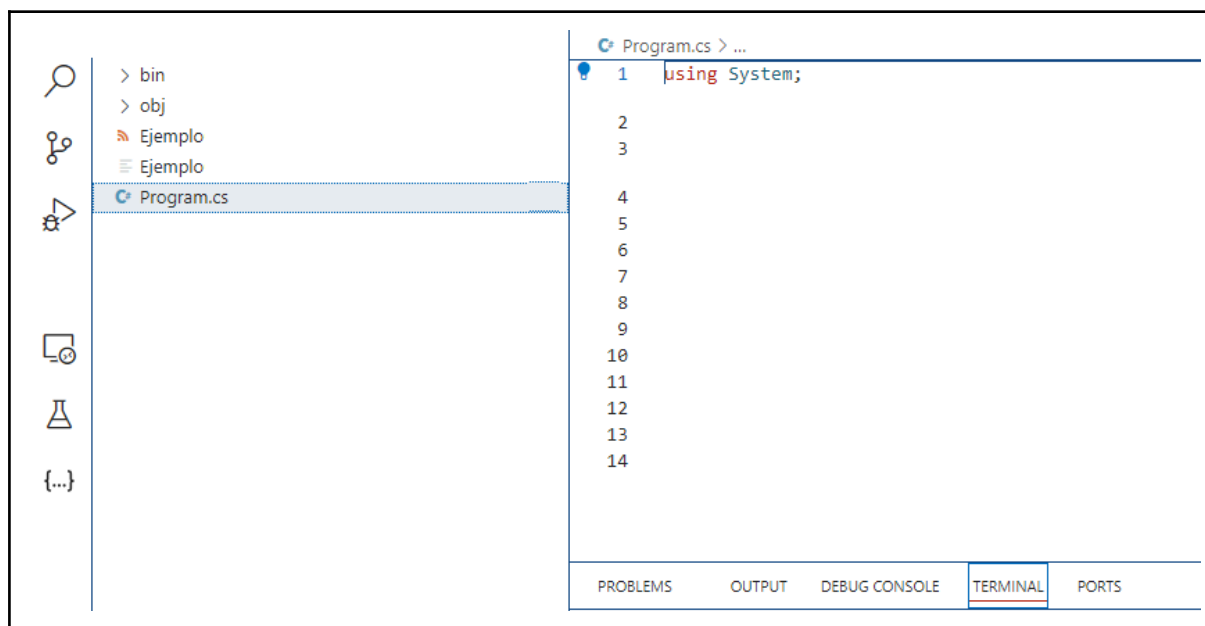
Una sentencia **if** esencialmente dice:

"Si la expresión evaluada, resulta ser verdadera (True), entonces ejecuta una vez el código en la expresión."

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado "EjemploC6":

```
cd documents
dotnet new console -o EjemploC6
cd EjemploC6
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System;                                     // EjemploC6.
class Program
{
    static void Main()
    {
        Console.Write("Dime tu nombre, por favor: ");
        string xname = Console.ReadLine();

        Console.WriteLine("Hola, " + xname + "!");
        Console.WriteLine("Vamos a hacer un test sobre Microsoft Windows");

        int puntos = 0;

        Console.Write("El comando COPY en Windows sirve para copiar archivos. (V/F)? : ");
        string xxx = Console.ReadLine();

        if (xxx == "V")
        {
            Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
            puntos = puntos + 1;
        }

        Console.WriteLine("Tu nota final en este test es:");
        Console.WriteLine(puntos);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a project named 'EJEMPLOC6' with files 'bin', 'obj', 'EjemploC6.csproj', 'EjemploC6.sln', and 'Program.cs'. The 'Program.cs' file is selected and open in the editor. The code in 'Program.cs' is as follows:

```

1 using System;
2 // EjemploC6.
3 class Program
4 {
5     static void Main()
6     {
7         Console.WriteLine("Dime tu nombre, por favor: ");
8         string xname = Console.ReadLine();
9
10        Console.WriteLine("Hola, " + xname + "!");
11
12        int puntos = 0;
13
14        Console.WriteLine("El comando COPY en Windows sirve para copiar archivos. (V/F)?");
15        string xxx = Console.ReadLine();
16
17    }
18 }

```

At the bottom of the window, the TERMINAL pane is active, showing the output of the program:

```

Dime tu nombre, por favor: Pepe
Hola, Pepe!
Vamos a hacer un test sobre Microsoft Windows
El comando COPY en Windows sirve para copiar archivos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
Tu nota final en este test es:
1
PS C:\Users\VACER\Documents\EjemploC6>

```

Nota: Es posible que en su terminal aparezcan dos problemas leves. Los trataremos en el siguiente ejemplo.

Preguntas: ¿Qué variables hemos utilizado para acumular puntos en nuestro test? ¿Cómo hemos comprobado que la respuesta es correcta? ¿Ya dominas la función **Console.ReadLine()**?

Fin del Ejemplo C6.

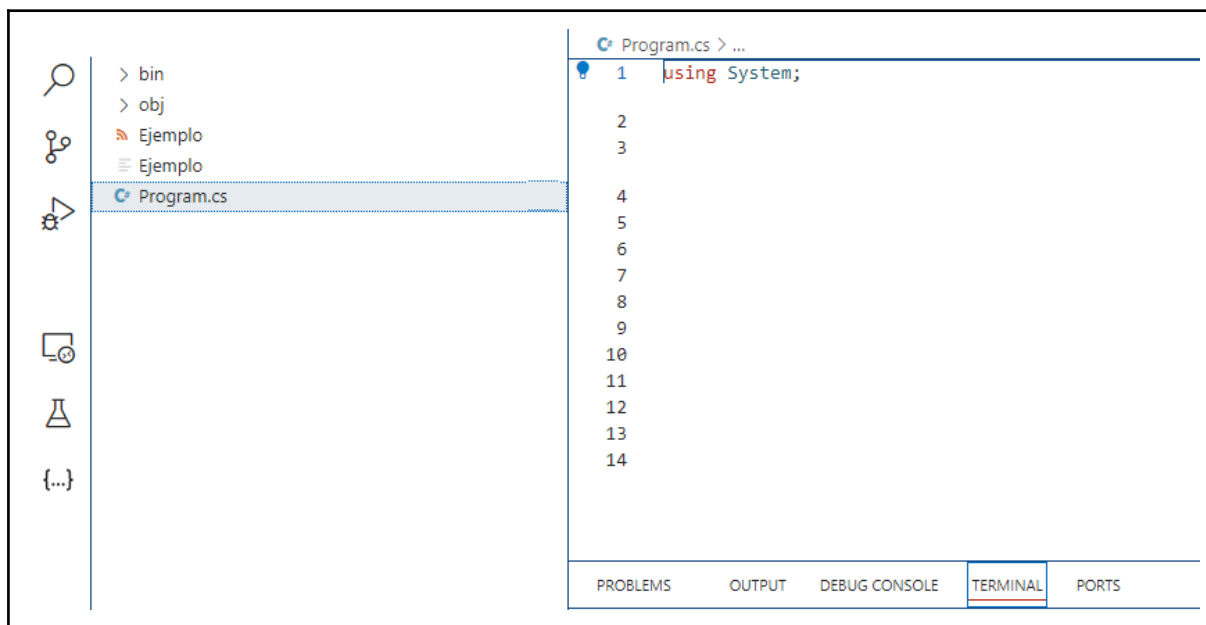
Ejemplo C6b. Creación de un programa que hace una pregunta a los alumnos de la asignatura lenguaje C# en una escuela de programación (creación de un test básico sobre Microsoft Windows).

Objetivo: Entender la sentencia **if** del lenguaje C# y validar las entradas “null” (vacías).

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC6b”:

```
cd documents
dotnet new console -o EjemploC6b
cd EjemploC6b
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; // EjemploC6b.

class Program
{
    static void Main()
    {
        Console.Write("Dime tu nombre, por favor: ");
        string? xname = Console.ReadLine();

        Console.WriteLine("Hola, " + xname + "!");
        Console.WriteLine("Vamos a hacer un test sobre Microsoft Windows");

        int puntos = 0;

        Console.Write("El comando COPY en Windows sirve para copiar archivos. (V/F)?: ");
        string? xxx = Console.ReadLine();

        if (string.IsNullOrEmpty(xxx))
        {
            Console.WriteLine("No entraste ninguna respuesta. Veamos otra pregunta.");
        }
        else
        {
            string respuesta = xxx.Trim().ToUpper();

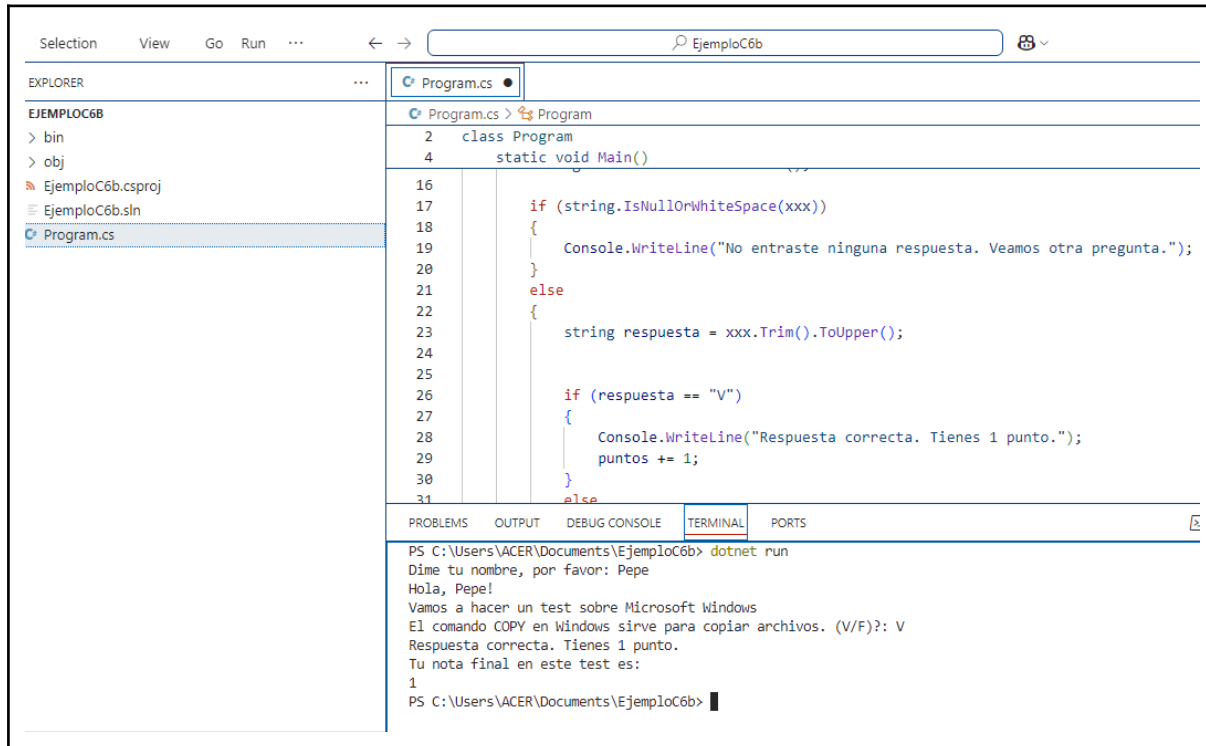
            if (respuesta == "V")
            {
                Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
                puntos += 1;
            }
            else
            {
                Console.WriteLine("Respuesta incorrecta.");
            }
        }

        Console.WriteLine("Tu nota final en este test es:");
        Console.WriteLine(puntos);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a project named 'EJEMPLOC6B' with files 'bin', 'obj', 'EjemploC6b.csproj', 'EjemploC6b.sln', and 'Program.cs'. The 'Program.cs' file is open in the editor. The code in 'Program.cs' is as follows:

```
2 class Program
4     static void Main()
{
16         if (string.IsNullOrEmpty(yyy))
17         {
18             Console.WriteLine("No entraste ninguna respuesta. Veamos otra pregunta.");
19         }
20     }
21     else
22     {
23         string respuesta = xxx.Trim().ToUpper();
24
25         if (respuesta == "V")
26         {
27             Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
28             puntos += 1;
29         }
30     }
31     else
```

At the bottom, the TERMINAL pane shows the output of the command 'dotnet run' executed in a PowerShell prompt. The output is:

```
PS C:\Users\ACER\Documents\EjemploC6b> dotnet run
Dime tu nombre, por favor: Pepe
Hola, Pepe!
Vamos a hacer un test sobre Microsoft Windows
El comando COPY en Windows sirve para copiar archivos. (V/F)? : V
Respuesta correcta. Tienes 1 punto.
Tu nota final en este test es:
1
PS C:\Users\ACER\Documents\EjemploC6b>
```

En el EjemploC6b hemos implementado la siguiente declaración para evitar errores leves en nuestro código: `string? xxx = Console.ReadLine()`
 ¿Cuál es la diferencia con `string xxx = Console.ReadLine()`?

Vamos a analizar la primera declaración para entender el concepto:

```
string? xxx = Console.ReadLine();
```

En esta línea, el símbolo `?` después del tipo `string` indica que la variable `xxx` puede tener un valor nulo (`null`). Esto forma parte de las características de referencia anulable (nullable reference types) introducidas en C# 8.0.

Cuando activas las referencias anulables en tu proyecto (habilitando `nullable enable`), se considera una buena práctica especificar si una variable de tipo referencia puede o no ser nula. Por lo tanto:

- `xxx` puede contener una cadena que el usuario ingrese desde el teclado.
- Si el usuario no ingresa nada o si `Console.ReadLine()` devuelve `null`, esto será aceptado sin problemas.

Vamos a analizar la segunda declaración para entender mejor el concepto:

```
string xxx = Console.ReadLine();
```

Aquí **no se permite** explícitamente que `xxx` sea nulo. Sin embargo:

- Si las referencias anulables están habilitadas, el compilador emitirá una advertencia, ya que `Console.ReadLine()` puede devolver `null` (por ejemplo, si el usuario simplemente presiona Enter).
- Si no tienes las referencias anulables activadas, `string` asume por defecto que `xxx` podría ser nulo, pero esto no está explicitado en el código.

En resumen:

- ``string?` deja claro que la variable **puede aceptar valores nulos** y lo maneja de manera explícita, lo que ayuda a prevenir errores en tiempo de ejecución relacionados con referencias nulas (`NullReferenceException`).
- ``string`` es más estricta cuando las referencias anulables están habilitadas, y **espera que no se le asigne un valor nulo**, aunque no siempre puede evitarlo de manera implícita.

Preguntas: ¿Qué variables hemos utilizado para acumular puntos en nuestro test? ¿Cómo hemos comprobado que la respuesta es correcta? ¿Ya dominas la función `Console.ReadLine( )`?

Nota: Novedades al ejecutar un bloque con condiciones en C# Versión 11.

Analice el siguiente ejemplo:

```
using System;

class Program
{
    static void Main()
    {
        /// Caso 1. Logical AND (&&)
        int age = 25;
        bool okTicket = true;

        if (age >= 18 && okTicket)
        {
            Console.WriteLine("Su ticket para entrar al cine es correcto!");
        }
        else
        {
            Console.WriteLine("Su ticket ha caducado. No puede pasar!");
        }

        /// Caso 2. Operación con bits: Bitwise AND operation [Novedad en C# 11]
        int num1 = 5; // Binary: 0101
        int num2 = 3; // Binary: 0011

        if ((num1 and num2) == 1) /// Operación con bits (bitwise operation)
        {
            Console.WriteLine("Bitwise AND result es igual a 1.");
        }
        else
        {
            Console.WriteLine("Bitwise AND result no es igual a 1.");
        }
    }
}
```

Observe que:

La primera declaración **if** utiliza **&&**, que verifica dos condiciones lógicas (**age** y **okTicket**) antes de otorgar acceso.

La segunda declaración **if** utiliza **and**, realizando una operación bit a bit en dos números (**num1** y **num2**), comprobando si su resultado de AND binario es igual a 1.

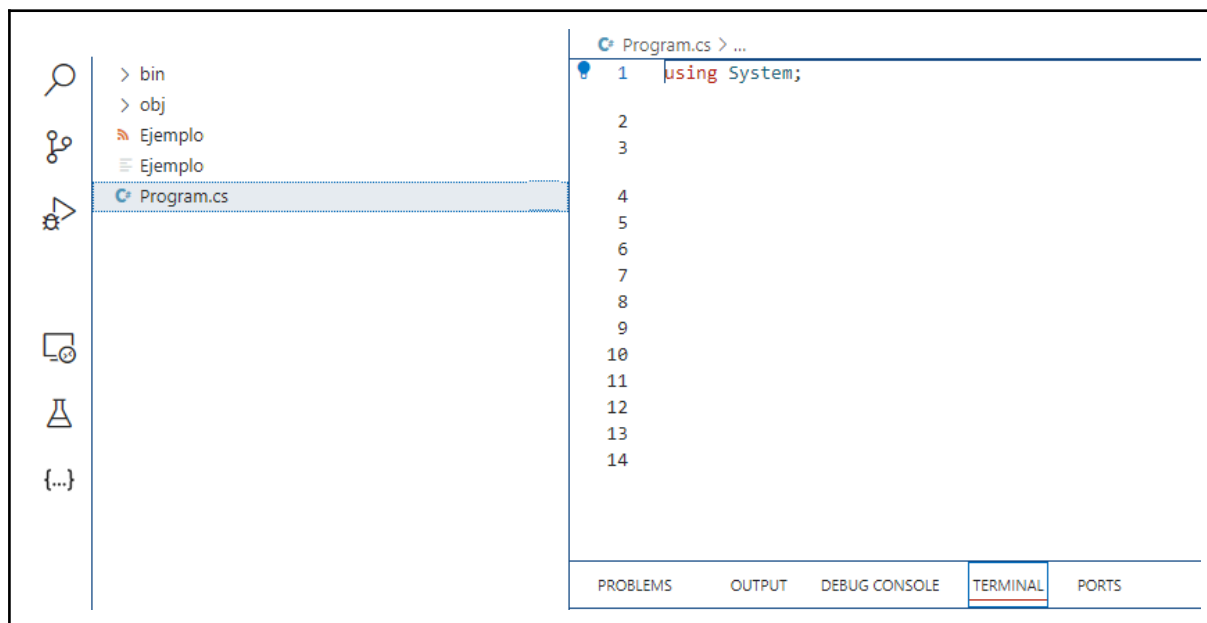
Ejemplo C7. Creación de un programa que hace preguntas a los alumnos de la asignatura lenguaje C# en una escuela de programación (creación de un test básico sobre Microsoft Windows).

Objetivo: Entender la sentencia **if** del lenguaje C#, validar las entradas “null” (vacías) y entender la secuencia de la programación estructurada.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC7”:

```
cd documents
dotnet new console -o EjemploC7
cd EjemploC7
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:


```

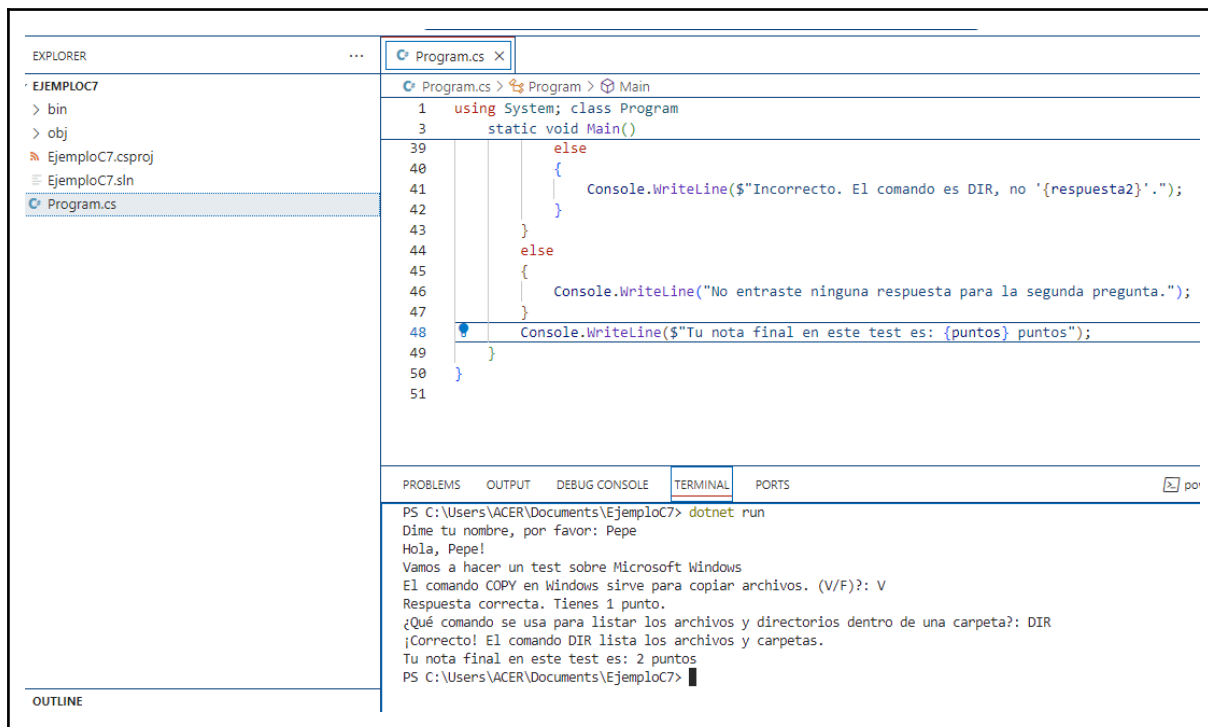
using System; class Program                                     /// Ejemplo C7.
{
    static void Main()
    {
        Console.WriteLine("Dime tu nombre, por favor: ");
        string? xname = Console.ReadLine(); int puntos = 0;
        Console.WriteLine("Hola, " + xname + "!");
        Console.WriteLine("Vamos a hacer un test sobre Microsoft Windows");
        Console.WriteLine("El comando COPY en Windows sirve para copiar archivos. (V/F)?: ");
        string? xxx = Console.ReadLine();
        if (string.IsNullOrEmpty(xxx))
        {
            Console.WriteLine("No entraste ninguna respuesta. Veamos otra pregunta.");
        }
        else
        {
            string respuesta = xxx.Trim().ToUpper();
            if (respuesta == "V")
            {
                Console.WriteLine("Correcto. Tienes 1 punto."); puntos += 1;
            }
            else
            {
                Console.WriteLine("Respuesta incorrecta.");
            }
        }
        Console.WriteLine("¿Qué comando se usa para listar los archivos y directorios dentro de una carpeta?: ");
        string? respuesta2 = Console.ReadLine();
        if (!string.IsNullOrEmpty(respuesta2))
        {
            string respuestaTrimmed = respuesta2.Trim().ToUpper();
            if (respuestaTrimmed == "DIR")
            {
                Console.WriteLine("Correcto. El comando DIR lista los archivos y carpetas."); puntos += 1;
            }
            else
            {
                Console.WriteLine($"Incorrecto. El comando es DIR, no '{respuesta2}'.");
            }
        }
        else
        {
            Console.WriteLine("Respuesta inválida para la segunda pregunta.");
        }
        Console.WriteLine("Tu nota final en este test es:"); Console.WriteLine(puntos);
    }
}

```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a project named 'EJEMPLOC7' with files 'bin', 'obj', 'EjemploC7.csproj', 'EjemploC7.sln', and 'Program.cs'. The main editor displays the code in 'Program.cs', which includes a static void Main() method with several Console.WriteLine statements. The bottom pane shows the Terminal output, which displays the execution of the program, including prompts for a name and a test about Windows commands, resulting in a final score of 2 points.

```
1 using System; class Program
3 static void Main()
39 else
40 {
41     Console.WriteLine($"Incorrecto. El comando es DIR, no '{respuesta2}'.");
42 }
43 }
44 else
45 {
46     Console.WriteLine("No entraste ninguna respuesta para la segunda pregunta.");
47 }
48 Console.WriteLine($"Tu nota final en este test es: {puntos} puntos");
49 }
50 }
51
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\ACER\Documents\EjemploC7> dotnet run
Dime tu nombre, por favor: Pepe
Hola, Pepe!
Vamos a hacer un test sobre Microsoft Windows
El comando COPY en Windows sirve para copiar archivos. (V/F)? : V
Respuesta correcta. Tienes 1 punto.
¿Qué comando se usa para listar los archivos y directorios dentro de una carpeta?: DIR
¡Correcto! El comando DIR lista los archivos y carpetas.
Tu nota final en este test es: 2 puntos
PS C:\Users\ACER\Documents\EjemploC7>

Observe que el código se ejecuta perfectamente y el usuario obtiene dos puntos al responder las preguntas correctamente.

Ejemplo C8. Cálculo del volumen de un cubo.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla implementando una función (llamado “method” [método] en C#) definida por el usuario.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC8”:

```
cd documents
dotnet new console -o EjemploC8
cd EjemploC8
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System;                                // Ejemplo C8.

class Program
{
    // definición de funciones (methods)
    static int Multiplica(int a, int b, int c)
    {
        Console.WriteLine("Multiplicaremos los siguientes valores:");
        Console.WriteLine(a + ", " + b + ", " + c);
        int resultado = a * b * c;
        return resultado;
    }

    // - - - - -

    // programa principal (main program)
    static void Main()
    {
        string ax = "Vamos a calcular el volumen de un cubo (poliedro regular).";
        Console.WriteLine(ax);
        string bx = "Todas las aristas miden lo mismo -en metros-";
        Console.WriteLine(bx);

        int total = Multiplica(3, 3, 3);
        Console.WriteLine("El total es:");
        Console.WriteLine(total);
        Console.WriteLine("metros cubicos.");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

The screenshot shows the Visual Studio Code interface with the following components:

- EXPLORER:** Displays the project structure for 'EJEMPLO8', including 'bin', 'obj', 'EjemploC8.csproj', 'EjemploC8.sln', and the selected 'Program.cs'.
- Program.cs:** Contains the following code:


```

1  using System;
2
3  0 references | Windsurf: Refactor | Explain
4  class Program
5  {
6      // definicion de funciones
7      1 reference | Windsurf: Refactor | Explain | X
8      static int Multiplica(int a, int b, int c)
9      {
10         Console.WriteLine("Multiplicaremos los siguientes valores:");
11         Console.WriteLine(a + ", " + b + ", " + c);
12         int resultado = a * b * c;
13         return resultado;
14     }
15
16     // programa principal (main program)
17     0 references | Windsurf: Refactor | Explain | X
18     static void Main()
19     {
20         // ...
21     }
22 }
      
```
- TERMINAL:** Shows the command 'dotnet run' and its output:


```

PS C:\Users\ACER\Documents\EjemploC8> dotnet run
Vamos a calcular el volumen de un cubo (poliedro regular).
Todas las aristas miden lo mismo -en metros-
Multiplicaremos los siguientes valores:
3, 3, 3
El total es:
27
metros cubicos.
PS C:\Users\ACER\Documents\EjemploC8>
      
```
- Bottom Panel:** Includes tabs for 'OUTLINE' and 'TIMELINE'.

Observe que el código se ejecuta perfectamente y el usuario obtiene el total de forma correcta. La función multiplica los tres valores pasados como parámetros. En este caso, la función ha multiplicado 3 x 3 x 3, cuyo resultado es 27.

Recuerde que un **método** (method) en C# es un bloque de código que realiza una tarea específica con un objeto o una clase. Podríamos decir que su equivalente en Python es la función definida por el usuario. En los siguientes ejemplos hablaremos de funciones y métodos indistintamente.

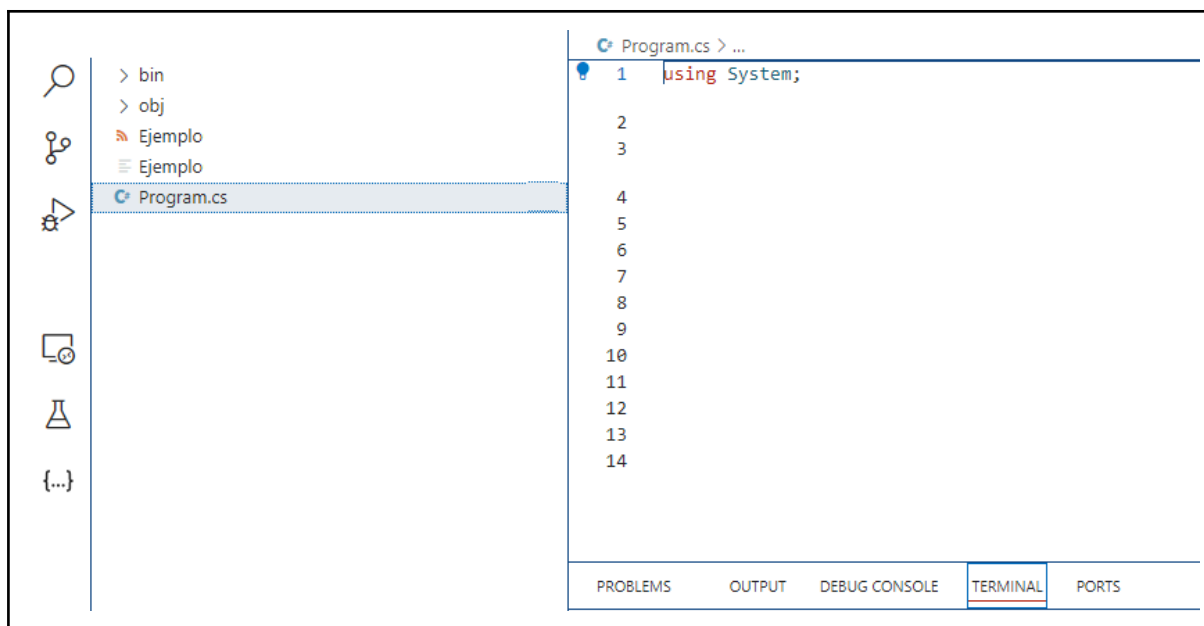
Ejemplo C9. Creación de un test básico sobre Microsoft Windows (implementando funciones definidas por el usuario).

Objetivo: Entender el funcionamiento de una función (método) definida por el usuario. Entender cómo se define una función y llamada a la función definida.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC9”:

```
cd documents
dotnet new console -o EjemploC9
cd EjemploC9
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program          // Ejemplo C9
{
    static void Main()
    {
        Console.Write("Dime tu nombre, por favor: ");
        string xname = Console.ReadLine();
        Console.WriteLine("Hola, " + xname + "!");
        string a = "Vamos a hacer un test de dos preguntas sobre";
        string b = " Microsoft Windows.";
        Console.WriteLine(a + b);
        Examen();
        Console.WriteLine("Fin de la prueba.");
    }
    // -----
    static void Examen()          // definimos la función Examen
    {
        int puntos = 0;

        // PREGUNTA 1
        Console.Write("El comando COPY en Windows sirve para copiar archivos. (V/F)? : ");
        string xxx = Console.ReadLine();
        if (xxx == "V")
        {
            Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
            puntos += 1;
        }

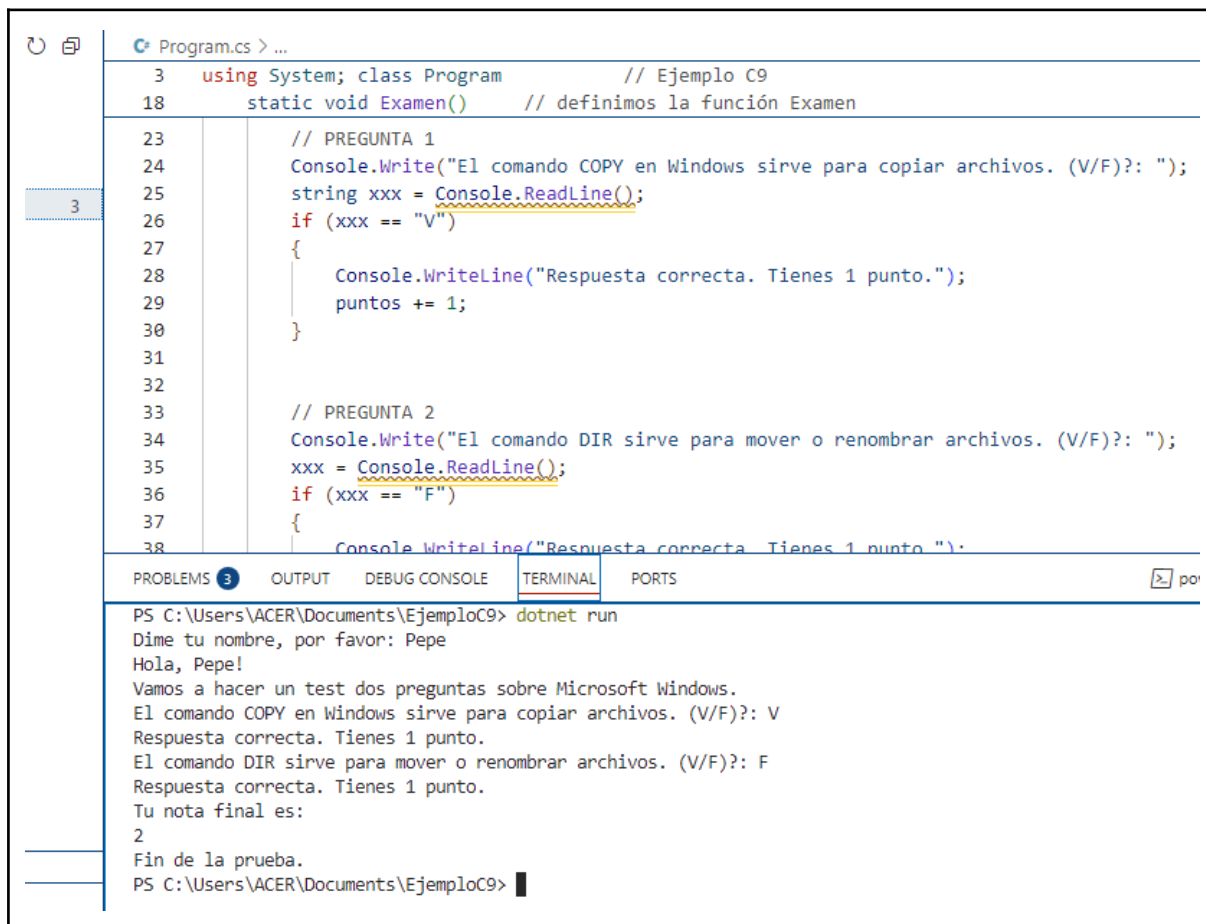
        // PREGUNTA 2
        Console.Write("El comando DIR sirve para mover o renombrar archivos. (V/F)? : ");
        xxx = Console.ReadLine();
        if (xxx == "F")
        {
            Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
            puntos += 1;
        }

        Console.WriteLine("Tu nota final es:");
        Console.WriteLine(puntos);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



The screenshot shows the Visual Studio Code interface. The top pane displays a C# file named `Program.cs` with the following code:

```

3  using System; class Program          // Ejemplo C9
18  static void Examen()                // definimos la función Examen

23      // PREGUNTA 1
24      Console.Write("El comando COPY en Windows sirve para copiar archivos. (V/F)? ");
25      string xxx = Console.ReadLine();
26      if (xxx == "V")
27      {
28          Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
29          puntos += 1;
30      }

31
32
33      // PREGUNTA 2
34      Console.Write("El comando DIR sirve para mover o renombrar archivos. (V/F)? ");
35      xxx = Console.ReadLine();
36      if (xxx == "F")
37      {
38          Console.WriteLine("Respuesta correcta. Tienes 1 punto.");

```

The bottom pane shows the **TERMINAL** output:

```

PS C:\Users\ACER\Documents\EjemploC9> dotnet run
Dime tu nombre, por favor: Pepe
Hola, Pepe!
Vamos a hacer un test dos preguntas sobre Microsoft Windows.
El comando COPY en Windows sirve para copiar archivos. (V/F)? V
Respuesta correcta. Tienes 1 punto.
El comando DIR sirve para mover o renombrar archivos. (V/F)? F
Respuesta correcta. Tienes 1 punto.
Tu nota final es:
2
Fin de la prueba.
PS C:\Users\ACER\Documents\EjemploC9>

```

Observe que el código se ejecuta perfectamente y el usuario obtiene la nota final de forma correcta cuando responde a las preguntas correctamente.

Reto: Modifique el código para que no aparezca ningún error leve.

Algunas claves eliminar errores leves:

- ``string?`` deja claro que la variable **puede aceptar valores nulos** y lo maneja de manera explícita, lo que ayuda a prevenir errores en tiempo de ejecución relacionados con referencias nulas (`NullReferenceException`).
- ``string`` es más estricta cuando las referencias anulables están habilitadas, y **espera que no se le asigne un valor nulo**, aunque no siempre puede evitarlo de manera implícita.

Ejemplo C10. Creación de un test sobre Microsoft Windows (implementando dos funciones definidas por el usuario).

Objetivo: Entender el funcionamiento de una función definida por el usuario. Llamada a varias funciones desde el programa principal.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC10”:

```
cd documents
dotnet new console -o EjemploC10
cd EjemploC10
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program          // Ejemplo C10
{
    static void Main()
    {
        Console.Write("Dime tu nombre, por favor: ");
        string xname = Console.ReadLine();
        Console.WriteLine("Hola, " + xname + "!");
        string a = "Vamos a hacer un test de dos preguntas sobre";
        string b = " Microsoft Windows.";
        Console.WriteLine(a + b);
        Examen();
        Felicitar();
        Console.WriteLine("Fin de la prueba.");
    }

    // -----
    static void Felicitar()          // definimos la función Felicitar
    {
        string feliz_Msg = "Felicitaciones. Siga estudiando así!";
        Console.WriteLine(feliz_Msg);
    }

    // -----
    static void Examen()             // definimos la función Examen
    {
        int puntos = 0;

        // PREGUNTA 1
        Console.Write("El comando TREE en Windows sirve para copiar archivos. (V/F)?: ");
        string xxx = Console.ReadLine();
        if (xxx == "F")
        {
            Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
            puntos += 1;
        }

        // PREGUNTA 2
        Console.Write("El comando DEL sirve para mover o renombrar archivos. (V/F)?: ");
        xxx = Console.ReadLine();
        if (xxx == "F")
        {
            Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
            puntos += 1;
        }

        Console.WriteLine("Tu nota final es:");
        Console.WriteLine(puntos);
    }
}
```

Al terminar de escribir el código, deberá grabar su archivo. A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

```

1 using System; class Program
2 {
3     static void Main()
4     {
5         string xname;
6         Console.Write("Dime tu nombre, por favor: ");
7         xname = Console.ReadLine();
8         Console.WriteLine("Hola, " + xname + "!");
9
10        string a = "Vamos a hacer un test de dos preguntas sobre";
11        string b = " Microsoft Windows.";
12        Console.WriteLine(a + b);
13
14        examen();
15        felicitacion();
16        Console.WriteLine("Fin de la prueba.");

```

Dime tu nombre, por favor: Pablo
 Hola, Pablo!
 Vamos a hacer un test de dos preguntas sobre Microsoft Windows.
 El comando TREE en Windows sirve para copiar archivos. (V/F)? : F
 Respuesta correcta. Tienes 1 punto.
 El comando DEL sirve para mover o renombrar archivos. (V/F)? : F
 Respuesta correcta. Tienes 1 punto.
 Tu nota final es:
 2
 Felicitaciones, siga estudiando así.
 Estamos orgullosos de usted.
 Fin de la prueba.
 PS C:\Users\ACER\Documents\EjemploC10>
 PS C:\Users\ACER\Documents\EjemploC10>

Observe que el código se ejecuta perfectamente y el usuario obtiene la nota final de forma correcta cuando responde a las preguntas correctamente.

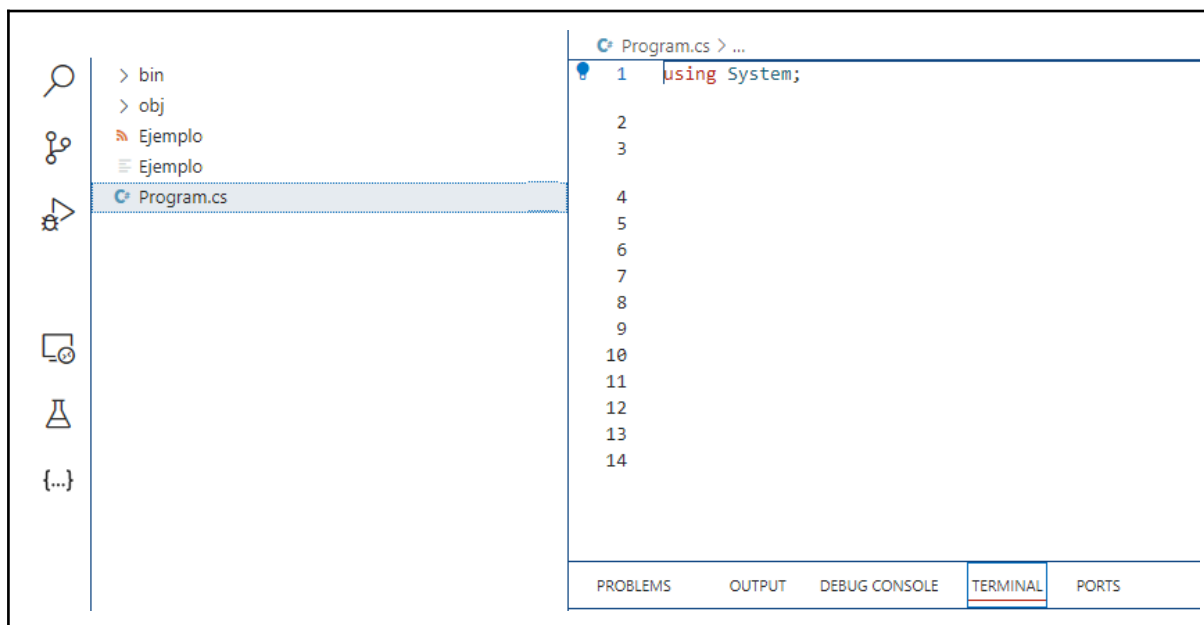
Ejemplo C11. Creación de un test sobre Microsoft Windows (implementando funciones con parámetros).

Objetivo: Entender el funcionamiento de una función definida por el usuario utilizando parámetros.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC11”:

```
cd documents
dotnet new console -o EjemploC11
cd EjemploC11
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program          // Ejemplo C11
#### Definición de dos funciones: función examen() y función felicitar()
{
    // - - - - -
    static void Main()
    {
        Console.WriteLine("Dime tu nombre, por favor: ");
        string xname = Console.ReadLine();
        Console.WriteLine("Hola, " + xname + "!");
        string a = "Vamos a hacer un test de dos preguntas sobre";
        string b = " Microsoft Windows.";
        Console.WriteLine(a + b);
        Examen();
        Console.WriteLine("Fin de la prueba.");
    }
    // - - - - -
    static void Examen()
    {
        int puntos = 0;
        // PREGUNTA 1
        Console.WriteLine("El comando FORMAT en Windows sirve para copiar archivos. (V/F)?");
        string xxx = Console.ReadLine();
        if (xxx == "F")
        {
            Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
            puntos++;
        }
        // PREGUNTA 2
        Console.WriteLine("El comando REN sirve para copiar archivos. (V/F)? ");
        xx = Console.ReadLine();
        if (xx == "F")
        {
            Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
            puntos++;
        }
        Felicitacion(puntos);
    }
    // - - - - -
    static void Felicitacion(int p)
    {
        Console.WriteLine("Felicitaciones, siga estudiando asi.");
        Console.WriteLine("Estamos orgullosos de usted.");
        Console.WriteLine("Tu nota final es:");
        Console.WriteLine(p);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

The screenshot shows the Visual Studio Code interface with a C# file named Program.cs open. The code defines a class Program with a static method Examen() and a static method Felicitation(int p). The Examen() method prompts the user for their name, asks two questions about Windows commands (FORMAT and REN), and calculates a final score based on correct answers. The Felicitation method displays the score and congratulatory messages.

```

1  using System; class Program
15  static void Examen()
31      Console.WriteLine("Respuesta correcta. Tienes 1 punto.");
32      puntos++;
33  }
34      Felicitation(puntos);
35  }
36
37  1 reference | Windsurf: Refactor | Explain | Generate Documentation | X
38  static void Felicitation(int p)
39  {
40      Console.WriteLine("Felicitaciones, siga estudiando así.");
41      Console.WriteLine("Estamos orgullosos de usted.");
42      Console.WriteLine("Tu nota final es:");
43      Console.WriteLine(p);
44  }

```

The terminal output shows the program's execution:

```

Dime tu nombre, por favor: Pablo
Hola, Pablo!
Vamos a hacer un test de dos preguntas sobre Microsoft Windows.
El comando FORMAT en Windows sirve para copiar archivos. (V/F)? F
Respuesta correcta. Tienes 1 punto.
El comando REN sirve para copiar archivos. (V/F)? F
Respuesta correcta. Tienes 1 punto.
Felicitaciones, siga estudiando así.
Estamos orgullosos de usted.
Tu nota final es:
2
Fin de la prueba.
PS C:\Users\ACER\Documents\EjemploC11>
PS C:\Users\ACER\Documents\EjemploC11>

```

Observe que el código se ejecuta perfectamente y el usuario obtiene la nota final de forma correcta cuando responde a las preguntas correctamente.

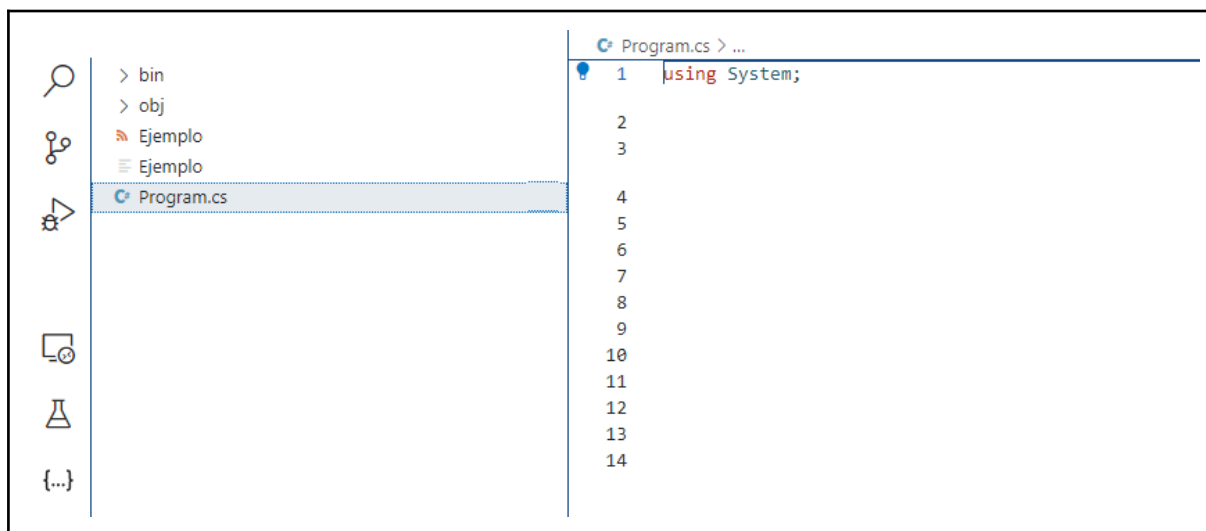
Ejemplo C12. Creación de una función que suma 4 números (implementando una función con parámetros).

Objetivo: Entender el funcionamiento de una función definida por el usuario utilizando parámetros.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC12”:

```
cd documents
dotnet new console -o EjemploC12
cd EjemploC12
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program           // Ejemplo C12

{
    // Definición de funciones
    static int Sumar(params int[] args)
    {
        int resultado = 0;
        foreach (var arg in args)
        {
            resultado += arg;
        }
        Console.WriteLine("Sumaremos los siguientes valores:");
        Console.WriteLine(string.Join(" ", args));
        return resultado;
    }
    // - - - - -

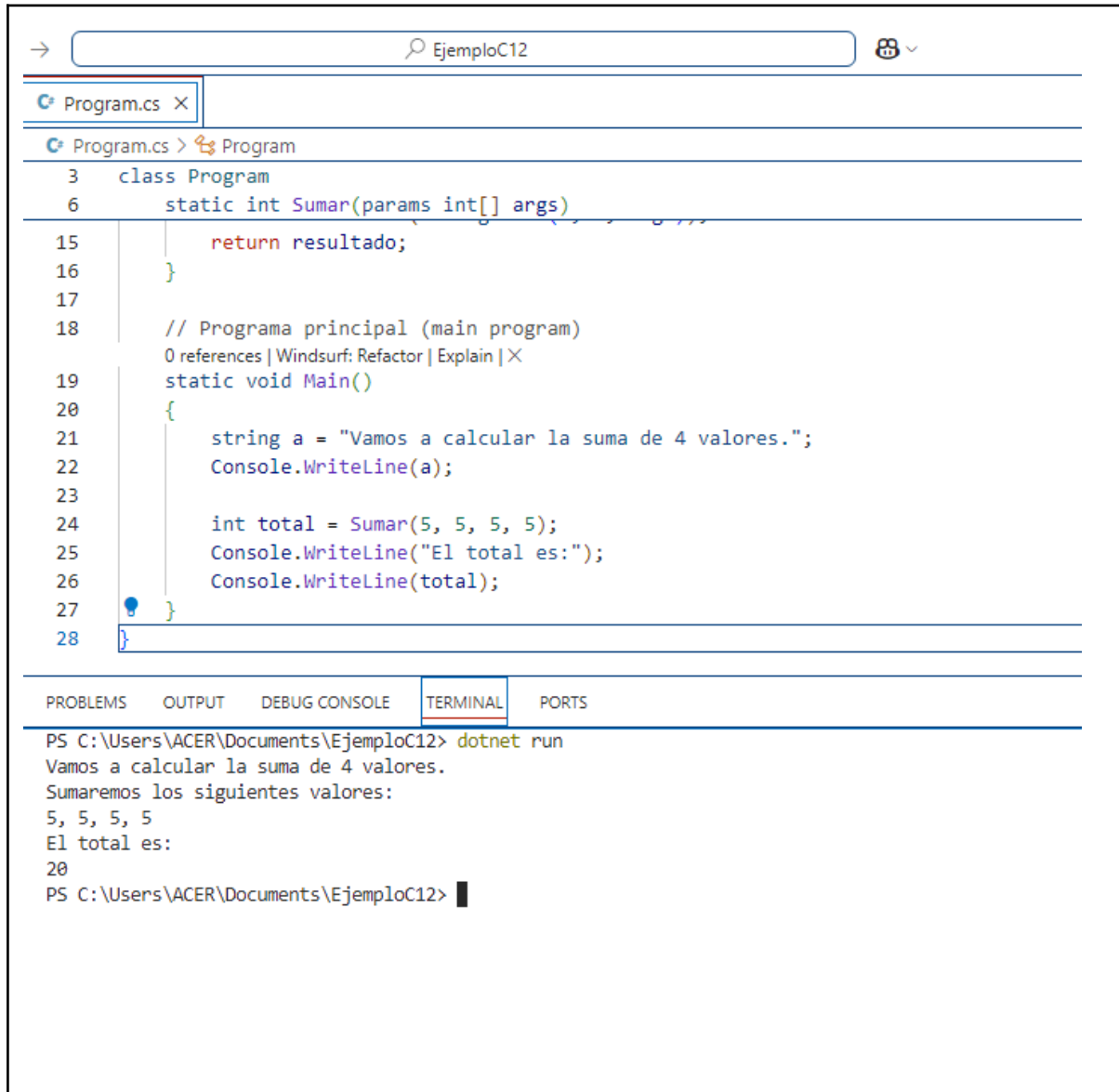
    // Programa principal (Main program)
    static void Main()
    {
        string ax = "Vamos a calcular la suma de 4 valores.";
        Console.WriteLine(ax);

        int total = Sumar(5, 5, 5, 5);
        Console.WriteLine("El total es:");
        Console.WriteLine(total);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



The screenshot shows the Visual Studio Code interface. The top bar displays the file name 'EjemploC12'. The editor window shows a C# file named 'Program.cs' with the following code:

```
3  class Program
6      static int Sumar(params int[] args)
15      {
16          return resultado;
17      }
18      // Programa principal (main program)
19      static void Main()
20      {
21          string a = "Vamos a calcular la suma de 4 valores.";
22          Console.WriteLine(a);
23
24          int total = Sumar(5, 5, 5, 5);
25          Console.WriteLine("El total es:");
26          Console.WriteLine(total);
27      }
28  }
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\ACER\Documents\EjemploC12> dotnet run
Vamos a calcular la suma de 4 valores.
Sumaremos los siguientes valores:
5, 5, 5, 5
El total es:
20
PS C:\Users\ACER\Documents\EjemploC12>
```

Observe que el código se ejecuta perfectamente y el usuario obtiene como resultado la suma de los cuatro valores enviados como parámetros a la función Sumar. El resultado de la suma es: 20.

Observe que hemos creado una función en el bloque “definición de funciones”:

```
static int Sumar(params int[] args)
    . . .
```

Acto seguido, para que el programa haga correctamente su trabajo, simplemente escribimos el nombre seguido de paréntesis en el “programa principal” de la función que queremos ejecutar.

Para calcular la suma de cuatro valores, llamaremos a la función de la siguiente forma (mencionando que **total** será “**int**”, un valor entero):

```
int total= Sumar(5,5,5,5)
```

Para imprimir en la consola el valor del total de la suma, escribiremos:

```
Console.WriteLine(total) ;
```

Reto: Transforme el EjemploC12 para que la función **Sumar** calcule la suma de tres valores sin utilizar un bucle en el código.

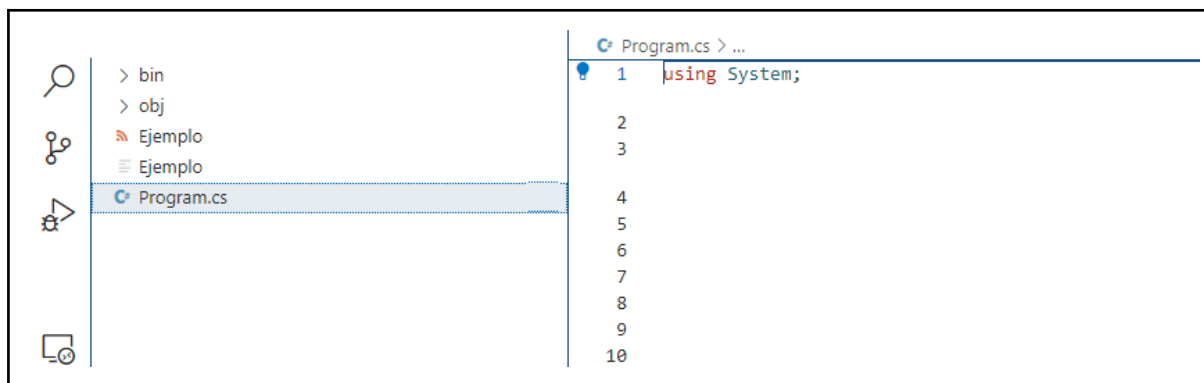
Ejemplo C13. Cálculo del volumen de un cubo.

Objetivo: Crear una aplicación que realiza cálculos básicos y muestra los resultados en pantalla utilizando una función.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC13”:

```
cd documents  
dotnet new console -o EjemploC13  
cd EjemploC13  
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program           // Ejemplo C13

{
    // Definición de funciones

    static int Multiplifica(int a, int b, int c)
    {
        Console.WriteLine("Multiplicaremos los siguientes valores:");
        Console.WriteLine($"{a} {b} {c}");
        int resultado = a * b * c;
        return resultado;
    }

    // - - - - -

    // Programa principal (Main program)

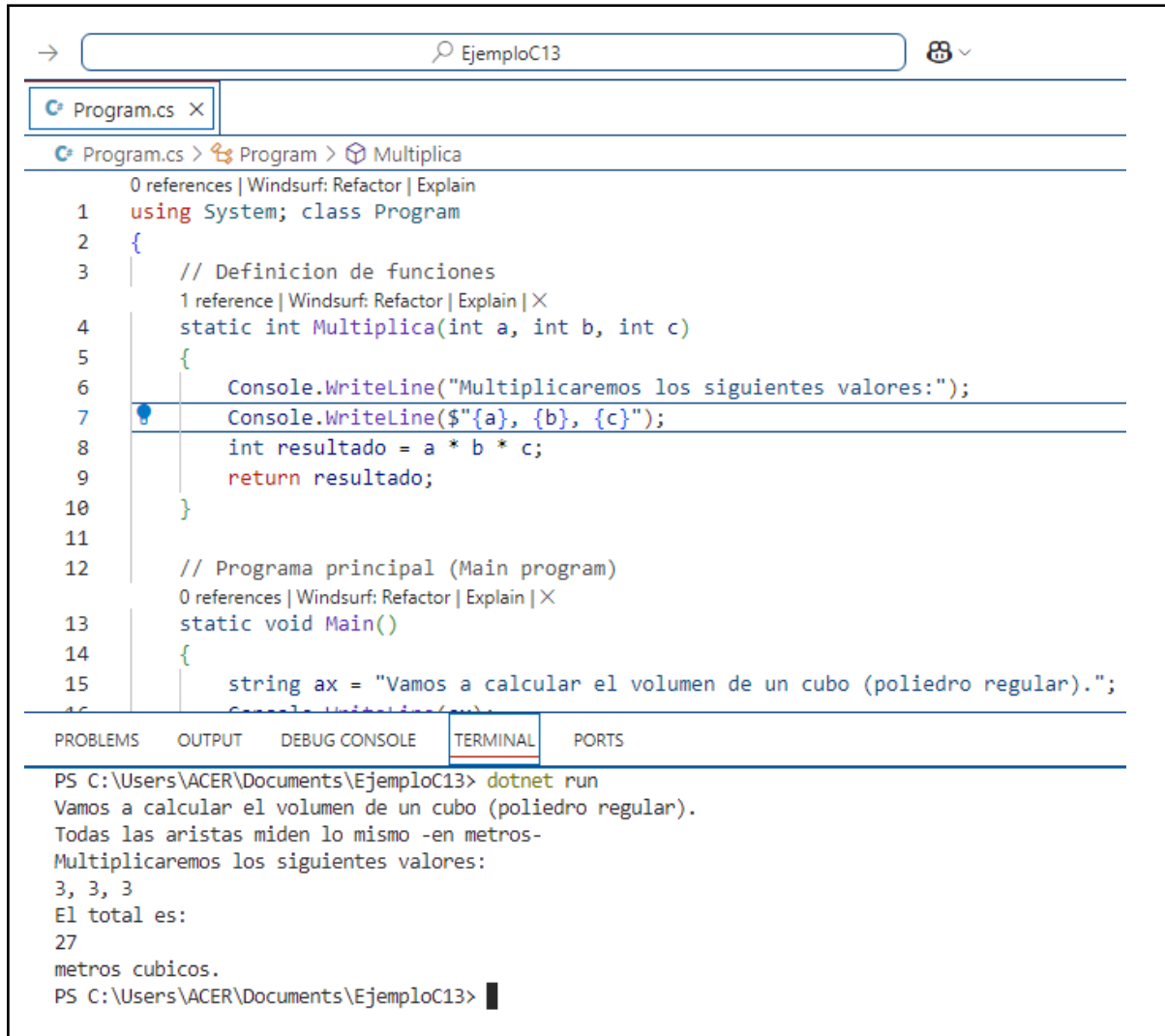
    static void Main()
    {
        string ax = "Vamos a calcular el volumen de un cubo (poliedro regular).";
        Console.WriteLine(ax);
        string bx = "Todas las aristas miden lo mismo -en metros-";
        Console.WriteLine(bx);

        int total = Multiplifica(3, 3, 3);
        Console.WriteLine("El total es:");
        Console.WriteLine(total);
        Console.WriteLine("metros cubicos.");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run



The screenshot shows the Visual Studio Code interface. The top bar indicates the file 'EjemploC13'. The editor displays a C# file named 'Program.cs' with the following code:

```

1  using System; class Program
2  {
3      // Definicion de funciones
4      static int Multiplica(int a, int b, int c)
5      {
6          Console.WriteLine("Multiplicaremos los siguientes valores:");
7          Console.WriteLine($"{a}, {b}, {c}");
8          int resultado = a * b * c;
9          return resultado;
10     }
11
12     // Programa principal (Main program)
13     static void Main()
14     {
15         string ax = "Vamos a calcular el volumen de un cubo (poliedro regular).";
16         Console.WriteLine(ax);
17     }
18 }

```

Below the editor, the 'TERMINAL' tab is active, showing the output of the command 'dotnet run' executed in the directory 'PS C:\Users\ACER\Documents\EjemploC13>'. The output is:

```

PS C:\Users\ACER\Documents\EjemploC13> dotnet run
Vamos a calcular el volumen de un cubo (poliedro regular).
Todas las aristas miden lo mismo -en metros-
Multiplicaremos los siguientes valores:
3, 3, 3
El total es:
27
metros cubicos.
PS C:\Users\ACER\Documents\EjemploC13>

```

Observe que el código se ejecuta perfectamente y el usuario obtiene como resultado el producto de los tres valores enviados como parámetros a la función **Multiplica**. El resultado de la suma es: 27.

Preguntas: ¿Qué variables hemos utilizado? ¿Ya dominas la técnica para imprimir una cadena de texto y una variable numérica en una misma sentencia **Console.WriteLine()**? Intenta optimizar las 3 últimas líneas del código.

Fin del Ejemplo C13.

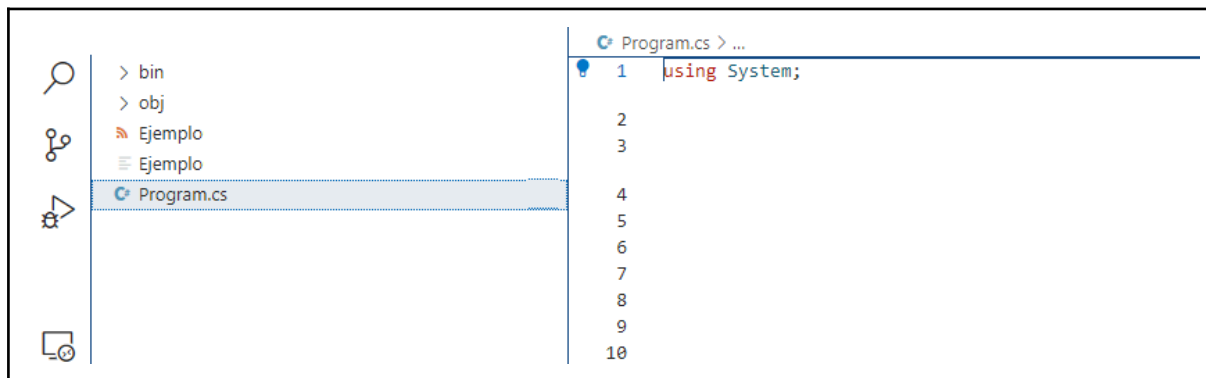
Ejemplo C14. Cálculo del volumen de una esfera (I).

Objetivo: Crear una aplicación que realiza cálculos utilizando una función.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new console -o EjemploC14
cd EjemploC14
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program          // Ejemplo C14
{
    // volume= 4/3 * pi * r * r * r
    static double Multiplica(double a, double b, double c)
    {
        double resultado = a * b * c; return resultado;
    }
    // - - - - -
    static void Main()
    {
        string ax = "Vamos a calcular el volumen de una esfera."; Console.WriteLine(ax);
        string bx = "El radio mide 5 metros."; Console.WriteLine(bx);

        double total = Multiplica(5, 5, 5);
        Console.WriteLine("El total es:");
        double volume = 4.0 / 3.0 * 3.14 * total;
        Console.WriteLine(volume); Console.WriteLine("metros cubicos.");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

The screenshot shows the Visual Studio Code interface with a file named 'Program.cs' open. The code defines a class 'Program' with a static method 'Multiplica' and a 'Main' method. The 'Main' method prints a message, asks for a radius, and calculates the volume of a sphere using the 'Multiplica' method. The terminal at the bottom shows the command 'dotnet run' being executed, and the output matches the program's logic.

```

1  using System; class Program
4      static double Multiplica(double a, double b, double c)
6          double resultado = a * b * c;
7          return resultado;
8      }
9
10     static void Main()
11     {
12         string ax = "Vamos a calcular el volumen de una esfera.";
13         Console.WriteLine(ax);
14         string bx = "El radio mide 5 metros.";
15         Console.WriteLine(bx);
16
17         double total = Multiplica(5, 5, 5);
18         Console.WriteLine("El total es:");
19         double volume = 4.0 / 3.0 * 3.14 * total;
20         Console.WriteLine(volume);

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\ACER\Documents\EjemploC14> dotnet run
Vamos a calcular el volumen de una esfera.
El radio mide 5 metros.
El total es:
523,333333333334
metros cubicos.

```

Observe que el código se ejecuta perfectamente y el usuario obtiene como resultado el producto de los tres valores enviados como parámetros a la función **Multiplica** para después multiplicar este resultado por el número **PI** y a su vez multiplicar este resultado por 4/3. El resultado de la suma es: 523,33.

Preguntas: ¿Qué variables hemos utilizado? Intenta mejorar este programa.

Fin del Ejemplo C14.

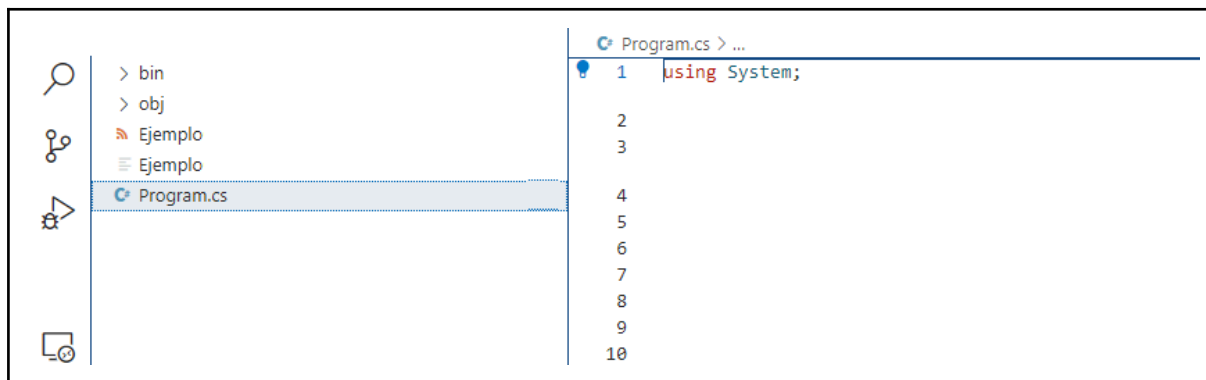
Ejemplo C15. Cálculo del volumen de una esfera (II).

Objetivo: Crear una aplicación que realiza cálculos utilizando la clase **Math**.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new console -o EjemploC15
cd EjemploC15
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program // Ejemplo C15
{
    // Definición de funciones
    static double Volume(double radio)
    {
        double volume = (4.0 / 3.0) * Math.PI * Math.Pow(radio, 3); return volume;
    }
    // - - - - -
    static void Main()
    {
        // Preguntamos la longitud del radio al usuario
        Console.Write("Entre el radio de la esfera: ");
        double radio = Convert.ToDouble(Console.ReadLine());

        // Calculamos el volumen llamando a la función "Volume"
        double volumen = Volume(radio);
        Console.WriteLine($"El volumen de la esfera es: {volumen:F2}");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

The screenshot shows the Visual Studio Code editor with a file named `Program.cs`. The code defines a `Program` class with a `Volume` method and a `Main` method. The `Volume` method calculates the volume of a sphere using the formula $V = \frac{4}{3} \pi r^3$. The `Main` method prompts the user for the radius, reads the input, and calls the `Volume` method. The terminal at the bottom shows the command `dotnet run` being executed, followed by the program's output: "Entre el radio de la esfera: 9" and "El volumen de la esfera es: 3053,63".

```

1  using System; class Program
2  {
3      // Definición de funciones
4      static double Volume(double radio)
5      {
6          double volume = (4.0 / 3.0) * Math.PI * Math.Pow(radio, 3);
7          return volume;
8      }
9
10     static void Main()
11     {
12         // Preguntamos la longitud del radio al usuario
13         Console.WriteLine("Entre el radio de la esfera: ");
14         double radio = Convert.ToDouble(Console.ReadLine());
15
16         // Calculamos el volumen llamando a la función "Volume"
17     }
18 }

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\ACER\Documents\EjemploC15> dotnet run
Entre el radio de la esfera: 9
El volumen de la esfera es: 3053,63
PS C:\Users\ACER\Documents\EjemploC15>

```

Nota importante:

Math.PI es una constante proporcionada por la clase **Math** que representa el valor de π (pi), aproximadamente 3,14159.

Math.Pow(radio, 3) eleva la variable **radio** (que representa el radio de la esfera) a la potencia de 3.

Reto: Intenta optimizar y mejorar este programa.

Fin del Ejemplo C15.

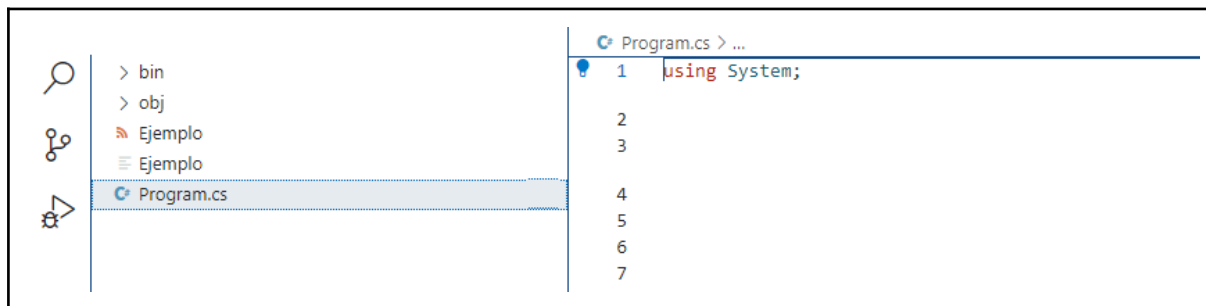
Ejemplo C16. Cálculo del volumen de un cilindro.

Objetivo: Crear una aplicación que realiza cálculos utilizando la clase **Math**.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new console -o EjemploC16
cd EjemploC16
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos el código del archivo **.cs** por el siguiente contenido:

```
using System; class Program                // Ejemplo C16

{
    // Función que realiza el Cálculo del volumen de un cilindro
    static double CalcularVolumen(double radius, double height)
    {
        double resultadoVolumen = Math.PI * Math.Pow(radius, 2) * height;
        return resultadoVolumen;
    }
    // - - - - -

    static void Main()
    {
        Console.WriteLine("Entre el radio del cilindro: ");
        double radio = Convert.ToDouble(Console.ReadLine());

        Console.WriteLine("Entre la altura del cilindro: ");
        double altura = Convert.ToDouble(Console.ReadLine());
        double volumen = CalcularVolumen(radio, altura);
        string bx = "volumen del cilindro con";
        Console.WriteLine($"El {bx} radio {radio} y altura {altura} es: {volumen:F2}");
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos el terminal en Visual Studio Code (CTRL+ñ) y teclearemos:

dotnet run

The screenshot shows the Visual Studio Code editor with a file named 'EjemploC16'. The code is a C# program that calculates the volume of a cylinder. The code is as follows:

```

1 using System; class Program
2 {
3     // Function to calculate the volume of a cylinder
4     static double CalcularVolumen(double radius, double height)
5     {
6         double resultadoVolumen = Math.PI * Math.Pow(radius, 2) * height;
7         return resultadoVolumen;
8     }
9
10    static void Main()
11    {
12        // Main program
13        Console.WriteLine("Entre el radio del cilindro: ");
14        double radio = Convert.ToDouble(Console.ReadLine());
15
16        Console.WriteLine("Entre la altura del cilindro: ");
17        double altura = Convert.ToDouble(Console.ReadLine());
18    }

```

Below the code editor, the 'TERMINAL' tab is active, showing the output of the 'dotnet run' command:

```

PS C:\Users\ACER\Documents\EjemploC16> dotnet run
Entre el radio del cilindro: 9
Entre la altura del cilindro: 8
El volumen de un cilindro con radio 9 y altura 8 es: 2035,75
PS C:\Users\ACER\Documents\EjemploC16>

```

Fin del Ejemplo C16.

Ejemplo C17. Hola Mundo (utilizando WPF).

Objetivo: Crear una aplicación que imprima en pantalla “hola mundo” en una ventana nueva utilizando WPF.

Esta es la estructura básica de nuestro proyecto:

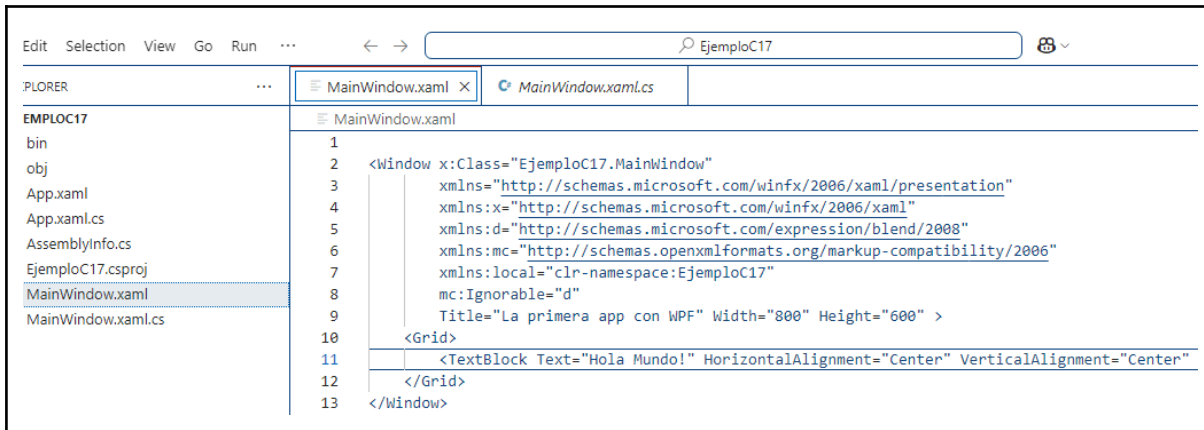
- **EjemploC17:** Nombre de nuestro proyecto.
- **Dependencias (dependencies) :** Contiene referencias a las librerías.NET necesarias.
- **Propiedades (properties):** Contiene ajustes (settings) del proyecto.
- **MainWindow.xaml :** Este archivo define la interfaz de la ventana principal (user interface de “main window”).
- **MainWindow.xaml.cs :** Este archivo contiene código C# asociado a MainWindow.xaml (en él se puede programar la lógica de la aplicación y podemos manipular eventos (event handlers)).
- **App.xaml :** Este archivo contiene recursos de la aplicación y es el punto de entrada de ésta (entry point).
- **App.xaml.cs :** Este archivo contiene código C# asociado a App.xaml.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC17
cd EjemploC17
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Ahora reemplazaremos el código del archivo **.xaml** (que contiene la programación para la interfaz -UI- por el siguiente contenido):



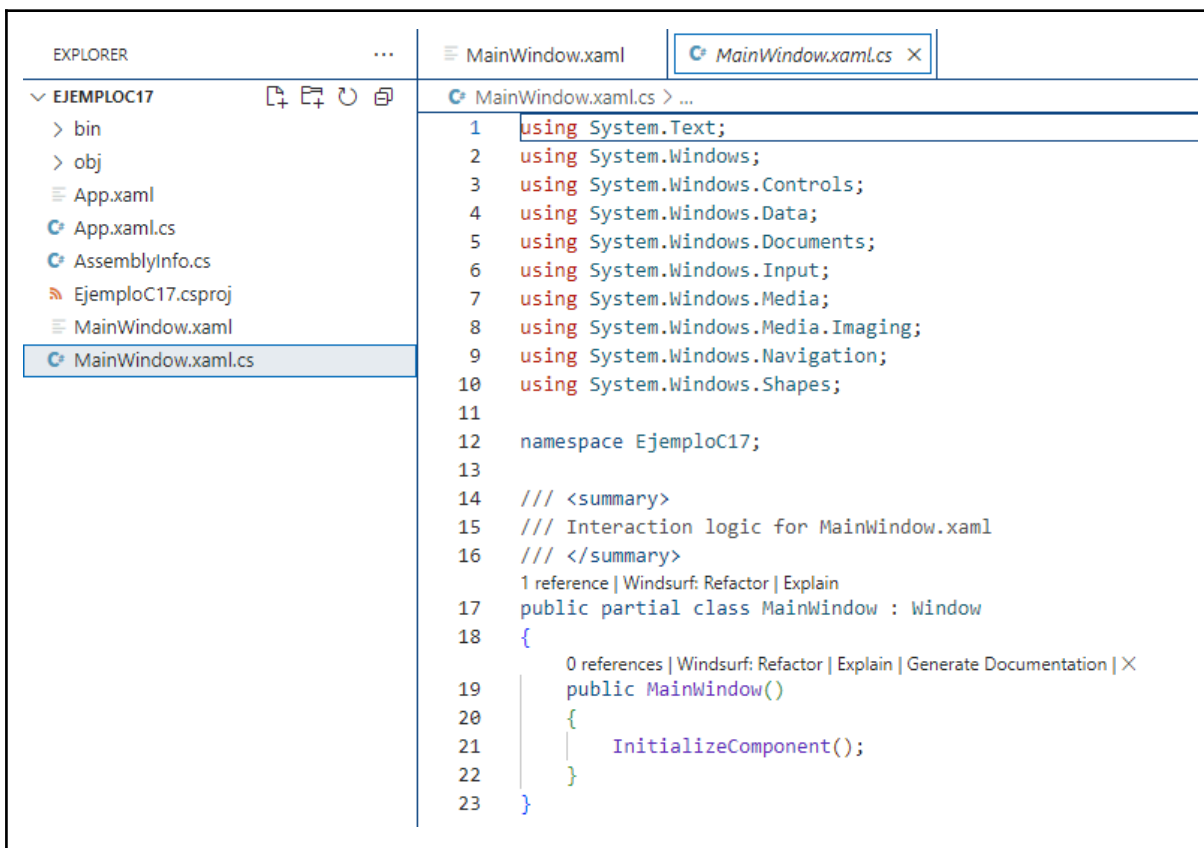
```

1  <Window x:Class="EjemploC17.MainWindow"
2
3      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
4      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
5      xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6      xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
7      xmlns:local="clr-namespace:EjemploC17"
8      mc:Ignorable="d"
9      Title="La primera app con WPF" Width="800" Height="600" >
10
11      <Grid>
12          <TextBlock Text="Hola Mundo!" HorizontalAlignment="Center" VerticalAlignment="Center"
13      </Grid>
14  </Window>

```

En el cuadro anterior hemos programado un **TextBlock** (bloque de texto) dentro del bloque **Grid** (Rejilla) donde irá el mensaje “Hola Mundo!”.

Después reemplazaremos el código del archivo **MainWindow.xaml.cs** por:



```

1  using System.Text;
2  using System.Windows;
3  using System.Windows.Controls;
4  using System.Windows.Data;
5  using System.Windows.Documents;
6  using System.Windows.Input;
7  using System.Windows.Media;
8  using System.Windows.Media.Imaging;
9  using System.Windows.Navigation;
10 using System.Windows.Shapes;
11
12 namespace EjemploC17;
13
14 /// <summary>
15 /// Interaction logic for MainWindow.xaml
16 /// </summary>
17 public partial class MainWindow : Window
18 {
19     public MainWindow()
20     {
21         InitializeComponent();
22     }
23 }

```

Importante:

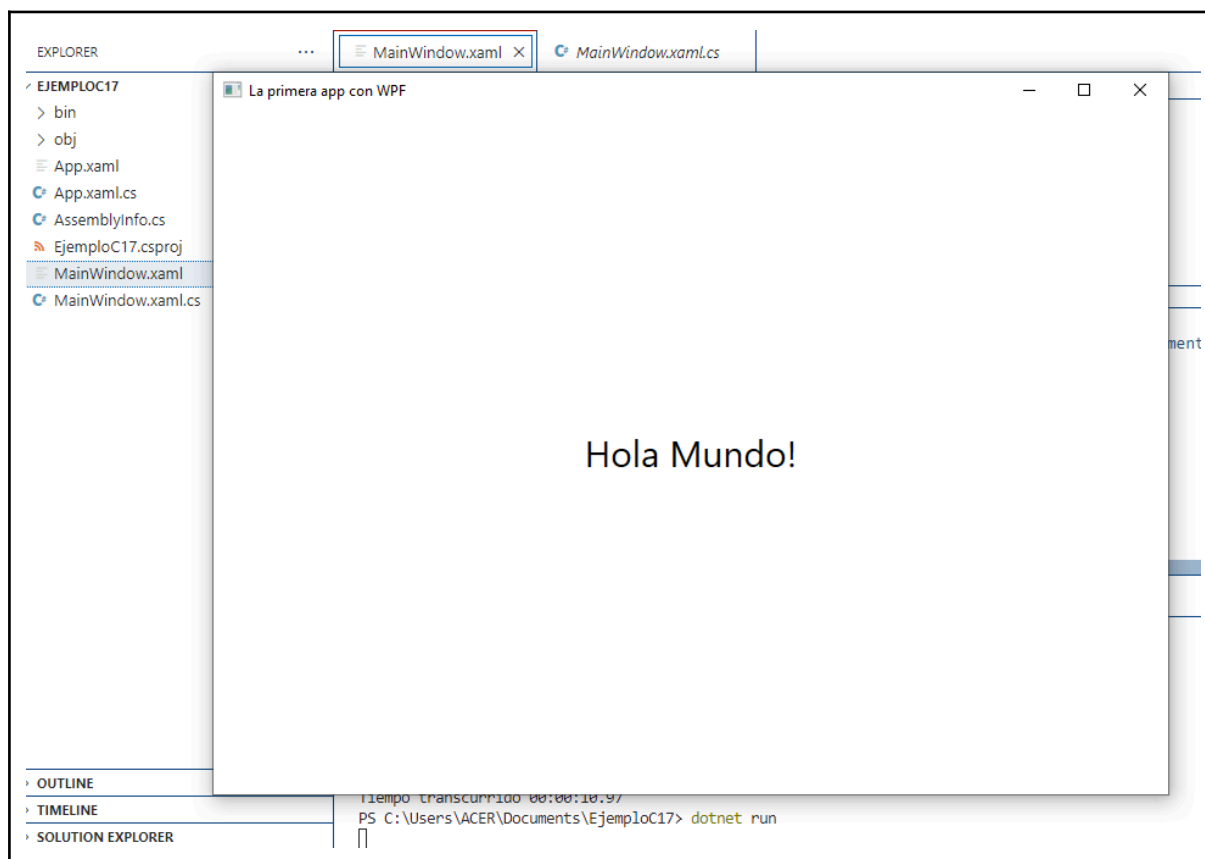
En esta ocasión no ha sido preciso modificar el archivo escrito en C# **MainWindow.xaml.cs**: Este es el archivo de código principal . Aunque no necesitamos escribir ningún código en C# para este ejemplo sencillo, este archivo es donde añadiríamos la lógica para manejar las interacciones del usuario, la manipulación de datos y otros comportamientos dinámicos de la ventana creada en nuestro ejemplo.

Utilice las siguientes instrucciones en el terminal para ejecutar el proyecto:

```
dotnet build
```

```
dotnet run
```

A continuación se ejecutará el proyecto creado y verá la siguiente ventana:



Ejemplo C18. Hola Mundo (utilizando WPF).

Objetivo: Crear una aplicación que imprima en pantalla “Hola Mundo C!” en una ventana nueva cambiando la propiedad **Text** de un bloque de texto.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC18
cd EjemploC18
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Importante:

El Grid es un panel de diseño en WPF. Ayuda a organizar dónde se posicionan los elementos de la interfaz de usuario en la ventana creada. Es una estructura visual.

El código C# buscará las siguientes dos condiciones:

- El Nombre: El **TextBlock** debe tener un nombre (usando **x:Name**) para poder referenciarlo.
- La Propiedad: Que el **TextBlock** tenga una propiedad **Text** que se pueda modificar."

Haga estos cambios en el código de estos **dos** archivos y analice el efecto al **referenciar** el bloque de texto llamado “**MyTextBlock**”:

MainWindow.xaml

```
<Window x:Class="EjemploC18.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:EjemploC18"
        mc:Ignorable="d"
        Title="La primera app con WPF" Width="800" Height="600" >
    <Grid>
        <TextBlock x:Name="MyTextBlock" Text="Hola Mundo!"
            HorizontalAlignment="Center"
            VerticalAlignment="Center" />
    </Grid>
</Window>
```

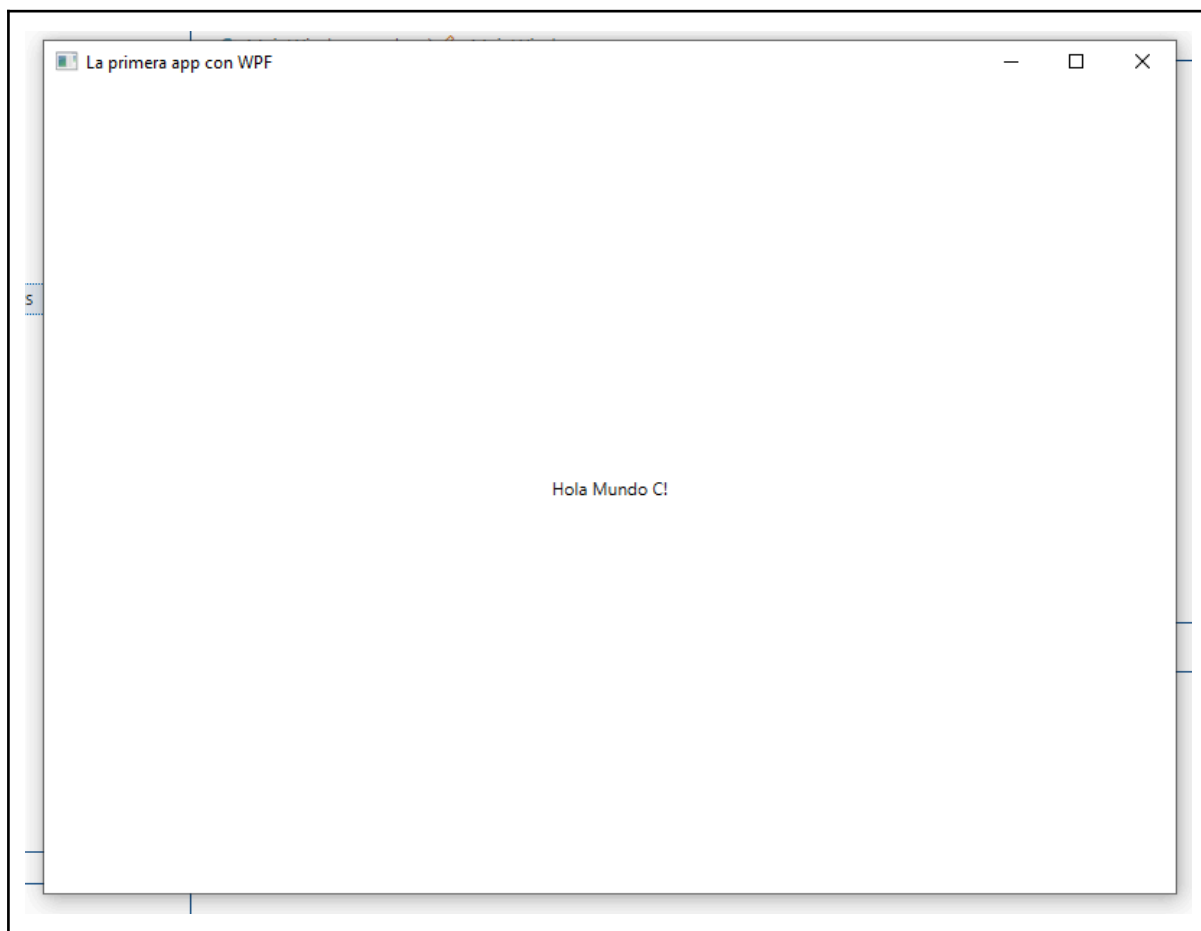
MainWindow.xaml.cs

```
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace EjemploC18;

/// <summary>
/// Interaction logic for MainWindow.xaml
/// </summary>
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        MyTextBlock.Text = "Hola Mundo C!";
    }
}
```

A continuación ejecute el proyecto creado y verá la siguiente ventana:



Ejemplo C19. Hola Mundo (con cambio de color y utilizando WPF).

Objetivo: Crear un programa que imprima “Hola Mundo C#” en una ventana modificando las propiedades de un objeto para cambiar el color de un texto.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC19
cd EjemploC19
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios en el código de estos **dos** archivos y analice el efecto al **cambiar las propiedades** del bloque de texto llamado “**MyTextBlock**”.

MainWindow.xaml

```
<Window x:Class="EjemploC19.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:EjemploC19"
        mc:Ignorable="d"
        Title="La primera app con WPF" Width="800" Height="600" >
    <Grid>
        <TextBlock x:Name="MyTextBlock" Text="Hola!"
            HorizontalAlignment="Center"
            VerticalAlignment="Center" />
    </Grid>
</Window>
```

MainWindow.xaml.cs

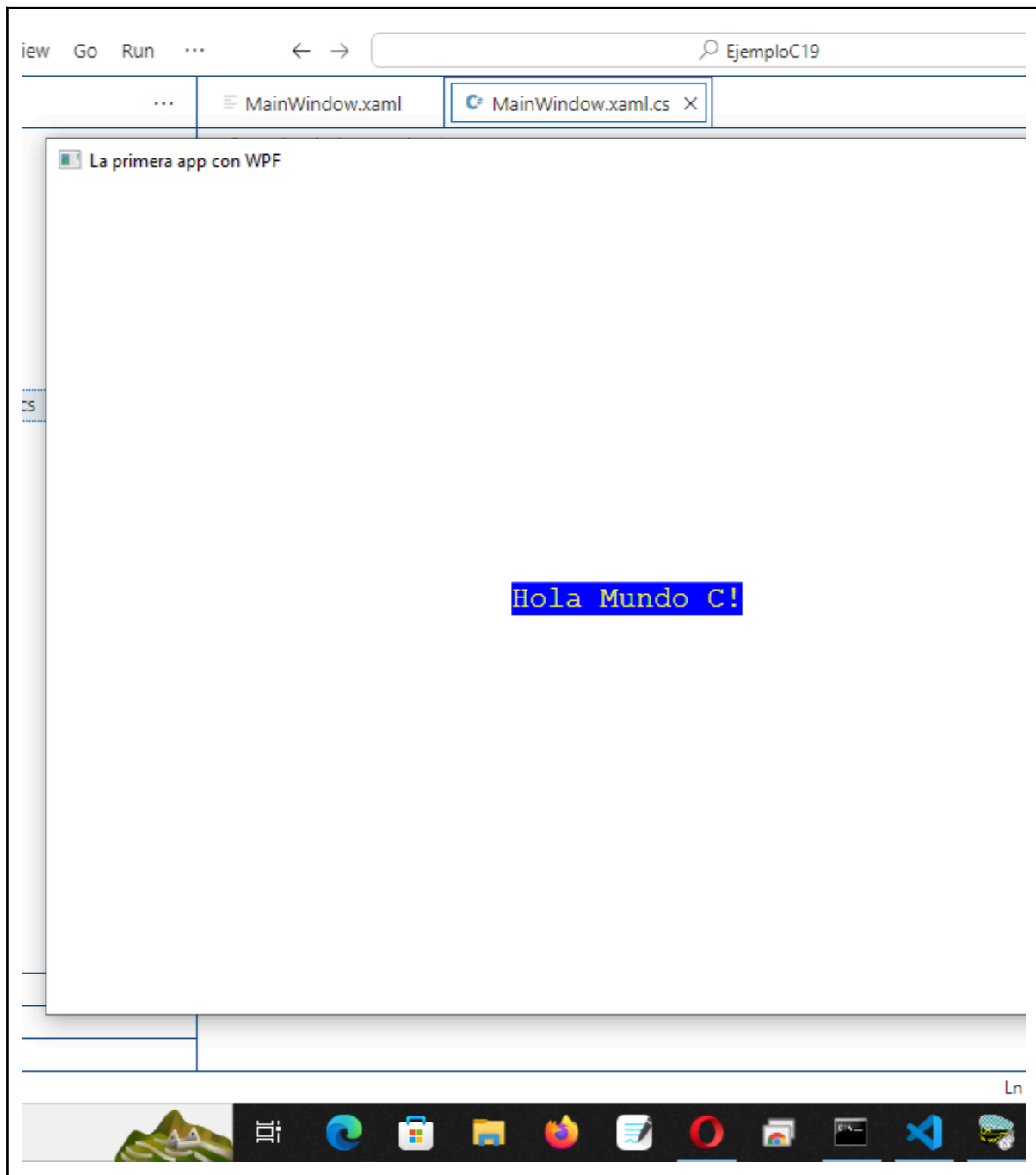
```
using System.Text; using System.Windows;
using System.Windows.Controls;
using System.Windows.Data; using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media; using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace EjemploC19;

/// <summary>
/// Interaction logic for MainWindow.xaml
/// </summary>
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        MyTextBlock.Text = "Hola Mundo C!";
        MyTextBlock.Foreground = Brushes.Yellow;
        MyTextBlock.FontSize = 20;
        MyTextBlock.Background = Brushes.Blue;
        MyTextBlock.FontFamily = new FontFamily("Courier New");
    }
}

/// Recuerde que la fuente recomendada por Microsoft es "Segoe UI".
```

A continuación ejecute el proyecto creado y verá la siguiente ventana:



Ejemplo C20. Hola Mundo (con cambio de color y utilizando WPF).

Objetivo: Crear un programa que imprima “Hola Mundo C#” en una ventana modificando las propiedades de un objeto para cambiar el color de diferentes textos.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC20
cd EjemploC20
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios en el código de estos **dos** archivos y analice el efecto al **cambiar las propiedades** del bloque de texto llamado “**MyTextBlock**”.

MainWindow.xaml

```
<Window x:Class="EjemploC20.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:EjemploC20"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
    <Grid>
        <TextBlock x:Name="MyTextBlock1" Text="Hola Mundo!" HorizontalAlignment="Center"
            VerticalAlignment="Center" />
        <TextBlock x:Name="MyTextBlock2" Text="Hola Munro!" HorizontalAlignment="Center"
            VerticalAlignment="Center"
            Margin="0,40,0,0"/>
    </Grid>
</Window>
```

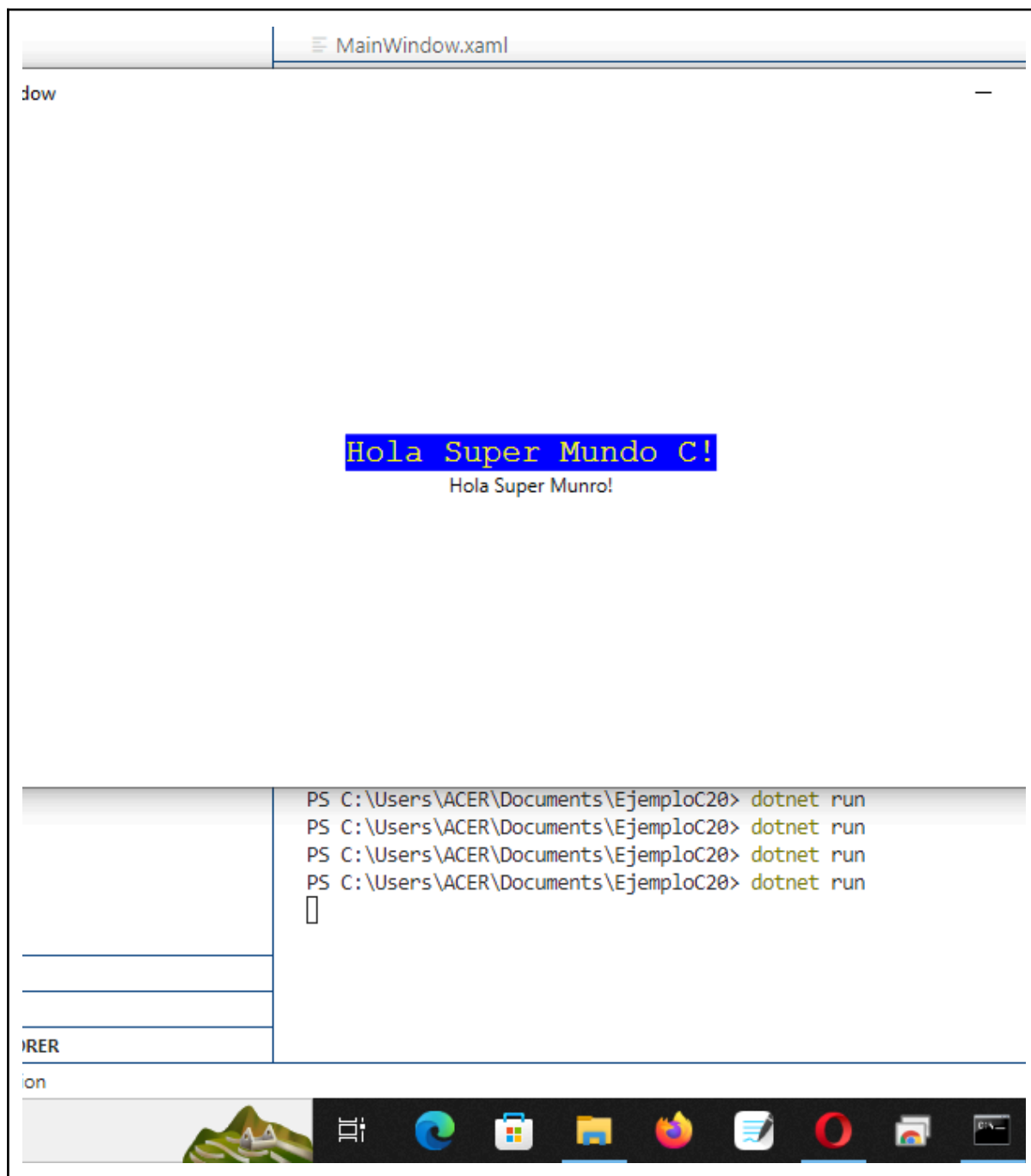
MainWindow.xaml.cs

```
using System.Text; using System.Windows;
using System.Windows.Controls;
using System.Windows.Data; using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation; using System.Windows.Shapes;

namespace EjemploC20;

/// <summary>
/// Interaction logic for MainWindow.xaml
/// </summary>
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        MyTextBlock1.Text = "Hola Super Mundo C!";
        MyTextBlock1.Foreground = Brushes.Yellow;
        MyTextBlock1.FontSize = 20;
        MyTextBlock1.Background = Brushes.Blue;
        MyTextBlock1.FontFamily = new FontFamily("Courier New");
        MyTextBlock2.Text = "Hola Super Munro!";
    }
}
```

A continuación ejecute el proyecto creado y verá la siguiente ventana:



Ejemplo C21. Hola Mundo (con cambio de color y utilizando WPF).

Objetivo: Crear un programa que imprima “Hola Mundo C#” en una ventana modificando las propiedades de un objeto para cambiar el color de diferentes textos.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC21
cd EjemploC21
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios en el código de estos **dos** archivos y analice el efecto al **cambiar las propiedades** del bloque de texto llamado “**MyTextBlock**”.

MainWindow.xaml

```
<Window x:Class="EjemploC21.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="EJEMPLOC21" Height="300" Width="300">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition />
            <RowDefinition />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition />
            <ColumnDefinition />
        </Grid.ColumnDefinitions>

        <TextBlock Text="1" Grid.Row="0" Grid.Column="0" HorizontalAlignment="Center"
VerticalAlignment="Center" />
        <TextBlock Text="2" Grid.Row="0" Grid.Column="1" HorizontalAlignment="Center"
VerticalAlignment="Center" />
        <TextBlock Text="3" Grid.Row="1" Grid.Column="0" HorizontalAlignment="Center"
VerticalAlignment="Center" />
        <TextBlock Text="4" Grid.Row="1" Grid.Column="1" HorizontalAlignment="Center"
VerticalAlignment="Center" />
    </Grid>
</Window>
```

MainWindow.xaml.cs

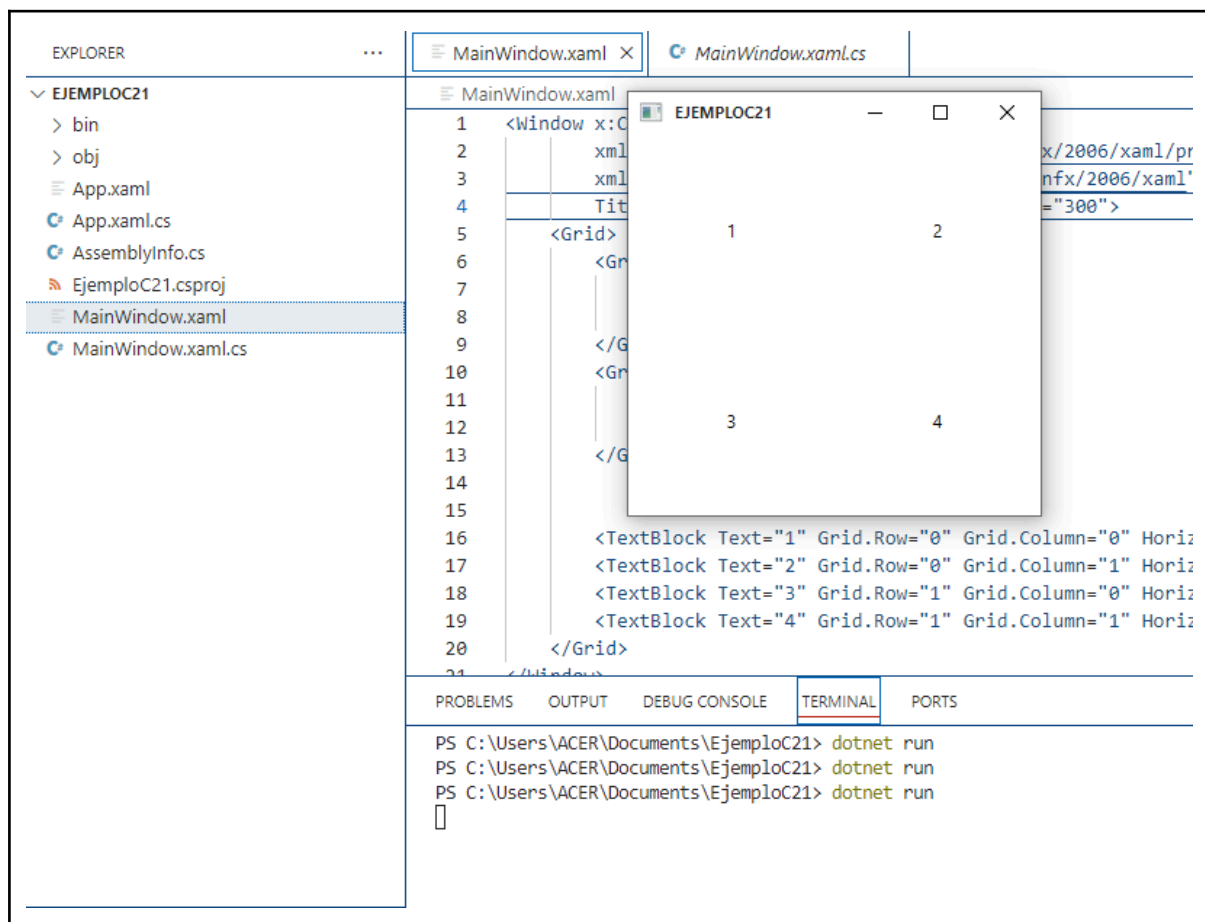
```
using System.Text; using System.Windows;
using System.Windows.Controls; using System.Windows.Data;
using System.Windows.Documents; using System.Windows.Input;
using System.Windows.Media; using System.Windows.Media.Imaging;
using System.Windows.Navigation; using System.Windows.Shapes;

namespace EjemploC21;

/// <summary> /// Interaction logic for MainWindow.xaml /// </summary>

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

A continuación ejecute el proyecto creado y verá la siguiente ventana:



Preguntas:

¿Es cierto que el número 4 está en las coordenadas: (1,1) y que corresponde a Row=1 y Column=1?

¿Es cierto que el número 1 corresponde a Row=0 y Column=0?

Ejemplo C22. Hola Mundo (con cambio de color y utilizando WPF).

Objetivo: Escribir un programa para crear un array de 25 x 80 posiciones para almacenar palabras en cada posición.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC22
cd EjemploC22
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios en el código de estos **dos** archivos y analice el código.

MainWindow.xaml

```
<Window x:Class="EjemploC22.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="EJEMPLOC22" Height="300" Width="300">

    <Grid>

        <Grid.RowDefinitions>
            <RowDefinition />
            <RowDefinition />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition />
            <ColumnDefinition />
        </Grid.ColumnDefinitions>

        <TextBlock Text="Array" Grid.Row="0" Grid.Column="0" HorizontalAlignment="Center"
VerticalAlignment="Center" />
        <TextBlock Text="de" Grid.Row="0" Grid.Column="1" HorizontalAlignment="Center"
VerticalAlignment="Center" />
        <TextBlock Text="25 lineas y" Grid.Row="1" Grid.Column="0"
HorizontalAlignment="Center" VerticalAlignment="Center" />
        <TextBlock Text="80 columnas creado!" Grid.Row="1" Grid.Column="1"
HorizontalAlignment="Center" VerticalAlignment="Center" />

    </Grid>

</Window>
```

MainWindow.xaml.cs

```
using System.Text; using System.Windows;
using System.Windows.Controls; using System.Windows.Data;
using System.Windows.Documents; using System.Windows.Input;
using System.Windows.Media; using System.Windows.Media.Imaging;
using System.Windows.Navigation; using System.Windows.Shapes;
namespace EjemploC22;
/// <summary> /// Interaction logic for MainWindow.xaml /// </summary>
public partial class MainWindow : Window
{
    // Declara un array de 2 dimensiones para almacenar "words" (palabras)
    private string[,] wordsArray = new string[25, 80];

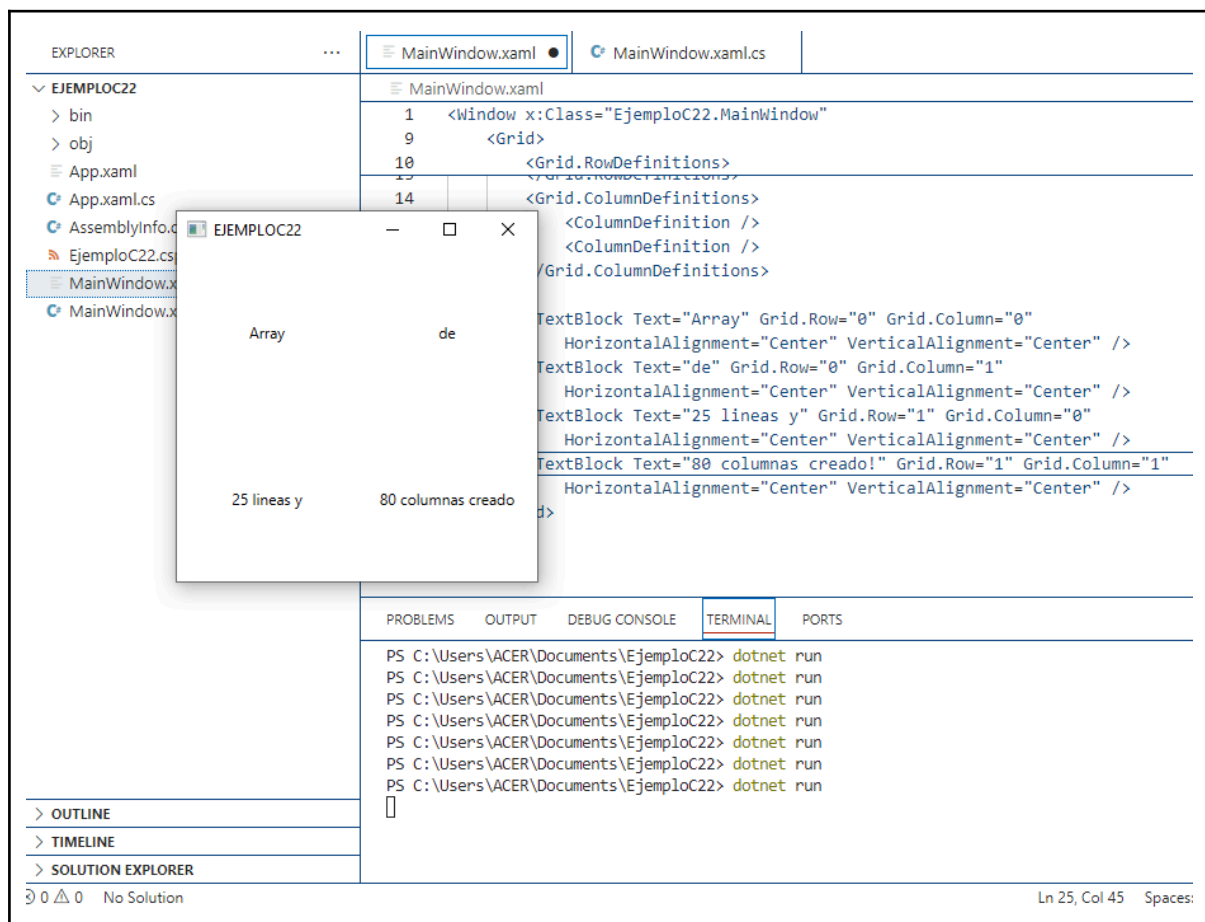
    public MainWindow()
    {
        InitializeComponent();
        InitializeWordsArray(); // Llamada al method para inicializar array
    }

    private void InitializeWordsArray()
    {
        // inicializar array con valores por defecto (strings vacíos)
        for (int row = 0; row < 25; row++)
        {
            for (int col = 0; col < 80; col++)
            {
                wordsArray[row, col] = ""; // valores por defecto (strings vacíos)
            }
        }

        // Ejemplo de cómo añadir valores al array
        wordsArray[0, 0] = "0,0 Hi";
        wordsArray[0, 1] = "0,1 Son";
        wordsArray[1, 5] = "1,5 How";
        wordsArray[24, 65] = "24,65 are you?";
    }

    // Ejemplo para obtener un word (palabra) del array
    public string GetWord(int row, int col)
    {
        if (row >= 0 && row < 25 && col >= 0 && col < 80)
        {
            return wordsArray[row, col];
        }
        else
        {
            return "Invalid Index";
        }
    }
}
```

A continuación ejecute el EjemploC22 creado y verá la siguiente ventana:



Preguntas:

¿Es cierto que el número 4 está en las coordenadas: (1,1) y que corresponde a Row=1 y Column=1?

Ejemplo C23. Creación de una matriz de puntos (utilizando WPF).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar palabras (strings) en cada posición. Se llenarán las celdas con la cadena “.”

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC23
cd EjemploC23
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios en el código de estos **dos** archivos y analice el código.

MainWindow.xaml

```
<Window x:Class="EjemploC23.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Visualización del Array" Height="600" Width="1200">
    <Grid>
        <DataGrid Name="WordsDataGrid" AutoGenerateColumns="False"
                  HorizontalAlignment="Stretch" VerticalAlignment="Stretch">
        </DataGrid>
    </Grid>
</Window>
```

/// End of file MainWindow.xaml

MainWindow.xaml.cs

```

using System.Collections.Generic; using System.Windows;
using System.Windows.Controls; using System.Windows.Data;
namespace EjemploC23
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            InitializeArrayAndGrid();
        }
        private void InitializeArrayAndGrid()
        {
            int rows = 25; int cols = 80;
            string[,] dataArray = new string[rows, cols];

            // Llenamos el array con "."
            for (int i = 0; i < rows; i++)
            {
                for (int j = 0; j < cols; j++)
                {
                    dataArray[i, j] = ".";
                }
            }
            // Creamos columns para el DataGrid
            for (int i = 0; i < cols; i++)
            {
                DataGridTextColumn column = new DataGridTextColumn();
                column.Header = i.ToString(); // Podemos cambiar headers si es preciso
                Binding binding = new Binding($"[{i}]"); // Bind to elemento en index i
                in the row data
                column.Binding = binding;
                WordsDataGrid.Columns.Add(column);
            }
            // Creamos una list para mantener las filas(rows) de datos para el DataGrid
            List<string[]> gridData = new List<string[]>();
            for (int i = 0; i < rows; i++)
            {
                string[] rowData = new string[cols];
                for (int j = 0; j < cols; j++)
                {
                    rowData[j] = dataArray[i, j];
                }
                gridData.Add(rowData);
            }
            // Set the ItemsSource of the DataGrid
            WordsDataGrid.ItemsSource = gridData;
        }
    }
}

```



```

/// Versión mejorada del archivo de la pag. anterior: MainWindow.xaml.cs con
/// funciones bien diferenciadas
using System.Collections.Generic; using System.Windows;
using System.Windows.Controls; using System.Windows.Data;
namespace EjemploC23
{
    public partial class MainWindow : Window
    {
        private const int NumRows = 25; private const int NumCols = 80;
        private string[,] _dataArray;

        public MainWindow()
        {
            InitializeComponent();
            InitializeData();
            PopulateGridColumnns();
            PopulateGridData();
        }
        private void InitializeData()
        {
            _dataArray = CreateAndFillArray(NumRows, NumCols, ".");
        }
        private string[,] CreateAndFillArray(int rows, int cols, string fillValue)
        {
            string[,] array = new string[rows, cols];
            for (int i = 0; i < rows; i++)
            {
                for (int j = 0; j < cols; j++)
                {
                    array[i, j] = fillValue;
                }
            }
            return array;
        }
        private void PopulateGridColumnns()
        {
            for (int i = 0; i < NumCols; i++)
            {
                WordsDataGrid.Columns.Add(CreateGridColumn(i));
            }
        }
        private DataGridTextColumn CreateGridColumn(int columnIndex)
        {
            DataGridTextColumn column = new DataGridTextColumn();
            column.Header = columnIndex.ToString();
            column.Binding = new Binding($"[{columnIndex}]");
            return column;
        }
        private void PopulateGridData()
        {
            WordsDataGrid.ItemsSource = ConvertArrayToList(_dataArray);
        }
        private List<string[]> ConvertArrayToList(string[,] array)
        {
            int rows = array.GetLength(0);
            int cols = array.GetLength(1);
            List<string[]> list = new List<string[]>();
            for (int i = 0; i < rows; i++)
            {
                string[] rowData = new string[cols];
                for (int j = 0; j < cols; j++)
                {
                    rowData[j] = array[i, j];
                }
                list.Add(rowData);
            }
            return list;
        }
    }
}

```

Ejemplo C24. Creación de una matriz con letras (utilizando WPF).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras o palabras (strings) en cada posición. Se llenarán las celdas con la cadena “.” para posteriormente almacenar la palabra “OK” y visualizarla en pantalla en la primera línea.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC24
cd EjemploC24
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios en el código de estos **dos** archivos y analice el código.

MainWindow.xaml

```
<Window x:Class="EjemploC24.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Visualización del Array" Height="600" Width="1200">
    <Grid>
        <DataGrid Name="WordsDataGrid" AutoGenerateColumns="False"
                  HorizontalAlignment="Stretch" VerticalAlignment="Stretch">
        </DataGrid>
    </Grid>
</Window>
```

/// End of file MainWindow.xaml

MainWindow.xaml.cs

```
using System.Collections.Generic; using System.Windows;
using System.Windows.Controls; using System.Windows.Data;
namespace EjemploC24

{
    public partial class MainWindow : Window
    {
        private string[,] dataArray;
        private List<string[]> gridData;
        private int rows = 25;
        private int cols = 80;

        public MainWindow()
        {
            InitializeComponent();
            InitializeArrayAndGrid();
            ModifyGrid(); /// Llamamos a ModifyGrid despues de llamar a InitializeComponent
        }

        private void InitializeArrayAndGrid()
        {
            dataArray = new string[rows, cols];
            gridData = new List<string[]>();

            // Llenamos el array con "."
            for (int i = 0; i < rows; i++)
            {
                for (int j = 0; j < cols; j++)
                {
                    dataArray[i, j] = ".";
                }
            }

            // Creamos columns para el DataGrid
            for (int i = 0; i < cols; i++)
            {
                DataGridTextColumn column = new DataGridTextColumn();
                column.Header = i.ToString();
                Binding binding = new Binding($"[{i}]");
                column.Binding = binding;
                WordsDataGrid.Columns.Add(column);
            }

            // Creamos una list para mantener las filas(rows) de datos para el DataGrid
            for (int i = 0; i < rows; i++)
            {
                string[] rowData = new string[cols];
                for (int j = 0; j < cols; j++)
                {
                    rowData[j] = dataArray[i, j];
                }
                gridData.Add(rowData);
            }

            // Set the ItemsSource of the DataGrid
            WordsDataGrid.ItemsSource = gridData;
        }

        // El código continúa en la siguiente página
    }
}
```

```

private void ModifyGrid()
{
    // Colocamos "O" in the first cell (row 0, column 0), primera letra de "OK"
    if (rows > 0 && cols > 0 && WordsDataGrid != null && WordsDataGrid.Items !=
null)
    {
        dataArray[0, 0] = "O";
        if (gridData.Count > 0 && gridData[0].Length > 0)
        {
            gridData[0][0] = "O"; // Update the underlying data
        }
    }

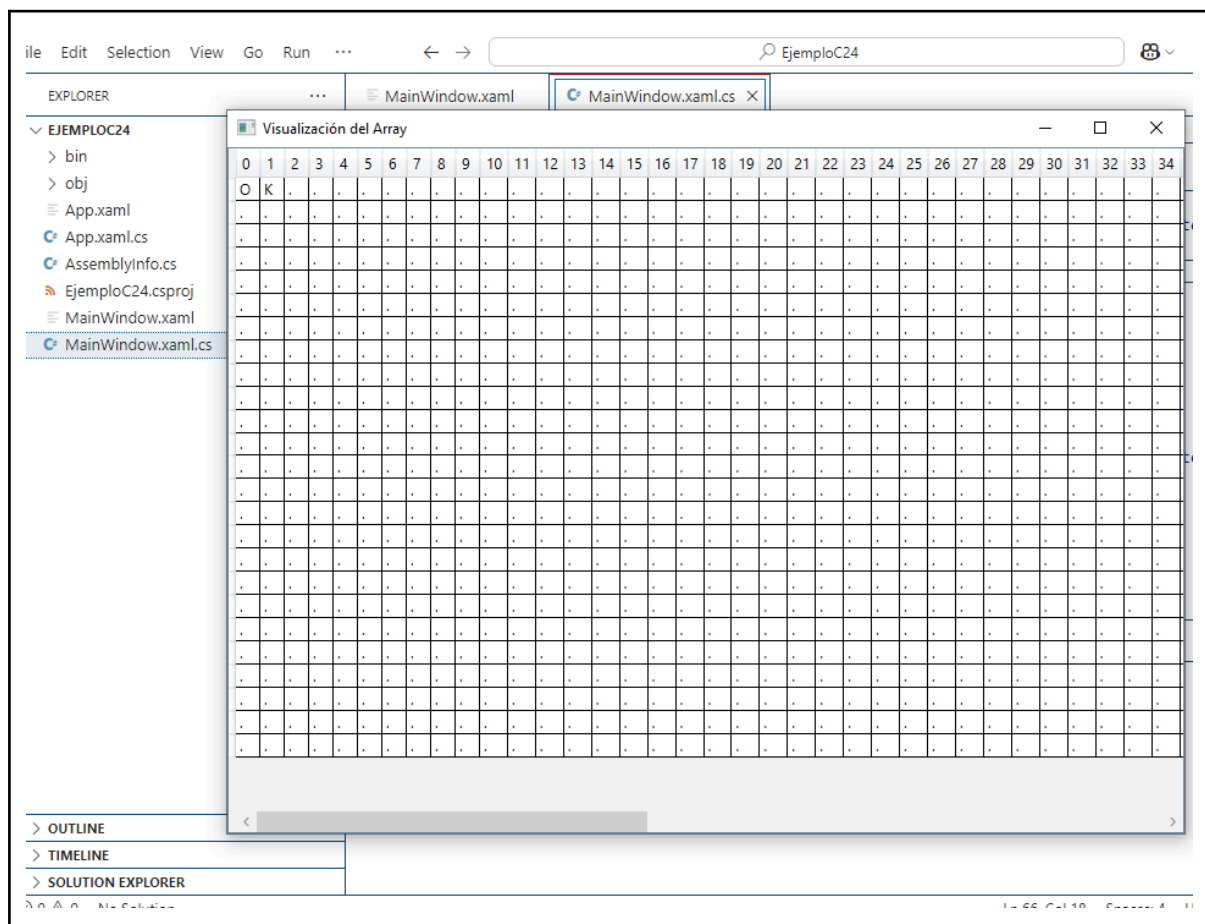
    // Colocamos "K" in the cell at (row 0, column 1), que es la segunda letra de
"OK"
    if (rows > 0 && cols > 3 && WordsDataGrid != null && WordsDataGrid.Items !=
null)
    {
        dataArray[0, 1] = "K";
        if (gridData.Count > 0 && gridData[0].Length > 3)
        {
            gridData[0][1] = "K"; // Update the underlying data
        }
    }

    // Refrescamos WordsDataGrid para imprimir las letras de la matriz
    if (WordsDataGrid != null && WordsDataGrid.Items != null)
    {
        WordsDataGrid.Items.Refresh();
    }
}
}

// Continúa en la siguiente página

```


A continuación ejecute el EjemploC24 creado y verá la siguiente ventana:



Preguntas:

¿Es cierto que la cadena “K” está en las coordenadas: (1,1) y que corresponde a Row=1 y Column=1?

¿Podría esta matriz ser la base para un videojuego?

Ejemplo C25. Creación de una tabla sin bordes utilizando una matriz con letras (utilizando WPF).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras o palabras (strings) en cada posición. Se llenarán las celdas con la cadena “.” para posteriormente almacenar la palabra “OK” y visualizarla en pantalla en la primera línea.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new wpf -o EjemploC25
cd EjemploC25
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios en el código de estos **dos** archivos y analice el código.

MainWindow.xaml

```
<Window x:Class="EjemploC25.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Visualización del Array" Height="600" Width="1200">
    <Grid>
        <DataGrid Name="WordsDataGrid" AutoGenerateColumns="False"
                  HorizontalAlignment="Stretch" VerticalAlignment="Stretch">
        </DataGrid>
    </Grid>
</Window>
```

/// End of file MainWindow.xaml

MainWindow.xaml.cs

```

using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Media;

namespace EjemploC23
{
    public partial class MainWindow : Window
    {
        private string[,] dataArray;
        private List<string[]> gridData;
        private int rows = 25;
        private int cols = 80;

        public MainWindow()
        {
            InitializeComponent();
            InitializeArrayAndGrid();
            ModifyGrid();
            MakeBordersWhite(); // Call this method to make borders white
        }

        private void InitializeArrayAndGrid()
        {
            dataArray = new string[rows, cols];
            gridData = new List<string[]>();

            // Llenamos el array con "."
            for (int i = 0; i < rows; i++)
            {
                for (int j = 0; j < cols; j++)
                {
                    dataArray[i, j] = ".";
                }
            }

            // Creamos columns para el DataGrid
            for (int i = 0; i < cols; i++)
            {
                DataGridTextColumn column = new DataGridTextColumn();
                column.Header = i.ToString();
                Binding binding = new Binding($"[{i}]");
                column.Binding = binding;
                WordsDataGrid.Columns.Add(column);
            }

            // Creamos una list para mantener las filas(rows) de datos para el DataGrid
            for (int i = 0; i < rows; i++)
            {
                string[] rowData = new string[cols];
                for (int j = 0; j < cols; j++)
                {
                    rowData[j] = dataArray[i, j];
                }
                gridData.Add(rowData);
            }

            // Set the ItemsSource of the DataGrid
            WordsDataGrid.ItemsSource = gridData;
        }
    }
}

```

```

private void ModifyGrid()
{
    // Place "A" in the first cell (row 0, column 0)
    if (rows > 0 && cols > 0 && WordsDataGrid != null && WordsDataGrid.Items !=
null)
    {
        dataArray[0, 0] = "A";
        if (gridData.Count > 0 && gridData[0].Length > 0)
        {
            gridData[0][0] = "A";
        }
    }

    // Place "C" in the cell at row 0, column 3
    if (rows > 0 && cols > 3 && WordsDataGrid != null && WordsDataGrid.Items !=
null)
    {
        dataArray[0, 3] = "C";
        if (gridData.Count > 0 && gridData[0].Length > 3)
        {
            gridData[0][3] = "C";
        }
    }

    // Force the DataGrid to refresh its view
    if (WordsDataGrid != null && WordsDataGrid.Items != null)
    {
        WordsDataGrid.Items.Refresh();
    }
}

private void MakeBordersWhite()
{
    // Set the overall border of the DataGrid to white (if any is still drawn)
    WordsDataGrid.BorderBrush = Brushes.White;
    WordsDataGrid.BorderThickness = new Thickness(0); // Ensure no thickness

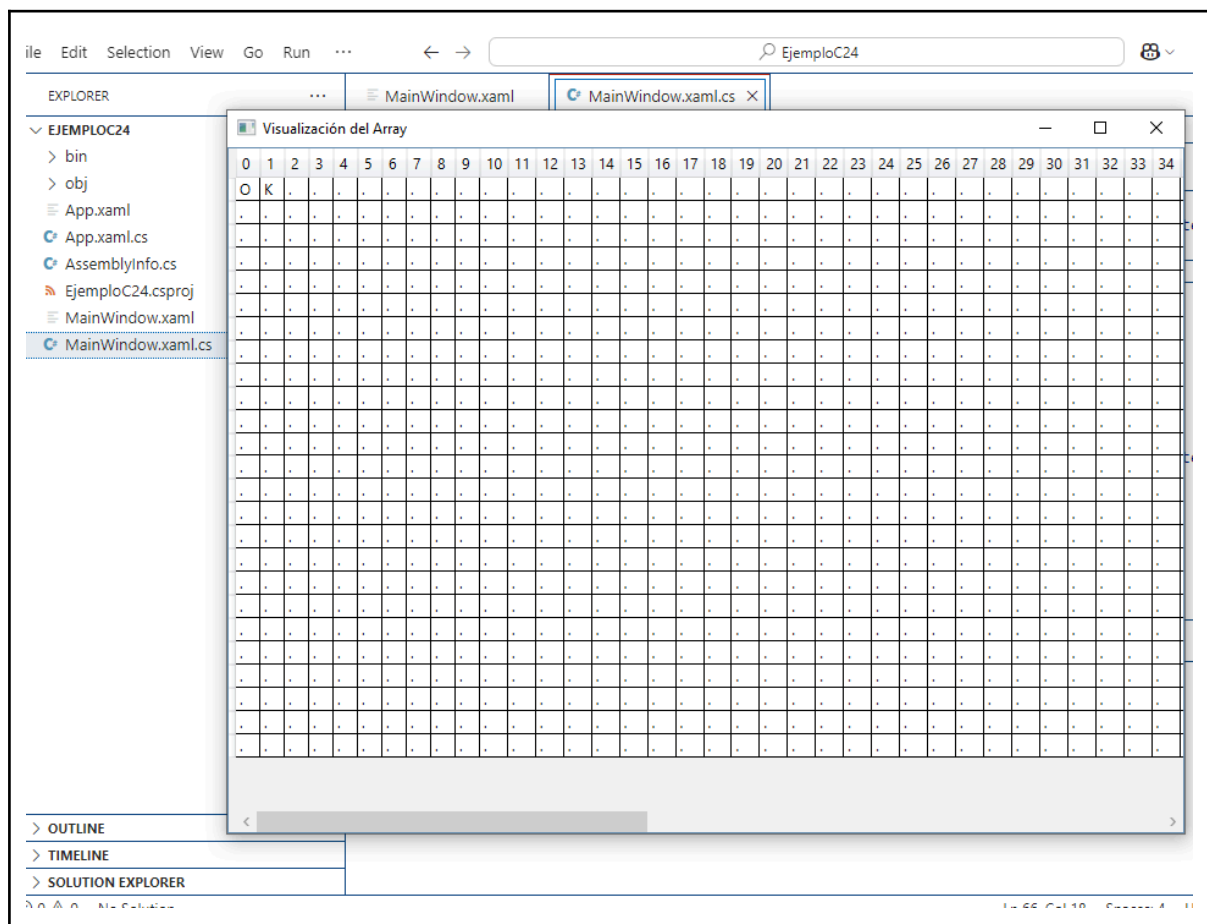
    // Remove row headers
    WordsDataGrid.RowHeaderWidth = 0;

    // Style for the DataGridCell to make its borders white
    Style cellStyle = new Style(typeof(DataGridCell));
    cellStyle.Setters.Add(new Setter(Control.BorderBrushProperty, Brushes.White));
    cellStyle.Setters.Add(new Setter(Control.BorderThicknessProperty, new
Thickness(0)));
    WordsDataGrid.CellStyle = cellStyle;

    // Style for the DataGridColumnHeader to make its separators white
    Style columnHeaderStyle = new Style(typeof(DataGridColumnHeader));
    columnHeaderStyle.Setters.Add(new Setter(Control.BorderBrushProperty,
Brushes.White));
    columnHeaderStyle.Setters.Add(new Setter(Control.BorderThicknessProperty, new
Thickness(0, 0, 1, 1))); // Adjust as needed
    WordsDataGrid.ColumnHeaderStyle = columnHeaderStyle;
}
}

```

A continuación ejecute el EjemploC25 creado y verá la siguiente ventana:



Preguntas:

¿Es cierto que la cadena “K” está en las coordenadas: (1,1) y que corresponde a Row=1 y Column=1?

¿Podría esta matriz ser la base para un videojuego?

Ejemplo C26. Creación de una ventana con un botón (utilizando WinForms).

Objetivo: Escribir un programa para crear una ventana con un botón utilizando **WinForms** en lugar de WPF.

En ocasiones, es preferible crear la interfaz de usuario únicamente con código (sin utilizar ningún asistente). Con ello se consigue mayor flexibilidad y una mejor integración con la lógica de la aplicación.

Para hacerlo, necesitamos tener presente los siguientes puntos:

1.

Inicializamos los controles (de nuestro UI) necesarios: Form, Button, Label, TextBox, Panel, etc.) usando sus respectivos constructores de clases.

2

Establecemos las propiedades de cada control como por ejemplo: Text, Location, Size, Anchor, Dock, BackColor, ForeColor o Font.

3.

Añadimos controles al formulario (Form):

Utilizamos el método Controls.Add() u otros controles de contenedor como Panel o GroupBox para añadir los controles a la jerarquía del UI.

4.

Añadimos manipuladores de eventos (event handlers) a los eventos de los controles creando métodos que manipulan los eventos (event handler methods) y los añadimos con el operador +=.

5.

Controlamos la disposición (layout) de los controles utilizando Location, Size, Anchor y Dock properties, o utilizando otros controles más sofisticados, como por ejemplo: TableLayoutPanel y FlowLayoutPanel.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC26
cd EjemploC26
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:


```

/// Program.cs

using System; using System.Drawing; using System.Windows.Forms; /// EjemploC26

public class CodeOnlyForm : Form
{
    private Button myButton;
    private Label myLabel;

    public CodeOnlyForm()
    {
        // Set the form's properties
        Text = "Ejemplo con C# y WinForms";
        Size = new Size(800, 600);
        StartPosition = FormStartPosition.CenterScreen;

        // Create the label
        myLabel = new Label();
        myLabel.Text = "Hello Mundo! "Form" creado con código únicamente!";
        myLabel.Location = new Point(20, 20);
        myLabel.AutoSize = true;

        // Create the button
        myButton = new Button();
        myButton.Text = "Haz clic aqui";
        myButton.Location = new Point(20, 60);
        myButton.Click += MyButton_Click; // Attach the event handler

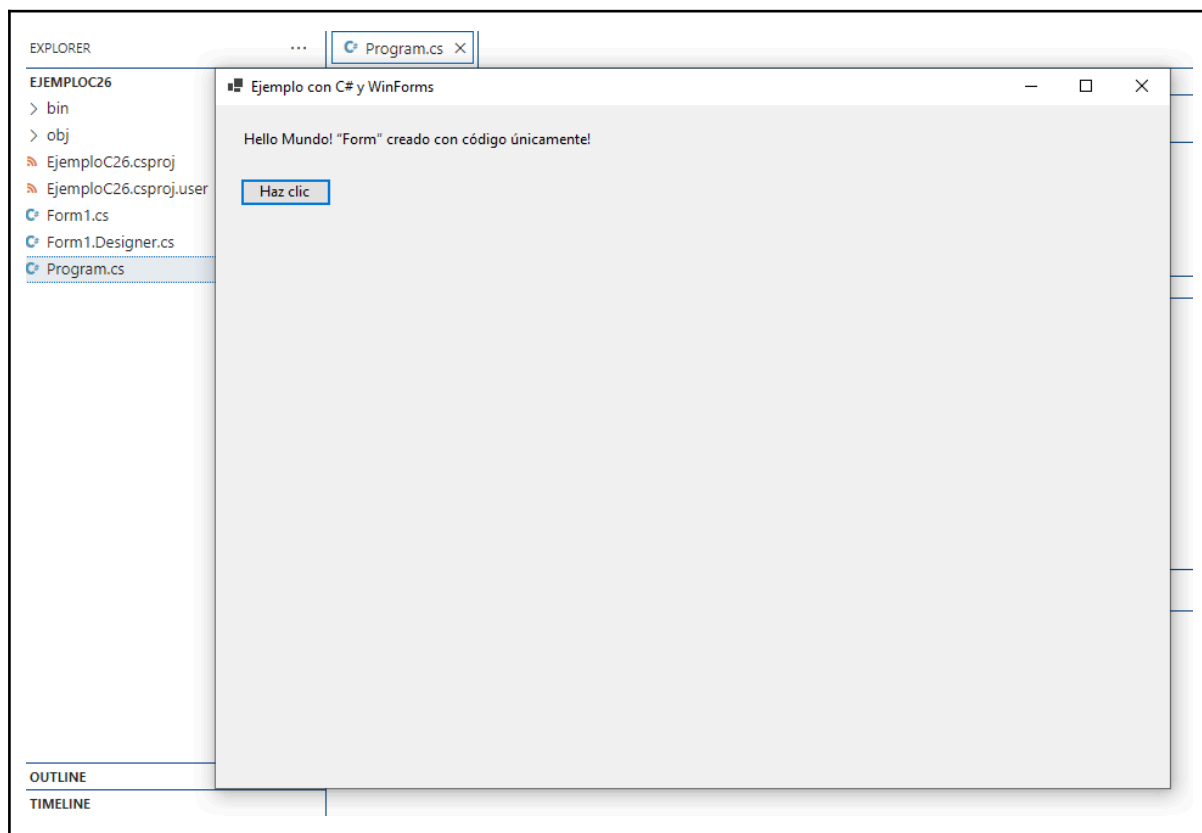
        // Add controls to the form's Controls collection
        Controls.Add(myLabel);
        Controls.Add(myButton);
    }

    private void MyButton_Click(object sender, EventArgs e)
    {
        myLabel.Text = "Has hecho clic!";
    }

    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new CodeOnlyForm());
    }
}

```

A continuación ejecute el EjemploC26 creado y verá la siguiente ventana:



Ejemplo C27. Creación de una ventana con un botón (utilizando WinForms).

Objetivo: Escribir un programa para crear una ventana con un botón utilizando WinForms en lugar de WPF.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC27
cd EjemploC27
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```
/// Program.cs
```

```
using System; using System.Drawing; using System.Windows.Forms; /// EjemploC27
public class CodeOnlyForm : Form
{
    private Button myButton; private Label myLabel;
    private Color borderColor = Color.Red; private int borderWidth = 3; // Thickness

    public CodeOnlyForm()
    {
        // Crear form's properties
        Text = "WinForms con Borde rojo"; ClientSize = new Size(800, 600);
        StartPosition = FormStartPosition.CenterScreen;
        Padding = new Padding(borderWidth); // Crea padding for the inner border

        // Cambia background color of the form to the border color
        BackColor = borderColor;

        // Crea Panel to hold the actual content, inside the border
        Panel contentPanel = new Panel();
        contentPanel.BackColor = SystemColors.Control; // sys background color for controls
        contentPanel.Dock = DockStyle.Fill; // fill inner area (excluding padding)

        // Create the label
        myLabel = new Label();
        myLabel.Text = "Hello desde WinForms!";
        myLabel.Location = new Point(20, 20);
        myLabel.AutoSize = true;
        contentPanel.Controls.Add(myLabel);

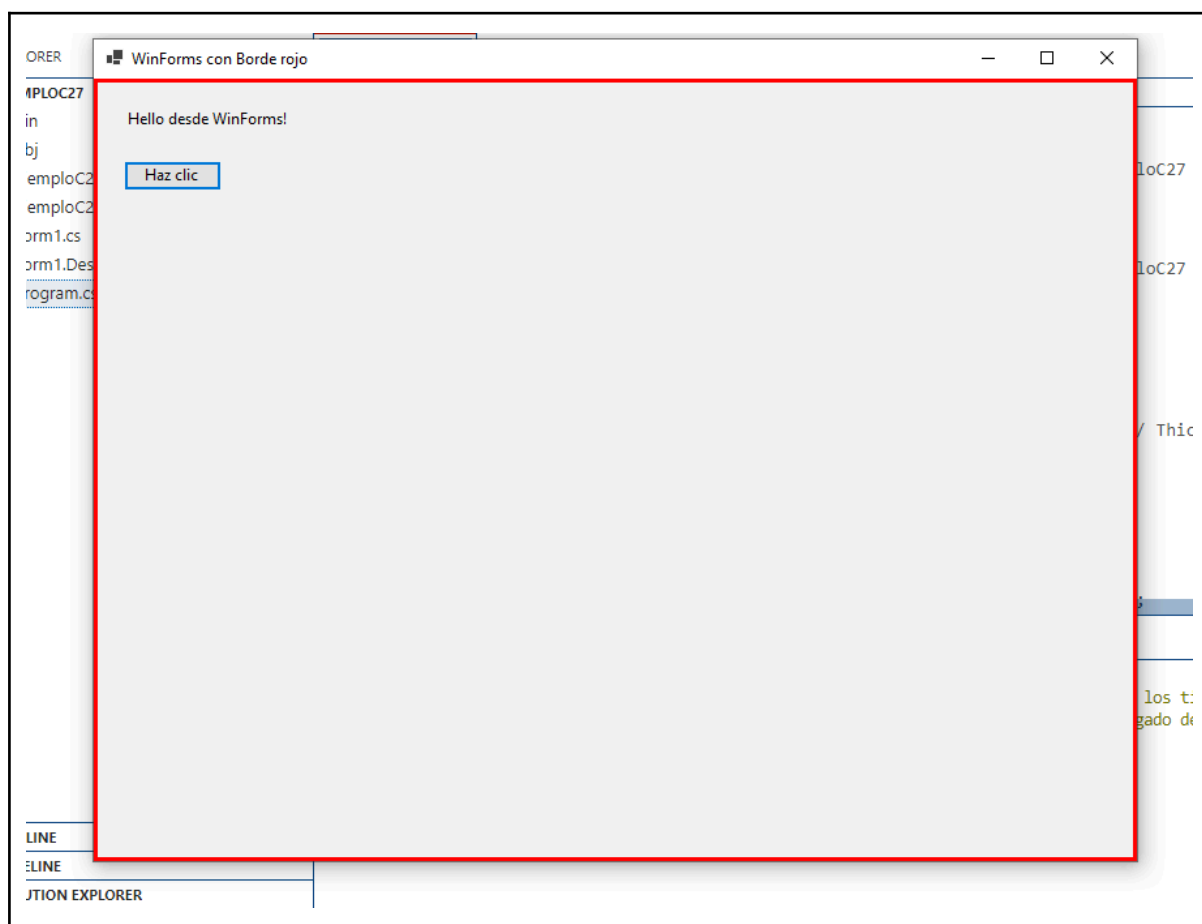
        // Create the button
        myButton = new Button(); myButton.Text = "Haz clic aquí";
        myButton.Location = new Point(20, 60);
        myButton.Click += MyButton_Click; // Adjuntar the event handler
        contentPanel.Controls.Add(myButton);

        Controls.Add(contentPanel); // Añade content panel to the form
    }

    private void MyButton_Click(object sender, EventArgs e)
    {
        myLabel.Text = "Haz hecho clic!";
    }

    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles(); Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new CodeOnlyForm());
    }
}
```

A continuación ejecute el EjemploC27 creado y verá la siguiente ventana:



Ejemplo C28. Creación de una matriz con letras (utilizando WinForms).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras o palabras (strings) en cada posición. Se llenarán las celdas con la cadena “.” para posteriormente almacenar la palabra “OK” y visualizarla en pantalla en la primera línea.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC28
cd EjemploC28
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs
using System; using System.Drawing; using System.Windows.Forms; /// EjemploC28
public class GridForm : Form
{
    private const int NumColumns = 80; private const int NumRows = 25;
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles;

    public GridForm()
    {
        Text = "Grid 80 x 25";
        // Calculate initial form size based on the number of rectangles
        ClientSize = new Size(NumColumns * 20, NumRows * 20); // Adjust size as needed
        StartPosition = FormStartPosition.CenterScreen;
        DoubleBuffered = true; // For smoother painting (suave)

        gridRectangles = new Rectangle[NumRows, NumColumns]; // Initialize array

        int rectangleWidth = ClientSize.Width / NumColumns; // Calc rectang dims and
positions
        int rectangleHeight = ClientSize.Height / NumRows;

        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
            }
        }
        Paint += GridForm_Paint; Resize += GridForm_Resize;
    }

    private void GridForm_Paint(object sender, PaintEventArgs e)
    {
        using (Pen pen = new Pen(gridColor))
        {
            for (int row = 0; row < NumRows; row++)
            {
                for (int col = 0; col < NumColumns; col++)
                {
                    e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);
                }
            }
        }
    }
}

/// el programa continúa en la siguiente página

```

```

private void GridForm_Resize(object sender, EventArgs e)
{
    // Recalculate rectangle dimensions and positions on resize
    int rectangleWidth = ClientSize.Width / NumColumns; int rectangleHeight =
    ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

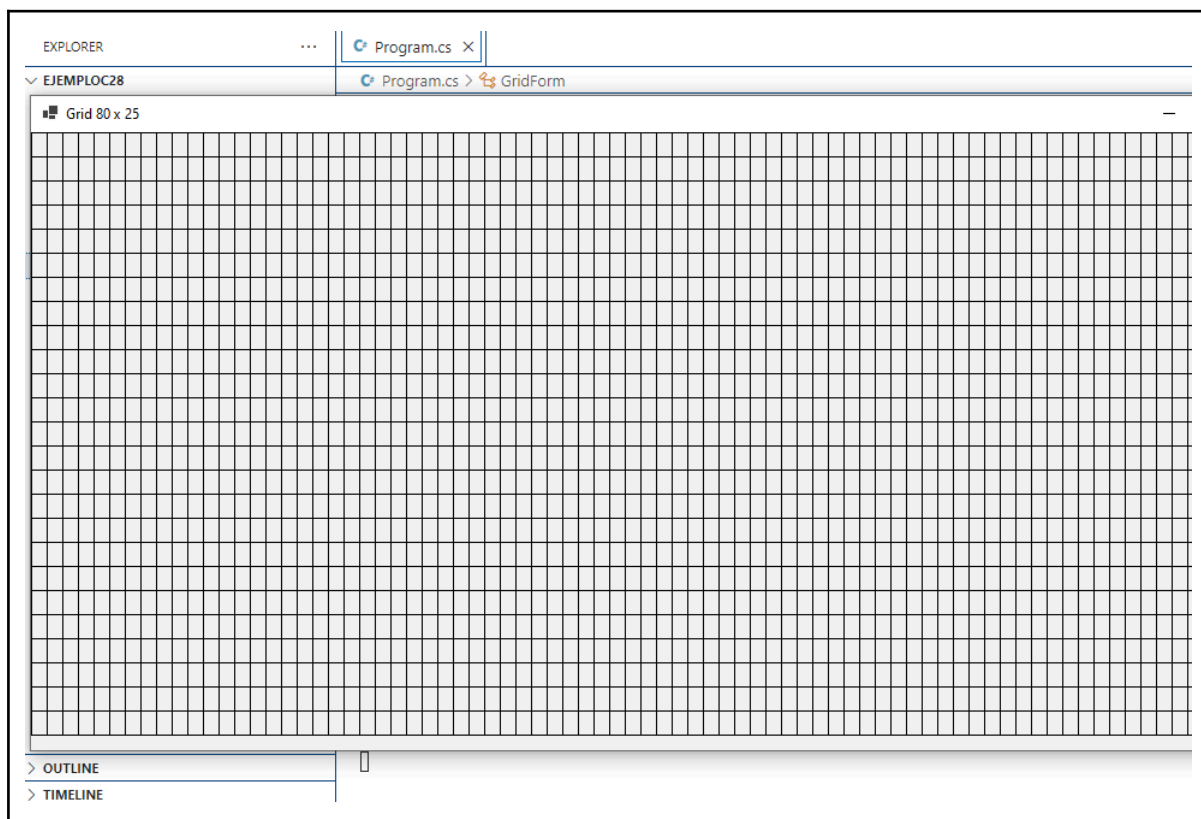
    Invalidate(); // Forzamos a super repaint of the form llamando a Invalidate()
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}
}

```

/// fin del programa EjemploC28

A continuación ejecute el EjemploC28 creado y verá la siguiente ventana:



Nota importante:

Para **declarar** una matriz de cadenas que contenga 3 elementos, haremos:

```
string[] freeItemsIDs= new string[3];
```

El operador **new** crea una nueva instancia de una matriz que puede almacenar 3 valores tipo string (cadena).

Para **asignar** valores a la matriz freeItemsIDs, escribiremos:

```
string[] freeItemsIDs= new string[3];
freeItemsIDs[0]="Free watch";
freeItemsIDs[1]="Free keyboard";
freeItemsIDs[2]="Free mouse";
```

Recuerde que el primer elemento de la matriz siempre es 0. Por tanto, el último valor utilizable como índice en esta matriz será 2.

Para acceder en la memoria del ordenador a un valor de la matriz y recuperar su valor con el fin de imprimirlo en la consola, escribiremos el siguiente código:

/// [Program.cs](#)

Ejemplo 28b.

```
string[] freeItemsIDs= new string[3];
freeItemsIDs[0]="Free watch";
freeItemsIDs[1]="Free keyboard";
freeItemsIDs[2]="Free mouse";

Console.WriteLine($"Primer valor: {freeItemsIDs[0]}");
Console.WriteLine($"Segundo valor: {freeItemsIDs[1]}");
Console.WriteLine($"Tercer valor: {freeItemsIDs[2]}");
```

Otra forma de implementar el anterior ejemplo podría ser:

```
/// Program.cs Ejemplo 28c.

string[] freeItemsIDs= ["Free watch", "Free keyboard", "Free mouse"];
Console.WriteLine($"Primer valor: {freeItemsIDs[0]}");
Console.WriteLine($"Segundo valor: {freeItemsIDs[1]}");
Console.WriteLine($"Tercer valor: {freeItemsIDs[2]}");
/// Opcional: comprobemos la longitud del array (matriz)
Console.WriteLine($"Existen: {freeItemsIDs.Length} valores en la matriz.");
```

Sin embargo, sería más efectivo recorrer toda la matriz con un bucle y la instrucción `foreach`:

```
/// Program.cs Ejemplo 28d.

string[] freeItemsIDs = { "Free watch", "Free keyboard", "Free mouse" };

foreach (string item in freeItemsIDs)
{
    Console.WriteLine(item);
}
```

A continuación, utilizamos una matriz para hacer la suma de dos valores enteros:

```
/// Program.cs Ejemplo 28e.

int[] inventos= {1971, 1978};
int sum=0;
foreach (int item in inventos)
{
    sum= sum+item;
}
Console.WriteLine($"La suma total de los valores de la matriz es: {sum}");
```

Reto: Copie y chequee las siguientes modificaciones del programa 28d para que se impriman los valores de la matriz utilizando un bucle con la instrucción **while**.

```
using System;

/// Program.cs                                Ejemplo 28f.

string[] freeItemsIDs = { "Free watch", "Free keyboard", "Free mouse" };

int index = 0;

while (index < freeItemsIDs.Length)
{
    // print cada elemento del array utilizando index
    Console.WriteLine(freeItemsIDs[index]);

    // Incrementamos index para movernos al next elemento del array
    index++;
}
```

Pregunta:

¿Pretende evaluar la expresión “(index < freeItemsIDs.Length)” cuántos elementos componen la matriz?

```
/// Program.cs                                Ejemplo 28g.

using System;

string[] freeItemsIDs = { "Free watch", "Free keyboard", "Free mouse" };

int index = 0;
int howmany = freeItemsIDs.Length;

while (index < howmany)
{
    Console.WriteLine(freeItemsIDs[index]); index++;
}
```

A continuación, utilizamos una matriz para hacer la suma de dos valores enteros (creamos un bucle utilizando la instrucción while):

```
using System;

int[] inventos = {1971, 1978};

int sum = 0;

int index = 0;

while (index < inventos.Length)
{
    sum = sum + inventos[index];

    index++;
}

// Print total sum
Console.WriteLine($"La suma total de los valores de la matriz es: {sum}");
```

Ejemplo C29. Creación de una matriz con cambio de colores de fondo (utilizando WinForms).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras o palabras (strings) en cada posición. Se llenarán las celdas con la cadena “.” para posteriormente almacenar la palabra “OK” y visualizarla en pantalla en la primera línea.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC29
cd EjemploC29
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs
using System; using System.Drawing; using System.Windows.Forms; /// EjemploC29

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1;           // Default line color is 1 (black); cyan=2
    private int backgroundColorSwitch = 0; // Default background color is 0 (white)
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles;

    public GridForm()
    {
        Text = "Grid of Rectangles with Color Switches";
        ClientSize = new Size(NumColumns * 20, NumRows * 20);
        StartPosition = FormStartPosition.CenterScreen;
        DoubleBuffered = true;
        BackColor = Color.White; // Set initial background color

        gridRectangles = new Rectangle[NumRows, NumColumns];

        // Calculate initial rectangle dimensions and positions
        int rectangleWidth = ClientSize.Width / NumColumns;
        int rectangleHeight = ClientSize.Height / NumRows;

        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
            }
        }

        Paint += GridForm_Paint;
        Resize += GridForm_Resize;
        KeyDown += GridForm_KeyDown; // Listen for key presses for both switches
    }

/// el programa EjemploC29 continúa en la siguiente página

```

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    // Determine the line color based on the switch
    switch (lineColorSwitch)
    {
        case 1:
            gridColor = Color.Black;
            break;
        case 2:
            gridColor = Color.Cyan;
            break;
        case 0:
            gridColor = Color.White;
            break;
        default:
            gridColor = Color.Black; // Default to black if switch is unexpected
            break;
    }

    // Determine the background color based on the switch
    switch (backgroundColorSwitch)
    {
        case 1:
            BackColor = Color.Black;
            break;
        case 0:
            BackColor = Color.White;
            break;
        default:
            BackColor = Color.White; // Default to white if switch is unexpected
            break;
    }

    using (Pen pen = new Pen(gridColor))
    {
        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);
            }
        }
    }
}

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    Invalidate();
}

```

/// el programa EjemploC29 continúa en la siguiente pág.


```

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    // Switch for line color
    if (e.KeyCode == Keys.D1) // Press '1' for black lines
    {
        lineColorSwitch = 1;
        Invalidate();
    }
    else if (e.KeyCode == Keys.D2) // Press '2' for cyan lines
    {
        lineColorSwitch = 2;
        Invalidate();
    }
    else if (e.KeyCode == Keys.D0 && !e.Shift) // Press '0' for white lines
    {
        lineColorSwitch = 0;
        Invalidate();
    }

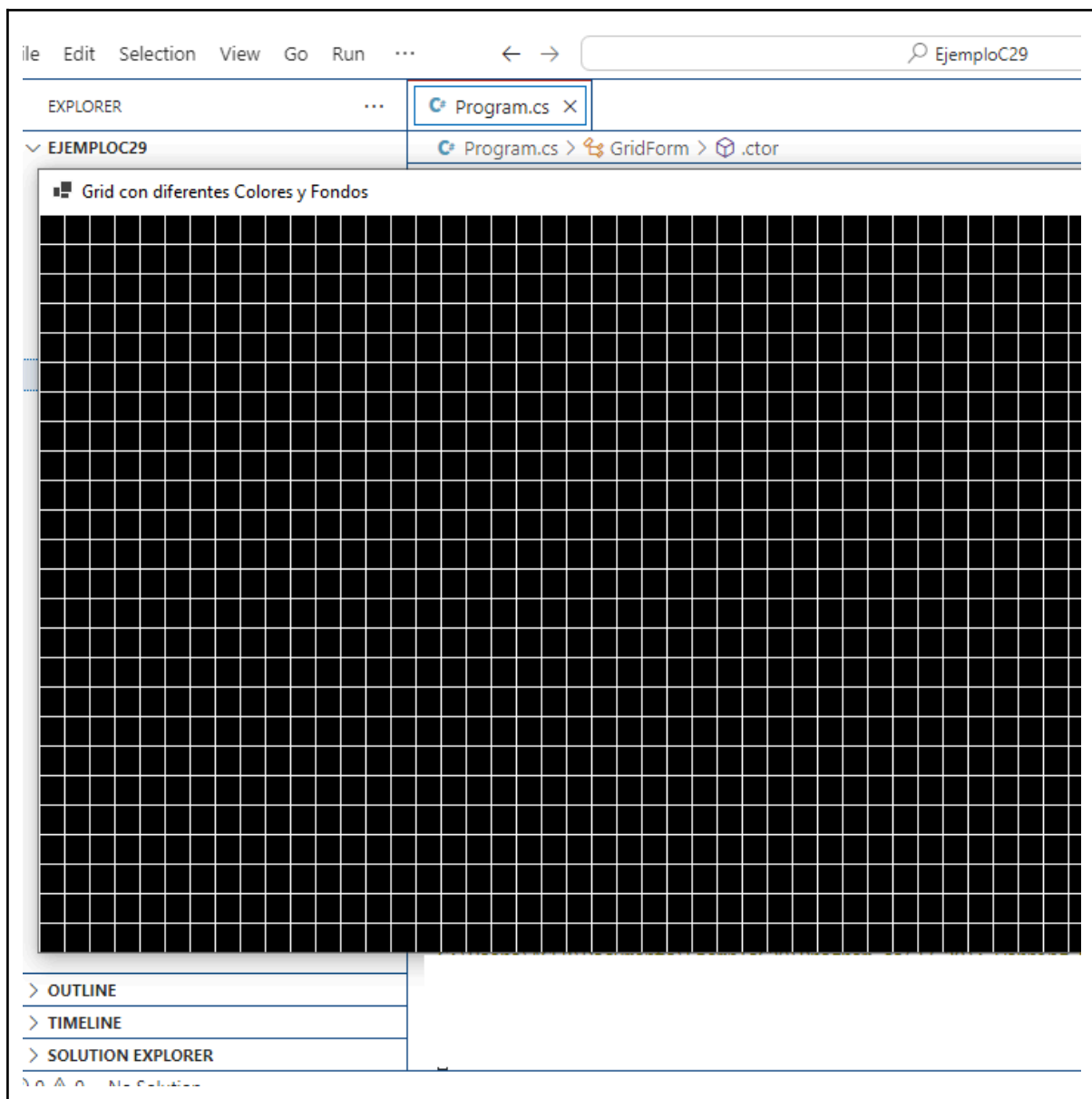
    // Switch for background color
    if (e.KeyCode == Keys.D1 && e.Shift) // Press Shift + '1' for black background
    {
        backgroundColorSwitch = 1;
        Invalidate();
    }
    else if (e.KeyCode == Keys.D0 && e.Shift) // Press Shift + '0' for white background
    {
        backgroundColorSwitch = 0;
        Invalidate();
    }
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}
}

```

/// fin del programa EjemploC29

A continuación ejecute el EjemploC29 creado y verá la siguiente ventana:



Ejemplo C30. Creación de una matriz con cambio de colores de fondo (utilizando WinForms).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras o palabras (strings) en cada posición. Se llenarán las celdas con la cadena “A” para posteriormente almacenar la palabra “OK” y visualizarla en pantalla en la primera línea.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC30
cd EjemploC30
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs
using System; using System.Drawing; using System.Windows.Forms; /// /// EjemploC30

```

```

using System;
using System.Drawing;
using System.Windows.Forms;

```

```

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1;
    private int backgroundColorSwitch = 0;
    private int fontColorSwitch = 2; // Default font color is cyan, cyan=2
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string[,] textGrid = new string[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 8);

    public GridForm()
    {
        Text = "Fixed 80x25 Grid of 'A's with Color Switches";
        ClientSize = new Size(NumColumns * 10, NumRows * 20);
        StartPosition = FormStartPosition.CenterScreen;
        DoubleBuffered = true;
        BackColor = Color.White;
        ForeColor = Color.Black; // Set default ForeColor for text

        // Populate the rectangle and text arrays with the fixed dimensions
        int rectangleWidth = ClientSize.Width / NumColumns;
        int rectangleHeight = ClientSize.Height / NumRows;

        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
                textGrid[row, col] = "A";
            }
        }

        Paint += GridForm_Paint;
        Resize += GridForm_Resize;
        KeyDown += GridForm_KeyDown;
    }
}

```

/// el programa EjemploC30 continúa en la siguiente pág.

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    switch (lineColorSwitch)          /// Determine line color
    {
        case 1:
            gridColor = Color.Black;
            break;
        case 2:
            gridColor = Color.Cyan;
            break;
        case 0:
            gridColor = Color.White;
            break;
        default:
            gridColor = Color.Black;
            break;
    }

    switch (backgroundColorSwitch)    /// Determine background color
    {
        case 1:
            BackColor = Color.Black; break;
        case 0:
            BackColor = Color.White; break;
        default:
            BackColor = Color.White; break;
    }

    // Determine font color
    Color textColor;
    switch (fontColorSwitch)
    {
        case 1:
            textColor = Color.Black;
            break;
        case 2:
            textColor = Color.Cyan;
            break;
        case 0:
            textColor = Color.White;
            break;
        default:
            textColor = ForeColor; break;
    }

    using (Pen pen = new Pen(gridColor))
    using (StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center })
    using (Brush textBrush = new SolidBrush(textColor))
    {
        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);
                e.Graphics.DrawString(textGrid[row, col], displayFont, textBrush,
gridRectangles[row, col], sf);
            }
        }
    }
}

/// el programa EjemploC30 continúa en la siguiente página.

```

```

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    Invalidate();
}

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    // Switch for line color
    if (e.KeyCode == Keys.D1) lineColorSwitch = 1;
    else if (e.KeyCode == Keys.D2) lineColorSwitch = 2;
    else if (e.KeyCode == Keys.D0 && !e.Shift && !e.Control) lineColorSwitch = 0; // Only
'0'

    // Switch for background color
    if (e.KeyCode == Keys.D1 && e.Shift) backgroundColorSwitch = 1;
    else if (e.KeyCode == Keys.D0 && e.Shift) backgroundColorSwitch = 0;

    // Switch for font color (using Control modifier)
    if (e.KeyCode == Keys.D1 && e.Control) fontColorSwitch = 1; // Ctrl + '1' for black
font
    else if (e.KeyCode == Keys.D2 && e.Control) fontColorSwitch = 2; // Ctrl + '2' for
cyan font
    else if (e.KeyCode == Keys.D0 && e.Control) fontColorSwitch = 0; // Ctrl + '0' for
white font

    Invalidate();
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}
}

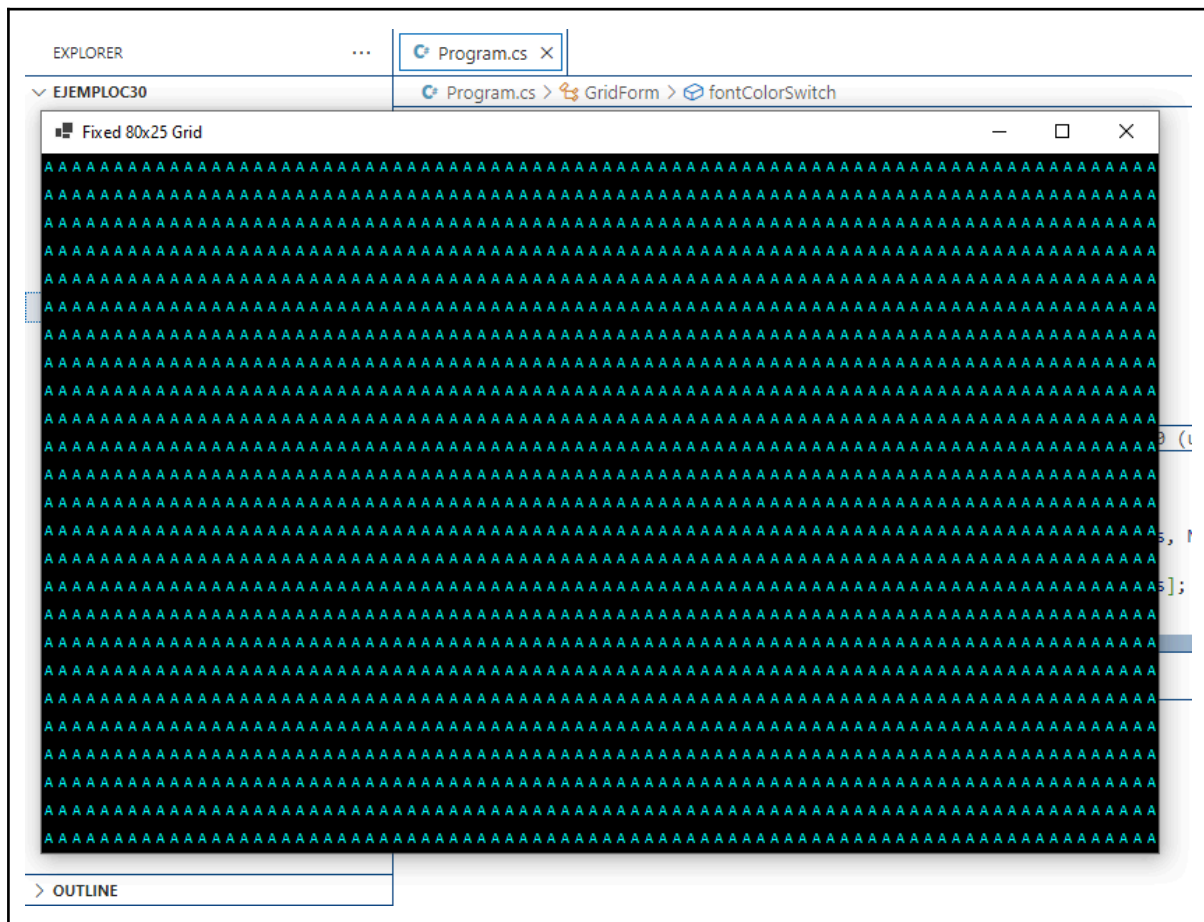
```

```

/// Fin del programa EjemploC30

```

A continuación ejecute el EjemploC30 creado y verá la siguiente ventana:



Nota:

Durante la ejecución del programa, puede cambiar los colores de ciertos elementos de la ventana pulsando ciertas combinaciones de teclas (CTRL+1 o SHIFT+1).

Si desea ver las líneas que delimitan las celdas, pulse simplemente “1”.

Si desea hacer las líneas invisibles pulse “0”.

Si desea que las líneas aparezcan en color “cyan”, pulse “2”.

Ejemplo C31. Creación de una matriz con cambio de colores de fondo e impresión de caracteres (letras) en la rejilla (grid).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras en posiciones especificadas por el usuario. Se llenarán las celdas con la cadena vacía “ ” para posteriormente almacenar la letra “H” y visualizarla en pantalla en la primera línea.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC31
cd EjemploC31
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:


```

/// Program.cs /// Este programa imprime letras en las coordenadas especificadas con la
///                /// función superprint()
using System; using System.Drawing; using System.Windows.Forms; /// /// EjemploC31

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1;
    private int backgroundColorSwitch = 0;
    private int globalFontColorSwitch = 0; // Global font color switch
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string[,] textGrid = new string[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 8);

    public GridForm()
    {
        Text = "Fixed 80x25 Grid with Superprint";
        ClientSize = new Size(NumColumns * 10, NumRows * 20);
        StartPosition = FormStartPosition.CenterScreen;
        DoubleBuffered = true;
        BackColor = Color.White;
        ForeColor = Color.Black;

        // Populate the rectangle array with the fixed dimensions
        int rectangleWidth = ClientSize.Width / NumColumns;
        int rectangleHeight = ClientSize.Height / NumRows;

        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
                textGrid[row, col] = ""; // Initialize textGrid with empty strings
            }
        }

        // Example of using superprint
        superprint(0, 0, "H", 1); // Use font color code 1 (black)
        superprint(5, 10, "*", 2); // Use font color code 2 (cyan)
        // superprint(79, 24, "Z", 0); // Última posición de la grid.Font color code 0 (white)

        Paint += GridForm_Paint;
        Resize += GridForm_Resize;
        KeyDown += GridForm_KeyDown;
    }
}

```

/// el programa EjemploC31 continúa en la siguiente página.

```
public void superprint(int column_num, int row_num, string stringx, int fontColorCode)
{
    if (column_num >= 0 && column_num < NumColumns && row_num >= 0 && row_num < NumRows)
    {
        textGrid[row_num, column_num] = stringx;
        // We don't directly set the color here anymore; it's handled in Paint
        Invalidate(); // Force a repaint to show the new string
    }
    else
    {
        Console.WriteLine($"Warning: superprint coordinates ({column_num},{row_num}) are
out of bounds.");
    }
}
```

/// el programa EjemploC31 continúa en la siguiente pág.

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    Color currentLineColor = gridColor;          /// Determine line color

    switch (lineColorSwitch)
    {
        case 1:
            currentLineColor = Color.Black; break;
        case 2:
            currentLineColor = Color.Cyan; break;
        case 0:
            currentLineColor = Color.White; break;
    }

    // Determine background color
    Color currentBackColor = BackColor;
    switch (backgroundColorSwitch)
    {
        case 1:
            currentBackColor = Color.Black; break;
        case 0:
            currentBackColor = Color.White; break;
    }
    BackColor = currentBackColor; // Apply background color

    using (Pen pen = new Pen(currentLineColor))
    using (StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center })
    {
        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);

                // Determine font color for the current cell
                Color currentFontColor;
                // For now, we'll use the global font color switch for all text
                switch (globalFontColorSwitch)
                {
                    case 1:
                        currentFontColor = Color.Black;
                        break;
                    case 2:
                        currentFontColor = Color.Cyan;
                        break;
                    case 0:
                        currentFontColor = Color.White;
                        break;
                    default:
                        currentFontColor = ForeColor;
                        break;
                }
                using (Brush textBrush = new SolidBrush(currentFontColor))
                {
                    e.Graphics.DrawString(textGrid[row, col], displayFont, textBrush,
gridRectangles[row, col], sf);
                }
            }
        }
    }
}

/// el programa EjemploC31 continúa en la siguiente pág.

```

```

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    Invalidate();
}

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    // Switch for line color
    if (e.KeyCode == Keys.D1) lineColorSwitch = 1;
    else if (e.KeyCode == Keys.D2) lineColorSwitch = 2;
    else if (e.KeyCode == Keys.D0 && !e.Shift && !e.Control) lineColorSwitch = 0;

    // Switch for background color
    if (e.KeyCode == Keys.D1 && e.Shift) backgroundColorSwitch = 1;
    else if (e.KeyCode == Keys.D0 && e.Shift) backgroundColorSwitch = 0;

    // Switch for global font color
    if (e.KeyCode == Keys.D1 && e.Control) globalFontColorSwitch = 1;
    else if (e.KeyCode == Keys.D2 && e.Control) globalFontColorSwitch = 2;
    else if (e.KeyCode == Keys.D0 && e.Control) globalFontColorSwitch = 0;

    Invalidate();
}

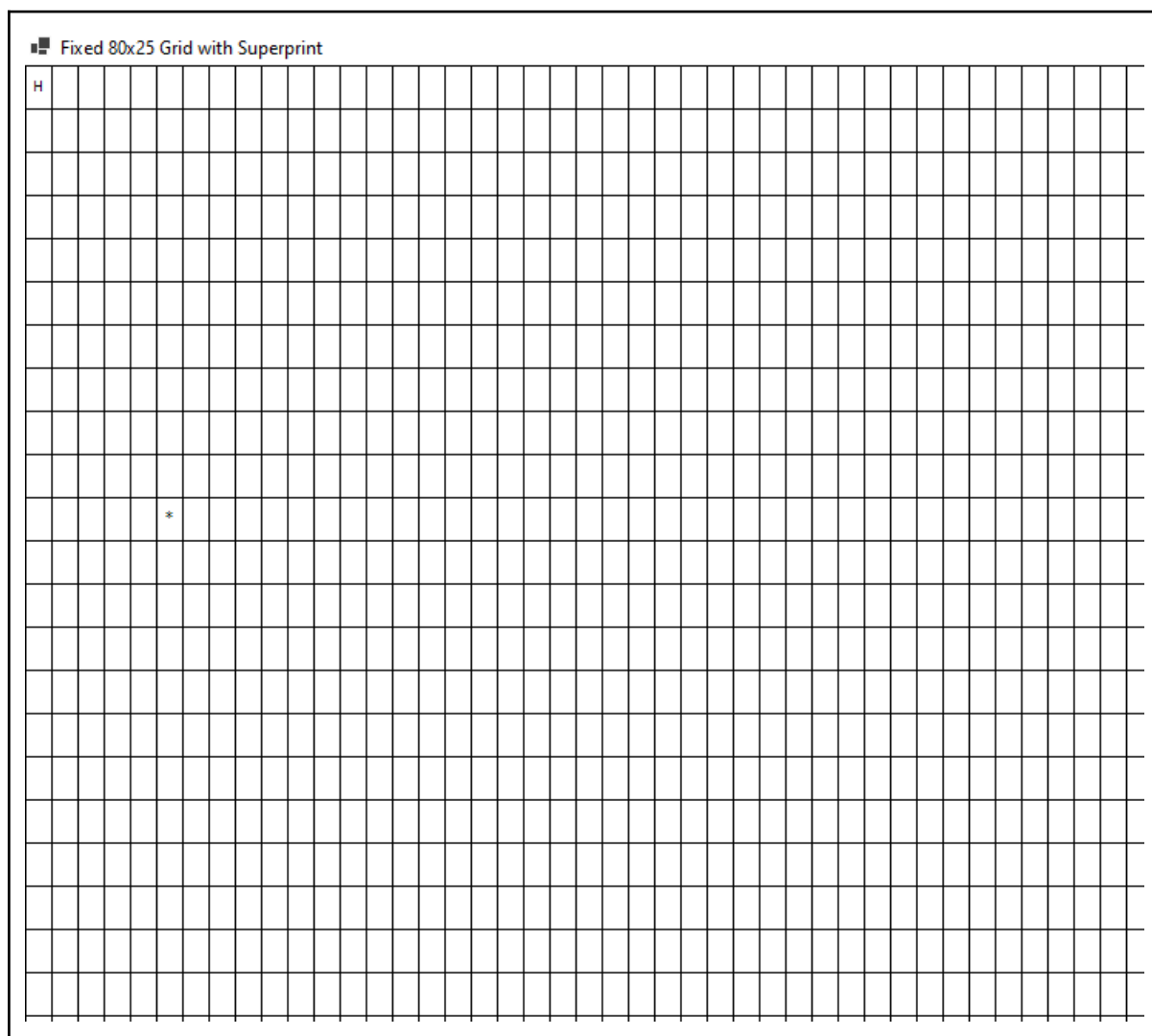
[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}

}

/// Fin del programa EjemploC31

```

A continuación ejecute el EjemploC31 creado y verá la siguiente ventana:



Durante la ejecución del programa, puede cambiar los colores de ciertos elementos de la ventana pulsando ciertas combinaciones de teclas (CTRL+1 o SHIFT+1).

Si desea ver las líneas que delimitan las celdas, pulse simplemente “1”.

Si desea hacer las líneas invisibles pulse “0”.

Si desea que las líneas aparezcan en color “cyan”, pulse “2”.

Ejemplo C32. Creación de una matriz con cambio de colores de fondo e impresión de palabras en la rejilla (grid).

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras en posiciones especificadas por el usuario. Se llenarán las celdas con la cadena vacía “ ” para posteriormente almacenar la palabra “**Pic**” y visualizarla en pantalla en la cuarta línea (que corresponde a row 3).

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC32
cd EjemploC32
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs /// Este programa imprime letras en las coordenadas especificadas con la
///              /// función superprint() y la función superprintword()
using System; using System.Drawing; using System.Windows.Forms; /// /// EjemploC32

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1;
    private int backgroundColorSwitch = 0;
    private int globalFontColorSwitch = 0;
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string[,] textGrid = new string[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 8);

    public GridForm()
    {
        Text = "Fixed 80x25 Grid with SuperprintWord";
        ClientSize = new Size(NumColumns * 10, NumRows * 20);
        StartPosition = FormStartPosition.CenterScreen;
        DoubleBuffered = true;
        BackColor = Color.White;
        ForeColor = Color.Black;

        // Populate the rectangle array with the fixed dimensions
        int rectangleWidth = ClientSize.Width / NumColumns;
        int rectangleHeight = ClientSize.Height / NumRows;

        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
                textGrid[row, col] = ""; // Initialize textGrid with empty strings
            }
        }

        // Example of using superprint and superprintword
        superprint(0, 0, "H", 1);
        superprint(5, 10, "*", 2);
        // superprint(79, 24, "Z", 0); // última posición en la grid
        superprintword(0, 3, "Pic", 2); // Print "Pic" at (0, 3) with cyan font
        superprintword(10, 5, "Hello", 0); // Print "Hello" at (10, 5) with white font

        Paint += GridForm_Paint;
        Resize += GridForm_Resize;
        KeyDown += GridForm_KeyDown;
    }
}

```

```

public void superprint(int column_num, int row_num, string stringx, int fontColorCode)
{
    if (column_num >= 0 && column_num < NumColumns && row_num >= 0 && row_num <
NumRows)
    {
        textGrid[row_num, column_num] = stringx;
        Invalidate(); // Force a repaint to show the new string
    }
    else
    {
        Console.WriteLine($"Warning: superprint coordinates ({column_num},{row_num})
are out of bounds.");
    }
}

public void superprintword(int column_num, int row_num, string wordx, int font_colour)
{
    for (int i = 0; i < wordx.Length; i++)
    {
        int currentColumn = column_num + i;
        if (currentColumn < NumColumns) // Make sure we don't go out of bounds
horizontally
        {
            superprint(currentColumn, row_num, wordx[i].ToString(), font_colour);
        }
        else
        {
            Console.WriteLine($"Warning: superprintword '{wordx}' at
({column_num},{row_num}) exceeds grid width.");
            break; // Stop printing if we reach the edge
        }
    }
}

```



```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    // Determine line color
    Color currentLineColor = gridColor;
    switch (lineColorSwitch)
    {
        case 1:
            currentLineColor = Color.Black; break;
        case 2:
            currentLineColor = Color.Cyan; break;
        case 0:
            currentLineColor = Color.White; break;
    }

    // Determine background color
    Color currentBackColor = BackColor;
    switch (backgroundColorSwitch)
    {
        case 1:
            currentBackColor = Color.Black; break;
        case 0:
            currentBackColor = Color.White; break;
        default:
            currentBackColor = Color.White; break;
    }
    BackColor = currentBackColor; // Apply background color

    // Determine font color
    Color currentFontColor;
    switch (globalFontColorSwitch)
    {
        case 1:
            currentFontColor = Color.Black; break;
        case 2:
            currentFontColor = Color.Cyan; break;
        case 0:
            currentFontColor = Color.White; break;
        default:
            currentFontColor = ForeColor; break;
    }

    using (Pen pen = new Pen(currentLineColor))
    using (StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center })
    using (Brush textBrush = new SolidBrush(currentFontColor))
    {
        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);
                e.Graphics.DrawString(textGrid[row, col], displayFont, textBrush,
gridRectangles[row, col], sf);
            }
        }
    }
}

```

```

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    Invalidate();
}

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    // Switch for line color
    if (e.KeyCode == Keys.D1) lineColorSwitch = 1;
    else if (e.KeyCode == Keys.D2) lineColorSwitch = 2;
    else if (e.KeyCode == Keys.D0 && !e.Shift && !e.Control) lineColorSwitch = 0;

    // Switch for background color
    if (e.KeyCode == Keys.D1 && e.Shift) backgroundColorSwitch = 1;
    else if (e.KeyCode == Keys.D0 && e.Shift) backgroundColorSwitch = 0;

    // Switch for global font color
    if (e.KeyCode == Keys.D1 && e.Control) globalFontColorSwitch = 1;
    else if (e.KeyCode == Keys.D2 && e.Control) globalFontColorSwitch = 2;
    else if (e.KeyCode == Keys.D0 && e.Control) globalFontColorSwitch = 0;

    Invalidate();
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}
}

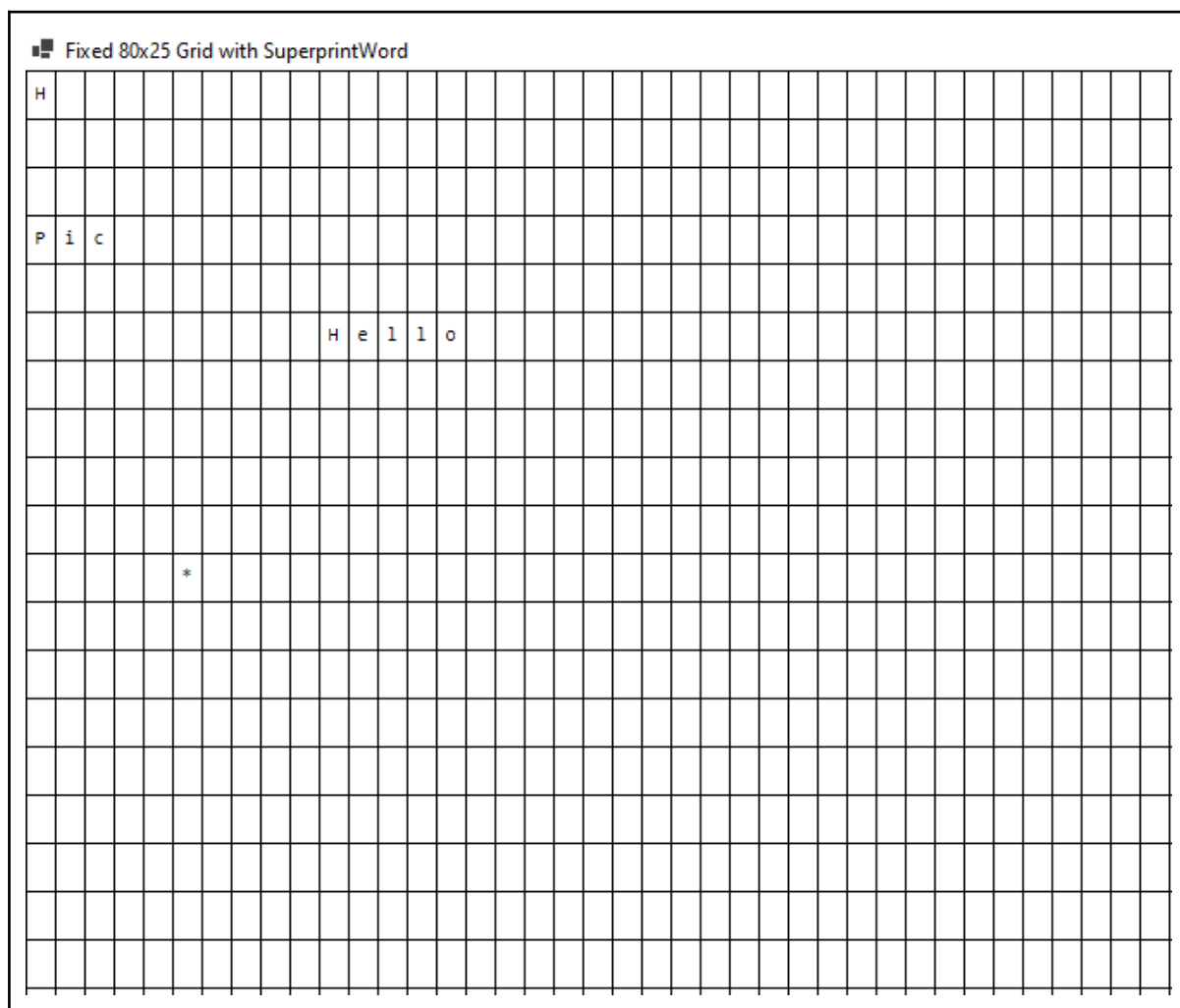
```

```

/// Fin del programa EjemploC32

```

A continuación ejecute el EjemploC32 creado y verá la siguiente ventana:



Durante la ejecución del programa, puede cambiar los colores de ciertos elementos de la ventana pulsando ciertas combinaciones de teclas (CTRL+1 o SHIFT+1).

Si desea ver las líneas que delimitan las celdas, pulse simplemente "1".

Si desea hacer las líneas invisibles pulse "0".

Si desea que las líneas aparezcan en color "cyan", pulse "2".

Ejemplo C33. Creación de una matriz con cambio de colores de fondo e impresión de palabras en la rejilla (grid). Implementación del menú clásico de Microsoft Windows con los elementos: “About”, “View” y “Settings”.

Objetivo: Escribir un programa para crear una matriz (array) de 25 x 80 posiciones para almacenar letras en posiciones especificadas por el usuario. Se llenarán las celdas con la cadena vacía “ ” para posteriormente almacenar la palabra **“Pic”** y visualizarla en pantalla en la cuarta línea (que corresponde a row 3).

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC33
cd EjemploC33
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs /// Este programa imprime letras en las coordenadas especificadas con la
///              /// función superprint() y la función superprintword()

```

```

using System;                                     /// CTRL+M hides MENU // EJEMPLOC33
using System.Drawing;
using System.Windows.Forms;

```

```

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1; // 1: Black, 2: Cyan, 0: White
    private int backgroundColorSwitch = 0; // 1: Black, 0: White
    private int globalFontColorSwitch = 0; // 1: Black, 2: Cyan, 0: White
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string[,] textGrid = new string[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 8);
    private MenuStrip mainMenuStrip;
    private ToolStripMenuItem aboutMenuItem;
    private ToolStripMenuItem viewMenuItem;
    private ToolStripMenuItem settingsMenuItem;
    private ToolStripMenuItem darkModeMenuItem;
    private ToolStripMenuItem classicModeMenuItem;
    private ToolStripMenuItem showRulerMenuItem;
    private ToolStripMenuItem hideRulerMenuItem;
    private bool isRulerVisible = false;

```

```

public GridForm()
{
    Text = "Fixed 80x25 Grid with Menu and Ruler";
    ClientSize = new Size(NumColumns * 10, NumRows * 20);
    StartPosition = FormStartPosition.CenterScreen;
    DoubleBuffered = true;
    BackColor = Color.White; ForeColor = Color.Black;

    mainMenuStrip = new MenuStrip();           /// Create the MenuStrip
    Controls.Add(mainMenuStrip);
    mainMenuStrip.Dock = DockStyle.Top;

    aboutMenuItem = new ToolStripMenuItem("About");    /// Create menu items
    viewMenuItem = new ToolStripMenuItem("View");
    settingsMenuItem = new ToolStripMenuItem("Settings");

    // Create submenu items for View
    showRulerMenuItem = new ToolStripMenuItem("Show Ruler");
    showRulerMenuItem.Click += ShowRulerMenuItem_Click;
    hideRulerMenuItem = new ToolStripMenuItem("Hide Ruler");
    hideRulerMenuItem.Click += HideRulerMenuItem_Click;

    // Create submenu items for Settings
    darkModeMenuItem = new ToolStripMenuItem("Dark Mode");
    darkModeMenuItem.Click += DarkModeMenuItem_Click;
    classicModeMenuItem = new ToolStripMenuItem("Classic Mode");
    classicModeMenuItem.Click += ClassicModeMenuItem_Click;

    // Add submenu items to View
    viewMenuItem.DropDownItems.Add(showRulerMenuItem);
    viewMenuItem.DropDownItems.Add(hideRulerMenuItem);

    // Add submenu items to Settings
    settingsMenuItem.DropDownItems.Add(darkModeMenuItem);
    settingsMenuItem.DropDownItems.Add(classicModeMenuItem);

    // Add top-level menu items to the MenuStrip
    mainMenuStrip.Items.Add(aboutMenuItem);
    mainMenuStrip.Items.Add(viewMenuItem);
    mainMenuStrip.Items.Add(settingsMenuItem);

    // Populate the rectangle array with the fixed dimensions
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
            textGrid[row, col] = ""; // Initialize textGrid with empty strings
        }
    }

    /// Example of using superprint and superprintword
    superprint(0, 0, "H", 1); superprint(79, 24, "Z", 0);
    superprintword(0, 3, "Pic", 2); superprintword(10, 5, "Hello", 0);

    Paint += GridForm_Paint;
    Resize += GridForm_Resize;
    KeyDown += GridForm_KeyDown;
}

```

```

private void ShowRulerMenuItem_Click(object sender, EventArgs e)
{
    superprintword(0, 0,
"0123456789012345678901234567890123456789012345678901234567890123456789", 1);
    isRulerVisible = true;
}

private void HideRulerMenuItem_Click(object sender, EventArgs e)
{
    superprintword(0, 0, new string(' ', 80), 0); // Print spaces to hide, using white
font
    isRulerVisible = false;
}

private void DarkModeMenuItem_Click(object sender, EventArgs e)
{
    backgroundColorSwitch = 1; // Black background
    globalFontColorSwitch = 2; // Cyan fonts
    lineColorSwitch = 1;      // Black lines
    Invalidate();
}

private void ClassicModeMenuItem_Click(object sender, EventArgs e)
{
    backgroundColorSwitch = 0; // White background (classic light)
    globalFontColorSwitch = 1; // Black fonts (classic dark)
    lineColorSwitch = 0;      // White lines (to appear invisible on white background)
    Invalidate();
}

public void superprint(int column_num, int row_num, string stringx, int fontColorCode)
{
    if (column_num >= 0 && column_num < NumColumns && row_num >= 0 && row_num < NumRows)
    {
        textGrid[row_num, column_num] = stringx;
        Invalidate();
    }
    else
    {
        Console.WriteLine($"Warning: superprint coordinates ({column_num},{row_num}) are
out of bounds.");
    }
}

public void superprintword(int column_num, int row_num, string wordx, int font_colour)
{
    for (int i = 0; i < wordx.Length; i++)
    {
        int currentColumn = column_num + i;
        if (currentColumn < NumColumns)
        {
            superprint(currentColumn, row_num, wordx[i].ToString(), font_colour);
        }
        else
        {
            Console.WriteLine($"Warning: superprintword '{wordx}' at
({column_num},{row_num}) exceeds grid width.");
            break;
        }
    }
}

```

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    // Determine line color
    Color currentLineColor = gridColor;
    switch (lineColorSwitch)
    {
        case 1:
            currentLineColor = Color.Black;
            break;
        case 2:
            currentLineColor = Color.Cyan;
            break;
        case 0:
            currentLineColor = Color.White;
            break;
    }

    // Determine background color
    Color currentBackColor = BackColor;
    switch (backgroundColorSwitch)
    {
        case 1:
            currentBackColor = Color.Black; break;
        case 0:
            currentBackColor = Color.White; break;
        default:
            currentBackColor = Color.White; break;
    }
    BackColor = currentBackColor; // Apply background color

    Color currentFontColor; // Determine font color
    switch (globalFontColorSwitch)
    {
        case 1:
            currentFontColor = Color.Black;
            break;
        case 2:
            currentFontColor = Color.Cyan;
            break;
        case 0:
            currentFontColor = Color.White;
            break;
        default:
            currentFontColor = ForeColor;
            break;
    }

    using (Pen pen = new Pen(currentLineColor))
    using (StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center })
    using (Brush textBrush = new SolidBrush(currentFontColor))
    {
        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);
                e.Graphics.DrawString(textGrid[row, col], displayFont, textBrush,
gridRectangles[row, col], sf);
            }
        }
    }
}

```



```

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    Invalidate();
}

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    // Toggle menu visibility with Ctrl+M
    if (e.Control && (e.KeyCode == Keys.M))
    {
        mainMenuStrip.Visible = !mainMenuStrip.Visible;
    }
    // Trigger superprintword with Ctrl+R
    else if (e.Control && (e.KeyCode == Keys.R))
    {
        superprintword(0, 0,
"012345678901234567890123456789012345678901234567890123456789", 1);
        isRulerVisible = true;
    }
    else
    {
        // Switch for line color
        if (e.KeyCode == Keys.D1) lineColorSwitch = 1;
        else if (e.KeyCode == Keys.D2) lineColorSwitch = 2;
        else if (e.KeyCode == Keys.D0 && !e.Shift && !e.Control) lineColorSwitch = 0;

        // Switch for background color
        if (e.KeyCode == Keys.D1 && e.Shift) backgroundColorSwitch = 1;
        else if (e.KeyCode == Keys.D0 && e.Shift) backgroundColorSwitch = 0;

        // Switch for global font color
        if (e.KeyCode == Keys.D1 && e.Control) globalFontColorSwitch = 1;
        else if (e.KeyCode == Keys.D2 && e.Control) globalFontColorSwitch = 2;
        else if (e.KeyCode == Keys.D0 && e.Control) globalFontColorSwitch = 0;
    }

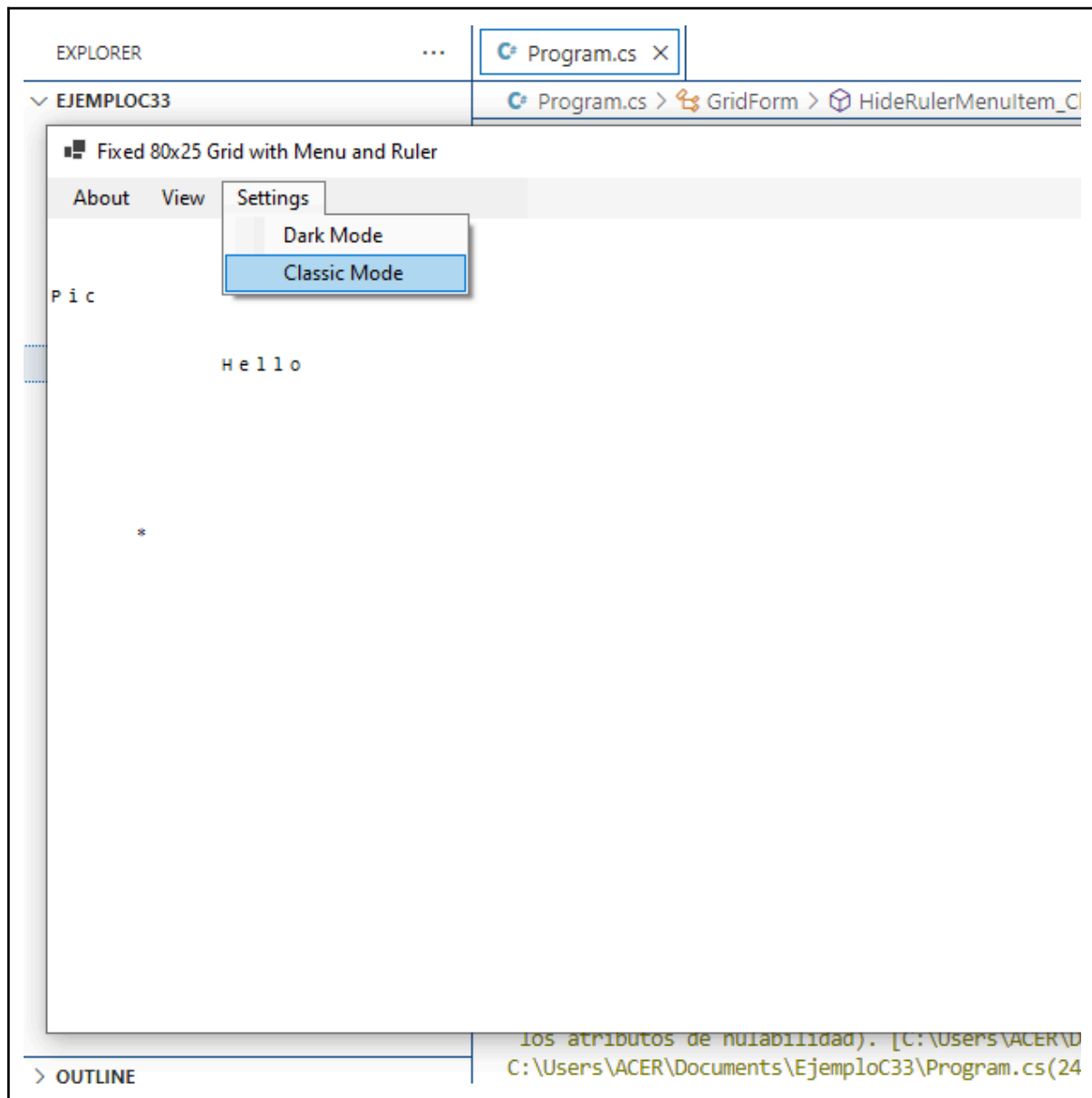
    Invalidate(); // Invalidate to ensure any visual changes are redrawn
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}
}

/// Fin del programa EjemploC33

```

A continuación ejecute el EjemploC33 creado y verá la siguiente ventana:



Durante la ejecución del programa, puede cambiar los colores de ciertos elementos de la ventana activando el menú principal.

Para ver la opción “Settings” (ajustes), pulse CTRL+M y seleccione “Dark mode” o “Classic Mode”.

Para ver la numeración de las columnas en la parte superior de la ventana, pulse “View” y seleccione “View Ruler”.

Ejemplo C34. Creación de un test de 3 preguntas. Implementación del menú clásico de Microsoft Windows con los elementos: “About”, ”Start”, “View” y “Settings”.

Objetivo: Escribir el código para realizar un test de 3 preguntas con un formulario. Las preguntas estarán almacenadas en una matriz. Las respuestas se introducirán por teclado.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC34
cd EjemploC34
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs ///                                     /// EjemploC34. Test de tres preguntas.

using System;
using System.Drawing;
using System.Windows.Forms;

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1; // 1: Black, 2: Cyan, 0: White
    private int backgroundColorSwitch = 0; // 1: Black, 0: White
    private int globalFontColorSwitch = 0; // 1: Black, 2: Cyan, 0: White
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string[,] textGrid = new string[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 8);
    private MenuStrip mainMenuStrip;
    private ToolStripMenuItem aboutMenuItem;
    private ToolStripMenuItem viewMenuItem;
    private ToolStripMenuItem settingsMenuItem;
    private ToolStripMenuItem darkModeMenuItem;
    private ToolStripMenuItem classicModeMenuItem;
    private ToolStripMenuItem showRulerMenuItem;
    private ToolStripMenuItem hideRulerMenuItem;
    private bool isRulerVisible = false;
    private ToolStripMenuItem startMenuItem;
    private ToolStripMenuItem startEasyTestMenuItem;
    private ToolStripMenuItem startDifficultTestMenuItem;
    private Label questionWidthLabel; // Label to display question width

    // Global variable for maximum questions
    private const int max_questions = 50;

    // Global array for the first 3 questions
    private string[] questions_array = new string[3]
    {
        "Question 1: Is Basic a computer programming language?",
        "Question 2: Is Cobol a computer programming language?",
        "Question 3: Is Pascal a computer programming language?"
    };

    //continúa más abajo. . .

```

```

public GridForm()
{
    Text = "Fixed 80x25 Grid with Initial Dark Mode";
    ClientSize = new Size(NumColumns * 10, NumRows * 20);
    StartPosition = FormStartPosition.CenterScreen;
    DoubleBuffered = true;
    BackColor = Color.White;
    ForeColor = Color.Black;

    // Create the MenuStrip
    mainMenuStrip = new MenuStrip();
    Controls.Add(mainMenuStrip);
    mainMenuStrip.Dock = DockStyle.Top;

    // Create menu items
    aboutMenuItem = new ToolStripMenuItem("About");
    aboutMenuItem.Click += AboutMenuItem_Click; // Attach event handler
    viewMenuItem = new ToolStripMenuItem("View");
    settingsMenuItem = new ToolStripMenuItem("Settings");
    startMenuItem = new ToolStripMenuItem("Start");

    // Create submenu items for View
    showRulerMenuItem = new ToolStripMenuItem("Show Ruler");
    showRulerMenuItem.Click += ShowRulerMenuItem_Click;
    hideRulerMenuItem = new ToolStripMenuItem("Hide Ruler");
    hideRulerMenuItem.Click += HideRulerMenuItem_Click;

    // Create submenu items for Settings
    darkModeMenuItem = new ToolStripMenuItem("Dark Mode");
    darkModeMenuItem.Click += DarkModeMenuItem_Click;
    classicModeMenuItem = new ToolStripMenuItem("Classic Mode");
    classicModeMenuItem.Click += ClassicModeMenuItem_Click;

    // Create submenu items for Start
    startEasyTestMenuItem = new ToolStripMenuItem("Start the easy test");
    startEasyTestMenuItem.Click += StartEasyTestMenuItem_Click;
    startDifficultTestMenuItem = new ToolStripMenuItem("Start the difficult test");
    startDifficultTestMenuItem.Click += StartDifficultTestMenuItem_Click;

    // Add submenu items to View
    viewMenuItem.DropDownItems.Add(showRulerMenuItem);
    viewMenuItem.DropDownItems.Add(hideRulerMenuItem);

    // Add submenu items to Settings
    settingsMenuItem.DropDownItems.Add(darkModeMenuItem);
    settingsMenuItem.DropDownItems.Add(classicModeMenuItem);

    // Add submenu items to Start
    startMenuItem.DropDownItems.Add(startEasyTestMenuItem);
    startMenuItem.DropDownItems.Add(startDifficultTestMenuItem);

    // Add top-level menu items to the MenuStrip
    mainMenuStrip.Items.Add(aboutMenuItem);
    mainMenuStrip.Items.Add(startMenuItem);
    mainMenuStrip.Items.Add(viewMenuItem);
    mainMenuStrip.Items.Add(settingsMenuItem);

    // Populate the rectangle array with the fixed dimensions
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;
    //public GridForm continua más abajo. . .

```

```

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
            textGrid[row, col] = ""; // Initialize textGrid with empty strings
        }
    }

    // Start in dark mode
    start_in_dark_mode();

    // Create label for question width display
    questionWidthLabel = new Label();
    questionWidthLabel.ForeColor = Color.Cyan; // Set font color
    questionWidthLabel.BackColor = Color.Black;
    questionWidthLabel.AutoSize = true;
    questionWidthLabel.Location = new Point(10, ClientSize.Height -
questionWidthLabel.Height - 10); // Bottom-left corner
    Controls.Add(questionWidthLabel);

    // Example of using superprint and superprintword (using current font color)
    // superprint(0, 0, "H", globalFontColorSwitch);
    // superprint(79, 24, "Z", globalFontColorSwitch);
    superprintword(1, 3, "Hello, developer.", globalFontColorSwitch);
    superprintword(1, 5, "Welcome to C#", globalFontColorSwitch);

    Paint += GridForm_Paint;
    Resize += GridForm_Resize;
    KeyDown += GridForm_KeyDown;
    Resize += GridForm_Resize; //listen to resize event.
}

private void start_in_dark_mode()
{
    backgroundColorSwitch = 1; // Black background
    globalFontColorSwitch = 2; // Cyan fonts
    lineColorSwitch = 1;      // Black lines
    BackColor = Color.Black;
    ForeColor = Color.Cyan;
}

//continúa más abajo. . .

```

```

private void test3()
{
    for (int i = 0; i < questions_array.Length; i++)
    {
        string question = questions_array[i];

        // Use a Form to display the question and a TextBox for input
        Form inputForm = new Form();
        inputForm.Text = "Question " + (i + 1);
        inputForm.StartPosition = FormStartPosition.CenterScreen;
        inputForm.BackColor = Color.Black;
        inputForm.ForeColor = Color.Cyan;

        Label questionLabel = new Label();
        questionLabel.Text = question;
        questionLabel.ForeColor = Color.Cyan;
        questionLabel.AutoSize = false;
        questionLabel.MaximumSize = new Size(560, 0); // Wrap text at 560px
        questionLabel.Size = new Size(560, questionLabel.PreferredHeight);
        questionLabel.Location = new Point(10, 10);

        TextBox answerTextBox = new TextBox();
        answerTextBox.BackColor = Color.Black;
        answerTextBox.ForeColor = Color.Cyan;
        answerTextBox.Location = new Point(10, questionLabel.Bottom + 10);
        answerTextBox.Width = 560;

        Button okButton = new Button();
        okButton.Text = "OK";
        okButton.DialogResult = DialogResult.OK;
        okButton.BackColor = Color.Black;
        okButton.ForeColor = Color.Cyan;
        okButton.Location = new Point(inputForm.ClientSize.Width - 90, answerTextBox.Bottom +
20);

    }

    /// private void test3() continúa más abajo. . .

```

```

// Calculate form size based on contents (with minimum width)
int desiredWidth = Math.Max(600, questionLabel.Width + 40); // leave padding
int desiredHeight = okButton.Bottom + 60;
inputForm.ClientSize = new Size(desiredWidth, desiredHeight);

// Adjust OK button location now that form size is known
okButton.Location = new Point(inputForm.ClientSize.Width - okButton.Width - 20,
answerTextBox.Bottom + 20);

// Add controls
inputForm.Controls.Add(questionLabel);
inputForm.Controls.Add(answerTextBox);
inputForm.Controls.Add(okButton);
inputForm.AcceptButton = okButton;

// Display width in main form
int questionFormWidth = inputForm.Width;
questionWidthLabel.Text = "Question Form Width: " + questionFormWidth;

string answer = "";
if (inputForm.ShowDialog() == DialogResult.OK)
{
    answer = answerTextBox.Text;
}
inputForm.Dispose();

if (answer.ToLower() == "yes")
{
    MessageBox.Show("Correct!", "Answer " + (i + 1));
}
else
{
    MessageBox.Show("Incorrect!", "Answer " + (i + 1));
}
}
}

/// end of private void test3()

```

```

//continúa más abajo. . .

```



```

private void about_form()
{
    Form aboutForm = new Form();
    aboutForm.Text = "About";
    aboutForm.StartPosition = FormStartPosition.CenterScreen;
    aboutForm.Size = new Size(300, 200);
    aboutForm.BackColor = Color.Black;
    aboutForm.ForeColor = Color.Cyan;

    Label messageLabel = new Label();
    messageLabel.Text = "Thanks for running this app.\n\njavier@brownbook.cloud";
    messageLabel.ForeColor = Color.Cyan;
    messageLabel.AutoSize = true;
    messageLabel.Location = new Point(10, 10);

    Button closeButton = new Button();
    closeButton.Text = "Close";
    closeButton.DialogResult = DialogResult.OK;
    closeButton.BackColor = Color.Black;
    closeButton.ForeColor = Color.Cyan;
    closeButton.Location = new Point(aboutForm.ClientSize.Width - closeButton.Width - 10,
aboutForm.ClientSize.Height - closeButton.Height - 10);

    aboutForm.Controls.Add(messageLabel);
    aboutForm.Controls.Add(closeButton);
    aboutForm.AcceptButton = closeButton;

    aboutForm.ShowDialog();
    aboutForm.Dispose();
}

// Event handler for "About" menu item
private void AboutMenuItem_Click(object sender, EventArgs e)
{
    about_form();
}

// Event handler for "Start the easy test"
private void StartEasyTestMenuItem_Click(object sender, EventArgs e)
{
    test3();
}

// Event handler for "Start the difficult test"
private void StartDifficultTestMenuItem_Click(object sender, EventArgs e)
{
    test3();
}

//continúa más abajo. . .

```

```

private void ShowRulerMenuItem_Click(object sender, EventArgs e)
{
    superprintword(0, 0,
"012345678901234567890123456789012345678901234567890123456789",
globalFontColorSwitch);
    isRulerVisible = true;
}

private void HideRulerMenuItem_Click(object sender, EventArgs e)
{
    superprintword(0, 0, new string(' ', 79), globalFontColorSwitch); // Use current font
color for spaces
    isRulerVisible = false;
}

private void DarkModeMenuItem_Click(object sender, EventArgs e)
{
    start_in_dark_mode(); Invalidate();
}

private void ClassicModeMenuItem_Click(object sender, EventArgs e)
{
    backgroundColorSwitch = 0; // White background (classic light)
    globalFontColorSwitch = 1; // Black fonts (classic dark)
    lineColorSwitch = 0;      // White lines (to appear invisible on white background)
    BackColor = Color.White;
    ForeColor = Color.Black;
    Invalidate();
}

public void superprint(int column_num, int row_num, string stringx, int fontColorCode)
{
    if (column_num >= 0 && column_num < NumColumns && row_num >= 0 && row_num < NumRows)
    {
        textGrid[row_num, column_num] = stringx; Invalidate();
    }
    else
    {
        Console.WriteLine($"Warning: superprint coordinates ({column_num},{row_num}) are
out of bounds.");
    }
}

public void superprintword(int column_num, int row_num, string wordx, int font_colour)
{
    for (int i = 0; i < wordx.Length; i++)
    {
        int currentColumn = column_num + i;
        if (currentColumn < NumColumns)
        {
            superprint(currentColumn, row_num, wordx[i].ToString(), font_colour);
        }
        else
        {
            Console.WriteLine($"Warning: superprintword '{wordx}' at
({column_num},{row_num}) exceeds grid width.");
            break;
        }
    }
}

//continúa más abajo. . .

```

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    // Determine line color
    Color currentLineColor = gridColor;
    switch (lineColorSwitch)
    {
        case 1:
            currentLineColor = Color.Black;
            break;
        case 2:
            currentLineColor = Color.Cyan;
            break;
        case 0:
            currentLineColor = Color.White;
            break;
    }

    // Determine background color
    Color currentBackColor = BackColor;
    switch (backgroundColorSwitch)
    {
        case 1:
            currentBackColor = Color.Black;
            break;
        case 0:
            currentBackColor = Color.White;
            break;
        default:
            currentBackColor = Color.White;
            break;
    }
    BackColor = currentBackColor; // Apply background color

    // Determine font color
    Color currentFontColor;
    switch (globalFontColorSwitch)
    {
        case 1:
            currentFontColor = Color.Black;
            break;
        case 2:
            currentFontColor = Color.Cyan;
            break;
        case 0:
            currentFontColor = Color.White;
            break;
        default:
            currentFontColor = ForeColor;
            break;
    }

    //continúa más abajo. . .

```

```

        using (Pen pen = new Pen(currentLineColor))
        using (StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center })
        using (Brush textBrush = new SolidBrush(currentFontColor))
        {
            for (int row = 0; row < NumRows; row++)
            {
                for (int col = 0; col < NumColumns; col++)
                {
                    e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);
                    e.Graphics.DrawString(textGrid[row, col], displayFont, textBrush,
gridRectangles[row, col], sf);
                }
            }
        }
    }
}

```

```

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }
    //update the location of label.
    questionWidthLabel.Location = new Point(10, ClientSize.Height -
questionWidthLabel.Height - 10);
    Invalidate();
}

```

//continúa más abajo. . .

```

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    // Toggle menu visibility with Ctrl+M
    if (e.Control && (e.KeyCode == Keys.M))
    {
        mainMenuStrip.Visible = !mainMenuStrip.Visible;
    }
    // Trigger superprintword with Ctrl+R
    else if (e.Control && (e.KeyCode == Keys.R))
    {
        superprintword(0, 0,
"012345678901234567890123456789012345678901234567890123456789",
globalFontColorSwitch);
        isRulerVisible = true;
    }
    else
    {
        // Switch for line color
        if (e.KeyCode == Keys.D1) lineColorSwitch = 1;
        else if (e.KeyCode == Keys.D2) lineColorSwitch = 2;
        else if (e.KeyCode == Keys.D0 && !e.Shift && !e.Control) lineColorSwitch = 0;

        // Switch for background color
        if (e.KeyCode == Keys.D1 && e.Shift) backgroundColorSwitch = 1;
        else if (e.KeyCode == Keys.D0 && e.Shift) backgroundColorSwitch = 0;

        // Switch for global font color
        if (e.KeyCode == Keys.D1 && e.Control) globalFontColorSwitch = 1;
        else if (e.KeyCode == Keys.D2 && e.Control) globalFontColorSwitch = 2;
        else if (e.KeyCode == Keys.D0 && e.Control) globalFontColorSwitch = 0;
    }

    Invalidate(); // Invalidate to ensure any visual changes are redrawn
}

//private void MyTimer_Tick(object sender, EventArgs e)
//{
//    //update the location of label.
//    questionWidthLabel.Location = new Point(10, ClientSize.Height -
questionWidthLabel.Height - 10);
//}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}

}

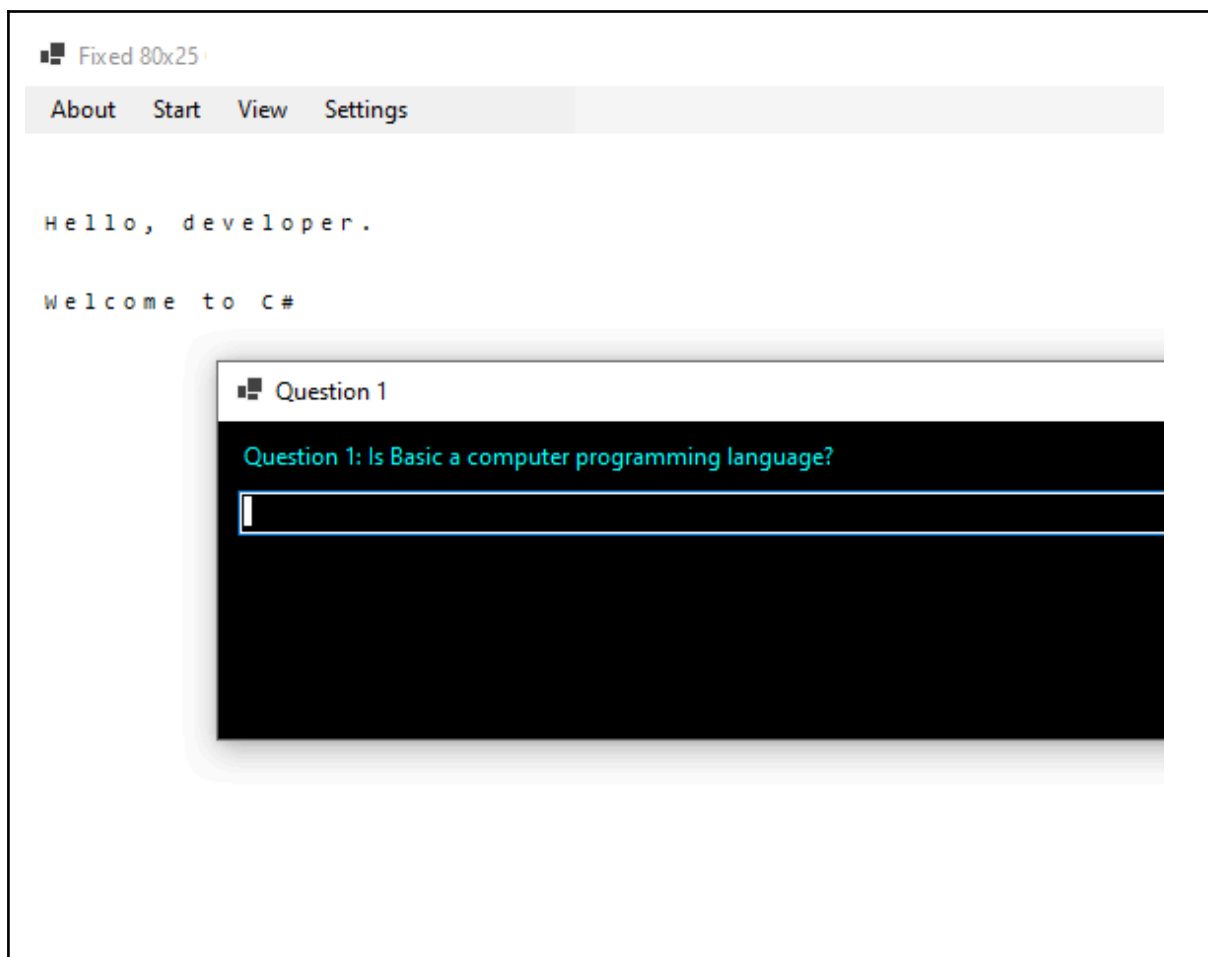
/// fin del EjemploC34

```

A continuación ejecute el EjemploC34 creado y verá la siguiente ventana:

Recuerde que durante la ejecución del programa, puede cambiar los colores de ciertos elementos de la ventana activando el menú principal. Para ello haga clic en “Settings” y elija una opción distinta a “Dark mode”.

Para empezar el test, pulse “Start” y elija un test.



¿Qué elementos hemos implementado en nuestro código al utilizar WinForms?

Hemos configurado correctamente una aplicación básica de WinForms heredando de **Form**, creado un punto de entrada **Main** con **Application.Run**, y utilizado controles estándar como **MenuStrip**, **ToolStripMenuItem**, **Label**, **TextBox** y **Button**.

Dibujo Personalizado:

Hemos utilizado el evento **Paint** (**GridForm_Paint**) para dibujar sobre la superficie del formulario usando **e.Graphics**. Este es el enfoque correcto para el renderizado personalizado en **WinForms**.

Doble Búfer:

Establecer **DoubleBuffered = true** es una buena práctica para el dibujo personalizado, ya que ayuda a reducir el parpadeo.

Manejo de Redimensionamiento:

El controlador de eventos **GridForm_Resize** recalcula correctamente los **gridRectangles** para adaptar las posiciones y tamaños de las celdas de la cuadrícula cuando se redimensiona el formulario.

Manejo de Eventos:

Hemos adjuntado correctamente los controladores de eventos (**+=**) a los elementos del menú, eventos de teclado y eventos del formulario (**Paint**, **Resize**, **KeyDown**).

Estructura del Menú:

La creación y población del **MenuStrip** con elementos **ToolStripMenuItem** anidados está implementada correctamente.

Atajos de Teclado:

Has implementado atajos de teclado (**Ctrl+M**, **Ctrl+R** y las teclas numéricas con modificadores) usando el evento **KeyDown**.

Objetos Desechables (Disposable):

Estás utilizando bloques **using** para **Pen**, **StringFormat** y **Brush** en el método **Paint**, lo cual es crucial para liberar correctamente los recursos GDI+. También estás liberando (**Dispose()**) los formularios creados dinámicamente (**inputForm**, **aboutForm**) después de mostrarlos con **ShowDialog()**.

Verificaciones de Límites:

superprint y **superprintword** incluyen verificaciones básicas para evitar dibujar fuera de los límites definidos de la cuadrícula.

Preguntas:

Enumere todas las funciones utilizadas en el código del EjemploC34.

¿Cómo añadiría un segundo test con preguntas con una complejidad más elevada?

Ejemplo C35. Creación de dos tests de 5 preguntas. Implementación del menú clásico de Microsoft Windows con los elementos: “About”, ”Start”, “View” y “Settings”.

Objetivo: Escribir el código para realizar dos tests de 5 preguntas con un formulario. Las preguntas estarán almacenadas en una matriz. Las respuestas se introducirán por teclado. Se programará un test fácil y un test más complicado. Se podrá seleccionar el tipo de test en el menú principal.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC35
cd EjemploC35
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs ///                                     /// EjemploC35. Test de cinco preguntas.

using System;
using System.Drawing;
using System.Windows.Forms;

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1; // 1: Black, 2: Cyan, 0: White
    private int backgroundColorSwitch = 0; // 1: Black, 0: White
    private int globalFontColorSwitch = 0; // 1: Black, 2: Cyan, 0: White
    private Color gridColor = Color.Black; // This variable is not currently used in
drawing
    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string[,] textGrid = new string[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 8);
    private MenuStrip mainMenuStrip;
    private ToolStripMenuItem aboutMenuItem;
    private ToolStripMenuItem viewMenuItem;
    private ToolStripMenuItem settingsMenuItem;
    private ToolStripMenuItem darkModeMenuItem;
    private ToolStripMenuItem classicModeMenuItem;
    private ToolStripMenuItem showRulerMenuItem;
    private ToolStripMenuItem hideRulerMenuItem;
    private bool isRulerVisible = false; // This variable is set but not used in logic
    private ToolStripMenuItem startMenuItem;
    private ToolStripMenuItem startEasyTestMenuItem;
    private ToolStripMenuItem startDifficultTestMenuItem;
    private Label questionWidthLabel; // Label (now at bottom, potentially for debug info)
    private Label scoreLabel; // Label to display the score (now at bottom)

    // Global variable for maximum questions (unused in this code)
    private const int max_questions = 50;

    // Global array for the questions
    private string[] questions_array = new string[5] // Size is 5
    {
        "Is Basic a computer programming language?" ,
        "Is Cobol a computer programming language?" ,
        "Is Pascal a computer programming language? ", // Added space
        "Does C# works well with .NET 8.0 in Microsoft Windows 10?" , // Added space
        "Does Visual Studio Code work well with C# in Microsoft Windows 10?" /// Question
5 // NEW question
    };

    // continúa más abajo. . .

```

```

// Variable to store the score
private int superpunto = 0;

public GridForm()
{
    Text = "Fixed 80x25";
    ClientSize = new Size(NumColumns * 10, NumRows * 20); // Initial size based on grid
cells
    StartPosition = FormStartPosition.CenterScreen;
    DoubleBuffered = true;
    BackColor = Color.White; // Initial setup, will be overridden by start_in_dark_mode
    ForeColor = Color.Black; // Initial setup, will be overridden by start_in_dark_mode

    // Create the MenuStrip
    mainMenuStrip = new MenuStrip();
    Controls.Add(mainMenuStrip);
    mainMenuStrip.Dock = DockStyle.Top;

    // Create menu items
    aboutMenuItem = new ToolStripMenuItem("About");
    aboutMenuItem.Click += AboutMenuItem_Click; // Attach event handler
    viewMenuItem = new ToolStripMenuItem("View");
    settingsMenuItem = new ToolStripMenuItem("Settings");
    startMenuItem = new ToolStripMenuItem("Start");

    // Create submenu items for View
    showRulerMenuItem = new ToolStripMenuItem("Show Ruler");
    showRulerMenuItem.Click += ShowRulerMenuItem_Click;
    hideRulerMenuItem = new ToolStripMenuItem("Hide Ruler");
    hideRulerMenuItem.Click += HideRulerMenuItem_Click;

    // Create submenu items for Settings
    darkModeMenuItem = new ToolStripMenuItem("Dark Mode");
    darkModeMenuItem.Click += DarkModeMenuItem_Click; // Attaching to the single valid
method
    classicModeMenuItem = new ToolStripMenuItem("Classic Mode");
    classicModeMenuItem.Click += ClassicModeMenuItem_Click;

    // Create submenu items for Start
    startEasyTestMenuItem = new ToolStripMenuItem("Start the easy test");
    startEasyTestMenuItem.Click += StartEasyTestMenuItem_Click;
    startDifficultTestMenuItem = new ToolStripMenuItem("Start the difficult test");
    startDifficultTestMenuItem.Click += StartDifficultTestMenuItem_Click;

    /// continúa en la página siguiente. . .

```

```

// Add submenu items to View
viewMenuItem.DropDownItems.Add(showRulerMenuItem);
viewMenuItem.DropDownItems.Add(hideRulerMenuItem);

// Add submenu items to Settings
settingsMenuItem.DropDownItems.Add(darkModeMenuItem);
settingsMenuItem.DropDownItems.Add(classicModeMenuItem);

// Add submenu items to Start
startMenuItem.DropDownItems.Add(startEasyTestMenuItem);
startMenuItem.DropDownItems.Add(startDifficultTestMenuItem);

// Add top-level menu items to the MenuStrip
mainMenuStrip.Items.Add(aboutMenuItem);
mainMenuStrip.Items.Add(startMenuItem);
mainMenuStrip.Items.Add(viewMenuItem);
mainMenuStrip.Items.Add(settingsMenuItem);

// Populate the rectangle array with the fixed dimensions
// This calculation is done again in Resize, could potentially be a separate method
int rectangleWidth = ClientSize.Width / NumColumns;
int rectangleHeight = ClientSize.Height / NumRows;

for (int row = 0; row < NumRows; row++)
{
    for (int col = 0; col < NumColumns; col++)
    {
        gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        textGrid[row, col] = ""; // Initialize textGrid with empty strings
    }
}

/// Continúa más abajo. . .

```

```

// Start in dark mode initially
start_in_dark_mode();

// Create label for question width display (moved to bottom)
questionWidthLabel = new Label();
questionWidthLabel.ForeColor = Color.Cyan;
questionWidthLabel.BackColor = Color.Black; // Match dark mode initial background
questionWidthLabel.AutoSize = true;
questionWidthLabel.Text = " "; // Initial text. Just testing

// Create label for score display (moved to bottom, above questionWidthLabel)
scoreLabel = new Label();
scoreLabel.ForeColor = Color.Cyan; // Match dark mode initial font color
scoreLabel.BackColor = Color.Black; // Match dark mode initial background
scoreLabel.AutoSize = true;
scoreLabel.Text = $"Score: {superpunto}"; // Initial score display

// Add labels to controls *before* calculating their initial position based on
ClientSize
Controls.Add(questionWidthLabel);
Controls.Add(scoreLabel);

// Set initial locations for labels based on initial ClientSize
// The Resize event will also set these positions correctly
// We need the heights of the labels, which are known after adding them to controls
int paddingBottom = 10; // Padding from the bottom edge
int paddingBetween = 5; // Padding between labels
int paddingLeft = 10; // Padding from the left edge

int scoreLabelY = ClientSize.Height - scoreLabel.Height - paddingBottom -
questionWidthLabel.Height - paddingBetween;
scoreLabel.Location = new Point(paddingLeft, scoreLabelY);

int questionLabelY = ClientSize.Height - questionWidthLabel.Height - paddingBottom;
questionWidthLabel.Location = new Point(paddingLeft, questionLabelY);

// Example of using superprint and superprintword (using current font color)
// Consider clearing the grid before writing new text for clarity
superprintword(1, 3, "Hello, developer.", globalFontColorSwitch);
superprintword(1, 5, "Welcome to C#", globalFontColorSwitch);

Paint += GridForm_Paint;
Resize += GridForm_Resize; // Make sure resize is handled
KeyDown += GridForm_KeyDown;
// Removed the duplicate Resize += GridForm_Resize;
}

```

/// Continúa en la página siguiente. . .

```
private void start_in_dark_mode()
{
    backgroundColorSwitch = 1; // Black background
    globalFontColorSwitch = 2; // Cyan fonts
    lineColorSwitch = 1;      // Black lines (visible on dark background)
    BackColor = Color.Black;
    ForeColor = Color.Cyan;
    UpdateLabelColors(); // Ensure label colors match the mode
}

private void classic_mode() // Created a separate method for classic mode logic
{
    backgroundColorSwitch = 0; // White background (classic light)
    globalFontColorSwitch = 1; // Black fonts (classic dark)
    lineColorSwitch = 0;      // White lines (to appear invisible on white background)
    BackColor = Color.White;
    ForeColor = Color.Black;
    UpdateLabelColors(); // Ensure label colors match the mode
}

// Continúa más abajo. . .
```

```

// This method runs the test questions in forward order
private void test3()
{
    // Reset score and display at the start of the test
    superpunto = 0;
    if (scoreLabel != null)
        scoreLabel.Text = $"Score: {superpunto}";

    // Optional: Clear the main grid before starting the test
    // ClearGrid();

    // Loop through questions from the first (index 0) to the last
    for (int i = 0; i < questions_array.Length; i++)
    {
        string question = questions_array[i];

        // Create a new Form for each question (consider reusing a form class for
better practice)
        Form inputForm = new Form();
        // Dialog title shows the original question number (1, 2, 3, 4, 5)
        inputForm.Text = "Question " + (i + 1); // Title shows original question number
        inputForm.StartPosition = FormStartPosition.CenterScreen;
        inputForm.BackColor = Color.Black; // Hardcoded color
        inputForm.ForeColor = Color.Cyan; // Hardcoded color
        Label questionLabel = new Label();
        questionLabel.Text = question; // Displays the question text
        questionLabel.ForeColor = Color.Cyan; // Hardcoded color
        questionLabel.AutoSize = false;
        questionLabel.MaximumSize = new Size(560, 0); // Wrap text at 560px
        questionLabel.Size = new Size(560, questionLabel.PreferredHeight); // Calculate
height
        questionLabel.Location = new Point(10, 10);

        TextBox answerTextBox = new TextBox();
        answerTextBox.BackColor = Color.Black; // Hardcoded color
        answerTextBox.ForeColor = Color.Cyan; // Hardcoded color
        answerTextBox.Location = new Point(10, questionLabel.Bottom + 10);
        answerTextBox.Width = 560;

        Button okButton = new Button();
        okButton.Text = "OK";
        okButton.DialogResult = DialogResult.OK;
        okButton.BackColor = Color.Black; // Hardcoded color
        okButton.ForeColor = Color.Cyan; // Hardcoded color
        // Initial location (will be adjusted)
        okButton.Location = new Point(inputForm.ClientSize.Width - 90,
answerTextBox.Bottom + 20);

        // Calculate form size based on contents (with minimum width)
        int desiredWidth = Math.Max(600, questionLabel.Width + 40); // leave padding
        int desiredHeight = okButton.Bottom + 60;
        inputForm.ClientSize = new Size(desiredWidth, desiredHeight);

    }
}

```

/// Continúa en la página siguiente. . .

```

        // Adjust OK button location now that form size is known
        okButton.Location = new Point(inputForm.ClientSize.Width - okButton.Width - 20,
answerTextBox.Bottom + 20);

        // Add controls
        inputForm.Controls.Add(questionLabel);
        inputForm.Controls.Add(answerTextBox);
        inputForm.Controls.Add(okButton);
        inputForm.AcceptButton = okButton;

        // Display width in main form - Could potentially show the dialog width here if
needed
        // int questionFormWidth = inputForm.Width;
        // if (questionWidthLabel != null)
        //     questionWidthLabel.Text = "Dialog Width: " + questionFormWidth;

        string answer = "";
        if (inputForm.ShowDialog() == DialogResult.OK)
        {
            answer = answerTextBox.Text;
        }
        inputForm.Dispose(); // Dispose the form after showing

        // Check the answer and update score
        // Note: Using (i+1) here for the MessageBox title shows the original question
number
        if (answer.Trim().ToLower() == "yes") // Added Trim() to handle
leading/trailing spaces
        {
            superpunto++; // Increment score for correct answer
            MessageBox.Show("Correct!", "Answer " + (i + 1));
        }
        else
        {
            MessageBox.Show("Incorrect!", "Answer " + (i + 1));
        }

        // Update the score display on the main form after each answer
        if (scoreLabel != null)
            scoreLabel.Text = $"Score: {superpunto}";

    } // End of loop

    // Show final score at the end of the test
    MessageBox.Show($"Test finished. Your final score is: {superpunto}", "Test
Results");

    // Clear the score label text after the test results are shown
    if (scoreLabel != null)
        scoreLabel.Text = string.Empty; // Using string.Empty is standard
}

```

/// Continúa en la página siguiente . . .


```

// This method (function) runs questions in reverse order
private void test5()
{
    // Reset score and display at the start of the test
    superpunto = 0;
    if (scoreLabel != null)
        scoreLabel.Text = $"Score: {superpunto}";

    // Optional: Clear the main grid before starting the test
    // ClearGrid();

    // Loop through questions from the last (index Length - 1) to the first (index 0)
    for (int i = questions_array.Length - 1; i >= 0; i--)
    {
        string question = questions_array[i];

        // Create a new Form for each question (consider reusing a form class for
better practice)
        Form inputForm = new Form();
        // Dialog title shows the original question number (5, 4, 3, 2, 1)
        // inputForm.Text = "Question " + (i + 1); /// shows question number
        inputForm.Text = "Question ";
        inputForm.StartPosition = FormStartPosition.CenterScreen;
        inputForm.BackColor = Color.Black; // Hardcoded color - consider using theme
colors
        inputForm.ForeColor = Color.Cyan; // Hardcoded color - consider using theme
colors

        Label questionLabel = new Label();
        questionLabel.Text = question; // Displays the question text
        questionLabel.ForeColor = Color.Cyan;
        questionLabel.AutoSize = false;
        questionLabel.MaximumSize = new Size(560, 0); // Wrap text at 560px
        questionLabel.Size = new Size(560, questionLabel.PreferredHeight); // Calculate
height based on wrapped text
        questionLabel.Location = new Point(10, 10);

        TextBox answerTextBox = new TextBox();
        answerTextBox.BackColor = Color.Black; // Hardcoded color
        answerTextBox.ForeColor = Color.Cyan; // Hardcoded color
        answerTextBox.Location = new Point(10, questionLabel.Bottom + 10);
        answerTextBox.Width = 560;

        Button okButton = new Button();
        okButton.Text = "OK";
        okButton.DialogResult = DialogResult.OK;
        okButton.BackColor = Color.Black;
        okButton.ForeColor = Color.Cyan;
        // Initial location (will be adjusted)
        okButton.Location = new Point(inputForm.ClientSize.Width - 90,
answerTextBox.Bottom + 20);

```

/// Continúa en la página siguiente . . .

```

// Calculate form size based on contents (with minimum width)
int desiredWidth = Math.Max(600, questionLabel.Width + 40); // leave padding
int desiredHeight = okButton.Bottom + 60;
inputForm.ClientSize = new Size(desiredWidth, desiredHeight);

// Adjust OK button location now that form size is known
okButton.Location = new Point(inputForm.ClientSize.Width - okButton.Width - 20,
answerTextBox.Bottom + 20);

// Add controls
inputForm.Controls.Add(questionLabel);
inputForm.Controls.Add(answerTextBox);
inputForm.Controls.Add(okButton);
inputForm.AcceptButton = okButton;

string answer = "";
if (inputForm.ShowDialog() == DialogResult.OK)
{
    answer = answerTextBox.Text;
}
inputForm.Dispose(); // Dispose the form after showing

// Check the answer and update score
if (answer.Trim().ToLower() == "yes") // Added Trim()
{
    superpunto++; // Increment score for correct answer
    /// MessageBox.Show("Correct!", "Answer " + (i + 1));
    MessageBox.Show("Well done! Correct!", "Message");
}
else
{
    /// MessageBox.Show("Incorrect!", "Answer " + (i + 1));
    MessageBox.Show("Incorrect!", "Message");
}

// Update the score display on the main form after each answer
if (scoreLabel != null)
    scoreLabel.Text = $"Score: {superpunto}";

} // End of loop

// Show final score at the end of the test
MessageBox.Show($"Test finished. Your final score is: {superpunto}", "Test
Results");

// Clear the score label text after the test results are shown
if (scoreLabel != null)
    scoreLabel.Text = string.Empty; // Using string.Empty is standard
}
/// end of private void test5()

```

/// Continúa en la página siguiente . . .

```
// Optional: Method to clear the text grid
private void ClearGrid()
{
    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            textGrid[row, col] = "";
        }
    }
    Invalidate(); // Request redraw
}
```

/// Continúa en la siguiente página. . .

```

private void about_form()
{
    Form aboutForm = new Form();
    aboutForm.Text = "About";
    aboutForm.StartPosition = FormStartPosition.CenterScreen;
    aboutForm.Size = new Size(300, 200); // Fixed size
    aboutForm.BackColor = Color.Black; // Hardcoded color
    aboutForm.ForeColor = Color.Cyan; // Hardcoded color

    Label messageLabel = new Label();
    messageLabel.Text = "Thanks for running this app.\njavier@brownbook.cloud";
    messageLabel.ForeColor = Color.Cyan; // Hardcoded color
    messageLabel.AutoSize = true;
    messageLabel.Location = new Point(10, 10);

    Button closeButton = new Button();
    closeButton.Text = "Close";
    closeButton.DialogResult = DialogResult.OK;
    closeButton.BackColor = Color.Black; // Hardcoded color
    closeButton.ForeColor = Color.Cyan; // Hardcoded color
    // Position button relative to form size
    closeButton.Location = new Point(aboutForm.ClientSize.Width - closeButton.Width -
10, aboutForm.ClientSize.Height - closeButton.Height - 10); // Corrected calculation error
here

    aboutForm.Controls.Add(messageLabel);
    aboutForm.Controls.Add(closeButton);
    aboutForm.AcceptButton = closeButton; // Allow Enter key to close

    aboutForm.ShowDialog();
    aboutForm.Dispose(); // Dispose the form
}

// Event handler for "About" menu item
private void AboutMenuItem_Click(object sender, EventArgs e)
{
    about_form();
}

```

/// Continúa en la página siguiente . . .

```

// Event handler for "Start the easy test"
private void StartEasyTestMenuItem_Click(object sender, EventArgs e)
{
    test3(); // Starts the test3 method (function) [questions in forward order]
}

// Event handler for "Start the difficult test"
private void StartDifficultTestMenuItem_Click(object sender, EventArgs e)
{
    test5(); // Starts the test5 function (reverse order with original numbering in
title)
}

private void ShowRulerMenuItem_Click(object sender, EventArgs e)
{
    // Prints the ruler on the grid
    superprintword(0, 0,
"0123456789012345678901234567890123456789012345678901234567890123456789",
globalFontColorSwitch);
    isRulerVisible = true;
    // Note: superprintword calls Invalidate for every character
}

private void HideRulerMenuItem_Click(object sender, EventArgs e)
{
    // Clears the ruler by printing spaces
    superprintword(0, 0, new string(' ', 79), globalFontColorSwitch); // Use current
font color for spaces
    isRulerVisible = false;
    // Note: superprintword calls Invalidate for every character, which is inefficient
}

```

/// Continúa en la página siguiente . . .

```

private void ClassicModeMenuItem_Click(object sender, EventArgs e)
{
    classic_mode(); // Call the new classic_mode method
    Invalidate(); // Request redraw to apply new colors
}

public void superprint(int column_num, int row_num, string stringx, int fontColorCode)
{
    if (column_num >= 0 && column_num < NumColumns && row_num >= 0 && row_num <
NumRows)
    {
        // Store the text in the grid data structure
        textGrid[row_num, column_num] = stringx;
        // Invalidate();
        // Call Invalidate() once after all print done
    }
    else
    {
        Console.WriteLine($"Warning: superprint coordinates ({column_num},{row_num})
are out of bounds.");
    }
}

public void superprintword(int column_num, int row_num, string wordx, int font_colour)
{
    for (int i = 0; i < wordx.Length; i++)
    {
        int currentColumn = column_num + i;
        if (currentColumn < NumColumns)
        {
            // Store character in grid. Pass the global font color.
            superprint(currentColumn, row_num, wordx[i].ToString(),
globalFontColorSwitch); // Using global switch
            // Invalidate is called *inside* superprint in the original code,
            // but that's inefficient. A better way is to call it once here.
            // For now, let's rely on the Paint event being triggered by other means
            // or call Invalidate() once after the loop if needed.
        }
        else
        {
            Console.WriteLine($"Warning: superprintword '{wordx}' at
({column_num},{row_num}) exceeds grid width.");
            break; // Stop printing if out of bounds
        }
    }
    // To make the text appear, debemos llamar a Invalidate() after printing
    // Call Invalidate() once here after the entire word is printed.
    Invalidate(); // Request redraw after the word is placed in the grid
}

```

/// Continúa en la página siguiente . . .

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    // Determine colors based on switches
    Color currentLineColor;
    switch (lineColorSwitch)
    {
        case 1: currentLineColor = Color.Black; break;
        case 2: currentLineColor = Color.Cyan; break;
        case 0: currentLineColor = Color.White; break;
        default: currentLineColor = Color.Black; break; // Default to Black
    }

    // Background color is already set on the form (BackColor property)
    // This switch logic is redundant here as BackColor is set in mode methods

    // Determine font color based on global switch
    Color currentFontColor;
    switch (globalFontColorSwitch)
    {
        case 1: currentFontColor = Color.Black; break;
        case 2: currentFontColor = Color.Cyan; break;
        case 0: currentFontColor = Color.White; break;
        default: currentFontColor = ForeColor; break;
    }

    using (Pen pen = new Pen(currentLineColor))
    using (StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
        LineAlignment = StringAlignment.Center })
    using (Brush textBrush = new SolidBrush(currentFontColor)) // Use the determined
font color
    {
        // Draw the grid and the text content from textGrid
        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                // Draw the cell rectangle
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);

                // Draw the text stored for this cell
                // Note: textGrid[row, col] should ideally only contain a single
character
                if (!string.IsNullOrEmpty(textGrid[row, col]))
                {
                    // Draw the first character if the string is longer than 1
                    string charToDraw = textGrid[row, col].Length > 0 ? textGrid[row,
col].Substring(0, 1) : "";
                    e.Graphics.DrawString(charToDraw, displayFont, textBrush,
gridRectangles[row, col], sf);
                }
            }
        }
    }
}

```

/// Continúa en la página siguiente . . .

```

private void GridForm_Resize(object sender, EventArgs e)
{
    // Recalculate cell dimensions based on new form size
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    // Update gridRectangles
    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    // Update the location of labels based on new ClientSize - Anclar a bottom-left
    int paddingBottom = 10; // Padding from the bottom edge
    int paddingBetween = 5; // Padding between labels
    int paddingLeft = 10;   // Padding from the left edge

    // Position the score label just above the questionWidthLabel
    if (scoreLabel != null && questionWidthLabel != null)
    {
        int questionLabelY = ClientSize.Height - questionWidthLabel.Height -
paddingBottom;
        questionWidthLabel.Location = new Point(paddingLeft, questionLabelY);

        int scoreLabelY = questionLabelY - scoreLabel.Height - paddingBetween;
        scoreLabel.Location = new Point(paddingLeft, scoreLabelY);
    }
    // If only scoreLabel exists
    else if (scoreLabel != null)
    {
        int scoreLabelY = ClientSize.Height - scoreLabel.Height - paddingBottom;
        scoreLabel.Location = new Point(paddingLeft, scoreLabelY);
    }
    // If only questionWidthLabel exists (original case)
    else if (questionWidthLabel != null)
    {
        int questionLabelY = ClientSize.Height - questionWidthLabel.Height -
paddingBottom;
        questionWidthLabel.Location = new Point(paddingLeft, questionLabelY);
    }

    Invalidate(); // Request redraw with updated grid rectangles
}

```

/// Continúa en la página siguiente . . .


```

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    // Toggle menu visibility with Ctrl+M
    if (e.Control && (e.KeyCode == Keys.M))
    {
        mainMenuStrip.Visible = !mainMenuStrip.Visible;
        e.Handled = true; // Mark event as handled
    }
    // Trigger superprintword (ruler) with Ctrl+R
    else if (e.Control && (e.KeyCode == Keys.R))
    {
        // Toggle ruler visibility
        if (!isRulerVisible)
        {
            ShowRulerMenuItem_Click(sender, e);
        }
        else
        {
            HideRulerMenuItem_Click(sender, e);
        }
        e.Handled = true; // Mark event as handled
    }
    else
    {
        bool colorChanged = false;
        // Switch for line color
        if (e.KeyCode == Keys.D1 && !e.Shift && !e.Control) { lineColorSwitch = 1;
colorChanged = true; }
        else if (e.KeyCode == Keys.D2 && !e.Shift && !e.Control) { lineColorSwitch = 2;
colorChanged = true; }
        else if (e.KeyCode == Keys.D0 && !e.Shift && !e.Control) { lineColorSwitch = 0;
colorChanged = true; }

        // Switch for background color (requires Shift)
        if (e.KeyCode == Keys.D1 && e.Shift) { backgroundColorSwitch = 1; BackColor =
Color.Black; colorChanged = true; }
        else if (e.KeyCode == Keys.D0 && e.Shift) { backgroundColorSwitch = 0;
BackColor = Color.White; colorChanged = true; }

        // Switch for global font color (requires Control)
        if (e.KeyCode == Keys.D1 && e.Control) { globalFontColorSwitch = 1; ForeColor =
Color.Black; colorChanged = true; }
        else if (e.KeyCode == Keys.D2 && e.Control) { globalFontColorSwitch = 2;
ForeColor = Color.Cyan; colorChanged = true; }
        else if (e.KeyCode == Keys.D0 && e.Control) { globalFontColorSwitch = 0;
ForeColor = Color.White; colorChanged = true; }

        if (colorChanged)
        {
            UpdateLabelColors(); // Update label colors if any color switch happened
            e.Handled = true; // Mark event as handled if color changed
        }
    }
}

```

/// Continúa en la página siguiente . . .

```

        // Only Invalidate if a relevant key was pressed and handled
        if (e.Handled)
        {
            Invalidate(); // Invalidate to ensure any visual changes are redrawn
        }
    }

    // Helper method to update label colors when mode changes via keys or menus
    private void UpdateLabelColors()
    {
        Color currentForeColor;
        switch (globalFontColorSwitch)
        {
            case 1: currentForeColor = Color.Black; break;
            case 2: currentForeColor = Color.Cyan; break;
            case 0: currentForeColor = Color.White; break;
            default: currentForeColor = ForeColor; break;
        }

        Color currentBackColor = BackColor; // Use the form's current BackColor

        if (questionWidthLabel != null)
        {
            questionWidthLabel.ForeColor = currentForeColor;
            questionWidthLabel.BackColor = currentBackColor;
        }
        if (scoreLabel != null)
        {
            scoreLabel.ForeColor = currentForeColor;
            scoreLabel.BackColor = currentBackColor;
        }
    }

    private void DarkModeMenuItem_Click(object sender, EventArgs e)
    {
        start_in_dark_mode();
        Invalidate(); // Request redraw to apply new colors
    }

    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new GridForm());
    }
}

/// Fin del ejemploC35

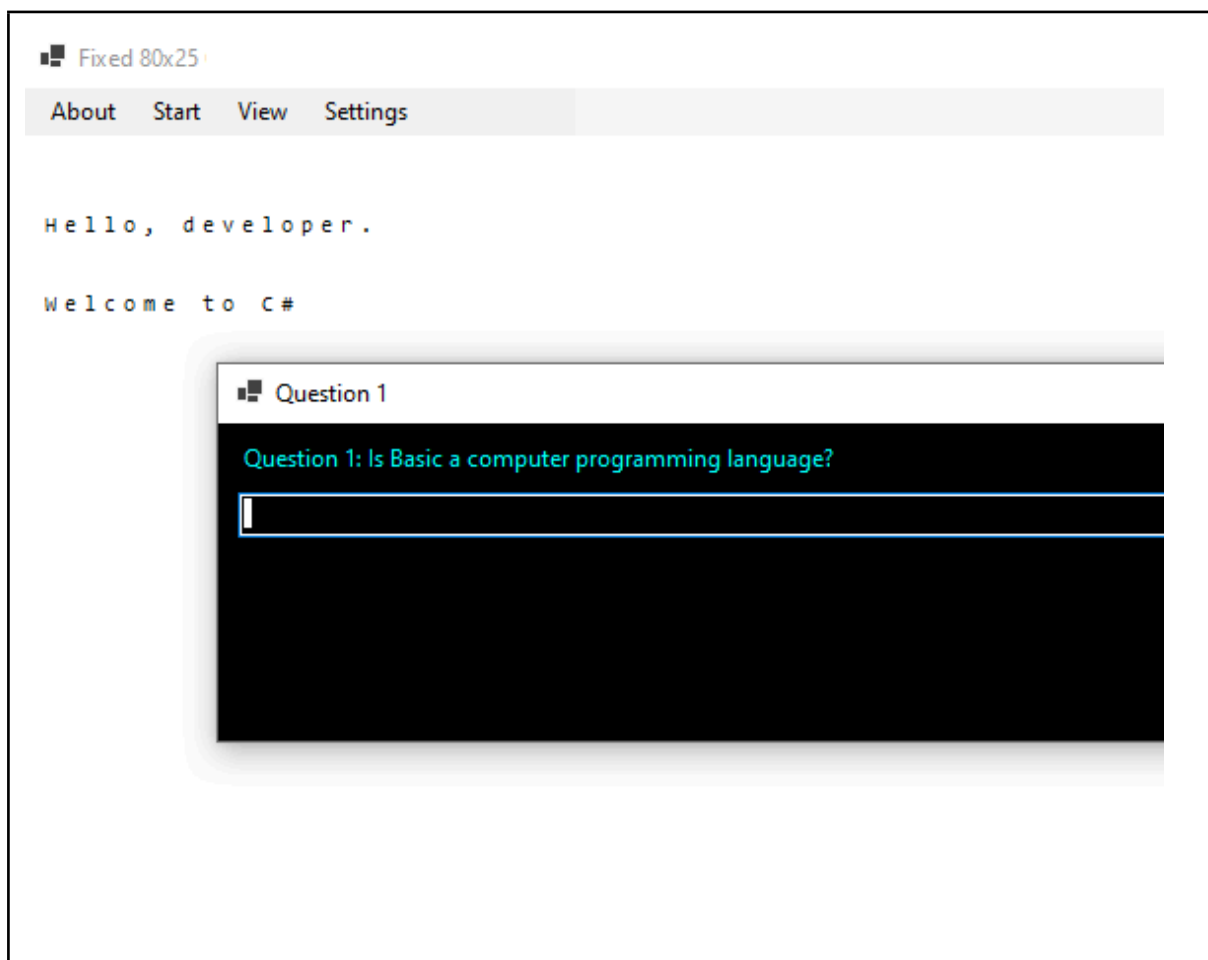
```

A continuación ejecute el EjemploC35 creado y verá la siguiente ventana:

Recuerde que durante la ejecución del programa, puede cambiar los colores de ciertos elementos de la ventana activando el menú principal.

Para empezar el test, pulse “Start” y elija un test (podrá elegir entre test fácil -easy- o test difícil -difficult-).

La función test3() ejecuta el test fácil. La función test5() ejecuta un test más complicado.



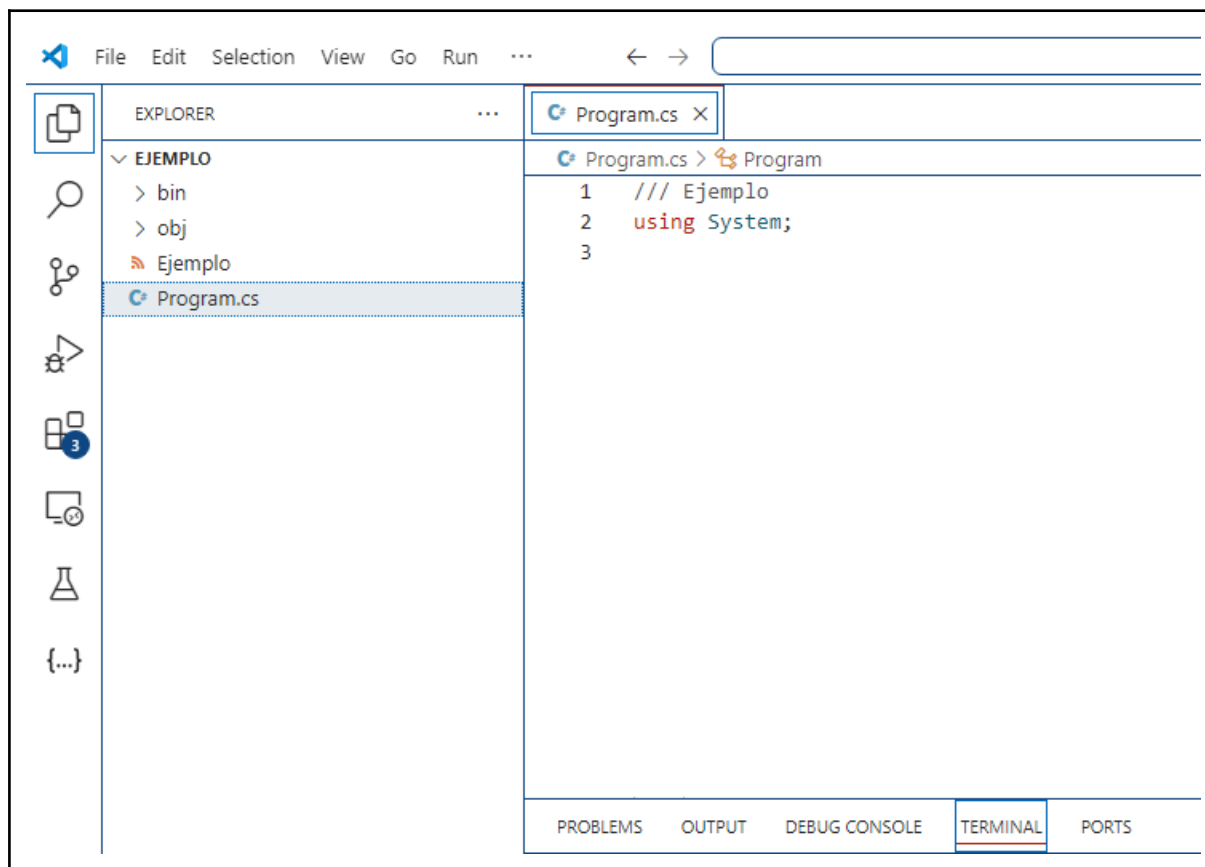
Ejemplo C36. Textos y Números aleatorios en el terminal.

Objetivo: Aplicación que imprime números aleatorios en el terminal.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC36”:

```
cd documents
dotnet new console -o EjemploC36
cd EjemploC36
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

```
/// Ejemplo C36. Generación de números aleatorios en el terminal.

using System;

class Program
{
    static string? SuperInput(string prompt)
    {
        Console.Write(prompt + " ");
        string? input = Console.ReadLine()?.Trim();
        return string.IsNullOrEmpty(input) ? null : input;
    }

    static void Main()
    {
        var name = SuperInput("Entre su nombre:");
        var displayName = name ?? "Usuario Anónimo";

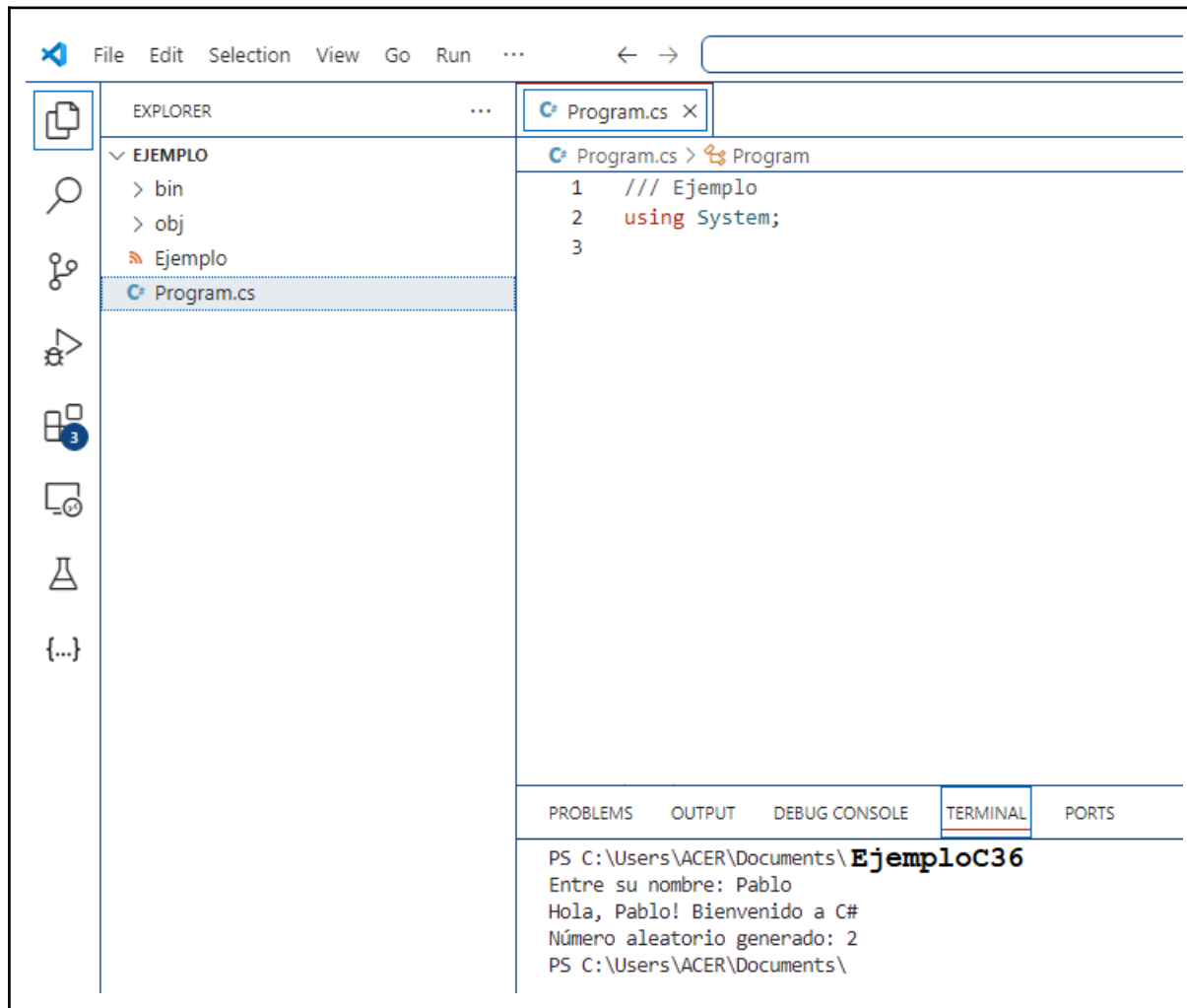
        Console.WriteLine(name != null
            ? $"Hola, {displayName}! Bienvenido a C#"
            : "No se ingresó ningún nombre válido. Hasta la vista.");

        Random random = new Random();
        int randomNumber = random.Next(1, 7);
        // el 1 está incluido, el 7 está excluido
        Console.WriteLine("Número aleatorio generado: " + randomNumber);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos la consola en VS Code (CTRL+ñ) y teclearemos:

dotnet run



Si el usuario no entra ningún nombre, no aparece ningún error. Hemos validado la entrada. Si no se proporciona ningún nombre, nuestro código informará al usuario de ello. Observe que se ha generado un número aleatorio, el número 2.

Reto: Modifique el código para que se muestren dos números aleatorios distintos en la consola.

Ejemplo C37. Creación de una clave (password) basada en un test de 3 preguntas.

Objetivo: Escribir el código para realizar un test de 3 preguntas con un formulario. Las preguntas estarán almacenadas en una matriz. Las respuestas se introducirán por teclado. Se generará una clave (password) en base a las respuestas entradas por el usuario que podrá utilizarse en cualquier servicio web.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC37
cd EjemploC37
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs ///
EjemploC37. Test de tres preguntas para generar
passwords

```

```

using System;
using System.Drawing;
using System.Windows.Forms;

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;
    private int lineColorSwitch = 1;
    private int backgroundColorSwitch = 0;
    private int globalFontColorSwitch = 0;
    private Color gridColor = Color.Black;
    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string[,] textGrid = new string[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 8);
    private MenuStrip mainMenuStrip;
    private ToolStripMenuItem aboutMenuItem;
    private ToolStripMenuItem viewMenuItem;
    private ToolStripMenuItem settingsMenuItem;
    private ToolStripMenuItem darkModeMenuItem;
    private ToolStripMenuItem classicModeMenuItem;
    private ToolStripMenuItem startMenuItem;
    private ToolStripMenuItem startEasyTestMenuItem;
    private ToolStripMenuItem startDifficultTestMenuItem;
    private Label questionWidthLabel;

    private const int max_questions = 50;
    private string[] questions_array = new string[3]
    {
        "Question 1: Is Basic a computer programming language?",
        "Question 2: Is Cobol a computer programming language?",
        "Question 3: Is Pascal a computer programming language?"
    };
};

/// Continúa en la siguiente página. . .

```



```

public GridForm()
{
    Text = "Password Generator Plus";
    ClientSize = new Size(NumColumns * 10, NumRows * 20);
    StartPosition = FormStartPosition.CenterScreen;
    DoubleBuffered = true;
    BackColor = Color.White; ForeColor = Color.Black;

    mainMenuStrip = new MenuStrip();
    Controls.Add(mainMenuStrip);
    mainMenuStrip.Dock = DockStyle.Top;

    aboutMenuItem = new ToolStripMenuItem("About");
    aboutMenuItem.Click += AboutMenuItem_Click;
    viewMenuItem = new ToolStripMenuItem("View");
    settingsMenuItem = new ToolStripMenuItem("Settings");
    startMenuItem = new ToolStripMenuItem("Start");

    darkModeMenuItem = new ToolStripMenuItem("Dark Mode");
    darkModeMenuItem.Click += DarkModeMenuItem_Click;
    classicModeMenuItem = new ToolStripMenuItem("Classic Mode");
    classicModeMenuItem.Click += ClassicModeMenuItem_Click;

    startEasyTestMenuItem = new ToolStripMenuItem("Generate password and start the
test");
    startEasyTestMenuItem.Click += StartEasyTestMenuItem_Click;
    startDifficultTestMenuItem = new ToolStripMenuItem("Generate a strong password");
    startDifficultTestMenuItem.Click += StartDifficultTestMenuItem_Click;

    settingsMenuItem.DropDownItems.Add(darkModeMenuItem);
    settingsMenuItem.DropDownItems.Add(classicModeMenuItem);

    startMenuItem.DropDownItems.Add(startEasyTestMenuItem);
    startMenuItem.DropDownItems.Add(startDifficultTestMenuItem);

    mainMenuStrip.Items.Add(aboutMenuItem);
    mainMenuStrip.Items.Add(startMenuItem);
    mainMenuStrip.Items.Add(viewMenuItem); mainMenuStrip.Items.Add(settingsMenuItem);

    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
            textGrid[row, col] = "";
        }
    }

    }

    /// Continúa en la siguiente página. . .

```

```
start_in_dark_mode();

questionWidthLabel = new Label();
questionWidthLabel.ForeColor = Color.Cyan;
questionWidthLabel.BackColor = Color.Black;
questionWidthLabel.AutoSize = true;
questionWidthLabel.Location = new Point(10, ClientSize.Height -
questionWidthLabel.Height - 10);
Controls.Add(questionWidthLabel);

superprintword(1, 3, "Hello, developer.", globalFontColorSwitch);
superprintword(1, 5, "Welcome to C#", globalFontColorSwitch);

Paint += GridForm_Paint;
Resize += GridForm_Resize;
KeyDown += GridForm_KeyDown;
}

/// Continúa en la siguiente página. . .
```

```
private void start_in_dark_mode()
{
    backgroundColorSwitch = 1; globalFontColorSwitch = 2;
    lineColorSwitch = 1;
    BackColor = Color.Black;    ForeColor = Color.Cyan;
}

/// Continúa en la siguiente página. . .
```

```

private void test3()
{
    for (int i = 0; i < questions_array.Length; i++)
    {
        string question = questions_array[i];

        Form inputForm = new Form();
        inputForm.Text = "Question " + (i + 1);
        inputForm.StartPosition = FormStartPosition.CenterScreen;
        inputForm.BackColor = Color.Black; inputForm.ForeColor = Color.Cyan;

        Label questionLabel = new Label();
        questionLabel.Text = question; questionLabel.ForeColor = Color.Cyan;
        questionLabel.AutoSize = false;
        questionLabel.MaximumSize = new Size(560, 0);
        questionLabel.Size = new Size(560, questionLabel.PreferredHeight);
        questionLabel.Location = new Point(10, 10);

        TextBox answerTextBox = new TextBox();
        answerTextBox.BackColor = Color.Black; answerTextBox.ForeColor = Color.Cyan;
        answerTextBox.Location = new Point(10, questionLabel.Bottom + 10);
        answerTextBox.Width = 560;

        Button okButton = new Button();
        okButton.Text = "OK"; okButton.DialogResult = DialogResult.OK;
        okButton.BackColor = Color.Black; okButton.ForeColor = Color.Cyan;

        int desiredWidth = Math.Max(600, questionLabel.Width + 40);
        int desiredHeight = answerTextBox.Bottom + 80;
        inputForm.ClientSize = new Size(desiredWidth, desiredHeight);

        okButton.Location = new Point(inputForm.ClientSize.Width - okButton.Width - 20,
        answerTextBox.Bottom + 20);

        inputForm.Controls.Add(questionLabel);
        inputForm.Controls.Add(answerTextBox);
        inputForm.Controls.Add(okButton); inputForm.AcceptButton = okButton;

        questionWidthLabel.Text = "Get a pen now! ";

        string answer = "";
        if (inputForm.ShowDialog() == DialogResult.OK)
        {
            answer = answerTextBox.Text;
        }
        inputForm.Dispose();

        MessageBox.Show(answer.ToLower() == "yes" ? "Correct!" : "Incorrect!", "Answer
" + (i + 1));
        if (i == 2)
        {
            Random random = new Random(); int randomNumber = random.Next(1, 4);
            questionWidthLabel.Text = "Write down your password: " + i + "$$$123Pen$$$"
+ randomNumber;
        }
    }
}

```

```

private void about_form()
{
    Form aboutForm = new Form();
    aboutForm.Text = "About";
    aboutForm.StartPosition = FormStartPosition.CenterScreen;
    aboutForm.Size = new Size(300, 200);
    aboutForm.BackColor = Color.Black;
    aboutForm.ForeColor = Color.Cyan;

    Label messageLabel = new Label();
    messageLabel.Text = "Thanks for running this app.\n\njavier@brownbook.cloud";
    messageLabel.ForeColor = Color.Cyan;
    messageLabel.AutoSize = true;
    messageLabel.Location = new Point(10, 10);

    Button closeButton = new Button();
    closeButton.Text = "Close";
    closeButton.DialogResult = DialogResult.OK;
    closeButton.BackColor = Color.Black;
    closeButton.ForeColor = Color.Cyan;
    closeButton.Location = new Point(aboutForm.ClientSize.Width - closeButton.Width -
10, aboutForm.ClientSize.Height - closeButton.Height - 10);

    aboutForm.Controls.Add(messageLabel); aboutForm.Controls.Add(closeButton);
    aboutForm.AcceptButton = closeButton;

    aboutForm.ShowDialog(); aboutForm.Dispose();
}

private void AboutMenuItem_Click(object sender, EventArgs e) => about_form();
private void StartEasyTestMenuItem_Click(object sender, EventArgs e) => test3();
private void StartDifficultTestMenuItem_Click(object sender, EventArgs e) => test3();

```

/// Continúa en la siguiente página. . .

```

private void DarkModeMenuItem_Click(object sender, EventArgs e)
{
    start_in_dark_mode();
    Invalidate();
}

private void ClassicModeMenuItem_Click(object sender, EventArgs e)
{
    backgroundColorSwitch = 0; globalFontColorSwitch = 1;
    lineColorSwitch = 0;
    BackColor = Color.White; ForeColor = Color.Black;
    Invalidate();
}

public void superprint(int column_num, int row_num, string stringx, int fontColorCode)
{
    if (column_num >= 0 && column_num < NumColumns && row_num >= 0 && row_num <
NumRows)
    {
        textGrid[row_num, column_num] = stringx;
        Invalidate();
    }
}

public void superprintword(int column_num, int row_num, string wordx, int font_colour)
{
    for (int i = 0; i < wordx.Length; i++)
    {
        int currentColumn = column_num + i;
        if (currentColumn < NumColumns)
        {
            superprint(currentColumn, row_num, wordx[i].ToString(), font_colour);
        }
    }
}

/// Continúa en la siguiente página. . .

```

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    Color currentLineColor = lineColorSwitch switch
    {
        1 => Color.Black,
        2 => Color.Cyan,
        0 => Color.White,
        _ => Color.Black
    };

    Color currentBackColor = backgroundColorSwitch == 1 ? Color.Black : Color.White;
    BackColor = currentBackColor;

    Color currentFontColor = globalFontColorSwitch switch
    {
        1 => Color.Black,
        2 => Color.Cyan,
        0 => Color.White,
        _ => ForeColor
    };

    using (Pen pen = new Pen(currentLineColor))
    using (StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center })
    using (Brush textBrush = new SolidBrush(currentFontColor))
    {
        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);
                e.Graphics.DrawString(textGrid[row, col], displayFont, textBrush,
gridRectangles[row, col], sf);
            }
        }
    }
}

```

/// Continúa en la siguiente página. . .

```

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    questionWidthLabel.Location = new Point(10, ClientSize.Height -
questionWidthLabel.Height - 10);
    Invalidate();
}

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    if (e.Control && (e.KeyCode == Keys.M))
    {
        mainMenuStrip.Visible = !mainMenuStrip.Visible;
    }
    else
    {
        if (e.KeyCode == Keys.D1) lineColorSwitch = 1;
        else if (e.KeyCode == Keys.D2) lineColorSwitch = 2;
        else if (e.KeyCode == Keys.D0 && !e.Shift && !e.Control) lineColorSwitch = 0;

        if (e.KeyCode == Keys.D1 && e.Shift) backgroundColorSwitch = 1;
        else if (e.KeyCode == Keys.D0 && e.Shift) backgroundColorSwitch = 0;

    }

    Invalidate();
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}

}

/// Fin del Ejemplo C37.

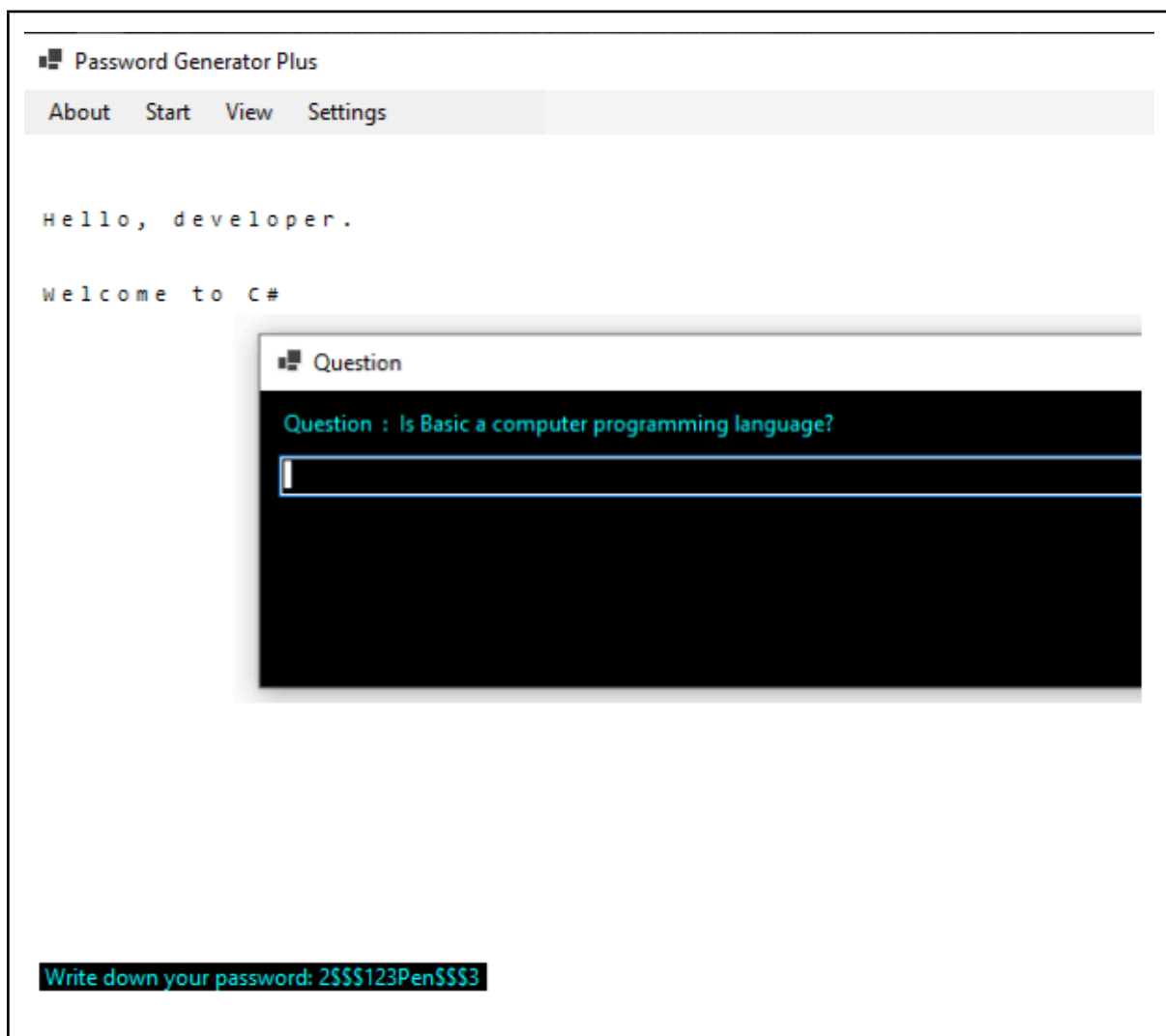
```


A continuación ejecute el EjemploC37 creado y verá la siguiente ventana:

Recuerde que durante la ejecución del programa, puede cambiar los colores de ciertos elementos de la ventana activando el menú principal.

Para empezar el test, pulse “Start” y elija un test (podrá elegir entre test fácil -easy- o test difícil -difficult-).

La función test3() ejecuta el test fácil y genera una clave (password) después de responder tres preguntas sobre informática.



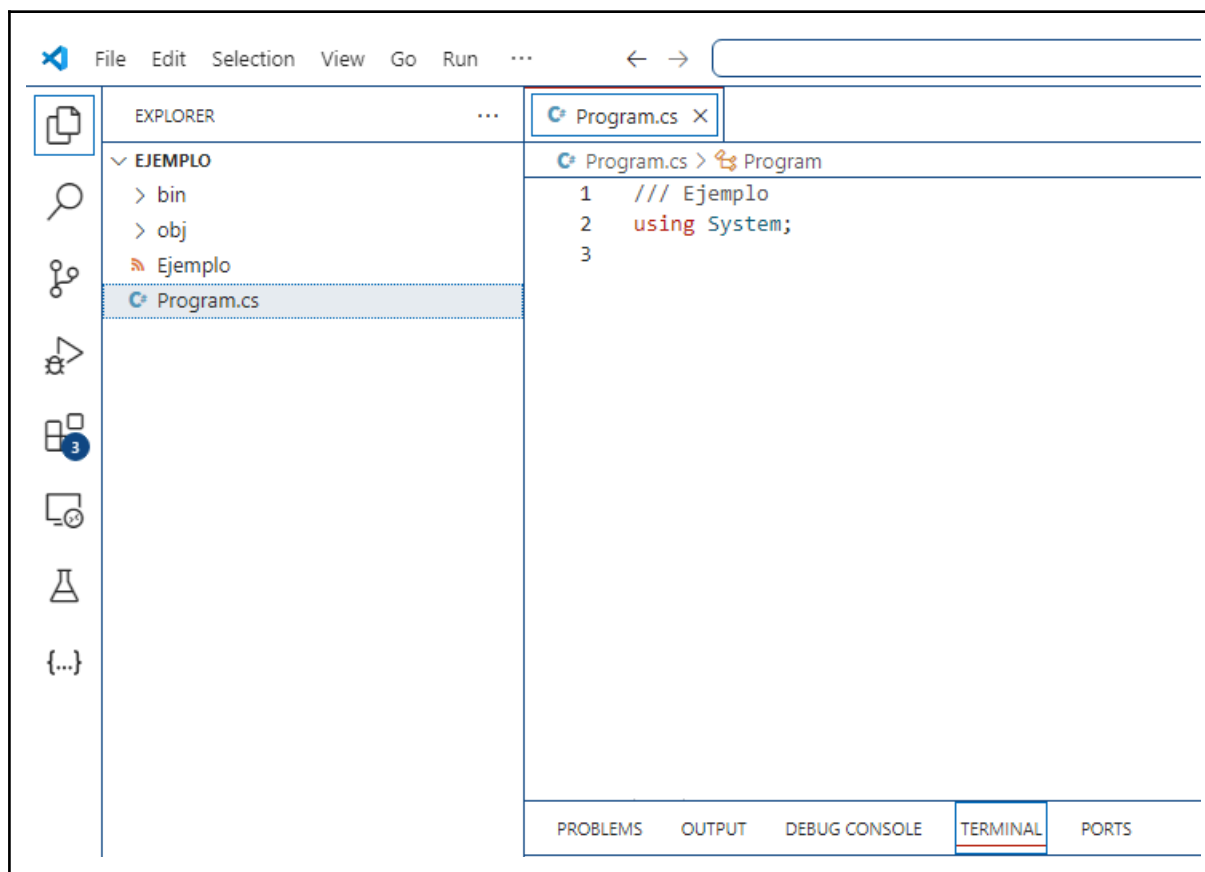
Ejemplo C38. Creación de claves semi aleatorias.

Objetivo: Aplicación que imprime claves semi aleatorias en el terminal combinando palabras que se han entrado por teclado.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC38”:

```
cd documents
dotnet new console -o EjemploC38
cd EjemploC38
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

```
using System;                                     /// EjemploC38. Generador de contraseñas.

class Program
{
    static void Main()
    {
        Console.Write("Ingresa la primera palabra: ");
        string palabra1 = Console.ReadLine();

        Console.Write("Ingresa la segunda palabra: ");
        string palabra2 = Console.ReadLine();

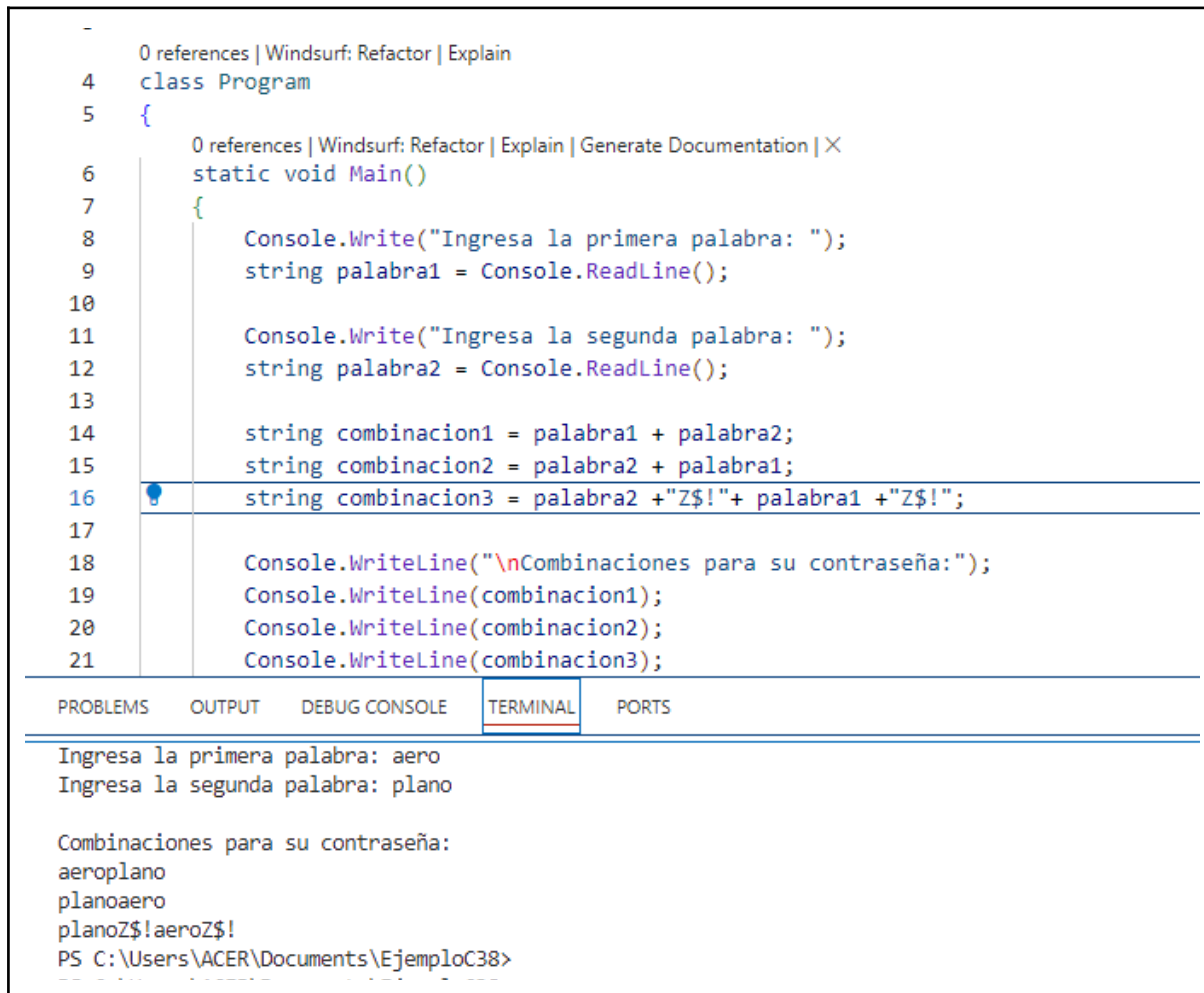
        string combinacion1 = palabra1 + palabra2;
        string combinacion2 = palabra2 + palabra1;
        string combinacion3 = palabra2 + "Z$!" + palabra1 + "Z$!";

        Console.WriteLine("\nCombinaciones para su contraseña:");
        Console.WriteLine(combinacion1);
        Console.WriteLine(combinacion2);
        Console.WriteLine(combinacion3);
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos la consola en VS Code (CTRL+ñ) y teclearemos:

dotnet run



```

-
0 references | Windsurf: Refactor | Explain
4 class Program
5 {
    0 references | Windsurf: Refactor | Explain | Generate Documentation | X
6     static void Main()
7     {
8         Console.Write("Ingresa la primera palabra: ");
9         string palabra1 = Console.ReadLine();
10
11        Console.Write("Ingresa la segunda palabra: ");
12        string palabra2 = Console.ReadLine();
13
14        string combinacion1 = palabra1 + palabra2;
15        string combinacion2 = palabra2 + palabra1;
16        string combinacion3 = palabra2 + "Z$!" + palabra1 + "Z$!";
17
18        Console.WriteLine("\nCombinaciones para su contraseña:");
19        Console.WriteLine(combinacion1);
20        Console.WriteLine(combinacion2);
21        Console.WriteLine(combinacion3);
    }
}

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Ingresa la primera palabra: aero
Ingresa la segunda palabra: plano

Combinaciones para su contraseña:
aeroplano
planoaero
planoZ$!aeroZ$!
PS C:\Users\ACER\Documents\EjemploC38>

```

Observe que no hemos validado las entradas. Si no se proporciona ninguna palabra, nuestro código no informará al usuario de ello. Observe que se han generado tres contraseñas débiles con un patrón predecible.

Reto: Modifique el código para que se incorporen dos números aleatorios diferentes al inicio y al final de cada contraseña.

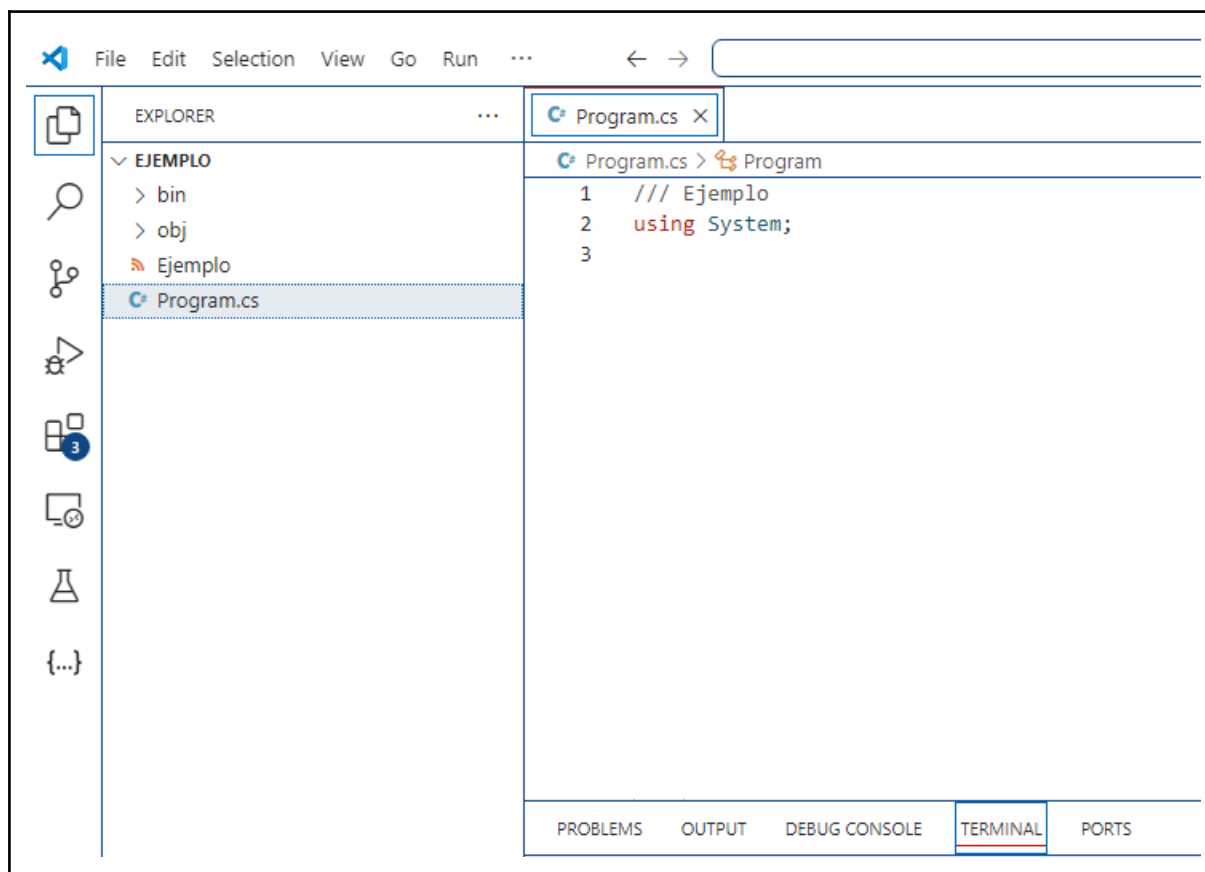
Ejemplo C39. Juego de los dados.

Objetivo: Generar dos números aleatorios entre el 1 y el 6 utilizando el método **Random.Next** que pertenece a la clase **Random** en C#.

Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC39”:

```
cd documents
dotnet new console -o EjemploC39
cd EjemploC39
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

```
using System;

class Program
{
    static void Main()
    {
        Random random = new Random();

        int numero1 = random.Next(1, 7); // 1 a 6 inclusive
        int numero2 = random.Next(1, 7);

        Console.WriteLine("Juego de los dados.");
        Console.WriteLine("Vamos a generar dos números aleatorios entre 1 y 6.");
        Console.WriteLine("Primer número aleatorio: " + numero1);
        Console.WriteLine("Segundo número aleatorio: " + numero2);
    }
}
```

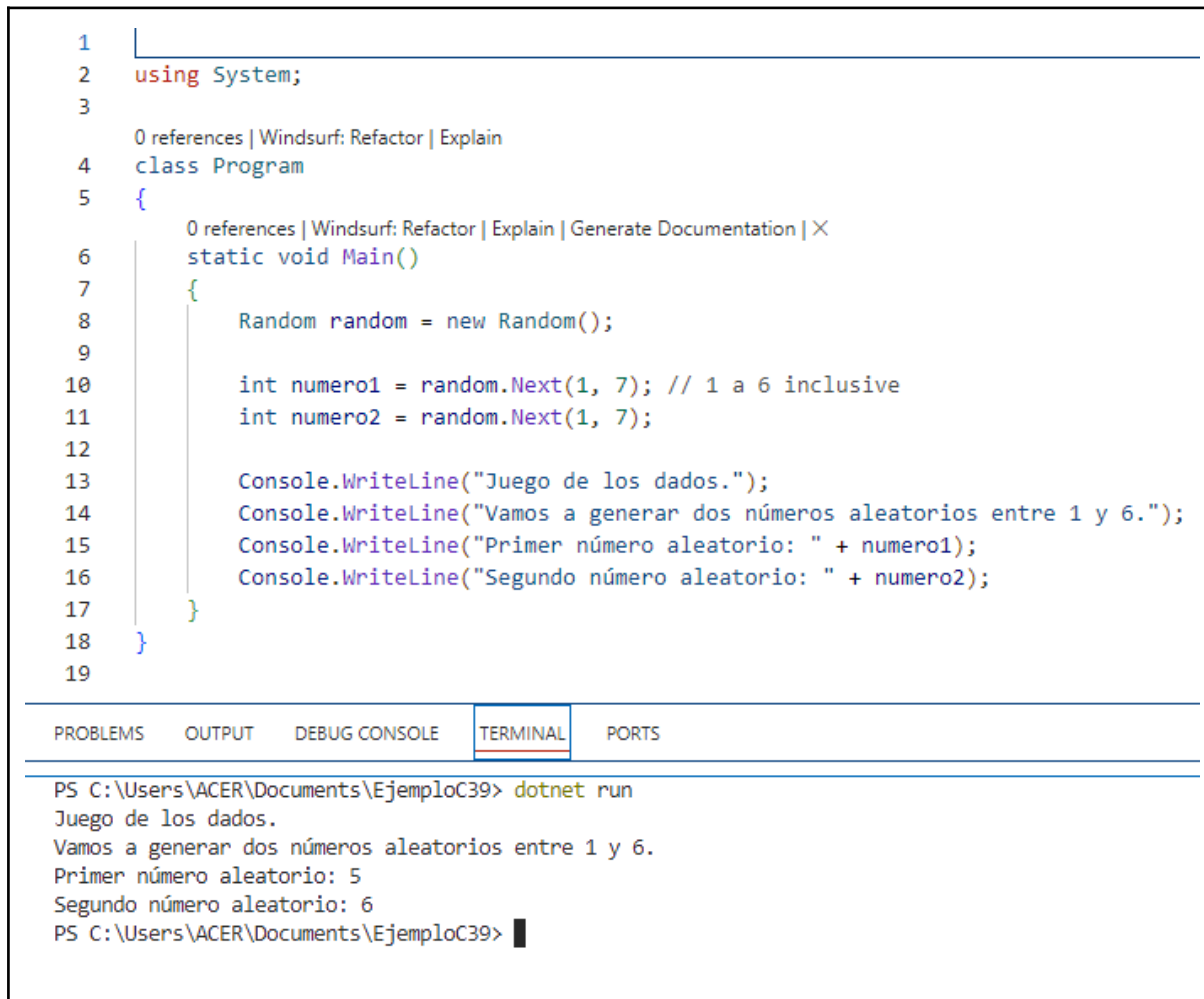
Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

Para obtener 2 números aleatorios enteros entre 1 y 6 (ambos inclusive, el 7 está excluido), es necesario que construyamos la siguiente instrucción:

```
int numero1 = random.Next(1,7)
```

A continuación abriremos la consola en VS Code (CTRL+ñ) y teclearemos:

dotnet run



The screenshot displays the Visual Studio Code interface. The top pane shows a C# file named Program.cs with the following code:

```
1
2 using System;
3
4 class Program
5 {
6     static void Main()
7     {
8         Random random = new Random();
9
10        int numero1 = random.Next(1, 7); // 1 a 6 inclusive
11        int numero2 = random.Next(1, 7);
12
13        Console.WriteLine("Juego de los dados.");
14        Console.WriteLine("Vamos a generar dos números aleatorios entre 1 y 6.");
15        Console.WriteLine("Primer número aleatorio: " + numero1);
16        Console.WriteLine("Segundo número aleatorio: " + numero2);
17    }
18 }
19
```

The bottom pane shows the output of the program:

```
PS C:\Users\ACER\Documents\EjemploC39> dotnet run
Juego de los dados.
Vamos a generar dos números aleatorios entre 1 y 6.
Primer número aleatorio: 5
Segundo número aleatorio: 6
PS C:\Users\ACER\Documents\EjemploC39> █
```

Observe que se han generado dos números aleatorios: el 5 y el 6. Si vuelve a ejecutar el programa, la combinación generada será diferente.

Reto: Modifique el código para que se incorporen dos números aleatorios entre 1 y 49.

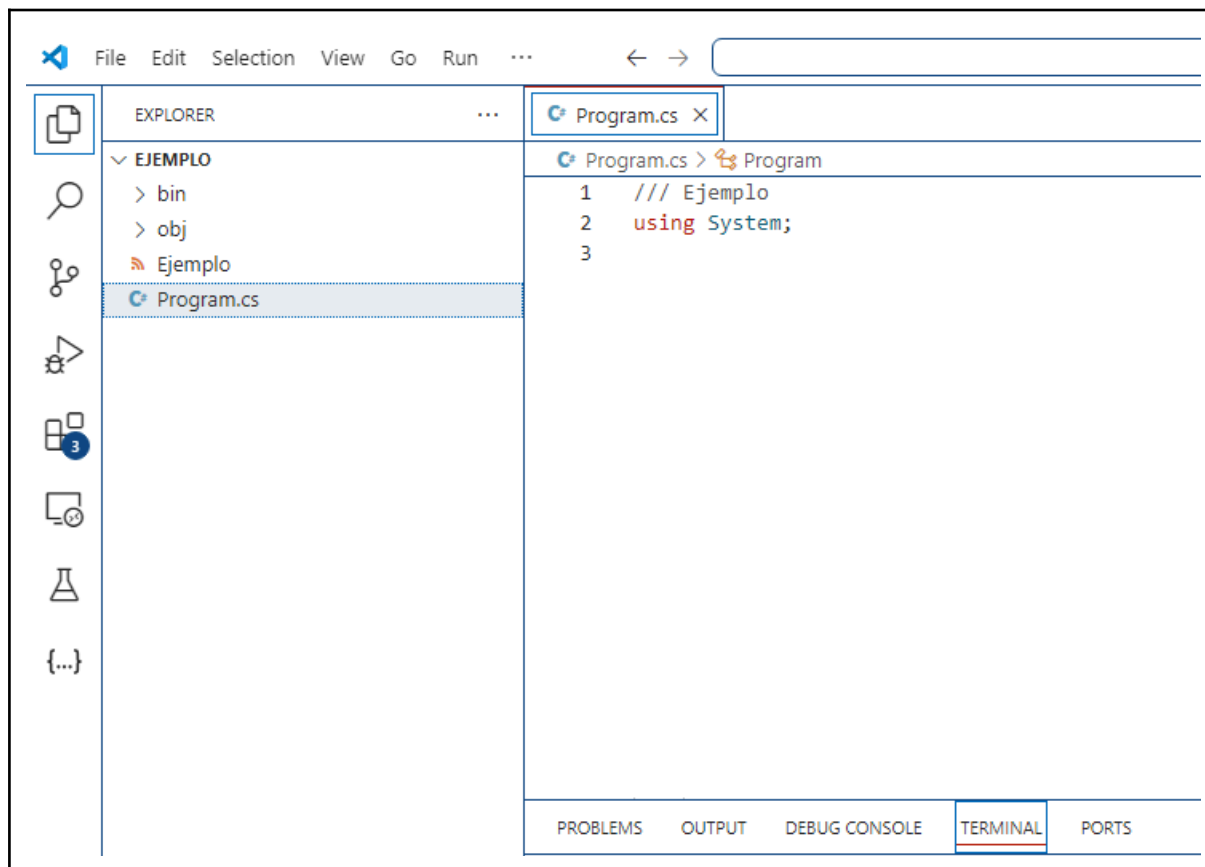
Ejemplo C40. Gráfico de un cubo en perspectiva isométrica.

Objetivo: Generar un gráfico de un cubo en perspectiva isométrica con caracteres ASCII (codificación UTF8) .

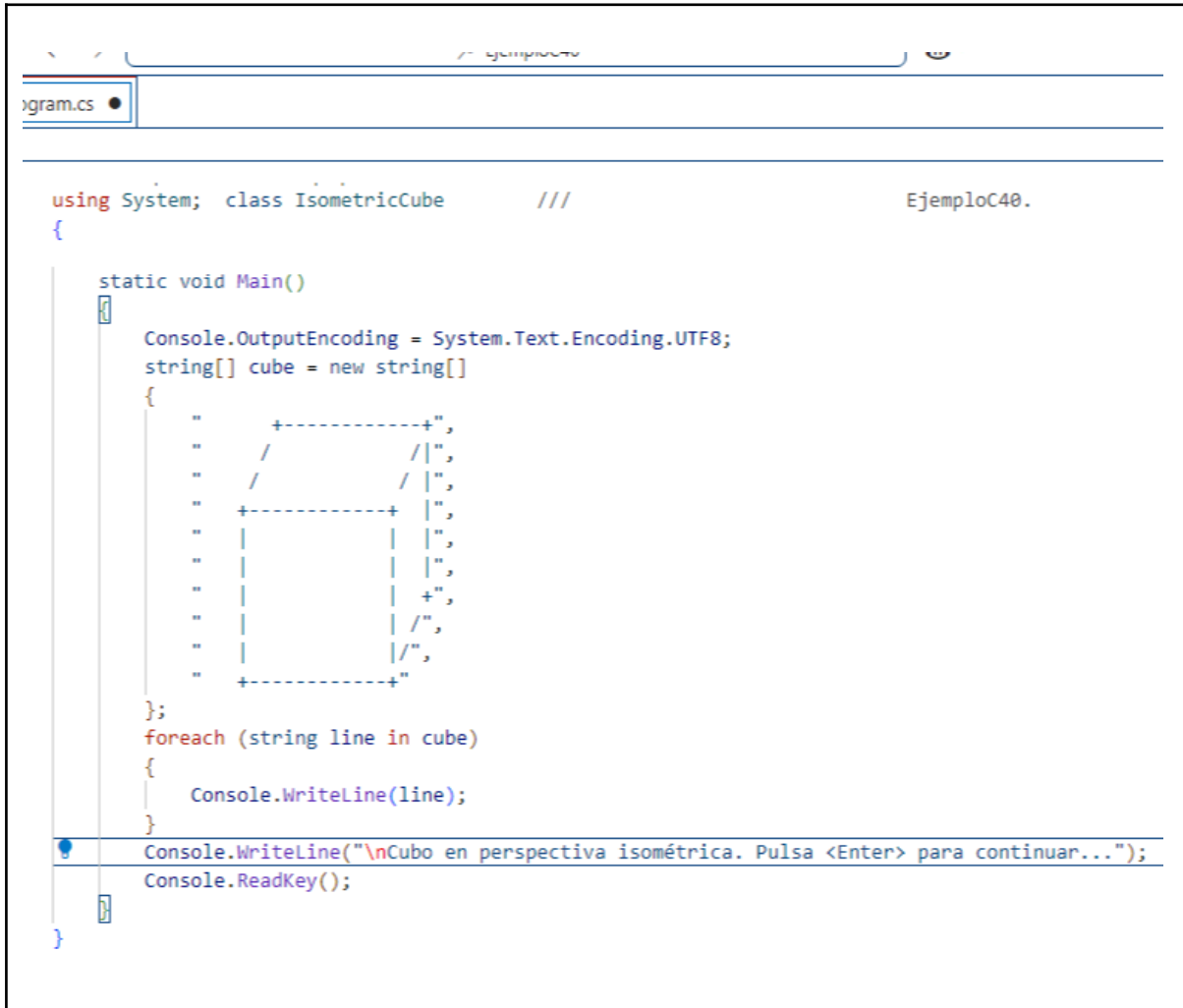
Ahora teclearemos en el símbolo de sistema (command prompt) las siguientes instrucciones para crear un proyecto de prueba llamado “EjemploC40”:

```
cd documents
dotnet new console -o EjemploC40
cd EjemploC40
code .
```

Visual Studio Code se abrirá y veremos la siguiente pantalla:



Ahora reemplazaremos la líneas 1,2 y 3 por el siguiente contenido:

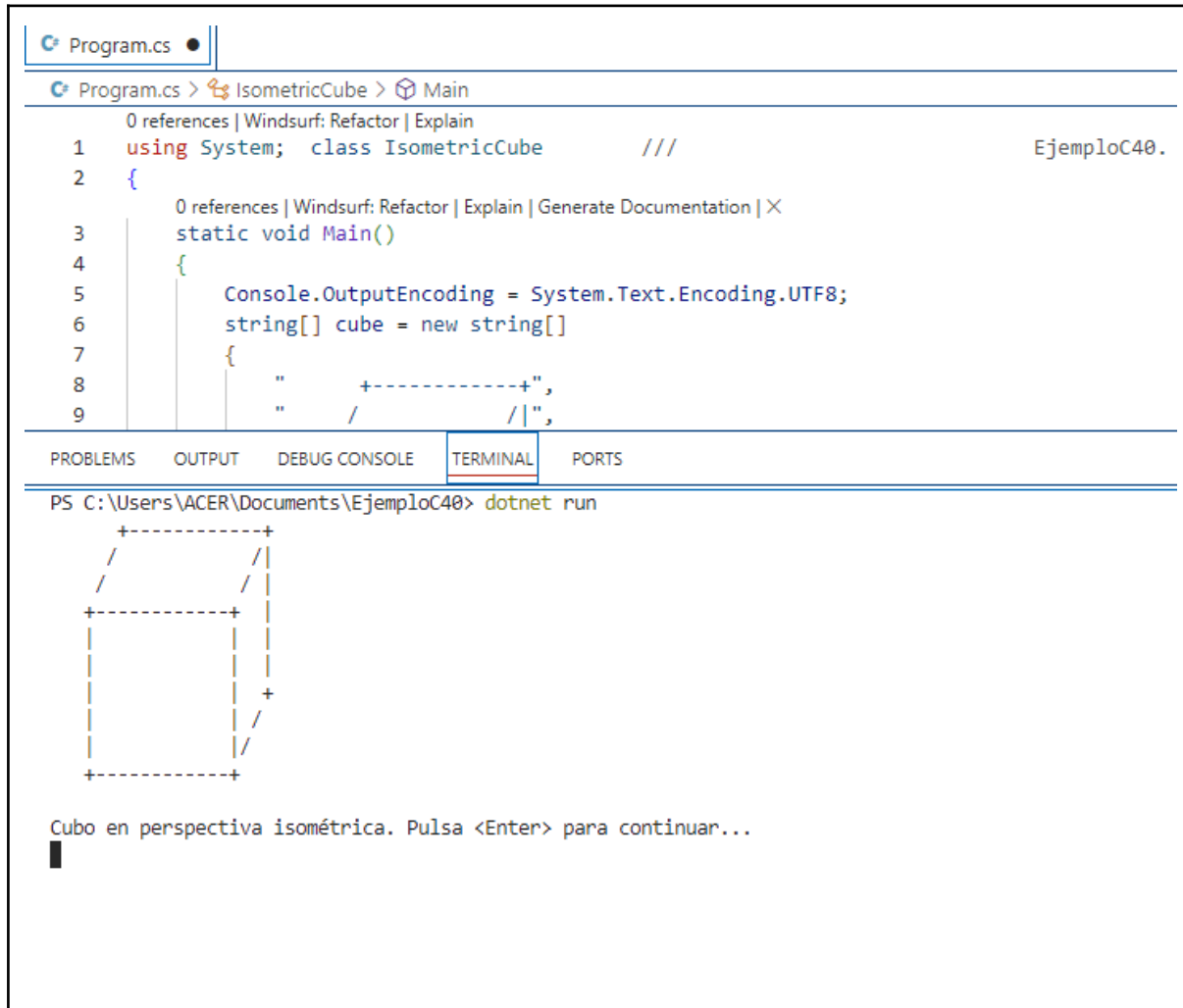


```
using System; class IsometricCube    /// EjemploC40.
{
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;
        string[] cube = new string[]
        {
            "      +-----+",
            "      /       /\n",
            "      /       /\n",
            "      +-----+ |\n",
            "      |       | |\n",
            "      |       | |\n",
            "      |       | |\n",
            "      |       | /\n",
            "      |       | /\n",
            "      +-----+"
        };
        foreach (string line in cube)
        {
            Console.WriteLine(line);
        }
        Console.WriteLine("\nCubo en perspectiva isométrica. Pulsa <Enter> para continuar...");
        Console.ReadKey();
    }
}
```

Al terminar de escribir el código, teclearemos CTRL+S para grabar nuestro archivo.

A continuación abriremos la consola en VS Code (CTRL+ñ) y teclearemos:

dotnet run



The screenshot shows the Visual Studio Code interface. The top editor pane displays a C# file named `Program.cs` with the following code:

```

1  using System; class IsometricCube      /// EjemploC40.
2  {
3      0 references | Windsurf: Refactor | Explain
4      static void Main()
5      {
6          Console.OutputEncoding = System.Text.Encoding.UTF8;
7          string[] cube = new string[]
8          {
9              "      +-----+",
              "      /           /\",

```

The bottom pane shows the **TERMINAL** tab. The command prompt shows the command `dotnet run` being executed. The output displays an ASCII art representation of a cube in isometric perspective, constructed using dashed lines and slashes. Below the cube, the text reads: "Cubo en perspectiva isométrica. Pulsa <Enter> para continuar..." followed by a cursor.

Observe que se ha generado un cubo con caracteres ASCII en perspectiva isométrica.

Reto: Modifique el código para que se genere un segundo cubo a la derecha del primero. ¿Podría servir este código como base para un videojuego?

Ejemplo C41. Creación de un test de 3 preguntas utilizando una cuadrícula que simula el modo texto (consola).

Objetivo: Escribir el código para realizar un test de 3 preguntas dentro de una cuadrícula de 80 columnas y 25 líneas. Las preguntas estarán almacenadas en una matriz. Las respuestas se introducirán por teclado.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC41
cd EjemploC41
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs ///
using System; using System.Drawing;
using System.Runtime.InteropServices; using System.Windows.Forms;

public class GridForm : Form
{
    private const int NumColumns = 80; private const int NumRows = 25;

    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string?[,] textGrid = new string?[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 10, FontStyle.Regular,
GraphicsUnit.Pixel);
    private bool showGridLines = false;
    private bool isDarkMode = false;

    private int? cursorCol = null;
    private int? cursorRow = null;
    private bool showCursor = true;
    private System.Windows.Forms.Timer cursorTimer;

    public GridForm()
    {
        Text = "Fixed 80x25 Grid with Superprint and Superinput";
        ClientSize = new Size(NumColumns * 10, NumRows * 20);
        StartPosition = FormStartPosition.CenterScreen;
        DoubleBuffered = true;
        BackColor = Color.White; ForeColor = Color.Black; KeyPreview = true;

        int rectangleWidth = ClientSize.Width / NumColumns;
        int rectangleHeight = ClientSize.Height / NumRows;

        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
                textGrid[row, col] = "";
            }
        }

        cursorTimer = new System.Windows.Forms.Timer();
        cursorTimer.Interval = 500;
        cursorTimer.Tick += (s, e) =>
        {
            showCursor = !showCursor;
            Invalidate();
        };
        cursorTimer.Start();

        Paint += GridForm_Paint;
        Resize += GridForm_Resize;
        KeyDown += GridForm_KeyDown;
        Shown += GridForm_Shown;
    }
}
/// Continúa en la siguiente página. . .

```

```

private void GridForm_Shown(object? sender, EventArgs e)
{
    superprint(1, 1, "Hola desarrollador!");
    superprint(1, 3, "Presiona Ctrl+C para cambiar el color de la consola");
    superprint(1, 5, "Presiona Ctrl+G para mostrar/ocultar la cuadrícula -grid-");
    string nombre = superinput(1, 9, "Tu nombre: ");
    superprint(1, 9, $"Bienvenido, {nombre}");

    var questions = new (string question, string answer)[]
    {
        ("Madrid es la capital de España.", "V"),
        ("El océano más grande del mundo es el Atlántico.", "F"),
        ("La fórmula química del agua es H2O.", "V"),
    };

    int startRow = 11;
    foreach (var (question, correctAnswer) in questions)
    {
        superprint(1, startRow, question);
        string userAnswer = superinput(1, startRow + 1, "V/F?");

        if (userAnswer.ToUpper() == correctAnswer)
        {
            superprint(1, startRow + 3, "Correcto!");
        }
        else
        {
            superprint(1, startRow + 3, $"Incorrecto! La respuesta correcta es {correctAnswer}.");
        }

        startRow += 5;
    }
}

```

/// Continúa en la siguiente página. . .

```

public void superprint(int column_num, int row_num, string stringx)
{
    for (int i = 0; i < stringx.Length; i++)
    {
        int currentColumn = column_num + i;

        if (row_num >= 0 && row_num < NumRows && currentColumn >= 0 && currentColumn <
NumColumns)
        {
            textGrid[row_num, currentColumn] = stringx[i].ToString();
        }
    }
    Invalidate();
}

public void superprintword(int column_num, int row_num, string wordx)
{
    for (int i = 0; i < wordx.Length; i++)
    {
        int currentColumn = column_num + i;
        if (row_num >= 0 && row_num < NumRows && currentColumn < NumColumns)
        {
            superprint(currentColumn, row_num, wordx[i].ToString());
        }
    }
}

```

/// Continúa en la siguiente página. . .

```

public string superinput(int column_num, int row_num, string prompt)
{
    superprintword(column_num, row_num, prompt);
    int inputStartCol = column_num + prompt.Length;
    int currentCol = inputStartCol;
    string userInput = "";

    cursorRow = row_num;
    cursorCol = currentCol; showCursor = true;
    Invalidate();

    bool enterPressed = false;

    KeyPressEventHandler keyPressHandler = (sender, e) =>
    {
        if (e.KeyChar == (char)Keys.Enter)
        {
            enterPressed = true;
        }
        else if (e.KeyChar == (char)Keys.Back)
        {
            if (userInput.Length > 0)
            {
                userInput = userInput[..^1];
                currentCol--;
                superprint(currentCol, row_num, " ");
            }
        }
        else if (!char.IsControl(e.KeyChar))
        {
            userInput += e.KeyChar;
            superprint(currentCol, row_num, e.KeyChar.ToString());
            currentCol++;
        }

        cursorCol = currentCol;
        Invalidate();
    };

    KeyPress += keyPressHandler;

    while (!enterPressed)
    {
        Application.DoEvents();
    }

    KeyPress -= keyPressHandler;
    cursorCol = null;
    cursorRow = null;
    Invalidate();

    return userInput;
}

```

/// Continúa en la siguiente página. . .

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    using Pen pen = new Pen(ForeColor);
    using StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center };
    using Brush textBrush = new SolidBrush(ForeColor);
    Brush cursorBrush = isDarkMode ? Brushes.Cyan : Brushes.Black;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            if (showGridLines)
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);

            e.Graphics.DrawString(textGrid[row, col] ?? "", displayFont, textBrush,
gridRectangles[row, col], sf);
        }
    }

    if (cursorCol.HasValue && cursorRow.HasValue && showCursor)
    {
        Rectangle cursorRect = gridRectangles[cursorRow.Value, cursorCol.Value];
        e.Graphics.FillRectangle(cursorBrush, cursorRect);
    }
}

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    Invalidate();
}

```

/// Continúa en la siguiente página. . .


```

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    if (e.Control && e.KeyCode == Keys.C)
    {
        if (!isDarkMode)
        {
            BackColor = Color.Black;
            ForeColor = Color.Cyan;
            displayFont = new Font("Consolas", 10, FontStyle.Regular,
GraphicsUnit.Pixel);
            isDarkMode = true;
        }
        else
        {
            BackColor = Color.White;
            ForeColor = Color.Black;
            displayFont = new Font("Consolas", 10, FontStyle.Regular,
GraphicsUnit.Pixel);
            isDarkMode = false;
        }

        Invalidate();
    }

    if (e.Control && e.KeyCode == Keys.G)
    {
        showGridLines = !showGridLines;
        Invalidate();
    }
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}
}
/// Fin del EjemploC41.

```

A continuación ejecute el EjemploC41 creado y verá la siguiente ventana:



```
Fixed 80x25 Grid with Superprint and Superinput

Hola desarrollador!

Presiona Ctrl+C para cambiar el color de la consola

Presiona Ctrl+G para mostrar/ocultar la cuadrícula -grid-

Bienvenido, Javier

Madrid es la capital de España.
V/F?V

Correcto!

El océano mas grande del mundo es el Atlantico.
V/F?f

Correcto!

La formula química del agua es H2O.
V/F?v

Correcto!
```

Observe el test de 3 preguntas que se ha implementado con la función `superinput` y el control del cursor. Es posible añadir preguntas a la matriz **questions**.

Reto: Al añadir más preguntas, tendrá un problema: Las preguntas sobrepasarán el límite de las 25 líneas de la cuadrícula (grid). ¿Cómo lo solucionaría? ¿Cómo añadiría un acumulador de preguntas correctas para mostrar el resultado del test?

Ejemplo C42. Creación de un test de 4 preguntas utilizando una cuadrícula que simula el modo texto (consola).

Objetivo: Escribir el código para realizar un test de 4 preguntas dentro de una cuadrícula de 80 columnas y 25 líneas. Las preguntas estarán almacenadas en una matriz. Las respuestas se introducirán por teclado.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new winforms -n EjemploC42
cd EjemploC42
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Form1.cs y **Form1.Designer.cs** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs    /// EjemploC42. /// Test de 4 preguntas con msgbox y grid implementados
using System;    using System.Drawing;
using System.Runtime.InteropServices; using System.Windows.Forms;

public class GridForm : Form
{
    private const int NumColumns = 80;
    private const int NumRows = 25;    // grid para modo terminal

    private Rectangle[,] gridRectangles = new Rectangle[NumRows, NumColumns];
    private string?[,] textGrid = new string?[NumRows, NumColumns];
    private Font displayFont = new Font("Consolas", 10, FontStyle.Regular,
GraphicsUnit.Pixel);
    private bool showGridLines = false; private bool isDarkMode = false;

    private int? cursorCol = null; private int? cursorRow = null;
    private bool showCursor = true;
    private System.Windows.Forms.Timer cursorTimer;

    private int superpuntos = 0; // Variable to track the total score

    public GridForm()
    {
        Text = "Fixed 80x25 Grid with Superprint and Superinput";
        ClientSize = new Size(NumColumns * 10, NumRows * 20);
        StartPosition = FormStartPosition.CenterScreen;
        DoubleBuffered = true;
        BackColor = Color.White; ForeColor = Color.Black;
        KeyPreview = true;

        int rectangleWidth = ClientSize.Width / NumColumns;
        int rectangleHeight = ClientSize.Height / NumRows;

        for (int row = 0; row < NumRows; row++)
        {
            for (int col = 0; col < NumColumns; col++)
            {
                gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
                textGrid[row, col] = "";
            }
        }

        cursorTimer = new System.Windows.Forms.Timer();
        cursorTimer.Interval = 500;
        cursorTimer.Tick += (s, e) =>
        {
            showCursor = !showCursor; Invalidate();
        };
        cursorTimer.Start();

        Paint += GridForm_Paint;
        Resize += GridForm_Resize;
        KeyDown += GridForm_KeyDown; Shown += GridForm_Shown;
    }
}
/// Continúa en la siguiente página. . .

```

```

private void GridForm_Shown(object? sender, EventArgs e)
{
    superprint(1, 1, "Bienvenido!");
    superprint(1, 3, "Presiona Ctrl+C para cambiar el color de la consola");
    superprint(1, 5, "Presiona Ctrl+G para mostrar/ocultar la cuadrícula -grid-");

    string nombrex = superinput(1, 9, "Tu nombre: ");
    superprint(1, 9, $"Bienvenido, {nombrex}");

    var questions = new (string question, string answer)[]
    {
        ("Madrid es la capital de España.", "V"),
        ("El océano mas grande del mundo es el Atlantico.", "F"),
        ("El lago grande del mundo es el Caspio.", "V"),
        ("La fórmula química del agua es H2O.", "V"),
    };

    foreach (var (question, correctAnswer) in questions)
    {
        ClearGrid(); // Clear the grid before displaying each question
        int startRow = 1; // Reset the starting line for each question

        superprint(1, startRow, question);
        string userAnswer = superinput(1, startRow + 1, "V/F?");

        if (userAnswer.ToUpper() == correctAnswer)
        {
            superpuntos++; // Increment the score for a correct answer
            MessageBox.Show("Correcto!", "Respuesta", MessageBoxButtons.OK,
grid
                MessageBoxIcon.Information);
        }
        else
        {
            MessageBox.Show($"Incorrecto! La respuesta correcta es {correctAnswer}.",
                "Respuesta", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }

        System.Threading.Thread.Sleep(1000); // Wait for 1 second before clearing the
    }

    // Display the final score in a new MessageBox
    MessageBox.Show($"Puntuación final: {superpuntos}/{questions.Length}",
        "Resultado Final", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
}

```

/// Continúa en la siguiente página. . .

```

private void ClearGrid()
{
    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            textGrid[row, col] = ""; // Clear each cell
        }
    }
    Invalidate(); // Refresh the grid to reflect changes
}

public void superprint(int column_num, int row_num, string stringx)
{
    for (int i = 0; i < stringx.Length; i++)
    {
        int currentColumn = column_num + i;

        if (row_num >= 0 && row_num < NumRows && currentColumn >= 0 && currentColumn <
NumColumns)
        {
            textGrid[row_num, currentColumn] = stringx[i].ToString();
        }
    }
    Invalidate();
}

public void superprintword(int column_num, int row_num, string wordx)
{
    for (int i = 0; i < wordx.Length; i++)
    {
        int currentColumn = column_num + i;
        if (row_num >= 0 && row_num < NumRows && currentColumn < NumColumns)
        {
            superprint(currentColumn, row_num, wordx[i].ToString());
        }
    }
}

```

/// Continúa en la siguiente página. . .

```

public string superinput(int column_num, int row_num, string prompt)
{
    superprintword(column_num, row_num, prompt);
    int inputStartCol = column_num + prompt.Length;
    int currentCol = inputStartCol;
    string userInput = "";

    cursorRow = row_num;
    cursorCol = currentCol;
    showCursor = true;
    Invalidate();

    bool enterPressed = false;

    KeyPressEventHandler keyPressHandler = (sender, e) =>
    {
        if (e.KeyChar == (char)Keys.Enter)
        {
            enterPressed = true;
        }
        else if (e.KeyChar == (char)Keys.Back)
        {
            if (userInput.Length > 0)
            {
                userInput = userInput[..^1];
                currentCol--;
                superprint(currentCol, row_num, " ");
            }
        }
        else if (!char.IsControl(e.KeyChar))
        {
            userInput += e.KeyChar;
            superprint(currentCol, row_num, e.KeyChar.ToString());
            currentCol++;
        }

        cursorCol = currentCol; Invalidate();
    };

    KeyPress += keyPressHandler;

    while (!enterPressed)
    {
        Application.DoEvents();
    }

    KeyPress -= keyPressHandler;
    cursorCol = null;
    cursorRow = null;
    Invalidate();

    return userInput;
}

/// Continúa en la siguiente página. . .

```

```

private void GridForm_Paint(object sender, PaintEventArgs e)
{
    using Pen pen = new Pen(ForeColor);
    using StringFormat sf = new StringFormat { Alignment = StringAlignment.Center,
LineAlignment = StringAlignment.Center };
    using Brush textBrush = new SolidBrush(ForeColor);
    Brush cursorBrush = isDarkMode ? Brushes.Cyan : Brushes.Black;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            if (showGridLines)
                e.Graphics.DrawRectangle(pen, gridRectangles[row, col]);

            e.Graphics.DrawString(textGrid[row, col] ?? "", displayFont, textBrush,
gridRectangles[row, col], sf);
        }
    }

    if (cursorCol.HasValue && cursorRow.HasValue && showCursor)
    {
        Rectangle cursorRect = gridRectangles[cursorRow.Value, cursorCol.Value];
        e.Graphics.FillRectangle(cursorBrush, cursorRect);
    }
}

```

/// Continúa en la siguiente página. . .


```

private void GridForm_Resize(object sender, EventArgs e)
{
    int rectangleWidth = ClientSize.Width / NumColumns;
    int rectangleHeight = ClientSize.Height / NumRows;

    for (int row = 0; row < NumRows; row++)
    {
        for (int col = 0; col < NumColumns; col++)
        {
            gridRectangles[row, col] = new Rectangle(col * rectangleWidth, row *
rectangleHeight, rectangleWidth, rectangleHeight);
        }
    }

    Invalidate();
}

private void GridForm_KeyDown(object sender, KeyEventArgs e)
{
    if (e.Control && e.KeyCode == Keys.C)
    {
        if (!isDarkMode)
        {
            BackColor = Color.Black;    ForeColor = Color.Cyan;
            displayFont = new Font("Consolas", 10, FontStyle.Regular,
GraphicsUnit.Pixel);
            isDarkMode = true;
        }
        else
        {
            BackColor = Color.White;
            ForeColor = Color.Black;
            displayFont = new Font("Consolas", 10, FontStyle.Regular,
GraphicsUnit.Pixel);
            isDarkMode = false;
        }

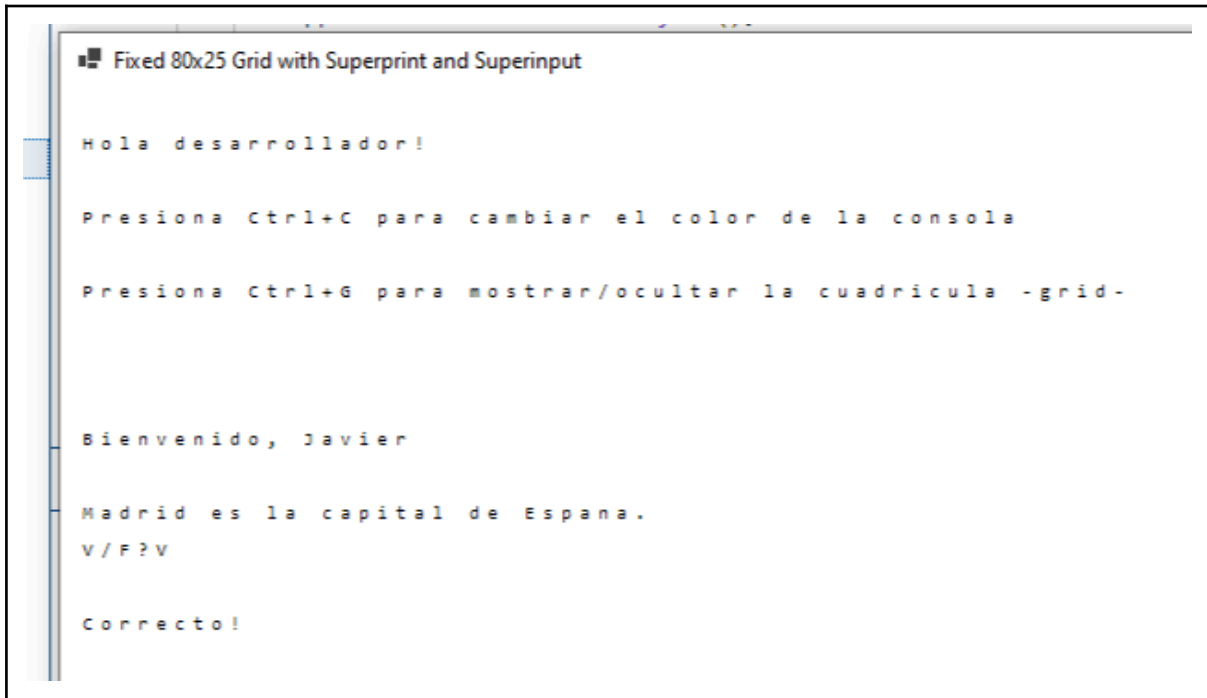
        Invalidate();
    }

    if (e.Control && e.KeyCode == Keys.G)
    {
        showGridLines = !showGridLines;    Invalidate();
    }
}

[STAThread]
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new GridForm());
}
}
/// Fin del código del EjemploC42.

```

A continuación ejecute el EjemploC42 creado y verá la siguiente ventana:



```
Fixed 80x25 Grid with Superprint and Superinput

Hola desarrollador!

Presiona Ctrl+C para cambiar el color de la consola

Presiona Ctrl+G para mostrar/ocultar la cuadrícula -grid-

Bienvenido, Javier

Madrid es la capital de España.
V/F?V

Correcto!
```

Observe el test (de 4 preguntas) que se ha implementado con la función `superinput` y el control del cursor. Es posible añadir preguntas a la matriz **questions**.

Reto: ¿Cómo añadiría otros colores a la ventana que muestra el resultado del test?

Ejemplo C43a. Juego de los dados (II).

Objetivo: Generar dos números aleatorios entre el 1 y el 6 utilizando el metodo **Random.next** que pertenece a la clase **Random** en C#. Programar el juego con dos tiradas de dados como mínimo implementando la pregunta: “¿Quiere intentarlo de nuevo? ”.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new console -o EjemploC43a
cd EjemploC43a
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

EjemploC43a.sln y **EjemploC43a.csproj** se modificarán con el código de **Program.cs**. No necesita utilizar Visual Designer.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs
EjemploC43a

using System;

class Program
{
    static void Main()
    {
        Random random = new Random();
        bool jugarDeNuevo = true;

        while (jugarDeNuevo)
        {
            int numero1 = random.Next(1, 7); // 1 a 6 inclusive
            int numero2 = random.Next(1, 7);

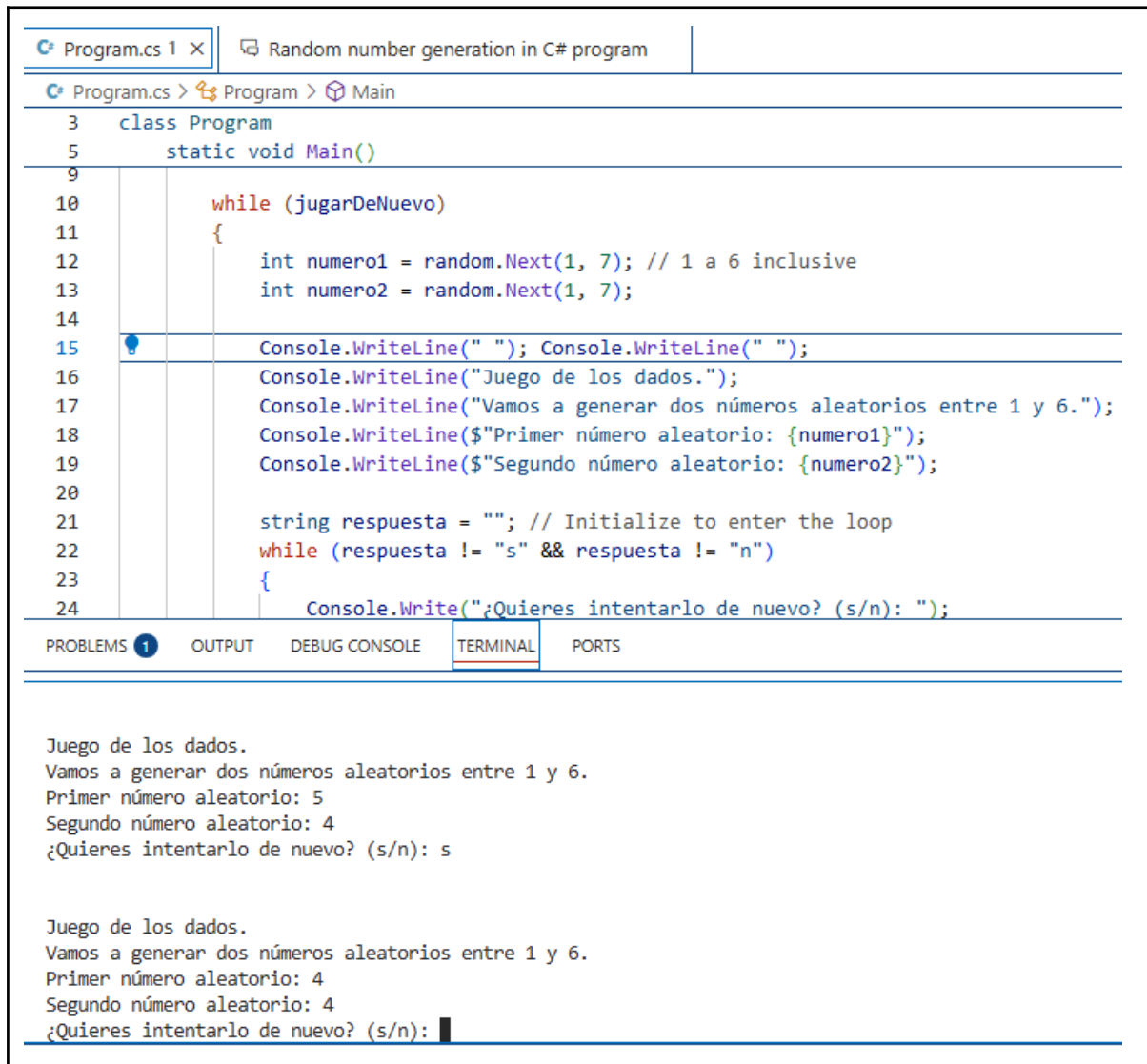
            Console.WriteLine("Juego de los dados.");
            Console.WriteLine("Vamos a generar dos números aleatorios entre 1 y 6.");
            Console.WriteLine($"Primer número aleatorio: {numero1}");
            Console.WriteLine($"Segundo número aleatorio: {numero2}");

            string respuesta = ""; // Prepara var. respuesta
            while (respuesta != "s" && respuesta != "n")
            {
                Console.Write("¿Quieres intentarlo de nuevo? (s/n): ");
                respuesta = Console.ReadLine()?.Trim().ToLower();
                if (respuesta != "s" && respuesta != "n")
                {
                    Console.WriteLine("Entrada no válida. Por favor, ingresa 's' o 'n'.");
                }
            }

            jugarDeNuevo = (respuesta == "S"); ///como ahora respuesta ya no es bool true, saldrá del loop
        }
    }
}

```

A continuación ejecute el **EjemploC43a** creado y verá la siguiente ventana:



The screenshot shows a Visual Studio Code editor with a C# file named Program.cs. The code is a simple dice game program. It uses a while loop to repeatedly generate two random numbers between 1 and 6, inclusive, and prints them to the console. The loop continues as long as the user enters 's' (si) or 'n' (no) when prompted to try again. The output window shows the program's execution, including the generated numbers and the user's input.

```

3  class Program
5      static void Main()
9
10     while (jugarDeNuevo)
11     {
12         int numero1 = random.Next(1, 7); // 1 a 6 inclusive
13         int numero2 = random.Next(1, 7);
14
15         Console.WriteLine(" "); Console.WriteLine(" ");
16         Console.WriteLine("Juego de los dados.");
17         Console.WriteLine("Vamos a generar dos números aleatorios entre 1 y 6.");
18         Console.WriteLine($"Primer número aleatorio: {numero1}");
19         Console.WriteLine($"Segundo número aleatorio: {numero2}");
20
21         string respuesta = ""; // Initialize to enter the loop
22         while (respuesta != "s" && respuesta != "n")
23         {
24             Console.Write("¿Quieres intentarlo de nuevo? (s/n): ");

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Juego de los dados.
Vamos a generar dos números aleatorios entre 1 y 6.
Primer número aleatorio: 5
Segundo número aleatorio: 4
¿Quieres intentarlo de nuevo? (s/n): s

Juego de los dados.
Vamos a generar dos números aleatorios entre 1 y 6.
Primer número aleatorio: 4
Segundo número aleatorio: 4
¿Quieres intentarlo de nuevo? (s/n): █

```

Observe que mientras la variable de tipo lógico (booleana) **jugarDeNuevo** sea **true**, se seguirá ejecutando la parte del programa donde se crean los números aleatorios. Recuerde que el valor booleano puede ser verdadero (true) o falso (false). Es decir, todo lo que sea distinto de true, será falso.

Ejemplo C44a. Cálculo del volumen de un cubo utilizando matrices.

Objetivo: Escribir el código para calcular el volumen de un poliedro utilizando una matriz.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents
dotnet new console -o EjemploC44a
cd EjemploC44a
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

EjemploC44a.sln y **EjemploC44a.csproj** se modificarán con el código de **Program.cs**.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

/// Program.cs /// EjemploC44a
///
///                                     Cálculo del volumen de un poliedro utilizando
matrices

using System;

class Program
{
    public static int[] myArray = { 3, 4, 5 };           /// array global público

    // definición de funciones
    static int Multiplica(int a, int b, int c)          /// la función devuelve un int
    {
        Console.WriteLine("Multiplicaremos los siguientes valores:");
        Console.WriteLine(a + ", " + b + ", " + c);
        int resultado = a * b * c;
        return resultado;
    }

    // - - - - -

    // programa principal (main program)
    static void Main()
    {
        string ax = "Vamos a calcular el volumen de un poliedro.";
        Console.WriteLine(ax);
        string bx = "Las aristas no miden lo mismo -en metros-";
        Console.WriteLine(bx);

        int total = Multiplica(myArray[0], myArray[1], myArray[2]);
        Console.WriteLine("El total es:");
        Console.WriteLine(total);
        Console.WriteLine("metros cubicos.");
    }
}

// Importante:
// En este ejemplo, hemos declarado un array global llamado myArray.
// El ARRAY global public myArray (declarado como public static) puede
// ser usado en cualquier parte dentro de la clase Program en
// Program.cs. Se Puede acceder a él desde cualquier método,
// incluyendo Main y Multiplica, usando Program.myArray (si se
// accede desde fuera de la clase).
// o simplemente myArray (si se accede desde dentro de la clase).

```

A continuación ejecute el **EjemploC44a** creado y verá la siguiente ventana:

The screenshot shows the Visual Studio Code interface. The top part is the editor with a file named `Program.cs`. The code is as follows:

```

1  using System;
2
3  0 references
4  class Program
5  {
6      3 references
7      public static int[] myArray = { 3, 4, 5 };
8
9      // definicion de funciones
10     1 reference
11     static int Multiplica(int a, int b, int c)
12     {
13         Console.WriteLine("Multiplicaremos los siguientes valores:");
14         Console.WriteLine(a + ", " + b + ", " + c);
15         int resultado = a * b * c;
16         return resultado;
17     }
18 }

```

The bottom part of the screenshot shows the **TERMINAL** tab. The output is as follows:

```

PS C:\Users\ACER\Documents\EjemploC44a>
PS C:\Users\ACER\Documents\EjemploC44a> & 'c:\Users\ACER\.vscode\extensions\ms-dotnettools.86_64\vsdbg.exe' '--interpreter=vscode' '--connection=f22a59b37751466d8e86631b54d00484'
Vamos a calcular el volumen de un poliedro.
Las aristas no miden lo mismo -en metros-
Multiplicaremos los siguientes valores:
3, 4, 5
El total es:
60
metros cubicos.
PS C:\Users\ACER\Documents\EjemploC44a>

```

Observe que en este ejemplo, hemos declarado un array global llamado **myArray**. El array global public **myArray** (declarado como public static) puede ser usado en cualquier parte dentro de la clase Program en Program.cs.

Ejemplo C45a. Cálculo de la suma de los elementos de una matriz pública (global).

Objetivo: Escribir el código para calcular la suma de los elementos de una matriz pública (global) utilizando un método global.

Utilice las siguientes instrucciones para crear un proyecto de prueba:

```
cd documents  
dotnet new console -o EjemploC45a  
cd EjemploC45a  
code .
```

Visual Studio Code se abrirá y veremos los archivos que componen nuestro proyecto.

Haga estos cambios únicamente en el código del script **Program.cs**:

```

using System;                                     // EjemploC45a

public class Yes
{
    public static int[] myArray = { 0, 4, 5 };    // Public Global array

    public static int AddValues(int x, int y)    // Public Global method
    {
        return x + y;
    }
}

class Program                                     // Programa principal
{
    static void Main()
    {
        int a = Yes.myArray[1]; // Access array via Yes class
        int b = Yes.myArray[2];

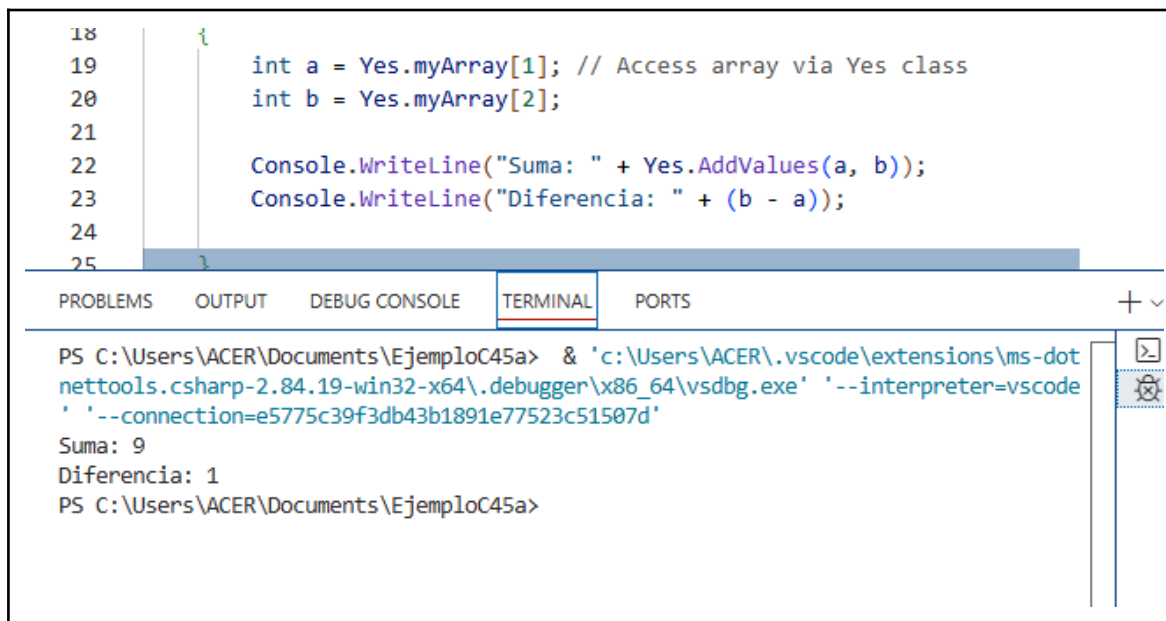
        Console.WriteLine("Suma: " + Yes.AddValues(a, b));
        Console.WriteLine("Diferencia: " + (b - a));

    }
}

```

Observe que en este ejemplo, hemos declarado un array global llamado **myArray**. El array global public **myArray** (declarado como public static) puede ser usado en cualquier parte dentro de la clase Program en Program.cs. Lo mismo ocurre con la función **AddValues**. Como en su momento se declararon dentro de la clase **Yes**, al llamarlas, deberemos poner delante la clase a la que pertenecen (se han marcado en negrita para su mejor comprensión).

A continuación ejecute el EjemploC45a creado y verá la siguiente ventana:



The screenshot shows a Visual Studio Code editor with a C# file named EjemploC45a.cs. The code is as follows:

```
18 {  
19     int a = Yes.myArray[1]; // Access array via Yes class  
20     int b = Yes.myArray[2];  
21  
22     Console.WriteLine("Suma: " + Yes.AddValues(a, b));  
23     Console.WriteLine("Diferencia: " + (b - a));  
24  
25 }
```

Below the editor, the TERMINAL tab is active, showing the command used to run the program and its output:

```
PS C:\Users\ACER\Documents\EjemploC45a> & 'c:\Users\ACER\.vscode\extensions\ms-dotnettools.csharp-2.84.19-win32-x64\.debugger\x86_64\vsdbg.exe' '--interpreter=vscode' '--connection=e5775c39f3db43b1891e77523c51507d'  
Suma: 9  
Diferencia: 1  
PS C:\Users\ACER\Documents\EjemploC45a>
```

Observe que se ha tomado el segundo elemento de la matriz, `Yes.myArray[1]`, y el tercero, `Yes.myArray[2]` de la matriz para hacer los cálculos.

Importante: Recuerde que para cambiar el valor de un elemento, lo haremos mediante la orden:

`Yes.myArray[0] = 10;`

En este caso, el primer valor del array, que era 0, lo hemos cambiado por el número 10.

Reto 1: Compruebe que el siguiente ejemplo suma todos los elementos del array y muestra el resultado:

```
using System;

public class Yes
{
    public static int[] myArray = { 2, 4, 6, 8, 10 };           // array público
}

class Program
{
    static void Main()
    {
        int sum = 0;  int i = 0;                               // inicializar contador y sum

        while (i < Yes.myArray.Length)                        //sumar elementos hasta encontrar último
        {
            sum += Yes.myArray[i];
            i++;
        }

        Console.WriteLine("Suma total de los elementos del array: " + sum);
    }
}
```

Reto 2: Modifique el ejemplo anterior para que se muestren todos los sumandos y el resultado de la operación.

Ejemplo C46. Script de comprobación de conexión entre Unity y C#

Objetivo: Escribir el código necesario para comprobar que Unity 6 está configurado correctamente y conectado con Microsoft Visual Studio Code y Microsoft C#.

“Unity Hub” es una herramienta de Unity 6 para gestionar proyectos. Desde esta utilidad se puede instalar una versión anterior de Unity independiente para gestionar proyectos antiguos. Además, “Unity Hub” permite gestionar diferentes paquetes de recursos.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

- a .** Abra la aplicación Unity Hub.
- b .** Haga clic en “New Project”.
- c .** Elija la plantilla “3D (Core)”.
- d.** Escriba el siguiente nombre de proyecto: “U1-HelloWorldProject”.

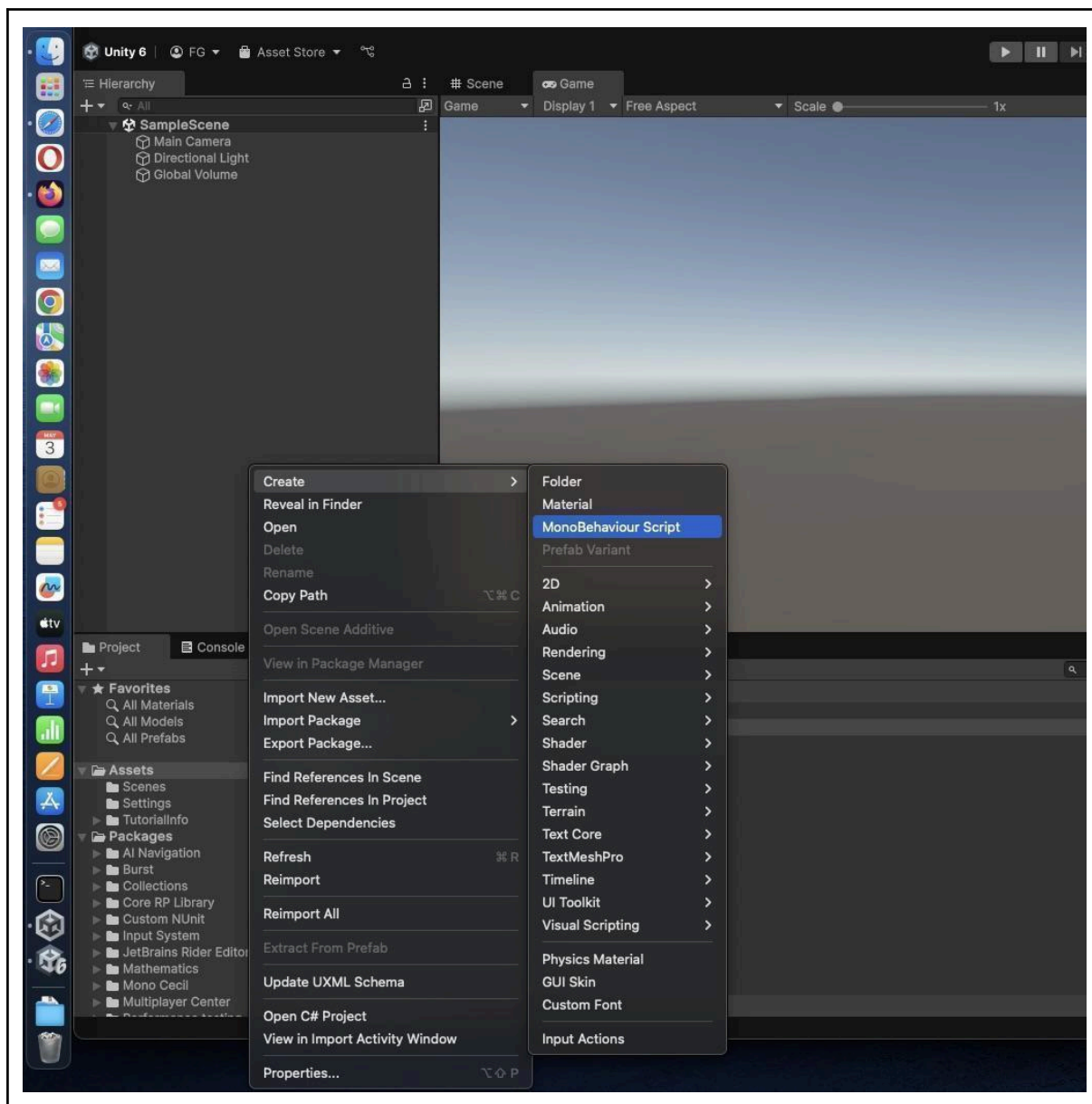
Paso 2.

a.

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

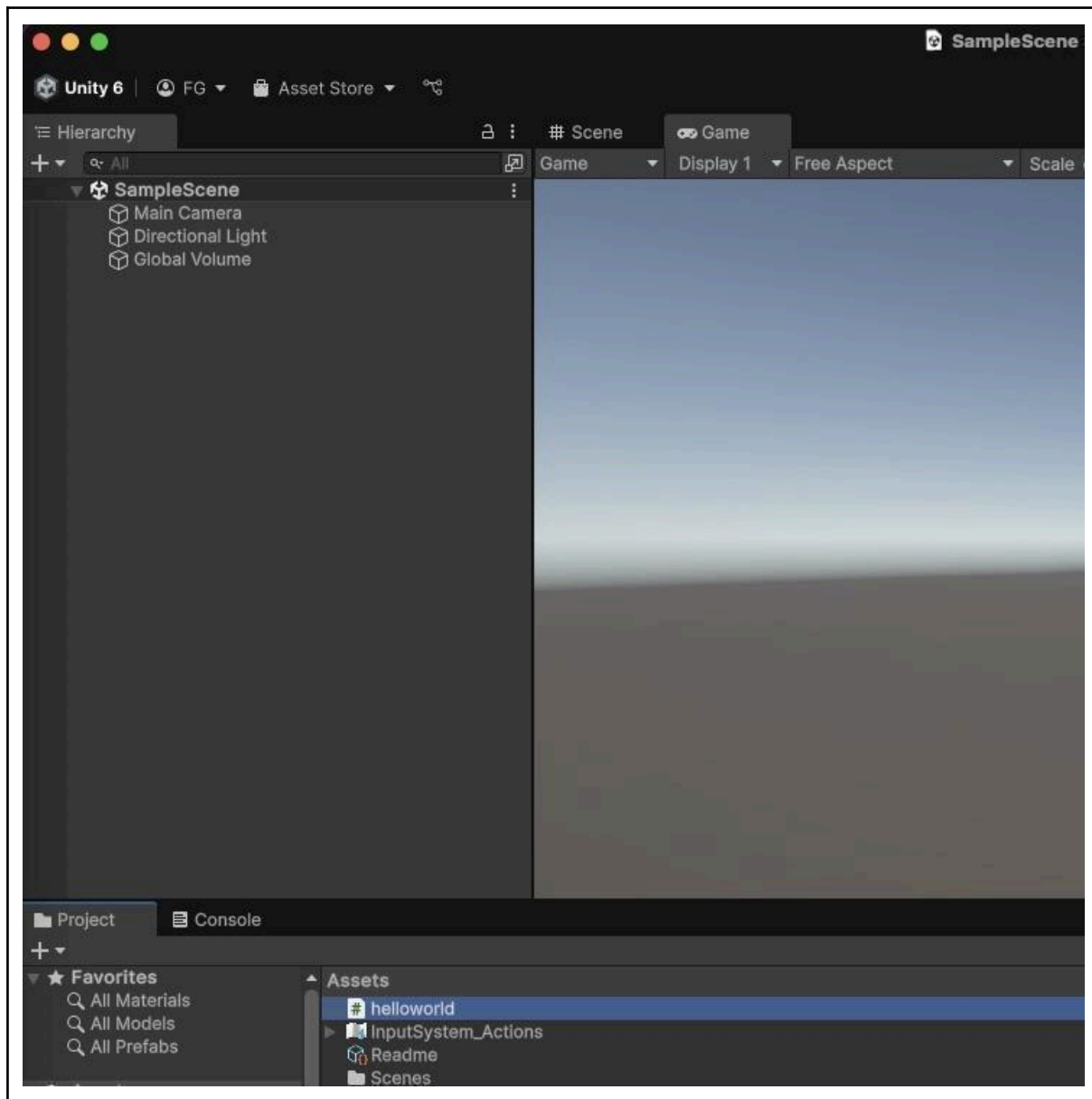
b.

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “HelloWorld”.



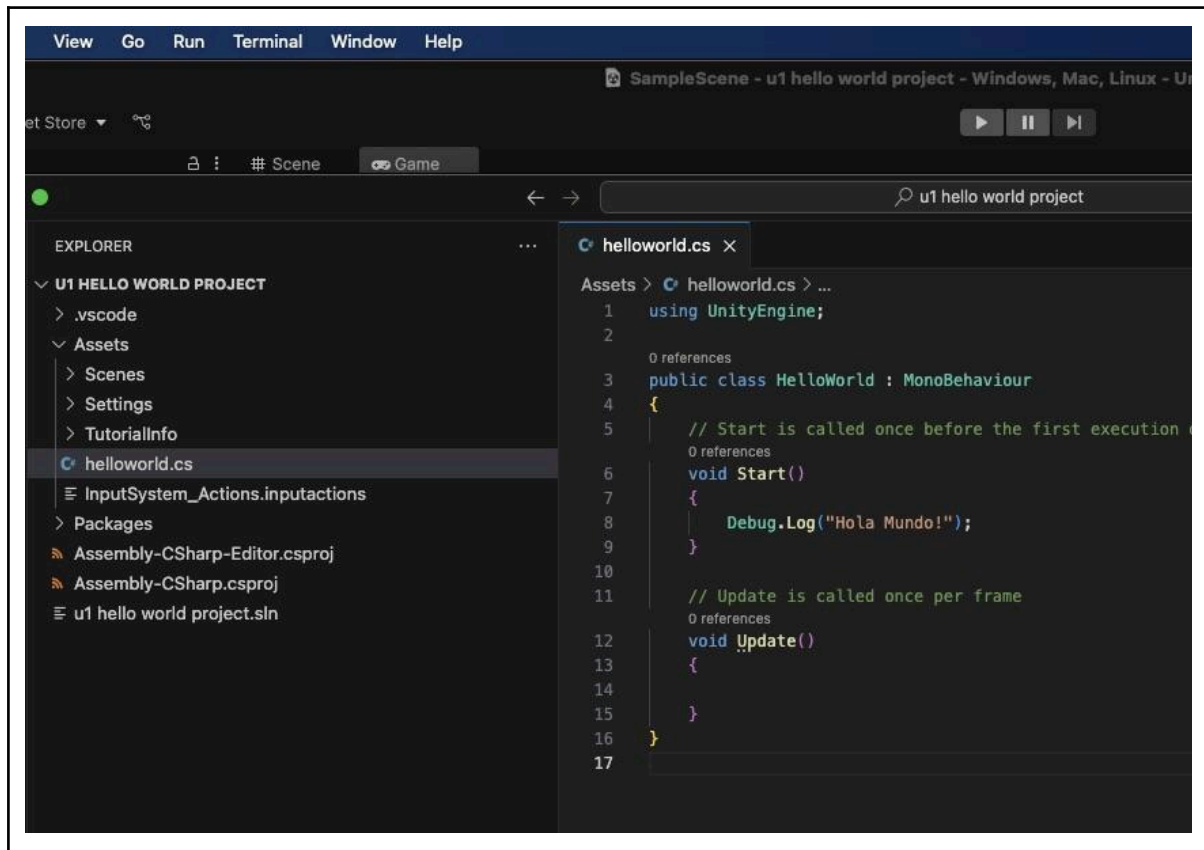
Paso 3.

Haga doble clic en el archivo “HelloWorld.cs” que acaba de renombrar.



Paso 4.

Se abrirá Microsoft Visual Studio con el archivo “HelloWorld.cs”. Borre el código que aparece por defecto y copie el que se muestra en esta imagen:



Nota:

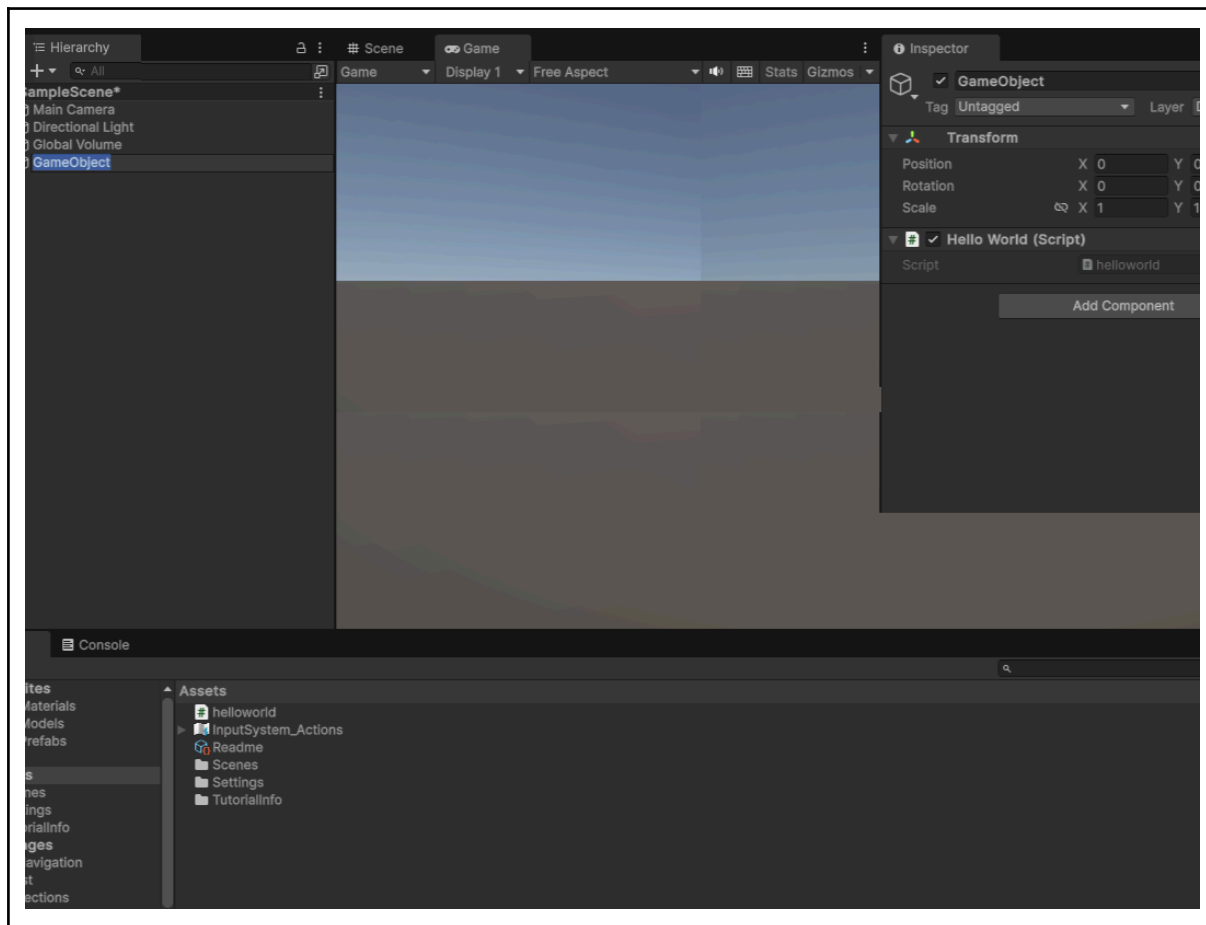
Recuerde que el nombre de la clase creada, HelloWorld, debe coincidir siempre con el nombre del archivo que contiene el script. En nuestro ejemplo el archivo que contiene el código se llama Helloworld.cs.

No olvide grabar el archivo que acaba de modificar y cierre Microsoft Visual Studio.

Paso 5.

Vuelva a Unity y siga estos pasos:

- En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “Create Empty” (crear vacío).
- Seleccione “GameObject”.
- En el “Inspector”, haga clic en “Add Component” (añadir componente).
- Busque y seleccione “HelloWorld” (si no aparece en la lista, teclee el nombre “HelloWorld” para que aparezca).



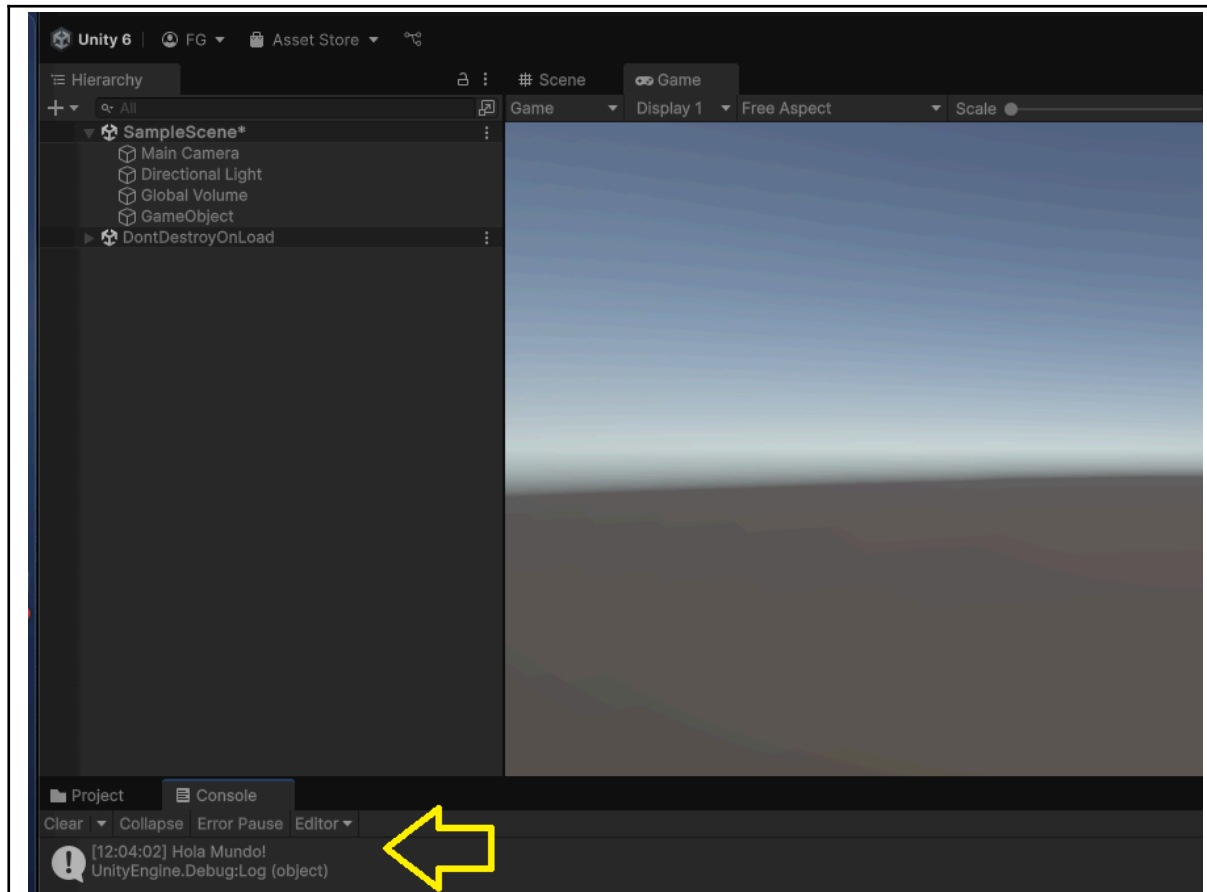
Nota:

Recuerde que un “script” siempre tiene que ir asociado a un “GameObject”.

Paso 6.

Haga clic en el botón “Play” que está en el centro de la parte superior de la pantalla principal.

Abra la consola (para ello haga clic en “Window”; después haga clic en “General” y finalmente haga clic en “Console”).



Observe que junto a la flecha amarilla que aparece en la imagen podemos leer la frase “Hola Mundo!”. Esto indica que nuestro programa escrito en lenguaje C# con Microsoft Visual Studio Code se comunica correctamente con Unity 6.

Reto 1a:

Implemente los siguientes cambios en el programa anterior para crear dos variables llamadas **interruptor** e **interruptor2**.

Objetivo:

Comprobar que Unity muestra en el Inspector las variables públicas creadas.

```
using UnityEngine;

// This script is attached to the GameObject called "GameObject" in the scene

public class HelloWorld : MonoBehaviour
{
    public int interruptor1 = 1;
    public int interruptor2 = 1;

    // Start is called once before 1st execution of Update after MonoBehaviour is created
    void Start()
    {
        // Check that Unity can see public variables and work out (hacer) calculations:
        // These values are set in the Inspector and can be changed there or with code.

        interruptor2 = 2 * interruptor1;
        Debug.Log("Hello world!!");
    }

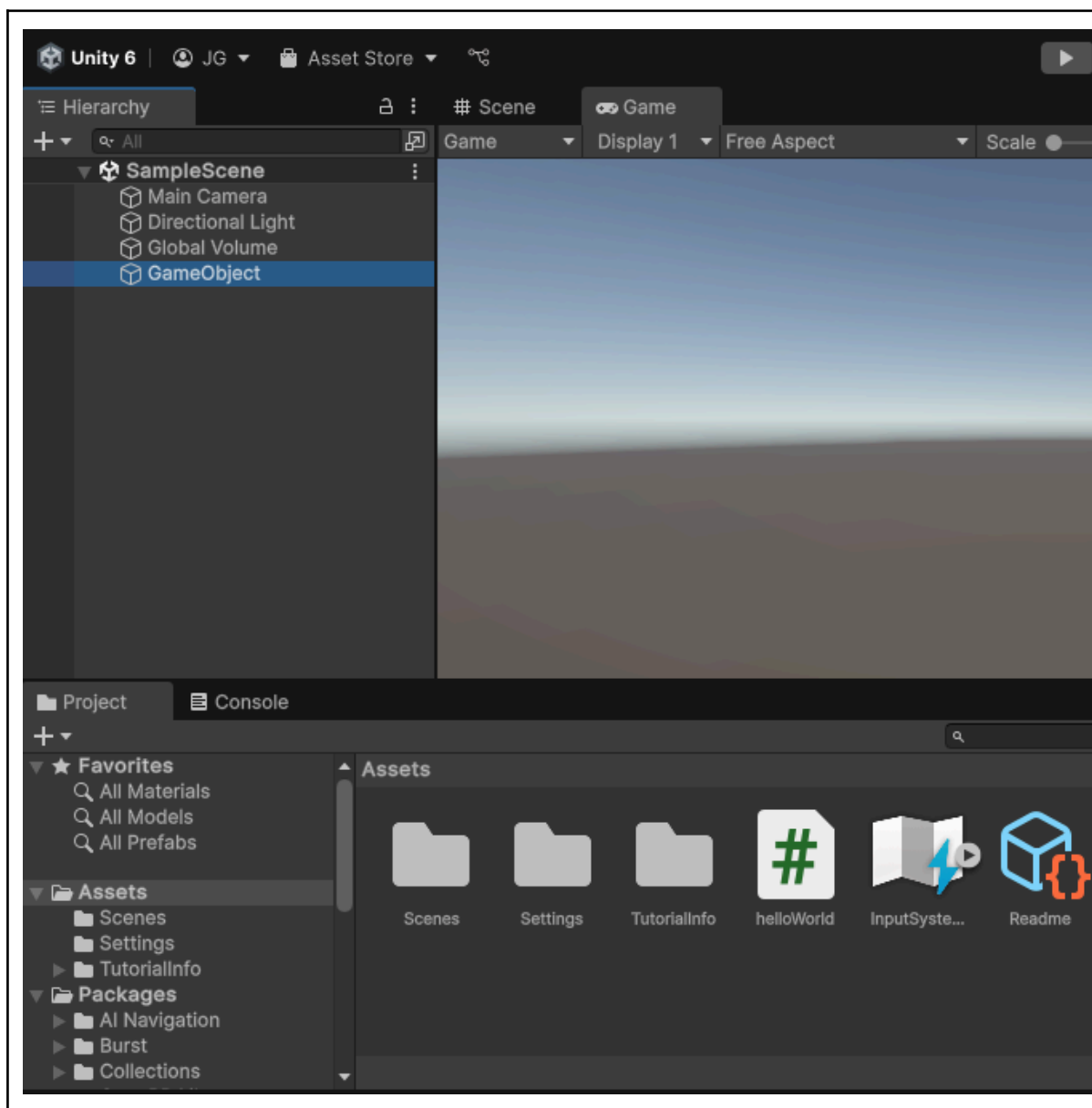
    // Update is called once per frame
    void Update()
    {
    }
}
```

Grabe los cambios realizados en el código para que Unity actualice el Inspector.

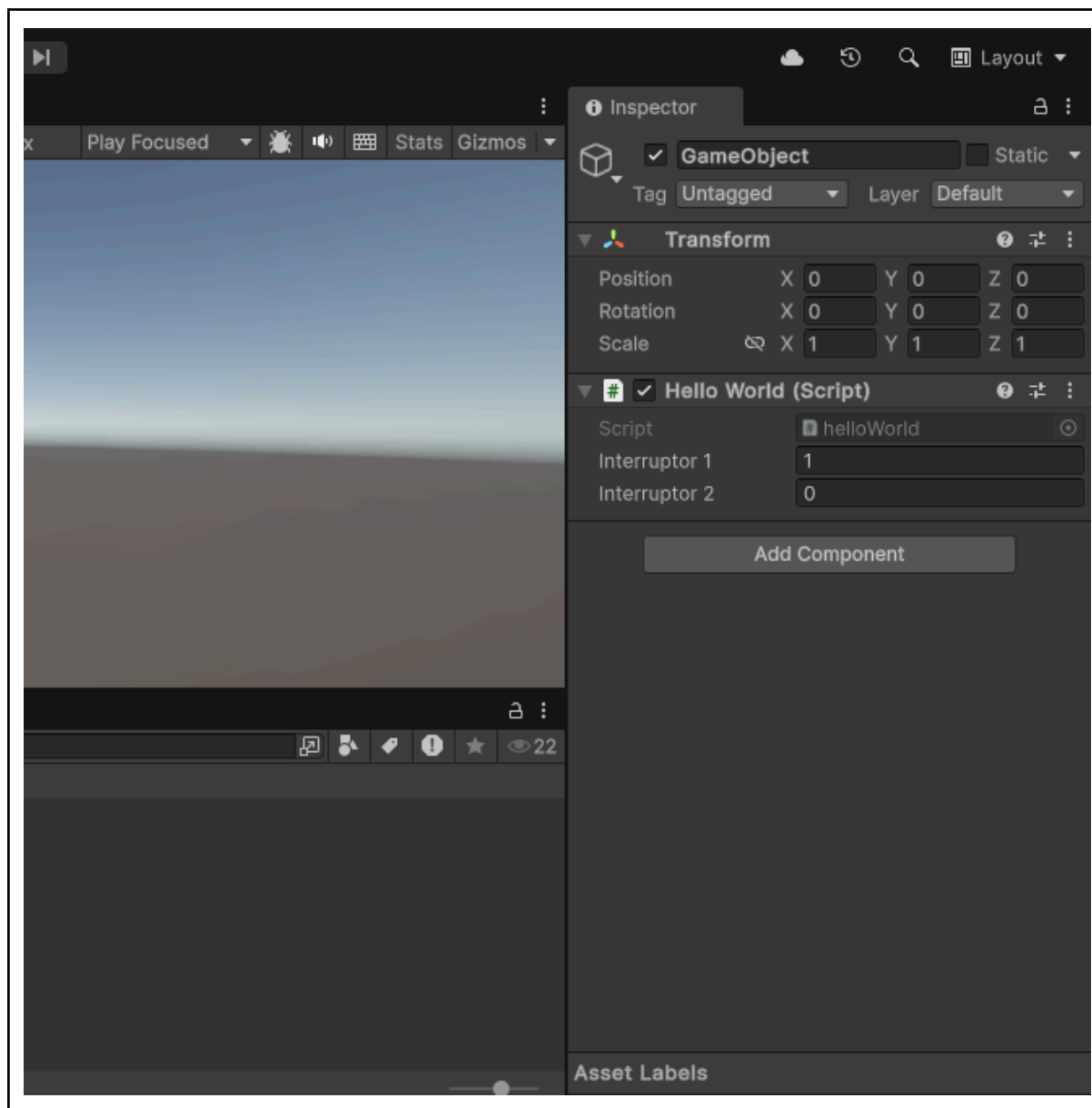
Haga clic en el botón “Play” que está en el centro de la parte superior de la pantalla principal para ejecutar su código.

Abra la consola (para ello haga clic en “Window”; después haga clic en “General” y finalmente haga clic en “Console”).

Haga clic en el objeto “GameObject” en la ventana “Hierarchy”.



A continuación compruebe que los valores creados en el código como variables públicas **interruptor1** e **interruptor2** aparecen en el “Inspector”.



Reto 1b:

Cree un “textbox” (caja de texto) con fondo negro y el siguiente texto en color blanco: “Hola mundo Unity!”. Cambie también el color del fondo del “canvas” (lienzo) a negro.

1. Cambie el color del fondo del “canvas” (lienzo) a negro (Set Camera Background to black):

- Seleccione “**Main Camera**” en “**Hierarchy**”.
- En el Inspector, localice “**Camera component**”.
- Cambie “(Environment) **Background color property**” a negro (RGB 0, 0, 0).

2. Cree “**Hello, World!**” Text (UI Canvas):

- Haga clic con el botón derecho del ratón en “**Hierarchy; UI; Text - TextMeshPro**” (debe importar “TMP Essentials”).
- Seleccione el objeto **Text (TMP)** que acaba de crear.
- A continuación, en el Inspector:
 - Escriba “Hello, World!” en el campo **Text Input**.
 - Cambie el **Color** a “**white**” y cambie “**Font Size**” si lo desea.
 - Cambie “**Alignment**” para centrar el texto de forma horizontal y vertical.

Reto 2:

Realice las siguientes modificaciones en el archivo "HelloWorld.cs" para cambiar el color de fondo de la escena a negro.

```
void Start()
{
    Debug.Log("Hello, World!");

    // recuerde que Main Camera debe estar etiquetado (tagged) como "MainCamera"

    Camera mainCamera = Camera.main;

    mainCamera.backgroundColor = Color.black;
}
```

Ejemplo C47. Impresión de un mensaje con Unity y C#.

Objetivo: Escribir el código necesario para comprobar que Unity 6 puede imprimir mensajes a través de Microsoft Visual Studio Code con código escrito en C#.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “3D (Core)”.

Escriba el siguiente nombre de proyecto: “U2-ProjectBotonClic”.

Paso 2.

En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “UI” y después elija “Canvas” (esto creará un entorno o lienzo vacío para los elementos (UI)).

Paso 3.

En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho en “Canvas”.

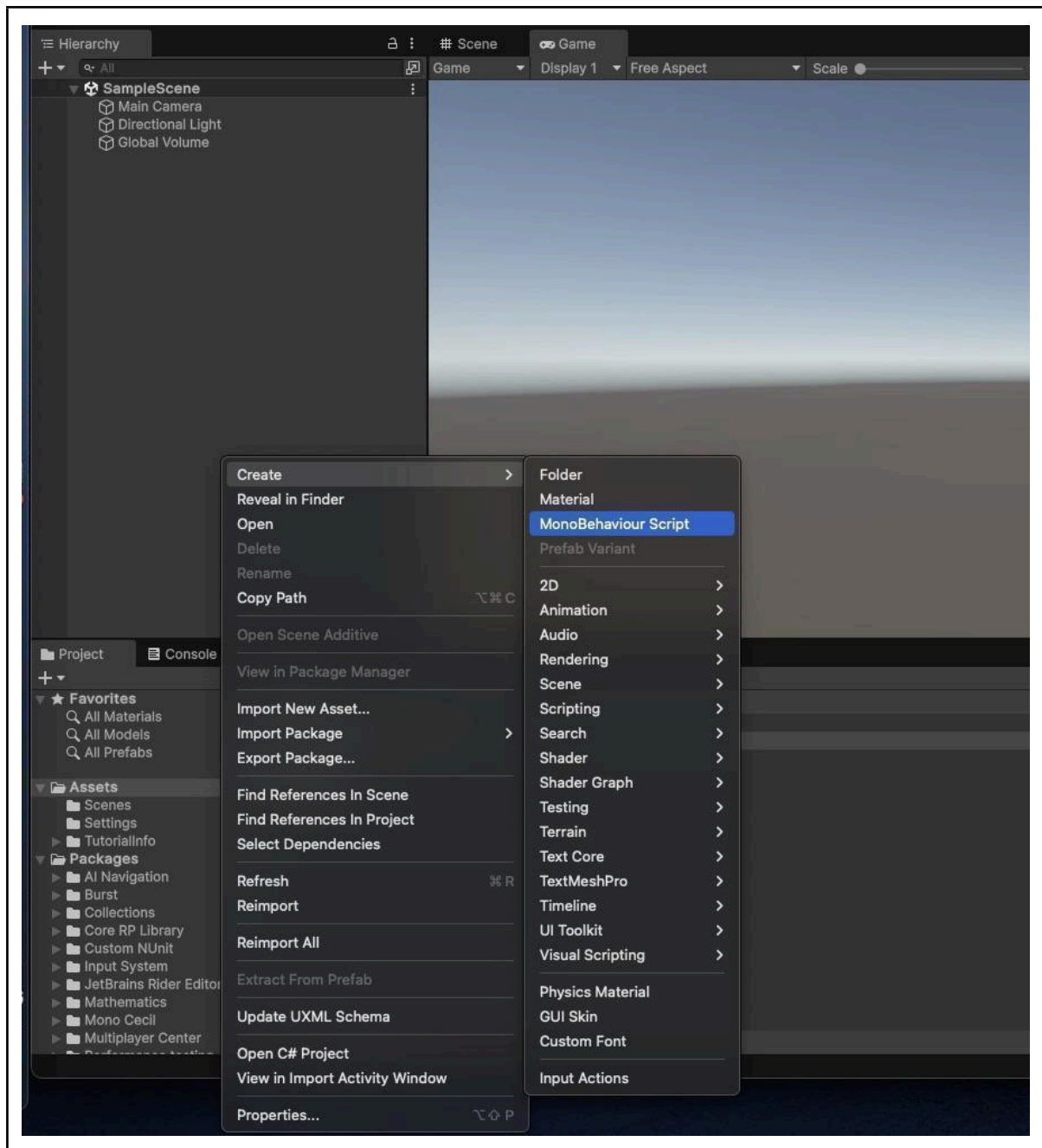
Seleccione “UI” y después seleccione “Button”.

Un botón aparecerá justo debajo de “Canvas”.

Puede mover el “Button” (botón) en la “Scene View” (vista escena).

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “BotonClicHandler”.



Paso 4.

Haga doble clic en el archivo “.cs” que acaba de renombrar.

Se abrirá Microsoft Visual Studio con el archivo “.cs”. Borre el código que aparece por defecto y copie el que se muestra en esta imagen:

```
using UnityEngine;

public class BotonClicHandler: MonoBehaviour
{
    public void OnButtonClick()
    {
        Debug.Log("Hola, ha hecho clic en el botón!");
    }
}
```

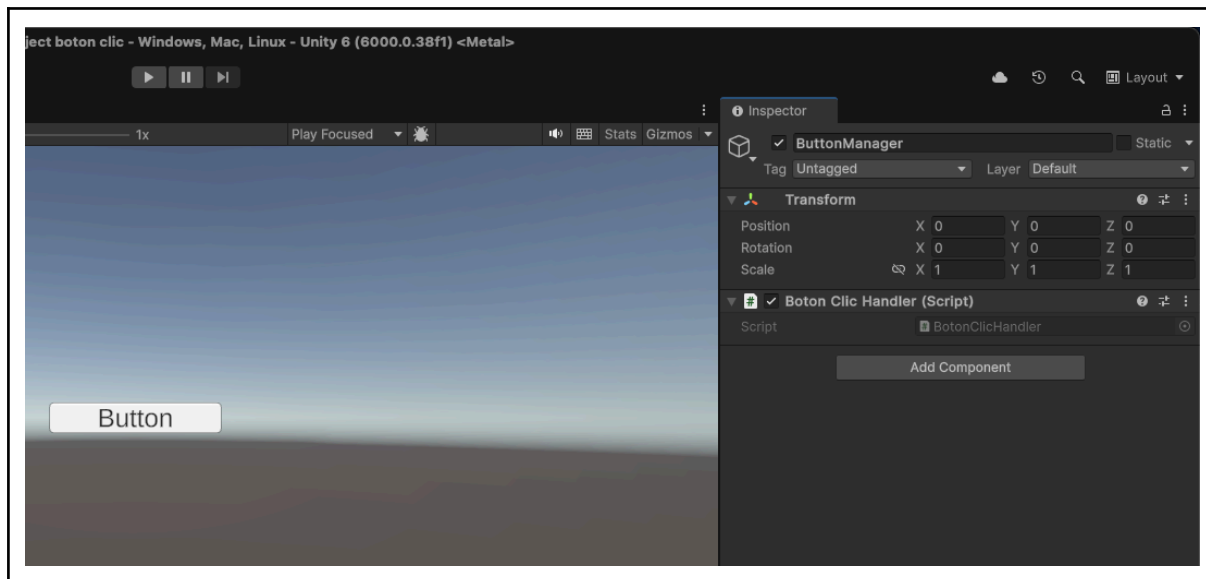
Nota:

No olvide grabar el archivo que acaba de modificar y cierre Microsoft Visual Studio.

Paso 5.

Vuelva a Unity y siga estos pasos:

- En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “Create Empty” (crear vacío).
- Renombre el objeto como “ButtonManager”.
- Seleccione el objeto creado y haga clic en “Add Component” (añadir componente -en el Inspector-).
- Busque y seleccione “BotonClicHandler” (si no aparece en la lista, teclee el nombre para que aparezca).



Paso 6.

Seleccione “Button” (botón) en “Hierarchy”.

En el Inspector, busque el componente “Button”.

Haga clic en “+” justo debajo de “On click()”.

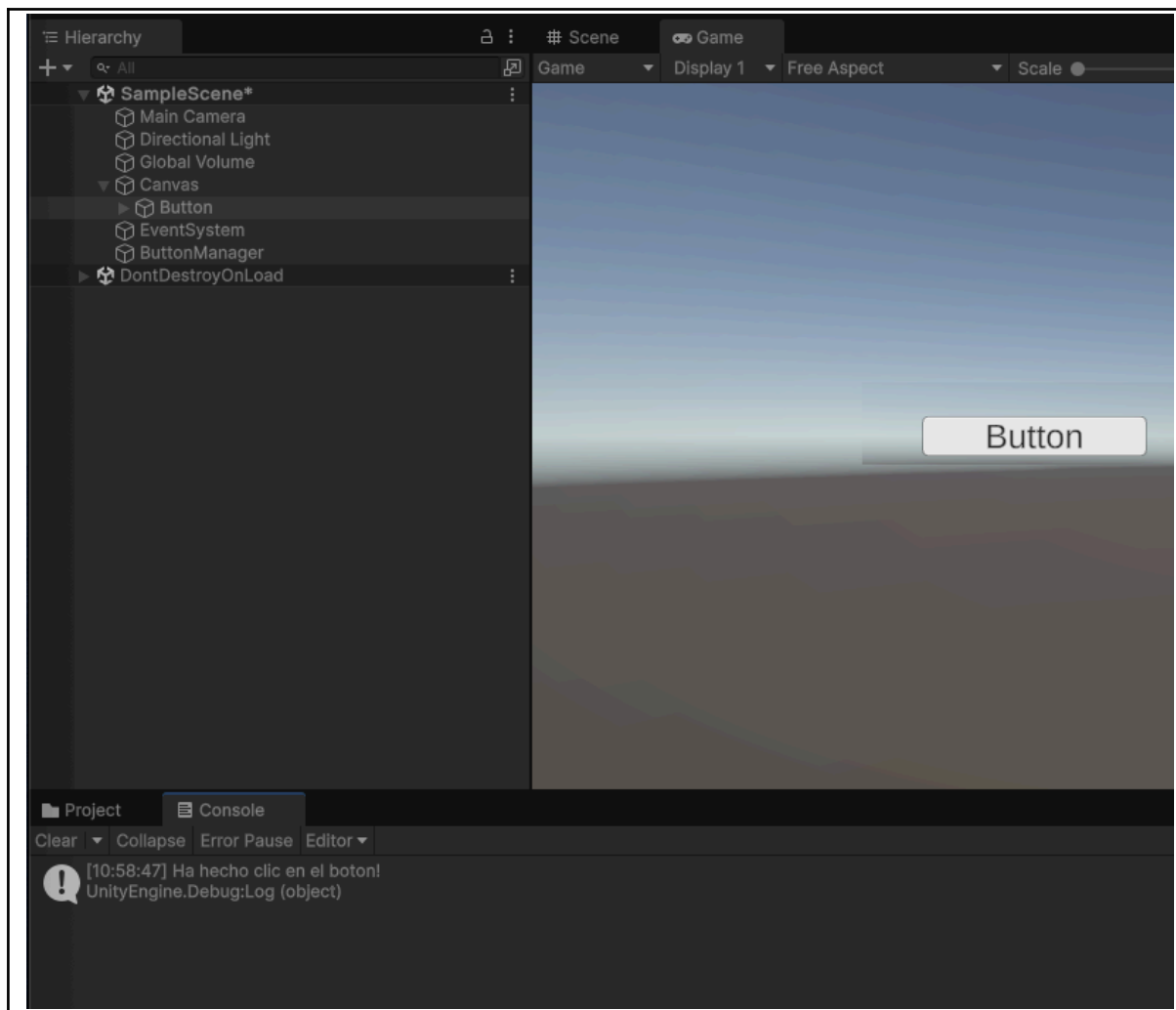
Arrastre el “ButtonManager” GameObject al campo vacío.

Elija en la lista desplegable (justo a la derecha de “RunTime Only”) “BotonClicHandler” y después elija “OnButtonClick()”.

Paso 7.

Haga clic en el botón “Play” que está en el centro de la parte superior de la pantalla principal.

Abra la consola (para ello haga clic en “Window”; después haga clic en “General” y finalmente haga clic en “Console”) para comprobar los mensajes enviados al usuario.



Observe que en la parte inferior de la imagen podemos leer la frase “Ha hecho clic en el botón!”. Esto indica que nuestro programa escrito en C# se comunica correctamente con Unity 6 y puede manejar los eventos en la escena creada.

Ejemplo C48. Impresión de un mensaje con respuesta con Unity y C#.

Objetivo: Escribir el código necesario para comprobar que Unity 6 puede imprimir mensajes a través de Microsoft Visual Studio Code con código escrito en C#.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “3D (Core)”.

Escriba el siguiente nombre del proyecto: “U3-UI Text Buttons Project”.

Paso 2.

En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “UI” y después elija “Canvas” (esto creará un entorno o lienzo vacío para los elementos (UI)).

Paso 3a.

En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho en “Canvas”.

Seleccione “UI” y después seleccione “Button”.

Un botón aparecerá justo debajo de “Canvas”.

Puede mover el “Button” (botón) en la “Scene View” (vista escena).

Paso 3b.

Ahora añadiremos un objeto Text (UI) para ver el mensaje:

En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho en “Canvas”.

Seleccione “UI” y después seleccione “Text - TextMeshPro”.

Puede mover el “Button” (botón) en la “Scene View” (vista escena).

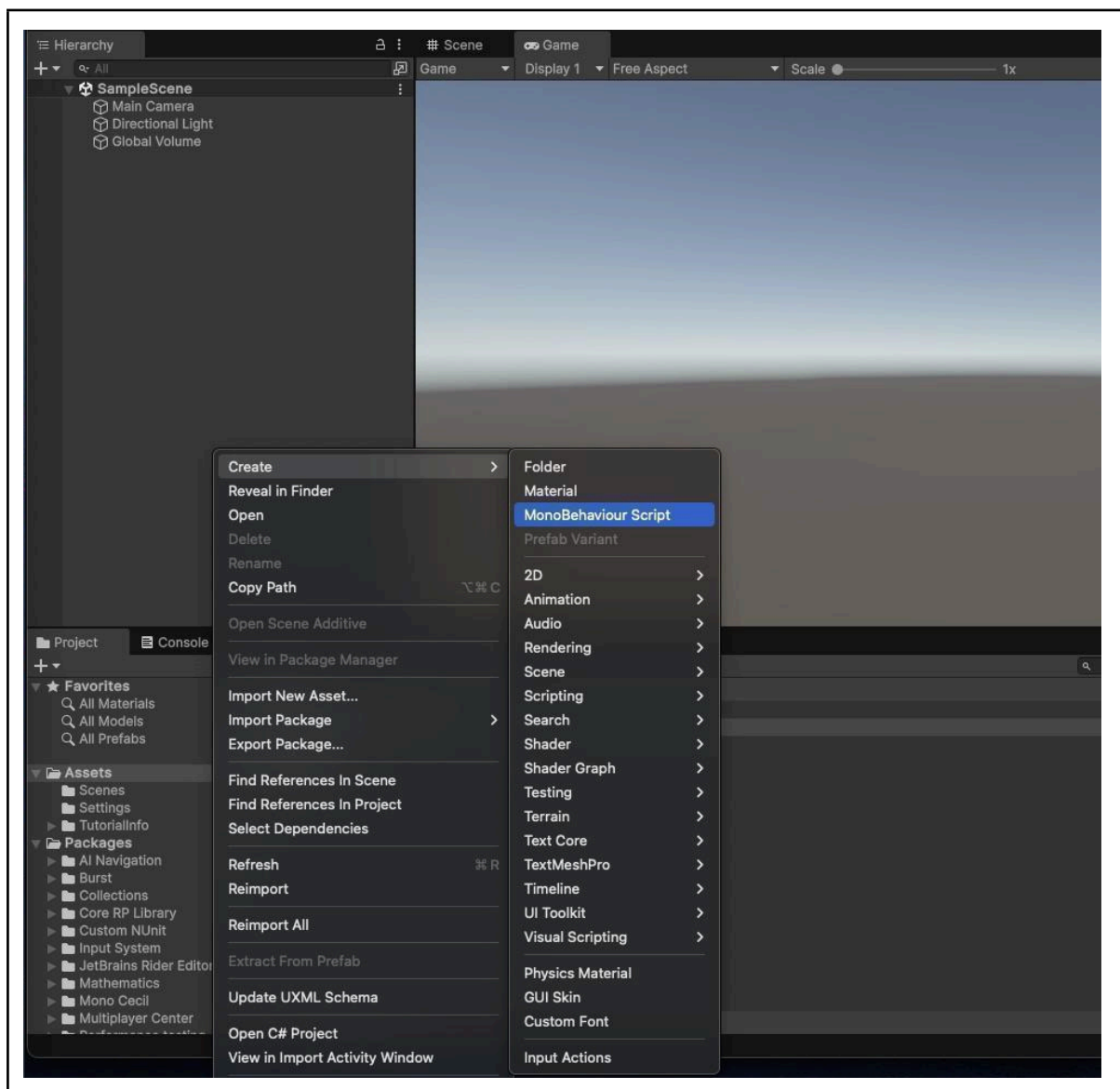
Borre el texto que aparece por defecto.

Cambie el estilo o el tamaño del texto si lo desea.

Paso 3c.

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “ButtonMessageDisplay”.



Paso 4.

Haga doble clic en el archivo “.cs” que acaba de renombrar en el paso anterior.

Se abrirá Microsoft Visual Studio con el archivo “.cs”. Borre el código que aparece por defecto y copie el que se muestra en esta imagen:

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class ButtonMessageDisplay: MonoBehaviour
{
    public TextMeshProUGUI messageText;

    public void OnButtonClick()
    {
        Debug.Log("OK, has presionado el botón!");
    }
}
```

Nota:

No olvide grabar el archivo que acaba de modificar y cierre Microsoft Visual Studio.

Paso 5.

Vuelva a Unity y siga estos pasos:

- a. En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “Create Empty” (crear vacío).
- b. Renombre el objeto como “UIManager”.
- c. Añada (add component) el script “ButtonMessageDisplay” al “UIManager”.
- d. Arrastre el objeto “Text UI” en el campo del script llamado “Message Text” en el Inspector.

Paso 6.

Seleccione “Button” (botón) en “Hierarchy”.

En el Inspector, busque el componente “Button (script)”.

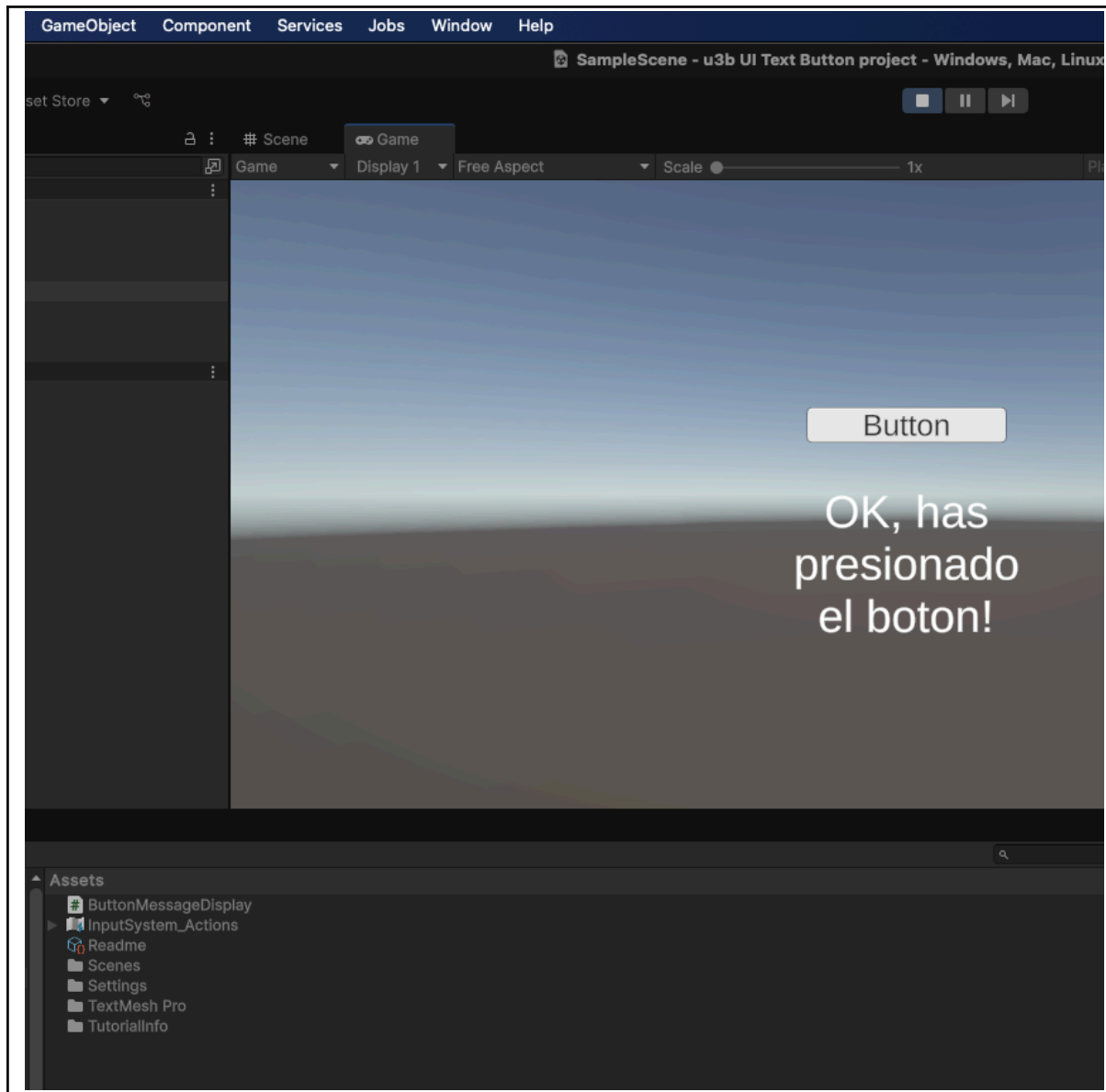
Haga clic en “+” justo debajo de “On click()”.

Arrastre el “UIManager” al campo vacío del objeto (object).

Elija en la lista desplegable (justo a la derecha de “RunTime Only”) “ButtonMessageDisplay()” y después elija “OnButtonClick()”.

Reto:

Antes de ejecutar la aplicación que debe visualizar el mensaje “OK, has presionado el botón”, falta un paso importante. ¿Cuál es?



Ejemplo C49. Creación de dos botones e impresión de un mensaje con respuesta con Unity y C#.

Objetivo: Escribir el código necesario para comprobar que Unity 6 puede imprimir mensajes cuando los botones creados en el “canvas” son pulsados por el usuario.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “2D (Core)”.

Escriba el siguiente nombre del proyecto: “U4-UI Text Buttons Project 2D”.

Paso 2. Comprobaciones y creación de objetos.

- **TextMeshPro:**

- Asegúrese que “TextMeshPro essential resources” se cargan correctamente (Window > TextMeshPro > Import TMP Essential Resources).

- **UI Canvas:**

- Un “Canvas GameObject” es requerido en la escena (Haga clic con el botón derecho del ratón en “Hierarchy” > UI > Canvas). Este objeto es el contenedor para los elementos de la interfaz del usuario (UI).

- **UI Buttons:**

- Cree dos “Button GameObjects” como hijos del Canvas (Haga clic con el botón derecho del ratón en Canvas > UI > Button - TextMeshPro).

- **TextMeshPro Text:**

- Cree un “TextMeshPro Text (UGUI) GameObject” como hijo del Canvas (Haga clic con el botón derecho del ratón en Canvas > UI > Text - TextMeshPro). Este es el objeto que imprimirá los mensajes al pulsar el botón 1 o el botón 2.

Paso 3. Creación del código escrito en C#.

A continuación crearemos el código necesario para controlar las interacciones entre los botones y el texto que indica qué botón se ha pulsado.

- El código necesita referencias a los dos botones y el objeto “TextMeshPro”.
- También necesitamos crear funciones que serán llamadas cada vez que un botón es pulsado. Las funciones actualizarán la propiedad “Text” del objeto “TextMeshPro”.

```
using UnityEngine; using UnityEngine.UI;          /// UI components (Button)
using TMPro;                                   /// TMPro components

public class ButtonMessageDisplay : MonoBehaviour
{
    // Public variables:          (estas variables son visibles en el inspector)

    public Button button1; public Button button2;
    public TMPro.TextMeshProUGUI messageText; // Este componente es el TMP text object

    /// Deberá probar TMProTextMeshProUGUI o TMProTextMeshProGUI dependiendo de la versión de Unity;

    void Start()
    {
        // This runs once when the script starts

        if (button1 != null)      // listener to button1's click event
        {
            button1.onClick.AddListener(OnButton1Pressed);
        }

        if (button2 != null)      // listener to button2's click event
            button2.onClick.AddListener(OnButton2Pressed);

        if (messageText == null) // check if text object is assigned
            Debug.LogError("Message Text (TextMeshProUGUI) is not assigned!");
    }

    void OnButton1Pressed()      // Function called when Button 1 is clicked
    {
        if (messageText != null)
            messageText.text = "button 1 was pressed"; // Update the text
    }

    void OnButton2Pressed()      // Function called when Button 2 is clicked
    {
        if (messageText != null)
            messageText.text = "button 2 was pressed"; // Update the text
    }
}

/// Grabe este archivo con el nombre ButtonMessageDisplay.cs
```

Paso 4. Composición final.

A continuación siga los siguientes pasos para unir todos los componentes creados anteriormente:

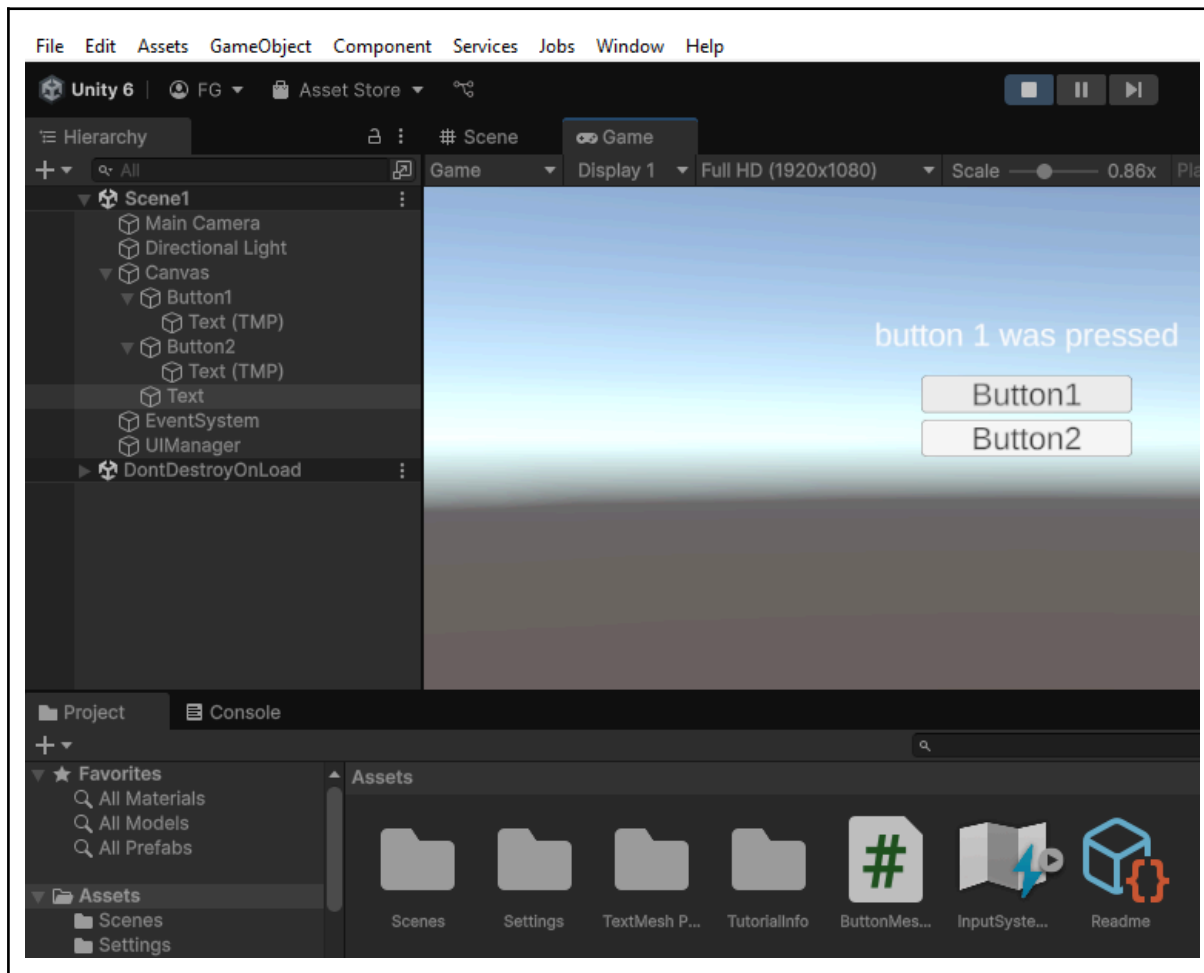
1. Añada un “empty GameObject” a su escena y llámelo "UIManager".
2. Arrastre el archivo **ButtonMessageDisplay.cs** dentro del "UIManager GameObject” en “Hierarchy” o en el Inspector.
3. Asigne Referencias en el Inspector:
 - Seleccione el "UIManager GameObject”.
 - En el Inspector, arrastre el “Button 1 GameObject” desde “Hierarchy” dentro del campo "Button 1".
 - En el Inspector, arrastre el “Button 2 GameObject” desde “Hierarchy” dentro del campo "Button 2".
 - Arrastre el “TextMeshProUGUI GameObject” dentro del campo "Message Text".

Resumen:

- El script utiliza variables públicas para exponer los objetos “Button1”, “Button2” y TextMeshPro al Editor de Unity.
- En el método “Start()”, agregamos mediante código "listeners" al evento “onClick” de cada botón.
- Cuando se hace clic en un botón en el juego, su “listener” correspondiente activa la función especificada (OnButton1Pressed u OnButton2Pressed). Estas funciones luego acceden al objeto “messageText” asignado y cambian su propiedad de texto a la cadena deseada.

Haga clic en “File” y grabe la escena para que se compile el proyecto.

A continuación, haga clic en el botón “Play” que está en el centro de la parte superior de la pantalla principal para ver el resultado.



Reto 1: Cambie el texto que aparece en los botones 1 y 2.

Puede hacerlo de dos formas:

Método 1:

Hacer los cambios manualmente en el Inspector:

- En la ventana “**Hierarchy**”, haga click en la flecha que apunta hacia abajo (la cual está junto al botón del cual quiere cambiar alguna propiedad).
- Seleccione el componente “**Text**” dentro del botón.
- En el “**Inspector**”, edite el campo “**Text**” para incluir el texto que quiera que aparezca en el botón.

Método 2:

Hacer los cambios con código (C#):

- Haga referencia al componente “**button's text**” y utilice el siguiente bloque de código para modificarlo:

```
using UnityEngine;
using UnityEngine.UI;

public class ChangeButtonText : MonoBehaviour
{
    public Button myButton;
    public Text buttonText;

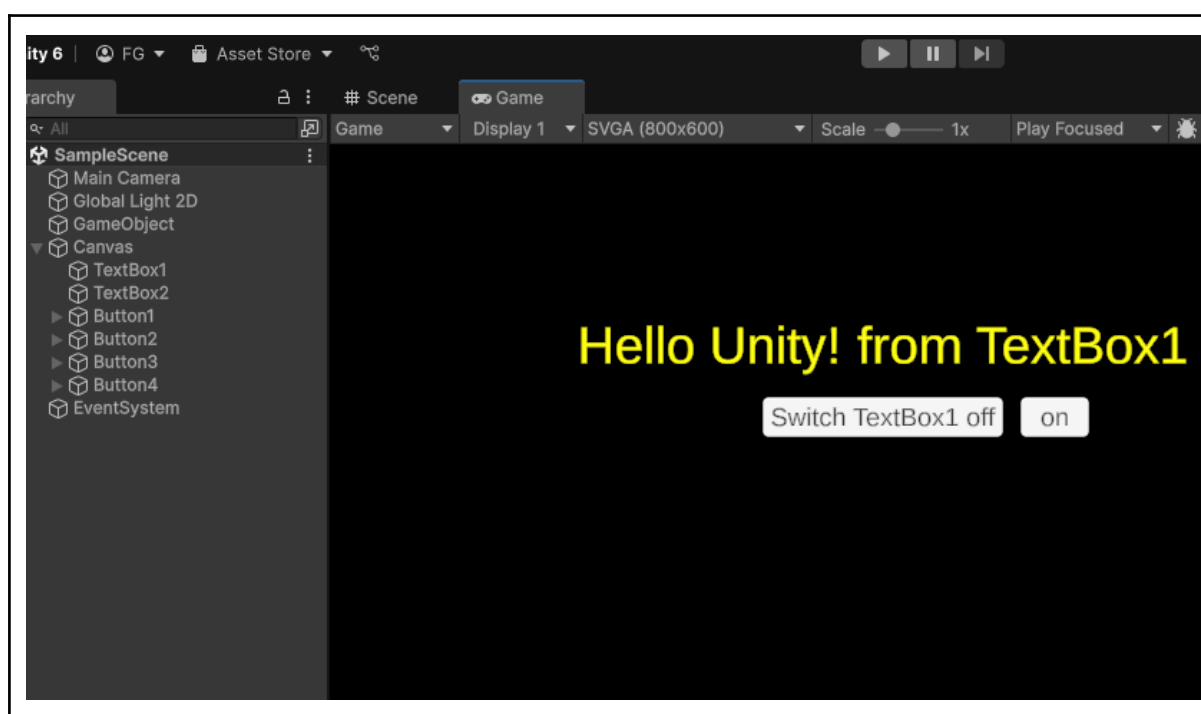
    void Start()
    {
        buttonText = myButton.GetComponentInChildren<Text>();
        buttonText.text = "El Nuevo Texto está aquí";
    }
}

/// Recuerde que:
/// Si está usando TextMeshPro,
/// Deberá cambiar Text por TextMeshProUGUI y añadir using TMPro;
```


Reto 2

Cree una caja de texto llamada “TextBox1” (debe ser un elemento **UI Text** o un elemento **TextMeshPro Text**) que se pueda hacer invisible cuando haga clic en un botón llamado “Button1”.

El botón tiene un texto en el que se puede leer: “Switch TextBox1 off”. Al pulsarlo desaparece de la pantalla el texto “Hello Unity! from TextBox1”.



Continúa en la página siguiente. . .

Puede hacerlo de la siguiente forma:

Borre del canvas los siguientes objetos: Button1, Button2 y UIManager

1. En la ventana “**Hierarchy**”, cree un objeto nuevo llamado “**Button1**”.
Seleccione “Button1”: En la ventana “**Hierarchy**”, donde se encuentra ahora mismo, encuentre y seleccione "Button1" ().
2. En la ventana **Inspector**, desplácese hacia abajo hasta que encuentre "Button component".
3. **Ahora debemos añadir un evento “OnClick”:**
 - En la sección “**OnClick ()**”. Haga clic en el símbolo +.
4. **Asigne el “TextBox1 GameObject”:**
 - Un nuevo hueco aparecerá. Arrastre el "Textbox1 GameObject" desde la ventana **Hierarchy** dentro del campo "**None (Object)**" en el evento OnClick.
5. **Seleccione la función necesaria:**
 - Haga clic en el menú desplegable "**No Function**".
 - Busque y seleccione **GameObject --> SetActive (bool)**.
6. **Enviar el parámetro False a la función SetActive():**
 - Aparecerá un “checkbox” debajo de la función. Debe estar desmarcado (**Unchecked**). Cuando está desmarcado (vacío) estamos enviando el parámetro **False** a la función SetActive(), lo que hace que el GameObject **TextBox1** con nuestro mensaje se haga invisible.

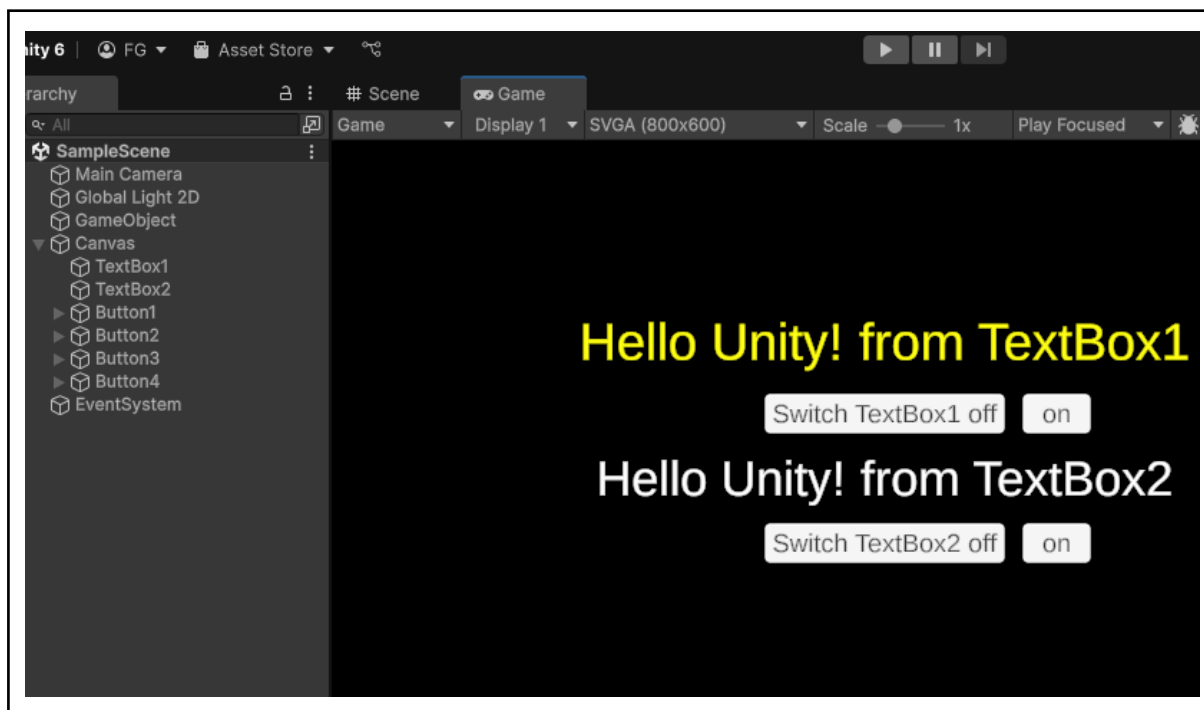
Ahora, cuando ejecutamos la app y hacemos clic en "Button1", desaparece el objeto "Textbox1". Repita los pasos del 1 al 6 para cuantos botones necesite en su juego.

Reto 3 (Nombre de este mini proyecto: “U1b Hello World Project_”.

En la siguiente pantalla puede observar algunos cambios:

Implemente los botones necesarios para controlar un segundo mensaje en un nuevo objeto llamado Textbox2.

A continuación, incluya botones “on” para volver a hacer visibles los mensajes de los objetos Textbox1 y Textbox2 como se ve en la imagen:



Grabe los cambios realizados en el proyecto “U1b Hello World Project_” para futuras referencias.

Ejemplo C50a. Preparación de la escena (intro) de una novela gráfica.

Objetivo: Escribir el código necesario en C# para visualizar fotografías y texto (introducción de la novela gráfica) en la pantalla utilizando botones.

Paso 1. Crear el Canvas en un nuevo proyecto 2D.

- Abra la aplicación Unity Hub y haga clic en “New Project”.
- Elija la plantilla “**2D (Core)**” y escriba el siguiente nombre: “**U1c_My_slide1**”.
- Copie el archivo `slide1.png` file en la carpeta “Assets”.
- Seleccione en el Inspector el archivo (**slide1.png**) que contiene la primera página de la introducción de su novela gráfica, cambie “Texture Type” a Sprite (2D and UI) y haga clic en “Apply”.
Recuerde que el Canvas debe contener todos los elementos UI (El Canvas también contiene el sistema de gestión de eventos).
- A continuación, en Hierarchy, haga clic en GameObject > UI > Canvas.

Paso 2. Crear la imagen (UI):

- Haga clic con el botón derecho del ratón en la ventana “Hierarchy”.
- Seleccione UI > Image. Esto creará un elemento **Image GameObject** (como hijo del Canvas). Seleccione este elemento **Image GameObject** en la ventana Hierarchy:
 - ◆ En el Inspector, busque **Source Image**. Arrastre el archivo `slide1.png` (que ahora es también un Sprite) desde la ventana Project hasta el campo **Source Image**.
 - ◆ Cambie la posición y tamaño de la imagen si lo desea cambiando los parámetros **Rect Transform** (position, width, height). También puede hacer clic en **Set Native Size** (para utilizar el tamaño original del archivo PNG).
 - ◆ Puede hacer invisible la imagen al iniciar su aplicación: En el Inspector, deje sin marcar la casilla que está junto al nombre del elemento Image GameObject (en la parte superior del Inspector).

Paso 3. Crear el botón (UI):

- Haga clic con el botón derecho del ratón en la ventana “Hierarchy”.
- Seleccione UI > Button - TextMeshPro .

También podría haber elegido UI > Button. Si eligió TextMeshPro, Unity puede necesitar que importe el paquete TMP Essentials.

- Seleccione el nuevo “Button GameObject” en “Hierarchy”.
- Si lo desea, puede cambiar el texto que está sobre el botón (expanda el botón en Hierarchy y seleccione Text (TMP) o Text child. En el Inspector, cambie Text field a “Show Slide 1” o “Ver Slide 1”.
- Cambie las coordenadas o el tamaño del botón si lo desea modificando el parámetro “Rect Transform”.

Paso 4. Crear un “GameObject” para el “Script” (que crearemos más adelante):

- Haga clic con el botón derecho del ratón en la ventana “Hierarchy”. Seleccione “Create Empty”.
- Seleccione el “GameObject” que acaba de crear y llámelo “**UIController**”. Este archivo contendrá más adelante el código escrito en C#.

Paso 5. Crear el Script en C#:

- Haga clic con el botón derecho del ratón en la ventana “Hierarchy”. Seleccione “Create Empty”.
- En la ventana “Project”, haga clic con el botón derecho del ratón en un área vacía (por ejemplo, en la carpeta “Assets” o en una subcarpeta “Scripts”).
- Renombre el nuevo Script como “ImageDisplayController” (debe tener el mismo nombre que la clase creada en el código).
- Abra el “script” que acaba de crear y reemplace el contenido con el ejemplo facilitado más adelante llamado “ImageDisplayController.cs”. No olvide grabar el archivo antes de cerrar Visual Studio Code.

Paso 6. Adjuntar el script creado al GameObject **UIController** (creado anteriormente):

- Haga clic con el botón derecho del ratón en la ventana “Hierarchy”. Seleccione “Create Empty”.
- Arrastre el archivo ImageDisplayController.cs desde la ventana “Project” hasta el GameObject **UIController** (que está en el Inspector).
- Nota: Puede encontrar el script ImageDisplayController.cs completo después del paso número 8.

Paso 7. Asignar la imagen a el script en el Inspector:

- Seleccione el GameObject **UIController** en la ventana “Hierarchy” y observe sus propiedades en la ventana del Inspector:
 - Deberá ver el componente Image Display Controller (Script). Debe contener un campo público llamado **Slide Image**.
- Arrastre el **Image GameObject** desde la ventana “Hierarchy” dentro del campo público **Slide Image** en el Inspector.
- Nota: El Image GameObject fue creado anteriormente cuando le asignó el archivo slide1.png).

Paso 8. Conectar el botón al **método** del script:

- Seleccione el botón GameObject en la ventana “Hierarchy”.
- En el Inspector, busque Button (Script) Component.
- Encuentre la lista On Click(). Haga clic en el símbolo + para añadir un evento nuevo.
- Arrastre el GameObject **UIController** desde la ventana “Hierarchy” dentro del (objeto) campo “None” que acaba de aparecer.
- Haga clic en la lista “No Function”.
- Haga clic en **ImageDisplayController** y seleccione la función **ShowSlide1()**.

```

///
/// Este script es parte del EjemploC50a y se llama:
/// ImageDisplayController.cs

using UnityEngine;
using UnityEngine.UI;    // Librería necesaria for UI elements Image and Button

public class ImageDisplayController : MonoBehaviour

{
    // Public variable to assign Image component in the Inspector
    public Image slideImage;

    void Start()
    {
        // Compruebe que image is [hidden] at the start
        // This is good, aunque puede hacerlo en el Inspector
        if (slideImage != null)
        {
            slideImage.gameObject.SetActive(false);
        }
    }

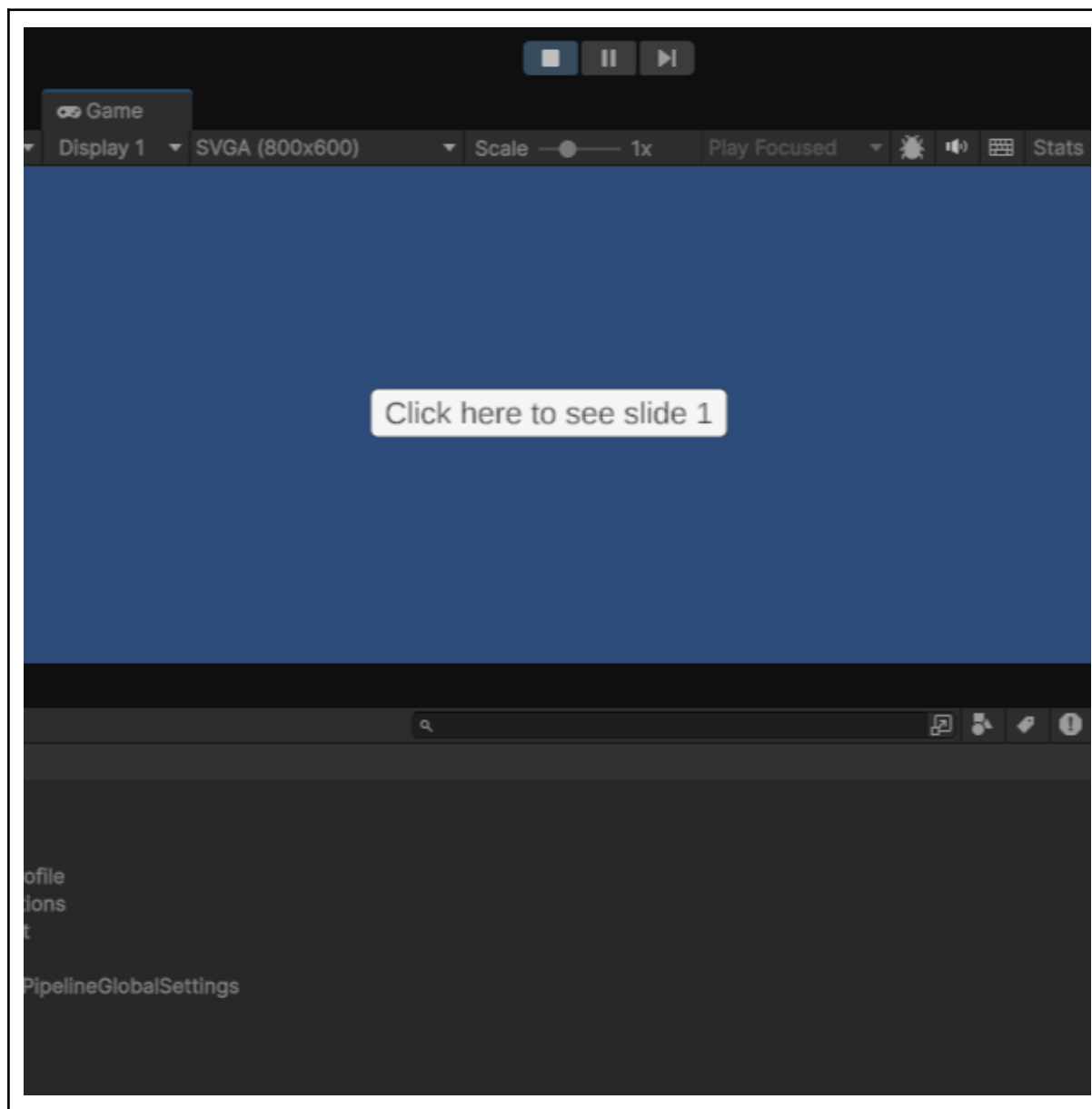
    // This method es llamado cuando Button1 is pressed
    public void ShowSlide1()
    {
        // Comprueba if the Image component is assigned

        if (slideImage != null)
        {
            // Activar GameObject que tiene the Image component

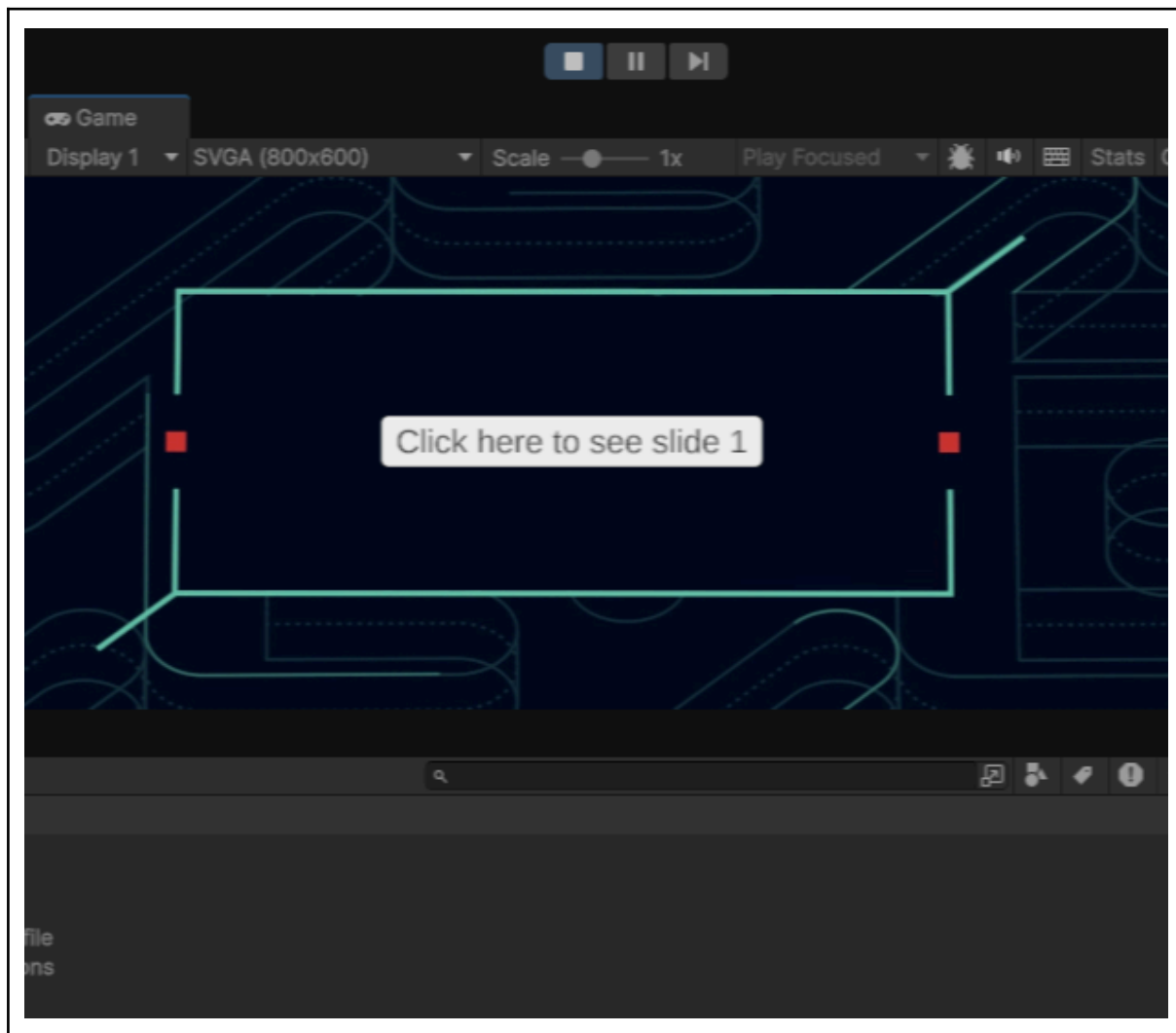
            slideImage.gameObject.SetActive(true);
            Debug.Log("slidel.png ahora es visible.");
        }
        else
        {
            Debug.LogError("Slide Image no ha sido asignada en el Inspector!");
            //Sol.: Asigna 'slideImage' var in 'Image Display Controller' script component
        }
    }
}

```


Al ejecutar la aplicación del **Ejemplo C50a**, aparecerá el botón “Click here to see Slide 1”:



A continuación, una vez el usuario pulse el botón, aparecerá en la pantalla la imagen slide1.png.



Reto:

Cree una historia gráfica con 3 imágenes y 3 botones. La primera imagen contendrá la primera página de la historia del primer computador IBM PC compatible, la segunda imagen contendrá la segunda página de la historia y así, sucesivamente. Cada botón llamará a una función para visualizar cada página de la historia gráfica. En la última imagen (o escena) pregunta cuál es el resultado de sumar $120+5$ al usuario. Si el usuario responde correctamente, visualiza en pantalla el siguiente mensaje: “¡Felicidades!”.

Ejemplo C51a. Lectura e impresión de un fichero de texto.

Objetivo: Escribir el código necesario en C# para comprobar que Unity 6 puede imprimir mensajes, nombres o números contenidos en un fichero de texto con extensión .txt.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “2D (Core)”.

Escriba el siguiente nombre: “**U1bxt Hello World Project_txt**”.

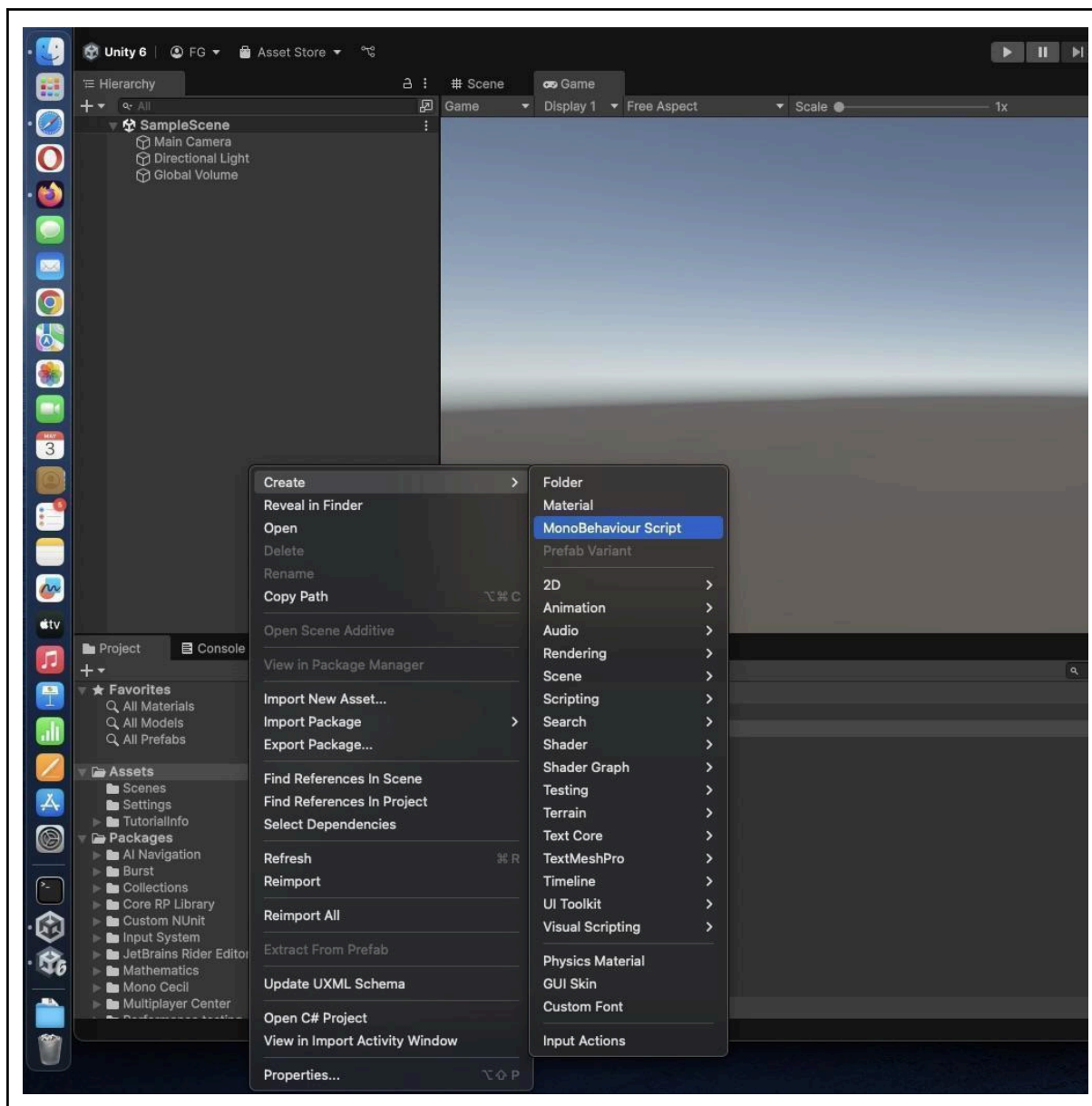
Paso 2.

a.

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

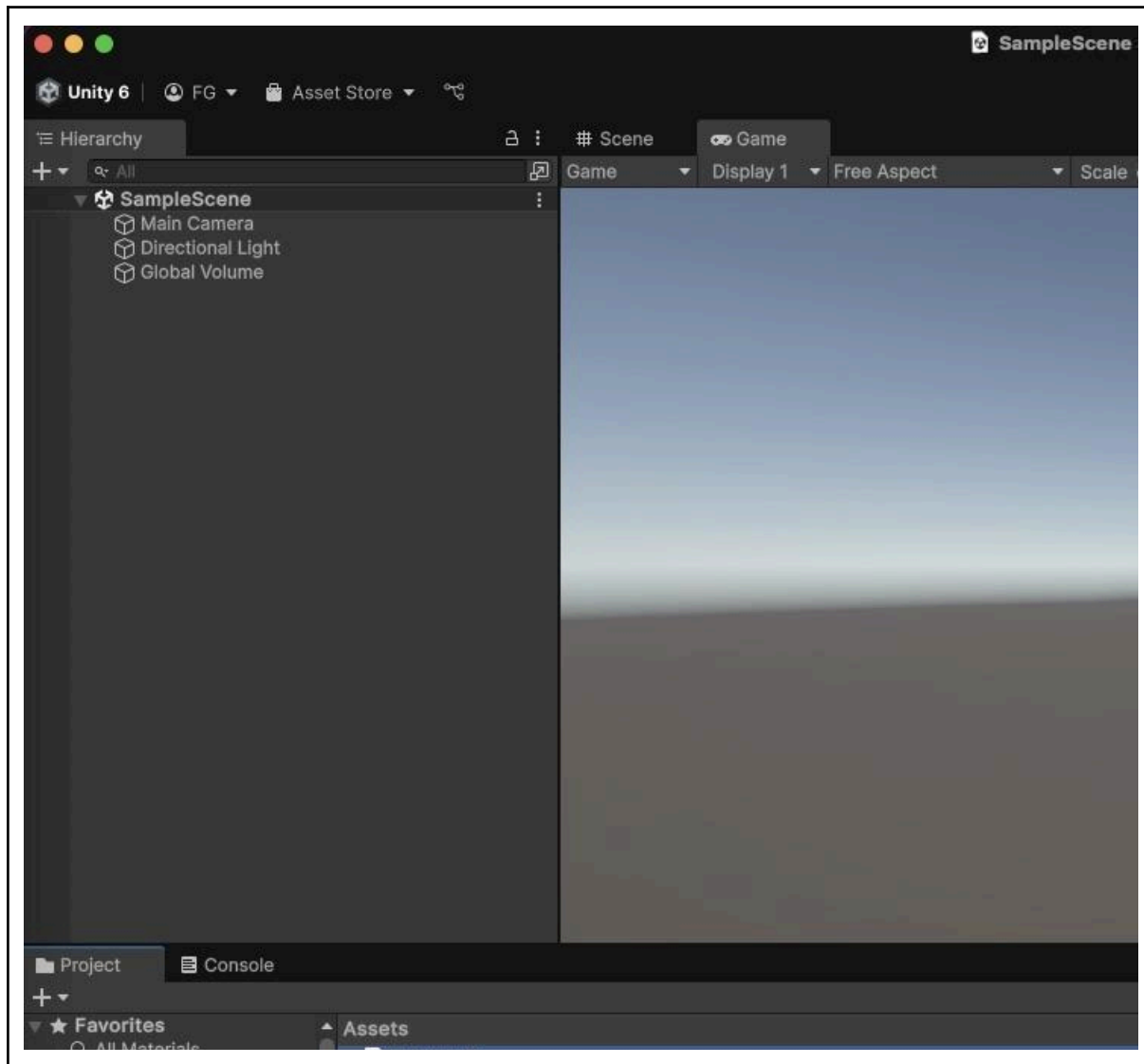
b.

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “FileManager”.



Paso 3.

Busque y seleccione en “Assets” el script “FileManager”. Haga doble clic en el script “FileManager”.



Paso 4.

Se abrirá Microsoft Visual Studio con el archivo “FileManager.cs”. Borre el código que aparece por defecto y copie el que se muestra en esta imagen:

```
using UnityEngine; using UnityEngine.UI; using System.IO; using System.IOEnumeration;

public class FileManager : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    string[] namesArray;
    string myFilePath;
    string fileName;

    void Start()
    {
        Debug.Log("Hello! Vamos a leer un fichero txt!");
        Camera mainCamera = Camera.main; mainCamera.backgroundColor = Color.black;
        /// ojo! este es path de entorno de desarrollo
        fileName = "names.txt";
        myFilePath = Application.dataPath + "/" + fileName;
        /// llama a la funcion ReadFromFile para leer la primera y segunda question
        string q1 = ReadFromFile(0); print(q1);
        string q2 = ReadFromFile(1); print(q2);
    }

    public string ReadFromFile(int superIndex )    /// superIndex es el num de la pregunta q se quiere leer
    {
        /// Each name (which is a line in the txt file) will be put into the array.
        /// Usaremos el method ReadAllLines
        namesArray = File.ReadAllLines(myFilePath);
        string question = namesArray[superIndex];
        /// Debug.Log(question);
        return (question);
    }
}
```

Notas:

Recuerde que el nombre de la clase creada, FileManager, debe coincidir siempre con el nombre del archivo que contiene el script. En nuestro ejemplo el archivo que contiene el código se llama FileManager.cs. Guarde el archivo que acaba de modificar y cierre Microsoft Visual Studio.

No olvide crear un archivo de texto (con cualquier editor de textos) llamado **names.txt** con dos líneas. Ponga una frase en cada línea y grabe el archivo en la carpeta “Assets” de su proyecto.

Paso 5.

Vuelva a Unity y siga estos pasos:

- En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “Create Empty” (crear vacío).
- Seleccione “GameObject”.
- En el “Inspector”, haga clic en “Add Component” (añadir componente).
- Busque y seleccione “FileManager” (si no aparece en la lista, teclee el nombre “FileManager” para que aparezca).

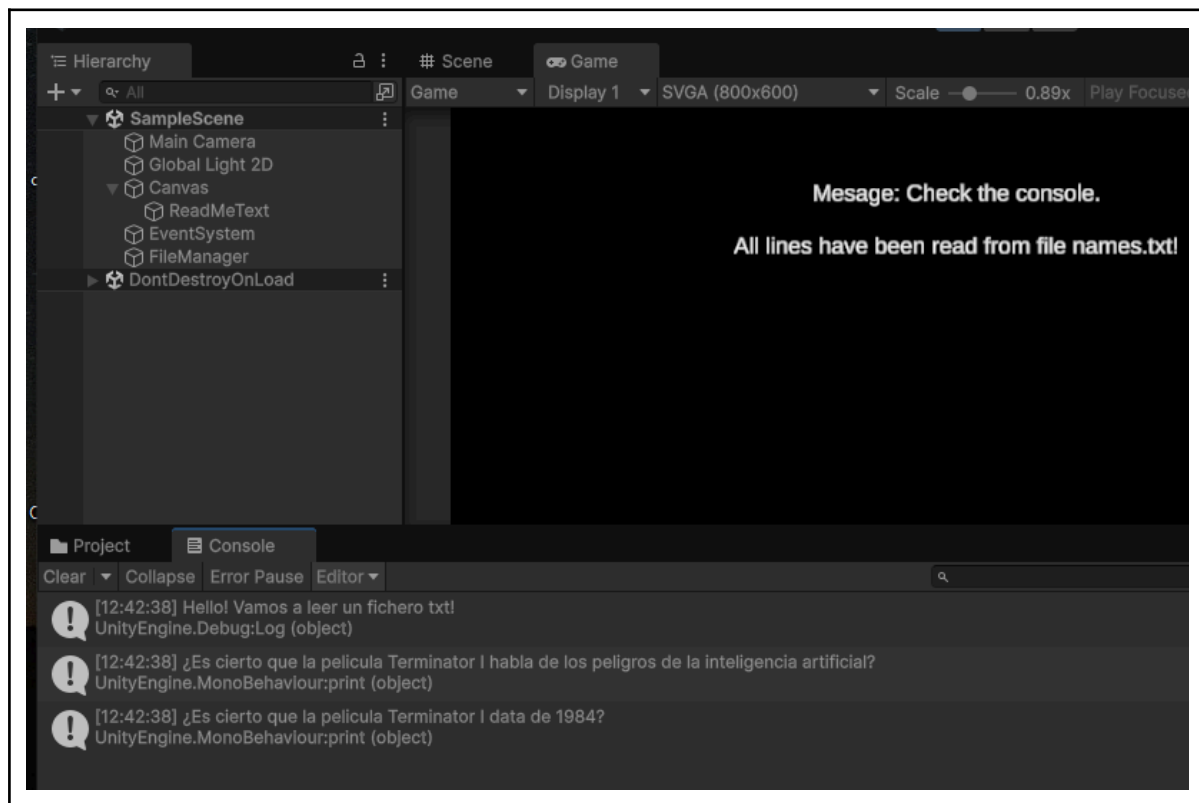
Recuerde que un “script” siempre tiene que ir asociado a un “GameObject”

Paso 6.

Haga clic en el botón “Play” que está en el centro de la parte superior de la pantalla principal.

Abra la consola (para ello haga clic en “Window”; después haga clic en “General” y finalmente haga clic en “Console”).

Observe que en la consola podemos leer las líneas que están grabadas en el archivo de texto **names.txt**. Esto indica que nuestro programa escrito en lenguaje C# con Microsoft Visual Studio Code ha leído con éxito las líneas del fichero de texto que se encuentra en la carpeta “Assets”.



Ejemplo C51b. Lectura e impresión de ficheros de texto.

Objetivo: Escribir el código necesario en C# para comprobar que Unity 6 puede imprimir mensajes, nombres o números contenidos en un fichero de texto (pasado como parámetro) con extensión .txt.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “2D (Core)”.

Escriba el siguiente nombre: **“U1bxt Hello World Project_txt plus”**.

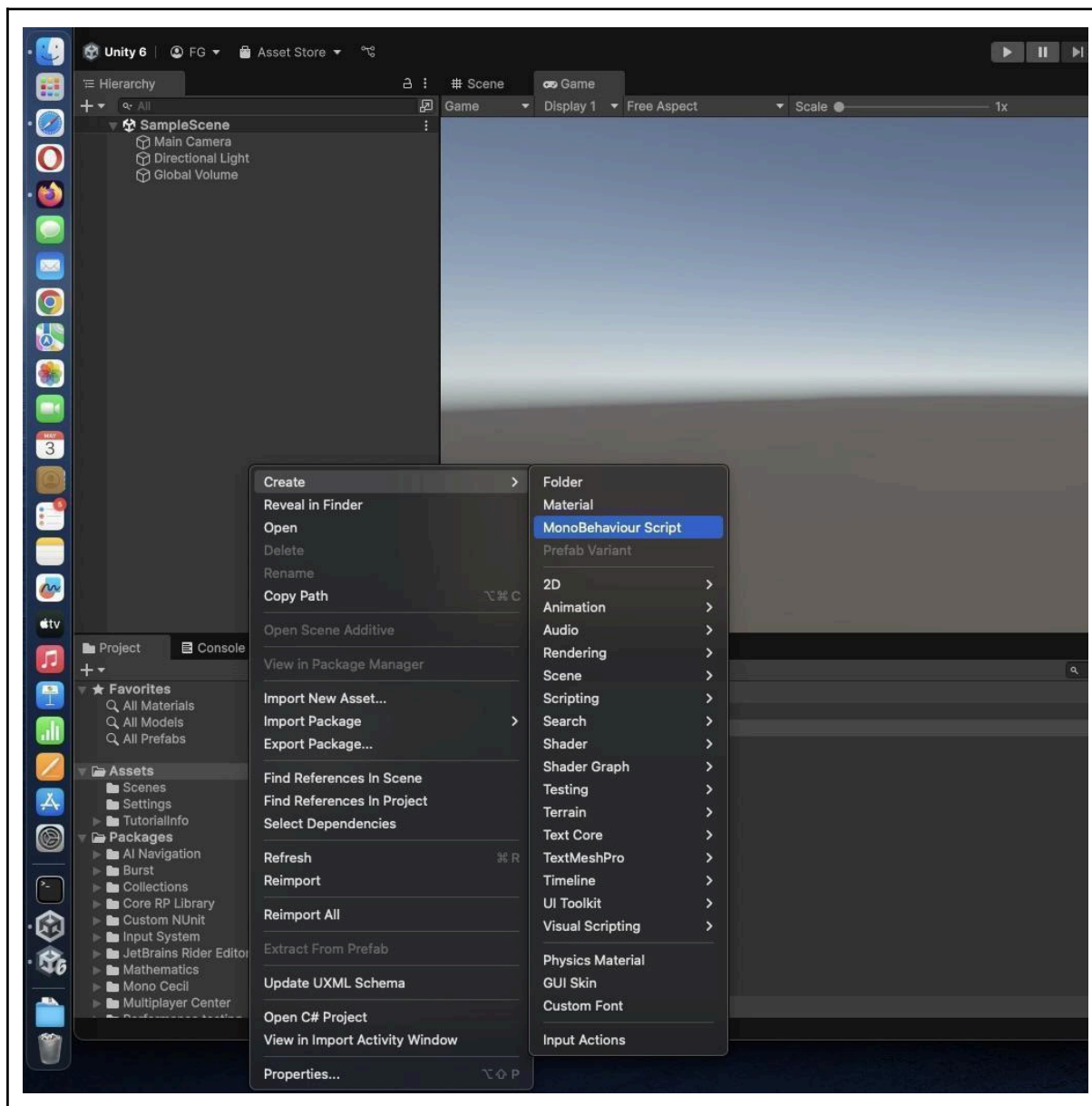
Paso 2.

a.

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

b.

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”.



Paso 3.

Busque y seleccione en “Assets” el script que acaba de crear. Haga doble clic en el script.

Paso 4.

Se abrirá Microsoft Visual Studio. Borre el código que aparece por defecto y copie el que se muestra en esta imagen:

```
using UnityEngine;                                     // CreadTXTfile.cs
using System.IO;

/// <summary>
/// Reads text files and returns lines or words (e.g., questions or answers) by index.
/// Now supports dynamic file paths via an optional parameter.
/// </summary>

public class FileManager : MonoBehaviour
{
    public static FileManager Instance { get; private set; }

    [SerializeField] private string _defaultFileName = "names.txt";

    private string[] _cachedLines;
    private string _lastLoadedFilePath; //Track last loaded file to avoid redundant reads

    void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }

    /// El código continúa en la siguiente página
```

```

/// Loads file and caches its lines. Called if GetQuestion() detects a new file.
///
private void LoadFile(string filePath)
{
    if (!File.Exists(filePath))
    {
        Debug.LogError($"File not found: {filePath}");
        return;
    }

    try
    {
        _cachedLines = File.ReadAllLines(filePath);
        _lastLoadedFilePath = filePath; // Remember the loaded file

        Debug.Log($"Loaded {_cachedLines.Length} lines from: {filePath}");
    }
    catch (System.Exception e)
    {
        Debug.LogError($"Error reading {filePath}: {e.Message}");
    }
}

///
/// Gets a line from the specified file (or defaults to _defaultFileName).
/// Example: GetQuestion(0, "custom.txt") or GetQuestion(1) for default file.
///
public string GetQuestion(int lineIndex, string fileName = null)
{
    // Use default file if none provided
    string fileToRead = fileName ?? _defaultFileName;
    string filePath = Path.Combine(Application.dataPath, fileToRead);

    // Reload file only if it's different from the last loaded one
    if (_cachedLines == null || filePath != _lastLoadedFilePath)
    {
        LoadFile(filePath);
    }

    // Validate line index
    if (lineIndex < 0 || lineIndex >= _cachedLines.Length)
    {
        Debug.LogError($"Invalid index: {lineIndex}. File has {_cachedLines.Length}
lines.");
        return string.Empty;
    }

    return _cachedLines[lineIndex];
}

}

/// Como se usa este procedimiento desde otro script:
///
/// string question = FileManager.Instance.GetQuestion(0, "names.txt");
///
/// ==> Obtiene el texto de la pregunta num. 1 del fichero por defecto (names.txt).
///
/// Fin

```

Guarde el archivo que acaba de modificar con el nombre **CreadTXTfile.cs** y cierre Microsoft Visual Studio.

No olvide crear el archivo de texto **names.txt** con dos líneas. Ponga una frase en cada línea y grabe el archivo en la carpeta “Assets” de su proyecto.

Paso 5.

Vuelva a Unity y siga estos pasos:

- En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “Create Empty” (crear vacío).
- Seleccione “GameObject”.
- En el “Inspector”, haga clic en “Add Component” (añadir componente).
- Busque y seleccione “**CreadTXTfile**” (si no aparece en la lista, teclee el nombre “**CreadTXTfile**” para que aparezca).

Paso 6.

a.

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

b.

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”.

c.

Busque y seleccione en “Assets” el script que acaba de crear. Haga doble clic en el script.

d.

Se abrirá Microsoft Visual Studio. Borre el código que aparece por defecto y copie el que se muestra en esta imagen:

```

using System.Runtime.ExceptionServices;                                     // keyboardManager.cs
using UnityEngine;

public class KeyboardManager : MonoBehaviour
{
    private AudioSource audio1;

    void Start()
    {
        // Find the GameObject with the tag "Audio1" and get its AudioSource component
        GameObject audioObject = GameObject.FindWithTag("Audio1");
        if (audioObject != null)
        {
            audio1 = audioObject.GetComponent<AudioSource>();
        }
        else
        {
            Debug.LogWarning("No GameObject with tag 'Audio1' found.");
        }
    }

    void Update()
    {
        // Detecta que la tecla '1' is pressed
        if (Input.GetKeyDown(KeyCode.Alpha1))
        {
            if (audio1 != null)
            {
                PlayAudio1.Play(); // llama al proc indep Public Static Class PlayAudio1
                                   // que esta en PlayAudio1.cs

                string question1 = FileManager.Instance.GetQuestion(0, "names.txt");
                                   // llama al proc indep Public GetQuestion de la clase
                                   // FileManager que está en CreadTxtFile.cs

                Debug.LogWarning($"Record number 0. Question 1 loaded: {question1}");
            }
            else
            {
                Debug.LogWarning("AudioSource not assigned.");
            }
        }
    }
}

```

Guarde el archivo que acaba de modificar con el nombre **keyboardManager.cs** y cierre Microsoft Visual Studio.

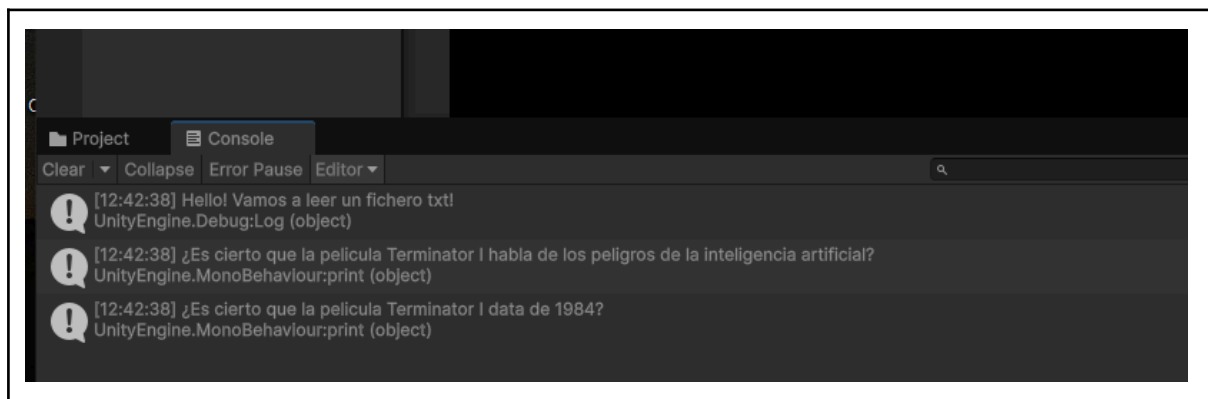
- En la ventana “Hierarchy window” (ventana jerarquía), haga clic con el botón derecho y elija “Create Empty” (crear vacío).
- Seleccione “GameObject”.
- En el “Inspector”, haga clic en “Add Component” (añadir componente).
- Busque y seleccione “**GO2_keyboardManager**” (si no aparece en la lista, teclee el nombre “**keyboardManager**” para que aparezca).

Paso 7.

Haga clic en el botón “Play” que está en el centro de la parte superior de la pantalla principal. Pulse la tecla correspondiente al número 1. Se leerá el archivo de texto grabado anteriormente.

Abra la consola (para ello haga clic en “Window”; después haga clic en “General” y finalmente haga clic en “Console”).

Observe que en la consola podemos leer las líneas que están grabadas en el archivo de texto **names.txt**. Esto indica que nuestro programa escrito en lenguaje C# con Microsoft Visual Studio Code ha leído con éxito las líneas del fichero de texto que se encuentra en la carpeta “Assets”.



Reto:

Al pulsar la tecla correspondiente al número 1, se leyó el archivo de texto y también debió de reproducirse un archivo de audio mp3 que simula el clic de un ratón. ¿Qué ajustes son necesarios para que **PlayAudio1.cs** (en el cuadro) concuerde con el código del script mostrado previamente? Lea atentamente:

```
using UnityEngine;                                     /// PlayAudio1.cs

public static class PlayAudio1
{
    private static AudioSource _audioSource; private const string AudioTag = "Audio1";

    static PlayAudio1()                                // Inicialización estática (se ejecuta una sola vez)
    {
        GameObject audioObject = GameObject.FindWithTag(AudioTag);

        if (audioObject != null)
        {
            _audioSource = audioObject.GetComponent();
            if (_audioSource != null)
            {
                _audioSource.playOnAwake = false; // Asegura que no se reproduzca al
inicio
            }
            else
            {
                Debug.LogError($"No AudioSource found on G.O. with tag '{AudioTag}'.");
            }
        }
        else
        {
            Debug.LogError($"No GameObject with tag '{AudioTag}' found.");
        }
    }

    // Método público para reproducir el audio desde cualquier script
    public static void Play()
    {
        if (_audioSource != null && _audioSource.clip != null)
        {
            _audioSource.Play();
        }
        else
        {
            Debug.LogWarning("Audio1 not initialized or missing AudioClip.");
        }
    }

    // Método para asignar un AudioSource manualmente (opcional)
    public static void SetAudioSource(AudioSource audioSource)
    {
        _audioSource = audioSource;
        if (_audioSource != null)
        {
            _audioSource.playOnAwake = false;
        }
    }
}

// Para reproducir el sonido tendrá que hacer la llamada: PlayAudio1.Play();
// Debe estar en otro script (ver script anterior).
// Llama al proc indep Public Static Class PlayAudio1 que está en este file PlayAudio1.cs
```


Ejemplo C52. Suma de dos números enteros entrados por teclado.

Objetivo: Escribir el código necesario en C# para comprobar que Unity 6 puede sumar variables enteras y visualizar la suma.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “2D (Core)”.

Escriba el siguiente nombre: “**U2 Hello Sumas**”.

Situado en la escena elija GameObject > UI > Canvas para crear un nuevo objeto Canvas.

Haga clic con el botón derecho en Hierarchy, elija UI > Input Field (TMP) para crear dos objetos tipo “input field”. Necesitará importar la librería TextMeshPro essentials. Llame a los objetos creados "InputField1" e "InputField2".

Haga clic con el botón derecho en el Canvas, seleccione UI > Button (TMP) para crear un nuevo objeto button. Llámelo "CalculateButton".

Haga clic con el botón derecho en el Canvas, seleccione UI > Text (TMP) para crear un nuevo objeto text box. Llámelo "ResultText".

Posicione cada elemento UI (Hierarchy) en el canvas (Probablemente necesite alargar el objeto "ResultText" para que quepa el resultado).

Paso 2. Cree un nuevo script.

En la ventana de Proyecto, cree el script "**calculator**". A continuación reemplace el código por las siguientes líneas:

```
using UnityEngine; using TMPro; // Required for TextMeshProUGUI and TMP_InputField
using UnityEngine.UI;           // Required for Button

public class Calculator : MonoBehaviour
{
    // Public variables to link UI elements from the Unity Editor
    public TMP_InputField inputField1; public TMP_InputField inputField2;
    public TextMeshProUGUI resultText; public Button calculateButton;

    /// This method is used to set up event listeners for UI elements.
    void Start()
    {
        // Add a listener to the button's click event.
        // When the calculateButton is clicked, llamamos a CalculateSum method
        calculateButton.onClick.AddListener(CalculateSum);
    }

    /// Calculates the sum of two integers entered in the input fields.
    /// It parses the text from the input fields, performs calculation,
    /// and displays the result in the resultText UI element.
    void CalculateSum()
    {
        int num1 = 0;   int num2 = 0;

        // Try to parse the text from the first input field into an integer.
        // int.TryParse is used 4 safe parsing, return .T.(if
successful).F.otherwise.
        if (int.TryParse(inputField1.text, out num1))
        {
            Debug.Log("Parsed num1: " + num1);   ///just debugging
        }
        else
        {
            // If parsing fails, log an error and display an error message to the
user.
            Debug.LogError("Invalid number in Input Field 1: " + inputField1.text);
            resultText.text = "Error: Invalid number in Input 1";
            return; // Exit the method as we cannot proceed with calculation.
        }

        // Attempt to parse the text from the second input field into an integer.
        if (int.TryParse(inputField2.text, out num2))
        {
            Debug.Log("Parsed num2: " + num2);   ///just debugging
        }
        else
        {
            // If parsing fails, log an error and display an error message to the
user.
            Debug.LogError("Invalid number in Input Field 2: " + inputField2.text);
            resultText.text = "Error: Invalid number in Input 2";
            return; // Exit the method.
        }

        int sum = num1 + num2;   // Calculate the sum of the two parsed integers.

        // Display result in resultText UI element. Convert integer sum to string
resultText.text = "Result: " + sum.ToString(); Debug.Log("Sum: " + sum);
    }
}
```

Paso 3. Adjunte el nuevo script a un nuevo GameObject.

Cree un GameObject vacío en la escena (Game Object; Create Empty) y llámelo “CalculatorManager”.

Arrastre el archivo **Calculator.cs** desde la ventana de proyecto dentro del GameObject “CalculatorManager” que está en la ventana “Hierarchy”.

Paso 4. Asigne los UI Elements a las variables del script.

Seleccione el GameObject "CalculatorManager" GameObject en “Hierarchy”.

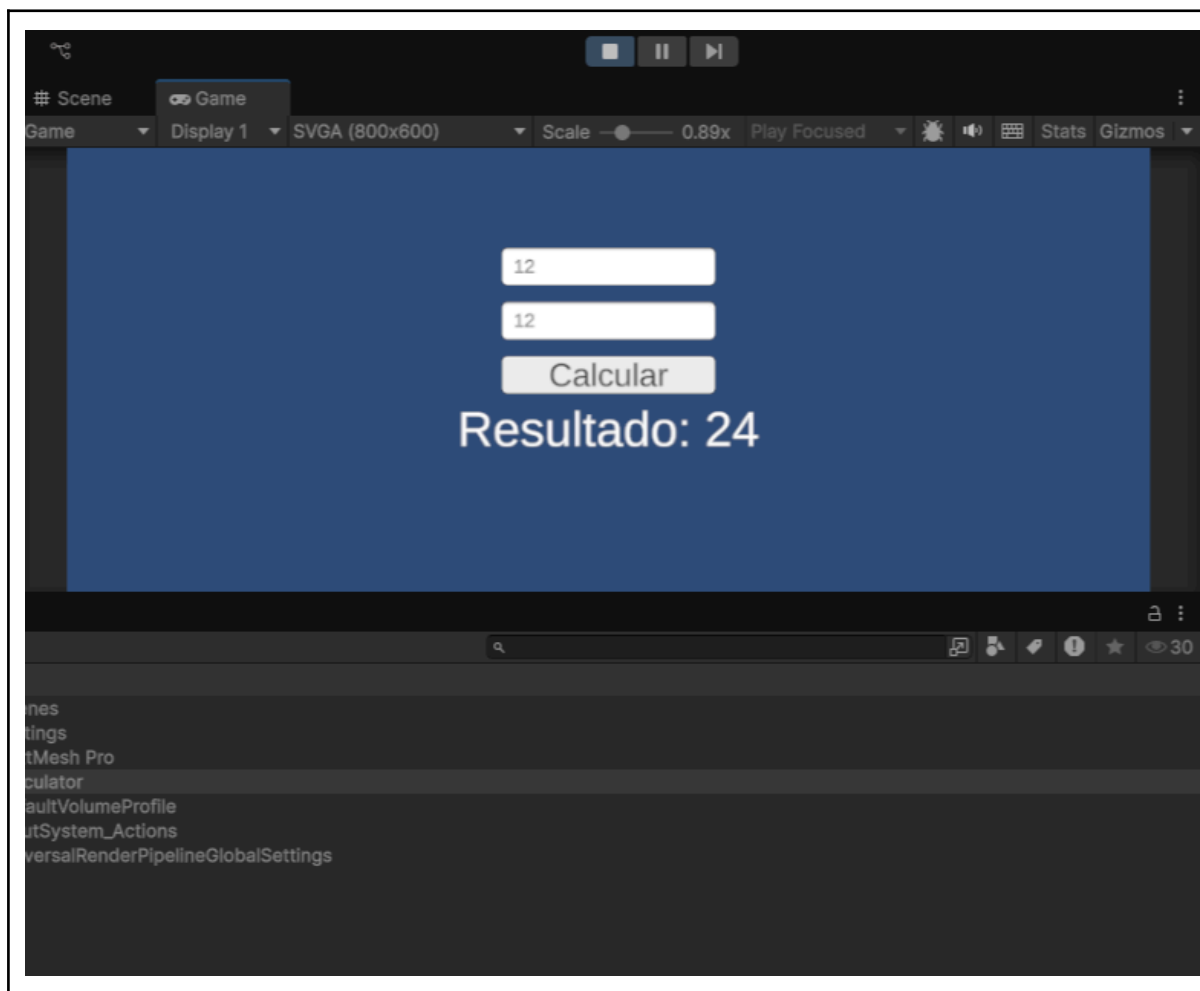
En el Inspector, verá las variables públicas que se declararon en el script Calculator (inputField1, inputField2, resultText, calculateButton).

Arrastre el elemento "InputField1" desde la ventana Hierarchy al hueco Input Field 1 en el Inspector. (Repita el proceso con el elemento InputField2).

Arrastre el elemento "ResultText" desde la ventana Hierarchy al hueco de Result Text en el Inspector.

Arrastre el elemento "CalculateButton" desde la ventana Hierarchy dentro del hueco de Calculate Button en el Inspector.

Paso 5. Ejecute la aplicación. Entre un número entero en cada caja de texto y haga clic en el botón “Calcular”. Deberá ver este resultado:



Ejemplo C53. Creación en tres pasos de un cubo con un mensaje.

Objetivo: Crear dos escenas en nuestra aplicación (o juego). El texto "Game Over" deberá aparecer centrado en la segunda escena, después de ser lanzada al pulsar un botón (trigger) en la primera escena. Para ello utilizaremos el script SceneSwitcher.cs.

Parte 1: Crear SampleScene si no existe todavía (ésta será la primera escena)

1. Abra un nuevo Proyecto 2D (o un proyecto 2D existente).

- **Llame a su nuevo proyecto: “Slide11-GameOver”.**

2. Crear SampleScene (si todavía no existe):

- En la ventana **Project**, localice Assets > Scenes.
- Haga clic con el botón derecho (en un área vacía) y seleccione Create > Scene.
- Renombre la nueva escena como “SampleScene”.

3. Abra “SampleScene” para crear el botón “TriggerScene2Button”:

- **Haga** doble clic en “SampleScene” en la ventana “Project” para abrirla en la ventana “Hierarchy”.
- En la ventana **Hierarchy**, haga clic con el botón derecho del ratón y seleccione UI > Button - Text (TextMeshPro).
- Se creará un Canvas nuevo y el sistema de eventos.
- **En** “Hierarchy”, seleccione el botón que acaba de crear. En la ventana **Inspector**, cambie su nombre. Escriba “**TriggerScene2Button**”.
- **Cambie el texto del botón Button**. En el Inspector, cambie el campo Text Input. Escriba “**Go to SampleScene2**”.
- **Cambie la posición del botón (Opcional)**.

4. Cree en la carpeta “Assets” el archivo “SceneSwitcher.cs” con el código del siguiente cuadro :

```
/// SceneSwitcher.cs

using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneSwitcher : MonoBehaviour
{
    public void LoadSceneByName(string sceneName)
    {
        SceneManager.LoadScene(sceneName);
    }
}
```

5. Adjunte el script “SceneSwitcher.cs” a un “GameObject” vacío:

- En la ventana “**Hierarchy**”, haga clic con el botón derecho del ratón y seleccione “Create Empty”.
- Cambie el nombre al “**GameObject**” que acaba de crear. Llámelo **SceneManagerObject**.
- Arrastre el script **SceneSwitcher** desde la ventana Project hasta el GameObject llamado “SceneManagerObject” en “Hierarchy”.

6. Configure el botón para llamar a “SceneSwitcher.cs”:

- Seleccione **TriggerScene2Button** en la ventana **Hierarchy**.
- En la ventana **Inspector**, busque el “Button component”.
- Busque la lista “OnClick()”. Haga clic en el botón + para añadir una nueva entrada.
- Arrastre el objeto **SceneManagerObject** desde “Hierarchy” hasta el campo None (Object) de la nueva entrada vacía.
- Haga clic en el menú “No Function” y haga clic en la opción **SceneSwitcher > LoadSceneByName** (string).
- Escriba en el campo que acaba de aparecer: **SampleScene2** (escribalo sin errores. Debe coincidir con el nombre que eligió al crear la segunda escena).

Parte 2: Preparar SampleScene2 (la segunda escena)

1. Crear SampleScene2:

- En la ventana “**Project**”, seleccione Assets > Scenes.
- A continuación, haga clic con el botón derecho del ratón para crear una nueva escena:. Seleccione Create > Scene.
- Renombre la nueva escena como “**SampleScene2**”.
- Haga doble clic en “**SampleScene2**” en la ventana “Project” para abrirla.

2. Crear el Textbox (TextMeshPro):

- En la ventana “**Hierarchy**”, haga clic con el botón derecho del ratón y seleccione UI > Text - TextMeshPro. Se creará automáticamente un Canvas (lienzo) y el sistema de eventos.
- **(Opcional):** En la ventana “Hierarchy”, seleccione el texto que acaba de crear. En el Inspector, cambie el nombre del texto y llámelo: GameOverText.
- **Cambie el contenido del texto:** En el Inspector, busque el componente TextMeshPro - Text:
 - En el campo “Text Input” , escriba “Game Over”.
 - Cambie el tamaño de la fuente. Por ejemplo, 72.
Opcionalmente puede **centrar el texto**.

Nota: Nuestra segunda escena solo tiene un mensaje que muestra el fin del juego. Sin embargo, en esta escena se podría diseñar otro nivel del juego, un puzzle, un resumen de la historia, etc..

Parte 3: Adición de escenas en los ajustes (Build Settings).

Unity necesita saber qué escenas deben incluirse en el proyecto y el orden de aparición que deben seguir. Veamos cómo se incluyen y ordenan las escenas:

- Haga clic en “File” > “Build Settings” (ajustes de construcción del proyecto).
- Arrastre **SampleScene** desde la ventana **Project** hasta la lista “**Scenes In Build**”. Debería aparecer con el índice en 0.
- Arrastre **SampleScene2** desde la ventana **Project** hasta la lista “**Scenes In Build**” Debería aparecer con el índice en 1.
- **Importante:** La escena con el índice 0 es normalmente la primera escena que se carga al iniciar el juego o la app. En nuestro caso, la primera escena es “ SampleScene”.
- **Cierre la ventana** “Build Settings”.

Parte 4. Comprobación del botón de cambio de escena.

- Vaya a la escena principal “SampleScene”.
- En la ventana Proyecto, haga doble clic en la escena “SampleScene” para fijar la escena activa.
- Haga clic en el botón “Play” en el Editor.
- Haga clic en el botón “TriggerScene2Button” (el texto que aparece sobre el botón es: “Go to Scene2”).
 - Ahora debería ver la escena llamada SampleScene2, donde aparece el texto en el centro de la pantalla “Game Over”.

Ejemplo C54. Creación en tres pasos de un cubo en 3D con un mensaje en el Canvas.

Objetivo: Escribir el código necesario en C# para comprobar que Unity puede imprimir mensajes sobre objetos creados en el “canvas” en escenarios en 3D .

Veamos en primer lugar algunos conceptos básicos sobre objetos 3D:

En Unity, la forma de escalar un cubo (objeto 3D) y un elemento de **Texto** de la interfaz de usuario (UI) es muy diferente.

El cubo se controla con el componente **Transform**, mientras que el Texto usa un **RectTransform** y se ve afectado por el componente **Canvas Scaler** de su Canvas padre. Las "tres opciones de modo de escala" (scale mode options) son específicas del **Canvas Scaler** y afectan a todos los elementos de la UI, incluido el **Texto**.

Cómo escalar un cubo 3D

Un cubo 3D es un **GameObject** estándar en la escena. Su escala se controla directamente desde la propiedad **localScale** de su componente **Transform**.

- **Escalado Directo:** Podemos cambiar los valores X, Y y Z para estirar o encoger el cubo en el mundo 3D.
- **Escalado Jerárquico:** Si el cubo es hijo de otro **GameObject**, su escala final es una multiplicación de su **localScale** y la de todos sus padres.

No existen "modos de escala" (scale mode options) para un cubo 3D como los hay para la UI. Su tamaño es absoluto dentro del espacio 3D, sujeto a su escala local (**localScale**) y la de sus padres.

Cómo escalar un Texto de UI y los tres modos del Canvas

Los elementos de UI, como el **Texto**, existen dentro de un **Canvas**. El componente **Canvas Scaler** en el Canvas principal determina cómo estos elementos se escalan con respecto a la pantalla. Aquí puede ver los tres modos de escala:

1. Constant Pixel Size (Tamaño de Píxel Constante)

Este es el modo por defecto. Con esta opción, los elementos de UI mantendrán su tamaño en píxeles sin importar la resolución de la pantalla.

- **Scale Factor (Factor de Escala):** Se puede ajustar un factor para escalar todos los elementos de UI de manera uniforme.
- **Comportamiento:** Un texto de tamaño 24 se verá de 24 píxeles de alto tanto en una pantalla 1080p como en una 4K. Esto significa que parecerá más pequeño en la pantalla de mayor resolución, porque los píxeles son más pequeños.

2. Scale With Screen Size (Escalar con el Tamaño de la Pantalla)

Este modo ajusta el tamaño de la UI junto con la resolución de la pantalla. Es la opción **más utilizada** por los programadores para crear interfaces que se vean consistentes en diferentes dispositivos.

- **Reference Resolution (Resolución de Referencia):** A menudo, establecemos una resolución base (ej. 1920x1080). Unity escalará la UI hacia arriba o hacia abajo comparando la resolución actual con la de referencia.
- **Screen Match Mode (Modo de Coincidencia):** Decide cómo se realiza el escalado si la relación de aspecto de la pantalla es diferente a la de referencia. Podemos hacer que coincida con el ancho, el alto o una mezcla de ambos.

3. Constant Physical Size (Tamaño Físico Constante)

Este modo intenta que el tamaño físico real de tus elementos de UI sea el mismo en todos los dispositivos, sin importar la resolución.

- **Physical Unit (Unidad Física):** Podemos especificar la unidad de medida (milímetros, pulgadas, puntos, etc.).
- **Comportamiento:** Un botón configurado para medir 2 cm de ancho debería medir 2 cm reales tanto en un móvil pequeño como en una tableta grande, siempre que el dispositivo informe correctamente de sus DPI (puntos por pulgada). Es útil para garantizar que los elementos táctiles tengan siempre un tamaño cómodo.

Ahora que hemos repasado cómo interactúan textos y objetos 3D en Unity, veamos un ejemplo práctico:

Objetivo: Escribir el código necesario para comprobar que Unity puede imprimir mensajes sobre objetos creados en el “canvas” en escenarios en 3D .

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “3D (Core)”.

Escriba el siguiente nombre del proyecto: “U5- Project Cube”.

Paso 2. Cree el Canvas UI:

1. Add (añada) un Canvas:

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”.

- Seleccione UI > Canvas. Esto creará un “Canvas” (lienzo) en la escena.

- Busque el apartado “**Render Mode**” en el Inspector (recuerde que el Canvas debe estar seleccionado en “Hierarchy”) y asegúrese que el elemento “**Canvas component**” tiene activada la opción “**Screen Space - Overlay**”.

2. Add (añada) un UI Element (Opcional):

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”.

- Seleccione cualquier elemento UI, haga clic en UI > Text, Button, o Image (esto añadirá el elemento elegido en el Canvas).

Paso 3. Añada un cuboide de referencia de 5m x 1m x 5m:

1. Añada un cuboide (Cube):

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”.
- Seleccione 3D Object > Cube. Esto creará un “cubo” en la escena.

2. Ajuste Cube's Scale y cambie su nombre a “Cube100”:

- Seleccione “Cube100” en la ventana Hierarchy.
- En el Inspector, ajuste (en el componente “Transform”) los valores de “Scale” a `(5, 1, 5)`.

3. Cambie la Posición del objeto “Cube100”:

- Cambie la posición del cubo en el Inspector para colocarlo donde usted desee. Originalmente lo hemos dejado en la posición (0,0,0).

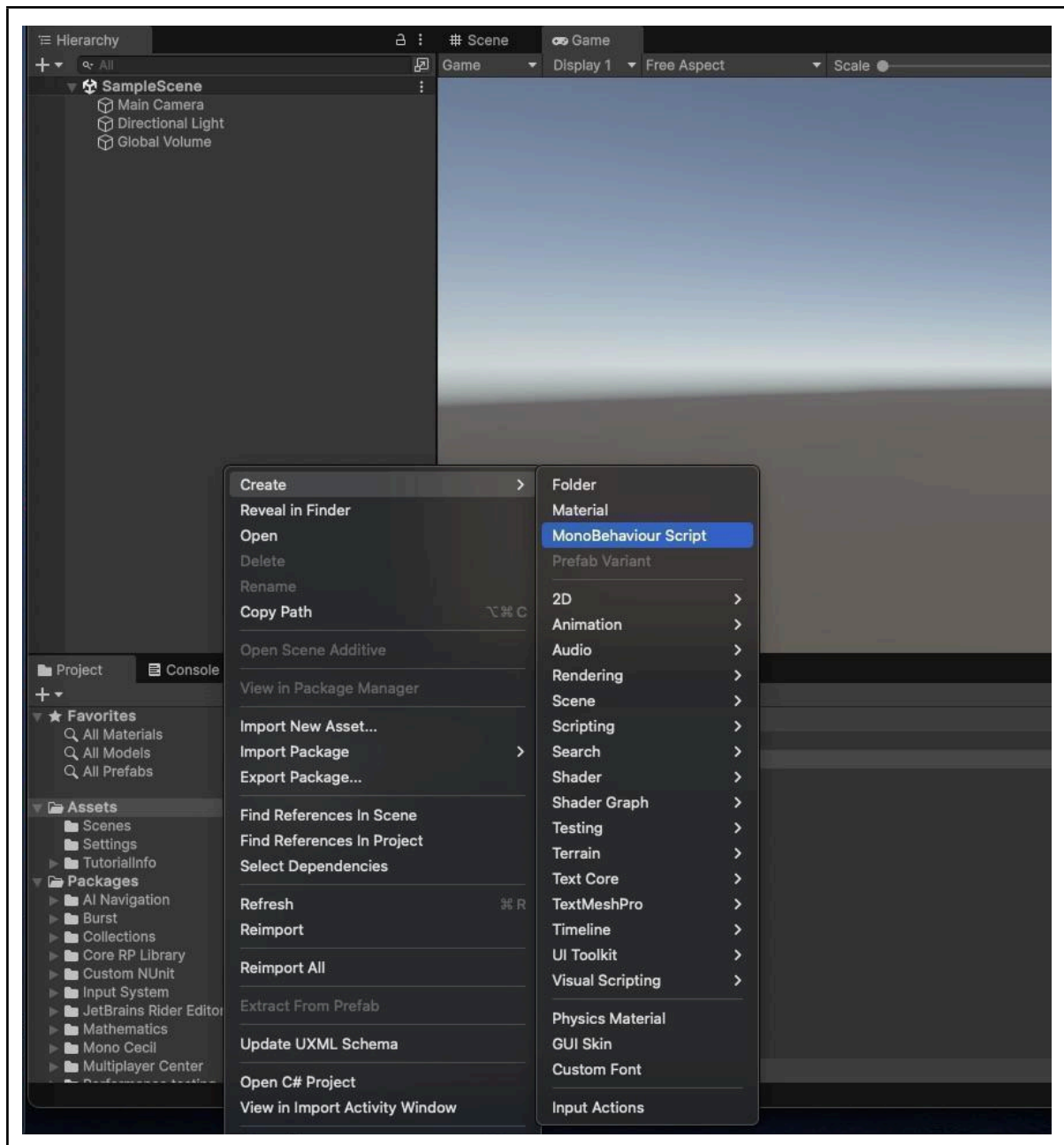
Este cubo lo haremos invisible más adelante. Lo hemos creado para tener un objeto de referencia para nuestro diseño.

Reto:

Para visualizar la cadena "Hola!" en la cara frontal de un nuevo objeto `Cube101`, utilizaremos el componente `TextMesh`. En la página siguiente verá cómo creamos el script. Analice el ejemplo y realice los cambios oportunos:

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “createCube1”.



Haga doble clic en el archivo “.cs” que acaba de renombrar y copie el siguiente programa:

```
using UnityEngine;
using TMPro;

public class CreateCube1 : MonoBehaviour
{
    [Header("Cube Settings")]
    [SerializeField] private string cubeName = "Cube101";
    [SerializeField] private string cubeTag = "101";
    [SerializeField] private Vector3 cubePosition = Vector3.zero;
    [SerializeField] private Vector3 cubeScale = new Vector3(5, 1, 5);
    [SerializeField] private bool enablePhysics = false;
    [SerializeField] private Color cubeColor = Color.white;

    [Header("Text Settings")]
    [SerializeField] public string text_in_the_cube = "Tu texto aquí"; //find esta var. mas adelante
    [SerializeField] private Color textColor = Color.black;
    [SerializeField] private float textSize = 14f; // Default size 14
    [SerializeField] private float textOffset = 0.01f; // Small offset from surface

    void Awake()
    {
        CreateCubeInCenter();
    }

    /// [el código continúa en la siguiente página. . . ]
}
```

```

void CreateCubeInCenter()
{
    if (GameObject.Find(cubeName) != null)
    {
        Debug.LogWarning($"{cubeName} ya existe!"); return;
    }

    // Create cube
    GameObject cube = GameObject.CreatePrimitive(PrimitiveType.Cube);
    cube.name = cubeName;
    cube.tag = cubeTag;
    cube.transform.position = cubePosition;
    cube.transform.localScale = cubeScale;
    cube.GetComponent<Renderer>().material.color = cubeColor;

    if (enablePhysics)
    {
        cube.AddComponent<Rigidbody>();
    }

    // Crear y configurar TextMeshPro
    GameObject textObject = new GameObject("CubeText");
    textObject.transform.SetParent(cube.transform);

    // Probamos DOS posiciones: Position text on front face (Z+ direction)
    // textObject.transform.localPosition = new Vector3(0, 0, cubeScale.z/2 + textOffset);
    textObject.transform.localPosition = new Vector3(0, 0, -0.5f);

    // Ojo! Incluir la siguiente linea si el usuario no ve el texto correctamente
    //textObject.transform.rotation = Quaternion.Euler(0, 180, 0); // Face camera

    TextMeshPro textMesh = textObject.AddComponent<TextMeshPro>();

    text_in_the_cube = "Hola!"; // Asignar el texto al cubo 101
    textMesh.text = text_in_the_cube; // text que va en el cubo

    textMesh.color = textColor;

    textMesh.fontSize = textSize;
    textMesh.enableAutoSizing = true; // Enable auto-sizing: Muy importante
    textMesh.fontSizeMin = 12; // Minimum size when auto-sizing
    textMesh.fontSizeMax = textSize; // Maximum size (your default size)

    textMesh.alignment = TextAlignmentOptions.Center;
    textMesh.verticalAlignment = VerticalAlignmentOptions.Middle;

    // Ajustamos rect transform to match cube face
    textMesh.rectTransform.sizeDelta = new Vector2(cubeScale.x, cubeScale.y);

    Debug.Log($"Cube created with text: '{text_in_the_cube}' (Auto-sized font)");
}
}

```

A continuación siga estos pasos para comprobar que el código funciona:

1. Adjunte su código (createCube1.cs) a un nuevo objeto vacío GameObject en su escena.
2. Renombre el GameObject recién creado. Llámelo “**GO1_create Cube1**”.
3. Ejecute la escena. El texto “**Hola!**” deberá aparecer sobre el objeto ‘Cube100’.

Puede, si lo desea, repetir el proceso para nuevos objetos ‘cube2’ y ‘cube3’ creando nuevos “scripts” para los objetos “cube2” y “cube3”.



Ejemplo C55. Creación en tres pasos de un cubo en 3D con un mensaje para hacer sonar un fichero mp3.

Objetivo: Escribir el código necesario en C# para comprobar que Unity puede imprimir mensajes sobre objetos creados en el “canvas” en escenarios en 3D y hacer sonar ficheros mp3.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “3D (Core)”.

Escriba el siguiente nombre del proyecto: “U5-Cube-39fl”.

Paso 2. Cree el Canvas UI:

1. Add (añada) un Canvas:

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”.

- Seleccione UI > Canvas. Esto creará un “Canvas” (lienzo) en la escena.

- Busque el apartado “**Render Mode**” en el Inspector (recuerde que el Canvas debe estar seleccionado en “Hierarchy”) y asegúrese que el elemento “**Canvas component**” tiene activada la opción “**Screen Space - Overlay**”.

2. Add (añada) un UI Element (Opcional):

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”.

- Seleccione cualquier elemento UI, haga clic en UI > Text, Button, o Image (esto añadirá el elemento elegido en el Canvas).

Paso 3. Añada un cuboide de referencia de 5m x 1m x 5m:

1. Añada un cuboide (Cube):

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”.
- Seleccione 3D Object > Cube. Esto creará un “cubo” en la escena.

2. Ajuste Cube's Scale y cambie su nombre a “Cube100”:

- Seleccione “Cube100” en la ventana Hierarchy.
- En el Inspector, ajuste (en el componente “Transform”) los valores de “Scale” a `(5, 1, 5)`.

3. Cambie la Posición del objeto “Cube100”:

- Cambie la posición del cubo en el Inspector para colocarlo donde usted desee. Originalmente lo hemos dejado en la posición (0,0,0).

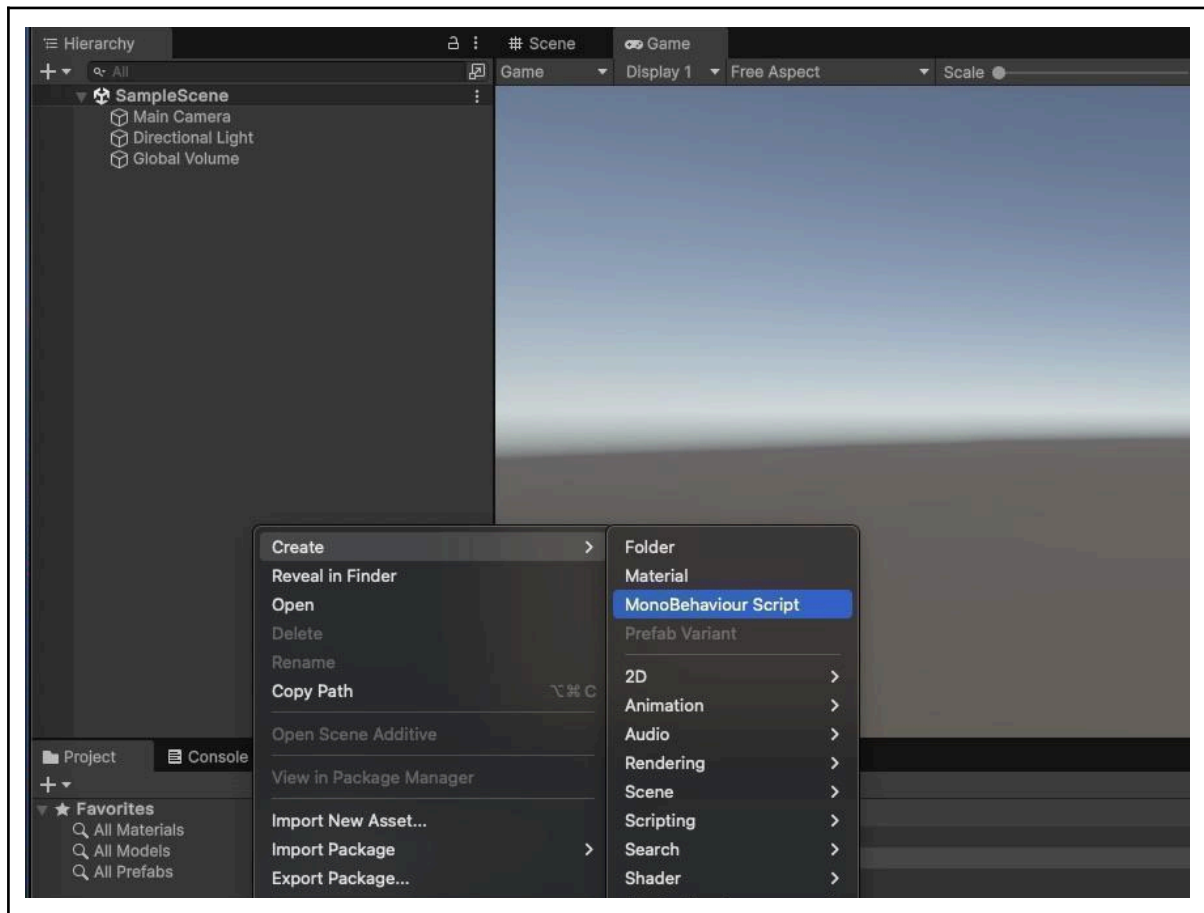
Este cubo lo haremos invisible más adelante. Lo hemos creado para tener un objeto de referencia para nuestro diseño.

Reto 1: Crear un texto sobre un cuboide

Para visualizar la cadena "Pulse <1>" en la cara frontal de un nuevo objeto `Cube101`, utilizaremos el componente `TextMesh`. En la página siguiente verá cómo creamos el script. Analice el ejemplo y realice los cambios oportunos:

En la ventana "Project window" (ventana de proyecto), haga clic con el botón derecho del ratón dentro de "Assets Folder" (carpeta de recursos).

Seleccione "Create" y después seleccione "MonoBehaviour Script". Se habrá creado un archivo C# llamado "NewMonoBehaviourScript". A continuación renombre el script que se acaba de crear como "createCube1".



Haga doble clic en el archivo “.cs” que acaba de renombrar y copie el siguiente programa:

```
using UnityEngine;
using TMPro;

public class CreateCube1 : MonoBehaviour
{
    [Header("Cube Settings")]
    [SerializeField] private string cubeName = "Cube101";
    [SerializeField] private string cubeTag = "101";
    [SerializeField] private Vector3 cubePosition = Vector3.zero;
    [SerializeField] private Vector3 cubeScale = new Vector3(5, 1, 5);
    [SerializeField] private bool enablePhysics = false;
    [SerializeField] private Color cubeColor = Color.white;

    [Header("Text Settings")]
    [SerializeField] public string text_in_the_cube = "Tu texto aquí"; //find esta var. mas adelante
    [SerializeField] private Color textColor = Color.black;
    [SerializeField] private float textSize = 14f; // Default size 14
    [SerializeField] private float textOffset = 0.01f; // Small offset from surface

    void Awake()
    {
        CreateCubeInCenter();
    }

    /// [el código continúa en la siguiente página. . . ]
```



```

void CreateCubeInCenter()
{
    if (GameObject.Find(cubeName) != null)
    {
        Debug.LogWarning($"{cubeName} ya existe!"); return;
    }

    // Create cube
    GameObject cube = GameObject.CreatePrimitive(PrimitiveType.Cube);
    cube.name = cubeName;
    cube.tag = cubeTag;
    cube.transform.position = cubePosition;
    cube.transform.localScale = cubeScale;
    cube.GetComponent<Renderer>().material.color = cubeColor;

    if (enablePhysics)
    {
        cube.AddComponent<Rigidbody>();
    }

    // Crear y configurar TextMeshPro
    GameObject textObject = new GameObject("CubeText");
    textObject.transform.SetParent(cube.transform);

    // Probamos DOS posiciones: Position text on front face (Z+ direction)
    // textObject.transform.localPosition = new Vector3(0, 0, cubeScale.z/2 + textOffset);
    textObject.transform.localPosition = new Vector3(0, 0, -0.5f);

    // Ojo! Incluir la siguiente linea si el usuario no ve el texto correctamente
    //textObject.transform.rotation = Quaternion.Euler(0, 180, 0); // Face camera

    TextMeshPro textMesh = textObject.AddComponent<TextMeshPro>();

    text_in_the_cube = "Pulse <1>"; // Asignar el texto del num. 1 al cubo 101
    textMesh.text = text_in_the_cube;    // text que va en el cubo

    textMesh.color = textColor;

    textMesh.fontSize = textSize;
    textMesh.enableAutoSizing = true; // Enable auto-sizing: Muy importante
    textMesh.fontSizeMin = 12; // Minimum size when auto-sizing
    textMesh.fontSizeMax = textSize; // Maximum size (your default size)

    textMesh.alignment = TextAlignmentOptions.Center;
    textMesh.verticalAlignment = VerticalAlignmentOptions.Middle;

    // Ajustamos rect transform. Ojo! multiplied x 2 para que quepa el texto
    textMesh.rectTransform.sizeDelta = new Vector2(2*cubeScale.x, cubeScale.y);

    Debug.Log($"Cube created with text: '{text_in_the_cube}' (Auto-sized font)");
}
}

```

A continuación siga estos pasos:

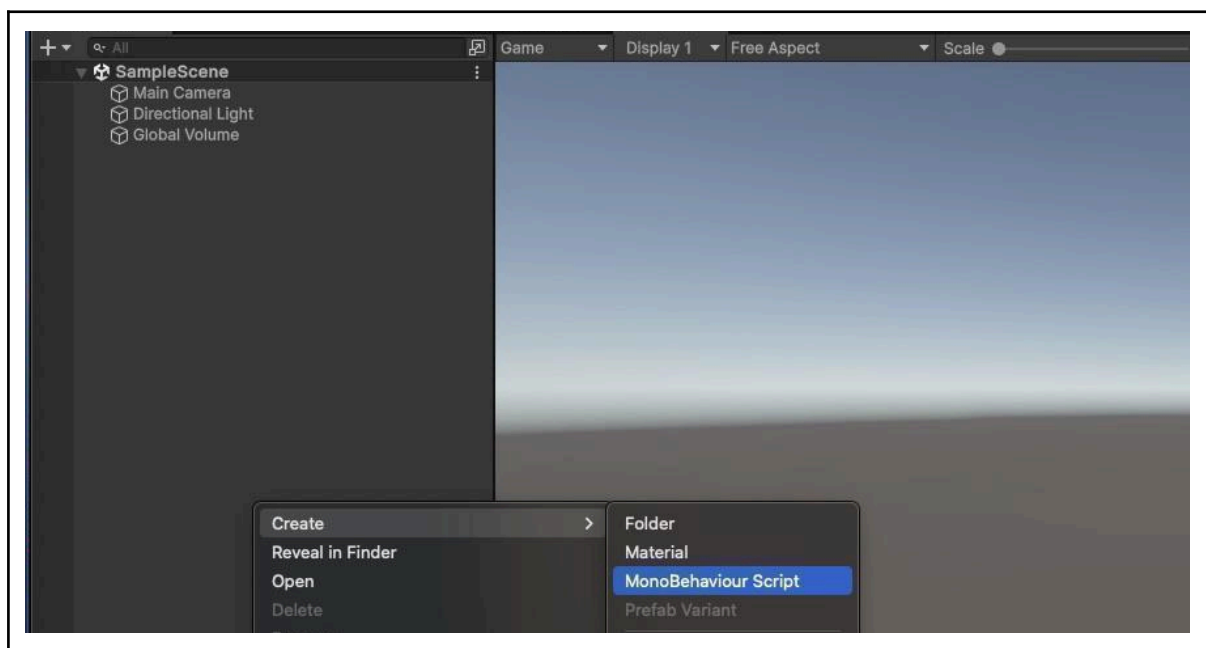
1. Adjunte su código (createCube1.cs) a un nuevo objeto vacío GameObject en su escena.
2. Renombre el GameObject recién creado. Llámelo “GO1_create Cube1”.

Reto 2: Incluir sonido a nuestro proyecto

Para hacer sonar un archivo de audio cuando se pulse la tecla correspondiente al número 1, necesitaremos crear un programa para controlar el teclado. En la página siguiente verá cómo creamos el script. Analice el ejemplo y realice los cambios oportunos:

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “keyboardManager”.



```

/// keyboardManager.cs

/// Recuerde crear una carpeta llamada "Sounds" en el directorio "Assets" de Unity de
/// su proyecto e incluir en la carpeta "Sounds" un archivo llamado "bip-sound-001.mp3".
/// Puede encontrar este archivo en cualquier web de audios gratuitos.
/// Cree un objeto AudioSource en "Hierarchy" llamado Audio_Source1 y cree un tag
/// llamado "Audio1" para este audio. Adjunte el fichero bip-source-001.mp3 al
/// campo Audio Resource del objeto Audio Source creado.

using UnityEngine;

public class KeyboardManager : MonoBehaviour
{
    private AudioSource audio1;

    void Start()
    {
        // Find the GameObject with the tag "Audio1" and get its AudioSource component
        GameObject audioObject = GameObject.FindWithTag("Audio1");
        if (audioObject != null)
        {
            audio1 = audioObject.GetComponent<AudioSource>();
        }
        else
        {
            Debug.LogWarning("No GameObject with tag 'Audio1' found.");
        }
    }

    void Update()
    {
        // Detecta when the '1' key is pressed
        if (Input.GetKeyDown(KeyCode.Alpha1))
        {
            if (audio1 != null)
            {
                audio1.Play();
            }
            else
            {
                Debug.LogWarning("AudioSource not assigned.");
            }
        }
    }
}

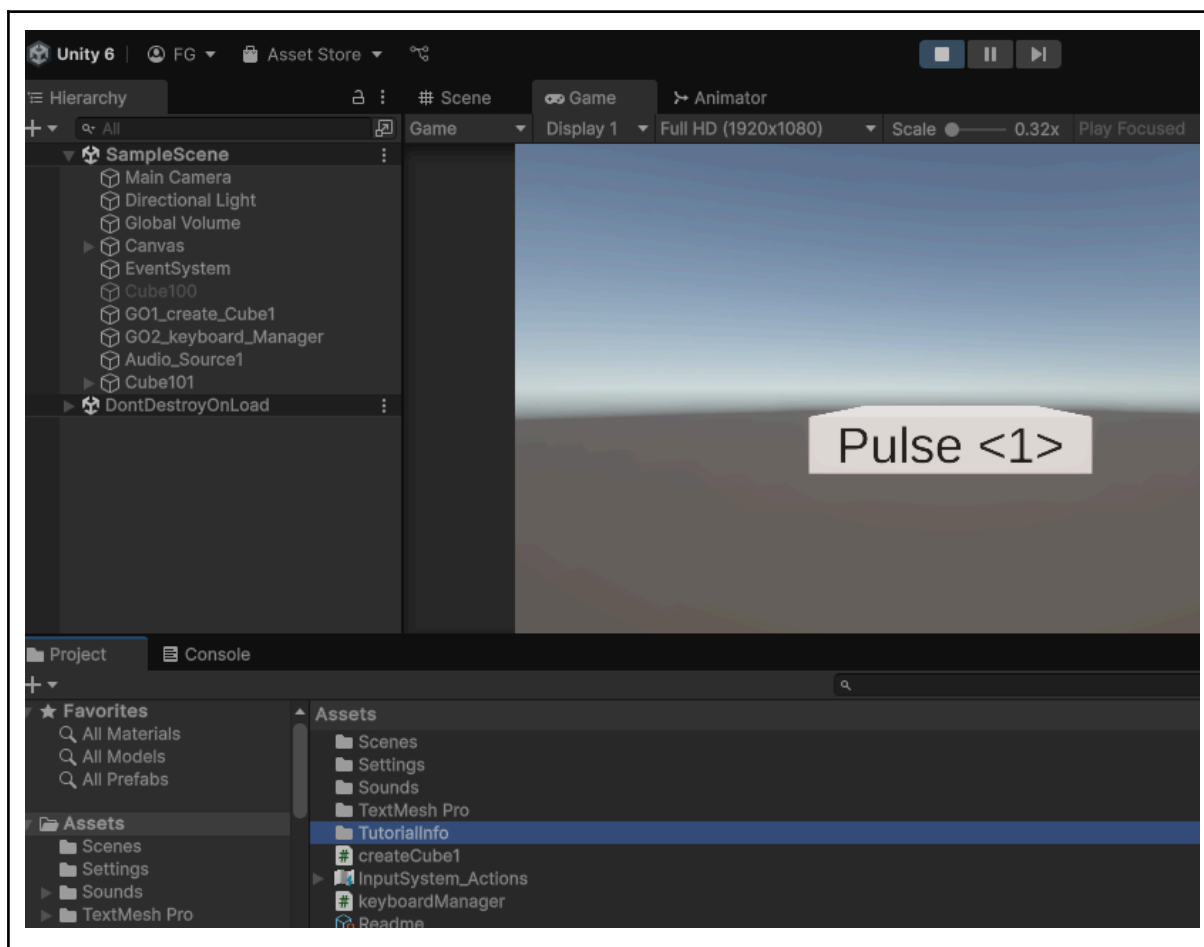
```

A continuación siga estos pasos para comprobar que el código funciona:

1. Adjunte su código (keyboardManager.cs) a un nuevo objeto vacío GameObject en su escena.
2. Renombre el GameObject recién creado. Llámelo “GO2_keyboard_Manager”.
3. Ejecute la escena. El texto “**Pulse <1>**” deberá aparecer sobre el objeto `Cube100`.

Al ejecutar el proyecto y pulsar la tecla **1**, el archivo mp3 sonará. Puede, si lo desea, repetir el proceso para nuevos objetos `cube2` y `cube3` creando nuevos “scripts” para estos objetos con el fin de reproducir diferentes sonidos.

Si el audio suena **dos** veces, no olvide seleccionar **Audio_Source1** en “Hierarchy” y a continuación, en el Inspector, desmarcar **Play On Awake**.



Ejemplo C56. Creación en tres pasos de un cubo en 3D con un mensaje para hacer sonar un fichero mp3 con una clase pública

Objetivo: Escribir el código necesario en para comprobar que Unity puede imprimir mensajes sobre objetos creados en el “canvas” en escenarios en 3D y hacer sonar ficheros mp3 utilizando una clase pública.

Una clase pública es un módulo de código reutilizable, accesible desde cualquier punto del proyecto. Su función principal es ejecutar tareas específicas, como reproducir un archivo de audio. Imagina una función universal que se puede usar en cualquier parte del proyecto.

Paso 1. Ejecutar Unity y crear un nuevo proyecto.

Abra la aplicación Unity Hub.

Haga clic en “New Project”.

Elija la plantilla “3D (Core)”.

Escriba el siguiente nombre del proyecto: “U5-Cube-39fl-plus”.

Paso 2. Cree el Canvas UI:

1. Add (añada) un Canvas:

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”. Seleccione UI > Canvas. Esto creará un “Canvas” (lienzo) en la escena.

- Busque el apartado “**Render Mode**” en el Inspector (recuerde que el Canvas debe estar seleccionado en “Hierarchy”) y asegúrese que el elemento “**Canvas component**” tiene activada la opción “**Screen Space - Overlay**”.

2. Add (añada) un UI Element (Opcional):

- En el editor, haga clic con el botón derecho del ratón en la ventana “Hierarchy”. Seleccione cualquier elemento UI, haga clic en UI > Text, Button, o Image (esto añadirá el elemento elegido en el Canvas).

Paso 3. Añada un cuboide de referencia de 5m x 1m x 5m:

1. Añada un cuboide (Cube):

- En el editor de Unity, haga clic con el botón derecho del ratón en la ventana “Hierarchy”.
- Seleccione 3D Object > Cube. Esto creará un “cubo” en la escena.

2. Ajuste Cube's Scale y cambie su nombre a “Cube100”:

- Seleccione “Cube100” en la ventana Hierarchy.
- En el Inspector, ajuste (en el componente “Transform”) los valores de “Scale” a `(5, 1, 5)`.

3. Cambie la Posición del objeto “Cube100”:

- Cambie la posición del cubo en el Inspector para colocarlo donde usted desee. Originalmente lo hemos dejado en la posición (0,0,0).

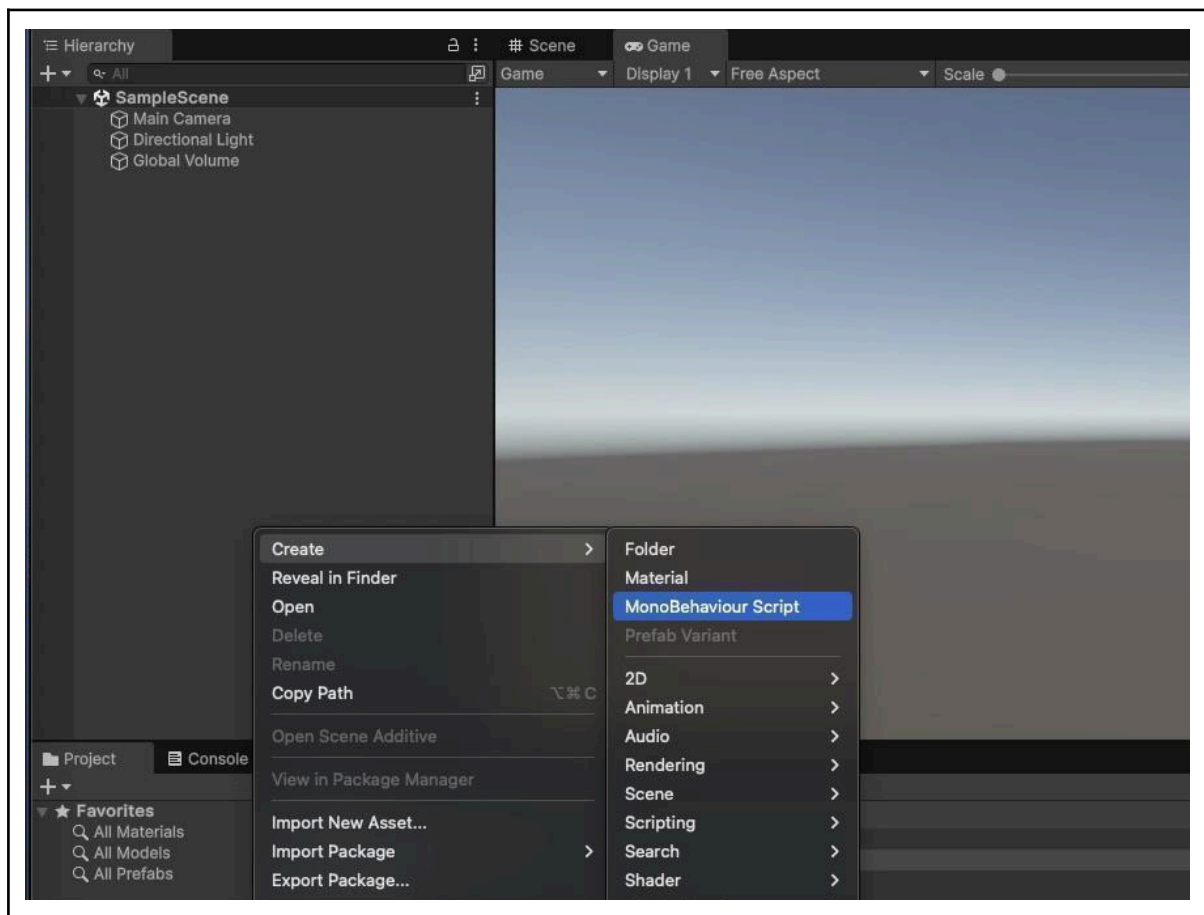
Este cubo lo haremos invisible más adelante. Lo hemos creado para tener un objeto de referencia para nuestro diseño.

Reto 1: Crear un texto sobre un cuboide

Para visualizar la cadena "Pulse <1>" en la cara frontal de un nuevo objeto `Cube101`, utilizaremos el componente `TextMesh`. En la página siguiente verá cómo creamos el script. Analice el ejemplo y realice los cambios oportunos:

En la ventana "Project window" (ventana de proyecto), haga clic con el botón derecho del ratón dentro de "Assets Folder" (carpeta de recursos).

Seleccione "Create" y después seleccione "MonoBehaviour Script". Se habrá creado un archivo C# llamado "NewMonoBehaviourScript". A continuación renombre el script que se acaba de crear como "createCube1".



Haga doble clic en el archivo “.cs” que acaba de renombrar y copie el siguiente programa:

```
/// CreateCube1.cs

using UnityEngine;
using TMPro;

public class CreateCube1 : MonoBehaviour
{
    [Header("Cube Settings")]
    [SerializeField] private string cubeName = "Cube101";
    [SerializeField] private string cubeTag = "101";
    [SerializeField] private Vector3 cubePosition = Vector3.zero;
    [SerializeField] private Vector3 cubeScale = new Vector3(5, 1, 5);
    [SerializeField] private bool enablePhysics = false;
    [SerializeField] private Color cubeColor = Color.white;

    [Header("Text Settings")]
    [SerializeField] public string text_in_the_cube = "Tu texto aquí"; //find esta var. mas adelante
    [SerializeField] private Color textColor = Color.black;
    [SerializeField] private float textSize = 14f; // Default size 14
    [SerializeField] private float textOffset = 0.01f; // Small offset from surface

    void Awake()
    {
        CreateCubeInCenter();
    }

    /// [el código continúa en la siguiente página. . . ]
```

```

void CreateCubeInCenter()
{
    if (GameObject.Find(cubeName) != null)
    {
        Debug.LogWarning($"{cubeName} ya existe!"); return;
    }

    // Create cube
    GameObject cube = GameObject.CreatePrimitive(PrimitiveType.Cube);
    cube.name = cubeName;
    cube.tag = cubeTag;
    cube.transform.position = cubePosition;
    cube.transform.localScale = cubeScale;
    cube.GetComponent<Renderer>().material.color = cubeColor;

    if (enablePhysics)
    {
        cube.AddComponent<Rigidbody>();
    }

    // Crear y configurar TextMeshPro
    GameObject textObject = new GameObject("CubeText");
    textObject.transform.SetParent(cube.transform);

    // Probamos DOS posiciones: Position text on front face (Z+ direction)
    // textObject.transform.localPosition = new Vector3(0, 0, cubeScale.z/2 + textOffset);
    textObject.transform.localPosition = new Vector3(0, 0, -0.5f);

    // Ojo! Incluir la siguiente linea si el usuario no ve el texto correctamente
    //textObject.transform.rotation = Quaternion.Euler(0, 180, 0); // Face camera

    TextMeshPro textMesh = textObject.AddComponent<TextMeshPro>();

    text_in_the_cube = "Pulse <1>"; // Asignar el texto del num. 1 al cubo 101
    textMesh.text = text_in_the_cube;    // text que va en el cubo

    textMesh.color = textColor;

    textMesh.fontSize = textSize;
    textMesh.enableAutoSizing = true; // Enable auto-sizing: Muy importante
    textMesh.fontSizeMin = 12; // Minimum size when auto-sizing
    textMesh.fontSizeMax = textSize; // Maximum size (your default size)

    textMesh.alignment = TextAlignmentOptions.Center;
    textMesh.verticalAlignment = VerticalAlignmentOptions.Middle;

    // Ajustamos rect transform. Ojo! multiplied x 2 para que quepa el texto
    textMesh.rectTransform.sizeDelta = new Vector2(2*cubeScale.x, cubeScale.y);

    Debug.Log($"Cube created with text: '{text_in_the_cube}' (Auto-sized font)");
}
}

```

/// [fin del archivo CreateCube1.cs]

A continuación siga estos pasos:

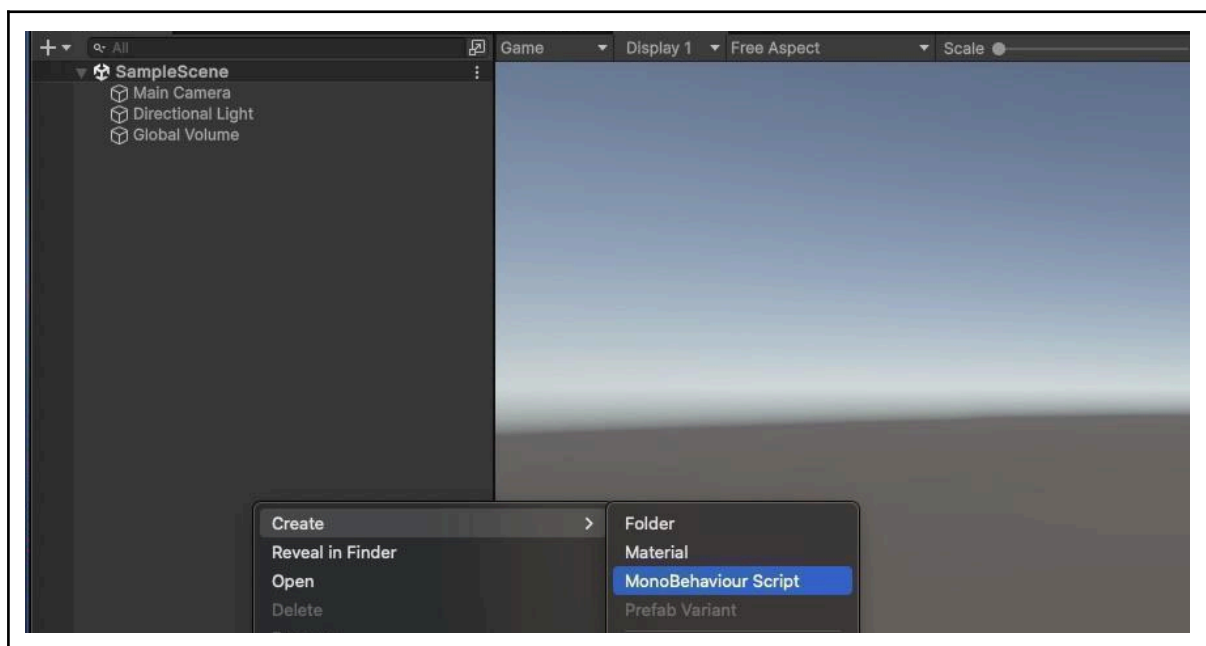
1. Adjunte su código (createCube1.cs) a un nuevo objeto vacío GameObject en su escena.
2. Renombre el GameObject recién creado. Llámelo “GO1_create Cube1”.

Reto 2: Incluir sonido a nuestro proyecto

Para hacer sonar un archivo de audio cuando se pulse la tecla correspondiente al número 1, necesitaremos crear un programa para controlar el teclado. En la página siguiente verá cómo creamos el script. Analice el ejemplo y realice los cambios oportunos:

En la ventana “Project window” (ventana de proyecto), haga clic con el botón derecho del ratón dentro de “Assets Folder” (carpeta de recursos).

Seleccione “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “keyboardManager”.



```

/// keyboardManager.cs

/// Recuerde crear una carpeta llamada "Sounds" en el directorio "Assets" de Unity de
/// su proyecto e incluir en la carpeta "Sounds" un archivo llamado "bip-sound-001.mp3".
/// Puede encontrar este archivo en cualquier web de audios gratuitos.
/// Cree un objeto AudioSource en "Hierarchy" llamado Audio_Source1 y cree un tag
/// llamado "Audio1" para este audio. Adjunte el fichero bip-source-001.mp3 al
/// campo Audio Resource del objeto Audio Source creado.

using UnityEngine;

public class KeyboardManager : MonoBehaviour
{
    private AudioSource audio1;

    void Start()
    {
        // Find the GameObject with the tag "Audio1" and get its AudioSource component
        GameObject audioObject = GameObject.FindWithTag("Audio1");
        if (audioObject != null)
        {
            audio1 = audioObject.GetComponent<AudioSource>();
        }
        else
        {
            Debug.LogWarning("No GameObject with tag 'Audio1' found.");
        }
    }

    void Update()
    {
        // Detecta when the '1' key is pressed
        if (Input.GetKeyDown(KeyCode.Alpha1))
        {
            if (audio1 != null)
            {
                PlayAudio1.Play();
                // Llamada al proc independiente PUBLIC STATIC CLASS PlayAudio1 that
                // can be found in PlayAudio1.cs
            }
            else
            {
                Debug.LogWarning("AudioSource not assigned.");
            }
        }
    }
}

```

Seleccione de nuevo “Create” y después seleccione “MonoBehaviour Script”. Se habrá creado un nuevo archivo C# llamado “NewMonoBehaviourScript”. A continuación renombre el script que se acaba de crear como “PlayAudio1”.

```

/// PlayAudio1.cs

using UnityEngine;

public static class PlayAudio1 // PUBLIC PROCEDURE. Can be called DESDE cualq SCRIPT.
{
    private static AudioSource _audioSource;
    private const string AudioTag = "Audio1";

    // Inicialización estática (se ejecuta una sola vez)
    static PlayAudio1()
    {
        GameObject audioObject = GameObject.FindWithTag(AudioTag);

        if (audioObject != null)
        {
            _audioSource = audioObject.GetComponent<AudioSource>();
            if (_audioSource != null)
            {
                _audioSource.playOnAwake = false; // Asegura que no se reproduzca al inicio
            }
            else
            {
                Debug.LogError($"No AudioSource found on GameObject with tag
'{AudioTag}'.");
            }
        }
        else
        {
            Debug.LogError($"No GameObject with tag '{AudioTag}' found.");
        }
    }

    // Método público para reproducir el audio desde cualquier script
    public static void Play()
    {
        if (_audioSource != null && _audioSource.clip != null)
        {
            _audioSource.Play();
        }
        else
        {
            Debug.LogWarning("Audio1 not initialized or missing AudioClip.");
        }
    }

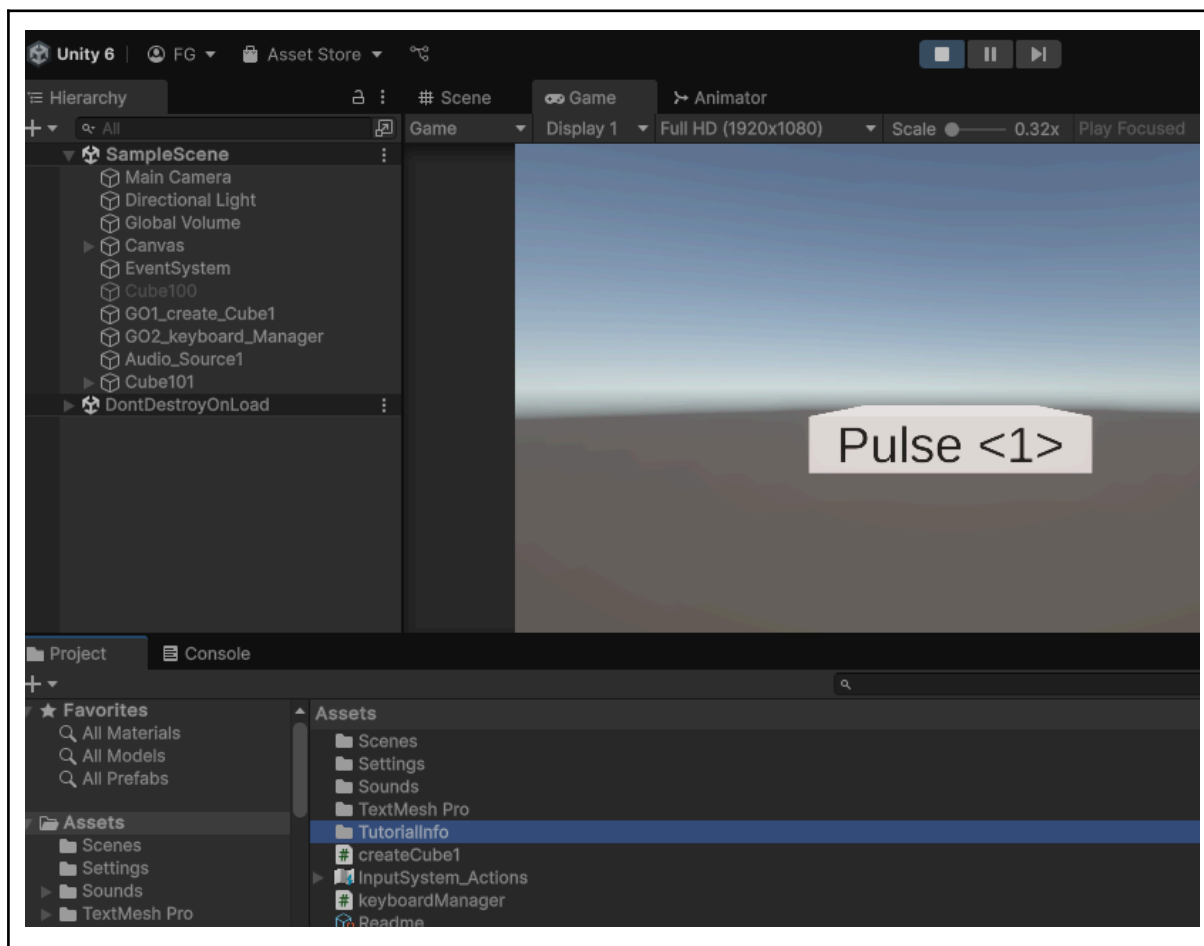
    // Método para asignar un AudioSource manualmente (opcional)
    public static void SetAudioSource(AudioSource audioSource)
    {
        _audioSource = audioSource;
        if (_audioSource != null)
        {
            _audioSource.playOnAwake = false;
        }
    }
}

```

A continuación siga estos pasos para comprobar que el código funciona:

1. Adjunte su código (**keyboardManager.cs**) a un nuevo objeto vacío GameObject en su escena.
2. Renombre el GameObject recién creado. Llámelo “**GO2_keyboard_Manager**”.
3. Ejecute la escena. El texto “**Pulse <1>**” deberá aparecer sobre el objeto `Cube100`.

Al ejecutar el proyecto y pulsar la tecla **1**, el archivo mp3 sonará. Si el audio suena **dos** veces, no olvide seleccionar **Audio_Source1** en “Hierarchy” y a continuación, en el Inspector, desmarcar la opción **Play On Awake**.



APÉNDICE A

test50_apendice_A.py

50 preguntas de repaso

```

# Test de repaso. 50 preguntas sobre Linux, Microsoft Windows y Python 3.
# Archivo: test50_apendice_A.py
# definir las funciones: test1() y felicitar() ahora:

def test1():
    puntos=0

    #PREGUNTA 1
    xxx = input("El comando DIR en Windows sirve para ver los archivos de una carpeta
(V/F)? : ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 2
    xxx = input("El comando DEL sirve para borrar archivos. (V/F)? : ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 3
    xxx = input("Las variables en Python pueden considerarse contenedores de datos. (V/F)? :
")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 4
    xxx = input("En un programa Python, debe asignarse un nombre diferente a cada variable.
(V/F)? : ")
    if xxx=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 5
    x = input("Para ver el valor de la variable X en Python, podemos usar la instrucción
print(X). (V/F)? : ")
    if x=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 6
    x = input("Para ver el valor de la variable Z en Python, podemos usar la instrucción
print(Z). (V/F)? : ")
    if x=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif

    #PREGUNTA 7
    x = input("Para ver el valor de la cadena Y en Python, podemos usar la instrucción
print(Y). (V/F)? : ")
    if x=="V":
        print("Respuesta correcta. Tienes 1 punto.")
        puntos=puntos+1
    #endif
    ##### continúa en la hoja siguiente

```



```

#PREGUNTA 8
x = input("Para ver el valor de la cadena YZ en Python, podemos usar la instrucción
print(YZ). (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 9
x = input("A una variable en Python se le puede asignar el valor True de tipo booleano
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 10
x = input("A una variable en Python se le puede asignar el valor False de tipo booleano
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 11
x = input("A una variable n en Python se le puede asignar el valor 1 de tipo entero
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 12
x = input("A una variable n en Python se le puede asignar el valor 0 de tipo entero
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 13
x = input("Los programas Python se almacenan en ficheros de texto con la extensión .py
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

```

continúa en la hoja siguiente

```

#PREGUNTA 14
x = input(" El editor de textos vi es muy utilizado por los programadores de Python en
Linux (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 15
x = input("En la década de los 90, numerosos programadores utilizaban el editor de
textos del famoso Comandante Norton (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 16
x = input("El Comandante Norton fue el programador del archivo ejecutable cn.exe para
MS-DOS (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 17
x = input("El archivo cn.exe para MS-DOS puede editar archivos con la extensión .py
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 18
x = input("El archivo cn.exe para MS-WINDOWS puede editar archivos con la extensión .py
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 19
x = input("EL programa equivalente en Linux del archivo cn.exe para MS-Windows se llama
Midnight Commander (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

```

continúa en la hoja siguiente

```

#PREGUNTA 20
x = input("El archivo Midnight Commander Editor para Linux puede editar archivos con la
extensión .py (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 21
x = input("El archivo .py de nuestro programa Python puede funcionar en Linux haciendo
muy pocos cambios (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 22
x = input("Para ejecutar en el terminal el archivo.py de nuestro programa Python
escribiremos: python3 archivo.py (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 23
x = input("Para utilizar el modulo FLET en un archivo.py necesitamos Python version 3.8
(V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 24
x = input("Para utilizar el módulo FLET en un proyecto necesitamos instalarlo antes
desde el terminal de Windows (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 25
x = input("Para utilizar el módulo FLET en un proyecto necesitamos instalarlo con la
orden: pip install flet (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#####

```

```

#PREGUNTA 26
x = input("Para utilizar el módulo FLET en un proyecto necesitamos instalarlo con la
orden: pip install flet (V/F)? : ")
if x=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 27
x = input("FLET es un conjunto de herramientas (framework) para Python que facilita al
programador el desarrollo del interfaz de usuario (frontend) (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 28
x = input("Cualquier programador puede usar FLET libremente porque es un conjunto de
herramientas Open-Source (software de código abierto). (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 29
x = input("El software de código abierto es un software cuyo conjunto de instrucciones
puede ser modificado o utilizado libremente. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

#PREGUNTA 30a
x = input("El software de código abierto fomenta la colaboración entre equipos de
desarrolladores. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

```

continúa en la hoja siguiente

```

#PREGUNTA 30b
x = input("Python es un lenguaje de programación interpretado. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 31!
x = input("C# soporta la programación orientada a objetos. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 32
x = input("La función 'len()' se usa para obtener la longitud de una lista en
Python. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 33
x = input("En Python, las listas no son inmutables. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 34!
x = input("El método 'estrip()' elimina los espacios en blanco al inicio y al final de
una cadena en Python. (V/F)? ")
if x=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 35
x = input("Python es case-sensitive, es decir, distingue entre mayúsculas y minúsculas.
(V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

```

continúa en la hoja siguiente

```

#PREGUNTA 36
x = input("El método 'append()' se usa para agregar un elemento al final de una lista
en Python. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 37!
x = input("Unity 6 puede conectarse con Unity Hub para gestionar proyectos creados con
diferentes versiones de Unity. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 38!
x = input("Unity 6 permite crear videojuegos basados en botones y mensajes de texto
utilizando C# (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 39
x = input("El método 'isnumeric()' devuelve True si todos los caracteres de una cadena
son numéricos en Python. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 40!
x = input("Unity 6 puede ejecutar código escrito en lenguaje C#. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 41
x = input("El método 'replace()' reemplaza todas las ocurrencias de una subcadena por
otra en una cadena en Python. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

```

continúa en la hoja siguiente

```

#PREGUNTA 42
x = input("El operador '**' se usa para la exponenciación en Python. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 43
x = input("El método 'strip()' elimina los espacios en blanco al inicio y al final de
una cadena en Python. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 44
x = input("El método 'join()' se usa para unir elementos de una lista en una cadena en
Python. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 45
x = input("El operador '/' en Python realiza una división entera. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 46
x = input("El método 'sort()' ordena una lista en Python de forma ascendente por
defecto. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

#PREGUNTA 47
x = input("En Python, 'lambda' se usa para crear funciones anónimas. (V/F)? ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#elseif

```

continúa en la hoja siguiente

```

#PREGUNTA 48
x = input("El método 'pop()' elimina y devuelve el último elemento de una lista en
Python. (V/F)? : ")
if x=="V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

##### la respuesta correcta a la pregunta siguiente es F

#PREGUNTA 49!
x = input("En Python, 'lamda' se usa para crear funciones anónimas. (V/F)? : ")
if x=="F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos=puntos+1
#endif

print("Tu nota en este test es:")
print(puntos)

##### definición de la función felicitar()
def felicitar():
    print("Felicitaciones, siga estudiando así.")
    print("Estamos orgullosos de usted.")

#### programa principal (main program)

xname = input("Dime tu nombre, por favor: ")
print("Hola, " + xname + "!")
a="Vamos a hacer un test sobre de 50 preguntas sobre"
b=" Microsoft Windows y Python 3."
print(a+b)
test1()
felicitar()
print("Fin de la prueba. Fin del archivo test50_apendice_B.py ")

##### fin

```


APÉNDICE B

¿Cómo podría mejorar el código de los ejemplos?

Busca los errores con ayuda de Copilot iA

Clue: Tests (con matrices) escritos en Python 3

```

# Test 1 de repaso sobre Python 3 con iA.
# Archivo: test50_apendice_B.py
#
# Este programa mejora la forma de guardar la información de las preguntas y las
# respuestas. Se implementa una lista donde se guardan las preguntas y las
# respuestas correctas.
# De este modo, no tenemos que repetir 50 veces los bloques if-endif para comprobar
# si las respuestas del usuario son correctas.
# Observe también que se ha implementado un bucle 'for' para recorrer la lista y
# hacer las preguntas. También se ha implementado el método upper()
# que convierte la entrada del usuario # a mayúsculas.
#
# Definir la función:

def test_verdadero_falso():
    """ Lista de preguntas y respuestas
    preguntas = [
        {
            "pregunta": "Python es un lenguaje de programación interpretado. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "En Python, las listas son inmutables. (V/F)",
            "respuesta": "F"
        }
    ]

    puntos = 0

    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print(";Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuacion final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_falso()

if __name__ == "__main__":
    test_verdadero_falso()

# Observe que únicamente se ha escrito el código para dos preguntas.
# puede seguir añadiendo preguntas y respuestas a la lista de preguntas.
#
##### fin del archivo test50_apendice_B.py

```

```

## Test 2 de repaso sobre Python 3.
## test_printXporX.py --> Test sobre la instruccion print(X) y print(X*X)

xname = input("Dime tu nombre, por favor: ")

print("Hola, " + xname + "!"); print("Vamos a hacer un test sobre Microsoft Windows")

puntos=0

##1er pregunta
A="imprimir una variable llamada X en pantalla: print o (X):"
xxx = input("""Cual es la primera orden que debemos teclear para """ + A)

if xxx=="print":
    print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
#elseif

##2a pregunta
A="imprimir una variable llamada X en pantalla: print o (X):"
xxx = input("""Cual es la segunda parte de la orden que debemos teclear para """ + A)

if xxx=="(X)":
    print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
#elseif

##3a pregunta
A="imprimir el cuadrado de la variable X en pantalla: print o (X*X):"
xxx = input("""Cual es la primera orden que debemos teclear para """+A)

if xxx=="print":
    print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
#elseif

##4a pregunta
A="imprimir el cuadrado de la variable X en pantalla: print o (X*X):"
xxx = input("""Cual es la segunda parte de la orden que debemos teclear para """+A)

if xxx=="(X*X)":
    print("Respuesta correcta. Tienes 1 punto."); puntos=puntos+1
#elseif

print("Tu nota final en este test es:"); print(puntos)

#### fin del archivo

```

```
# Test 3a de repaso sobre Python 3.
# Archivo: testMarketing.py

# Recuerde que el objetivo financiero principal y fundamental
# de cualquier empresa comercial o de ventas es la Maximización de
# los Beneficios (Ganancias) a largo plazo.

a= "El marketing es el arte de encontrar compradores para sus"
b= "productos y servicios obteniendo"
c= "beneficios"
d= "pérdidas."

print(a)
print(b)
print(c)


# Importante:

# El marketing es el proceso de identificar las necesidades y deseos de

# los clientes, y luego crear y ofrecer productos o servicios que

# satisfagan esas necesidades, generando beneficios tanto para la empresa

# como para los consumidores. Todo ello implica, desde la investigación

# de mercado y el desarrollo de productos hasta la publicidad, las

# ventas y la distribución. El marketing trata de conectar con

# los clientes ofreciendo valor y construyendo relaciones a largo plazo.
```

```

# Test 3b de repaso sobre Python 3.
# Archivo: marketing5.py
# Observe que se ha implementado un bucle 'for' para recorrer la lista y
# hacer las preguntas. Corrija los posibles errores en el código.
#
# Definir la función:
def test_verdadero_falso():

    preguntas = [
        {
            "pregunta": "El marketing comienza con entender quién es el comprador y qué
problema necesita resolver. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Crear un producto o servicio útil no es suficiente; hay que
comunicar su valor de forma efectiva. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "El enfoque debe estar en el valor percibido por el cliente, no solo
en las características del producto. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "El marketing exitoso implica dirigirse a un público específico,
entender sus motivaciones y ofrecerles una solución clara. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "El proceso de compra no es una interacción de valor, donde el
cliente evalúa lo que recibe frente a lo que paga. (V/F)",
            "respuesta": "F"
        }
    ]

    puntos = 0

    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print(";Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuacion final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_falso()

if __name__ == "__main__":
    test_verdadero_falso()

#### Observe que únicamente se ha escrito el código para 5 preguntas. #### fin del archivo

```

```

# Test 3c de repaso sobre Python 3 y seguridad en dispositivos Android.
# Archivo: moreAndroidSecurity.py
# Observe que se ha implementado un bucle 'for' para recorrer la lista y
# hacer las preguntas. Corrija los posibles errores en el código.
#
# Definir la función:
def test_verdadero_falso():

    preguntas = [
        {
            "pregunta": "Desactivar Gemini en una tablet con Android incrementa la privacidad
del usuario. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Desactivar el modo AI en <Google search> incrementa la privacidad
del usuario. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Desactivar el <rastreo de ubicación> en dispositivos Android
incrementa la privacidad del usuario. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Desactivar el acceso a la cámara y el micrófono de dispositivos
Android incrementa la privacidad del usuario. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Utilizar Chrome y la aplicación <Google Password Manager> incrementa
la privacidad del usuario. (V/F)",
            "respuesta": "F"
        }
    ]

    puntos = 0

    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print("¡Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuacion final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_falso()

if __name__ == "__main__":
    test_verdadero_falso()

#### Observe que únicamente se ha escrito el código para 5 preguntas. #### fin del archivo

```

```

# Test 4 de repaso.
# Archivo: marketing5X.py
# Empezamos por definir la función (corrija los posibles errores en el código):

def test_verdadero_falso():

    preguntas = [
        {
            "pregunta": "El valor no está en el producto en sí, sino en cómo el cliente lo
percibe. La experiencia de compra y la reputación influyen directamente en esa percepción.
(V/F)",
            "respuesta": "v"
        },
        {
            "pregunta": "El precio debe reflejar el valor percibido, no solo el coste de
producción. Un precio demasiado bajo puede disminuir la percepción de calidad. (V/F)",
            "respuesta": "v"
        },
        {
            "pregunta": "Para destacar en el mercado, es esencial ofrecer algo único. Esto
puede ser una característica, una experiencia o incluso una historia detrás del producto.
(V/F)",
            "respuesta": "v"
        },
        {
            "pregunta": " La confianza del cliente es un activo valioso. Las reseñas,
recomendaciones y la coherencia en la calidad construyen una reputación sólida. (V/F)",
            "respuesta": "v"
        },
        {
            "pregunta": "Delegar tareas a expertos (como diseño, publicidad o logística)
puede mejorar la eficiencia y la calidad del producto final. (V/F)",
            "respuesta": "v"
        }
    ]

    ##### Un cliente satisfecho no solo vuelve, sino que recomienda.

    puntos = 0

    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print(";Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuacion final es: {puntos} de {len(preguntas)}")

##### end def test_verdadero_falso()

if __name__ == "__main__":
    test_verdadero_falso()

# El marketing exitoso se basa en relaciones duraderas, no en ventas únicas.
# Observe que únicamente se ha escrito el código para 5 preguntas.##### fin del archivo

```

```

# Test 5 de repaso
# Archivo: marketing5gaps.py          [Preguntas tipo COMPLETA LA FRASE. . .)
# Se ha implementado un bucle 'for' para recorrer la lista y hacer las preguntas.
# Empezamos por definir la función (corrija los posibles errores en el código):

def test_verdadero_falso():

    preguntas = [
        {
            "pregunta": "Complete la frase: Un cliente satisfecho no solo vuelve, sino
que...",
            "respuesta": "recomienda"
        },
        {
            "pregunta": "Complete la frase: Delegar tareas a expertos (como diseño,
publicidad o logística) puede mejorar la eficiencia y la calidad del producto...",
            "respuesta": "final"
        }
    ]

    puntos = 0

    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Completa la frase: ").strip().lower()

        if respuesta_usuario == pregunta["respuesta"]:
            print(";Correcto! Tienes un punto.")
            puntos += 1
        else:
            print(f"Incorrecto. La respuesta correcta era: {pregunta['respuesta']}")

    print(f"\nTu puntuacion final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_falso()

if __name__ == "__main__":
    test_verdadero_falso()

# Recuerde: Un cliente satisfecho no solo vuelve, sino que recomienda.

```



```

# Test 6 de repaso
# Archivo: marketing5options.py
# Se ha implementado un cambio: Ahora el usuario puede elegir entre tres opciones
correctas.
# Empezamos por definir la función (corrija los posibles errores en el código):

def test_verdadero():

    preguntas = [
        {
            "pregunta": "Complete la frase: Un cliente satisfecho no solo vuelve, sino
que...",
            "opciones": ["recomienda", "manda", "firma"],
            "respuesta": "recomienda"
        },
        {
            "pregunta": "Complete la frase: Delegar tareas a expertos (como diseño,
publicidad o logística) puede mejorar la eficiencia y la calidad del producto...",
            "opciones": ["final", "inicial", "intermedio"],
            "respuesta": "final"
        }
    ]

    puntos = 0

    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        print("Opciones:")
        for idx, opcion in enumerate(pregunta["opciones"], 1):
            print(f"{idx}. {opcion}")
        r = input("Elige la opción correcta (escribe la palabra completa):
").strip().lower()

        if r == pregunta["respuesta"]:
            print(";Correcto! Tienes un punto.")
            puntos += 1
        else:
            print(f"Incorrecto. La respuesta correcta era: {pregunta['respuesta']}")

    print(f"\nTu puntuación final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero()

if __name__ == "__main__":
    test_verdadero()

```

```

# Test 7 de repaso sobre ciberseguridad y envío de documentación por email
# Archivo: marketing7.py [ Corrija los posibles errores en el código. ]
#
# Definir la función:

def test_verdadero_false():

    preguntas = [
        {
            "pregunta": "Debemos pasar la foto del DNI a blanco y negro si vamos a enviarla a
un proveedor por email. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Si vamos a enviar un DNI a un proveedor por email, difuminaremos
antes la fecha de expedición y la firma. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Si vamos a enviar el DNI por email, difuminaremos antes la firma.
(V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Si vamos a enviar el DNI por email, difuminaremos antes en el
reverso la numeración del equipo del DNI. (V/F)",
            "respuesta": "V"
        },
        {
            "pregunta": "Si vamos a enviar un DNI por email, difuminaremos antes en el
reverso los datos de las últimas tres líneas. (V/F)",
            "respuesta": "V"
        }
    ]

    puntos = 0

    for i, pregunta in enumerate(preguntas):
        print(f"Pregunta {i + 1}: {pregunta['pregunta']}")
        respuesta_usuario = input("Tu respuesta: ").upper()

        if respuesta_usuario == pregunta["respuesta"]:
            print(";Correcto! Tienes un punto.")
            puntos += 1
        else:
            print("Incorrecto.")

    print(f"\nTu puntuacion final es: {puntos} de {len(preguntas)}")

#### end def test_verdadero_false()

if __name__ == "__main__":
    test_verdadero_false()

##### fin del archivo

```

```

# Script 8 de repaso sobre LA COMUNICACIÓN EN ENTORNOS TIC (módulo 1)
# Archivo: hello-tabulateTIC1.py [ Corrija los posibles errores en el código.
]
#
import os
os.system('cls' if os.name == 'nt' else 'clear')

from tabulate import tabulate

### SLIDE 1 #####

table_data = [
    ["COMUNICACIÓN EN ENTORNOS TIC", " "],
    [" "],
    [" "],
    ["Mod. (I)", " "],
    [" "],
    ["COMUNICACIÓN INTERPERSONAL SEGÚN EL MODELO", " "],
    ["CLIENTE-SERVIDOR", " "]
]

headers = [" ", "Pag.1"] ### # Definir table headers (encabezados)
table1 = tabulate(table_data, headers, tablefmt="fancy_grid") # Crear table
print(table1) ### this is slide 1
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

#####

### SLIDE 2

table_data = [
    ["1. COMUNICACIÓN EN ENTORNOS TIC", " "],
    [" "],
    ["1.1. Barreras de comunicación entre el cliente y el personal TIC", " "],
    [" "],
    [" - Dificultad del cliente en expresar lo que quiere: ", " "],
    [" Muchos clientes no dominan el lenguaje técnico y ello", " "],
    [" dificulta que comuniquen sus necesidades correctamente", " "],
    [" "],
    [" - Dificultad del informático para explicarse:", " "],
    [" El personal TIC suele utilizar un lenguaje muy técnico", " "],
    [" que el cliente no entiende y ello genera confusión.", " "]
]

headers = [" ", "Pag.2"] ### # Definir table headers (encabezados)
table2 = tabulate(table_data, headers, tablefmt="fancy_grid") # Crear table
print(table2) ### this is slide 2
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

#####

```

SLIDE 3

```

table_data = [
    ["1.2. Personalización del lenguaje", " "],
    [" ", " "],
    [" - Adaptar el lenguaje técnico al nivel de conocimiento del cliente:", " "],
    ["    Utilizar términos sencillos y evitar jergas técnicas.", " "],
    [" ", " "],
    [" - Uso de ejemplos y analogías:", " "],
    ["    Explicar conceptos técnicos mediante ejemplos cotidianos", " "],
    ["    que el cliente pueda entender fácilmente.", " "],
    [" ", " "],
    [" - Confirmación de comprensión:", " "],
    ["    Preguntar al cliente si ha entendido la información", " "],
    ["    y aclarar cualquier duda que pueda tener.", " "],
    [" ", " "],
    [" - Claridad del mensaje:", " "],
    ["    Explicar conceptos de forma sencilla evitando", " "],
    ["    ambigüedades.", " "]
]

headers = [" ", "Pag.3"]      ### # Definir table headers (encabezados)
table3 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table3)                ## this is slide 3
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

```

```
#####
```

SLIDE 4

```

table_data = [
    ["1.3. Comunicación en entornos remotos", " "],
    [" ", " "],
    [" - Uso de herramientas de comunicación adecuadas:", " "],
    ["    Utilizar videollamadas, correos electrónicos", " "],
    ["    o chats según la preferencia del cliente.", " "],
    [" ", " "],
    [" - Establecimiento de horarios claros:", " "],
    ["    Acordar horarios para reuniones virtuales", " "],
    ["    y respetarlos para evitar malentendidos.", " "],
    [" ", " "],
    [" - Documentación detallada:", " "],
    ["    Proporcionar documentación escrita clara", " "],
    ["    que el cliente pueda consultar posteriormente.", " "],
    [" ", " "],
    [" - ¡Importante!", " "],
    ["    El uso de herramientas digitales como las videollamadas", " "],
    ["    requiere mayor precisión y empatía, que que se pierde", " "],
    ["    mucha información no verbal.", " "]
]

headers = [" ", "Pag.4"]      ### # Definir table headers (encabezados)
table4 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table4)                ## this is slide 4
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
#####

```

SLIDE 5

```
table_data = [
    ["2. APLICACIÓN DE LA INTELIGENCIA RELACIONAL PARA UN DIÁLOGO FLUIDO CON EL CLIENTE", "
"],
    [" ", " "],
    ["2.1. Detección de necesidades del analfabeto TIC", " "],
    [" ", " "],
    [" - Inteligencia emocional: Reconocer emociones propias y ajenas para mejorar la
interacción", " "],
    [" - Empatía: Ponerse en el lugar del cliente para comprender sus dificultades", " "],
    [" - Técnicas para animar al receptor a hablar: Crear ambiente cómodo que invite a
participar", " "],
    [" - Uso de analogías: Explicar conceptos técnicos mediante comparaciones con
situaciones", " "],
    [" cotidianas", " "]
]
```

```
headers = [" ", "Pag.5"]      ### # Definir table headers (encabezados)
table5 = tabulate(table_data, headers, tablefmt="fancy_grid")    # Crear table
print(table5)                ## this is slide 5
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

```
#####
```

SLIDE 6

```
table_data = [
    [" ", " "],
    [" ", " "],
    ["2.2. Escucha activa", " "],
    [" ", " "],
    [" - Prestar atención real al cliente, sin interrumpir,", " "],
    [" mostrando interés y validando sus aportes.", " "],
    [" ", " "],
    [" ", " "],
    [" ", " "]
]
```

```
headers = [" ", "Pag.6"]      ### # Definir table headers (encabezados)
table6 = tabulate(table_data, headers, tablefmt="fancy_grid")    # Crear table
print(table6)                ## this is slide 6
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

```
#####
```

SLIDE 7

```
table_data = [
    [" ", " "],
    [" ", " "],
    ["2.3. Asertividad en la comunicación", " "],
    [" ", " "],
    [" - Expresar ideas con claridad y respeto, sin", " "],
    [" imponer ni ceder en exceso.", " "],
    [" ", " "],
    [" ", " "],
    [" ", " "]
]

headers = [" ", "Pag.7"]      ### # Definir table headers (encabezados)
table7 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table7)                ## this is slide 7
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

#####
```

SLIDE 8

```
table_data = [
    [" ", " "],
    [" ", " "],
    ["2.4. Diferentes maneras de preguntar", " "],
    [" ", " "],
    [" - Usar preguntas abiertas, cerradas o", " "],
    [" reflexivas según el objetivo de la conversación.", " "],
    [" ", " "],
    [" ", " "],
    [" ", " "]
]

headers = [" ", "Pag.8"]      ### # Definir table headers (encabezados)
table8 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table8)                ## this is slide 8
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

#####
```

SLIDE 9

```
table_data = [
    [" ", " "],
    [" ", " "],
    ["2.5. Comunicación no verbal", " "],
    [" ", " "],
    [" - Gestos, tono de voz y postura influyen en la percepción del", " "],
    [" mensaje, incluso en videollamadas.", " "],
    [" ", " "],
    [" ", " "],
    [" ", " "]
]

headers = [" ", "Pag.9"]      ### # Definir table headers (encabezados)
table9 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table9)                ## this is slide 9
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

#####
```

SLIDE 10

```
n=10
table_data = [
    [" ", " "],
    [" ", " "],
    ["2.6. Cultura del servicio", " "],
    [" ", " "],
    [" - Priorizar la satisfacción del cliente, mostrando,", " "],
    [" disposición, amabilidad y compromiso con la solución.", " "],
    [" ", " "],
    [" ", " "],
    [" ", " "]
]

headers = [" ", "Pag."+str(n)]
table10 = tabulate(table_data, headers, tablefmt="fancy_grid")
print(table10)
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

print(" ")
print("Fin de la presentación del módulo I. Gracias por su atención.")
print("\n")
```

```

# Script 9a de repaso sobre la implementación del módulo tkinter para entornos gráficos
# Archivo: hello-tabulateTIC2tk.py          [ Corrija los posibles errores en el código. ]

# Recuerda instalar en el terminal el módulo tk con la orden:
# pip install tk                          ## El módulo tk es el módulo tkinter para implementar un GUI
# Este script crea un diálogo (presentación) de una diapositiva (slide)

import tkinter as tk
from tkinter import messagebox

# Crear main window
root = tk.Tk()
root.title("tk: Productos TIC")
root.geometry("600x400")

# Define funciones para los botones

def show_message():
    # Create custom dialog window
    dialog = tk.Toplevel(root)
    dialog.title("Productos TIC")
    dialog.geometry("600x400")

    singleSpace="\u2800"          # Using invisible space for alignment
    doubleSpace="\u2800\u2800"
    tripleSpace="\u2800\u2800\u2800"
    sentences1 = [
        "MODULO I",
        " ",
        "PRESENTACION DEL PRODUCTO TIC AL CLIENTE",
        " "
    ]
    message = "\n".join(sentences1)

    # Create label with Courier font
    label = tk.Label(
        dialog,
        text=message,
        font=("Courier", 12),
        justify=tk.LEFT,
        padx=20,
        pady=20
    )
    label.pack(expand=True)

    # Add OK button to close dialog
    ok_button = tk.Button(
        dialog,
        text="OK",
        command=dialog.destroy,
        font=("Courier", 10)
    )
    ok_button.pack(pady=10)

    # Make dialog modal
    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

```



```
def close_window():
    root.destroy()

# Crear botones
button1 = tk.Button(root, text="Open Presentation", command=show_message)
button2 = tk.Button(root, text="Close", command=close_window)

# Place botones
button1.pack(pady=10)
button2.pack(pady=10)

root.mainloop()          ##### Run app

### Fin del script 9a
```

```

# Script 9b de repaso sobre COMUNICACIÓN EN ENTORNOS TIC (módulo 2)
# Archivo: hello-tabulateTIC222tk.py          [ Corrija los posibles errores en el código. ]

# Recuerda instalar en el terminal el módulo tk con la orden:
# pip install tk                               ## El módulo tk es el módulo tkinter para implementar el GUI
# Este script crea una serie de diálogos encadenados (presentación) de 6 slides
# Utilizaremos el script 9a como base para desarrollar este ejemplo:

import tkinter as tk
from tkinter import messagebox

# Crear main window
root = tk.Tk()
root.title("tk: Productos TIC")
root.geometry("600x400")

# Define funciones para los botones
def create_navigation_buttons(dialog, next_func=None, prev_func=None):
    """Helper function to create navigation buttons"""
    button_frame = tk.Frame(dialog)
    button_frame.pack(pady=10)

    if prev_func:
        back_button = tk.Button(
            button_frame,
            text="Back",
            command=lambda: [dialog.destroy(), prev_func()],
            font=("Arial", 10)
        )
        back_button.pack(side=tk.LEFT, padx=5)

    if next_func:
        next_button = tk.Button(
            button_frame,
            text="Next",
            command=lambda: [dialog.destroy(), next_func()],
            font=("Arial", 10)
        )
        next_button.pack(side=tk.LEFT, padx=5)

### Continúa en la siguiente página. . .

```

```

def close_window():
    root.destroy()

def show_message1():
    dialog = tk.Toplevel(root)
    dialog.title("Productos TIC")
    dialog.geometry("600x400")

    sentences1 = [
        "MODULO II",
        " ",
        "PRESENTACION DEL PRODUCTO TIC AL CLIENTE",
        " "
    ]
    message = "\n".join(sentences1)

    label = tk.Label(
        dialog,
        text=message,
        font=("Courier", 12),
        justify=tk.LEFT,
        padx=20, pady=20
    )
    label.pack(expand=True)

    create_navigation_buttons(dialog, next_func=show_message2)

    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

```

Continúa en la siguiente página. . .

```

def show_message2():
    dialog = tk.Toplevel(root)
    dialog.title("Productos TIC")
    dialog.geometry("600x400")

    sentences2 = [
        "1. CUIDADO DE LA IMAGEN PERSONAL Y PROFESIONAL",
        " ",
        "La primera impresión es crucial al presentar productos TIC al cliente.",
        " ",
        "Una imagen profesional y cuidada genera confianza y credibilidad.",
        " ",
        "Esto implica:",
        " ",
        "- Vestir de manera adecuada: Usar ropa limpia y apropiada para el entorno.",
        "- Mantener una buena higiene y cuidado personal.",
        "- Cuidar el lenguaje corporal: Adoptar una postura abierta y segura.",
        "- Hablar con claridad y confianza (comunicación verbal).",
        "- Además, debemos cuidar cómo nos presentamos a través del ordenador.",
        "- Colocar un fondo ordenado, con buena iluminación y una conexión estable."
    ]
    message = "\n".join(sentences2)

    label = tk.Label(
        dialog, text=message,
        font=("Courier", 12),
        justify=tk.LEFT,
        padx=20, pady=20
    )
    label.pack(expand=True)

    create_navigation_buttons(dialog, next_func=show_message3, prev_func=show_message1)

    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

```

Continúa en la siguiente página. . .

```

def show_message3():
    dialog = tk.Toplevel(root)
    dialog.title("Productos TIC")
    dialog.geometry("600x400")

    sentences3 = [
        "2. VENTA DEL PRODUCTO TIC",
        " ",
        " ",
        "La venta del producto TIC requiere:",
        " ",
        "- Atención al beneficio del cliente",
        "- Atención al beneficio del usuario final",
        "- Enfoque en el resultado",
        "- Manejo de expectativas y objeciones"
    ]
    message = "\n".join(sentences3)

    label = tk.Label(
        dialog, text=message,
        font=("Courier", 12),
        justify=tk.LEFT,
        padx=20, pady=20
    )
    label.pack(expand=True)

    create_navigation_buttons(dialog, next_func=show_message4, prev_func=show_message2)

    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

```

Continúa en la siguiente página. . .

```

def show_message4():
    dialog = tk.Toplevel(root)
    dialog.title("Productos TIC")
    dialog.geometry("600x400")

    sentences4 = [
        "3. LA PERSUASIÓN. APLICACIÓN DE TÉCNICAS DE PERSUASIÓN",
        " ",
        " ",
        "La venta del producto TIC requiere:",
        " ",
        "- Estructuración adecuada para la venta: ICARO + cierre",
        "- Transmisión de confianza y seguridad",
        "- Entender la problemática del cliente digital",
        "- Entender la perspectiva del cliente ante las soluciones que se le ofrecen",
        "- Manejar la comunicación no verbal efectivamente en las ventas remotas",
        "- Manejar la comunicación no verbal en la venta exitosa",
        "- Entender la diferencia entre convencimiento y manipulación"
    ]
    message = "\n".join(sentences4)

    label = tk.Label(
        dialog, text=message,
        font=("Courier", 12),
        justify=tk.LEFT,
        padx=20, pady=20
    )
    label.pack(expand=True)

    create_navigation_buttons(dialog, next_func=show_message5, prev_func=show_message3)

    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

```

Continúa en la siguiente página. . .

```

def show_message5():
    dialog = tk.Toplevel(root)
    dialog.title("Productos TIC")
    dialog.geometry("600x400")

    sentences5 = [
        "4. LA NEGOCIACIÓN EXITOSA EN EL MARCO GANAR-GANAR",
        " ",
        " ",
        "La venta exitosa del producto TIC requiere:",
        " ",
        "- Negociación de los extras:",
        " ",
        "Utilidades adicionales, mejoras del producto, etc.",
        " ",
        "- Negociación del tiempo de desarrollo y de los recursos",
        " ",
        " "
    ]
    message = "\n".join(sentences5)

    label = tk.Label(
        dialog, text=message,
        font=("Courier", 12),
        justify=tk.LEFT,
        padx=20, pady=20
    )
    label.pack(expand=True)

    create_navigation_buttons(dialog, next_func=show_message6, prev_func=show_message4)

    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

```

Continúa en la siguiente página. . .

```

def show_message6():
    dialog = tk.Toplevel(root)
    dialog.title("Productos TIC")
    dialog.geometry("600x400")

    sentences6 = [
        "5. PRESENTACIÓN DE UN PRODUCTO TIC",
        " ",
        " ",
        " La venta exitosa del producto TIC requiere:",
        " ",
        " - Herramientas TIC de apoyo",
        " ",
        " - Manejo de una herramienta relevante en el sector (ultima versión del
producto).",
        " ",
        " ",
        " ",
        " ",
        "FIN DE LA PRESENTACIÓN.",
        " "
    ]

    message = "\n".join(sentences6)

    label = tk.Label(
        dialog, text=message,
        font=("Courier", 12),
        justify=tk.LEFT,
        padx=20, pady=20
    )
    label.pack(expand=True)

    create_navigation_buttons(dialog, prev_func=show_message5) # No next function for last
slide

    dialog.transient(root)
    dialog.grab_set()
    root.wait_window(dialog)

# Crear botones PARA MAIN SCREEN
button1 = tk.Button(root, text="Open Presentation", command=show_message1)
button2 = tk.Button(root, text="Close", command=close_window)

# Place botones
button1.pack(pady=10)
button2.pack(pady=10)

root.mainloop()          ##### Run app

### Fin del script 9b

```



```

# Script 10 de repaso sobre COMUNICACIÓN EN ENTORNOS TIC (módulo 3)
# Archivo: hello-tabulateTIC3.py          [ Corrija los posibles errores en el código. ]

import os
os.system('cls' if os.name == 'nt' else 'clear')

from tabulate import tabulate

### SLIDE 1 ##### SCRIPT 10.

table_data = [
    ["COMUNICACIÓN EN ENTORNOS TIC", " ", " "],
    [ " ", " " ],
    [ " ", " " ],
    ["Mod. (III)", " ", " "],
    [ " ", " " ],
    ["DISEÑO DE LA APLICACIÓN TIC", " ", " "],
    [ " ", " ", " " ]
]

headers = [ " ", "Pag.1"]          ### # Definir table headers (encabezados)

table1 = tabulate(table_data, headers, tablefmt="fancy_grid")    # Crear table

print(table1)                    ## this is slide 1

xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

#####

### SLIDE 2

table_data = [
    ["1. ORGANIZACIÓN DEL TRABAJO EN EL DISEÑO DE APPS", " ", " "],
    [ " ", " " ],
    [" - Planificación del trabajo: Intro, analisis, diseño, implementacion, evaluacion", " " ],
    [ " ", " " ],
    [" - Ambientes de desarrollo eficientes", " ", " "],
    [ " ", " " ],
    [" - Tecnicas de gestion de tareas en equipos presenciales y remotos", " ", " "],
    [ " ", " " ],
    [" - Distribución del trabajo en el equipo de desarrollo", " ", " "],
    [ " ", " " ],
    [" - Manejo de apps relevantes en el sector con la última versión del producto", " ", " " ]
]

headers = [ " ", "Pag.2"]          ### # Definir table headers (encabezados)
table2 = tabulate(table_data, headers, tablefmt="fancy_grid")    # Crear table
print(table2)                    ## this is slide 2
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

### Continúa en la siguiente página. . .

```

```
#####
```

```
### SLIDE 3
```

```
table_data = [
    ["2. GESTIÓN DEL TIEMPO EN UN PROYECTO TIC", " "],
    [" ", " "],
    [" - Requisitos funcionales y no funcionales", " "],
    [" - Estimación del tiempo de desarrollo", " "],
    [" - Priorizar tareas y gestión de imprevistos", " "],
    [" - Herramienta GTD -Getting things done-", " "],
    [" - Metodo Pomodoro", " "],
    [" - Técnicas para evitar la procrastinación", " "],
    [" ", " "]
]

headers = [" ", "Pag.3"]      ### # Definir table headers (encabezados)
table3 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table3)                ## this is slide 3
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

```
#####
```

```
### SLIDE 4
```

```
table_data = [
    ["3. LIDERAZGO EN PROYECTOS TIC", " "],
    [" ", " "],
    [" - Tipos de lideres (agresivo, inhibido, asertivo)", " "],
    [" - Claves de un liderazgo motivador:", " "],
    [" - Interacciones entre el propietario del producto y el jefe del proyecto", " "],
    [" - Problema del <<yo lo resuelvo>> en el trabajo en equipo", " "],
    [" - Toma de decisiones: Solución más apropiada para el cliente", " "],
    [" - Procedimiento para comunicar información valiosa para el equipo:", " "],
    [" - Documentación del trabajo realizado y control de versiones", " "],
    [" ", " "]
]

headers = [" ", "Pag.4"]      ### # Definir table headers (encabezados)
table4 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table4)                ## this is slide 4
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

```
### Continúa en la siguiente página. . .
```

```
#####
```

```
### SLIDE 5
```

```
table_data = [
    ["4. ANEXO I: TÉCNICA POMODORO", " "],
    [" ", " "],
    ["La técnica Pomodoro es un método de gestión del tiempo que puede ser útil en las
fases de", " "],
    ["construcción y programación de aplicaciones (intro, analisis, implementacion y
evaluacion). ", " "],
    [" ", " "],
    ["Los aspectos clave de esta técnica son:", " "],
    [" ", " "],
    [" - Intervalos de trabajo: Consiste en trabajar de manera concentrada durante 25 min",
" "],
    [" - Descansos cortos: Después de cada intervalo de 25 min (pomodoro), descansar
durante 5 min", " "],
    [" - Descansos largos: Después de completar 4 pomodoros, tomar un descanso de 15-30
min", " "]
]
```

```
headers = [" ", "Pag.5"]      ### # Definir table headers (encabezados)
table5 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table5)                ## this is slide 5
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

```
#####
```

```
### SLIDE 6
```

```
table_data = [
    ["5. ANEXO II: BENEFICIOS DE LA TÉCNICA POMODORO EN EL DISEÑO DE APPS", " "],
    [" ", " "],
    [" - Mejora la concentración: Al trabajar en intervalos cortos, el enfoque es mayor", "
"],
    [" - Aumenta la productividad: Ayuda a gestionar mejor el tiempo y evitar la
procrastinación", " "],
    [" - Facilita la planificación: Se puede dividir una tarea grande en partes más
manejables", " "],
    [" - Reduce el agotamiento: Los descansos regulares ayudan a mantener la energia y
motivacion", " "],
    [" - Fomenta la autodisciplina: Al seguir un horario estructurado, se desarrolla una
mejor gestión del tiempo", " "],
    [" ", " "],
    [" ", " "],
    ["Fin de la presentación del mod. III ", " "]
]
```

```
headers = [" ", "Pag.6"]
table6 = tabulate(table_data, headers, tablefmt="fancy_grid")
print(table6)
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

```
print(" ")
print("Fin de la presentación del módulo III. Gracias por su atención.")
print("\n")
```

```
### Fin del script 10
```

```

# Script 11 de repaso sobre COMUNICACIÓN EN ENTORNOS TIC (módulo 4)
# Archivo: hello-tabulateTIC4.py          [ Corrija los posibles errores en el código. ]

import os
os.system('cls' if os.name == 'nt' else 'clear')

from tabulate import tabulate

### SLIDE 1 ##### SCRIPT 10.

table_data = [
    ["COMUNICACIÓN EN ENTORNOS TIC", " ", " "],
    [ " ", " " ],
    [ " ", " " ],
    ["Mod. (IV)", " ", " "],
    [ " ", " " ],
    ["CONFLICTOS Y PROBLEMAS EN EL DESARROLLO DE APPS", " " ],
    [ " ", " " ]
]

headers = [ " ", "Pag.1"]      ### # Definir table headers (encabezados)

table1 = tabulate(table_data, headers, tablefmt="fancy_grid")    # Crear table

print(table1)                ## this is slide 1

xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

#####

### SLIDE 2

table_data = [
    ["1. CONTROL DE STRESS EN EL DESARROLLO DE PROYECTOS", " " ],
    [ " ", " " ],
    [" TÉCNICAS DE CONTROL: ", " " ],
    [ " ", " " ],
    [" - Actuación recomendada cuando el proyecto cambia de rango o espacio temporal. ", " " ],
    [ " ", " " ],
    [" - Enfrentamiento al caso en el que no se cumple la fecha de entrega.", " " ],
    [ " ", " " ],
    [" - Superación de la crisis cuando falla el test (evaluacion) y no se sabe por qué", " " ],
    [ " ", " " ],
    [" - Alternativas cuando la previsión de la dificultad de un trabajo ha fallado", " " ]
]

headers = [ " ", "Pag.2"]      ### # Definir table headers (encabezados)
table2 = tabulate(table_data, headers, tablefmt="fancy_grid")    # Crear table
print(table2)                ## this is slide 2
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

### Continúa en la siguiente página. . .

```

```
#####
```

SLIDE 3

```
table_data = [
    ["2. ESTRATEGIAS PARA RESOLUCIÓN DE CONFLICTOS", " "],
    [" ", " "],
    [" - Caso del producto que no cubre las necesidades del cliente", " "],
    [" - Caso del proyecto que no es eficiente", " "],
    [" - Caso del cliente que pone limitaciones temporales no realistas", " "],
    [" - Caso en que la calidad está reñida con los medios", " "],
    [" - Caso en el que el resultado no coincide con el diseño", " "],
    [" - Caso en el que el equipo de desarrollo no se entiende entre sí", " "],
    [" - Caso en el que el jefe de proyecto no se entiende con el equipo", " "],
    [" - Caso en el que el cliente no se entiende con el equipo de desarrollo", " "],
    [" - Caso en el que el cliente no se entiende con el jefe de proyecto", " "],
    [" ", " "]
]
```

```
headers = [" ", "Pag.3"]      ### # Definir table headers (encabezados)
table3 = tabulate(table_data, headers, tablefmt="fancy_grid") # Crear table
print(table3)                ## this is slide 3
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

```
#####
```

SLIDE 4

```
table_data = [
    ["3. INTERACCIÓN SOCIAL EN EL CONFLICTO", " "],
    [" ", " "],
    [" - Estrategias para pedir ayuda", " "],
    [" - Actitud para hacer y recibir críticas", " "],
    [" - Manifestación de desacuerdos con otras decisiones", " "],
    [" - Petición de cambios de conducta a los componentes del equipo", " "],
    [" - Estrategias para poner límites", " "],
    [" - Estrategias para hacer frente a jefes de proyecto agresivos", " "],
    [" - Estrategias ante situaciones embarazosas", " "],
    [" ", " "]
]
```

```
headers = [" ", "Pag.4"]      ### # Definir table headers (encabezados)
table4 = tabulate(table_data, headers, tablefmt="fancy_grid") # Crear table
print(table4)                ## this is slide 4
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')
```

Continúa en la siguiente página. . .

SLIDE 5

```

table_data = [
    ["4. ANEXO I: TÉCNICA POMODORO", " "],
    [" ", " "],
    ["La técnica Pomodoro es un método de gestión del tiempo que puede ser útil en las
fases de", " "],
    ["construcción y programación de aplicaciones (intro, analisis, implementacion y
evaluacion). ", " "],
    [" ", " "],
    ["Los aspectos clave de esta técnica son:", " "],
    [" ", " "],
    [" - Intervalos de trabajo: Consiste en trabajar de manera concentrada durante 25 min",
" "],
    [" - Descansos cortos: Después de cada intervalo de 25 min (pomodoro), descansar
durante 5 min", " "],
    [" - Descansos largos: Después de completar 4 pomodoros, tomar un descanso de 15-30
min", " "]
]

headers = [" ", "Pag.5"]      ### # Definir table headers (encabezados)
table5 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table5)                ## this is slide 5
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

```

```
#####
```

SLIDE 6

```

table_data = [
    ["5. ANEXO II: BENEFICIOS DE LA TÉCNICA POMODORO EN EL DISEÑO DE APPS", " "],
    [" ", " "],
    [" - Mejora la concentración: Al trabajar en intervalos cortos, el enfoque es mayor", "
"],
    [" - Aumenta la productividad: Ayuda a gestionar mejor el tiempo y evitar la
procrastinación", " "],
    [" - Facilita la planificación: Se puede dividir una tarea grande en partes mas
manejables", " "],
    [" - Reduce el agotamiento: Los descansos regulares ayudan a mantener la energia y
motivacion", " "],
    [" - Fomenta la autodisciplina: Al seguir un horario estructurado, se desarrolla una
mejor gestión del tiempo", " "],
    [" ", " "],
    [" ", " "],
    ["Fin de la presentación del mod. IV ", " "]
]

headers = [" ", "Pag.6"]      ### # Definir table headers (encabezados)
table6 = tabulate(table_data, headers, tablefmt="fancy_grid")  # Crear table
print(table6)                ## this is slide 6
xx = input("Pulse <ENTER> para continuar: "); print("\n\n")
os.system('cls' if os.name == 'nt' else 'clear')

print(" ")
print("Fin de la presentación del módulo IV. Gracias por su atención.")
print("\n")

```

Fin del script 11

APÉNDICE C

Busque y corrija los errores en la siguiente tabla:

Comandos Linux vs Microsoft Windows

Linux	Microsoft Windows
pwd Muestra el directorio actual	cd Muestra el directorio actual
ls Lista archivos y carpetas	dir Lista archivos y carpetas
cd <directorio> Cambia de directorio	cd <directorio> Cambia de directorio
mkdir <carpeta> Crea un directorio/carpeta nueva	mkdir <carpeta> Crea un directorio/carpeta nueva
rm <archivo> Elimina un archivo	del <archivo> Elimina un archivo
rm -r <carpeta> Elimina un directorio y su contenido	rmdir /s <carpeta> Elimina un directorio y su contenido
cp <origen> <destino> Copia archivos o carpetas	copy <origen> <destino> Copia archivos o carpetas
mv <origen> <destino> Mueve o renombra archivos	move <origen> <destino> Mueve o renombra archivos
cat <archivo> Muestra el contenido de un archivo	type <archivo> Muestra el contenido de un archivo
grep "texto" <archivo> Busca un texto dentro de un archivo	findstr "texto" <archivo> Busca un texto dentro de un archivo
ps aux Muestra procesos en ejecución	tasklis Muestra procesos en ejecución
clear Limpia la pantalla	cls Limpia la pantalla

APÉNDICE D (i)

Proyecto sencillo para visualizar registros de una base de datos SQLite

```
#
# Programa xsumal.py desarrollado en Python 3 para Microsoft Windows 10
#
# El script visualiza el primer registro de la tabla page1 y
#
# permite su modificación mediante la entrada de una cadena nueva:
#
# newline1
#
# El primer registro de la tabla page1 contiene un párrafo largo
#
# de la primera página del libro o un nombre de una persona
#
# Cómo construir la cadena para pasarla como orden SQLite?
#
# La base de datos se llama test11.db. Puede descargar el archivo .db visitando:
#
# https://github.com/javier928/xxupdate
#
# La tabla 1 se llama page1. La tabla contiene párrafos de un libro o
# preguntas de un test
#
# La tabla 1 (y cada tabla de la bd solo contiene 1 registro. Cada registro
# contiene los párrafos de esa pag)
#
# La tabla 2 se llama page2. La tabla contiene más párrafos del libro o más
# preguntas de un test.
#
# La tabla 9 se llama page9. La tabla contiene más párrafos del libro. El libro
# contiene 10 páginas (10 tablas).
#
# La tabla tiene los siguientes campos (todos son tipo texto excepto
# booknn y pagenn, que son numéricos):
#
# line1, line2...line20
#
# ¿Qué campo estamos utilizando para filtrar registros? pagenn
#
#
#####

import os
import sqlite3
import tabulate

def cls():
    os.system("cls")

## El código continúa en la siguiente página
```



```
def superprint(ppp,qqq):  
    ##si ppp es '1' se imprime una línea en blanco antes de imprimir el valor de qqq  
    if ppp=="1":  
        print(" ")  
    print(qqq)
```

```
def printlongline():  
    print("-----")  
    print("-----")
```

El código continúa en la siguiente página

```

## Preguntamos qué database (ebook) queremos abrir:
## Esta parte se puede modificar para abrir diferentes libros (databases)
cls()
superprint("0"," Bienvenido a SQL database editor lite")
print(" ")
zx= input(" Qué database quiere abrir? <Enter 0 para salir, entre 1 para abrir test11.db>
")

conn= sqlite3.connect("test11.db")
print(" ")
print(" SQLite database <test11.db> se ha abierto correctamente. ")

## preguntamos que TABLE (page of the book) queremos ACTUALIZAR
print(" ")
Xplineyyy= input(" Qué (TABLE or PAGE) number quiere ver? <Entre 0 para salir> ")
Xpyyy=int(Xplineyyy)

## construimos orden SQLite para contar registros en la tabla 1. SELECT field1,field2,etc.
from table1 (or pagel)
cursor1= "SELECT LINE1,"
cursor2= "LINE2, LINE3, LINE4, LINE5,LINE6, LINE7, LINE8, LINE9,LINE10,LINE11, LINE12,
LINE13,LINE14,LINE15,LINE16, LINE17, LINE18, LINE19,LINE20, PAGENN, "
cursor3= "LINE1N, LINE2N, LINE3N FROM PAGE"+Xplineyyy
cursor= conn.execute(cursor1+cursor2+cursor3)

cursor1= "SELECT COUNT(*)"
cursor2= " "
cursor3= "FROM PAGE"+Xplineyyy
#pagel is table1 in sqlite database test.db
cursor= conn.execute(cursor1+cursor2+cursor3)

print(" ")
plineyyy= input(" Qué (record) number quiere editar? <Entre 0 para salir> ")
pyyy=int(plineyyy)

### falta cambiar el filtro en la orden SELECT para mostrar pagenn= plineyyy
cursor1= "SELECT LINE1,"
cursor2= "LINE2, LINE3, LINE4, LINE5, LINE6, LINE7, LINE8, LINE9,LINE10,LINE11, LINE12,
LINE13,LINE14,LINE15,LINE16, LINE17, LINE18, LINE19,LINE20,PAGENN, "
cursor3= "LINE1N, LINE2N, LINE3N FROM PAGE"+Xplineyyy+" "
cursor4= "WHERE PAGENN="+plineyyy+" "
cursor= conn.execute(cursor1+cursor2+cursor3+cursor4)

### Continúa en la página siguiente...

```

```

cls()
for row in cursor:

print("-----\n")
print(" Fields:      Content:")

print("-----\n")
print(" line1:      ", row[0].strip(), "gasta "+str(row[21])+" eur")
print(" line2:      ", row[1].strip(), "gasta "+str(row[22])+" eur")
print(" line3:      ", row[2].strip(), "gasta "+str(row[23])+" eur")
print(" line4:      ", row[3])
print(" line5:      ", row[4])
print(" line6:      ", row[5])
print(" line7:      ", row[6])
print(" line8:      ", row[7])
print(" line9:      ", row[8])
print(" line10:     ", row[9])
print(" line11:     ", row[10])
print(" line12:     ", row[11])
print(" line13:     ", row[12])
print(" line14:     ", row[13])
print(" line15:     ", row[14])
print(" line16:     ", row[15])
print(" line17:     ", row[16])
print(" line18:     ", row[17])
print(" line19:     ", row[18])
print(" line20:     ", row[19], "Total: "+str(row[21]+row[22]+row[23])+" eur")
print(" pagenn:     ", row[20])

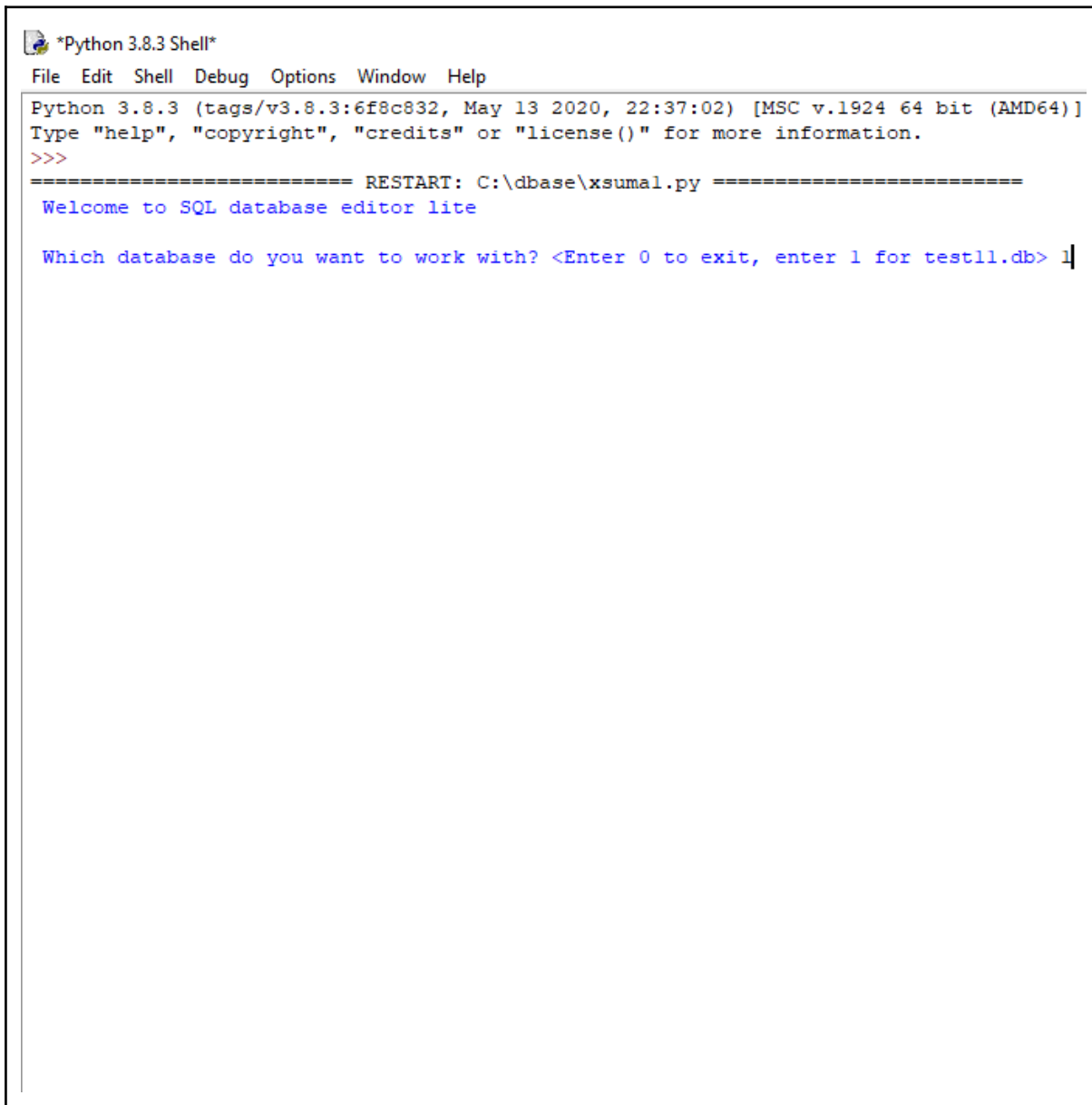
print("\n-----")

conn.close()

## Fin del código ## Puede descargar el código fuente (y la base de datos SQLite) de este ejemplo visitando:
## https://github.com/javier928/xxupdate

```

Al ejecutar el código podemos ver que tenemos que responder la primera pregunta.



```
*Python 3.8.3 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite

Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1
```

Si tecleamos 1 y pulsamos <Enter>, estaremos de acuerdo en abrir el archivo test11.db y veremos la siguiente pregunta:

A continuación podemos ver la siguiente pregunta (donde se filtra la tabla que queremos ver-la cual contiene una página de nuestro hipotético libro-):

```
*Python 3.8.3 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite

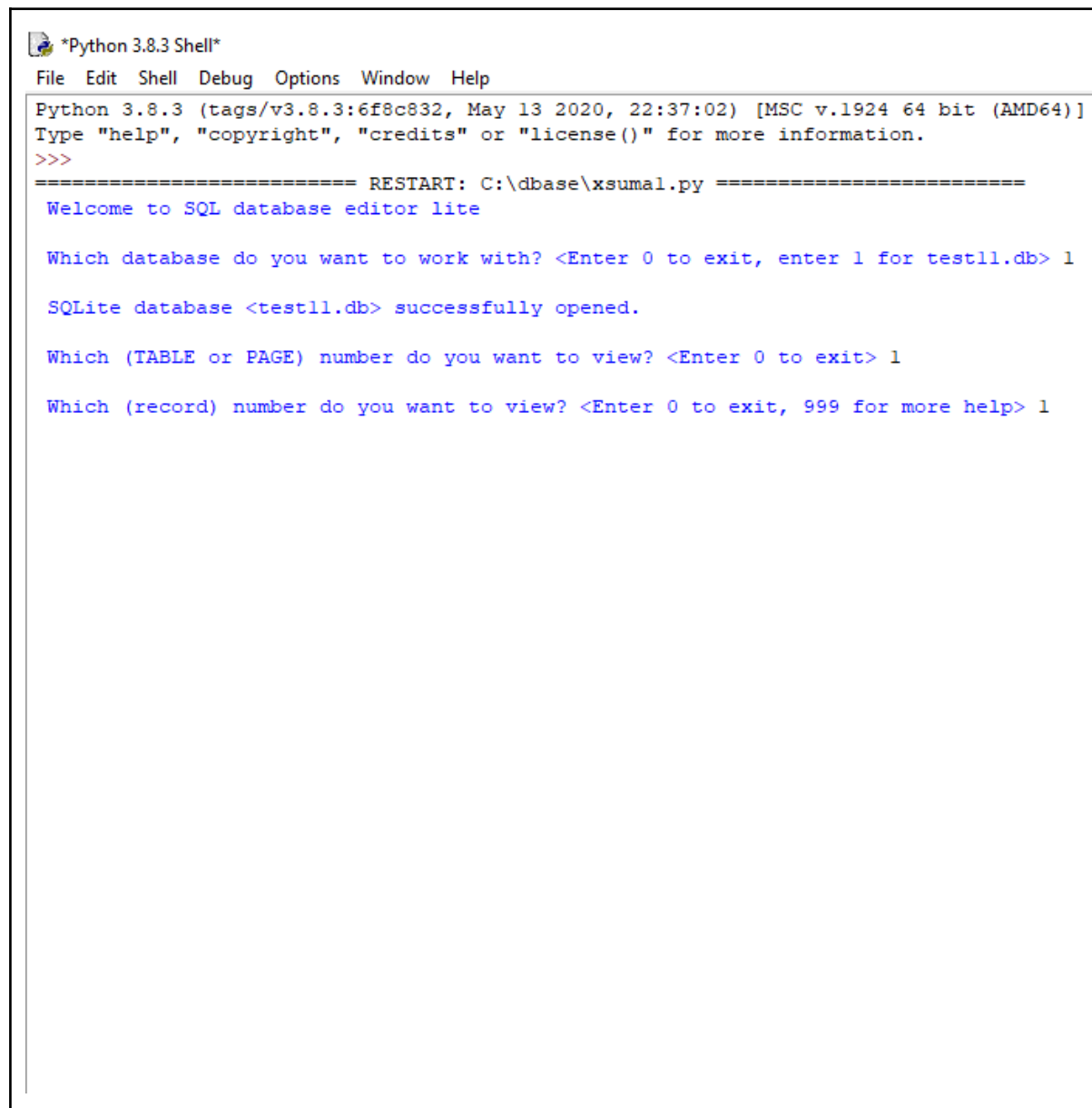
Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1

SQLite database <test11.db> successfully opened.

Which (TABLE or PAGE) number do you want to view? <Enter 0 to exit> 1
```

Si tecleamos 1 y pulsamos <Enter>, veremos la siguiente pregunta:

A continuación podemos ver la siguiente pregunta (donde se filtra qué registro -record- visualizaremos (recuerde que cada tabla únicamente contiene un registro):



```
*Python 3.8.3 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:37:02) [MSC v.1924 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite

Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1

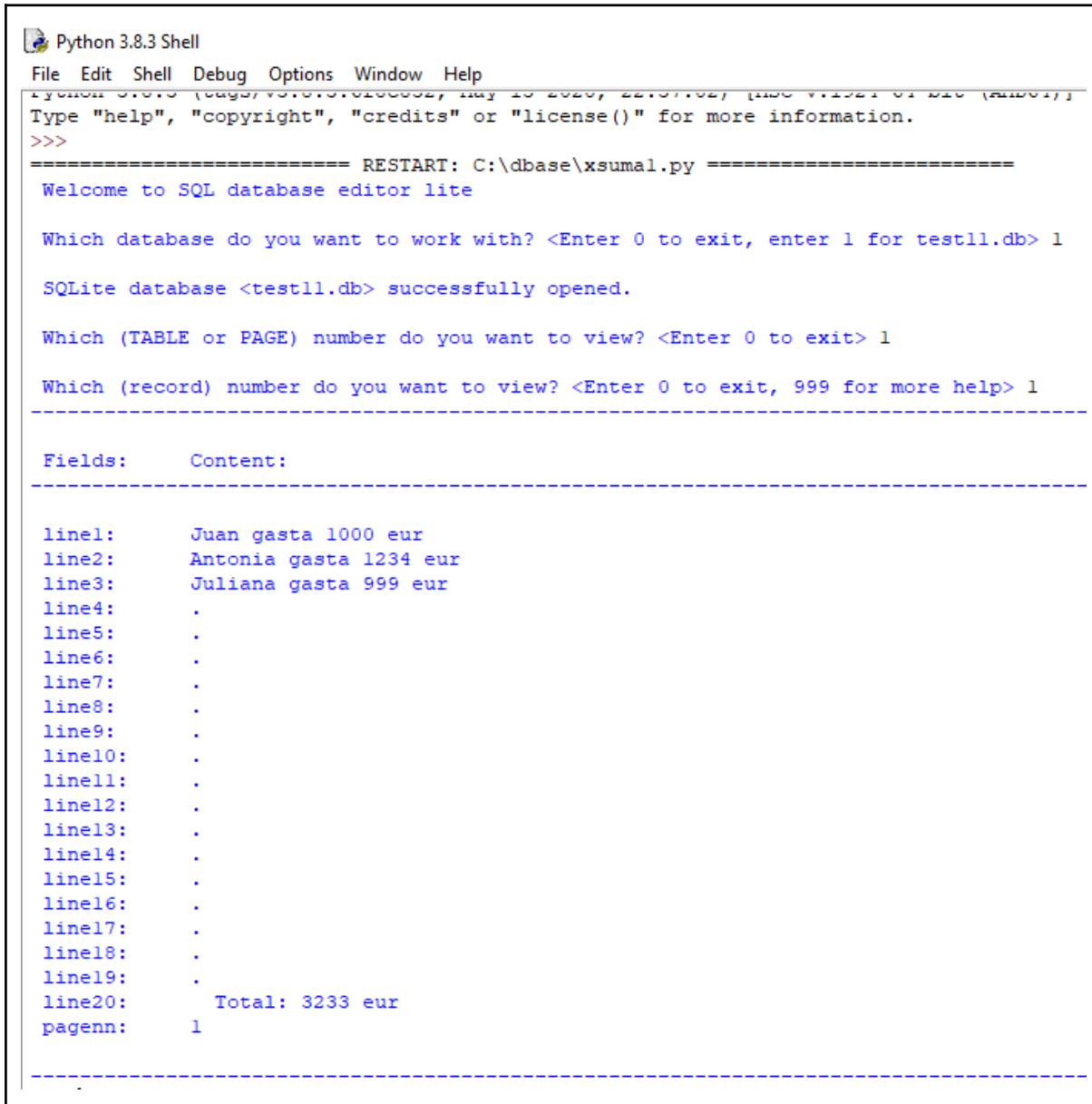
SQLite database <test11.db> successfully opened.

Which (TABLE or PAGE) number do you want to view? <Enter 0 to exit> 1

Which (record) number do you want to view? <Enter 0 to exit, 999 for more help> 1
```

Si tecleamos 1 y pulsamos <Enter>, veremos el contenido del registro seleccionado:

Contenido del registro:



```
Python 3.8.3 Shell
File Edit Shell Debug Options Window Help
Python 3.8.3 (tags/v3.8.3:6f8c832, May 15 2020, 22:07:02) [AMD64]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\dbase\xsumal.py =====
Welcome to SQL database editor lite

Which database do you want to work with? <Enter 0 to exit, enter 1 for test11.db> 1

SQLite database <test11.db> successfully opened.

Which (TABLE or PAGE) number do you want to view? <Enter 0 to exit> 1

Which (record) number do you want to view? <Enter 0 to exit, 999 for more help> 1
-----
Fields:      Content:
-----
line1:      Juan gasta 1000 eur
line2:      Antonia gasta 1234 eur
line3:      Juliana gasta 999 eur
line4:      .
line5:      .
line6:      .
line7:      .
line8:      .
line9:      .
line10:     .
line11:     .
line12:     .
line13:     .
line14:     .
line15:     .
line16:     .
line17:     .
line18:     .
line19:     .
line20:     Total: 3233 eur
pagenn:     1
-----
```

Nota:

Observe que en el campo LINE20 se ha sumado el contenido de los campos LINE1N, LINE2N y LINE3N (no se ha sumado el contenido de LINE1, LINE2 y LINE3).

Si necesita sumar otros valores, tendrá que cambiar el contenido de los campos LINE1N, LINE2N y LINE3N en el primer registro de la tabla seleccionada, que en este caso es la número 1.

APÉNDICE D (ii)

Convertir una lista de temperaturas Celsius a temperaturas Fahrenheit y visualizar los resultados con un gráfico.

Para visualizar gráficos de temperaturas en Celsius y Fahrenheit, podemos usar la biblioteca **Matplotlib**, que funciona correctamente en paralelo con **NumPy**. Utilice el siguiente ejemplo para ver cómo funcionan las librerías mencionadas:

```
import numpy as np
import matplotlib.pyplot as plt

# Datos
temperaturas_celsius = np.array([A, B, C, D, E])
temperaturas_fahrenheit = (temperaturas_celsius * 9/5) + 32

# Crear gráfico
plt.plot(temperaturas_celsius, temperaturas_fahrenheit, marker='o', color='blue')
plt.title('Conversión de Temperaturas')
plt.xlabel('Temperatura (°C)')
plt.ylabel('Temperatura (°F)')
plt.grid(True)
plt.show()
```

Ojo: Las variables A, B, C, D y E no se han declarado correctamente. Estos valores numéricos de dos dígitos deben leerse en la base de datos **TEST11Data.db**. Puede encontrar el archivo en la siguiente dirección:

<https://github.com/javier928/xxupdate>

```
import os; import sqlite3; import numpy as np; import matplotlib.pyplot as plt
conn = sqlite3.connect("test11data.db")

cursor1 = "SELECT LINE1 FROM PAGE1 LIMIT 1 OFFSET 0"
cursor = conn.execute(cursor1)
A = int(cursor.fetchone()[0]); print("Loading data. Reading A=",A)

cursor1 = "SELECT LINE1 FROM PAGE1 LIMIT 1 OFFSET 1"
cursor = conn.execute(cursor1)
B = int(cursor.fetchone()[0]); print("Loading data. Reading B=",B)

cursor1 = "SELECT LINE1 FROM PAGE1 LIMIT 1 OFFSET 2"
cursor = conn.execute(cursor1)
C = int(cursor.fetchone()[0]); print("Loading data. Reading C=",C)

D=0      ## ojo, este valor no se lee de la base de datos: debe completar esta línea
E=0      ## ojo, este valor no se lee de la base de datos: estudie los párrafos anteriores

cursor.close(); conn.close()

temp_celsius= np.array([A,B,C,D,E]); temp_fahrenheit= (temp_celsius * 9/5 + 32)
plt.plot(temp_celsius, temp_fahrenheit, marker='o', color='blue')
plt.title('Conversión de Temperaturas'); plt.xlabel('Temp. (°C)'); plt.ylabel('Temp. (°F)')
plt.grid(True); plt.show()
```


APÉNDICE D (iii)

Ejemplo de utilización de color en un script

Para visualizar textos en color en un script en Python, podemos usar la biblioteca **colorama**, que funciona correctamente en paralelo con la mayoría de librerías para manipular textos. Utilice el siguiente ejemplo para ver cómo funcionan las librerías mencionadas:

```
##
## colorama2.py - - - > Ejemplo de utilización de color en un script.
##
## Nota: cls borra la pantalla en Windows PCs
##

import os
os.system('cls' if os.name == 'nt' else 'clear')

from colorama import init, Fore, Style

# Inicializa colorama
init(autoreset=True)

xname = input("Dime tu nombre, por favor: ")

print("Hola, " + xname + "!")

a = "Vamos a hacer un test dos preguntas sobre"
# Dejamos "Microsoft Windows" en color (azul brillante, por ejemplo)
b = f" {Fore.BLUE}{Style.BRIGHT}Microsoft Windows"
print(a + b)

puntos = 0

xxx = input("El comando COPY en Windows sirve para copiar archivos. (V/F)? ")

if xxx == "V":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos = puntos + 1
else:
    # Agregamos el "else" para mostrar el error en ROJO si la respuesta es incorrecta
    print(f"{Fore.RED}Respuesta incorrecta.")
#endif

xxx = input("El comando DIR en Windows sirve para mover o renombrar archivos. (V/F)? ")

if xxx == "F":
    print("Respuesta correcta. Tienes 1 punto.")
    puntos = puntos + 1
else:
    # Agregamos el "else" para mostrar el error en ROJO si la respuesta es incorrecta
    print(f"{Fore.RED}Respuesta incorrecta.")
#endif

print("Tu nota final en este test es:"); print(puntos)
```

APÉNDICE E

Trabajo en equipo y seguridad informática (I)

Para compartir los archivos y programas vistos en este libro con los miembros de su equipo, debería comprimirlos con una clave de más de 13 caracteres (lo puede hacer con la utilidad 7zip gratuita) antes de enviarlos por email a cualquier colega. No envíe sus archivos a través de Internet sin cifrar.

Otra opción segura para compartir archivos por email es Protonmail. Este servicio web puede comprimir y cifrar los archivos que quiera enviar a los miembros de su equipo. Ambos miembros del equipo deben tener una cuenta Protonmail para aumentar la seguridad. No use la clave que usa en su correo electrónico habitual en la web de Protonmail.

No comparta con su colega la clave de acceso a ningún archivo por email. Planifique una reunión con su colega y díglela la palabra secreta. No deben compartirla con nadie. No la comparta por Signal, Telegram o Whatsapp.

Borre las cookies al menos una vez por semana. Si cambia de ordenador en su oficina o puesto de trabajo, cambie su clave de acceso. Bloquee el ordenador cada vez que se levanta de su puesto de trabajo.

Si cambia de empresa o de oficina, borre todos sus datos en su ordenador antes de marcharse y cambie la clave en su correo electrónico personal.

Si instaló un navegador diferente al proporcionado por Microsoft Windows, borre las cookies, el historial y desinstale el navegador. A continuación cambie sus claves de Microsoft Outlook, Proton mail y Gmail y resetee la computadora. Asegúrese que al abrir el correo de cualquiera de estos servicios utilizados, no puede ver su correo electrónico.

Si compartió algún documento con sus colegas en la web de Microsoft www.office.com o a través de OneDrive, Proton Drive o Google Drive, quite los accesos a los archivos compartidos desde otro ordenador o desde su teléfono móvil.

Es importante recordar que en el ámbito de la informática, la seguridad absoluta no existe. Sin embargo, podemos tomar algunas medidas de protección.

En 2022 se registraron en España 298,000 delitos informáticos. Fuente: Statista. Recuperado el 1 de Abril de 2025, de <https://www.statista.com/topics/7011/cyber-crime-in-spain/#topicOverview>

Las fuerzas de seguridad de numerosos países recomiendan tomar una serie de medidas de prevención para proteger la información almacenada en redes locales, dispositivos móviles (incluidos teléfonos y tabletas). Entre estas medidas, podemos resaltar las siguientes:

1. No se fíe de llamadas no solicitadas: Si recibe una llamada de alguien que afirma ser de una empresa de tecnología o de su banco, cuelgue y contacte directamente a la institución utilizando los números de teléfono oficiales.
2. Nunca comparta información personal o financiera a través de llamadas: Los bancos y las empresas legítimas nunca pedirán contraseñas, números de tarjetas de crédito o información de cuentas a través de una llamada telefónica.
3. Verifique la identidad del interlocutor: Si no está seguro de la identidad de la persona que llama, pídale que proporcione un número de contacto verificable para que pueda llamarle y comprobar que es la persona que dice ser.
4. Mantenga su software actualizado: Instale las últimas actualizaciones de Microsoft Windows, Android e iOS así como de todas las aplicaciones instaladas en sus dispositivos para protegerse de las vulnerabilidades conocidas.
5. Utilice una contraseña fuerte y única para cada cuenta: Evite contraseñas fáciles de adivinar relacionadas con su familia o trabajo y cambie las contraseñas regularmente (cada 60 días). No siga un patrón para crearlas.
6. Active la autenticación de dos factores: Esta función agrega una capa adicional de seguridad al requerir un código de verificación además de su contraseña. Tanto Google como Microsoft, tienen aplicaciones oficiales gratuitas de autenticación de dos factores para móviles y tabletas. Estas son: Google Authenticator y Microsoft Authenticator.
7. Ponga en modo avión su dispositivo cuando no lo necesite o cuando vaya a descansar. Verifique que, al activar el modo avión, no cuenta con acceso a internet ni a dispositivos Bluetooth.

Ejemplo de trabajo en equipo y seguridad informática (II)

Hoy en día es muy común en el mundo empresarial enviar archivos, informes, listas con miles de datos e incluso bases de datos en cualquiera de los siguientes formatos: DBF, XLS, XLSX, 7Z o ZIP.

En la empresa donde comenzaste a trabajar esta semana, utilizan la aplicación gratuita **7za.exe** para comprimir archivos desde la línea de comandos. Te han pedido comprimir tres archivos cuyos nombres comienzan con la palabra "hola", utilizando el formato ZIP.

¿Qué comando deberías usar con **7za.exe** para comprimir los archivos?

```
7za.exe a -tzip archive.zip hola*
```

Durante el día, un compañero te envía tres informes por correo electrónico, comprimidos en un archivo ZIP llamado **archive.zip**. La utilidad predeterminada de Microsoft Windows para descomprimir archivos no funciona correctamente, pero recuerdas que la aplicación **7za.exe** también permite descomprimir archivos con extensión **.zip**.

¿Qué comando deberías utilizar en la línea de comandos para descomprimir el archivo **archive.zip**, si necesitas leer los informes posteriormente para ponerte al día?

```
7za.exe x archive.zip
```

Reto 1:

¿Qué crees que haría la siguiente orden?

```
7za.exe a -tzip -p1234 archive.zip hola*
```

Reto 2:

¿Qué crees que haría la siguiente orden?

```
7za.exe x archive.zip -p1234
```

Recuerda que la utilidad **7za.exe** es gratuita (open source) y se puede utilizar libremente (incluso con fines comerciales).

La utilidad **7za.exe** es compatible con los siguientes formatos: **7z, zip, gzip, bzip2, tar, xz, and cab.**

Entre las recomendaciones generales para el fortalecimiento de mecanismos de autenticación, diversos organismos de ciberseguridad, como en INCIBE o el NIST (Estados Unidos), han señalado la necesidad de evitar patrones numéricos predecibles en sistemas basados en PIN o códigos cortos. Estos patrones presentan una alta probabilidad de ser seleccionados y, en consecuencia, serán utilizados en ataques de fuerza bruta, análisis estadístico o explotación de bases de datos filtradas. Evite estas combinaciones numéricas de alto riesgo: **0000, 1234, 1111, 1212, 2222, 3333, 4444, 5555, 6666, 7777, 8888, 9999, 1004, 1010, 1122, 2000, 2001, 4321 y 6969.**

APÉNDICE F

Unidades de almacenamiento:

Las unidades de almacenamiento se utilizan para medir la capacidad de guardar datos en dispositivos electrónicos.

$$1 \text{ byte (B)} = 8 \text{ bits}$$

$$1 \text{ KB (Kilobyte)} = 1024 \text{ Bytes}$$

$$1 \text{ MB (Megabyte)} = 1024 \text{ KB}$$

$$1 \text{ GB (Gigabyte)} = 1024 \text{ MB}$$

$$1 \text{ TB (Terabyte)} = 1024 \text{ GB}$$

$$1 \text{ PB (Petabyte)} = 1024 \text{ TB}$$

$$1 \text{ EB (Exabyte)} = 1024 \text{ PB}$$

$$1 \text{ ZB (Zettabyte)} = 1024 \text{ EB}$$

$$1 \text{ YB (Yottabyte)} = 1024 \text{ ZB}$$

APÉNDICE G

Dispositivos de almacenamiento y velocidades de transferencia

Los discos duros y los “pendrives” son dispositivos electrónicos que funcionan como unidades de almacenamiento en el mundo informático que se utilizan para guardar datos.

En la siguiente tabla puede ver una lista de velocidades máximas **teóricas** de transferencia de datos basada en los diferentes estándares de USB:

- **USB 2.0 (Hi-Speed USB):** Hasta 480 Mbps
- **USB 3.2 Gen 1 (SuperSpeed USB, antes USB 3.0/3.1 Gen 1):** Hasta 5 Gbps
- **USB 3.2 Gen 2 (SuperSpeed+ USB, antes USB 3.1 Gen 2):** Hasta 10 Gbps
- **USB 3.2 Gen 2x2 (SuperSpeed+ USB 20Gbps):** Hasta 20 Gbps
- **USB4 Gen 2x2 (USB4 20Gbps):** Hasta 20 Gbps
- **USB4 Gen 3x2 (USB4 40Gbps):** Hasta 40 Gbps
- **USB4 V2 (USB4 80Gbps):** Hasta 80 Gbps

APÉNDICE H

La Obligación de ser Autónomo para Vender productos Online en España

Si un programador, desarrollador o emprendedor va a montar una tienda virtual para vender productos en España, debe darse de alta como autónomo antes de realizar la primera venta. No importa si es una tienda pequeña, si se vende desde casa o si se usa una plataforma de un tercero como Shopify o WooCommerce con Wordpress: la actividad económica debe estar registrada legalmente.

Si no se tienen beneficios al principio también es necesario comunicar el inicio de la actividad. No importa si los ingresos son bajos o si se está empezando una prueba. La ley exige el alta desde el momento en que hay actividad económica habitual.

En España, la obligación de darse de alta como autónomo (trabajador por cuenta propia) está ligada a la habitualidad y la finalidad lucrativa de la actividad, no sólo al volumen de ventas (aunque el Salario Mínimo Interprofesional se usa a menudo como referencia).

Si se está en fase de pruebas, se debe consultar con un asesor fiscal sobre opciones como el alta temporal o valorar si realmente se está realizando actividad económica habitual.

APÉNDICE I

La nueva Ley de Inteligencia Artificial de la Unión Europea (EU AI Act) se enfoca en controlar el riesgo de la IA para proteger a las personas.

1. Transparencia obligatoria (Transparencia)

- Aplicaciones web: Si una web usa bots, asistentes o genera contenido automático (texto, imágenes), la ley exige avisar claramente a los usuarios que están hablando o interactuando con una IA.
- Redes Wi-Fi: Si se usa IA para optimizar la red o filtrar tráfico, debemos informar a los usuarios que sus datos de red están siendo analizados por algoritmos inteligentes.

2. Control y supervisión humana (Supervisión)

- Aplicaciones web: Si una web usa IA para tomar decisiones importantes (como aprobar un registro, calificar a un usuario o gestionar pagos), un humano debe poder revisar, corregir o frenar esa decisión.
- Redes Wi-Fi: Las decisiones automáticas que puedan bloquear a usuarios o restringir el acceso a la red no pueden dejarse 100% a ciegas a la IA; debe haber opción de intervención humana.

3. Ciberseguridad y protección de datos (Seguridad)

- Aplicaciones web: Los modelos de IA integrados no deben ser vulnerables a ataques o manipulaciones que expongan datos de los usuarios.
- Redes Wi-Fi: Al gestionar el tráfico de muchos usuarios simultáneos, la IA debe garantizar la privacidad total de las conexiones y evitar sesgos o discriminación al priorizar el ancho de banda.

Bibliografía

¿Qué es el régimen especial de trabajo autónomo? (2025) Obtenido del link:

<https://portal.seg-social.gob.es/wps/portal/importass/importass/Colectivos/Trabajo+Autonomo>

Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial

<https://www.boe.es/doue/2024/1689/L00001-00144.pdf>

Introduction to Flet (2024). Recuperado el 17 de Agosto de 2024, de <https://flet.dev/docs>

Beginner 's Guide to Python (2024). Recuperado el 1 de Agosto de 2024, de <https://wiki.python.org/moin/BeginnersGuide>

LiveCode Ltd. (2010). LiveCode UserGuide.

LiveCode Ltd. (2014c). Getting Started with LiveCode Development. Obtenido de LiveCode Lessons: <http://lessons.runrev.com/m/4603>

LiveCode Ltd. (2024). Creating an 8-puzzle Game. Obtenido de LiveCode Lessons: <https://lessons.livecode.com/m/102552>

Picjumbo.com. Free Images. Recuperado el 26 de Noviembre de 2025, de <https://picjumbo.com/free-images/nature/>

Cómo desarrollar apps para Microsoft Windows con Python 3 (2024). Francisco Javier García Estévez. Obtenido con un iPad de 2017 con el link: <https://books.apple.com/es/book/co-mo-desarrollar-apps-para-microsoft-windows-con-python/id6698850137?l=en-GB>

Introducción al diseño y configuración de software en redes locales

© 2025 by Javier García is licensed under Creative Commons
Attribution-NonCommercial-NoDerivatives 4.0 International.

To view a copy of this license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

Texto (c) de Javier García 2026

Javier928@gmail.com